# Table of Contents

Chapter 1: Glossary: Core System Design Terms Every Engineer Should Know....2
Chapter 2: How It Actually Works: Kafka, Databases, Caching & Core Infrastructure Explained....6
Chapter 3: Requirements Gathering, Estimation & Interview Approach....17
Chapter 4: Networking, Client-Server & HTTP Basics....37
Chapter 5: Scalability Fundamentals....57
Chapter 6: Database Basics....78
Chapter 7: Advanced Database Concepts: Indexing, Replication & Sharding....105
Chapter 8: Caching Strategies....135
Chapter 9: CDNs, Proxies & DNS....165
Chapter 10: Asynchronous Systems: Message Queues & Event-Driven Architecture....186
Chapter 11: API Design & Communication Protocols....214
Chapter 12: Distributed Systems Concepts: CAP Theorem, Consistency & Consensus....241
Chapter 13: Reliability & Fault Tolerance....265
Chapter 14: Object-Oriented Design & SOLID Principles (LLD Foundations)....285
Chapter 15: Design Patterns for LLD....314
Chapter 16: Putting It Together: HLD Case Study Walkthroughs....344
Chapter 17: Security & Authentication....385
Chapter 18: Production & Operations Knowledge....407
Chapter 19: Microservices Architecture & Distributed Systems Patterns....422
Chapter 20: When Your Tech Lead Says This... (Rapid-Fire Judgment Calls)....435
Chapter 21: Real Interview Experience: Verbatim Questions Asked at Real Companies....453
Chapter 22: Top 10 Most Asked System Design Questions....461
Chapter 23: Top 10 Most Asked LLD Questions....478

# Chapter 1: Glossary: Core System Design Terms Every Engineer Should Know

Every other chapter in this book assumes you already know the vocabulary of system design. If any of these terms feel unfamiliar, start here, five minutes now will make every later chapter click instead of feeling like a wall of jargon.

**Traffic, Load & Performance Metrics**

- **QPS / RPS (Queries Per Second / Requests Per Second):** how many requests a system handles every second, on average or at peak. It's the number system design interviews build almost everything else around, since it drives how many servers, how much database capacity, and how much caching a design needs.
- **Latency:** how long a single request takes from start to finish, usually measured in milliseconds. Low latency means a fast individual response, it says nothing about how many requests the system can handle at once.
- **Throughput:** how much total work a system can process per unit of time (requests per second, messages per second). A system can have great latency but poor throughput if it can only handle one request at a time, no matter how fast that one request is.
- **p50 / p95 / p99 (percentile latency):** p50 is the median response time, half of all requests are faster than this. p99 is the time below which 99% of requests complete, it captures the slow "tail" that an average can hide. A system with a great average latency can still have a p99 that frustrates one in every hundred users.
- **Rate limiting:** capping how many requests a client is allowed to make in a given time window (for example, 100 requests per minute per API key), rejecting or delaying requests beyond that cap. It protects a service from being overwhelmed by any single client, abusive or accidental.
- **Throttling:** closely related to rate limiting, but usually describes slowing a client down (delaying or queueing its requests) rather than outright rejecting them once it exceeds an allowed rate.
- **Backpressure:** a signal that flows backward through a system telling an upstream producer to slow down because a downstream consumer can't keep up, instead of letting requests pile up endlessly in a queue until something falls over.
- **Concurrency vs. parallelism:** concurrency means a system is dealing with multiple tasks that are in progress at overlapping times (not necessarily running at the exact same instant), while parallelism means tasks are genuinely executing at the same time, typically on multiple CPU cores.

**Scaling & Architecture**

- **Horizontal vs. vertical scaling:** horizontal scaling adds more machines to share the load (more, typically smaller, servers). Vertical scaling adds more power to one machine (more CPU, more RAM). Horizontal scaling has no hard ceiling and gives you redundancy; vertical scaling is simpler but eventually hits the limit of the biggest machine you can buy.
- **Load balancer:** a component that sits in front of multiple server instances and distributes incoming requests across them, so no single instance takes all the traffic, and unhealthy instances can be automatically removed from rotation.
- **Statelessness:** designing a server so it doesn't store any client-specific data between requests, any instance can handle any request. This is the real prerequisite for easy horizontal scaling, since a stateful server ties a client to one specific machine.

- **Sharding / partitioning:** splitting a large dataset across multiple databases or servers, each holding a subset of the data (for example, users A-M on one shard, N-Z on another), so no single machine has to hold or serve all of it.
- **Replication:** keeping copies of the same data on multiple machines, for availability (a copy survives if one machine fails), read scaling (more machines can answer read queries), and reduced latency (a copy can live closer to users in a given region).
- **CDN (Content Delivery Network):** a network of servers spread across many physical locations that cache and serve content from a location close to the user, instead of every request traveling back to a single origin server.
- **Reverse proxy:** a server that sits in front of one or more backend servers and forwards client requests to them, commonly used for load balancing, TLS termination, and hiding backend infrastructure from the outside world.
- **API gateway:** a single entry point that sits in front of a system's backend services, routing requests to the right service while handling cross-cutting concerns like authentication, rate limiting, and logging in one place instead of in every service.
- **Microservices vs. monolith:** a monolith is one deployable application containing all of a system's functionality. Microservices split that functionality into many small, independently deployable services. Microservices buy independent scaling and deployment at the cost of network calls and operational complexity that a monolith doesn't have.

**Data, Consistency & Distributed Systems**

- **Cardinality:** the number of distinct values a column, field, or metric label can take. A "status" column with 3 possible values has low cardinality; a "user ID" column has very high cardinality. Cardinality matters because high-cardinality fields make some indexes and metrics systems much more expensive, a metrics dashboard sliced by unique user ID can explode into millions of unique time series.
- **ACID:** the guarantees a database transaction gives you: Atomicity (all or nothing), Consistency (the database only ever ends up in a valid state), Isolation (concurrent transactions don't see each other's half-finished work), and Durability (a committed change survives a crash). This is what lets you trust a database with things like money.
- **BASE:** a looser alternative to ACID used by many distributed/NoSQL systems: Basically Available, Soft state, Eventually consistent. It trades strict correctness guarantees for higher availability and scalability.
- **Eventual consistency:** a guarantee that if no new updates are made, all replicas of a piece of data will eventually converge to the same value, but a read right after a write might briefly see stale data.
- **Strong consistency:** a guarantee that every read reflects the most recent write, no matter which replica or node answers it. It's more expensive to provide (often costing latency or availability) but never shows stale data.
- **CAP theorem:** a distributed data store can only fully guarantee two of three properties at once: Consistency, Availability, and Partition tolerance. Since a network partition can always happen, the real, practical choice most systems face is between consistency and availability during that partition.
- **Idempotency / idempotency key:** an operation is idempotent if running it once or many times leaves the system in the same end state. An idempotency key is a unique ID the client attaches to a request so the server can recognize and safely ignore a retried duplicate instead of, say, charging a customer twice.
- **Quorum (W/R):** in leaderless/quorum-based replication, W is the number of replicas that must acknowledge a write and R is the number that must respond to a read for the operation to be

considered successful. Choosing W and R lets a system tune its own trade-off between consistency and availability.

- **Leader-follower replication:** one node (the leader) accepts all writes and streams the changes to one or more followers, which apply them and can serve reads. Simple to reason about, but the leader is a bottleneck and single point of failure for writes until a failover completes.
- **Consensus:** the process by which a group of distributed nodes agree on a single value or decision (like who the current leader is) even in the presence of failures or network issues. Raft and Paxos are the two best-known consensus protocols.

**Caching**

- **Cache hit / cache miss:** a cache hit means the requested data was found in the cache and returned quickly; a cache miss means it wasn't, so the system has to fetch it from the slower source of truth (usually a database) and typically stores it in the cache for next time.
- **TTL (Time to Live):** how long a piece of cached data is allowed to stay before it's considered stale and either refreshed or evicted. A short TTL keeps data fresher; a long TTL reduces load on the underlying data source.
- **Cache eviction (LRU, LFU):** the policy a cache uses to decide what to remove when it's full. LRU (Least Recently Used) evicts whatever hasn't been accessed in the longest time; LFU (Least Frequently Used) evicts whatever has been accessed the fewest times.
- **Cache invalidation:** explicitly removing or updating a cached value when the underlying data changes, so the cache doesn't keep serving data that's now known to be wrong, rather than waiting for its TTL to naturally expire.
- **Hot key:** a single cache or database key that receives a disproportionate share of all traffic (think a celebrity's profile, or a viral post), which can overwhelm the one node responsible for that key even if the rest of the system has plenty of spare capacity.

**Reliability & Operations**

- **SLA / SLO / SLI:** an SLI (Service Level Indicator) is a measured metric, like "99.95% of requests succeeded." An SLO (Service Level Objective) is the internal target for that metric, like "99.9% success rate." An SLA (Service Level Agreement) is the external, often contractual, promise made to customers, usually with a formal consequence (like a service credit) if it's missed.
- **Availability (the "nines"):** the percentage of time a system successfully serves requests, usually expressed as a number of nines (99.9%, 99.99%). Each additional nine cuts allowed downtime by roughly 10x and gets progressively more expensive to achieve.
- **Fault tolerance:** a system's ability to keep working correctly, possibly in a degraded way, even when some of its components fail, rather than failing completely the moment one piece breaks.
- **Circuit breaker:** a safeguard that stops calling a dependency once it's clearly failing (instead of letting every request wait out a timeout against it), giving the struggling dependency a chance to recover and freeing up resources on the calling side.
- **Retry with backoff:** retrying a failed request after a delay that increases with each attempt (exponential backoff), instead of retrying immediately and repeatedly, which would just add more load to an already struggling dependency.
- **Failover:** automatically switching traffic or responsibility to a backup component (a standby replica, a secondary region) when the primary one fails, ideally with minimal disruption to the end user.
- **Health check:** a lightweight endpoint or signal that tells a load balancer, orchestrator, or monitoring system whether a given instance is actually able to handle traffic right now, used to pull unhealthy instances out of rotation automatically.

**Messaging & Asynchronous Systems**

- **Message queue:** a component that holds messages between a producer (something that creates work) and a consumer (something that processes it), letting the two operate independently in time so the consumer being briefly slow or down doesn't block the producer.
- **Pub/sub (publish/subscribe):** a messaging pattern where a publisher sends a message once and every subscriber interested in that type of message receives its own copy, without the publisher needing to know who's listening.
- **Webhook:** a way for one system to notify another of an event by making an HTTP request to a URL the second system provided in advance, instead of the second system having to repeatedly poll and ask "did anything happen yet?"
- **Dead-letter queue (DLQ):** a separate queue where messages get routed after they fail to process successfully a set number of times, so one broken "poison message" doesn't get retried forever and block every healthy message behind it.
- **At-least-once / at-most-once / exactly-once delivery:** at-least-once means a message might be delivered more than once but never lost (so consumers need to handle duplicates). At-most-once means a message might be lost but is never duplicated. Exactly-once, the ideal, is expensive and often achieved in practice by combining at-least-once delivery with idempotent processing on the consumer side.

# Chapter 2: How It Actually Works: Kafka, Databases, Caching & Core Infrastructure Explained

Before you can design systems, you need to understand what the building blocks actually do underneath the buzzwords. Every chapter after this one assumes you already know roughly how Kafka, Redis, Postgres, MongoDB, and indexes work. This chapter is where that assumption gets earned. Each topic below starts from zero, builds up with a plain-language analogy, and ends with the one-line version you'd actually say out loud in an interview.

---

## Q1. [Foundations] How does Kafka actually work under the hood?

**Answer:**

Think of Kafka as a very long, append-only notebook that many people can write new pages into, and many other people can read from, each at their own pace, without anyone's copy of the notebook ever getting erased.

**The core pieces**

- Topic: a named stream of data, like "orders" or "payment-events." This is the notebook's title.
- Partition: a topic is split into multiple partitions so multiple machines can share the work. Each partition is literally an ordered, append-only log file on disk, messages are only ever added to the end.
- Broker: a single Kafka server. A cluster has many brokers, and each partition physically lives on one or more of them.
- Producer: the client that writes (appends) new messages into a partition.
- Consumer: the client that reads messages from a partition, starting from wherever it last left off.
- Offset: a simple, ever-increasing number marking a consumer's position in a partition, like a bookmark in a book. Kafka doesn't track "who has read what" for you, each consumer just remembers its own offset and can rewind or skip ahead.
- Consumer group: a set of consumers sharing the work of reading one topic. Kafka guarantees each partition is read by only one consumer within a group at a time, which is how the work gets parallelized safely.
- Replication: each partition is copied to other brokers for durability. One broker is the leader for that partition and accepts all writes, the others are followers that copy the leader's log in order, and one of them is promoted to leader automatically if the original leader dies.

**Why ordering only works within a partition**

Kafka only guarantees message order inside a single partition, not across an entire topic. If order matters for a particular entity (say, all events for one user), the producer routes every message for that entity to the same partition, usually by hashing a key like the user ID. That way, that one user's events always land in the same ordered log, even though the topic as a whole is spread across many partitions.

**Why Kafka is so fast**

- Sequential disk writes: appending to the end of a file is dramatically faster than writing to random locations on disk, and that's the only kind of write Kafka ever does.

- It leans on the operating system's page cache instead of managing its own complex in-memory cache, which keeps the code simple and lets recently written data get read back from memory without touching disk at all.
- Consumers, not the broker, track progress. The broker doesn't have to maintain per-consumer read state, it just serves whatever offset a consumer asks for, which keeps the broker simple and lets it scale to huge numbers of consumers.

**Kafka is a distributed, replicated, append-only log split into partitions for parallelism, and its speed comes from sequential disk writes plus pushing the bookkeeping of "what have I read" onto each consumer instead of the broker.**

**Interview Answer:**
"Kafka stores messages as an ordered log per partition, producers append to it, consumers track their own offset to read from it, and multiple consumers can replay the same data independently since reading never deletes anything. It's fast because writes are always sequential appends, and it scales because a topic is split across partitions that can live on different brokers."

---

## Q2. [Foundations] How does MongoDB, or a NoSQL database in general, actually store data, and how is that different from a relational database?

**Answer:**

A relational database like Postgres is like a spreadsheet: every row in a table must have the exact same columns. MongoDB is more like a folder full of separate JSON files, each one (called a document) can have its own fields, nested objects, and arrays, and two documents in the same collection don't need to look alike.

### What a document actually looks like

A MongoDB document is stored on disk in BSON, a binary version of JSON. A single "blog post" document might directly embed its own array of comments inside it, instead of storing comments in a separate table and joining them at query time. This embedding is the central design idea in MongoDB: model your data around how you'll read it, not around normalized tables.

### Why there's no real JOIN

MongoDB has a limited $lookup operator that can stitch together two collections, but it's not built the way a SQL join is, and it's discouraged for anything performance-sensitive. The intended pattern is to embed related data directly in one document so a single read fetches everything you need, trading some data duplication for fewer round trips.

### How it actually stores and finds data

- Under the hood, MongoDB's default storage engine (WiredTiger) is still B-tree based, so once you're inside the engine, indexing works very similarly to a relational database's.
- MongoDB scales horizontally by sharding collections: each document is assigned to a shard based on a shard key, spreading both storage and write load across many servers natively.

### When to reach for it, and when not to

Document databases fit naturally when your data is already shaped like nested objects and its structure varies or evolves quickly, like product catalogs where different categories have different attributes, content management, or user-generated content. They're a weaker fit when you need strict relational integrity or multi-entity transactions, like a ledger where a debit and a credit across two different accounts must succeed or fail together.

**MongoDB stores flexible, self-contained JSON-like documents instead of fixed-column rows, favors embedding related data over joining tables, and scales out by sharding documents across servers using a shard key.**

**Interview Answer:**

"MongoDB stores data as JSON-like documents instead of rows in fixed tables, so each document can have a different shape and related data is usually embedded rather than joined. That makes it a good fit for data that's naturally nested or evolves quickly, but a weaker fit than a relational database when you need strong multi-table transactions."

---

## Q3. [Foundations] How does a database index actually work internally?

**Answer:**

Without an index, finding one row means scanning every row in the table in order, called a full table scan, exactly like reading every page of a book to find a single sentence. An index is a separate, sorted data structure that lets the database jump almost straight to the right row instead.

### The B-tree, explained with an analogy

Most database indexes are B-trees. A B-tree is a balanced, multi-level tree where each node holds several sorted key values along with pointers to child nodes. It's like the index at the back of a textbook: instead of reading every page to find every mention of "caching," you look up "caching" alphabetically in the index, and it tells you exactly which pages to flip to. A B-tree is a multi-level, self-balancing version of that same idea, one that keeps working efficiently even as the table grows to millions of rows.

### Why lookups stay fast as data grows

Because a B-tree stays balanced and each node holds many keys, you typically only need to traverse three or four levels deep to find any value, even in a table with millions of rows. That's why an indexed lookup is described as O(log n): every time the table grows by 10x, you might only need one extra level of the tree, not 10x more work.

### The trade-off nobody mentions: writes get slower

Every INSERT, UPDATE, or DELETE has to update not just the table's actual data, but every single index built on that table too, since each index is its own separate B-tree that must stay in sync. This is why adding indexes isn't free: more indexes speeds up reads but slows down writes, and a table with a dozen unnecessary indexes can make every write noticeably slower.

### Composite (multi-column) indexes

A composite index covers multiple columns together, sorted by the first column, then the second within that, and so on, like a phone book sorted by last name and then first name. It only helps a query that filters on the leading column(s) in that same order; an index on (last_name, first_name) won't speed up a search that only filters by first_name.

**An index is a separate, balanced B-tree structure that lets the database jump straight to matching rows instead of scanning the whole table, and that speed at read time is paid for with extra work on every write.**

**Interview Answer:**

"An index is basically a sorted lookup structure, usually a B-tree, sitting alongside the table so the database doesn't have to scan every row. It turns a lookup from O(n) into roughly O(log n), but every index also has to be updated on every write, so there's a real trade-off between read speed and write speed."

## Q4. [Foundations] Why is Redis so fast?

**Answer:**

Redis keeps all of its data in RAM instead of on disk. RAM is roughly 100,000 times faster to access than a spinning hard disk, and still tens of times faster than even a modern SSD, so simply avoiding disk on the hot path already explains most of Redis's speed.

**A single-threaded event loop**

Redis processes commands one at a time on a single main thread. That sounds like it should be slow, but it removes the overhead that multi-threaded systems pay for constantly: locking shared data structures and switching context between threads competing for the same memory. Because each individual Redis command is extremely simple and fast, a single thread can still push through hundreds of thousands of operations per second without ever needing to coordinate with another thread.

**Purpose-built, simple data structures**

Redis's core types, strings, hashes, lists, sets, and sorted sets, are implemented as efficient, low-level structures in C, giving most operations O(1) or O(log n) time complexity. There's no general-purpose query planner deciding how to execute an arbitrary SQL query, every operation is a direct, predictable data structure operation.

**Persistence is a safety net, not the hot path**

Redis can optionally save snapshots to disk (RDB) or append every write to a log (AOF), but this happens asynchronously in the background. It exists so Redis can recover its data after a restart, it is not part of the path a read or write takes while the server is running, so it doesn't slow down normal operations.

*The analogy*

Querying a disk-based database is like walking to a library to find a book. Querying Redis is like glancing at a sticky note already sitting on your desk, everything is already right there, no walk required.

**When Redis is the wrong primary store**

Because everything lives in RAM, your dataset can't exceed available memory without expensive scaling, and Redis doesn't support the kind of rich ad-hoc querying and multi-table joins a relational database does. It's typically used as a cache or a fast auxiliary store in front of a real database, not as a full replacement for one.

**Redis is fast because it operates entirely in memory, avoids threading overhead with a single-threaded event loop, and uses simple, purpose-built data structures, with disk persistence handled asynchronously in the background.**

**Interview Answer:**

"Redis is fast mainly because everything lives in RAM instead of on disk, and because a single-threaded event loop avoids the locking and context-switch overhead you'd get from multiple threads fighting over the same data. Its data structures are also simple and purpose-built, so most operations are O(1). Persistence to disk happens in the background and doesn't sit on the hot path."

## Q5. [Foundations] How does PostgreSQL actually work, and in which scenarios should you actually reach for it?

**Answer:**

Postgres is a relational database: data lives in tables with a fixed set of columns, and relationships between tables are enforced with foreign keys and constraints rather than left to the application to manage.

**What's actually happening under the hood**

- Process-per-connection: each client connection gets its own OS process (unlike MySQL, which typically uses one thread per connection), which is part of why Postgres has historically needed a connection pooler like PgBouncer in front of it at high concurrency.
- Storage in pages: data is stored on disk in fixed-size 8KB pages, and an index (usually a B-tree, same idea as covered above) points into those pages to find rows quickly.
- MVCC (Multi-Version Concurrency Control): instead of locking a row every time it's read, Postgres keeps multiple versions of a row and gives each transaction a consistent snapshot, so readers and writers don't block each other for ordinary operations.
- A query planner: when you run a SQL query, Postgres doesn't just execute it literally, it inspects available indexes and table statistics and chooses an execution plan it estimates will be fastest, which is why the same query can suddenly get faster or slower after data grows or an index is added.

**When to reach for Postgres**

- Your data has real relationships and correctness matters: financial transactions, inventory counts, anything where a group of changes across multiple tables must all succeed or all fail together (a transaction).
- You need flexible, ad-hoc querying: joins, aggregations, and reporting queries you didn't necessarily plan for upfront.
- You want one mature, battle-tested system that can also handle semi-structured data reasonably well through its jsonb column type, often removing the need for a second, separate NoSQL system.

**When you might look elsewhere**

If you need write throughput beyond what a single primary can handle even after tuning and read replicas, or if your data is genuinely document-shaped and changes shape constantly at very large scale, a purpose-built NoSQL system or a sharded architecture may fit better.

**Postgres uses MVCC so readers and writers don't block each other, stores data in indexed pages on disk, and is the right default whenever your data has real relationships and correctness (transactions) matters more than raw horizontal write scale.**

**Interview Answer:**
"Postgres is a relational database that uses MVCC to let reads and writes happen concurrently without blocking, stores rows in 8KB pages backed by B-tree indexes, and picks an execution plan per query through its planner. I'd reach for it by default whenever data has real relationships and I need transactional correctness, and only look elsewhere if I hit write-scale limits a single primary genuinely can't handle."

---

## Q6. [Foundations] How is a message queue like RabbitMQ structurally different from Kafka?

**Answer:**

RabbitMQ is built around the classic idea of a queue: a message is pushed in, delivered to one consumer, and then removed. It's like a shared to-do list where each task gets crossed off the moment someone picks it up, nobody else sees that task again.

Kafka is built around a durable, replayable log instead. Messages stay in the log for a configured retention period, hours or days, even after being read, and multiple independent consumers can each read the very same messages at their own pace, since reading never removes anything. It's closer to a recorded broadcast that many different people can watch on their own schedule, without erasing it for anyone else.

**When to reach for RabbitMQ**

- Complex routing: fan-out to multiple queues based on rules, priority queues, request/reply patterns between services.
- Simple job or task distribution, where each task should be handled exactly once by exactly one worker and then discarded.

**When to reach for Kafka**

- High-throughput event streaming, where you're producing a continuous flow of events rather than discrete one-off tasks.
- Multiple different systems need to independently consume the same stream (analytics, search indexing, and a notification service all reading the same "order placed" events, for example).
- You may need to replay historical events, for debugging, backfilling a new consumer, or reprocessing after a bug fix.

**RabbitMQ removes a message once a consumer takes it (one-time task distribution); Kafka retains messages in a durable log that many independent consumers can each replay at their own pace.**

**Interview Answer:**
"RabbitMQ is a traditional broker where a message is delivered once and then gone, which is great for task distribution. Kafka keeps messages in a retained, ordered log that multiple independent consumers can each read at their own pace and even replay later, which is why it fits event streaming and analytics use cases better than a classic queue does."

---

## Q7. [Foundations] How does DNS resolution actually work, step by step?

**Answer:**

DNS turns a human-readable domain name into the actual IP address a browser needs to open a connection. It's like asking around to find someone's home address: you check your own address book first, then ask people who are progressively more likely to know the exact answer.

**The actual chain of lookups**

- Browser cache: the browser first checks if it already resolved this domain recently and just reuses that answer.
- OS and router cache: if not, the operating system, and sometimes the home router, checks its own cache next.
- Recursive resolver (usually your ISP's or a public one like 8.8.8.8): if nobody local has the answer, this resolver takes over the rest of the search on your behalf.
- Root DNS server: the resolver asks a root server, which doesn't know the final answer but knows which server handles the domain's top-level suffix (like ".com") and points there.
- TLD server: this server doesn't know the final IP either, but it knows which authoritative name server is responsible for that exact domain, and points there.
- Authoritative name server: this is the domain's own "reception desk," it holds the actual DNS records and returns the real IP address.

- The resolver caches this answer (respecting a TTL, or time-to-live) and hands it back to your browser, which finally opens a connection to that IP.

**Why caching at every layer matters**

If every single request required walking this entire chain, page loads would be noticeably slower. Caching at the browser, OS, and resolver level means the full root-to-authoritative walk usually only happens once per domain, per TTL window, for a huge number of users sharing the same resolver.

**DNS resolution is a chain of increasingly specific lookups, from local caches, to a resolver, to root servers, to TLD servers, to the domain's own authoritative server, cached at every layer so the full walk rarely has to repeat.**

**Interview Answer:**
"DNS resolution starts with local caches, and if those miss, a resolver walks a chain from the root servers, to the TLD servers for the domain suffix, to the domain's own authoritative name server, which holds the actual IP. Every layer caches the answer for its TTL, which is why most lookups never need the full chain."

---

## Q8. [Foundations] How does a CDN actually make content load faster?

**Answer:**

A CDN operates a large network of servers, often called edge nodes or points of presence, spread across many physical locations worldwide. Instead of shipping every file from one central warehouse to every customer on Earth, it's like having small local warehouses in every city, each restocked from the central warehouse only occasionally, so most deliveries are short and fast.

**What actually happens on a request**

- A user requests a file (an image, a script, a video segment).
- Routing, usually via DNS or anycast, sends that request to the nearest edge node instead of the original origin server.
- If that edge node already has a cached copy (a cache hit), it serves the file directly, no trip to the origin server needed.
- If it doesn't (a cache miss), it fetches the file once from the origin, caches it locally, and serves that cached copy to this and future nearby requests until it expires.

**Why this actually speeds things up**

Physical distance directly adds latency, data can only travel so fast, and network hops add delay too. Serving from a server a few miles away instead of one on the other side of the planet cuts a meaningful chunk of that round-trip time, on top of massively reducing load on the origin server itself since most requests never reach it at all.

**A CDN caches content at edge servers physically close to users, so most requests are served locally instead of traveling all the way to a distant origin server, cutting latency and origin load.**

**Interview Answer:**
"A CDN is a network of edge servers close to users. Requests get routed to the nearest one, and if that edge already has the file cached, it serves it directly without ever hitting the origin. That cuts latency from physical distance and dramatically reduces load on the origin, since most requests are absorbed at the edge."

---

## Q9. [Foundations] How does Elasticsearch work, and why is it fast for search?

**Answer:**

Elasticsearch is fast at full-text search because of an inverted index, which stores data in the opposite direction from a normal database index.

**Forward index vs inverted index**

A normal database index typically maps a row (or a key) to where its data lives, like a book's table of contents mapping chapter titles to page numbers. An inverted index instead maps each word to the list of documents that contain it, exactly like the index at the back of a textbook: if you want every page that mentions "caching," you don't scan the whole book, you jump straight to "caching" in the back index and read off the page numbers listed there.

**Why that's the right structure for search**

A search query like "find every document containing both 'kafka' and 'replication'" becomes a fast set-intersection over two short lists of document IDs, instead of scanning every document's full text one at a time. That's fundamentally why full-text search engines use inverted indexes instead of the B-trees relational databases rely on.

**What Elasticsearch adds on top**

- It's built on Apache Lucene, which is the actual low-level library that builds and searches the inverted index.
- Elasticsearch adds sharding and replication so an index can scale across many machines, and a REST API and query language on top so applications can talk to it easily.
- It also supports relevance scoring, ranking results by how well they match rather than just returning an unordered list of matches, which is essential for search-quality features like "most relevant first."

**Elasticsearch is fast for text search because it uses an inverted index, word to documents instead of document to words, turning most searches into a quick lookup and set intersection instead of scanning full text.**

**Interview Answer:**

"Elasticsearch is fast because it's built on an inverted index, mapping each word to the documents containing it, so a search for multiple terms becomes a fast intersection of short lists instead of scanning every document. It runs on top of Lucene, and Elasticsearch itself adds sharding, replication, and a query API on top of that."

---

## Q10. [Foundations] How does leader election actually work in a distributed system (Zookeeper, etcd, Raft)?

**Answer:**

In a cluster of multiple nodes, you often need exactly one node in charge at any given moment, to avoid two nodes making conflicting decisions at the same time. Consensus protocols like Raft (used inside etcd) solve this with voting.

**How the election actually happens**

- Nodes start as followers. If a follower stops hearing from a leader for too long, it becomes a candidate and requests votes from the rest of the cluster.
- Other nodes vote for the first valid candidate they hear from in that round. Whoever collects votes from a majority of the cluster becomes the new leader.

- The leader then sends periodic heartbeats to prove it's still alive. As long as followers keep hearing heartbeats, they defer to it and don't start a new election.
- If the leader crashes or gets disconnected, heartbeats stop, followers notice, and a new election begins automatically.

*The analogy*

It's like a classroom electing a class representative: as long as the rep keeps showing up (heartbeats), everyone defers to them, but if they disappear for too long, the class holds a new vote rather than waiting forever.

**Why the majority requirement matters**

Requiring a majority, not just any vote, guarantees at most one leader can win in any given round, even if the cluster is temporarily split by a network partition, since only one side of a split can possibly contain a majority of the nodes. This is what prevents two nodes from both believing they're the leader at the same time.

**Leader election uses majority voting, a candidate becomes leader only once more than half the cluster votes for it, and periodic heartbeats keep that leader in place until it stops responding and a new election starts.**

**Interview Answer:**

"Leader election works through majority voting. A node that stops hearing leader heartbeats becomes a candidate, requests votes, and only becomes leader if it gets a majority. The leader then sends heartbeats to hold that position, and a new election only starts if those heartbeats stop. Requiring a majority is what prevents two leaders from existing at once during a network split."

---

## Q11. [Foundations] How does database replication actually work under the hood?

**Answer:**

Every write to the leader (primary) database is first recorded in a Write-Ahead Log, often called a WAL, a simple, sequential record of every change, before the actual data files on disk are updated. Replication works by streaming this same WAL to follower servers, which replay it in the exact same order to reconstruct an identical copy of the data.

*The analogy*

It's like handing someone your diary entries as you write them, in order, rather than handing them your finished photo album. They reconstruct the whole story themselves by replaying every entry in the same sequence you originally wrote it, ending up with the same result without you ever having to hand over the finished product directly.

**Why the WAL comes first**

Writing to the log before touching the actual data files means that if the database crashes mid-write, it can recover cleanly by replaying the log from the last confirmed point. That same durability mechanism is reused for replication almost for free, since followers just need a copy of that same ordered log to stay in sync.

**Synchronous vs asynchronous replication**

- Synchronous: the leader waits for at least one follower to confirm it received the WAL entry before telling the client the write succeeded. Safer (no data loss if the leader dies immediately after), but slower.

- Asynchronous: the leader confirms the write immediately and streams the WAL to followers in the background. Faster, but a small window of unreplicated data can be lost if the leader crashes at exactly the wrong moment.

**What this makes possible**

Read replicas exist because followers are already reconstructing a full, up-to-date copy of the data this way, so read traffic can be spread across them. Failover works the same way: if the leader dies, a follower that's replayed the WAL up to nearly the same point can be promoted to the new leader with minimal data loss.

**Replication works by streaming the leader's write-ahead log to followers, which replay every change in the same order to reconstruct an identical copy, enabling both read scaling and failover.**

**Interview Answer:**
"Replication streams the leader's write-ahead log to followers, and they replay it in order to build an identical copy of the data. Whether that's synchronous or asynchronous is a durability-versus-latency trade-off. This same mechanism is what lets you scale reads across replicas and promote a replica to leader on failover."

---

## Q12. [Foundations] How does sharding actually split data across machines, mechanically?

**Answer:**

Sharding splits one large dataset across multiple independent database servers, so no single machine has to hold or serve all of it. The interesting part is exactly how a given piece of data is assigned to a specific shard.

**The naive approach: hash(key) % N**

The simplest scheme hashes a key (like a user ID) and takes the remainder when divided by the number of shards, N. It's simple and spreads data evenly, but it has a serious problem: if you add or remove even one shard, N changes, and almost every key's hash % N result changes with it, meaning nearly the entire dataset has to be physically moved to new shards all at once.

**The better approach: consistent hashing**

Consistent hashing places shards at fixed points around an imaginary circle (a hash ring) instead. Each key is hashed to a point on that same ring, and it belongs to whichever shard is the next one clockwise from it. Adding or removing a single shard only reshuffles the keys between that shard and its immediate neighbor on the ring, not the entire dataset.

*The analogy*

Consistent hashing is like a circular seating chart. When one guest leaves or a new one arrives, only their immediate neighbors need to shift seats, everyone else stays exactly where they were.

**A practical wrinkle: hot shards**

Even with a good hashing scheme, one shard can still end up overloaded if a single key gets disproportionately more traffic than others, a celebrity account's data, for example. Real systems handle this with techniques like virtual nodes (spreading each physical shard across many points on the ring) to smooth out uneven load.

**Naive hash(key) % N sharding forces almost a full data reshuffle whenever the shard count changes; consistent hashing places shards on a ring so adding or removing one shard only moves the keys near it.**

**Interview Answer:**

"Basic hash-mod sharding is simple but brittle, changing the number of shards reshuffles almost everything. Consistent hashing fixes that by placing shards on a ring, so a key just goes to the next shard clockwise, and adding or removing a shard only affects its immediate neighbors on the ring, not the whole dataset."

---

## Q13. [Foundations] How does a modern web server handle thousands of concurrent connections at once?

**Answer:**

The old model gave every connection its own dedicated OS thread that sat and waited on it. That doesn't scale past a few thousand connections, because every thread carries real memory overhead and the OS has to keep switching between them, even when most are doing nothing but waiting.

**The event loop model**

Modern high-performance servers like Nginx and Node.js instead use an event loop with non-blocking I/O. A single thread registers its interest in many sockets at once using an OS mechanism like epoll (on Linux), and the OS tells that thread exactly which sockets actually have data ready, right now. The thread only does work on a connection when there's genuinely something to do, instead of a dedicated thread sitting idle on each one.

*The analogy*

It's one waiter efficiently covering every table in a restaurant by only walking over when a table raises its hand, instead of hiring one dedicated waiter per table who stands there the whole time even when nobody needs anything.

**Why this scales so much further**

- Far less memory overhead: no per-connection thread stack, just a lightweight registration with the OS.
- No costly context-switching between thousands of idle threads.
- A single core can juggle tens of thousands of mostly-idle connections, because the actual CPU work only happens in short bursts when data is truly ready.

**The trade-off**

This model is excellent for I/O-heavy workloads (lots of waiting on network or disk), but a single event loop thread can get stuck if a request needs heavy CPU computation, since that blocks the loop from checking on any other connection until it finishes. That's why CPU-heavy work is typically offloaded to separate worker threads or processes even in an event-loop-based server.

**Event-loop servers use non-blocking I/O and OS mechanisms like epoll so one thread can watch thousands of sockets and only act on the ones actually ready, avoiding the memory and context-switch cost of one thread per connection.**

**Interview Answer:**

"An event-loop server registers interest in many sockets at once through something like epoll, and only wakes up to do work on the sockets that actually have data ready, instead of dedicating a thread to sit and wait on each connection. That's why one thread can handle thousands of mostly-idle connections, though CPU-heavy work still needs to be offloaded so it doesn't block the loop."

# Chapter 3: Requirements Gathering, Estimation & Interview Approach

Every system design interview lives or dies on this chapter. Skipping requirements and estimation to jump straight to boxes and arrows might feel like progress, but it signals to an interviewer that you're optimizing for looking busy, not for solving the right problem, and that habit shows up on the job as scope creep and rework. Use these questions to practice asking the right clarifying questions before you design anything, and to make sure your capacity numbers are grounded in real math instead of guesses.

**War Story:** *A team once spent three weeks building a beautifully sharded, highly available notification system for what turned out to be an internal tool used by forty people, because nobody asked the product manager what scale actually mattered before writing a line of code.*

**Avoid:** jumping straight to "I'd use microservices and Kafka" before asking what the system needs to do • guessing at numbers silently instead of stating your assumptions out loud

**Aim For:** explicitly separating functional from non-functional requirements before designing anything • stating your back-of-envelope assumptions out loud and checking whether they're reasonable

## Q1. [Junior] What's the difference between a functional requirement and a non-functional requirement? Give me an example of each for a URL shortener.

**Answer:**

A functional requirement describes what the system does, the actual features and behaviors a user or client interacts with. **A non-functional requirement describes how well the system does it, the quality attributes that shape the architecture.**

**Functional requirements for a URL shortener**

- A user can submit a long URL and receive a short one back.
- Visiting the short URL redirects to the original long URL.
- A user can optionally set a custom alias or an expiration date.

**Non-functional requirements for a URL shortener**

- Redirects must complete in well under 100 milliseconds.
- The service must be highly available, since a broken redirect breaks every link that points to it.
- Short codes must not collide with each other.

The functional list defines the feature set you'd build for an MVP. The non-functional list defines the constraints that decide your data store, caching strategy, and replication approach, and it's usually the non-functional side that determines whether the design is actually hard.

**Interview Answer:**

"Functional requirements are the features, what the system lets a user do, like shortening a URL or setting a custom alias. Non-functional requirements are the quality attributes: how fast, how available, how consistent the system needs to be. For a URL shortener, creating and redirecting short links are functional. Low latency on redirects and high availability are non-functional, and honestly those non-functional constraints usually decide the actual architecture more than the feature list does."

## Q2. [Junior] What does "availability" mean as a non-functional requirement, and how is it usually expressed?

**Answer:**

Availability measures the percentage of time a system is capable of successfully responding to requests, as opposed to being down or unreachable.

**How it's expressed**

- It's usually written as a percentage of uptime over a year, like 99.9% or 99.99%, informally called "three nines" or "four nines."
- **Each additional nine represents roughly a 10x reduction in allowed downtime.** 99.9% allows about 8.7 hours of downtime per year, while 99.99% allows about 52 minutes per year.
- It's often written into a formal Service Level Agreement, or SLA, that has real consequences, like service credits, if it's violated.

**Why it matters for design**

Higher availability targets push you toward redundancy: multiple servers across multiple availability zones, no single point of failure, health checks and automatic failover, and graceful degradation instead of hard failures. Chasing an extra nine of availability gets exponentially more expensive and complex, so the target should match actual business need rather than being set as high as possible by default.

**Interview Answer:**
"Availability is the percentage of time the system successfully serves requests, usually expressed as a number of nines, like 99.9% or 99.99% uptime per year. Each nine you add cuts allowed downtime by roughly 10x, and each one gets progressively harder and more expensive to achieve, since it usually means more redundancy, more failover automation, and more operational complexity. I'd want to know the target early because it directly shapes how much redundancy the design needs."

---

## Q3. [Junior] Walk me through how you'd estimate QPS for a service with 50 million daily active users, where each user performs about 20 actions per day.

**Answer:**

Back-of-envelope QPS estimation converts a daily user count into a request rate you can design around.

**Step by step**

1. Total daily actions: 50,000,000 users multiplied by 20 actions each equals 1,000,000,000 actions per day.
2. Convert to average QPS: divide by the number of seconds in a day, roughly 86,400. That gives about 11,574 requests per second on average.
3. Account for peak traffic: real traffic is not evenly spread across 24 hours. A common rule of thumb is that peak traffic runs 2 to 3x the average, so peak QPS would be roughly 25,000 to 35,000 requests per second.
4. State your assumptions out loud: are all 20 actions the same type of request, or a mix of reads and writes? If it's a mix, break it down, for example 18 reads and 2 writes per user per day, and calculate QPS separately for each since reads and writes often hit different systems.

**Why this matters**

This number directly drives decisions later: how many servers you need, whether a single database can handle the write load, and whether you need caching or a CDN to absorb read traffic. **Getting the order of magnitude right matters far more than precision to the decimal point.**

**Interview Answer:**
"I'd start with total daily actions: 50 million users times 20 actions is 1 billion actions a day. Dividing by 86,400 seconds gives about 11,500 QPS on average. Since traffic isn't evenly distributed, I'd apply a peak multiplier, typically 2 to 3x, landing around 25,000 to 35,000 QPS at peak. I'd also ask whether those actions are mostly reads or writes, since that changes which part of the system needs to scale first."

---

## Q4. [Junior] You're told a service needs to store 10 million records that average 1 KB each. How much total storage is that, and how would you think about growth over the next 3 years?

**Answer:**

This is a straightforward multiplication problem, but the value of the question is in showing your thought process, not just the final number.

**Base calculation**

10,000,000 records multiplied by 1 KB equals 10,000,000 KB, which is about 10 GB.

**Thinking about growth**

- **Ask whether 10 million is the current count or the day-one count.** If new records are added over time, you need a growth rate, for example "500,000 new records per month."
- Project forward: at 500,000 per month, that's 6 million new records per year, or 18 million over 3 years, roughly 1.8x the starting size in raw record count.
- Multiply the 3-year total record count by the per-record size to get 3-year storage: roughly 28 million records times 1 KB is about 28 GB.
- Add a buffer for replication. If data is replicated 3x for durability, multiply the total by 3, giving about 84 GB.
- Add a buffer for indexes and metadata, which commonly add 20 to 50 percent overhead on top of raw data size.

**Why this matters**

Interviewers care less about the exact final gigabyte figure and more about whether you remember to account for growth, replication, and overhead rather than stopping at the naive multiplication.

**Interview Answer:**
"10 million records at 1 KB each is 10 GB of raw data today. For 3-year growth, I'd ask for a growth rate, say 500,000 new records a month, which adds up to about 18 million more records over 3 years, so roughly 28 GB of raw data by year three. Then I'd factor in replication, commonly 3x for durability, and index overhead of maybe 20 to 50 percent, which could put the real footprint closer to 80 to 100 GB."

---

## Q5. [Junior] Design a URL shortening service like bit.ly. What questions would you ask before writing anything on the board?

**Answer:**

Before designing, it helps to pin down scope, scale, and behavior so the design targets the real problem instead of a generic guess.

**Scope questions**

- Do we need custom aliases and expiration dates, or just auto-generated short codes?
- Do we need analytics, like click counts per link?

**Scale questions**

- How many URLs are created per day, and what's the read to write ratio? **Shortened links are usually read far more often than they're created.**
- How long do links need to stay alive: forever, or with an expiration policy?

**Behavior questions**

- What happens if two users try to shorten the same long URL, do they get the same short code or different ones?
- Should short codes be guessable or unguessable, does that matter for security?

Once these are answered, a typical follow-up design covers: a hashing or counter-based scheme for generating short codes, a key-value store for the mapping (a simple database that looks up a value directly by its key, like a giant dictionary), a cache in front of it since reads dominate, and a redirect service that returns an HTTP 301 or 302.

**Interview Answer:**
"I'd first ask about scope: do we need custom aliases, expiration, or click analytics, or is this just create and redirect? Then scale: how many links are created per day and what's the read to write ratio, since shortened URLs are typically read far more than written. Then behavior: does the same long URL always map to the same short code? Those answers tell me whether I need a cache, how I generate codes, and how much storage and throughput to design for."

---

## Q6. [Junior] I want you to design a system for a logistics company. That's all I'm going to tell you, go ahead.

**Answer:**

This prompt is intentionally underspecified, and the right first move is not to start drawing boxes, it's to ask questions that narrow "logistics company" down to a concrete problem.

**Questions to ask first**

1. What part of logistics: package tracking, route optimization, warehouse inventory, or delivery dispatch?
2. Who are the users: internal operations staff, drivers, or end customers tracking a package?
3. What scale are we talking about: hundreds of packages a day or millions?
4. Is this read-heavy (customers checking status) or write-heavy (constant location updates from drivers)?
5. Are there real-time requirements, like live map tracking, or is periodic status polling acceptable?

**Why this matters**

"Logistics company" could mean a package tracking API, a real-time fleet GPS system, or a warehouse management system, and each of those has a completely different data model and bottleneck. **Picking one without asking would mean designing the wrong system entirely, no matter how good the execution is.**

Once the scope is narrowed, for example to "a package tracking system where customers check delivery status and drivers push location updates," the rest of the interview can proceed like a normal system design problem: define functional requirements, non-functional requirements, and estimate scale.

**Interview Answer:**
"Before I design anything, I'd narrow this down. Logistics covers a lot of ground: package tracking, route planning, warehouse management, fleet dispatch. I'd ask which of those we're focused on, who the primary users are, roughly what scale we're operating at, and whether there's a real-time component like live location updates. Once I have a specific problem, like customer-facing package tracking with driver location updates, I can define functional and non-functional requirements and move into the actual design."

---

## Q7. [Mid] Design a system for a photo sharing app similar to Instagram. Give me the first five questions you would ask before diving into the design.

**Answer:**

For a broad, feature-rich product like this, the first questions should cut the huge possible surface area down to a design you can actually finish in the time available.

**Five questions to ask**

1. What's the core feature set for this session: upload and view photos, a following-based feed, likes and comments, or all of it? I'd want to agree on a smaller slice to design deeply rather than a shallow pass over everything.
2. What's the scale: how many daily active users, and roughly how many photos are uploaded per day?
3. Is the feed generated in real time when requested, or pre-computed and pushed to users, since that decision drives a large part of the architecture?
4. Do we need to support video as well as photos, since video changes storage and encoding requirements significantly?
5. What are the latency expectations for loading the feed and for uploading a photo, since those are very different operations with different tolerances?

**Why this matters**

**Instagram-scale prompts are deliberately broad.** Asking these questions upfront prevents spending 15 minutes designing a generic photo storage system only to discover the interviewer actually wanted a deep dive on feed generation.

**Interview Answer:**
"I'd start by narrowing scope: are we focusing on upload and storage, the feed, or social interactions like likes and comments, since trying to design all of it shallowly wastes time. Then I'd ask about scale, daily active users and photo upload volume, whether the feed is computed on read or pushed ahead of time, whether video is in scope, and what latency targets matter for feed loads versus uploads. Those five answers tell me where to spend the bulk of the interview."

---

## Q8. [Mid] A product manager tells you "the system needs to be fast." How do you turn that into a concrete, measurable requirement?

**Answer:**

"Fast" is not a design constraint by itself, it's a signal that a real number is missing, and the job is to extract that number through targeted follow-up questions.

**Questions to ask**

- Fast for which operation specifically? Reading a cached page and writing a financial transaction have very different acceptable latencies.
- What's the target in milliseconds, and is that an average, a p95, or a p99 (the latency below which 95% or 99% of requests fall)? **A system can have a great average latency while a meaningful fraction of users experience terrible tail latency.**
- Is there an existing baseline being compared against, like "faster than our current system's 800ms response time"?
- Does "fast" mean end-to-end user-perceived latency, including network and rendering, or just server-side processing time?

**Turning it into a requirement**

Once answered, restate it concretely, for example: "The search endpoint should return results in under 200ms at the 95th percentile, measured server-side, excluding client rendering time." That statement is testable, it can be measured in production, and it gives a clear target for choosing between architectural options like adding a cache or an index.

**Interview Answer:**
"I wouldn't accept 'fast' as a requirement, I'd push for specifics: fast for which operation, what's the actual target in milliseconds, and is that an average or a percentile like p95 or p99, since tail latency often matters more than the average. I'd also ask if there's a current baseline we're trying to beat. I'd turn the answer into something concrete and testable, like 'p95 latency under 200ms for the search endpoint,' so it can actually guide design decisions and be measured later."

---

## Q9. [Mid] Walk me through estimating the storage needed for a service that lets 5 million daily active users upload one photo per day, averaging 2 MB per photo, retained for 5 years.

**Answer:**

**This estimate combines a daily volume figure with a long retention window, so it's worth working through both pieces separately before combining them.**

**Step by step**

1. Daily raw upload volume: 5,000,000 users times 2 MB equals 10,000,000 MB, or about 10 TB per day.
2. Annual volume: 10 TB per day times 365 days is about 3.65 PB per year.
3. Five-year volume: 3.65 PB times 5 is about 18.25 PB, assuming a flat daily active user count and no growth, which is worth flagging as a simplifying assumption.
4. Add replication: photo storage typically needs durability, often 3x replication or erasure coding with similar overhead, pushing the figure toward roughly 55 PB.
5. Add thumbnails and multiple resolutions: apps almost always generate several derivative sizes for feeds, thumbnails, and full view, which can add another 20 to 40 percent on top.

**Practical implication**

At this scale, the answer isn't "buy bigger disks," it's object storage like a blob store, with a CDN in front for frequently accessed images and a tiered storage strategy that moves older, rarely accessed photos to cheaper cold storage.

**Interview Answer:**
"Daily upload volume is 5 million users times 2 MB, about 10 TB a day, or roughly 3.65 PB a year. Over 5 years, assuming flat usage, that's about 18 PB of raw data. Factoring in 3x replication for durability pushes it to around 55 PB, and generating multiple thumbnail sizes adds more on top. At that scale I'd design around object storage with a CDN for hot images and a lifecycle policy to move older photos into cheaper cold storage."

---

## Q10. [Mid] Explain the difference between average QPS and peak QPS, and why an interviewer would care whether you calculate both.

**Answer:**

Average QPS is total requests divided evenly across a time period, usually a full day. Peak QPS is the highest request rate the system actually experiences during a shorter, busier window.

**Why they diverge**

- Traffic isn't uniform. Most consumer applications see a surge during daytime and evening hours and a lull overnight, so a handful of hours can carry a disproportionate share of daily traffic.
- Seasonal or event-driven spikes make the gap even larger, like a retail site on a sale day or a ticketing platform when a popular event goes on sale.
- A common estimation shortcut is to assume peak is 2 to 3x average for typical consumer traffic, though this should be validated against the actual product if real data is available.

**Why it matters for the design**

If you only design for average QPS, the system will fall over during exactly the moments it matters most, like a shopping site during a flash sale. Capacity planning, auto-scaling thresholds, and load testing targets all need to be based on peak, not average. **Designing purely for average load is a common mistake that looks fine on paper and fails in production.**

**Interview Answer:**
"Average QPS spreads total daily requests evenly across 24 hours, but real traffic clusters around certain hours, so peak QPS, the busiest short window, is usually 2 to 3x higher than average for typical consumer apps, and can be far higher during specific events. I calculate both because if I only size the system for average load, it'll fail exactly when it matters most, during the traffic spike, so capacity planning and auto-scaling need to be built around peak, not average."

---

## Q11. [Mid] Design a ride sharing app's backend. How would you scope down the problem to an MVP you could realistically design in 45 minutes?

**Answer:**

A ride sharing platform touches matching, pricing, payments, maps, notifications, and driver management. **Trying to cover all of it in 45 minutes guarantees a shallow, unfinished design, so the goal is to pick a core slice and go deep.**

**Scoping approach**

1. Identify the core user journey: a rider requests a trip, the system matches them to a nearby available driver, and the trip proceeds to completion. This is the heart of the product.
2. Explicitly defer secondary features: dynamic pricing, driver ratings, payment processing, and promotions can be mentioned as out of scope for this session, with a one-line acknowledgment that they exist.
3. Pick the hardest part of the core journey to focus depth on, typically driver matching, since it involves geospatial queries, real-time location updates, and a matching algorithm under time pressure.

**Resulting MVP definition**

"A rider opens the app, requests a ride from their current location, the system finds and assigns the nearest available driver within a few seconds, and both parties get real-time trip status until drop-off." That's narrow enough to design end to end with real depth in the time available, and it can be explicitly stated as the plan before writing anything on the board.

**Interview Answer:**

"I'd narrow to the core journey: a rider requests a trip, gets matched to a nearby driver, and the trip completes, and I'd explicitly call out pricing, payments, and ratings as out of scope for now so we're aligned on where the depth is going. I'd spend the bulk of the time on driver matching specifically, since that's the hardest part, involving geospatial lookups and real-time location data, rather than spreading effort thin across every feature the product has."

---

## Q12. [Mid] Estimate the bandwidth required for a video streaming service with 1 million concurrent viewers watching a 5 Mbps stream.

**Answer:**

**Bandwidth estimation for streaming multiplies concurrent viewers by the per-stream bitrate, since bandwidth is about data flowing right now, not data accumulated over a day.**

**Step by step**

1. Total bandwidth: 1,000,000 concurrent viewers multiplied by 5 Mbps equals 5,000,000 Mbps, which is 5,000 Gbps, or 5 Tbps.
2. Convert to bytes if needed for infrastructure planning: 5 Tbps divided by 8 is about 625 GB per second of data being served.
3. Recognize this is not served from one place: a single origin server or even a single data center cannot push 5 Tbps, so this immediately motivates a CDN, where the bulk of viewers are served from edge locations close to them rather than a central origin.
4. Ask about bitrate variability: real streaming services use adaptive bitrate, offering multiple quality levels, so viewers on weaker connections might stream at 1 or 2 Mbps instead of 5, which would lower the actual aggregate figure and should be reflected as a range rather than a single number.

**Practical implication**

This number justifies architectural choices: a CDN with regional edge caching, adaptive bitrate streaming to reduce load on constrained clients, and origin servers that only need to serve a small fraction of total traffic since most requests are cache hits at the edge.

**Interview Answer:**

"Multiplying 1 million concurrent viewers by 5 Mbps gives 5 Tbps of total bandwidth, or about 625 GB per second. No single origin can serve that, so this number is really the justification for a CDN, where most

viewers are served from nearby edge nodes rather than a central server. I'd also flag that real streaming uses adaptive bitrate, so the actual aggregate bandwidth would vary depending on each viewer's connection quality, making 5 Tbps more of an upper bound than a fixed figure."

---

## Q13. [Mid] Design a news feed system. What are the read and write patterns you'd want to understand before deciding your data model?

**Answer:**

**A news feed's architecture is driven almost entirely by the shape of its read and write traffic, so understanding that shape before designing is essential.**

**Read pattern questions**

- How often does a typical user refresh or scroll their feed compared to how often new content is posted? Feeds are usually read far more often than they're written to.
- Does the feed need to be real-time, or is a delay of a few seconds or minutes acceptable?
- How far back does a user typically scroll? This affects whether you need to keep generating older feed content or can cap history.

**Write pattern questions**

- What's the distribution of how many followers a typical account has, and are there outliers, like accounts with millions of followers, since these create very different write loads?
- Are posts and interactions like likes or comments the only writes, or does the system also need to handle edits and deletes that ripple through the feed?

**Why this drives the data model**

If reads dominate heavily and follower counts are mostly small, a fan-out-on-write model, where a post is pushed into each follower's feed at write time, keeps reads fast and cheap. If some accounts have millions of followers, fan-out-on-write becomes prohibitively expensive for those posts, so a hybrid approach is common: fan-out-on-write for typical accounts and fan-out-on-read, generating the feed at request time, for high-follower accounts.

**Interview Answer:**
"I'd want to understand the read to write ratio, since feeds are almost always read far more than written, and whether the feed needs to be real-time or can tolerate a short delay. On the write side, I'd ask about follower count distribution, since a normal user with a few hundred followers and a celebrity with millions of followers need different approaches. That's what pushes me toward a hybrid model: fan-out-on-write for typical users to keep reads fast, and fan-out-on-read for high-follower accounts to avoid an expensive write storm."

---

## Q14. [Mid] A business gives you an SLA of 99.99% uptime for a payment processing system. Translate that into concrete design constraints.

**Answer:**

**99.99% uptime allows about 52 minutes of downtime per year, which is a tight budget that rules out a lot of simple architectural choices.**

**What this SLA forces into the design**

1. No single point of failure: every critical component, application servers, load balancers, and the database, needs at least one redundant instance ready to take over.
2. Multi-availability-zone deployment at minimum, since a single data center outage would likely burn through the entire yearly downtime budget in one incident.
3. Automated failover: manual intervention is too slow, a person paged at 3am can easily take 10 to 20 minutes to respond, which alone could be a meaningful fraction of the annual budget.
4. Health checks and circuit breakers, a circuit breaker stops calling a dependency that's clearly failing instead of letting every request wait on it, so that a failing dependency, like a downstream fraud check service, degrades gracefully rather than taking the whole payment flow down.
5. Careful handling of deploys: rolling deployments and the ability to roll back quickly, since bad deploys are a common cause of outages that count against the SLA.
6. Idempotency for payment operations, an idempotent operation produces the same result no matter how many times it's repeated, since retries during failover could otherwise cause duplicate charges, and duplicate charges on a payment system are arguably worse than downtime.

**Why this matters**

The SLA number by itself is abstract, but translating it into "we cannot tolerate manual failover" and "we need redundancy in every tier" is what actually shapes the architecture diagram, the on-call process, and the deployment strategy.

**Interview Answer:**

"99.99% uptime is about 52 minutes of downtime a year, which rules out manual failover and single points of failure anywhere in the stack. I'd design with redundant instances across multiple availability zones, automated health checks and failover, circuit breakers around downstream dependencies like fraud checks, and rolling deployments with fast rollback. For payments specifically, I'd also make sure operations are idempotent, since retries during a failover could otherwise cause duplicate charges, which would be worse than the downtime itself."

---

## Q15. [Mid] How would you budget your time in a 45-minute system design interview across requirements, estimation, high-level design, and deep dives?

**Answer:**

Time budgeting matters because running out of time during the deep dive, having never gotten past a shallow high-level design, is one of the most common ways this interview goes poorly.

**A reasonable allocation**

1. Requirements gathering, 5 to 7 minutes: clarify functional scope, non-functional priorities, and agree on what's in and out of scope for the session.
2. Back-of-envelope estimation, 3 to 5 minutes: QPS, storage, and bandwidth at a level of precision that informs design choices, not a math exercise for its own sake.
3. High-level design, 12 to 15 minutes: draw the major components, data flow, and API shape, and get to a coherent end-to-end picture before diving into any one piece.
4. Deep dive, 15 to 20 minutes: pick 1 to 2 components with the interviewer, often the hardest or most interesting part, like the matching algorithm in a ride sharing app or the fan-out strategy in a feed system, and go deep on trade-offs, failure modes, and scaling.
5. Wrap-up, 2 to 3 minutes: summarize the design, mention what was deferred, and note how it would evolve at 10x scale if there's time.

**Why flexibility matters**

**This is a guideline, not a rigid script.** If the interviewer signals they want to spend more time on one section, like pushing hard on estimation, it's worth following that lead rather than rigidly sticking to a pre-planned split, since the interviewer's questions are the clearest signal of what they actually want to evaluate.

**Interview Answer:**

"Roughly, I'd spend the first 5 to 7 minutes on requirements, 3 to 5 minutes on back-of-envelope estimation, then 12 to 15 minutes getting a coherent high-level design end to end before diving into any one piece. That leaves 15 to 20 minutes for a deep dive on the hardest component, and a couple minutes at the end to summarize and mention what I'd tackle next at higher scale. I'd stay flexible though, if the interviewer wants to push deeper on one section, I'd follow that instead of sticking rigidly to my plan."

---

## Q16. [Mid] Design a rate limiter for an API gateway. What's the first clarifying question you'd ask about scale?

**Answer:**

A rate limiter's design changes significantly depending on how many distinct clients it needs to track and how much traffic flows through it, so scale is the right place to start.

**First questions to ask**

- How many unique clients or API keys need independent rate limits, hundreds, thousands, or millions? This determines whether limiter state can fit comfortably in memory on a single node or needs a distributed store.
- What's the request volume the gateway handles overall, since the rate limiter sits on the hot path of every request and cannot become the bottleneck itself.
- Is the gateway a single instance or a fleet of many instances behind a load balancer? If it's a fleet, rate limit counters need to be shared across instances, typically through a centralized store like Redis, rather than kept locally, or a client could exceed their limit by hitting different instances.

**Why this shapes the design**

**If there's a small number of clients and a single gateway instance, an in-memory counter with a sliding window, an algorithm that counts requests in a moving time window instead of a fixed one, avoiding a burst right at a window boundary, is simple and sufficient.** If there are millions of clients across a distributed fleet of gateway instances, the design needs a shared, low-latency store for counters, careful attention to the extra latency that store adds to every request, and a strategy for what happens if that store becomes temporarily unavailable, since a failed rate limit check should probably fail open rather than block all traffic.

**Interview Answer:**

"I'd first ask how many unique clients need independent limits and whether the gateway runs as a single instance or a fleet behind a load balancer, since that determines whether I can get away with in-memory counters or need a shared store like Redis. If it's a distributed fleet, local counters would let a client exceed their limit by spreading requests across instances, so I'd need centralized, low-latency counters, and I'd also want to decide upfront whether to fail open or closed if that store becomes unavailable."

---

## Q17. [Senior] Walk me through estimating QPS, storage, and bandwidth end to end for a Twitter-like service with 200 million daily active users, each posting 2 tweets a day and reading 100 tweets a day.

**Answer:**

**This question chains three estimation types together, so the key is keeping each one clearly separated and stating assumptions as you go.**

**QPS**

1. Writes: 200,000,000 users times 2 tweets equals 400,000,000 tweets per day. Divided by 86,400 seconds, that's about 4,630 write QPS on average.
2. Reads: 200,000,000 users times 100 tweet reads equals 20,000,000,000 reads per day. Divided by 86,400, that's about 231,500 read QPS on average.
3. Apply a peak multiplier of roughly 2 to 3x: peak writes land around 10,000 to 14,000 QPS, and peak reads around 460,000 to 700,000 QPS.
4. Note the read to write ratio here is roughly 50 to 1, which strongly suggests heavy caching and read replicas, extra copies of the database that only serve read queries, rather than trying to scale the write path to match.

**Storage**

1. Assume an average tweet, with metadata, is about 300 bytes of text plus overhead, call it 1 KB including metadata and indexing for simplicity.
2. 400,000,000 tweets per day times 1 KB is about 400 GB of new data per day, or roughly 146 TB per year before replication.
3. With media attachments, if say 20 percent of tweets include an image averaging 200 KB, that adds roughly 16 TB per day on top, which dominates the total and should be called out explicitly as the bigger driver of storage than the text itself.

**Bandwidth**

1. Outbound bandwidth is dominated by reads: 20 billion reads a day, each maybe 1 to 2 KB of payload including some media thumbnails, works out to tens of terabytes per day of egress, which is a strong signal to lean heavily on CDN caching for popular or trending content.

**Why this matters**

The exercise isn't really about the exact digits, it's about correctly identifying that reads dominate QPS by roughly 50x, and that media, not text, dominates storage, since those two conclusions directly drive the architecture toward heavy caching, read replicas, and a separate media storage and delivery pipeline.

**Interview Answer:**

"Writes come out to about 400 million tweets a day, roughly 4,600 QPS average and maybe 12,000 at peak. Reads are 20 billion a day, about 230,000 QPS average and up to 700,000 at peak, so reads outnumber writes by roughly 50 to 1, which tells me to invest heavily in caching and read replicas rather than scaling writes to match. For storage, tweet text is a few hundred gigabytes a day, but attached media likely adds an order of magnitude more, so media storage and a CDN become the real architectural driver, not the tweet text itself."

---

## Q18. [Senior] You're given a vague prompt: "Design something like Google Docs." How do you narrow the scope without asking so many questions that you eat up all your time?

**Answer:**

The risk with a broad, feature-rich prompt like this is two failure modes at once: asking so few questions that you design the wrong thing, or asking so many that you burn 15 minutes before drawing a single box. **The fix is to ask a small number of high-leverage questions and then propose a scope rather than waiting to be told one.**

**High-leverage questions, kept to three or four**

1. Is the core challenge here real-time collaborative editing, meaning multiple people typing in the same document at once, or is it closer to a document storage and sharing product with version history?
2. Roughly what scale, in terms of concurrent editors per document and total documents, are we designing for?
3. Do we need to support rich formatting and embedded objects, or is plain text sufficient for this session?

**Proposing scope instead of waiting**

After a couple of quick answers, state the plan directly: "I'll focus on real-time collaborative editing for a single document, multiple users typing concurrently with changes syncing live, and I'll set aside sharing permissions, commenting, and version history unless we have time at the end." Stating this out loud does two things: it shows deliberate scoping rather than guessing, and it gives the interviewer an easy chance to redirect if that's not the part they wanted to focus on.

**Why this balance matters**

Asking is valuable, but only up to the point of resolving genuine ambiguity that would send the design in a different direction. Once the ambiguity that matters is resolved, further questions have diminishing returns and should be replaced by a stated assumption plus a design that can be adjusted if it's wrong.

**Interview Answer:**

"I'd ask three quick things: is the hard part real-time collaborative editing or more document storage and sharing, what scale of concurrent editors per document we're targeting, and whether rich formatting is in scope. Then rather than asking more, I'd state my scope directly: real-time collaborative editing on a single document as the core, with sharing and version history set aside unless we have time. That gets me moving quickly while giving the interviewer an easy opening to redirect if I've guessed wrong."

---

## Q19. [Senior] For a global e-commerce checkout system, walk me through how you'd prioritize consistency versus availability versus latency, and how that priority might differ across the different subsystems involved: inventory, payment, cart.

**Answer:**

A checkout flow is actually several subsystems with different correctness requirements stitched together, and treating them all the same way leads to either an overly conservative cart or an unsafe inventory count.

**Cart**

- Priority: availability and low latency over strict consistency.
- Reasoning: if the cart briefly shows slightly stale contents across two devices, it's a minor inconvenience, not a financial error, and users strongly prefer a cart that always responds instantly.

**Inventory**

- Priority: closer to strong consistency, but the exact mechanism depends on how tight the inventory is.

- Reasoning: overselling a popular item disappoints customers and creates costly refund and support work, but a full distributed lock on every inventory check would crush latency and throughput. A common middle ground is to use an optimistic check at reservation time, a short-lived reservation that decrements available stock, combined with a final consistent check at payment confirmation, rather than serializing every single cart addition against the master inventory count.

**Payment**

- Priority: strong consistency, non-negotiable.
- Reasoning: double-charging or losing a charge is a direct financial and trust failure. This subsystem should favor correctness even if it means higher latency or, in rare failure scenarios, temporarily rejecting a checkout rather than risking an inconsistent charge state. Idempotency keys on payment requests are essential here to make retries safe.

**Best Choice**

Treat the three subsystems differently rather than picking one consistency model for the whole checkout flow: available and fast for the cart, a reservation-based near-consistent model for inventory, and strictly consistent for payment.

**Why?**

The cost of being wrong is completely different in each case: a stale cart is invisible to the business, oversold inventory is an operational cost, and a payment error is a direct financial and legal liability. **Good system design matches the consistency guarantee to the actual cost of inconsistency, subsystem by subsystem, rather than applying one blanket policy across an entire flow.**

**Interview Answer:**

"I wouldn't apply one consistency model to the whole checkout flow. The cart can favor availability and low latency, since a briefly stale cart across devices is harmless. Inventory I'd lean toward a reservation-based approach, an optimistic hold at add-to-cart with a final consistent check at payment, balancing overselling risk against throughput. Payment has to be strongly consistent with idempotent requests, since double-charging or dropping a charge is a real financial and trust problem. The right consistency level really depends on the actual cost of being wrong in each subsystem."

---

## Q20. [Senior] Your back-of-envelope math says a service needs 3 GB/day of storage, but given the numbers we discussed earlier, that seems off by a lot. Walk me back through your math.

**Answer:**

The right response to this kind of pushback is not to defend the original number, it's to retrace the calculation step by step out loud, since the goal of estimation in an interview is transparency of reasoning, not protecting a prior answer.

**How to retrace it**

1. Restate the inputs that were agreed on earlier, for example "we said 10 million daily active users, each uploading one photo averaging 500 KB."
2. Redo the multiplication explicitly: 10,000,000 times 500 KB equals 5,000,000,000 KB, which is about 5 TB per day, not 3 GB.

3. Identify where the error likely crept in: a common mistake is dropping a unit conversion, for example treating KB as if it were MB, or forgetting to multiply by the daily active user count at all and only calculating for a much smaller sample.
4. State the corrected number clearly and note the two to three orders of magnitude difference explicitly, since silently fixing it without acknowledging the size of the gap can look like the error wasn't understood.

**Why this response matters**

**Being off on a first pass is normal and not itself a problem, everyone makes unit conversion slips under time pressure.** What matters is being able to methodically find the error when it's flagged, rather than hand-waving past it or doubling down on an answer that's clearly wrong. This also demonstrates the habit of sanity-checking estimates against intuition, since 3 GB/day for 10 million users is obviously too small the moment you say it out loud.

**Interview Answer:**

"Let me redo it. We said 10 million daily active users each uploading one 500 KB photo, so that's 10 million times 500 KB, which is 5,000,000,000 KB, or about 5 TB a day, not 3 GB. I think I dropped a unit conversion earlier, likely treating kilobytes as megabytes. That's actually a difference of almost three orders of magnitude, so thanks for catching it, 5 TB a day is the number I'd carry forward into the rest of the design."

---

## Q21. [Senior] Design a distributed messaging system like WhatsApp. Walk me through how message delivery guarantees change your requirements gathering compared to a typical CRUD service.

**Answer:**

A typical CRUD service usually tolerates "best effort" semantics, if a request fails, the client can retry and the user notices little difference. **A messaging system carries an implicit promise that a sent message actually arrives, and that promise needs to be made explicit before any design work starts.**

**Extra requirements questions specific to messaging**

1. What delivery guarantee is expected: at-most-once, at-least-once, or exactly-once? Most chat systems target at-least-once delivery combined with client-side deduplication, since true exactly-once across a distributed system is expensive and often unnecessary if duplicates can be safely discarded.
2. What ordering guarantee is expected within a conversation? Users expect messages to appear in the order they were sent, at least within a single chat, which affects whether you need per-conversation sequencing rather than relying purely on arrival time at the server.
3. What happens when the recipient is offline? Messages need to be durably stored and delivered on reconnect, which means the system needs persistent per-user message queues, not just a live pub-sub broadcast.
4. Do we need delivery and read receipts? This adds another data path, tracking per-message state per recipient, distinct from the message content path itself.
5. Is end-to-end encryption in scope? This changes where message content can be inspected, cached, or indexed, which affects search and moderation features.

**Why this changes the design fundamentally**

These answers push the design toward a durable, ordered, per-conversation log rather than a simple database table of messages, plus a separate delivery tracking mechanism and an online presence system to

know when to push versus queue. None of this complexity would surface if requirements gathering treated this like a generic "store and retrieve records" CRUD problem.

**Interview Answer:**

"Unlike a typical CRUD service, a messaging system carries an implicit guarantee that sent messages actually arrive, so I'd ask about delivery semantics, usually at-least-once with client-side deduplication, and ordering within a conversation. I'd also ask what happens when a recipient is offline, since that requires durable, per-user queues rather than pure real-time broadcast, whether read receipts are needed, since that's a separate state-tracking path from the message content itself, and whether end-to-end encryption is in scope, since that affects search and moderation entirely."

---

## Q22. [Senior] How would you estimate the number of servers needed to handle peak traffic for a ticket sales platform, knowing that traffic on sale day can spike 50x over average?

**Answer:**

This kind of extreme, event-driven spike is a different problem than steady-state peak traffic, and it changes both the estimation approach and the infrastructure strategy.

**Step by step**

1. Establish baseline average QPS the normal way: total daily requests divided by seconds in a day.
2. Apply the stated spike factor directly rather than a generic peak multiplier: if average is, say, 2,000 QPS, a 50x spike means designing for around 100,000 QPS during the sale window.
3. Estimate per-server capacity through a reasonable assumption or known benchmark, for example "each application server can handle roughly 500 requests per second under this workload," then divide: 100,000 divided by 500 is 200 servers needed at peak.
4. Add a safety margin, commonly 20 to 30 percent, to absorb uneven load distribution and avoid running at exactly 100 percent capacity, which pushes the estimate to roughly 250 to 260 servers.

**Why raw server count isn't the whole answer**

**Provisioning 250 servers permanently for a spike that lasts a few minutes a handful of times a year is wasteful.** The real design conversation is about elasticity: auto-scaling that can react fast enough, or more realistically, pre-warming capacity ahead of a known sale time since reactive auto-scaling is often too slow for a spike this sudden and steep. It's also worth mentioning a queue in front of the ticket purchase flow, so that instead of trying to serve 100,000 concurrent purchase attempts instantly, users are placed in a virtual waiting room and processed at a rate the backend can sustain, which is exactly the pattern real ticketing platforms use.

**Interview Answer:**

"I'd calculate baseline average QPS, then apply the given 50x spike factor directly rather than a generic 2 to 3x multiplier, since this is an extreme, known event. If baseline is 2,000 QPS, that's 100,000 QPS at peak. Dividing by an assumed per-server capacity gives a raw server count, plus a safety margin. But the more important point is that permanently provisioning for that spike is wasteful, so I'd push toward pre-warmed auto-scaling ahead of the known sale time, and a queue-based waiting room in front of the purchase flow so the backend processes demand at a sustainable rate instead of getting hit all at once."

## Q23. [Senior] Design a system for a global ad click tracking pipeline expected to process 1 million events per second at peak. What non-functional requirements dominate this design, and why?

**Answer:**

At this event rate, the dominant concerns shift away from typical request-response latency and toward throughput, durability, and correctness of aggregation under massive parallel load.

**Dominant non-functional requirements**

1. Throughput above almost everything else: the system has to sustain sustained ingestion at 1 million events per second without falling behind, since a backlog compounds quickly at this scale and can take a long time to drain.
2. Durability without full synchronous consistency: losing a small percentage of click events is often acceptable for billing accuracy at aggregate scale, but losing large chunks or an entire time window is not, since that directly affects advertiser billing and trust. This usually points toward a durable, replicated log like a distributed commit log rather than a synchronous write to a traditional relational database.
3. Idempotency and deduplication: at this scale, network retries and at-least-once delivery are unavoidable, so downstream consumers need a way to detect and discard duplicate click events, since double-counting clicks directly inflates advertiser bills.
4. Eventual consistency for aggregates: real-time dashboards can tolerate a short delay, seconds to low minutes, before reflecting the latest click counts, which is what allows batching and buffering to absorb the volume rather than trying to update a single global counter synchronously for every click.
5. Horizontal scalability of ingestion: the ingestion layer needs to be partitioned, typically by advertiser or ad campaign ID, so that load spreads across many machines rather than bottlenecking on a single writer.

**Why these dominate over things like strict latency**

**Nobody is waiting on a click event response in real time the way they wait on a page load, so per-event latency in the tens of milliseconds is fine.** What breaks the system at this scale is a component that can't scale horizontally, a synchronous consistency requirement that limits throughput, or a lack of deduplication that corrupts billing data, so those are the requirements to nail down first.

**Interview Answer:**

"At 1 million events a second, throughput and horizontal scalability dominate over strict latency, since nobody's waiting on an individual click response the way they wait on a page load. I'd prioritize a durable, replicated log for ingestion rather than synchronous writes to a relational database, partitioned by campaign ID so load spreads across machines. Idempotency and deduplication matter a lot too, since at-least-once delivery is unavoidable at this scale and duplicate clicks would inflate advertiser billing. Aggregation can be eventually consistent, dashboards updating within seconds is completely acceptable."

## Q24. [Senior] A stakeholder tells you the new inventory system needs to be "eventually consistent is fine everywhere." Do you push back on that, and if so, where?

**Answer:**

This is worth pushing back on, because "everywhere" is almost never actually true once you look at the specific operations inventory systems perform, and accepting the blanket statement without examining it risks building overselling straight into the design.

**Where eventual consistency is genuinely fine**

- Displaying a product's stock level on a browsing page, like "in stock" or "only 3 left," can tolerate a short staleness window. A slightly outdated count here is a minor UX issue, not a correctness failure.
- Analytics and reporting on inventory trends over time can clearly tolerate delay.

**Where it likely is not fine**

- The actual decrement that happens when a customer completes checkout needs a consistency guarantee strong enough to prevent two customers from both successfully purchasing the last unit of an item. If this path is fully eventually consistent with no coordination, overselling becomes a predictable, recurring problem rather than a rare edge case.
- Reversals, like an order cancellation restocking inventory, need to reliably apply, or reported stock will drift from physical reality over time, and now that drift compounds silently, which is arguably worse than an outage because it goes unnoticed until a customer complaint or an audit finds it.

**How to push back constructively**

Rather than rejecting the direction outright, propose the split explicitly: "Browsing and display can absolutely be eventually consistent, that's the right choice for performance. But I'd want a stronger consistency guarantee, like a reservation with a short-lived lock or a conditional decrement, specifically at the point of checkout, otherwise we risk overselling limited-stock items." **This reframes the conversation from a binary consistency choice to identifying exactly which operation needs the guarantee.**

**Interview Answer:**

"I would push back, but narrowly. Eventual consistency is genuinely fine for displaying stock counts while browsing, since a slightly stale number is harmless. But the actual checkout decrement needs a stronger guarantee, otherwise two customers could both successfully buy the last unit of a limited item, which is overselling, not a minor inconsistency. I'd propose keeping browsing eventually consistent for performance, but using a conditional decrement or short-lived reservation at the moment of purchase, so the blanket statement gets replaced with a per-operation decision."

---

## Q25. [Senior] You have 10 minutes left in an interview and haven't finished discussing non-functional requirements or done any capacity estimation. What do you do?

**Answer:**

**With 10 minutes left and two significant gaps, the priority is to deliver the most valuable signal possible rather than trying to force both topics in fully and rushing both.**

**Triage the remaining time**

1. Quickly state the non-functional requirements out loud as assumptions rather than a full discussion: "given what we've covered, I'm going to assume we need high availability given this is user-facing, and low write latency since this is on the critical path of checkout, let me know if that's wrong." This takes 30 seconds and closes most of the gap without a full back-and-forth.

2. Spend the bulk of the remaining time, 6 to 7 minutes, on capacity estimation, since it's usually more concrete, more clearly gradeable, and directly connects to design decisions already made, like whether a single database can handle the write load discussed earlier.
3. Tie the estimation numbers back to the design explicitly: "at 50,000 peak QPS, a single database instance won't keep up, which is why I put a cache and read replicas in the design earlier," rather than leaving the numbers floating disconnected from the diagram.
4. In the final minute, explicitly name what was skipped: "if we had more time, I'd want to dig into replication strategy and failure recovery in more detail."

**Why this order over the alternative**

Non-functional requirements can be reasonably stated as informed assumptions and verified quickly, while capacity estimation is a skill that's harder to fake or gloss over and is usually weighted heavily, so it deserves the larger remaining share of time rather than splitting 10 minutes evenly and doing both poorly.

**Interview Answer:**

"I'd quickly state the non-functional requirements as explicit assumptions rather than a full discussion, something like assuming high availability and low write latency given this is user-facing and on the critical path, and confirm that's reasonable. Then I'd spend the bulk of the remaining time on capacity estimation, since it's concrete and ties directly back into decisions I've already made in the design, like why I added caching. In the last minute, I'd name what I didn't get to, like replication details, so it's clear I know it's still open."

---

## Q26. [Senior] Design a multi-region deployment for a system with a legal requirement that European user data never leaves the EU. Walk me through how that constraint changes your requirements gathering and your estimation.

**Answer:**

**A data residency requirement like this is a non-functional constraint that overrides a lot of otherwise natural architectural choices, and it needs to be surfaced early since it affects nearly every layer of the design.**

**How it changes requirements gathering**

1. Ask exactly what "data" is covered: just personally identifiable information, or all associated records including logs, backups, and analytics events, since the scope of the constraint determines how much of the system needs regional isolation versus just the sensitive fields.
2. Ask whether EU users can ever be served by a non-EU region during a regional outage, or whether availability must be sacrificed to maintain the residency guarantee, since these two requirements can directly conflict during a failover scenario.
3. Ask whether cross-region features are needed at all, like a global leaderboard or a friend list that spans EU and non-EU users, since that data would need careful handling to avoid pulling EU personal data across the boundary while still supporting the feature.
4. Clarify backup and disaster recovery expectations, since backups are often overlooked but are still a copy of the data and would violate the same residency rule if replicated to a non-EU region.

**How it changes estimation**

- Capacity and storage estimates now need to be done per region rather than as one global pool, since EU traffic and EU data must be sized and provisioned within EU infrastructure specifically, and cannot be smoothed out by shifting spare capacity from another region.

- Cross-region replication for disaster recovery needs to happen within EU-only infrastructure, for example across multiple EU availability zones or EU-only regions, rather than to a global backup region, which may mean less redundancy diversity than a system without this constraint and should be called out as a trade-off.
- Latency estimation for EU users becomes simpler in one sense, since they're guaranteed to be served from EU infrastructure, but any feature that needs to reference non-EU data, like a global counter, now requires a cross-region call pattern that has to be designed carefully to avoid pulling restricted data back across the boundary.

**Why this matters**

Getting this wrong isn't just a performance or cost problem, it's a compliance and legal risk, so this constraint should be treated as a hard boundary condition on the architecture, decided before any regional topology is drawn, rather than an optimization to bolt on afterward.

**Interview Answer:**

"I'd first pin down exactly what counts as covered data, just personal information or everything including logs and backups, and whether EU users can ever fail over to a non-EU region during an outage, since that could directly conflict with the residency requirement. Estimation then has to be done per region rather than globally, since EU capacity and storage can't borrow from other regions, and disaster recovery replication has to stay within EU infrastructure, which likely means less redundancy diversity than I'd normally design for, and that trade-off is worth stating explicitly."

---

**If you remember nothing else from this chapter:** *nailing requirements gathering means you've already gotten a large share of the interview right, no matter what you draw next.*

# Chapter 4: Networking, Client-Server & HTTP Basics

Nearly every system design and backend discussion assumes you have a working knowledge of how a request actually moves across the network. If you're shaky here, it will catch up with you later in the interview the moment the conversation touches latency, connection handling, or protocol choice, even if you don't immediately realize why. Work through these questions to close that gap early, before you're thirty minutes into a design round that depends on it.

**War Story:** *A stubborn "works on my machine" bug once turned out to be an engineer testing over HTTP/1.1 while production ran behind a load balancer that only behaved correctly under HTTP/2, an entire day burned chasing a phantom race condition that was actually a protocol mismatch.*

**Avoid:** treating HTTP status codes as decoration rather than a contract with real meaning • not being able to explain what changes in browser behavior between a 301 and a 302

**Aim For:** reaching for the right verb and status code without being prompted • explaining connection reuse or keep-alive trade-offs unprompted

## Q1. [Junior] What is the client-server model, and why do we structure most systems this way instead of having every machine talk to every other machine directly?

**Answer:**

**The client-server model splits responsibility between two roles.** A client initiates requests, typically a browser, mobile app, or another service. A server listens for requests, processes them, and sends back a response. The server usually owns the data and the business logic, while the client is responsible for presenting results and capturing user input. For example, when you load a banking app, your phone is the client requesting your account balance, and the bank's server is the one holding and returning that data.

**Why this structure wins over peer to peer for most systems**

- Centralizing data and logic on the server makes it much easier to control access, apply consistent business rules, and keep data consistent across every user.
- Clients can be thin and simple, since heavy lifting like storage, computation, and coordination lives on the server.
- It's easier to scale, secure, and monitor a small number of servers than to trust and coordinate an unbounded number of peer machines.

**Where it breaks down**

- A single server, or even a single cluster, can become a bottleneck or a single point of failure, which is why we add load balancers, replicas, and caching layers on top of this basic model.
- Some workloads, like file sharing or blockchain, genuinely benefit from peer to peer designs where no central authority is needed.

**Interview Answer:**

"The client-server model separates the system into two roles: the client, which initiates requests like loading a page or calling an API, and the server, which owns the data and logic and returns a response. We use it because centralizing state and business rules on the server makes systems easier to secure, scale, and

reason about, compared to a peer to peer model where every node would need to coordinate with every other node directly."

---

## Q2. [Junior] Walk me through what happens, at a high level, from the moment I type a URL into my browser to when the page shows up on my screen.

**Answer:**

This is one of the most common opening questions in networking interviews because it touches almost every concept in this chapter.

**Step by step**

1. The browser parses the URL to extract the protocol, hostname, and path.
2. The browser checks its own cache, then the OS, then a DNS resolver, to translate the hostname into an IP address.
3. The browser opens a TCP connection to that IP address on the appropriate port, port 443 for HTTPS.
4. If it's HTTPS, a TLS handshake happens on top of TCP to negotiate encryption and verify the server's certificate.
5. The browser sends an HTTP request over that encrypted connection, with a method, headers, and possibly a body.
6. The request travels across the network as packets, gets routed through intermediate hops, and reaches the server, possibly passing through a load balancer or reverse proxy first.
7. The server processes the request, maybe talking to a database or other services, and sends back an HTTP response with a status code, headers, and a body.
8. The browser receives the response, parses the HTML, and starts fetching additional resources like CSS, JavaScript, and images, often repeating steps 2 through 7 for each one.
9. The browser renders the page as resources arrive.

**Why this question matters**

**It forces you to connect DNS, TCP, TLS, HTTP, and rendering into one coherent mental model instead of treating them as isolated facts.**

**Interview Answer:**

"First the browser resolves the hostname to an IP address through DNS. Then it opens a TCP connection to that IP, and if it's HTTPS, layers a TLS handshake on top to encrypt the connection. Once that's established, the browser sends an HTTP request, the server processes it and returns a response, and the browser parses that response, fetching any additional assets like CSS or images the same way, before finally rendering the page."

---

## Q3. [Junior] What's the difference between an IP address and a domain name, and why do we need both?

**Answer:**

An IP address is a numeric identifier assigned to a device on a network, like 142.250.72.14, that routers use to actually deliver packets. **A domain name, like example.com, is a human readable label that maps to one or more IP addresses.**

**Why we need both**

- Machines route traffic based on numeric addresses because that's efficient to process and compact to store in packet headers.
- Humans can't reliably remember or type long numeric addresses, especially since a service might use different IPs in different regions or over time.
- Domain names give us a stable identity for a service even when the underlying IP addresses change, for example when a server is replaced or traffic is rerouted.

**A bit more detail**

- IPv4 addresses are 32 bits, written as four decimal numbers like 192.168.1.1, giving about 4.3 billion addresses, which is why IPv6 exists with a much larger 128 bit address space.
- A single domain name can map to multiple IP addresses for load balancing, and a single IP address can sometimes host multiple domains using virtual hosting.

**Interview Answer:**
"An IP address is the actual numeric location routers use to deliver packets, while a domain name is a human friendly alias that DNS translates into one or more IP addresses. We need both because machines need something compact and structured to route on, but people need something memorable, and domain names also let us change the underlying IP addresses, for example during a migration or failover, without anyone needing to update how they refer to the service."

---

## Q4. [Junior] What is DNS, and what would break if it went down?

**Answer:**

DNS, the Domain Name System, is a distributed, hierarchical system that translates human readable domain names into IP addresses. **It works like a phone book that the entire internet queries constantly.**

**How it's structured**

- At the top are root servers, which know where to find servers for each top level domain like .com or .org.
- Below that are TLD servers, which point to the authoritative name servers for a specific domain.
- Authoritative name servers hold the actual DNS records for a domain, like its A record mapping to an IP address.

**What breaks if DNS goes down**

- Browsers and applications would no longer be able to translate domain names into IP addresses, so any service accessed by hostname would become unreachable, even if the servers behind it are perfectly healthy.
- Anything hardcoded to an IP address would still work, which is a useful fact during incident response, but that's rare in practice.
- Because DNS is so foundational, most organizations rely on caching at multiple levels, browser, OS, and resolver, to reduce blast radius when a DNS provider has an outage.

**Interview Answer:**
"DNS is the system that translates domain names into IP addresses, structured as a hierarchy of root servers, top level domain servers, and authoritative name servers for each domain. If DNS goes down, clients can no longer resolve hostnames to IPs, so even perfectly healthy servers become unreachable by name. That's why

DNS records are heavily cached at the browser, OS, and resolver level, and why many companies use multiple DNS providers for redundancy."

---

## Q5. [Junior] What's the difference between TCP and UDP?

**Answer:**

Both are transport layer protocols that sit on top of IP and define how data actually moves between two endpoints, but they make very different trade-offs. For example, loading a webpage over HTTPS relies on TCP so no byte of the page is lost or scrambled, while a live video call uses UDP so a dropped frame is simply skipped rather than stalling the whole call waiting for a retransmit.

**TCP**

- Connection oriented: requires a handshake before data flows.
- Reliable: guarantees delivery, retransmits lost packets, and delivers data in order.
- Has flow control and congestion control to avoid overwhelming the network or receiver.
- Higher overhead per packet due to acknowledgments and sequencing.

**UDP**

- Connectionless: no handshake, you just start sending datagrams.
- Unreliable: no guarantee of delivery, ordering, or duplicate protection.
- Minimal overhead, which makes it fast and lightweight.
- Leaves reliability, if needed, to the application layer.

**Best Choice**

**Depends entirely on the use case: TCP for anything that needs correctness and completeness, UDP for anything that needs speed and can tolerate loss.**

**Why?**

TCP is the right choice when every byte matters and order matters, like loading a web page or transferring a file, because a missing chunk would corrupt the result. UDP is the right choice when a fresher packet is more valuable than a late, guaranteed one, like video calls, live game state, or DNS queries, where retransmitting old data is often pointless.

**Interview Answer:**

"TCP is connection oriented and reliable, it guarantees ordered, complete delivery through handshakes, acknowledgments, and retransmission, which makes it right for things like web pages or file transfers where every byte matters. UDP skips all of that overhead, it just sends datagrams with no delivery guarantee, which makes it faster and better suited for real time use cases like video calls or gaming, where a late retransmitted packet is often worse than just dropping it and moving on."

---

## Q6. [Junior] What does it mean when we say HTTP is a stateless protocol?

**Answer:**

**Stateless means the server doesn't retain any memory of previous requests from a client.** Each HTTP request is processed independently, using only the information contained in that request itself, headers, cookies, body, and query parameters.

**Why this matters**

- The server doesn't need to know anything about a client's request history to handle the current one, which makes it simple to scale, since any server behind a load balancer can handle any request.
- It also means anything that looks like "state" from the user's perspective, like being logged in, has to be explicitly carried on each request, usually through a cookie or token, or reconstructed from a database.

**The trade-off**

- Statelessness makes horizontal scaling and failover easy, since there's no session data pinned to a specific server that would be lost if that server goes down.
- It pushes the burden of tracking identity and session data to the client, or to a shared store like Redis, rather than to server memory.

**Interview Answer:**

"Stateless means the server treats every HTTP request as a fresh, standalone event, it doesn't remember anything about previous requests from that client. Anything that needs to persist across requests, like being logged in, has to be carried explicitly, usually through a cookie or token that the client sends with every request, or looked up from a shared data store. This is actually a feature, because it means any server behind a load balancer can handle any request, which makes scaling much simpler."

---

## Q7. [Junior] What's the difference between latency and throughput, and why can't you always improve one by improving the other?

**Answer:**

**Latency is how long a single request takes from start to finish, usually measured in milliseconds.** Throughput is how much work the system can handle over a period of time, usually measured in requests per second.

**Why they're independent**

- A system can have very low latency but low throughput, for example a single threaded server that responds instantly to one request but can only handle one at a time.
- A system can have high throughput but high latency, for example a batch pipeline that processes millions of records per hour but takes minutes for any individual record to complete.
- Adding more servers, or more threads, generally increases throughput by letting more work happen in parallel, but it doesn't make any individual request finish faster, in fact contention for shared resources can make individual requests slower.

**A concrete analogy**

Think of a highway. Latency is how long it takes one car to drive from one end to the other. Throughput is how many cars pass a given point per hour. Adding more lanes increases throughput without changing how fast any single car travels, and a traffic jam can increase latency without actually reducing the total number of cars that eventually get through.

**Interview Answer:**

"Latency measures how long one request takes, throughput measures how many requests the system handles per unit of time. They don't move together, you can have a fast single request that only happens once at a time, low latency but low throughput, or a system processing huge volumes where each individual item still takes a while, high throughput but high latency. Scaling horizontally usually improves throughput by adding parallel capacity, but it won't make an individual request complete faster."

## Q8. [Mid] Walk me through DNS resolution step by step, starting from when the browser doesn't have anything cached.

**Answer:**

This question checks whether you actually understand the hierarchy behind "DNS translates names to IPs" rather than just the one line summary.

**Step by step, assuming a completely cold cache**

1. The browser checks its own local cache, finds nothing.
2. The OS checks its local resolver cache, finds nothing.
3. The request goes to a recursive resolver, often provided by the ISP or a public service.
4. The recursive resolver queries a root name server, which doesn't know the answer but returns a referral to the correct top level domain, TLD, server, for example the .com server.
5. The resolver queries the TLD server, which returns a referral to the authoritative name server for the specific domain.
6. The resolver queries the authoritative name server, which returns the actual record, typically an A or AAAA record with the IP address.
7. The recursive resolver caches this result according to its TTL and returns it to the OS, which also caches it, and finally to the browser.

**Why caching matters at every layer**

Each layer caches the result for the record's TTL, so subsequent lookups for the same hostname are much faster and don't repeat this whole chain. **This is also exactly why the first request to a brand new hostname is slower than later ones.**

**Interview Answer:**
"With nothing cached, the browser and OS both miss, so the query goes to a recursive resolver, which asks a root server, gets referred to the right TLD server, gets referred again to the domain's authoritative name server, and finally gets the actual IP address back. That answer gets cached at the resolver, OS, and browser level according to its TTL, so every subsequent lookup for that hostname is fast until the TTL expires, which is also why the very first request to a new host has extra DNS latency baked in."

## Q9. [Mid] When would you choose UDP over TCP for a system you're designing?

**Answer:**

**The decision comes down to whether the application can tolerate loss and reordering better than it can tolerate the overhead and latency of guaranteed delivery.**

**Good fits for UDP**

- Live video and voice calls, where a dropped or late frame is better replaced by a newer one than retransmitted, since by the time it arrives it's stale anyway.
- Real time multiplayer games, where the next position update supersedes an old one, so retransmitting a missed packet just adds lag.
- DNS queries, small, simple request-response exchanges where retrying at the application layer is cheap and simpler than paying for a TCP handshake every time.

- Metrics and telemetry pipelines, where losing an occasional data point is acceptable and you'd rather not backpressure the whole system waiting for acknowledgments.

**Why not just always use TCP**

TCP's reliability guarantees come from retransmission and in order delivery, both of which introduce latency, especially under packet loss, since TCP will stall delivering later data until an earlier missing packet is recovered. For latency sensitive, loss tolerant workloads, that stall is worse than just dropping the packet.

**Best Choice**

UDP, often with a thin application layer protocol built on top to add just the reliability features you actually need, like QUIC does.

**Why?**

You get to choose exactly which guarantees you pay for instead of accepting TCP's one size fits all reliability model, which is why so much modern infrastructure, including HTTP/3, is built on top of UDP.

**Interview Answer:**
"I'd reach for UDP when the workload can tolerate loss better than it can tolerate the latency of TCP's retransmission and ordering guarantees, things like live video, voice, real time game state, or DNS queries. In those cases a stale retransmitted packet is often worse than just dropping it and sending the next update. If I need some reliability without full TCP overhead, I'd build a thin custom layer on top of UDP, which is exactly the approach QUIC and HTTP/3 take."

---

## Q10. [Mid] What actually happens to a piece of data, say a JSON object in my application, between the moment I call an API and the moment it arrives at the server as bytes on a wire?

**Answer:**

**This question is testing whether "sending data over the network" is a black box to you or something you can actually trace through layers.**

**Step by step**

1. **Serialization**: your in memory object, like a JSON object or a struct, gets converted into a byte representation, commonly JSON text, protobuf, or another wire format. This is necessary because memory representations like pointers and object references don't mean anything outside your process.
2. **Framing at the application layer**: the HTTP layer wraps those bytes with a request line, headers, and a body, forming a complete HTTP message.
3. **Transport layer segmentation**: TCP breaks that message into segments if it's larger than what fits in a single unit, adding sequence numbers so the receiver can reassemble it in order.
4. **Network layer packetization**: each TCP segment gets wrapped in an IP packet with source and destination IP addresses, and packets may be further fragmented depending on the network's maximum transmission unit.
5. **Link layer framing**: each packet is wrapped again into a frame with MAC addresses for the actual physical hop, whether that's Ethernet or Wi-Fi.
6. **Transmission**: these frames become electrical signals, light pulses, or radio waves, physically transmitted across the medium, routed hop by hop based on IP addresses until they reach the destination.

7. **On arrival**: the receiving machine reverses this process, unwrapping link, network, and transport headers, reassembling the original byte stream, and handing it to the receiving application, which deserializes those bytes back into an object it can work with.

**Interview Answer:**
"The object first gets serialized into bytes, typically JSON or protobuf, since raw in memory structures like pointers don't mean anything outside the process. Those bytes get wrapped into an HTTP message, which TCP breaks into segments, which get wrapped into IP packets with source and destination addresses, which get wrapped again into link layer frames for the physical hop. On the wire it's all just electrical or optical signals, and the receiving machine reverses every one of those steps, ending with deserialization back into an object the server code can use."

---

## Q11. [Mid] Why is the first request to a service often noticeably slower than the requests that follow?

**Answer:**

**This is almost always a stack of several distinct costs that only get paid once, then get amortized away by caching.**

**The main contributors**

- **DNS resolution**: if the hostname isn't cached anywhere, resolving it requires walking the full DNS hierarchy, which takes real time. Later requests reuse the cached record until its TTL expires.
- **TCP handshake**: establishing a new TCP connection requires a round trip before any data can flow. If the connection is kept alive and reused, later requests skip this entirely.
- **TLS handshake**: for HTTPS, there's an additional round trip, or more, to negotiate encryption parameters and verify certificates, though session resumption can shortcut this for reconnects.
- **Cold caches on the server side**: the first request might miss application level caches, trigger a cold start on a serverless function, or require warming up a database connection pool.
- **JIT and class loading**: in managed runtimes, the first call into a code path might trigger just in time compilation or lazy class loading that later calls skip.

**Why this matters in practice**

This is why health checks and warmup requests exist, why connection pooling and keep alive matter so much, and why serverless cold starts are a real operational concern worth designing around, sometimes via provisioned concurrency or periodic pings to keep instances warm.

**Interview Answer:**
"The first request usually pays for several one time costs that later requests skip: resolving DNS if it isn't cached, completing a TCP handshake, and for HTTPS, a TLS handshake on top of that, plus warming up any server side caches, connection pools, or cold serverless instances. Once the connection is established and caches are warm, subsequent requests reuse all of that, which is exactly why connection keep alive, DNS caching, and warmup strategies like provisioned concurrency matter so much for latency sensitive services."

---

## Q12. [Mid] If HTTP is stateless, how do we actually keep a user logged in across multiple requests?

**Answer:**

Since the server doesn't remember anything between requests by default, the client has to send enough information with every request for the server to reconstruct who it's talking to.

**Common approaches**

- **Cookies with server side sessions**: after login, the server creates a session record, usually in Redis or a database, and gives the browser a cookie containing a session ID. The browser sends that cookie automatically on every subsequent request, and the server looks up the session to know who's making the request.
- **Tokens, like JWTs**: instead of a server side session, the server issues a signed token containing the user's identity and claims. The client sends that token, typically in an Authorization header, and the server verifies its signature without needing to look anything up, since the token itself carries the state.

**Options**

**Server side sessions**

- Pros: easy to revoke immediately, since you just delete the session record, and you can store more data server side without bloating what's sent over the wire.
- Cons: requires a shared session store if you have multiple servers, adding infrastructure and a dependency.

**Tokens**

- Pros: stateless verification, no shared store needed, easy to scale across many servers.
- Cons: harder to revoke before expiry, since the token is self contained and valid until it expires unless you add a blocklist.

**Best Choice**

**Depends on the system, session cookies for traditional web apps with a central backend, tokens for distributed systems and APIs consumed by many different clients.**

**Why?**

Session cookies keep things simple and revocable for a monolith style backend, while tokens avoid a shared state dependency in a microservices world where many independent services need to verify identity without all calling back to one session store.

**Interview Answer:**
"HTTP itself doesn't remember anything, so we simulate state by having the client send identifying information on every request. The two common patterns are server side sessions, where a cookie carries a session ID that the server looks up against a shared store like Redis, and tokens like JWTs, where the client sends a signed token containing the user's identity that the server can verify without a lookup. I'd pick sessions for a simpler app that wants easy revocation, and tokens for a distributed system where many services need to verify identity independently."

---

## Q13. [Mid] What's the difference between a load balancer and an API gateway? Don't they both sit in front of your services and route traffic?

**Answer:**

They overlap in that both receive incoming traffic and forward it somewhere, but they're solving different layers of the problem, and in a real architecture you often use both together. For example, a load balancer

might spread traffic across ten identical order-service instances, while the API gateway sitting in front of it also checks the caller's auth token and applies rate limiting before the request ever reaches the load balancer.

**Load balancer**

- Primary job: distribute incoming traffic across multiple instances of the same service to spread load and provide failover.
- Operates at connection or request level, often layer 4 or layer 7, using algorithms like round robin, least connections, or consistent hashing.
- Generally unaware of business logic, it doesn't care what the request means, just where to send it.

**API gateway**

- Primary job: act as a single entry point for many different backend services, handling cross cutting concerns.
- Typically handles authentication, rate limiting, request validation, protocol translation, response transformation, and routing requests to the correct microservice based on path or header.
- Aware of API semantics, versioning, and often integrates with service discovery (a way for a service to look up a current, healthy address for another service instead of relying on a hardcoded one).

**Best Choice**

Use both, layered: an API gateway in front for routing, auth, and rate limiting, with a load balancer behind it, or built into it, distributing traffic across the healthy instances of whichever service the gateway routed to.

**Why?**

A load balancer alone doesn't understand API concerns like authentication or per client rate limits, and an API gateway alone typically isn't optimized to be the raw, high throughput traffic distributor that a dedicated load balancer is. **Together, the gateway handles "which service and is this request even allowed," and the load balancer handles "which specific healthy instance of that service should get it."**

**Interview Answer:**

"A load balancer distributes traffic across multiple instances of the same service to spread load and handle failover, it generally doesn't understand what the request means. An API gateway is a single entry point for many different services, handling things like authentication, rate limiting, and routing based on the API path, so it's aware of business and API level concerns. In practice they're complementary, I'd put an API gateway in front to handle routing and cross cutting concerns, with load balancing happening behind it, or built in, to spread traffic across healthy instances of whatever service it routed to."

---

## Q14. [Mid] Give me examples of when you'd return a 400, a 401, a 403, and a 404, and explain the actual difference between them.

**Answer:**

**These are all client error codes, meaning the client did something that prevented a successful response, but they mean very different things and mixing them up creates confusing APIs.**

**The distinctions**

- **400 Bad Request**: the request itself is malformed or fails validation, like a missing required field, invalid JSON, or a field that fails a format check. The client needs to fix the request itself.

- **401 Unauthorized**: the client hasn't proven who they are, meaning authentication failed or is missing entirely, like a missing or expired token. Despite the name, it's really about authentication, not authorization.
- **403 Forbidden**: the client is authenticated, the server knows who they are, but they're not allowed to perform this action, like a regular user trying to access an admin only endpoint.
- **404 Not Found**: the resource being requested simply doesn't exist at this URL, or the server is deliberately hiding its existence, which some APIs use intentionally instead of 403, to avoid revealing that a resource exists to someone who shouldn't know about it.

**Why getting this right matters**

Clients, monitoring systems, and API consumers use these codes to decide how to react automatically, retry, prompt for re-login, or give up, so returning the wrong code, like always returning 400 for everything, breaks that automation and makes debugging much harder for anyone integrating with your API.

**Interview Answer:**
"400 means the request itself is malformed, bad JSON or a missing field, and the client needs to fix it. 401 means authentication is missing or invalid, the server doesn't know who's asking. 403 means the server knows who you are but you're not allowed to do this. 404 means the resource doesn't exist at that path, or the server is deliberately not revealing whether it does. Getting these right matters because clients and monitoring tools react differently to each one, so collapsing them all into a generic error code breaks that."

---

## Q15. [Mid] Walk me through the TCP three way handshake, and explain why it adds latency before any actual data can be sent.

**Answer:**

**Before TCP can transmit application data, both sides need to agree that a connection exists and synchronize some initial bookkeeping, which takes a full round trip.**

**The handshake**

1. The client sends a SYN packet to the server, proposing an initial sequence number.
2. The server responds with a SYN-ACK packet, acknowledging the client's sequence number and proposing its own.
3. The client responds with an ACK packet, acknowledging the server's sequence number.
4. Only after this exchange completes can either side send actual application data.

**Why it adds latency**

Each of those three packets has to physically travel across the network, so the handshake alone costs one full round trip time, RTT, before a single byte of the actual HTTP request goes out. If you're also using TLS on top, that adds another one or two round trips before the encrypted request can be sent, so the very first exchange with a new server can cost several times the latency of later requests on the same connection.

**Why this matters for design**

This is exactly why HTTP connection reuse, keep alive, and connection pooling matter so much, reusing an existing TCP connection means skipping this handshake entirely for every request after the first, and it's also why protocols like QUIC, used in HTTP/3, are designed to reduce or eliminate this round trip cost by combining the transport and encryption handshakes.

**Interview Answer:**

"The three way handshake is SYN from the client, SYN-ACK from the server, then ACK from the client, and only after that can either side send real data. Each of those packets is a network trip, so the handshake alone costs a full round trip before anything useful happens, and adding TLS on top adds more round trips for the encryption negotiation. That's why keeping connections alive and reusing them matters so much for latency, and why newer protocols like QUIC try to collapse the transport and encryption handshakes into fewer round trips."

---

## Q16. [Mid] A client tells you they got a 200 OK response from your API, but they insist the request actually failed. How is that possible, and what would you check?

**Answer:**

A 200 status code only tells you that the HTTP transaction itself completed successfully, the server received the request and returned a response without a transport or server error. **It says nothing about whether the business operation the request represented actually succeeded.**

**Common reasons this happens**

- The API returns a 200 with an error encoded inside the response body, like `{"success": false, "error": "insufficient funds"}`, a pattern that's common but generally considered poor API design because it forces every client to parse the body just to know if the call worked.
- The request succeeded at one layer, for example the write was accepted and queued, but failed asynchronously later, and the 200 was returned before that downstream failure occurred.
- A caching layer or proxy served a stale cached 200 response that no longer reflects the current state.
- The client is misinterpreting a successful response, for example expecting a field that the API never actually promised to return.

**What I'd check**

1. Look at the actual response body, not just the status code, to see if there's an embedded error.
2. Check server side logs and traces for that specific request ID to see what actually happened downstream.
3. Check whether a cache or CDN sat between the client and the server and might have served a stale response.
4. Review the API contract to confirm what a 200 is actually supposed to guarantee here.

**Interview Answer:**
"A 200 only confirms the HTTP layer succeeded, it doesn't guarantee the business operation succeeded. I'd first check the response body for an embedded error or success flag, since some APIs conflate application errors with a 200 status, which honestly is a design smell. Then I'd trace that specific request through server logs to see if something failed asynchronously after the response was already sent, and I'd check whether a cache or proxy in between served a stale response instead of hitting the real backend."

---

## Q17. [Mid] What is TCP head of line blocking, and why does it matter for HTTP performance?

**Answer:**

**TCP guarantees in order delivery, which means if one packet is lost, every packet that arrived after it, even if they're already sitting in the receiver's buffer, has to wait until the missing one is retransmitted and arrives, before any of that data can be handed to the application.** For example, if a page loads its HTML,

CSS, and JSON data over one HTTP/2 connection and the packet carrying the CSS is lost, the JSON response sitting right behind it in the same TCP stream still has to wait, even though the browser could otherwise use it immediately.

**Why this hurts HTTP specifically**

- With HTTP/1.1 or HTTP/2, multiple requests and responses can be multiplexed over a single TCP connection to avoid opening many connections. That's great when everything arrives cleanly.
- But because it's all one TCP stream, a single lost packet belonging to just one of those multiplexed requests stalls delivery of every other request's data on that connection too, even though they're logically independent, this is head of line blocking at the transport layer.
- The result is that on a lossy network, like flaky mobile connections, multiplexing over TCP can actually make things feel worse, since one bad packet freezes everything sharing that connection.

**How this gets solved**

This is one of the main motivations behind QUIC and HTTP/3, which runs over UDP instead of TCP and implements its own independent streams, so a lost packet only blocks the specific stream it belongs to, not every other request multiplexed on the same connection.

**Interview Answer:**
"TCP guarantees ordered delivery, so if a packet is lost, everything behind it in the stream has to wait for that retransmission before the application can process any of it, even data that already fully arrived. That becomes a real problem for HTTP/2 style multiplexing, where many independent requests share one TCP connection, because a single lost packet for one request ends up stalling all the others sharing that connection. That's a core reason HTTP/3 moved to QUIC over UDP, where each stream can be blocked independently instead of the whole connection freezing."

---

## Q18. [Senior] You're building the networking layer for a real time multiplayer game where players see each other's positions update several times a second. Would you use TCP or UDP for position updates, and how would you actually structure it?

**Answer:**

**For position updates specifically, UDP is almost always the right call, but the interesting part of this question is what you build on top of it to make it usable.**

**Why UDP for position updates**

- Position updates are inherently ephemeral, a stale position from 200 milliseconds ago is actively harmful to render, since the player has already moved. Retransmitting a lost update and waiting for it is worse than just dropping it and using the next fresher one.
- TCP's ordered, reliable delivery would mean a single lost packet stalls delivery of every subsequent, newer position update behind it, which is exactly the opposite of what you want for something this latency sensitive.

**What I'd actually build on top of UDP**

- A sequence number on each update packet, so the client can detect out of order or duplicate packets and simply discard stale ones instead of trying to reconstruct exact ordering.
- Periodic full state snapshots plus lightweight delta updates in between, so a client that missed a few packets can resync quickly on the next snapshot instead of drifting forever.

- Client side interpolation and extrapolation, so the game still looks smooth to the player even if a packet or two is dropped.

**Where I'd still use TCP**

Anything where correctness matters more than freshness: chat messages, matchmaking, inventory changes, purchases, or anything that changes persistent account state. I'd run that over a separate reliable channel, either a second TCP connection or a reliable layer built on top of UDP, and keep it entirely separate from the fast, lossy position update stream.

**Interview Answer:**

"I'd use UDP for position updates, because a late, retransmitted update is worse than a dropped one, the player has already moved on. I'd add sequence numbers so clients can discard stale or out of order packets, mix in periodic full snapshots with deltas in between so a client that missed packets can resync quickly, and use client side interpolation to smooth over any gaps. For anything where correctness matters more than freshness, chat, purchases, inventory, I'd keep that on a separate reliable channel over TCP instead of mixing it into the fast path."

---

## Q19. [Senior] Your service's p99 latency looks fine, but throughput is dropping under load, users on the support team are reporting slow page loads even though your dashboards say latency is stable. Walk me through how that's possible and how you'd investigate.

**Answer:**

**This is a classic case of a metric telling a technically true but misleading story, and the investigation is really about finding where latency and throughput are decoupled.**

**How this can happen**

- p99 measures the latency of requests that actually complete and get measured. If a growing number of requests are timing out, getting dropped, or queued indefinitely upstream, at a load balancer or client library, they may never even reach the point where your p99 metric measures them, so your dashboard looks clean while real users experience failures or long waits before the request even lands.
- Throughput dropping under load, while measured latency looks stable, often points to a saturated resource somewhere upstream of where you're measuring, like connection pool exhaustion, a queue filling up, or a downstream dependency rejecting connections, none of which show up in a latency histogram that only tracks completed requests.

**How I'd investigate**

1. Check whether request volume reaching the service actually matches request volume being sent by clients, look for drops at the load balancer or gateway layer.
2. Look at connection pool metrics, both incoming and to downstream dependencies like the database, for saturation or queueing.
3. Check timeout and retry rates specifically, since those requests may be excluded from your latency percentiles entirely.
4. Look at queue depth on any async processing steps, a growing queue behind a fixed consumer rate will drop effective throughput even if each individual job's processing latency hasn't changed.

5. Correlate with client side or CDN level metrics if available, since server side dashboards only see what actually reaches the server.

**Interview Answer:**
"The likely explanation is that p99 only measures requests that actually complete and get recorded, so if requests are timing out, queueing, or getting rejected before they reach that measurement point, they're invisible to the dashboard even though users are experiencing them as failures or slowness. I'd check for drops between client and server request counts, look at connection pool and queue saturation both incoming and to downstream dependencies, and specifically look at timeout and retry rates, since those are exactly the requests that would be missing from a clean looking latency histogram."

---

## Q20. [Senior] Right after a deploy, you start seeing a spike in 502 and 504 errors from your load balancer. Walk me through how you'd debug this.

**Answer:**

**502 and 504 both mean the load balancer itself is healthy and reachable, but it's having trouble with the backend it's trying to talk to, which points the investigation squarely at the newly deployed instances.**

**What each code actually tells you**

- **502 Bad Gateway**: the load balancer got a response from the backend, but it was invalid or malformed, or the backend connection was reset unexpectedly, often because the new process crashed mid response or isn't speaking the expected protocol correctly.
- **504 Gateway Timeout**: the load balancer never got a response from the backend within its timeout window, which usually points to the backend being overloaded, stuck, or simply not ready to accept connections yet.

**What I'd check, roughly in order**

1. Confirm the rollout sequencing, are new instances being added to the load balancer's healthy pool before they've actually finished starting up and passing health checks.
2. Check application logs on the new instances for startup errors, crash loops, or slow initialization, like a database connection pool that hasn't finished warming up.
3. Check whether health check configuration matches actual readiness, a health check that returns healthy before the app is truly ready to serve traffic will cause the load balancer to route real requests to an instance that isn't ready.
4. Check for a sudden change in request handling time on the new version, maybe a new code path is slower or a downstream dependency call was added, pushing requests past the load balancer's timeout.
5. Check connection draining on old instances, if they're being terminated before finishing in flight requests, that also produces errors during the transition window.

**Interview Answer:**
"502 means the load balancer got a bad or broken response from the backend, 504 means it never got a response in time, so both point at the backend, specifically the instances that just changed. I'd first check whether new instances are being marked healthy and added to the pool before they're actually ready to serve traffic, then look at their startup logs for crashes or slow initialization, and check whether the health check is actually validating readiness rather than just process liveness. I'd also check connection draining on the old instances, since killing them mid request causes the exact same symptoms."

## Q21. [Senior] Design the layer of proxies, gateways, and load balancers you'd put in front of a public API that needs authentication, rate limiting, and routing to a dozen backend microservices.

**Answer:**

**This is really asking you to compose the concepts from this chapter into one coherent architecture, so I'd walk through it as layers from the internet inward.**

**Layer 1: DNS and edge**

Public hostname resolves, via DNS, to the entry point of the infrastructure, typically a managed layer 4 or layer 7 load balancer or a CDN edge in front of it, though deep CDN behavior is out of scope here.

**Layer 2: TLS termination and L7 load balancing**

Incoming connections terminate TLS here, so backend services don't each need to manage certificates, and requests get load balanced across multiple instances of whatever sits behind this layer for redundancy.

**Layer 3: API gateway**

This is where cross cutting, API aware concerns live:

- **Authentication**: validate tokens or API keys before anything else happens, reject unauthenticated requests immediately without burdening backend services.
- **Rate limiting**: enforce per client or per API key limits here, centrally, rather than duplicating that logic in every microservice.
- **Routing**: inspect the request path or headers and route to the correct one of the dozen backend microservices.
- **Request validation and protocol translation**: reject malformed requests early, and translate protocols if some backends expose gRPC while the public API is REST, for example.

**Layer 4: per service load balancing**

Once the gateway decides which microservice should handle a request, a load balancer, sometimes built into a service mesh, distributes that request across the healthy instances of that specific service.

**Layer 5: backend microservices**

Each service focuses purely on its own business logic, trusting that authentication and rate limiting already happened upstream, though I'd still validate authorization, not just authentication, at the service level for defense in depth.

**Why layer it this way**

Each layer has a single, well scoped responsibility, which makes the whole system easier to reason about, test, and scale independently, and it avoids duplicating concerns like auth or rate limiting inside every one of a dozen services.

**Interview Answer:**

"I'd layer it: DNS points to a load balancer that terminates TLS, in front of an API gateway that handles authentication, rate limiting, request validation, and routing to the right microservice based on path or header, and behind that, per service load balancing distributes requests across healthy instances of whichever service was selected. The key idea is centralizing cross cutting concerns like auth and rate limiting at the gateway so none of the dozen backend services need to reimplement them, while still doing authorization checks at the service level for defense in depth."

## Q22. [Senior] Your service keeps long lived TCP connections open with clients using HTTP keep alive, and now you need to do a rolling deploy without dropping in flight requests. What actually happens to those open connections, and how do you avoid errors?

**Answer:**

**The core tension is that a rolling deploy needs to terminate old instances, but those instances may be holding open TCP connections with requests still in flight, or waiting to serve the next request on a keep alive connection.**

**What happens if you do nothing special**

- If an instance is killed abruptly, any request currently being processed on it fails immediately, the client sees a connection reset or an empty response.
- Any keep alive connection sitting idle, waiting for the client's next request, also gets severed, and the client's next request on that connection fails until it opens a new one.

**How to avoid this: connection draining**

1. Remove the instance from the load balancer's pool first, so no new connections or requests get routed to it.
2. Signal the instance to stop accepting new requests on existing keep alive connections, often by returning a `Connection: close` header on the next response so well behaved clients open a fresh connection elsewhere.
3. Allow in flight requests already being processed to finish naturally, within a bounded grace period.
4. Only after that grace period, or once all connections are confirmed closed, terminate the instance.

**Other things I'd check**

- Client side retry behavior: idempotent requests should have automatic client retries on connection failure, since even careful draining has edge cases, like a request that started right as the drain began.
- Load balancer health check timing: make sure health checks detect the instance as unhealthy quickly enough that new traffic actually stops before the drain timeout is reached.
- Reasonable drain timeout: long enough for legitimate slow requests to finish, short enough that deploys don't take unreasonably long.

**Interview Answer:**
"Without care, killing an instance mid deploy drops any in flight request on it and severs idle keep alive connections, causing client side errors. I'd handle this with connection draining: pull the instance out of the load balancer pool so it stops receiving new traffic, let it finish requests it's already processing within a bounded grace period, and have it signal clients on keep alive connections to close and reconnect elsewhere, often via a Connection: close header. I'd pair that with idempotent client side retries as a safety net, since some in flight requests can still be caught right at the boundary."

## Q23. [Senior] Walk me through what TLS adds on top of the TCP handshake, and why that makes the very first HTTPS request to a host noticeably slower than the ones that follow.

**Answer:**

**TLS runs its own handshake on top of the already established TCP connection, and that handshake exists to solve a problem TCP doesn't address at all: encryption and identity verification.**

**What the TLS handshake actually does**

1. The client sends a ClientHello, listing supported TLS versions and cipher suites.
2. The server responds with a ServerHello, its chosen cipher suite, and its certificate.
3. The client verifies that certificate against a trusted certificate authority, confirming it's actually talking to the real server and not an impersonator.
4. Both sides perform a key exchange to agree on a shared symmetric encryption key, without ever transmitting that key in the clear.
5. Once this completes, both sides switch to encrypted communication using that shared key for the rest of the connection, since symmetric encryption is much faster than the asymmetric cryptography used during the handshake itself.

**Why the first request pays extra**

- The TCP handshake itself already costs one round trip before this even starts.
- TLS, depending on the version, adds one to two additional round trips on top of that before any encrypted application data, the actual HTTP request, can be sent, since TLS 1.3 reduced this from the two round trips typical in TLS 1.2, but it's still not zero.
- Certificate validation may itself trigger additional network calls, like checking certificate revocation status, adding more latency.
- All of this happens before the very first byte of the actual HTTP request goes out, so a cold connection to a new host can easily cost several times the round trip latency of a request on an already established, warm connection.

**Why later requests are fast**

If the connection is kept alive, all of this is paid once, and every subsequent request reuses the same encrypted TCP connection without repeating the handshake, and even for a brand new connection to a host visited recently, TLS session resumption can skip much of the handshake by reusing previously negotiated parameters.

**Interview Answer:**

"TLS adds its own handshake on top of TCP's, where the client and server agree on a cipher suite, the client verifies the server's certificate against a trusted authority, and both sides perform a key exchange to establish a shared encryption key, before any actual HTTP data can be sent encrypted. Each of those steps costs additional round trips beyond what TCP already cost, so the very first request to a new host can be several times slower than later ones. Once the connection is kept alive, or TLS session resumption is used, later requests skip most or all of that overhead."

---

## Q24. [Senior] Your team wants to move from a single layer of load balancers to a two tier setup, an L4 load balancer in front of a fleet of L7 load balancers. Why would you do that, and what does each layer actually decide?

**Answer:**

**This pattern shows up at real scale because L4 and L7 load balancing solve different problems, and stacking them lets each layer do the part it's actually good at.**

**What an L4 load balancer decides**

- Operates at the transport layer, making decisions based on IP address and port, without inspecting the actual content of the request.
- Extremely fast and lightweight, since it doesn't need to parse HTTP, terminate TLS, or understand application content, it just forwards packets or connections toward a backend based on a simple algorithm.
- Good at absorbing very high volumes of raw connections and providing a stable entry point, often the actual IP addresses published in DNS.

**What an L7 load balancer decides**

- Operates at the application layer, meaning it can inspect the actual HTTP request, path, headers, cookies, and route based on that content.
- Can terminate TLS, do content based routing, cookie based session affinity, and more intelligent health checking based on actual application responses, not just whether a port is open.
- More computationally expensive per request than L4, since it has to actually parse and understand the request.

**Why layer them together**

- The L4 layer handles massive raw connection volume cheaply and distributes it across many L7 load balancer instances, which lets the more expensive, content aware L7 logic scale horizontally rather than becoming a single bottleneck.
- It also gives you a layer of protection and stability, since the L4 tier changes rarely, while L7 load balancer configuration and instances can be updated, scaled, or replaced more dynamically underneath it.

**Interview Answer:**

"An L4 load balancer works at the transport layer, routing based on IP and port without looking at request content, which makes it extremely cheap and able to absorb huge connection volume. An L7 load balancer inspects the actual HTTP request, so it can do content based routing, TLS termination, and cookie based affinity, but that costs more per request. Stacking them lets the L4 tier cheaply spread massive raw traffic across a horizontally scalable fleet of L7 load balancers, so the more expensive, content aware logic isn't a single bottleneck and can scale independently underneath a stable entry point."

---

## Q25. [Senior] We've talked about TCP versus UDP and TCP head of line blocking. How do HTTP/2 and HTTP/3 change that picture?

**Answer:**

**Both protocols were designed to fix HTTP/1.1's limitations around parallelism, but they take different approaches, and HTTP/3 specifically targets the transport layer problem that HTTP/2 couldn't fully solve.**

**HTTP/2**

- Introduces multiplexing: many requests and responses share a single TCP connection using independent streams, instead of HTTP/1.1's approach of needing multiple connections, or serializing requests on one connection, to get parallelism.
- Still runs on top of TCP, which means it inherits TCP's guarantee of strictly ordered delivery for the entire connection.
- Because of that, a single lost packet still stalls every multiplexed stream sharing that TCP connection, even though logically those streams are independent, this is TCP level head of line blocking, and

HTTP/2 doesn't fix it, it actually makes the impact worse than HTTP/1.1 with multiple connections, since now more logical requests share the one connection that can get stalled.

**HTTP/3**

- Runs on QUIC, built on top of UDP instead of TCP, specifically to solve this problem.
- QUIC implements its own reliability and ordering, but per stream rather than per connection, so a lost packet only blocks the specific stream it belongs to, other streams keep flowing.
- QUIC also combines the transport and TLS handshakes into fewer round trips, and supports connection migration, so a connection can survive a client switching networks, like moving from Wi-Fi to cellular, without needing to reconnect from scratch.

**Why this matters**

HTTP/3 essentially took the multiplexing idea from HTTP/2 and rebuilt the transport layer underneath it to actually deliver on independent streams, rather than layering multiplexing on top of a transport, TCP, that was never designed with independent streams in mind.

**Interview Answer:**

"HTTP/2 introduced multiplexing so many requests share a single TCP connection, but because it's still TCP underneath, one lost packet stalls every multiplexed stream on that connection, which is TCP level head of line blocking, and it doesn't actually fix that problem. HTTP/3 solves it by running over QUIC, built on UDP, where reliability and ordering are implemented per stream rather than per connection, so a lost packet only blocks its own stream. QUIC also folds the transport and TLS handshakes together and supports connection migration across network changes, which is why HTTP/3 is the real fix, not just a patch on top of TCP."

---

**If you remember nothing else from this chapter:** *being sloppy about HTTP fundamentals usually means shipping subtle caching and retry bugs later.*

# Chapter 5: Scalability Fundamentals

It's easy to say "I would just add more servers," but much harder to explain what actually has to be true for that to work. This chapter tests whether you understand statelessness, load balancing, and the real limits of horizontal scaling, the difference between having scaled a real system and having only read about it.

**War Story:** *A service that "auto-scaled fine" in every load test fell over in production, because the load tests always warmed the cache first and real traffic never did, so the extra database load during a cold start cascaded into a full outage.*

**Avoid:** saying "just add more servers" to every scaling question without asking whether the service is stateless yet

**Aim For:** asking whether the service is stateless before proposing horizontal scaling • distinguishing between scaling reads and scaling writes

## Q1. [Junior] What's the difference between vertical scaling and horizontal scaling, and when would you reach for each?

**Answer:**

Vertical scaling, also called scaling up, means adding more resources, CPU, RAM, faster disk, to a single existing machine. Horizontal scaling, also called scaling out, means adding more machines and spreading load across all of them.

**Options**

- Vertical scaling
- Pros: simple, no code or architecture changes, no distributed system complexity, works well for naturally single-node systems like a primary database
- Cons: hits a hard hardware ceiling, the machine is a single point of failure, resizing often requires downtime, cost grows steeply at the high end
- Horizontal scaling
- Pros: close to unlimited growth by adding commodity machines, better fault tolerance since losing one node does not take down the system, easier to match capacity to demand
- Cons: requires the application to be stateless or to coordinate shared state, needs a load balancer and service discovery, adds real operational complexity

**Best Choice:** Start vertical while the system is small, but design for statelessness from day one so horizontal scaling is available later without a rewrite.

**Why?** Vertical scaling is the fastest way to buy headroom with zero architectural risk, and for a small system it's the right call. But every system that expects meaningful growth eventually needs horizontal scaling, because a single machine has a hard ceiling and is always a single point of failure. **The cheap insurance is keeping the app stateless early, so scaling out later becomes a configuration and infrastructure change instead of a redesign.**

**Interview Answer:**

"Vertical scaling means making one machine bigger, more CPU, memory, or disk. Horizontal scaling means adding more machines and distributing traffic across them with a load balancer. Vertical is simpler and fine

for small systems, but it hits a ceiling and stays a single point of failure. Horizontal scales much further and tolerates node failures, but it needs the application to be stateless or to externalize state somewhere shared. I'd start vertical for simplicity, but keep the app stateless from the start so scaling out later is a straightforward operational change."

---

## Q2. [Junior] What are the practical limits of vertical scaling, and why can't you just keep buying a bigger box?

**Answer:**

**Vertical scaling runs into limits on several fronts at once, not just raw hardware availability.**

**Hardware ceiling**

- There's a maximum CPU count, memory size, and I/O throughput available on any single machine, even the largest cloud instance types top out.
- Beyond a point, price per unit of extra capacity rises sharply, the biggest instances are disproportionately expensive.

**Availability risk**

- A single machine is a single point of failure. If it goes down, crashes, or needs a reboot for maintenance, the whole service goes down with it.
- Resizing a single machine typically requires downtime or at least a restart, which is disruptive for a live service.

**Diminishing returns**

- Software often doesn't scale linearly with added cores, lock contention (multiple threads waiting on each other for the same lock instead of doing work), single-threaded bottlenecks, and memory bandwidth limits mean doubling CPU doesn't double throughput.
- Vertical scaling does nothing for geographic distribution, users far from that one machine still see high latency.

**Interview Answer:**
"Vertical scaling has a hard ceiling, there's a biggest machine you can buy, and the cost curve gets steep well before you hit it. It's also a single point of failure, if that box goes down or needs a resize, the whole service is affected. And software often doesn't scale linearly with more cores anyway, due to lock contention or single-threaded hot paths, so doubling hardware doesn't guarantee doubling throughput. That's why any system expecting real growth needs a path to horizontal scaling, even if it starts on a single beefy machine."

---

## Q3. [Junior] What does it mean for an application server to be stateless, and why does that matter for scaling?

**Answer:**

A stateless application server does not store any client-specific or session-specific data in its own memory or local disk between requests. **Every request carries or looks up everything it needs, so any server can handle any request.**

**Why it matters for scaling**

- If a server holds session data locally, a client must keep hitting that exact server, which breaks the assumption that any of N servers can serve any request.
- Statelessness is what lets a load balancer distribute requests freely across a pool of interchangeable servers.
- It also makes auto-scaling safe, you can add or remove servers at any time without losing anyone's in-flight session data.

**How you achieve it**

1. Move session data out of process memory into a shared store like Redis or a database.
2. Use tokens (like JWTs) that carry the needed identity and claims with the request itself.
3. Keep any file uploads or generated artifacts in shared object storage, not local disk.

**Interview Answer:**
"Stateless means the server keeps nothing about a specific user or session in its own memory or disk between requests, everything needed to handle a request is either in the request itself or in a shared external store. That matters because it lets a load balancer send any request to any server interchangeably, and it lets you add or remove servers freely for scaling without worrying about losing someone's session. Practically, this means moving session state to something like Redis, using tokens that carry identity, and keeping file storage external rather than local to a box."

---

## Q4. [Junior] Explain how round robin load balancing works.

**Answer:**

**Round robin distributes incoming requests to a pool of servers in a fixed rotating order: request one goes to server A, request two to server B, request three to server C, then back to A, and so on.**

**Mechanics**

- The load balancer keeps an ordered list of healthy backend servers and a pointer to the next one to use.
- Each new request goes to the next server in line, cycling back to the start once it reaches the end.
- A weighted variant assigns more requests to servers with more capacity, for example a bigger server gets a weight of 2 and receives twice as many requests per cycle.

**Strengths and weaknesses**

- Strength: very simple to implement and reason about, low overhead on the load balancer.
- Weakness: it assumes every request is roughly equal cost and every server has equal capacity, which is often false. A server still processing a slow request can get piled on with more work anyway.

**Interview Answer:**
"Round robin sends requests to backend servers in a fixed rotating order, first request to server one, second to server two, and so on, cycling back around. It's simple and has almost no overhead, and a weighted version can send more traffic to bigger servers. The weakness is it doesn't account for how busy a server actually is right now, it just follows the rotation, so if one server is stuck on a slow request it can still get piled on with new ones."

---

## Q5. [Junior] What is a load balancer and where does it sit in a typical web architecture?

**Answer:**

**A load balancer is a component that sits in front of a pool of backend servers and distributes incoming client requests across them, so no single server takes all the traffic.** For example, a load balancer might sit in front of five identical checkout-service instances, forwarding each incoming request to whichever instance is healthy and least busy.

**Where it sits**

1. Client sends a request to a single public address, a domain name or IP that maps to the load balancer.
2. The load balancer picks a healthy backend server using some algorithm, round robin, least connections, or others.
3. It forwards the request to that server and returns the response to the client, often the client never knows which specific backend handled it.

**What it also does**

- Health checks: it periodically pings backend servers and stops sending traffic to any that are unhealthy or unresponsive.
- TLS termination: it can decrypt HTTPS traffic once at the edge, so backend servers deal in plain HTTP internally.
- It can operate at layer 4 (raw TCP/UDP, faster, less aware of content) or layer 7 (HTTP aware, can route by path or header, more flexible).

**Interview Answer:**
"A load balancer sits between clients and a pool of backend servers. Clients connect to one address, and the load balancer decides which backend server actually handles each request, using something like round robin or least connections. It also runs health checks so it stops routing to servers that are down, and it often handles TLS termination so backends only deal with plain HTTP. It can work at the TCP level for speed, or at the HTTP level for smarter routing based on path or headers."

---

## Q6. [Junior] If you take a service that stores session data in each server's local memory and put it behind a load balancer with multiple instances, what breaks?

**Answer:**

**The core problem is that session data now lives in only one place, but requests can land on any server, so most requests will hit a server that has no idea who the user is.**

**What actually breaks**

- A user logs in, their session gets created in server A's memory. Their next request gets routed to server B, which has never seen that session, so they appear logged out.
- Shopping carts, in-progress form state, or any per-user data stored locally behaves inconsistently, sometimes present, sometimes missing, depending on which server happens to answer.
- The failure is intermittent and hard to reproduce, since it depends on load balancer routing decisions.

**How to fix it**

1. Move session state to a shared, external store like Redis, so every server reads from the same place.
2. Alternatively, encode session data into a signed token the client holds and sends with each request, removing server-side session storage entirely.

3. As a short-term band-aid, sticky sessions can pin a client to one server, but that just delays the real fix and adds its own problems.

**Interview Answer:**
"The session lives only in one server's memory, but the load balancer can route any request to any server, so users start getting logged out or losing cart contents randomly depending on which instance answers. The real fix is to externalize session state into something shared, like Redis, or to use a signed token that carries the session data with the request. Sticky sessions can mask the symptom by pinning a user to one server, but that's a workaround, not a fix, and it brings its own scaling problems."

---

## Q7. [Mid] Compare round robin and least connections load balancing. Which would you pick for a service where request processing times vary a lot?

**Answer:**

Both distribute traffic across a server pool, but they use different signals to decide where the next request goes.

**Options**

- Round robin
- Pros: dead simple, minimal state to track, predictable and easy to debug
- Cons: ignores current server load entirely, a server stuck on a few expensive requests still gets new ones at the same rate as an idle server
- Least connections
- Pros: sends new requests to whichever server currently has the fewest active connections, which naturally adapts to servers that are slow or overloaded
- Cons: slightly more overhead since the load balancer must track active connection counts per server, and it can behave oddly if connection count isn't a good proxy for actual load, for example some connections are idle keep-alives

**Best Choice:** **Least connections for a service with highly variable request processing times.**

**Why?** When requests vary a lot in cost, some take 10 milliseconds, others take 2 seconds, round robin will happily send new work to a server that's already buried under slow requests, just because it's "next in line." Least connections actively routes around servers that are currently busy, which matches the real goal of a load balancer: keeping response times even across the fleet. The extra bookkeeping cost is small compared to the benefit for this workload shape.

**Interview Answer:**
"Round robin just cycles through servers in order regardless of how busy each one actually is, which is fine when requests are uniform in cost. Least connections tracks how many active connections each server has and sends new requests to the least busy one, which adapts much better when request cost varies a lot. For a service where some requests are cheap and others are expensive, I'd pick least connections, because it avoids piling more work onto a server that's already stuck processing something slow."

---

## Q8. [Mid] What are sticky sessions, and what problems do they solve versus what problems do they create?

**Answer:**

**Sticky sessions, also called session affinity, is a load balancer feature that pins a specific client to a specific backend server for the duration of their session, usually by inspecting a cookie or client IP.**

**What they solve**

- They let a service that stores session state in server memory keep working correctly behind a load balancer, without an immediate rearchitecture.
- They can also help with in-memory caches, since a user's repeated requests keep hitting a server that already has their data warmed up locally.

**What they create**

- Uneven load: if some users are more active than others, the servers holding their sessions get disproportionately busy, defeating the point of load balancing.
- Fragile scaling: removing a server for a deploy or scale-down event drops every session pinned to it, logging those users out or losing their in-progress state.
- Harder auto-scaling: new servers start empty and only gradually get traffic as new sessions form, so they don't immediately help under a load spike.
- A general crutch: sticky sessions often mask an underlying stateful design that will keep causing friction as the system grows.

**Interview Answer:**
"Sticky sessions pin a client to the same backend server for their whole session, usually via a cookie, so a stateful server keeps working behind a load balancer. It's a quick way to keep session-in-memory designs functional without a rewrite. The cost is that load gets uneven if some sessions are heavier than others, scaling down or restarting a server drops every session pinned to it, and new servers added during a scale-up start empty and take time to get real traffic. I treat it as a stopgap, not a long-term scaling strategy, the real fix is externalizing session state."

---

## Q9. [Mid] Walk me through how you'd take a stateful monolithic application and make it stateless so it can scale horizontally.

**Answer:**

**The goal is to remove anything that lives only on one server's disk or memory and move it somewhere every instance can reach equally.**

**Step by step**

1. **Audit state**: find everything the app keeps locally, in-memory sessions, uploaded files on local disk, in-process caches, background job state, local counters or locks.
2. **Externalize sessions**: move session data to a shared store like Redis or Memcached, or switch to signed client-side tokens like JWTs that carry the needed claims.
3. **Externalize file storage**: move uploads and generated files to object storage like S3, so any server can read or write them without depending on local disk.
4. **Externalize caching**: replace in-process caches with a shared cache layer so all instances see consistent cached data, or accept per-instance caching only for data that's fine to be slightly inconsistent.
5. **Externalize locks and coordination**: replace in-process locks with a distributed lock (via Redis or a coordination service like ZooKeeper or etcd) if the app needs mutual exclusion across instances.
6. **Externalize background jobs**: move scheduled or queued work into a shared job queue rather than an in-process scheduler, so any instance can pick up and process jobs.

7. **Verify with a kill test**: intentionally kill an instance mid-traffic and confirm no user-visible data is lost, this validates statelessness in practice, not just in theory.

**Interview Answer:**
"I'd start by cataloguing everything the app currently keeps locally, sessions, uploaded files, in-memory caches, locks, scheduled jobs. Then I'd move each of those to a shared place: sessions and locks into Redis, files into object storage, caches into a shared cache layer, and background work into a shared queue. Once none of that state lives only on one instance's disk or memory, any server can handle any request. I'd validate it by killing an instance under live traffic and confirming nothing breaks for users, that's the real proof it's stateless."

---

## Q10. [Mid] Explain consistent hashing and why it's used in load balancing and distributed caching.

**Answer:**

**Consistent hashing is a technique for mapping requests (or keys) to servers in a way that minimizes remapping when servers are added or removed.**

**The problem it solves**

- With plain hashing, like `server = hash(key) % N`, changing N (adding or removing a server) reshuffles almost every key's assignment, because the modulo changes for nearly everything. That causes a massive cache miss storm or connection reshuffle.

**How it works**

1. Both servers and keys are hashed onto the same conceptual ring, a fixed circular range of hash values, for example 0 to 2^32 minus 1.
2. Each key is assigned to the first server found going clockwise from the key's position on the ring.
3. Adding a server only affects the keys between it and the previous server on the ring, everything else stays assigned exactly as before.
4. Removing a server only reassigns that server's keys to the next server clockwise, again leaving everything else untouched.

**Why it matters**

- Only a small fraction of keys move when the server pool changes, roughly 1/N of keys, instead of nearly all of them.
- It's used in load balancers that want to route the same client or key to the same backend consistently, and heavily in distributed caches and databases for the same reason.
- Virtual nodes, placing each physical server at multiple points on the ring, smooth out uneven load distribution that plain consistent hashing can otherwise produce.

**Interview Answer:**
"Consistent hashing places both servers and keys on a hash ring, and each key goes to the next server clockwise from its position. The benefit over plain modulo hashing is that adding or removing a server only reshuffles a small slice of keys near that server on the ring, instead of remapping almost everything. That's why it's used for routing decisions in load balancers and heavily in distributed caches and databases, it keeps cache hit rates and routing stable as the server pool changes. Virtual nodes are usually added on top to even out load across servers."

## Q11. [Mid] Walk me through how you'd design auto-scaling for a web service that sees a predictable traffic spike every day at lunchtime.

**Answer:**

**Since the spike is predictable, the design should combine reactive auto-scaling with proactive scheduling rather than relying on reactive scaling alone.**

**Design**

1. **Baseline reactive auto-scaling**: set up a scaling policy on a metric like CPU utilization, request latency, or requests per instance, with a target range, for example scale out when average CPU crosses 60 percent.
2. **Scheduled pre-scaling**: add a scheduled action that increases the minimum instance count shortly before the known lunchtime spike, so capacity is already in place before demand hits, avoiding the lag of reactive scaling.
3. **Scale-in cooldown**: after the spike passes, scale back down gradually with a cooldown period, to avoid flapping if traffic dips and spikes again within a short window.
4. **Health checks integrated with the load balancer**: make sure new instances are only added to rotation once they pass readiness checks, so scaling doesn't send traffic to instances still warming up.
5. **Capacity ceiling and alerting**: set a maximum instance count as a safety net, and alert if the service hits that ceiling, since that likely means the spike is bigger than expected.

**Why combine both**

- Purely reactive scaling has lag: by the time CPU crosses the threshold, users have already felt slower response times, and new instances take time to boot and warm up.
- Purely scheduled scaling misses unexpected spikes on days that don't follow the usual pattern.

**Interview Answer:**
"Since the lunchtime spike is predictable, I wouldn't rely purely on reactive auto-scaling, because there's real lag between crossing a CPU threshold and having a warmed-up instance ready to take traffic. I'd add a scheduled scaling action that increases the minimum instance count a bit before lunch, so capacity is already there, and keep reactive scaling on top as a safety net for anything unexpected. I'd also make sure new instances pass health checks before receiving traffic, and set a cooldown period on scale-in to avoid flapping right after the spike passes."

---

## Q12. [Mid] Response times on your service are degrading under load. Walk me through how you'd identify the bottleneck.

**Answer:**

**The goal is to systematically narrow down which layer is actually saturated, rather than guessing.**

**Step by step**

1. **Check the load balancer and app tier first**: look at CPU, memory, and request queue depth on application servers. If app servers are maxed on CPU while the database is idle, the bottleneck is application compute.
2. **Check the database**: look at query latency, connection pool exhaustion, lock contention, and slow query logs. A common pattern is app servers waiting on database connections, not actually doing CPU work themselves.

3. **Check downstream dependencies**: third-party APIs, internal microservices, or a cache layer can each be the actual slow point, trace a single request end to end to see where time is really spent.
4. **Check the network and infrastructure layer**: DNS resolution, TLS handshake overhead, or a saturated network link can add latency that looks like an application problem but isn't.
5. **Use distributed tracing** if available, to get a per-request breakdown of time spent in each service or call, this is far faster than guessing from aggregate metrics alone.
6. **Reproduce under controlled load** with a load test tool if the issue is intermittent, so you can watch metrics in real time as load increases.

**Interview Answer:**
"I'd start by checking whether the application servers themselves are CPU or memory bound, versus just sitting there waiting on something else, like database connections or a downstream API. Then I'd look at the database specifically, connection pool saturation, slow queries, and lock contention are common culprits. If tracing is available, I'd use it to see exactly where time is spent for a single slow request, since that's much faster than guessing from dashboards. The key is narrowing down layer by layer instead of assuming it's the database or the app tier without evidence."

---

## Q13. [Mid] How do you scale reads differently from how you scale writes in a database-backed system?

**Answer:**

**Reads and writes have fundamentally different scaling levers because writes have to go somewhere authoritative, while reads can be served from many places.**

**Scaling reads**

- Add read replicas: copies of the primary database that serve read-only queries, letting you horizontally scale read capacity by adding more replicas.
- Route read traffic to replicas and write traffic to the primary, usually via the application or a proxy layer that understands the split.
- Accept eventual consistency: replicas lag slightly behind the primary, so a read right after a write might not reflect it yet, which needs to be acceptable for the use case or handled explicitly.

**Scaling writes**

- Writes must go to a single source of truth for a given piece of data, so replicas don't help write throughput directly.
- Vertical scaling of the primary buys some headroom, but has the same ceiling problems as any single machine.
- Beyond that, the real levers are partitioning or sharding the data across multiple primaries (each owning a subset of the data), or reducing write volume itself through batching, write buffering, or asynchronous processing via a queue.

**Why this asymmetry matters**

- Most systems are read heavy, so read replicas alone solve a huge portion of real-world scaling needs.
- Write scaling is fundamentally harder because it eventually requires splitting the data itself, which is a bigger architectural step, this is where sharding conversations start, though that's a deeper topic on its own.

**Interview Answer:**

"Reads scale relatively easily with read replicas, you add more copies of the data and route read traffic across them, accepting some replication lag. Writes are harder to scale because they need a single authoritative place to land for a given piece of data, so replicas don't help write throughput. To scale writes further you either scale the primary vertically, which has a ceiling, or partition the data across multiple primaries, which is a much bigger architectural change. Most systems are read heavy, so read replicas alone handle a lot of the real-world scaling need."

---

## Q14. [Mid] What metrics would you use to trigger auto-scaling, and what goes wrong if you scale on CPU utilization alone?

**Answer:**

**Auto-scaling needs a signal that accurately reflects when a service is actually struggling to keep up with demand, and CPU alone often doesn't capture that.**

**Useful metrics beyond CPU**

- Request latency (p50, p95, p99): directly reflects user experience, and can degrade even when CPU looks fine.
- Request queue depth or in-flight request count: shows whether requests are backing up waiting to be processed.
- Memory utilization: some workloads are memory bound, not CPU bound, and will fail before CPU ever looks high.
- Custom application metrics: for example, queue depth in a background job worker, or active connections for a connection-limited service.

**What goes wrong with CPU alone**

- I/O bound services (waiting on database calls, network calls) can have high latency and backed-up requests while CPU sits low, since the CPU is idle, waiting, not working. Scaling never triggers even though the service is struggling.
- CPU can spike briefly for unrelated reasons, like a garbage collection pause or a background job, causing unnecessary scale-out.
- CPU-based scaling reacts only after load has already caused a problem, and new instances take time to boot, so users feel the pain before scaling catches up.

**Best Choice:** Use a combination of latency and queue depth as primary signals, with CPU and memory as supporting signals.

**Why?** Latency and queue depth are much closer to what actually matters, whether the service is keeping up with demand, while CPU is only a proxy that fails for I/O bound workloads.

**Interview Answer:**

"CPU alone misses a lot, especially for I/O bound services that spend most of their time waiting on a database or network call rather than computing, so CPU can look fine while requests are actually piling up and latency is degrading. I'd scale primarily on request latency and queue depth or in-flight request count, since those reflect what users actually experience, and use CPU and memory as supporting signals rather than the sole trigger. That combination catches both compute-bound and I/O-bound bottlenecks."

## Q15. [Mid] Compare scaling the application server layer versus scaling the database layer. Which is usually harder, and why?

**Answer:**

Both layers can be scaled, but they differ enormously in how easy that scaling is to achieve safely.

**Options**

- Application server layer
- Pros: typically stateless by design (or made stateless), so scaling out is close to just adding more identical instances behind a load balancer
- Cons: still requires care around shared state, like sessions or local caches, and downstream dependencies must be able to handle the increased fan-out of requests
- Database layer
- Pros: read replicas offer relatively easy read scaling
- Cons: writes are much harder to scale because data has consistency and ordering requirements, a single row generally needs one authoritative owner. Scaling writes usually means sharding or partitioning, which touches schema design, query patterns, cross-shard transactions, and operational complexity significantly

**Best Choice:** **Application servers are generally far easier to scale than databases.**

**Why?** App servers hold no durable state of their own once made stateless, so adding more of them is close to a mechanical operation. Databases hold the actual state of the system, so scaling them means either accepting replication lag on reads or fundamentally restructuring how data is partitioned for writes, which is a much bigger architectural undertaking with real consistency trade-offs.

**Interview Answer:**
"Application servers are usually much easier to scale, especially once they're stateless, because adding more of them behind a load balancer is close to a mechanical operation with no data consistency concerns. Databases are harder because they hold the actual state, reads scale reasonably well with replicas, but scaling writes usually means partitioning or sharding the data, which touches schema design, cross-shard queries, and consistency guarantees. So in most systems, the database becomes the bottleneck and the harder scaling problem long before the app tier does."

## Q16. [Mid] What is the thundering herd problem in the context of auto-scaling, and how do you prevent it?

**Answer:**

**Thundering herd, in an auto-scaling context, happens when a sudden event causes a large number of instances or requests to act simultaneously, overwhelming a shared resource right when it's under the most stress.** For example, if a popular product page's cache entry expires during a traffic spike, dozens of servers can all miss the cache at the same instant and hammer the database with the same expensive query simultaneously.

**Common scenarios**

- A cache expires or a service restarts, and every instance simultaneously hits the database at once to refill or reconnect, spiking database load far beyond normal.

- A batch of new instances all boot at once during a scale-out event and all try to fetch the same configuration, register with the same service discovery system, or open connection pools to the database at the same moment.
- Health checks across a whole fleet fire at the exact same interval, creating a periodic load spike.

**How to prevent it**

1. **Stagger startup**: add jitter, a small random delay, to instance startup tasks like config fetches or connection pool warmup, so they don't all fire at once.
2. **Gradual traffic ramp**: bring new instances into rotation gradually rather than all at once, sometimes called a warm-up period.
3. **Backoff with jitter on retries**: when a shared resource is temporarily overwhelmed, retries should use exponential backoff plus randomness, not fire again immediately in lockstep.
4. **Connection pool limits**: cap how many connections each instance opens to the database, so a fleet-wide scale-out can't multiply into an unbounded connection spike.

**Interview Answer:**

"Thundering herd is when a bunch of instances or requests all act at the same instant, like a fleet of new instances all opening database connections the moment they boot during a scale-out, which can overwhelm the very resource you're trying to protect. I'd prevent it with jitter, adding small random delays to startup tasks like config fetches or connection warmup so instances don't all fire in lockstep, ramping new instances into traffic gradually rather than all at once, and using exponential backoff with jitter on retries against shared resources."

---

## Q17. [Mid] You have 10 stateless application servers behind a load balancer. What shared state or coordination problems can still bite you?

**Answer:**

**Being stateless at the request-handling level doesn't automatically eliminate every form of shared state, several categories still need explicit handling.**

**Things that still need coordination**

- **Rate limiting**: if each instance tracks request counts locally, a client could get 10x the intended rate limit by spreading requests across all 10 instances. This needs a shared counter, typically in Redis.
- **Distributed locks**: any operation that must not run concurrently, like a scheduled job that should only execute once, needs a shared lock, not an in-process one, or it'll run 10 times.
- **Caching consistency**: if each instance keeps a local in-memory cache, the 10 instances can disagree on the same piece of data at the same time, since invalidation on one instance doesn't touch the others.
- **Configuration and feature flags**: if config is loaded once at boot and cached locally, updating it means every instance needs a mechanism to pick up the change, or you get inconsistent behavior across the fleet during a rollout.
- **Idempotency**: with retries and multiple instances, the same logical operation, like a payment charge, can be attempted more than once from different instances, requiring an idempotency key checked against shared storage.

**Interview Answer:**

"Statelessness at the request level doesn't mean there's no shared state to worry about. Rate limiting needs a shared counter, otherwise a client can multiply their limit by spreading requests across instances. Anything

that should run exactly once, like a scheduled job, needs a distributed lock, not a local one. Local in-memory caches can drift out of sync across instances. And operations like payments need idempotency keys checked against shared storage, since retries can hit different instances and otherwise double-execute. All of these push you toward a shared coordination layer like Redis or a database."

---

## Q18. [Senior] You're taking a monolith running on a single large box and need to move it to a horizontally scaled architecture. Walk me through your approach.

**Answer:**

**This is a migration problem as much as a design problem, the goal is to get to horizontal scaling without a risky big-bang rewrite.**

**Approach**

1. **Establish a baseline**: understand current traffic patterns, peak load, growth trajectory, and where the actual pain is today, wasted effort scaling a part of the system that isn't the bottleneck.
2. **Identify and remove local state**: audit for in-memory sessions, local file storage, in-process caches, and in-process schedulers, these all block horizontal scaling and must move to shared stores first.
3. **Put a load balancer in front early**: even before fully scaling out, running two instances behind a load balancer forces you to surface hidden state problems in a controlled way, rather than discovering them under a real traffic spike.
4. **Scale the database read path first**: add read replicas and route read-heavy queries to them, this is usually the highest leverage, lowest risk first step, since most systems are read heavy.
5. **Introduce auto-scaling on the now-stateless app tier**: with local state removed, this becomes largely a matter of configuring scaling policies and health checks.
6. **Address the write path last**: if writes are still a bottleneck after read scaling and app tier scaling, that's when partitioning, sharding, or splitting the monolith's data ownership becomes necessary, a much bigger step reserved for when it's actually needed.
7. **Validate incrementally**: after each step, load test and monitor before moving to the next, rather than changing everything simultaneously.

**Interview Answer:**
"I'd start by figuring out where the actual pain is today, rather than assuming. Then I'd remove local state, sessions, file storage, in-process caches, moving each to a shared store, since that's the real blocker to horizontal scaling. I'd put a load balancer in front of even two instances early, to surface hidden state issues in a controlled way. From there, read replicas usually give the biggest, lowest-risk win since most systems are read heavy, followed by auto-scaling the now-stateless app tier. I'd only reach for partitioning the write path if it's still the bottleneck after those steps, and validate incrementally rather than changing everything at once."

---

## Q19. [Senior] Go deep on consistent hashing: how does it handle a server being added or removed, and what are virtual nodes for?

**Answer:**

**Consistent hashing's main value is bounding how much remapping happens when the server pool changes, but the naive version has a real weakness that virtual nodes fix.**

**Handling server changes**

- **Adding a server**: the new server is placed at a position on the hash ring. It takes ownership of the range of keys between itself and the previous server going counter-clockwise, meaning only those keys need to move, everything else stays exactly where it was.
- **Removing a server**: its keys are reassigned to the next server clockwise on the ring. Again, only that server's keys move, the rest of the ring is undisturbed.
- Compare this to `hash(key) % N`, where changing N reshuffles nearly every key because the modulo operation changes for almost all values, consistent hashing avoids that entirely.

**The problem with naive consistent hashing**

- With a small number of physical servers placed at single points on the ring, the distribution of keys can be very uneven, one server might end up owning a much larger arc of the ring than another, purely by chance of where their hash values land.
- Adding or removing a server can also create a disproportionately large or small range for its neighbor.

**Virtual nodes as the fix**

- Instead of placing each physical server at one point on the ring, place it at many points, for example 100 to 200 virtual nodes per physical server, each an independent hash position mapping back to the same physical server.
- This averages out load distribution across the ring, since each physical server now owns many small, scattered ranges instead of one large contiguous range.
- It also smooths the impact of adding or removing a server, since that change is now spread across many small ranges instead of one potentially large one.

**Interview Answer:**

"Consistent hashing places servers and keys on a ring, and each key belongs to the next server clockwise. Adding a server only takes over the range between it and the previous server, removing one only hands its range to the next server, so only a small fraction of keys ever move, unlike modulo hashing where nearly everything reshuffles. The catch is that with few physical points on the ring, load distribution can be quite uneven. Virtual nodes fix that by placing each physical server at many points on the ring, which spreads its ownership across many small ranges, evening out the load and smoothing the impact of any single server joining or leaving."

---

## Q20. [Senior] Your database is clearly the bottleneck, not the application tier. Before you reach for sharding, what options would you explore?

**Answer:**

**Sharding is powerful but expensive to implement and operate, so it's worth exhausting less invasive options first.**

**Options to explore first**

1. **Query and index optimization**: slow, unindexed, or poorly written queries are a very common root cause. Profiling and fixing the worst offenders can buy significant headroom cheaply.
2. **Connection pooling**: too many application instances opening too many raw connections can exhaust the database's connection limit, a pooler like PgBouncer can multiplex connections efficiently.
3. **Read replicas**: if the workload is read heavy, offloading reads to replicas is far simpler than sharding and solves a large class of database bottleneck problems.

4. **Vertical scaling of the primary**: sometimes the simplest fix is a bigger database instance, more memory in particular often helps a lot since it increases what fits in the buffer cache.
5. **Caching hot data**: putting frequently read, infrequently changing data in front of the database in a cache layer reduces database load directly, though this chapter treats caching only briefly since it's covered elsewhere.
6. **Denormalization or precomputed aggregates**: if expensive joins or aggregations are the pain point, restructuring data to avoid computing them on every request can help significantly without touching the topology of the database.
7. **Archiving or partitioning old data within the same database**: moving cold data out of hot tables, or using native table partitioning, can shrink working set size without a full sharding rearchitecture.

**Why explore these first**

- Sharding introduces cross-shard transaction complexity, more operational overhead, and rebalancing challenges that are hard to undo, it's a one-way architectural door you want to walk through deliberately, not reactively.

**Interview Answer:**
"Before sharding, I'd check the cheaper levers first: are queries actually optimized and indexed properly, is connection pooling in place, is the workload read heavy enough that replicas alone would solve it, and would a bigger primary instance or more memory buy meaningful headroom. Caching hot data and denormalizing expensive joins can also remove a lot of load without touching the database's topology at all. Sharding is a one-way door, it adds real complexity around cross-shard transactions and rebalancing, so I'd only reach for it once those other options are genuinely exhausted."

---

## Q21. [Senior] How do you keep rate limiting and session state consistent across many horizontally scaled instances?

**Answer:**

**The core issue is that any state tracked per-instance, in local memory, becomes inconsistent the moment there's more than one instance, since a client's requests can land anywhere.**

**Rate limiting**

- Use a shared counter in a fast store like Redis, keyed by client identifier, for example `rate_limit:{user_id}:{window}`.
- Use an atomic increment operation with an expiry set on the key matching the rate limit window, so the counter resets automatically.
- For high throughput, consider a sliding window log or token bucket algorithm implemented with Redis Lua scripts to keep the check-and-increment atomic, avoiding race conditions where two near-simultaneous requests both read a stale count.
- At very high scale, some systems accept slightly relaxed accuracy, like a local approximate counter that periodically syncs to the shared store, trading strict accuracy for reduced load on the shared store.

**Session state**

- Store sessions in a shared, fast store like Redis or Memcached, keyed by a session ID that the client holds in a cookie or token.

- Alternatively, remove server-side session storage entirely by using signed, stateless tokens like JWTs, at the cost of harder immediate revocation, since the server no longer holds a canonical, instantly updatable copy.
- If session data must be revocable instantly, for example on logout, combine a short-lived JWT with a shared denylist checked on each request, or a versioned session pointer stored centrally.

**Interview Answer:**

"Both problems come from the same root cause, per-instance local state doesn't work once requests can land on any of many instances. For rate limiting, I'd use an atomic counter in Redis keyed per client with an expiry matching the rate window, using Lua scripting or a proper sliding window algorithm to avoid race conditions between simultaneous requests. For sessions, I'd store them in a shared store like Redis, or move to signed stateless tokens if I want to avoid a shared store entirely, accepting that instant revocation then needs an extra mechanism like a denylist, since the token itself can't be invalidated on the fly."

---

## Q22. [Senior] What breaks when you try to horizontally scale a WebSocket-based real-time service, and how do you handle it?

**Answer:**

**WebSockets are long-lived, stateful connections, which conflicts directly with the usual stateless-and-load-balance model used for regular HTTP services.**

**What breaks**

- **Connection affinity**: once a client establishes a WebSocket connection to a specific server, that connection stays open to that exact server. A standard load balancer can't just move it to another instance mid-connection the way it can route the next HTTP request elsewhere.
- **Cross-instance messaging**: if client A is connected to server 1 and client B is connected to server 2, and A sends a message meant for B, server 1 has no direct way to reach server 2's client, they don't share memory.
- **Scale-down disruption**: removing an instance during scale-in drops every WebSocket connection currently held by that instance, forcing every one of those clients to reconnect.
- **Load balancer support**: not every load balancer configuration supports WebSocket upgrade and long-lived connections cleanly, this needs to be verified, not assumed.

**How to handle it**

1. Use a load balancer that explicitly supports WebSocket connections and sticky routing for the life of the connection, layer 7 load balancers with WebSocket support handle the initial upgrade handshake correctly.
2. Introduce a shared message bus, like Redis pub/sub (publish/subscribe: one service publishes a message and every other service that's subscribed receives a copy) or a dedicated message broker, so any server can publish a message and any other server holding the relevant recipient's connection can deliver it. This decouples "who is connected to which server" from "who needs to receive this message."
3. On scale-down, drain connections gracefully, stop accepting new connections on the instance being removed, and give existing clients a chance to reconnect elsewhere before forcibly closing.
4. Track connection-to-server mapping in a shared registry, if the app needs to know where a specific user's live connection currently lives, for targeted delivery.

**Interview Answer:**

"WebSockets are long-lived and stateful, so the connection sticks to whichever server accepted it, that server can't be swapped out mid-connection the way HTTP requests get freely routed. The real problem is cross-instance messaging: if two clients are connected to different servers, one server has no way to reach the other's client directly. I'd solve that with a shared pub/sub layer, like Redis, so any server can publish a message and whichever server holds the target connection delivers it. I'd also make sure the load balancer explicitly supports WebSocket upgrades, and drain connections gracefully on scale-down rather than dropping them abruptly."

---

## Q23. [Senior] Explain the difference between scaling for throughput and scaling for latency. Can you improve one without helping the other?

**Answer:**

**Throughput and latency are related but distinct goals, and optimizing purely for one can leave the other unchanged or even make it worse.** For example, adding ten more worker instances behind a queue increases how many total orders you can process per second, throughput, without making any single order process any faster, latency, and can even make one order slower if it now waits longer in a bigger queue before a worker picks it up.

**Throughput**

- Throughput is how much total work the system can process per unit time, requests per second, messages per second, and so on.
- Scaling for throughput typically means adding more parallel capacity, more instances, more partitions, more workers, so more work happens concurrently.

**Latency**

- Latency is how long a single request takes from start to finish.
- Scaling for latency often means reducing the work or waiting inside a single request's path: faster queries, fewer network hops, caching, more efficient serialization, or reducing queueing delay.

**Can you improve one without the other?**

- Yes. Adding 10 more instances behind a load balancer can raise total throughput significantly, while doing nothing for the latency of any individual request, each request still takes the same time to process, there are just more of them happening in parallel.
- Conversely, optimizing a slow database query makes each request faster, which helps latency, but if the system was never throughput constrained, this may not raise the number of requests per second the system can sustain overall.
- They do interact though: if latency is caused by queueing (requests waiting for a free worker), adding throughput capacity can also reduce latency, because requests spend less time waiting in line. But if latency is caused by the inherent cost of the work itself, more capacity alone doesn't fix it.

**Interview Answer:**

"Throughput is how much work the system gets through per second, latency is how long any single request takes. You can scale one without the other, adding more instances raises total throughput but doesn't make an individual request any faster, it just lets more of them run in parallel. Optimizing a slow query reduces latency for each request but doesn't necessarily raise overall throughput if the system wasn't capacity constrained to begin with. They do connect when latency is caused by queueing, since more capacity reduces wait time in line, but if latency comes from the inherent cost of the work, adding servers alone won't fix it."

## Q24. [Senior] Your auto-scaling group keeps flapping, scaling up and then immediately back down repeatedly. Diagnose the likely causes and how you'd fix it.

**Answer:**

**Flapping usually means the scaling policy is reacting to a signal that oscillates faster than the system can stably respond to, or the thresholds are too tight around the current load.**

**Likely causes**

- **No cooldown period**: the scaling policy evaluates and acts again immediately after a scaling event, before the effect of the last scale-out or scale-in has even been reflected in the metrics.
- **Threshold too close to normal load**: if the scale-out threshold is, for example, 60 percent CPU and normal load hovers right around 58 to 62 percent, small natural fluctuations will repeatedly cross the line in both directions.
- **Slow instance warmup counted against the metric**: a newly added instance might start with artificially high CPU or error rates while it's warming up caches or JIT-compiling code, dragging the aggregate metric in a way that triggers an immediate scale-in decision or a confusing signal.
- **Metric aggregation lag or noise**: a metric that's noisy or laggy relative to real load, like a coarse one-minute average during rapidly changing traffic, can cause the scaling decision to always be reacting to stale information.

**Fixes**

1. Add proper cooldown periods after both scale-out and scale-in events, so the system waits long enough to observe the actual effect before acting again.
2. Widen the gap between scale-out and scale-in thresholds, for example scale out above 70 percent but only scale in below 40 percent, so normal fluctuation doesn't cross both lines.
3. Exclude or delay new instances from the aggregate metric until they've passed a warmup period and are handling steady traffic.
4. Use a smoothed or longer-window metric for scaling decisions, rather than a raw, noisy, short-interval reading.
5. Consider step scaling instead of simple threshold scaling, where the magnitude of the scaling action matches how far past the threshold the metric is, reducing overcorrection.

**Interview Answer:**

"Flapping usually comes from the scaling thresholds being too tight around normal load, or from missing cooldown periods, so the system reacts to noise instead of a real sustained change. I'd widen the gap between the scale-out and scale-in thresholds so normal fluctuation doesn't cross both, add proper cooldown windows so it waits to see the actual effect of the last action before deciding again, and exclude new instances from the aggregate metric until they're past their warmup period, since a cold instance can skew the signal. I'd also look at using a smoothed, longer window metric rather than a noisy short-interval one."

## Q25. [Senior] Compare how you'd scale a write-heavy system, like a metrics ingestion pipeline, versus a read-heavy system, like a content site. What's fundamentally different about the approach?

**Answer:**

**The two workloads stress completely different parts of the stack, so the scaling strategy needs to target the actual pressure point in each case.**

**Options**

- **Write-heavy system (metrics ingestion)**
- Pros of the typical approach: partition incoming writes across multiple ingestion nodes and storage partitions by a key like metric name or source ID, use a message queue in front of storage to absorb bursts and smooth out write pressure, batch writes together rather than committing one at a time to reduce per-write overhead
- Cons: partitioning writes means queries that need to aggregate across partitions become more expensive, and the system needs to handle backpressure gracefully when ingestion temporarily outpaces storage capacity
- **Read-heavy system (content site)**
- Pros of the typical approach: add read replicas, cache aggressively at multiple layers (CDN for static and semi-static content, application cache for computed results), scale the stateless application tier horizontally since most of the work is serving, not writing
- Cons: cached and replicated data can be stale, and cache invalidation logic adds its own complexity, though this chapter treats caching only briefly

**Best Choice:** There isn't a single best choice, the right approach is dictated by which side, reads or writes, is actually under pressure, and the two require genuinely different toolkits.

**Why?** Write scaling is fundamentally about spreading writes across multiple owners of data and absorbing bursts, usually via partitioning and queuing, because a single write path has a hard ceiling. Read scaling is about avoiding redundant work altogether, through replication and caching, because the same data is being requested repeatedly. Applying read-scaling techniques to a write-heavy problem, like just adding read replicas, does nothing, since replicas don't add write capacity.

**Interview Answer:**
"A write-heavy pipeline like metrics ingestion needs writes spread across partitions and usually a queue in front of storage to absorb bursts and smooth out load, since a single write path has a hard ceiling. A read-heavy content site instead leans on read replicas and aggressive caching at multiple layers, since the same data gets requested repeatedly and the win comes from avoiding redundant work. The key realization is that read-scaling tools, like adding replicas, don't help write throughput at all, and write-scaling tools, like partitioning, are overkill if the actual bottleneck is repeated reads of the same data. The approach has to match which side is actually under pressure."

---

## Q26. [Senior] How does session affinity interact with consistent hashing when the set of servers behind the load balancer changes frequently, say due to auto-scaling?

**Answer:**

**Consistent hashing is often used specifically to make session affinity survive a changing server pool better than naive sticky sessions would, but it doesn't eliminate the tension entirely.**

**The interaction**

- With naive sticky sessions, keyed off which server a client first connected to, any change to the server pool, especially removal during scale-in, breaks affinity for every client pinned to the removed server, forcing them all to fail over and lose local state at once.

- Consistent hashing improves this: a client's key hashes to a position on the ring, and only clients whose position falls in the specific range affected by an added or removed server are remapped. Everyone else keeps their existing affinity.
- However, auto-scaling under a metrics-driven policy can still add or remove servers frequently during a sustained spike, and each change remaps some non-zero slice of clients, so affinity isn't perfectly stable even with consistent hashing, it's just far less disruptive than the naive approach.

**Practical implications**

1. Whatever state was local to a remapped client's new server needs to be fetched or rebuilt, for example a warm cache miss, so consistent hashing reduces the blast radius of scaling events but doesn't remove the need for a fallback path when affinity does shift.
2. Virtual nodes help smooth which specific clients get remapped on any given scaling event, avoiding one physical server absorbing an outsized share of remapped clients.
3. The most robust long-term answer is still to reduce reliance on affinity in the first place, externalizing whatever session state motivated the affinity requirement, so a remap causes at worst a cache miss, not a functional failure.

**Interview Answer:**
"Consistent hashing helps because when a server is added or removed during an auto-scaling event, only the clients whose hash falls in the specific affected range on the ring get remapped, not everyone, unlike naive sticky sessions where removing a server drops every client pinned to it at once. But it doesn't make affinity perfectly stable, frequent auto-scaling still remaps some slice of clients on every change. Virtual nodes help spread that impact more evenly. The more robust fix long term is to reduce how much actually depends on affinity, by externalizing session state, so a remap costs a cache miss at worst rather than a functional break."

---

## Q27. [Senior] You need to scale this system to handle 10 times current traffic within 3 months, without a full rewrite. Walk me through how you'd approach it.

**Answer:**

**A 10x target in a short window means prioritizing the highest-leverage, lowest-risk changes first, and being explicit about what won't get fixed in this pass.**

**Approach**

1. **Load test to find the real breaking point today**: don't assume where the system falls over, measure it, since intuition about bottlenecks is often wrong.
2. **Triage by leverage and risk**: rank potential fixes by how much headroom they buy versus how risky and time consuming they are to implement, favor high-leverage, low-risk items first.
3. **Remove blocking state from the app tier**: if any local state remains, sessions, local caches, this is usually a fast, high-leverage fix that unlocks straightforward horizontal scaling of the app layer.
4. **Scale the app tier horizontally with auto-scaling**: once stateless, this is close to a configuration change and buys significant headroom quickly for compute-bound load.
5. **Add read replicas and caching for the database**: usually the highest-leverage database fix achievable without a schema rewrite, directly addresses the most common bottleneck class.
6. **Fix the worst-offending slow queries and add missing indexes**: targeted, low-risk, high-impact, don't need a redesign.

7. **Identify anything genuinely un-scalable in 3 months and mitigate rather than solve**: for example, if a single write-heavy table is a real ceiling and full sharding isn't feasible in the timeframe, consider interim measures like moving to a bigger primary, offloading part of the write volume asynchronously through a queue, or archiving old data to shrink the working set.
8. **Load test again after each major change**: validate incrementally rather than betting everything on a big final test right before the deadline.
9. **Set up monitoring and alerting on the new capacity ceiling**: know in advance what the next bottleneck will be so it's not a surprise past 10x.

**Interview Answer:**

"First I'd load test to find where the system actually breaks today, rather than guessing. Then I'd prioritize by leverage versus risk: removing any remaining local state so the app tier can scale horizontally with auto-scaling is usually fast and high impact, and adding database read replicas plus fixing the worst slow queries usually addresses the biggest bottleneck class without a schema rewrite. If there's a genuinely hard limit, like a single write-heavy table, I'd look for a mitigation that fits the timeframe, a bigger primary, offloading writes through a queue, or archiving old data, rather than attempting full sharding under time pressure. I'd load test again after each change and make sure I know what the next bottleneck will be before we hit it."

---

**If you remember nothing else from this chapter:** *horizontal scaling is a reward for good architecture, not a fix that works regardless of how the system was built.*

# Chapter 6: Database Basics

Database choice is one of the most consequential decisions you'll make in a design interview, and one of the easiest to rush. These questions push you to match a database to actual requirements, consistency needs, access patterns, and data shape, rather than defaulting to whatever you used most recently regardless of fit.

**War Story:** *A "temporary" missing index on a foreign key sat unnoticed for a year, until a routine cleanup job's DELETE statement locked the entire orders table for eleven minutes during peak checkout.*

**Avoid:** reaching for "add an index" as a reflex to any slow-query question without asking to see the query plan first

**Aim For:** asking for EXPLAIN output before diagnosing a slow query • distinguishing a schema problem from a query problem

## Q1. [Junior] What is a relational database, and how is it fundamentally different from a NoSQL database?

**Answer:**

A relational database organizes data into tables made up of rows and columns, where each table has a fixed schema defining the columns and their types. **Tables relate to each other through keys, and you query the data using SQL.** For example, an e-commerce orders table in a relational database has a fixed set of columns like order_id, customer_id, and total, and you could join it with a customers table in a single SQL query, while a NoSQL document store might let each order document include its own nested list of items with no two orders required to have the same shape.

**Core traits of relational databases**

- Fixed schema: every row in a table has the same columns.
- Relationships are explicit, via foreign keys linking one table to another.
- Strong support for joins, so you can combine data across tables in a single query.
- Strong consistency guarantees through ACID transactions.

**Core traits of NoSQL databases**

- Flexible or schema-less structure: documents or rows in the same collection can have different fields.
- Optimized for specific access patterns, like key lookups or nested document retrieval, often at the cost of flexible querying.
- Many NoSQL systems favor horizontal scalability and availability over strict consistency.
- No single definition, NoSQL covers document, key-value, wide-column, and graph databases, each with different trade-offs.

The real difference isn't "structured versus unstructured," it's that relational databases assume you'll query your data in many different, possibly unanticipated ways and enforce integrity centrally, while NoSQL databases usually assume you know your access pattern up front and optimize heavily for it.

**Interview Answer:**

"A relational database stores data in tables with a fixed schema and enforces relationships between tables through foreign keys, and you query it with SQL, including joins across tables. NoSQL is really an umbrella term for databases, document, key-value, wide-column, or graph, that trade some of that structure and consistency for flexibility or horizontal scalability. The real distinction is that relational databases are built for flexible, ad hoc querying with strong integrity guarantees, while NoSQL databases are usually built around one specific access pattern."

---

## Q2. [Junior] If you had to know just four databases cold for a system design interview, which four would you pick, and what's the one-line pitch for each?

**Answer:**

**Almost every real system design interview eventually narrows down to a choice between these four, and knowing what each one is genuinely bad at matters more than knowing what it's good at.**

**PostgreSQL, the default answer**

- One-line pitch: a rock-solid relational database with strong consistency, real transactions, and the richest query language of the four.
- Reach for it whenever correctness and relationships between data matter more than raw write throughput across continents.
- What it's bad at: horizontally scaling writes across many machines without real operational effort, and rigid schemas fight you when your data shape changes every week.

**MongoDB, the flexible one**

- One-line pitch: a document database where each record can have its own shape, which is perfect when your data doesn't want to sit still.
- Reach for it when different records in the same collection naturally have different fields, like products across wildly different categories.
- What it's bad at: multi-document transactions used to be a real weakness (modern versions support them, but they're heavier than a relational transaction), and joins across collections are awkward compared to SQL.

**Cassandra, the always-write one**

- One-line pitch: a wide-column store (each row can have a different, flexible set of columns, optimized for very fast writes) built to keep accepting writes across multiple regions even when part of the network is partitioned.
- Reach for it when you have massive write volume, need multi-region replication, and can tolerate eventual consistency.
- What it's bad at: ad hoc queries. You design your tables around the exact queries you'll run, and a new access pattern you didn't plan for often means a new table.

**DynamoDB, the managed one**

- One-line pitch: a fully managed, serverless key-value and document store that scales without you touching a single server.
- Reach for it when you want AWS to handle all the operational scaling work and your access patterns are known ahead of time.
- What it's bad at: flexible querying. Like Cassandra, you pay for that hands-off scaling with needing to know your query patterns up front.

**Interview Answer:**
"The four I'd want cold are PostgreSQL, MongoDB, Cassandra, and DynamoDB. Postgres is my default for anything needing real transactions and relationships. MongoDB is what I reach for when the data shape genuinely varies record to record. Cassandra is for massive, multi-region write volume where you can't ever reject a write. DynamoDB is the fully managed version of that same idea, when I'd rather AWS handle scaling than run the cluster myself. The interesting part of this question is always what each one is bad at, not just what it's good at."

---

## Q3. [Mid] You're building the order and payment system for a food delivery app. A lost or double-charged payment is a real financial and legal problem. Which of the four databases would you reach for, and why would the other three be a mistake here?

**Answer:**

**Money needs real transactions and strong consistency more than it needs massive scale, which points straight at PostgreSQL and away from the other three.**

**Why PostgreSQL fits**

- Full ACID transactions mean debiting a customer and crediting the restaurant either both happen or neither does, with no in-between state a retry or a crash could expose.
- Foreign keys and constraints let the database itself refuse to insert a payment record for an order that doesn't exist, catching bugs before they become financial discrepancies.
- Mature tooling for financial auditing, migrations, and reporting queries that join across orders, payments, and refunds.

**Why the other three would be a mistake**

- MongoDB's flexible schema doesn't buy you anything here, since a payment record has a fixed, well-understood shape, and you'd be giving up strong multi-document transaction guarantees for no real benefit.
- Cassandra's whole value proposition is high write throughput with eventual consistency, exactly backwards from what a payment ledger needs: you'd rather have a rejected write than an eventually-consistent balance.
- DynamoDB can be made to work with careful design, but you're fighting its access-pattern-first model for something that needs flexible, ad hoc financial queries and joins far more than it needs massive horizontal write scale.

**Interview Answer:**
"For a payment system, I'd pick PostgreSQL without much hesitation. The core requirement is correctness: a debit and a credit have to happen together or not at all, and that's exactly what ACID transactions guarantee. MongoDB and Cassandra both trade some consistency for flexibility or scale, and neither of those trades helps a payment ledger, they'd just introduce risk. DynamoDB could technically be forced to work, but you'd be fighting its access-pattern-first design for a workload that actually wants flexible queries and strong transactional guarantees, which is exactly Postgres's strength."

---

## Q4. [Mid] You're storing a marketplace's product catalog, and different sellers attach wildly different custom attributes to their listings depending on category.

## Which of the four databases fits, and why does a rigid relational schema fight you here?

**Answer:**

**A marketplace catalog is the textbook case for a document database, since the actual shape of "a product" changes from category to category, and MongoDB lets each document simply have the fields it needs.**

**Why a relational schema fights you**

- A rigid table needs a fixed set of columns, so a laptop's attributes (RAM, screen size, processor) and a t-shirt's attributes (size, color, material) either need separate tables, a sparse table full of mostly-null columns, or a generic key-value attributes table that turns every query into extra joins.
- Every time a seller invents a new attribute for a new category, you're running a schema migration, which doesn't scale to a marketplace with thousands of active categories.

**Why MongoDB fits**

- Each product is its own document, so a laptop's document and a t-shirt's document can have completely different fields with no wasted columns and no schema migration.
- You still get indexes on whatever fields are common across categories, like price or seller ID, for fast filtering.
- Modern MongoDB does support multi-document transactions if you occasionally need them, so you're not completely giving up consistency guarantees, just not leaning on them as the primary tool.

**Why not Cassandra or DynamoDB here**

- Both work best when you know your access patterns up front and design tables around them. A catalog with constantly evolving, ad hoc attributes and filters is a worse fit than a document model that just lets the shape vary naturally.

**Interview Answer:**
"This is the classic document database use case. A rigid relational table forces every product category into the same fixed columns, which means either a lot of wasted null columns or a painful generic attributes table. MongoDB just lets each product document carry whatever fields make sense for its category, so a new category with new attributes doesn't need a schema migration, it just needs a new shape of document. I'd still index the fields that are common across categories, like price, so filtering stays fast."

---

## Q5. [Mid] You need to ingest sensor readings from two million IoT devices continuously, spread across three regions, and the system must never reject a write, even during a regional outage. Which of the four databases fits, and why do the other three fall short?

**Answer:**

**This is Cassandra's exact reason for existing: it's built to keep accepting writes across multiple regions even when the network between them is partitioned, which is precisely the "never reject a write" requirement here.**

**Why Cassandra fits**

- A masterless, peer to peer architecture means there's no single leader that becomes a bottleneck or a single point of failure for writes.
- Tunable consistency lets you explicitly choose to accept a write as long as it lands on any available replica, favoring availability over immediate global consistency, which is exactly the trade-off this workload wants.
- Linear scalability: adding more nodes adds more write capacity in a predictable, roughly linear way, which matters at two million devices worth of continuous writes.

**Why the other three fall short**

- PostgreSQL's strong consistency model means a partition can force it to reject writes rather than risk inconsistency, the opposite of what this system needs.
- MongoDB can be configured for high availability, but its replication model still centers around a primary per shard, so a regional outage affecting that primary can interrupt writes to that shard until a new primary is elected.
- DynamoDB technically offers global tables for multi-region writes, but Cassandra's data model and battle-tested multi-region write patterns are the more natural fit for a workload built around massive, continuous, geographically distributed ingestion.

**Interview Answer:**

"Cassandra was basically built for this exact scenario. Its masterless architecture means there's no single leader that a regional outage can knock out, and its tunable consistency lets me explicitly favor availability, accept the write somewhere, worry about full consistency later, over rejecting it. Postgres would rather reject a write than risk consistency during a partition, which is the wrong trade-off here, and MongoDB's primary-per-shard model means a regional outage can still interrupt writes until a new primary is elected. For massive, continuous, multi-region ingestion where you can never say no to a write, Cassandra is the natural fit."

---

## Q6. [Senior] You're building a mobile app backend on AWS that needs to handle unpredictable traffic spikes with zero infrastructure management and consistent single-digit millisecond reads at any scale. Which of the four databases fits, and what do you give up compared to Postgres or MongoDB?

**Answer:**

**DynamoDB is built exactly for this: fully managed, scales automatically with traffic, and gives predictable low-latency reads at effectively any scale, but you pay for that by giving up flexible, ad hoc querying.**

**Why DynamoDB fits**

- Fully managed and serverless: no servers to provision, patch, or scale manually, AWS handles partitioning and scaling behind the scenes as traffic grows or shrinks.
- Predictable single-digit millisecond latency at any scale, because reads and writes are designed around a partition key that AWS uses to route requests directly to the right partition.
- Auto-scaling (or on-demand capacity mode) absorbs unpredictable spikes without a human needing to intervene during a traffic surge.

**What you give up compared to Postgres or MongoDB**

- Flexible querying: DynamoDB wants you to know your access patterns up front and design your primary key and secondary indexes around them. A new query pattern you didn't plan for often means redesigning your table or adding a new index, not writing an ad hoc query.

- Joins: there's no join operation. Combining data that lives under different keys means multiple round trips or denormalizing data across items, which shifts complexity from query time to write time.
- Rich transactions: DynamoDB does support transactions across a limited number of items now, but nowhere near the flexibility of a relational transaction spanning arbitrary tables and conditions.

**Interview Answer:**

"DynamoDB fits because it's fully managed and gives predictable low-latency performance at any scale without me provisioning or scaling servers myself, which is exactly what 'zero infrastructure management under unpredictable traffic' is asking for. The trade-off is that I give up flexible querying and joins, I have to know my access patterns ahead of time and design my keys and indexes around them, rather than writing whatever query I need later like I could in Postgres or MongoDB. That's a fine trade for a mobile backend with known access patterns and unpredictable scale, but it would hurt for a workload that needs a lot of ad hoc reporting."

---

## Q7. [Junior] Explain the ACID properties using a real example, like a bank transfer.

**Answer:**

**ACID is a set of guarantees a database transaction gives you so that your data stays correct even when things go wrong.** Take a transfer of 100 dollars from Account A to Account B.

**Atomicity**

- The transaction is all or nothing. Debiting A and crediting B happen together, or neither happens.
- If the crediting step fails after the debit succeeds, atomicity rolls the debit back too.

**Consistency**

- The database moves from one valid state to another valid state.
- If there's a rule that balances can't go negative, the transaction is rejected rather than leaving a broken state.

**Isolation**

- Concurrent transactions don't see each other's half finished work.
- If someone checks Account A's balance mid transfer, they see either the balance before or after the transfer, never a partial update.

**Durability**

- Once the transaction commits, the change survives a crash or power loss.
- The bank doesn't lose your transfer if the server reboots a second later.

Together these four properties are what let you trust that a transaction either fully happened or fully didn't, with no in between state visible to anyone.

**Interview Answer:**

"ACID is the contract a database gives you for a transaction. Using a money transfer as the example: atomicity means the debit and credit both happen or neither does, consistency means the database only ever ends up in a valid state, like never allowing a negative balance, isolation means other transactions can't see the transfer half done, and durability means once it's committed, a crash right after won't lose it. Those four guarantees are why we trust databases with money and other critical state."

## Q8. [Junior] What's the difference between a primary key and a foreign key?

**Answer:**

**Both are ways of identifying and linking rows, but they serve different purposes.**

**Primary key**

- Uniquely identifies each row in a table.
- Cannot be null, and must be unique within that table.
- Often an auto incrementing integer or a UUID, though it can be a natural value like an email address.

**Foreign key**

- A column, or set of columns, in one table that references the primary key of another table.
- Used to establish and enforce a relationship, for example an `orders` table having a `customer_id` column that points to the `customers` table's primary key.
- The database can enforce referential integrity, refusing to insert an order with a `customer_id` that doesn't exist, or refusing to delete a customer who still has orders, depending on how the constraint is configured.

**Why the distinction matters**

- Primary keys are about identity within a table.
- Foreign keys are about relationships between tables, and they're what make relational joins meaningful instead of arbitrary.

**Interview Answer:**
"A primary key uniquely identifies a row within its own table and can never be null or duplicated. A foreign key is a column in one table that points to the primary key of another table, which is how you model relationships, like an order belonging to a customer. The database can enforce that relationship, rejecting an order that references a customer that doesn't exist, which is a big part of what gives relational databases their data integrity guarantees."

---

## Q9. [Junior] What is normalization? Walk me through normalizing a simple orders table that repeats customer information on every row.

**Answer:**

Normalization is the process of organizing a schema to reduce redundant data and avoid update anomalies, usually by splitting one wide table into multiple related tables.

**The problem**

Imagine one `orders` table with columns: order_id, customer_name, customer_email, customer_address, product_name, price, quantity. Every time the same customer places an order, their name, email, and address get repeated. **If a customer changes their email, you have to update every single order row, and if you miss one, your data is now inconsistent.**

**Normalizing it**

1. Pull customer fields into a `customers` table: customer_id, name, email, address.
2. Pull product fields into a `products` table: product_id, name, price.
3. Keep `orders` with: order_id, customer_id, order_date.

4. Add an `order_items` table with: order_id, product_id, quantity, price_at_purchase, to support multiple products per order.

**What this buys you**

- Update a customer's email in one place.
- No wasted storage repeating the same strings.
- Referential integrity: an order can't reference a customer that doesn't exist.

**Trade-off**

- Reading a full order now requires joining across four tables instead of reading one row.

**Interview Answer:**

"Normalization means splitting a wide table into smaller related tables so each fact is stored once. In this case I'd pull customer details into a customers table, product details into a products table, and use an order_items table to link orders to products with quantity and price at purchase. That way updating a customer's email touches one row instead of every order they've ever placed. The cost is that reading a full order now needs joins across several tables instead of one flat row."

---

## Q10. [Junior] What is a database index, and why does it make reads faster?

**Answer:**

**An index is a separate data structure, usually a B-tree, that the database maintains alongside a table to let it find rows quickly without scanning every row.** For example, looking up a user by email in a 50 million row table without an index means scanning rows one by one, while an index on the email column lets the database jump almost directly to the matching row, similar to using a book's index instead of reading every page.

**Without an index**

- A query filtering on a column has to perform a full table scan, checking every row one by one.
- This is fine for a small table, but on millions of rows it's slow.

**With an index**

- The database keeps a sorted structure mapping column values to row locations.
- A lookup becomes a fast tree traversal, similar to looking up a word in a dictionary instead of reading the whole book.
- Range queries, sorting, and equality lookups on the indexed column all get faster.

**The trade-off**

- Every index has to be updated on every insert, update, or delete that touches the indexed column, so writes get slower.
- Indexes take up additional disk space.
- Indexes only help if the query actually uses them, which depends on how selective the column is and how the query is written.

**Interview Answer:**

"An index is an extra data structure, typically a B-tree, that the database keeps sorted by the indexed column so it can jump straight to matching rows instead of scanning the whole table. Without one, a query on a large table has to check every row. The cost is that every write has to update the index too, so indexes

speed up reads at the expense of write performance and extra storage, which is why you index columns you filter or sort on frequently, not every column."

---

## Q11. [Junior] What's the difference between a database transaction and a single standalone query?

**Answer:**

A single query is one operation, like one SELECT, INSERT, UPDATE, or DELETE statement, executed and completed independently. **A transaction is a group of one or more operations that are treated as a single unit of work.** For example, a single UPDATE statement that adds ten dollars to one account is a standalone query, but transferring money by debiting one account and crediting another needs to be wrapped in a transaction so both changes succeed or fail together.

**Single query**

- Executes and either succeeds or fails on its own.
- No need to coordinate with other statements.

**Transaction**

- Wraps multiple statements between a BEGIN and a COMMIT (or ROLLBACK).
- All statements inside succeed together, or if any fails, the whole group can be rolled back so none of the changes take effect.
- Needed whenever multiple statements must succeed or fail together to keep data valid, like debiting one account and crediting another.

**Why it matters**

- Without transactions, a crash or error partway through a multi step operation can leave your data in an inconsistent state, like money leaving one account but never arriving in the other.
- Transactions also interact with isolation levels, which control what concurrent transactions can see of each other's in progress work.

**Interview Answer:**
"A single query is one statement that runs on its own. A transaction groups multiple statements so they succeed or fail together as one unit, using BEGIN and COMMIT or ROLLBACK. I'd use a transaction any time correctness depends on multiple statements happening atomically, like transferring money between two accounts, where you never want the debit to succeed without the matching credit. Without wrapping those two updates in a transaction, a crash between them would leave the data in a broken, inconsistent state."

---

## Q12. [Junior] What's the difference between a SQL database and a document database like MongoDB, at a basic level?

**Answer:**

**Both store and query data, but they structure it very differently.**

**SQL database**

- Data lives in tables with a fixed set of columns.
- Related data usually lives in separate tables, joined together at query time.
- Schema changes, like adding a column, apply to the whole table.

**Document database**

- Data lives in collections of documents, usually JSON or BSON.
- Each document can nest related data inside itself, like an order document that embeds its line items directly.
- Documents in the same collection can have different fields, there's no single fixed schema enforced by the database.

**Practical difference**

- In SQL, fetching an order and its items typically means joining an `orders` table with an `order_items` table.
- In a document database, that same order and its items might be stored as one embedded document, fetched in a single read with no join.
- SQL tends to be better when your data has many relationships you need to query flexibly. Document databases tend to be better when you consistently read and write a single aggregate, like one order, as a unit.

**Interview Answer:**
"A SQL database stores data in fixed schema tables and relies on joins to bring related data together. A document database like MongoDB stores each record as a self contained document, often JSON, which can embed related data directly and doesn't require every document in a collection to have the same fields. So fetching an order with its items is a join in SQL but can be a single document read in MongoDB. The trade-off is SQL gives you more flexible querying across relationships, while documents are optimized for reading and writing one aggregate at a time."

---

## Q13. [Mid] I'm storing a product catalog where each product has a different set of attributes depending on category. Would you use a relational table or a document database, and why?

**Answer:**

This is a classic schema flexibility question, and the right answer depends on how much you also need cross category querying and reporting.

**Option: Relational table**

- Pros: strong consistency, easy joins with orders and inventory tables, mature tooling for reporting.
- Cons: variable attributes per category are awkward. You either create a huge table with mostly null columns, use an Entity Attribute Value pattern (a generic table storing rows of entity ID, attribute name, and value instead of real columns) that's painful to query, or create a separate table per category which multiplies complexity.

**Option: Document database**

- Pros: each product document can naturally hold whatever attributes its category needs, no schema migration needed to add a new category with new fields, and reads for a single product are fast since everything about it is in one document.
- Cons: harder to run ad hoc analytical queries across attributes, like "find all products across categories with a warranty over 2 years," and no enforced referential integrity with related tables like orders.

**Best Choice**

**A document database for the catalog itself, with relational tables for orders, inventory, and payments.**

**Why?**

Product attributes are exactly the kind of semi structured, per category variable data that document databases are built for, and catalog reads are usually single product lookups rather than complex joins. But orders, inventory, and payments have strict relationships and need transactional guarantees, so those stay relational. Many real e-commerce systems are polyglot for exactly this reason.

**Interview Answer:**

"I'd lean toward a document database for the catalog, because each product category has a different shape of attributes, and forcing that into a fixed relational schema means either a lot of nullable columns or an awkward EAV pattern. A document per product can hold whatever fields its category needs without schema migrations. I'd keep orders, inventory, and payments in a relational database though, since those need strong consistency and well defined relationships, so overall this ends up being a polyglot design rather than one database for everything."

---

## Q14. [Mid] Walk me through the four standard SQL isolation levels and what anomaly each one prevents.

**Answer:**

**Isolation levels control how much one transaction can see of another transaction's uncommitted or concurrently changing work.** Going from weakest to strongest:

**Read Uncommitted**

- Transactions can see uncommitted changes made by other transactions.
- Allows dirty reads, non-repeatable reads, and phantom reads.
- Rarely used in practice, mostly historical.

**Read Committed**

- A transaction only ever sees data that has been committed by other transactions.
- Prevents dirty reads.
- Still allows non-repeatable reads, if you read the same row twice in the transaction, it might have changed in between because another transaction committed an update.
- This is the default in PostgreSQL and Oracle.

**Repeatable Read**

- Guarantees that if you read a row once, you'll see the same value if you read it again within the same transaction.
- Prevents dirty reads and non-repeatable reads.
- Can still allow phantom reads in some databases, where a range query returns new rows inserted by another transaction, though MySQL's InnoDB actually prevents phantoms at this level using gap locking.
- This is MySQL's default.

**Serializable**

- The strictest level, transactions behave as if they ran one at a time, in some serial order.
- Prevents dirty reads, non-repeatable reads, and phantom reads.

- Achieved through heavier locking or optimistic conflict detection, which reduces concurrency and throughput.

**Interview Answer:**
"From weakest to strongest: read uncommitted lets you see other transactions' uncommitted changes, so you get dirty reads. Read committed fixes that by only showing committed data, but you can still get non-repeatable reads if a row changes between two reads in your transaction. Repeatable read fixes that by locking in the values you've read, but can still allow phantom rows to appear in range queries. Serializable is the strongest, transactions behave as if run one after another, which prevents all of these anomalies but costs you concurrency and throughput."

---

## Q15. [Mid] Explain a dirty read, a non-repeatable read, and a phantom read, with a concrete example for each.

**Answer:**

**These are the three classic concurrency anomalies that isolation levels are designed to prevent.**

**Dirty read**

- Transaction A updates a row but hasn't committed yet.
- Transaction B reads that uncommitted value.
- Transaction A then rolls back.
- Transaction B has now acted on data that never actually existed.
- Example: A increases a product's stock count from 10 to 100 as part of a larger transaction. B reads the stock as 100 and lets a customer order 50 units. A then rolls back because of an error elsewhere, so the real stock was still 10, and B oversold.

**Non-repeatable read**

- Transaction A reads a row.
- Transaction B updates and commits a change to that same row.
- Transaction A reads the same row again within its own transaction and gets a different value.
- Example: A reads an account balance of 500 dollars at the start of a report. Midway through generating the report, B transfers money out and commits. A reads the balance again and now sees 300, inconsistent with its earlier read.

**Phantom read**

- Transaction A runs a query with a WHERE clause and gets a set of rows.
- Transaction B inserts a new row matching that WHERE clause and commits.
- Transaction A re runs the same query and sees an extra row that wasn't there before.
- Example: A counts all orders placed today as part of a report, gets 100. B inserts a new order that also qualifies and commits. A re runs the count and now gets 101, within the same logical transaction.

**Interview Answer:**
"A dirty read is when you read another transaction's uncommitted change, and that transaction then rolls back, so you acted on data that never really existed. A non-repeatable read is when you read the same row twice in one transaction and get two different values because another transaction committed a change in between. A phantom read is similar but for a set of rows, you run the same range query twice and get a different number of rows because another transaction inserted or deleted a matching row in between."

## Q16. [Mid] When would you deliberately denormalize a schema, and what do you give up by doing it?

**Answer:**

Denormalization means intentionally introducing redundancy, duplicating or precomputing data, to make reads faster at the cost of write complexity and consistency risk.

**When it makes sense**

- Read heavy workloads where the same joined data is fetched constantly and the join is a measurable bottleneck.
- Reporting or analytics tables where you want a flattened view for fast aggregation.
- Storing a computed value, like an order's total, directly on the order row instead of recalculating it from line items on every read.
- Storing a denormalized copy of frequently accessed reference data, like a product's name and price on an order line item at the time of purchase, so historical orders don't change if the product is later renamed or repriced.

**What you give up**

- Data can become inconsistent if you update one copy and forget the other. Denormalized fields need an explicit strategy to stay in sync, like triggers, application logic, or scheduled reconciliation jobs.
- More storage is used since data is duplicated.
- Writes get more complex, since a single logical update might now need to touch multiple places.

**Best Choice**

Normalize by default, denormalize selectively where a specific read pattern is measurably slow or where you specifically want a historical snapshot, like the price at time of purchase.

**Why?**

**Denormalization is a targeted optimization, not a starting design principle.** Applying it everywhere upfront trades away the correctness guarantees normalization gives you before you've proven you need the performance.

**Interview Answer:**

"I'd denormalize when a specific read path is a proven bottleneck, or when I actually want a historical snapshot, like storing the price of a product on an order line so it doesn't change if the product is repriced later. The cost is that now there are two copies of data that can drift out of sync if I'm not careful about how they're updated. My default is to normalize first and only denormalize deliberately, in specific places, once I can point to the read pattern that justifies it."

## Q17. [Mid] I'm building the database for a social media feed showing posts from people a user follows. Would you reach for a relational database or a NoSQL database here?

**Answer:**

This depends heavily on scale and access pattern, but let's compare the realistic options.

**Option: Relational database**

- Pros: straightforward to model users, posts, and follows as tables with foreign keys, easy to enforce integrity, like a post always belonging to a real user, and to run ad hoc queries during early development.
- Cons: generating a feed usually means querying "give me posts from everyone this user follows, ordered by time," which at scale means large joins or fan out queries that get expensive as follower counts grow into the millions.

**Option: Wide-column or document database**

- Pros: many production feed systems precompute the feed and store it as a per user list, essentially "here are the post IDs for this user's feed," which is a fast key based lookup rather than a join. Wide-column stores like Cassandra are built for this kind of high write throughput, time ordered access pattern.
- Cons: more operational complexity, you need a separate fan out process that pushes new posts into followers' feeds, and eventual consistency means a new post might not appear instantly for every follower.

**Best Choice**

Relational for the underlying users, posts, and follows relationships and for a first version at moderate scale. **A wide-column or key-value store for the precomputed feed itself once scale demands it.**

**Why?**

Early on, a relational database is simpler to build and reason about, and joins are fine at moderate scale. Once follower counts and write volume grow large, the feed generation query becomes the bottleneck, and that's when a precomputed, fan out based feed backed by a fast key based store pays off.

**Interview Answer:**

"For a straightforward version I'd start relational, modeling users, posts, and follows with foreign keys, since it's easy to reason about and query flexibly early on. As scale grows, generating a feed by joining across everyone a user follows gets expensive, so I'd move to a fan out on write approach, precomputing each user's feed and storing it in a fast key based or wide-column store like Cassandra, which is built for high write throughput and simple lookups. The relational database still owns the source of truth for posts and relationships."

---

## Q18. [Mid] What's the difference between a key-value store and a wide-column store? Give me an example of a good use case for each.

**Answer:**

**Both are NoSQL categories optimized for fast lookups, but they organize data differently and suit different access patterns.**

**Key-value store**

- Structure: every item is a key mapped to an opaque value, often a blob or a small JSON document, with no visibility into its internal structure from the database's perspective.
- Access pattern: extremely fast lookups by key, but you generally can't query by the contents of the value.
- Good use case: session storage for a web application, where you look up a session token and get back a blob of session data, or a simple cache of computed results keyed by request parameters.

**Wide-column store**

- Structure: data is organized into rows identified by a key, but each row can have a large, flexible, sparse set of columns grouped into column families, and columns can be added per row without a global schema change.
- Access pattern: efficient range scans over a row key, and efficient retrieval of specific column families without reading unrelated columns.
- Good use case: time series data, like sensor readings or event logs, where you write huge volumes of data keyed by device ID and timestamp, and query ranges of time efficiently. Cassandra and HBase are common examples.

**Interview Answer:**

"A key-value store is the simplest model, a key maps to an opaque value, and it's great when your access pattern is purely 'give me the value for this key,' like session storage. A wide-column store, like Cassandra, organizes data into rows with a flexible, sparse set of columns and is built for very high write throughput plus efficient range scans over a row key, which makes it a strong fit for time series data like IoT sensor readings or event logs where you're constantly appending and querying by time range."

---

## Q19. [Mid] What's a composite index, and when does it help more than two separate single-column indexes?

**Answer:**

**A composite index, also called a multi column index, is a single index built across two or more columns together, in a specific order.**

**How it works**

- The index is sorted first by the first column, then by the second column within each value of the first, and so on.
- A query benefits from a composite index when its WHERE clause filters on a prefix of the indexed columns, in order, or when it can satisfy a required sort order.

**Why it beats two separate indexes**

- With two separate single column indexes, the database usually can only use one of them efficiently for a given query, or it has to do extra work combining results from both.
- A composite index on `(customer_id, order_date)` directly supports a query like "find all orders for this customer, sorted by date," using a single, efficient index scan.
- If you query only by `order_date` without `customer_id`, the composite index generally won't help, since the column order matters. This is why column order in the index matters as much as which columns are included.

**Trade-off**

- Composite indexes are larger, and they only help queries that match their column order, so they're not a blanket replacement for thinking through actual query patterns.

**Interview Answer:**

"A composite index is one index built across multiple columns in a specific order, rather than one index per column. It's the right call when queries consistently filter or sort on the same combination of columns together, like customer_id and order_date, because the database can satisfy that with a single efficient index scan instead of combining results from two separate indexes. The catch is that column order in the

index matters, a composite index on customer_id then order_date won't help a query that filters only on order_date."

---

## Q20. [Mid] Design a schema for an e-commerce checkout flow that needs to track inventory, orders, and payments. What tables would you create and how would they relate?

**Answer:**

**This needs to support atomic stock decrements, order creation, and payment tracking, all consistently.**

**Core tables**

1. `products`: product_id, name, price, description.
2. `inventory`: product_id (foreign key to products), quantity_available, reserved_quantity.
3. `customers`: customer_id, name, email, address.
4. `orders`: order_id, customer_id (foreign key), status, created_at, total_amount.
5. `order_items`: order_id (foreign key), product_id (foreign key), quantity, price_at_purchase.
6. `payments`: payment_id, order_id (foreign key), amount, status, payment_method, processed_at.

**Key design decisions**

- Store `price_at_purchase` on `order_items` rather than relying on `products.price`, so historical orders don't change if a price is later updated.
- Give `orders` a `status` field, like pending, paid, shipped, cancelled, to track the checkout lifecycle.
- Split `payments` from `orders` since one order might have a failed payment attempt followed by a successful retry, and you want that history.
- Use `reserved_quantity` on `inventory` to hold stock during checkout before payment confirms, then convert reservation to a real decrement on payment success, or release it back on failure or timeout.

**Transaction boundaries**

- Creating the order, decrementing or reserving inventory, and creating the order_items should happen inside a single database transaction, so a failure partway through doesn't leave stock decremented with no order recorded, or vice versa.
- Payment processing usually happens through an external payment provider, so it's tracked as a separate step with its own status, since you can't wrap an external API call in the same database transaction.

**Interview Answer:**
"I'd model products, inventory, customers, orders, order_items, and payments as separate tables. Order_items store the price at the time of purchase rather than pointing back to the live product price, so historical orders stay accurate. Inventory gets a reserved_quantity so I can hold stock during checkout before payment confirms, then either commit the decrement or release the reservation. Creating the order and reserving inventory happen in one transaction, but payment processing is tracked separately with its own status, since it goes through an external provider and can't be wrapped in the same database transaction."

## Q21. [Mid] What's the difference between an OLTP workload and an OLAP workload, and how does that affect database choice?

**Answer:**

**These describe two very different ways applications interact with data, and they push database design in opposite directions.**

**OLTP, Online Transaction Processing**

- Characterized by many short, simple transactions: insert an order, update a balance, look up a single customer.
- Optimized for low latency on individual reads and writes, and for correctness under concurrent access.
- Schema is usually normalized to keep writes cheap and consistent.
- Examples: the database behind a checkout flow, a banking app, a booking system.

**OLAP, Online Analytical Processing**

- Characterized by complex queries scanning huge volumes of historical data: aggregations, trends, reports across millions of rows.
- Optimized for scanning and aggregating large datasets quickly, often using column oriented storage so a query only reads the columns it needs.
- Schema is often denormalized, using star or snowflake schemas, to make aggregation queries fast and simple.
- Examples: a data warehouse computing monthly revenue by region, a business intelligence dashboard.

**Why this affects database choice**

- Running heavy analytical queries directly against your OLTP database can slow down or lock the very transactions your application depends on.
- Most systems separate the two: an OLTP database, like PostgreSQL or MySQL, handles live application traffic, and data is periodically pipelined into an OLAP system, like Redshift, BigQuery, or Snowflake, for reporting.

**Interview Answer:**
"OLTP workloads are lots of small, fast transactions, like placing an order or updating a balance, and they need a normalized schema and low latency reads and writes. OLAP workloads are big analytical queries scanning huge amounts of historical data to compute trends or reports, and they benefit from denormalized, often column oriented storage optimized for scanning and aggregating. Because those access patterns conflict, most systems keep them separate, an OLTP database for live application traffic and a dedicated data warehouse fed by a pipeline for analytics."

## Q22. [Mid] Explain optimistic locking versus pessimistic locking. When would you choose one over the other?

**Answer:**

Both are strategies for handling concurrent updates to the same data, but they differ in when they detect and resolve conflicts.

**Pessimistic locking**

- Acquires a lock on a row before reading or modifying it, blocking other transactions from touching it until the lock is released.
- Guarantees no conflict can happen, because concurrent access is physically prevented.
- Cost: reduces concurrency, since other transactions wait, and can lead to deadlocks if multiple transactions lock resources in different orders.
- Good fit: high contention scenarios where conflicts are likely and correctness matters more than throughput, like updating a shared inventory count during a flash sale.

**Optimistic locking**

- Doesn't lock anything upfront. Instead it reads a row along with a version number or timestamp, does its work, and on write checks whether the version still matches.
- If another transaction updated the row in the meantime, the version won't match, and the write is rejected, requiring the application to retry.
- Cost: under high contention, many retries can happen, wasting work.
- Good fit: low contention scenarios where conflicts are rare, so most writes succeed on the first try and you avoid the overhead of locking.

**Best Choice**

**Optimistic locking for most application level concurrency control, pessimistic locking for specific high contention hotspots.**

**Why?**

Most rows in most applications aren't being fought over at the same instant, so optimistic locking's low overhead wins in the common case. Reserve pessimistic locking for narrow, known hotspots like a limited inventory pool during a sale, where conflicts are frequent enough that retries would be wasteful.

**Interview Answer:**

"Pessimistic locking grabs a lock before touching a row so no one else can conflict with you, which guarantees correctness but reduces concurrency and can cause deadlocks. Optimistic locking doesn't lock upfront, it checks a version number on write and rejects the update if someone else changed the row first, requiring a retry. I default to optimistic locking since most data isn't under heavy contention, and I'd reach for pessimistic locking specifically for known hotspots, like a limited stock count during a flash sale, where conflicts are frequent enough that retries would hurt more than blocking would."

---

## Q23. [Mid] You need to store session data for a website with millions of concurrent users, each session just a small blob of key-value data. What kind of database fits best?

**Answer:**

**This access pattern, simple key lookup, high volume, low latency, points strongly away from a relational database.**

**Requirements analysis**

- Access pattern is almost always "give me the session data for this session ID," a pure key lookup, never a complex query or join.
- Needs to handle a very high volume of reads and writes with low latency.

- Session data is often short lived, expiring after a period of inactivity.
- Losing a session occasionally, forcing a re login, is usually an acceptable trade-off, so extremely strong durability guarantees aren't critical.

**Option: Relational database**

- Pros: easy to set up if you already have one.
- Cons: overkill for pure key lookups, and the throughput you'd get per node is far lower than a purpose built key-value store.

**Option: Key-value store, like Redis or DynamoDB**

- Pros: designed exactly for this access pattern, extremely fast key based reads and writes, built in support for expiration (TTL), scales horizontally to handle very high request volume.
- Cons: not meant for complex querying, but that's fine since sessions don't need it.

**Best Choice**

A key-value store, Redis for in memory speed or DynamoDB for a managed, durable option.

**Why?**

The access pattern is a textbook key-value use case, and these stores give you the throughput, low latency, and built in expiration that a relational database would require extra application logic to replicate poorly.

**Interview Answer:**

"Session storage is a pure key lookup workload, no joins, no complex queries, just get and set by session ID, with data that naturally expires. That's exactly what key-value stores are built for. I'd use Redis if I want in memory speed and don't mind sessions being somewhat ephemeral, or DynamoDB if I want a managed store with built in TTL support and durability. A relational database would work but adds overhead and complexity for a workload that's this simple and this high volume."

---

## Q24. [Senior] I'm designing the database for a banking system handling fund transfers between two accounts. Which isolation level would you pick, and what could go wrong if you picked a weaker one?

**Answer:**

Money movement is the canonical case for taking isolation levels seriously, because the anomalies aren't theoretical, they translate directly into financial errors.

**Requirements**

- Two updates, a debit and a credit, must be atomic and must never be visible to another transaction half done.
- The system must prevent both accounts from being read or updated in ways that create incorrect balances under concurrency.

**Weaker isolation levels and what breaks**

- Read uncommitted: another transaction could read a balance mid transfer, before the debit and credit are both applied, and make a decision based on money that doesn't actually exist yet, like approving a withdrawal against a balance that's about to be reduced.
- Read committed: prevents dirty reads, but a report or a second transaction reading the same balance twice could see it change between reads, and more importantly, doesn't fully protect

against write skew, where two transactions each read a value, both decide it's safe to proceed, and together violate an invariant like an overdraft limit.

- Repeatable read: better, but depending on the database, may still allow write skew in specific concurrent scenarios, like two concurrent transfers out of the same account that each individually check the balance is sufficient but together overdraw it.

**Best Choice**

Serializable isolation, or repeatable read combined with explicit row level locking (`SELECT FOR UPDATE`) on the accounts being modified.

**Why?**

**Financial correctness requires eliminating write skew entirely, not just dirty and non-repeatable reads.** Serializable guarantees the outcome is equivalent to running transactions one at a time, which is exactly the guarantee you want for money movement, and the throughput cost is acceptable because transfer volume per account is typically low relative to total database throughput.

**Interview Answer:**

"For fund transfers I'd use serializable isolation, or repeatable read with explicit row locks on the accounts involved, because weaker levels expose write skew, where two concurrent transfers each check a balance independently, both see it as sufficient, and together overdraw the account. That's not a rare edge case in banking, it's a directly financial bug. Serializable makes the system behave as if transfers happened one at a time, which is worth the throughput cost given that transfer volume per account is usually low relative to the database's overall capacity."

---

## Q25. [Senior] Your e-commerce inventory table is seeing heavy lock contention during flash sales, and checkout requests are timing out. How would you redesign the schema or transaction logic to reduce it?

**Answer:**

**This is a hot row problem, many concurrent transactions competing to update the same inventory rows for popular items.**

**Diagnose first**

1. Confirm it's row level lock contention on a small number of hot product rows, not a broader database bottleneck like connection pool exhaustion.
2. Check whether the current logic reads the quantity, checks it in application code, then writes, which is a classic race condition needing a lock or an atomic operation.

**Mitigations**

- Replace read then write logic with a single atomic decrement: `UPDATE inventory SET quantity = quantity - 1 WHERE product_id = ? AND quantity > 0`, checking the affected row count to know if it succeeded. This removes the separate read step and shrinks the lock window dramatically.
- Shard the hot row into multiple counter rows, for example splitting a single product's stock of 1000 into 10 rows of 100 each, and having checkout attempts hit a random shard, then reconciling if needed. This spreads contention across multiple rows instead of one.
- Move to a reservation model with a short lived hold: decrement an in memory or Redis backed counter first for a fast accept or reject decision, then confirm against the database asynchronously, keeping the hot path out of the relational database entirely.

- Queue checkout attempts for the hottest items instead of letting them all hit the database simultaneously, smoothing the load rather than serializing everyone through row locks.

**Best Choice**

Start with the atomic decrement statement, since it's a quick, low risk fix, then add a fast in memory reservation layer in front of the database if the flash sale volume still overwhelms it.

**Why?**

The atomic decrement removes the read-then-write race and shrinks lock duration immediately with minimal design change. If contention is still too high at extreme scale, moving the hot path decision making to a fast in memory layer avoids hammering the relational database's row locks altogether, while the database remains the durable source of truth reconciled shortly after.

**Interview Answer:**

"First I'd make sure the checkout logic uses a single atomic update, decrementing quantity directly in the WHERE clause instead of reading the quantity in application code and writing it back separately, since that read-then-write pattern is what causes long lock windows and race conditions. If that's still not enough under flash sale load, I'd put a fast in memory counter, like Redis, in front of the database to make the accept or reject decision immediately, and reconcile the actual database row asynchronously, keeping the extreme write volume off the relational database's row locks."

---

## Q26. [Senior] You have a table with 500 million rows, there's an index on the column you're filtering by, but queries are still slow. What are the possible causes?

**Answer:**

**An index existing doesn't guarantee it's being used effectively.** Several things could be going wrong.

**The index isn't being used at all**

- Run `EXPLAIN` on the query to confirm the database is actually choosing an index scan rather than a full table scan.
- A function applied to the column in the WHERE clause, like `WHERE LOWER(email) = ?`, prevents the database from using a plain index on `email` unless there's a matching functional index.
- An implicit type mismatch, like comparing a string column to a numeric literal, can also cause the planner to skip the index.

**The index isn't selective enough**

- If the indexed column has low cardinality, like a status column with only three possible values, the index may not narrow the search enough, and the planner might reasonably choose a full scan instead.

**Too many rows still match**

- If the filter matches a large fraction of the 500 million rows, say 20 percent, reading the index and then fetching each matching row from the table can be slower than just scanning the table directly, because of the extra random I/O from index lookups back to the table.

**The query needs more than the index provides**

- If the query also sorts, filters on additional columns, or needs columns not in the index, the database might do the index lookup and then a lot of additional work anyway. A composite index or a covering index that includes the extra needed columns can help.

**Stale statistics or an outdated query plan**

- The database's query planner relies on statistics about data distribution. If these are stale after major data growth or changes, the planner might pick a bad plan. Running an analyze or update statistics command can fix this.

**Interview Answer:**

"I'd start by running EXPLAIN to see if the index is even being used, since something like applying a function to the column in the WHERE clause can silently prevent that. If it is being used, I'd check selectivity, a low cardinality column might not narrow things down enough to be worth an index lookup versus a scan. I'd also consider whether the query needs additional columns not covered by the index, which might call for a composite or covering index, and I'd check whether table statistics are stale, since an outdated query plan can pick a bad strategy even with a good index available."

---

## Q27. [Senior] Walk me through how you'd decide between a document database and a relational database for a multi-tenant SaaS product storing each customer's configuration settings.

**Answer:**

**Configuration data is a common real world case where the "right" answer depends on how varied and how deeply nested the configuration actually is, and how much you need to query across tenants.**

**Option: Relational database**

- Pros: strong consistency, well suited if configuration has a mostly fixed, well known shape across tenants, easy to query across tenants for things like "find all tenants with feature X enabled," straightforward to enforce constraints like valid value ranges.
- Cons: if different tenants or different plan tiers need meaningfully different sets of configuration options, you end up with a table full of nullable columns, or a key-value style config table that loses type safety and is awkward to query.

**Option: Document database**

- Pros: naturally fits configuration that varies significantly by tenant or evolves frequently, since each tenant's config document can have its own shape without a schema migration, and reading a tenant's entire configuration is a single document fetch.
- Cons: harder to run structured cross tenant queries or enforce strict validation rules at the database level, that validation work usually moves into the application layer.

**Best Choice**

Store the tenant's core identifying and billing data relationally, since that needs strong consistency and cross tenant queries, and store the flexible configuration blob as a JSON column within the relational database, or as a separate document store if the config is large and frequently updated.

**Why?**

Modern relational databases like PostgreSQL support JSON or JSONB columns, which gives you the flexibility of document style storage for the variable part of the data while keeping it colocated with the strongly consistent relational data it belongs to, avoiding the operational overhead of running two separate database systems unless the configuration data's scale specifically demands a dedicated document store.

**Interview Answer:**

"I'd look at how much configuration actually varies between tenants. If it's fairly uniform, a relational table works fine and gives me easy cross tenant queries. If it varies a lot by tenant or plan tier, I'd store it as a JSONB column in PostgreSQL rather than reaching for a whole separate document database, since that gives me schema flexibility for the config blob while keeping it next to the tenant's core relational data, like billing and identity, which still needs strong consistency and constraint enforcement. I'd only split into a dedicated document store if the configuration data grew large or complex enough to justify the operational overhead."

---

## Q28. [Senior] What is a write skew anomaly, and how does serializable isolation prevent it?

**Answer:**

Write skew is a subtle anomaly where two transactions each read overlapping data, each makes a decision based on what they read, and both commit changes that are individually valid but together violate an invariant that spans both rows.

**Classic example**

A hospital rule says at least one doctor must be on call at all times. Two doctors, A and B, are both on call. Doctor A checks the on call list, sees both A and B on call, and decides it's safe to take themselves off call. At the same time, Doctor B checks the same list, also sees both on call, and decides it's safe to take themselves off call too. Both transactions commit. **Now nobody is on call, even though each transaction individually followed the rule based on what it read.**

**Why weaker isolation levels don't catch this**

- Read committed and even repeatable read only protect against one transaction interfering with another's read or write of the same row. Here, A and B are reading the same rows but writing different rows, their own respective doctor record, so row level locking on the row being written doesn't create a conflict.

**How serializable isolation prevents it**

- True serializable isolation ensures the outcome is equivalent to some serial execution of the transactions, one after the other.
- In a serial execution, whichever doctor's transaction ran second would have seen the first doctor already off call, and would have been blocked from also going off call, preserving the invariant.
- Databases implement this either through predicate locking, which locks based on the condition of the query rather than specific rows, or through serializable snapshot isolation, which detects these dependency cycles between transactions and aborts one of them, forcing a retry.

**Interview Answer:**

"Write skew happens when two transactions each read some overlapping data, each independently decides their write is safe based on what they read, and both commit, but together they violate a rule that depends on both rows. The classic example is two on call doctors each independently deciding it's fine to go off call because they see the other one is still on call, and both go off call at once. Regular row level locking doesn't catch this because they're writing different rows. Serializable isolation prevents it by ensuring the result is equivalent to running the transactions one at a time, using predicate locks or detecting the conflict and aborting one transaction to force a retry."

## Q29. [Senior] A single application needs strong consistency for account balances and very high write throughput for a user activity log. Would you use one database for both, or split them? Walk me through your reasoning.

**Answer:**

These two requirements pull in genuinely different directions, and forcing one database to handle both well usually means compromising on one of them.

**Requirement analysis**

- Account balances: low volume relative to the activity log, but every write must be strongly consistent, correctly isolated, and durable, since incorrect balances are a serious problem.
- Activity log: very high write volume, append only, rarely updated once written, and the consistency bar is much lower, an activity log entry arriving a few seconds late or even occasionally being lost isn't catastrophic the way a wrong balance is.

**Option: One relational database for both**

- Pros: simpler operationally, one system to manage, transactional guarantees available everywhere.
- Cons: the activity log's high write volume competes for the same resources, connections, disk I/O, lock manager, as the balance transactions, and a relational database's strong consistency machinery is overkill and a throughput ceiling for a simple append heavy log.

**Option: Split into two systems**

- Balances: a relational database, PostgreSQL or similar, using serializable isolation or row locking as needed, since correctness matters far more than raw throughput here and the volume is manageable.
- Activity log: a write optimized store, like a wide-column database such as Cassandra, or an append only log system like Kafka feeding into a data warehouse, since the pattern is high volume appends with eventual, not immediate, read consistency.

**Best Choice**

**Split them.**

**Why?**

The two workloads have opposite bottlenecks: balances need correctness under low to moderate volume, the activity log needs throughput under high volume with relaxed consistency. Running both on the same database means either over engineering the log's storage with unnecessary transactional overhead, or under protecting the balances by loosening guarantees database wide. Polyglot persistence lets each subsystem use the tool suited to its actual requirements, at the cost of running and operating two systems instead of one.

**Interview Answer:**

"I'd split them. Account balances need strong consistency under a relatively low, controlled volume of transactions, so a relational database with serializable isolation or explicit locking makes sense there. The activity log needs to absorb very high write throughput with much looser consistency requirements, an entry showing up a couple seconds late is fine, which is exactly what a write optimized store like Cassandra or an event log like Kafka is built for. Trying to serve both from one relational database means the log's write volume competes with the balance transactions for the same resources, and you end up compromising one workload to serve the other."

## Q30. [Senior] Explain how a B-tree index works under the hood, and why it's the default index structure in most relational databases.

**Answer:**

**A B-tree, short for balanced tree, is a self balancing tree data structure that keeps data sorted and allows searches, insertions, and deletions in logarithmic time.**

**Structure**

- The tree has a root node, internal nodes, and leaf nodes.
- Each node holds multiple sorted keys, not just one, and pointers to child nodes representing ranges between those keys.
- Leaf nodes contain the actual indexed values, along with pointers to the corresponding rows in the table, or in some databases, the leaf nodes are the table data itself, which is what a clustered index means.
- The tree stays balanced, meaning every leaf is roughly the same distance from the root, so lookups have predictable performance regardless of which value you're searching for.

**How a lookup works**

1. Start at the root, compare the search value against the keys stored there.
2. Follow the pointer to the child node representing the range the search value falls into.
3. Repeat at each level until reaching a leaf node.
4. The leaf node either contains the value directly, or a pointer to the row's location on disk.

**Why B-trees fit relational databases so well**

- Disk I/O is the dominant cost in database performance, and B-trees are designed around minimizing the number of disk reads needed per lookup, since each node can be sized to match a disk page, letting one read pull in many keys at once.
- Because keys are stored in sorted order, B-trees support not just exact match lookups but also efficient range queries, like `WHERE age BETWEEN 20 AND 30`, and ordered scans for ORDER BY, which is essential for typical SQL workloads.
- The tree height grows logarithmically with the number of rows, so even with hundreds of millions of rows, a lookup only takes a handful of node traversals.

**Interview Answer:**
"A B-tree is a balanced, sorted tree structure where each node holds multiple keys and points to child nodes covering ranges of values, so a lookup walks from the root down to a leaf comparing against a handful of keys at each level, giving you logarithmic time search. It's the default in relational databases because it's built around minimizing disk reads, each node is sized to match a disk page, and because keeping data sorted means it supports range queries and ordered scans efficiently, not just exact match lookups, which covers the vast majority of real SQL query patterns."

---

## Q31. [Senior] Your team wants to add a new secondary index to speed up a slow read query on a table that gets thousands of writes per second. What's the trade-off, and how would you decide if it's worth it?

**Answer:**

Every index is a bet that the read speedup is worth the write cost, and on a heavily written table that bet needs real evidence, not intuition.

**The trade-off**

- Every insert, update, or delete that touches the indexed column now also has to update the index's B-tree structure, which adds CPU and I/O overhead per write.
- At thousands of writes per second, even a modest per write overhead compounds into meaningful additional load, potentially pushing write latency up or reducing overall write throughput.
- Indexes also consume additional disk space and memory, since the database tries to keep frequently used indexes cached.

**How to decide**

1. Measure the actual cost of the slow query today: how often does it run, how slow is it, and what's the business impact of that slowness.
2. Estimate or test the index's write overhead in a staging environment under realistic write load, measuring write latency and throughput before and after adding the index.
3. Check whether an existing index can be extended into a composite index that covers this query instead of adding a brand new one, since that avoids the cost of maintaining an entirely separate structure.
4. Consider whether the read query could instead be served by a read replica, a cached materialized view, or a separate reporting store, keeping the primary write heavy table free of extra index overhead.
5. If you do add it, monitor write latency and throughput in production afterward to confirm the estimate held, and be ready to drop the index if the cost outweighs the benefit.

**Best Choice**

Add the index only after measuring both sides, and prefer extending an existing composite index or offloading the read elsewhere if the write path is genuinely write sensitive.

**Why?**

On a table already under heavy write load, guessing wrong in either direction is costly, an unnecessary index silently degrades write throughput across the whole system, while skipping a genuinely needed index leaves a real read problem unsolved. **Testing under realistic load turns this from a guess into a measured decision.**

**Interview Answer:**

"I'd want real numbers before adding it. I'd measure the current write throughput and latency on that table, then test the new index under a realistic write load in staging to see the actual overhead, since at thousands of writes per second even a small per write cost adds up. I'd also check whether extending an existing composite index could cover the query instead of adding a new one, or whether the read could be served from a replica or a separate store entirely, keeping the primary write path untouched. I'd only ship the index once the measured read benefit clearly outweighs the measured write cost, and I'd keep monitoring after it's live."

---

## Q32. [Senior] Compare a clustered index and a non-clustered index. How does the choice of clustered index affect the design of the rest of your schema?

**Answer:**

**These two index types differ in their relationship to the actual physical storage of the table's data, which has real consequences for schema design.**

**Clustered index**

- Determines the physical order in which rows are stored on disk. The table data itself is organized as the leaf level of the index.
- A table can have only one clustered index, since data can only be physically sorted one way at a time.
- Lookups by the clustered key are very fast, since finding the key means you've found the row itself, no extra step needed.
- Range queries on the clustered key are extremely efficient, since matching rows are physically adjacent on disk.

**Non-clustered index**

- A separate structure from the table data, storing the indexed column's values along with a pointer, or the clustered key value, back to the actual row.
- A table can have many non-clustered indexes.
- A lookup via a non-clustered index typically requires an extra step, following the pointer back to the actual row, sometimes called a bookmark lookup, unless the index is a covering index that includes all needed columns.

**How the clustered index choice affects schema design**

- Choosing a monotonically increasing key, like an auto incrementing integer, as the clustered index means new rows are always appended at the end physically, avoiding page splits and fragmentation.
- Choosing a naturally random key, like a UUID, as the clustered index causes new rows to be inserted at random points throughout the physical storage, causing frequent page splits, fragmentation, and slower writes over time, which is a well known pitfall.
- Because the clustered key is often embedded in every non-clustered index as the pointer back to the row, a large or frequently changing clustered key makes every non-clustered index larger and more expensive to maintain, so it's worth keeping the clustered key small and stable.

**Interview Answer:**

"A clustered index defines the physical order rows are stored in on disk, so a table can only have one, and looking up by that key is as fast as it gets since the data is right there. A non-clustered index is a separate lookup structure that points back to the row, and you can have many of them, but a lookup often needs an extra step to fetch the actual row unless it's a covering index. The choice of clustered key matters a lot for schema design, a sequential key like an auto incrementing ID keeps inserts appending cleanly at the end, while a random key like a UUID causes constant page splits and fragmentation, and since non-clustered indexes usually embed the clustered key internally, keeping that key small and stable also keeps every other index cheaper to maintain."

---

**If you remember nothing else from this chapter:** *almost every "the database is slow" story turns out to be a missing index or a query nobody ever ran EXPLAIN on.*

# Chapter 7: Advanced Database Concepts: Indexing, Replication & Sharding

Once a system needs to scale past one machine, database internals stop being optional knowledge. Use this chapter to practice reasoning about indexing trade-offs, replication lag, and sharding strategy from first principles, not just repeating the vocabulary.

**War Story:** *A shard key chosen for convenience once put every major customer's data on a single shard, because sharding by signup month meant one viral launch month became a permanent hotspot nobody noticed until re-sharding was painfully expensive.*

**Avoid:** picking a shard key based on what's easy to implement rather than actual query and write patterns

**Aim For:** asking about access patterns before proposing a shard key • raising rebalancing cost as a real, ongoing constraint, not a one-time decision

## Q1. [Junior] What is database replication, and why would you use it?

**Answer:**

**Replication means keeping copies of the same data on multiple machines instead of just one.** For example, a Postgres database might have one primary handling all writes and three read replicas spread across regions, so a user in Singapore reads from a nearby replica instead of a primary sitting in Virginia.

**Why you'd use it**

- Availability: if one machine dies, another copy can take over and serve traffic.
- Read scaling: multiple copies mean multiple machines can answer read queries at the same time.
- Geographic distribution: you can place a copy closer to users in different regions to cut latency.
- Durability: losing a disk doesn't mean losing the data if it's replicated elsewhere.

**How it typically works**

One node, usually called the leader or primary, accepts writes. It then streams those changes to one or more followers (replicas), which apply them and can serve read traffic. The core trade-off is that keeping copies in sync takes time and coordination, which is where most of the interesting problems in this chapter come from.

**Interview Answer:**

"Replication is about storing multiple copies of the same data on different machines so a single failure doesn't take the system down and so read traffic can be spread across more than one server. Typically one node is the leader that accepts writes, and it streams those changes to follower replicas. The benefit is availability, durability, and read scalability, but the cost is that keeping replicas in sync introduces lag and consistency trade-offs you have to design around."

## Q2. [Junior] What's the difference between a leader-follower replication setup and just running multiple independent database instances?

**Answer:**

**The key difference is that leader-follower replicas are copies of the same data, kept in sync automatically, while independent instances are separate databases with no relationship to each other.** For example, three followers replicating from one leader all eventually show the same balance for a given account, while three independent, unconnected databases could each show a completely different or missing balance for that same account.

**Leader-follower replication**

- All followers eventually hold the same data as the leader.
- Writes go to the leader, and it propagates changes to followers.
- Reads can be served from any follower (with some staleness), and the system stays coherent as one logical dataset.

**Independent instances**

- Each instance has its own data, its own writes, its own state.
- Nothing keeps them in sync; if you wrote a row to instance A, instance B has no idea it exists.
- This is what you'd get if you just spun up multiple database servers without setting up replication between them.

**Interview Answer:**
"Leader-follower replication keeps multiple nodes holding copies of the same logical dataset, synchronized through a change stream from the leader. Independent instances are just separate databases with no data relationship at all, so a write to one has no effect on the other. Replication gives you redundancy and read scaling for the same data. Independent instances are more like sharding or just running unrelated services, they don't help you scale or protect a single dataset."

---

## Q3. [Junior] What is replication lag, and why does it happen?

**Answer:**

**Replication lag is the delay between when a write is committed on the leader and when that same write becomes visible on a follower.** For example, a user updates their profile photo, and if they refresh the page a moment later and get routed to a follower that hasn't caught up yet, they might briefly see their old photo.

**Why it happens**

- Network latency: changes have to travel from the leader to each follower, which takes time, especially across regions.
- Follower load: if a follower is busy processing its own reads or is under-resourced, it falls behind applying incoming changes.
- Large transactions or bursts of writes: a follower can only apply changes as fast as it can process them, so a spike in write volume can create a backlog.
- Asynchronous replication: most systems don't wait for followers to confirm before telling the client the write succeeded, so there's always some window where followers are behind.

**Why it matters**

A user can write data, immediately read it back from a lagging replica, and see stale or missing data. This is the root cause behind a lot of confusing "the data disappeared" bug reports.

**Interview Answer:**

"Replication lag is the gap between a write landing on the leader and that write showing up on a follower. It happens because propagating changes takes network time, followers can fall behind under load, and most systems use asynchronous replication so the leader doesn't wait for followers to catch up before acknowledging the write. The practical impact is that a read from a follower right after a write can return stale data, which is why read-after-write consistency has to be designed for explicitly."

---

## Q4. [Junior] What is an index, and why does adding one speed up reads?

**Answer:**

An index is a separate data structure that stores a sorted or otherwise organized reference to rows, so the database can find matching rows without scanning the entire table.

**Without an index**

- The database has to perform a full table scan, checking every row to see if it matches your query's filter.
- This is fine for a table with a thousand rows, but on a table with a hundred million rows it's extremely slow.

**With an index**

- The database can jump almost directly to the rows that match, similar to using the index at the back of a textbook instead of reading every page.
- Most indexes are built as B-trees, which keep data sorted and let the database narrow down a search in logarithmic time instead of scanning linearly.

**The trade-off**

**Indexes speed up reads but cost extra storage and slow down writes, since every insert, update, or delete has to also update the index.** That trade-off is a recurring theme in this chapter.

**Interview Answer:**
"An index is an auxiliary structure, most commonly a B-tree, that lets the database locate rows matching a condition without scanning the whole table. Instead of checking every row, it can navigate to the right spot directly, similar to using a book's index instead of reading page by page. The cost is that every write to the table also has to update the index, so indexes are a deliberate trade-off between read speed and write overhead and storage."

---

## Q5. [Junior] You have a table with 100 million rows and no index on a column you filter by often. What happens when you query it?

**Answer:**

Without an index on that column, the database has to do a full table scan: it reads every one of the 100 million rows, checks whether it matches your filter, and only then decides whether to include it in the result.

**Why this is slow**

- The database can't skip any rows, it has no way to know where the matching ones are without checking each one.
- On a large table this means reading a huge amount of data off disk (or at best, out of memory) for every single query, even if only a handful of rows match.

- Query latency grows roughly linearly with table size, so as the table keeps growing, this query keeps getting slower.

**What to do about it**

Add an index on that column. **The database can then use the index to jump straight to the matching rows instead of scanning everything.** You'd typically confirm the problem first with `EXPLAIN` (or your database's query plan tool) to see that it's doing a sequential scan, then add the index and re-check the plan to confirm it switches to an index scan.

**Interview Answer:**
"With no index on that column, the database is forced into a full table scan, reading all 100 million rows and checking each one against the filter, even if only a few rows actually match. That's slow and gets slower as the table grows. I'd confirm this with an EXPLAIN plan showing a sequential scan, then add an index on that column so the database can jump to matching rows directly, and verify the plan now shows an index scan instead."

---

## Q6. [Junior] What is sharding, and how is it different from replication?

**Answer:**

Sharding means splitting your data across multiple machines, where each machine holds a different subset of the data. Replication means putting the same data on multiple machines.

**Sharding**

- Each shard holds a portion of the total dataset, for example users A through M on shard 1 and N through Z on shard 2.
- It scales both storage and write throughput, because no single machine has to hold or process all the data.
- It adds complexity: queries that need data from multiple shards, and referential integrity across shards, get harder.

**Replication**

- Every replica holds the full dataset (or a full copy of a shard).
- It scales read throughput and improves availability, but doesn't help you store more data than a single machine can hold.

**How they combine**

**In practice, large systems use both: the dataset is split into shards, and each shard is itself replicated for availability and read scaling.**

**Interview Answer:**
"Sharding splits your dataset across multiple machines so each one only holds a portion of the data, which is how you scale beyond what a single machine can store or handle for writes. Replication instead copies the same data across multiple machines, which scales reads and improves availability but not total capacity. In a real large-scale system you use both together, sharding the dataset and then replicating each shard for redundancy."

## Q7. [Mid] Walk me through what happens when a client writes data to a leader-follower replicated database, from the write itself to when it's visible on a read replica.

**Answer:**

**Step by step**

1. The client sends a write request, which is routed to the leader (followers typically reject writes).
2. The leader validates and applies the write to its own storage, recording it in a write-ahead log or equivalent change log.
3. The leader acknowledges the write to the client, in an asynchronous setup, this can happen before any follower has seen the change.
4. The leader streams the log entry to its followers, either by shipping the write-ahead log or by sending logical change events.
5. Each follower applies the change to its own copy of the data, in the same order it was applied on the leader.
6. Once applied, that data is now visible to reads served from that follower.

**Where delay comes from**

Steps 4 and 5 aren't instantaneous. Network transfer, follower processing speed, and follower load all determine how much lag exists between step 3 (client sees success) and step 6 (a replica read reflects it).

**Why order matters**

**Followers must apply changes in the same order the leader committed them, otherwise you can end up with inconsistent state, like an update being applied before the insert it depends on.**

**Interview Answer:**
"The client's write goes to the leader, which applies it locally and appends it to its write-ahead log. In an asynchronous setup, the leader acknowledges success to the client right away, without waiting on followers. It then streams that log entry to each follower, which applies it in the same order the leader did. Only once a given follower has applied the change does a read against that follower reflect it, and the time between the leader's acknowledgment and that follower catching up is exactly what we call replication lag."

---

## Q8. [Mid] Compare synchronous and asynchronous replication. Which would you choose for a banking system versus a social media feed?

**Answer:**

**Synchronous replication**

- The leader waits for one or more followers to confirm they've applied the write before acknowledging success to the client.
- Pros: strong durability guarantee, if the leader dies immediately after acknowledging, the data is safe on the confirmed follower.
- Cons: write latency increases (you're waiting on a network round trip), and if the synchronous follower is unreachable, writes can stall entirely.

**Asynchronous replication**

- The leader acknowledges the write as soon as it's applied locally, without waiting for followers.
- Pros: low, predictable write latency, and followers being slow or down doesn't block writes.
- Cons: if the leader fails before followers catch up, those last writes can be lost.

**Options**

- Fully synchronous to all replicas: safest, slowest, least available.
- Fully asynchronous: fastest, least safe.
- Semi-synchronous (wait for at least one follower, async to the rest): a common middle ground.

**Best Choice**

For the banking system: semi-synchronous or fully synchronous replication to at least one nearby replica. For the social media feed: asynchronous replication.

**Why?**

**A banking system cares more about not losing a confirmed transaction than about shaving a few milliseconds off write latency, losing money is unacceptable.** A social media feed can tolerate losing the last few seconds of a post or like in the rare event of a leader crash, and it benefits far more from low write latency and high availability than from strict durability guarantees.

**Interview Answer:**

"Synchronous replication waits for a follower to confirm before acknowledging the write, giving strong durability at the cost of latency and availability if that follower is slow or down. Asynchronous replication acknowledges immediately and ships changes in the background, which is fast but risks losing the most recent writes if the leader crashes before followers catch up. For a banking system I'd want at least semi-synchronous replication, because losing a confirmed transaction is unacceptable. For a social media feed, asynchronous is the right call, since losing a few seconds of posts in a rare crash is a fine trade for lower latency and higher write availability."

---

## Q9. [Mid] Explain how a B-tree index works and why databases favor it over other structures for general-purpose indexing.

**Answer:**

**Structure**

**A B-tree is a balanced, sorted tree structure.** Each node holds multiple keys and pointers to child nodes, and the tree stays balanced so every leaf is roughly the same distance from the root. Data is kept in sorted order across the leaves.

**How a lookup works**

1. Start at the root node, which contains a range of key values and pointers to child nodes.
2. Compare your search key against the values in the node to decide which child pointer to follow.
3. Repeat down the tree until you reach a leaf node, which points to the actual row (or contains the row itself, depending on the storage engine).
4. Because the tree is balanced and shallow (a few levels can index millions of rows), this takes only a handful of comparisons.

**Why it's the default choice**

- It supports exact match lookups, but also range queries (find everything between X and Y), because data is stored in sorted order.
- It supports ordered scans, which is useful for `ORDER BY` and pagination.
- It stays balanced automatically as data is inserted or deleted, keeping lookup time predictable.
- It performs well on disk, because B-trees are designed around block-based reads, each node maps naturally to a disk page.

**Interview Answer:**
"A B-tree is a balanced, sorted tree where each node holds keys and pointers to children, and every leaf sits at the same depth. Looking something up means comparing your key against each node's values and following the right pointer down until you hit a leaf, which takes only a few steps even on huge tables because the tree is shallow and balanced. Databases default to B-trees because they handle exact matches, range queries, and sorted scans well, and they're structured around disk pages, which makes them efficient for on-disk storage."

---

## Q10. [Mid] When would you use a hash index instead of a B-tree index, and what can't a hash index do that a B-tree can?

**Answer:**

**How a hash index works**

It runs the indexed value through a hash function to compute a bucket location, then stores a pointer to the row in that bucket. Looking up an exact value means hashing it and going straight to the bucket, which is extremely fast, often faster than a B-tree lookup for pure equality checks.

**When to use it**

- When your access pattern is exclusively exact-match lookups, like looking up a session by session ID or a cache key.
- When you don't need sorted output or range queries on that column.

**What it can't do**

- Range queries (`WHERE value BETWEEN X AND Y`) don't work, because hashing destroys the ordering between values, similar values can land in completely unrelated buckets.
- Sorting by the indexed column can't use the index, since there's no order preserved.
- Prefix matching (`LIKE 'abc%'`) doesn't work either, for the same reason.

**Options**

- Hash index: fastest for pure equality lookups, no range or sort support.
- B-tree index: slightly slower for pure equality, but supports ranges, sorting, and prefix matches.

**Best Choice**

**Use a hash index only when you're certain the access pattern is equality-only and will stay that way.** Otherwise, default to B-tree.

**Why?**

Most real applications eventually need range queries or sorting on a column they thought was equality-only, and B-tree's small overhead on exact matches is a much safer general-purpose trade-off than losing range query capability entirely.

**Interview Answer:**
"A hash index hashes the key to a bucket and jumps straight there, which makes exact-match lookups very fast, but it destroys ordering, so it can't support range queries, sorting, or prefix matches on that column. I'd reach for a hash index only for a narrow case like a pure key-value lookup, something like a session store, where I'm confident I'll never need a range scan. For anything more general purpose, I default to a B-tree, since the small overhead on equality lookups buys you range and sort support you'll likely need eventually."

---

## Q11. [Mid] What is a composite index, and how does column order in the index matter?

**Answer:**

A composite index (also called a multi-column index) is a single index built across two or more columns, storing entries sorted first by the first column, then by the second column within each value of the first, and so on.

**Why order matters**

- The index is only useful as a sorted structure on its leading column(s) in that specific order. A query that filters only on the second column of the index generally can't use the index efficiently, because the data isn't sorted by that column on its own.
- A common rule of thumb: put the column used for equality filters first, and the column used for range filters or sorting last.
- Example: an index on `(customer_id, created_at)` efficiently supports "find all orders for customer_id = 5 sorted by created_at" because within each customer_id, rows are sorted by created_at. The same index does not efficiently support "find all orders created_at > X" for an arbitrary customer, because that query isn't anchored on the leading column.

**Practical implication**

**Composite index column order should be chosen based on your actual query patterns, not the order columns happen to appear in the table or in a `WHERE` clause.** Getting it backwards means the database falls back to scanning far more of the index than necessary, or ignoring it entirely.

**Interview Answer:**
"A composite index covers multiple columns, sorted by the first column, then the second within each value of the first, and so on. The order of columns matters a lot, because the index is only efficiently searchable as a sorted structure starting from its leading column. If I put the equality-filtered column first and the range or sort column second, a query can jump to the right spot and then scan in order. Get the order backwards and the database can't use the index efficiently, even though it technically covers the right columns."

---

## Q12. [Mid] You added an index to speed up a slow query, but now writes to that table are noticeably slower. Explain why.

**Answer:**

**Why this happens**

**Every index you add has to be updated on every insert, update, or delete that touches the indexed column.** The database isn't just writing the row anymore, it's also:

- Finding the right place in the index's B-tree for the new or changed value.
- Inserting the new entry, which can trigger page splits if the tree node is full.

- Removing or updating the old entry on an update or delete.
- Doing this for every index on the table, so a table with five indexes pays this cost five times per write.

**Why it's especially noticeable at scale**

- On a large table, index updates increasingly involve random disk access rather than sequential writes, which is slower.
- High write throughput tables amplify this cost, since every one of those writes now fans out into multiple index updates.
- If the index is on a column with high write frequency and low selectivity (few distinct values), the overhead is paid often for relatively little read benefit.

**What to do about it**

Audit which indexes are actually used by your query patterns (most databases can report unused indexes), drop ones that aren't earning their keep, and consider whether a covering index or a narrower index would achieve the read speedup with less write overhead.

**Interview Answer:**
"Indexes aren't free, every insert, update, or delete has to also update each index on that table, which means finding the right spot in the index structure, inserting or removing an entry, and potentially splitting pages. If the table has several indexes, or is high write volume, that overhead adds up and shows up as slower writes. The fix isn't to avoid indexes, it's to be deliberate: keep the indexes that serve real query patterns, drop ones that aren't used, and periodically check whether an index is pulling its weight relative to its write cost."

---

## Q13. [Mid] Compare range-based sharding and hash-based sharding. Which would you pick for a table of user events keyed by timestamp?

**Answer:**

**Range-based sharding**

- Data is partitioned by ranges of the shard key, for example events from January go to shard 1, February to shard 2.
- Pros: range queries (like "give me all events from last week") stay within a small number of shards, which is efficient.
- Cons: prone to hot shards, if the shard key correlates with time and all recent writes go to the newest range, one shard takes all the write traffic while older shards sit idle.

**Hash-based sharding**

- The shard key is hashed, and the hash value determines which shard the data lands on.
- Pros: write traffic spreads evenly across shards, because hashing scrambles the natural ordering, avoiding hot spots.
- Cons: range queries become expensive, since a range of original key values is scattered across many shards, you can't query "last week's events" without hitting every shard.

**Best Choice**

For a table of user events keyed by timestamp, use a composite approach: shard by hash of user ID (not by timestamp directly), or use range sharding only if the shard key includes something other than raw time, like `hash(user_id)` combined with a time bucket.

**Why?**

Sharding directly by timestamp is close to a textbook example of a hot shard: since all new events have the current or near-current timestamp, every write in the system lands on whichever shard owns "now," while all other shards are cold. **Hashing by user ID spreads writes evenly across shards and still lets you efficiently query "this user's events over time" by scanning within one shard's timestamp range.**

**Interview Answer:**

"Range sharding keeps values in contiguous ranges together, which is great for range queries but risks a hot shard when the key correlates with time, because every current write lands on the same shard. Hash sharding distributes writes evenly by hashing the key, but it scatters ranges across shards, making range scans expensive. For timestamped user events, I would not shard directly on timestamp, that guarantees a hot shard on 'now'. I'd shard by hash of user ID instead, which spreads write load evenly and still lets me efficiently query one user's timeline within a shard."

---

## Q14. [Mid] What is directory-based sharding, and when would you reach for it over range or hash sharding?

**Answer:**

**How it works**

**Directory-based sharding uses an explicit lookup service (a directory or mapping table) that records which shard each key, or each range of keys, lives on.** Instead of computing the shard from a formula (a hash or a range boundary), the application asks the directory "which shard has this key" and gets an answer.

**Why you'd want this**

- Flexibility: you can move individual keys or groups of keys between shards without changing a formula, just update the directory entry.
- Uneven distribution support: you can deliberately place a very large or very hot tenant on its own dedicated shard, something a pure hash function can't express.
- Easier rebalancing: since the mapping is just data, migrating a subset of keys to a new shard is a targeted update, not a system-wide recomputation.

**Trade-offs**

- The directory itself becomes a critical piece of infrastructure, it needs to be highly available and fast, since every request depends on it.
- It adds a lookup hop (though this is usually cached aggressively) compared to computing the shard directly from a hash.

**Options**

- Range sharding: simple, good for range queries, prone to hot spots.
- Hash sharding: simple, spreads load evenly, poor for range queries and rebalancing.
- Directory-based: most flexible, best for uneven or evolving workloads, at the cost of operating another critical service.

**Best Choice**

Directory-based sharding when you have multi-tenant workloads with wildly uneven tenant sizes, like a SaaS platform where one enterprise customer generates as much traffic as a thousand small ones.

**Why?**

A hash function can't give one tenant its own dedicated shard or split a single oversized tenant across multiple shards, but a directory can express exactly that kind of custom placement, and it makes future rebalancing a data change instead of a re-architecture.

**Interview Answer:**

"Directory-based sharding uses an explicit mapping service that tracks which shard owns which key or key range, instead of computing it from a hash or fixed boundaries. That gives you a lot of flexibility, you can move specific keys between shards, or give one very large tenant a dedicated shard, which you can't express with a pure hash function. The cost is that the directory itself becomes critical infrastructure that needs to be fast and highly available. I'd reach for it in multi-tenant systems with very uneven tenant sizes, where hash or range sharding alone can't express the placement you actually need."

---

## Q15. [Mid] Your application reads its own write immediately after writing it, but the read goes to a replica that hasn't caught up yet, and the user sees stale data. How do you fix this?

**Answer:**

**This is the classic read-after-write (also called read-your-writes) consistency problem, caused by routing a read to a lagging replica right after a write went to the leader.**

**Options to fix it**

- Read your own writes from the leader: for a short window after a user writes something, route that specific user's reads to the leader instead of a replica.
- Track a version or timestamp: after writing, remember the write's log position or timestamp, and only serve reads from a replica once it has confirmed it's caught up to at least that point.
- Sticky sessions to a specific replica: route all of a user's reads (for some period) to a single replica, and have the application wait for that replica to catch up before returning from the write, so at least that replica is guaranteed current for this user.
- Reduce lag itself: use synchronous or semi-synchronous replication for the follower serving that read, so lag approaches zero.

**Best Choice**

Route reads-after-writes to the leader for a short, user-scoped window (often the simplest to implement), and only use replicas for reads that aren't immediately following a write from that same user.

**Why?**

It's the least invasive change (no need to track versions everywhere), directly targets the exact scenario causing complaints, and doesn't sacrifice the read scaling benefit of replicas for the vast majority of reads that aren't immediately after a write.

**Interview Answer:**

"This is a read-after-write consistency issue, caused by a read landing on a replica that hasn't applied the write yet. The simplest fix is to route a user's reads to the leader for a short window right after they write something, then fall back to replicas afterward. A more general fix is to track the write's log position and only serve replica reads once that replica has caught up to it. I'd start with the leader-routing approach since

it's simple and targets exactly the scenario causing the bug, without giving up replica read scaling for everything else."

---

## Q16. [Mid] You're scaling read-heavy traffic with read replicas. Walk me through how you'd route reads and writes, and what could go wrong.

**Answer:**

**Routing setup**

1. **All writes go to the leader.** This is enforced either by the driver/connection pool knowing which endpoint is the leader, or by a proxy layer that routes based on query type.
2. Reads are distributed across one or more followers, often round-robin or load-based, to spread query load.
3. The application (or a smart proxy) needs a way to distinguish "this read can tolerate staleness" from "this read needs the latest data," since not every read should go to a replica.

**What can go wrong**

- Stale reads: a replica lagging behind the leader can return outdated data, which surprises users who just wrote something (see read-after-write consistency).
- Replica failure: if a follower goes down, the routing layer needs to detect it and stop sending it traffic, otherwise requests time out or fail.
- Uneven replica lag: some replicas may lag more than others (due to network distance or load), so "any replica" isn't uniformly safe, you may need to pick the least-lagged one for certain reads.
- Write amplification on the leader from replication itself: as you add more followers, the leader has to stream changes to each of them, which can itself become a bottleneck at very high follower counts.
- Failover complexity: if the leader dies, promoting a follower to leader needs to happen carefully to avoid data loss or split-brain (two nodes both thinking they're leader).

**Interview Answer:**
"I'd send all writes to the leader and spread reads across the follower replicas, using a proxy or driver-level routing so the application doesn't have to know the topology directly. The main things that can go wrong are stale reads from lagging replicas, especially right after a write from the same user, replica failures that need to be detected and routed around, and lag being uneven across replicas so 'any replica' isn't always safe. I'd also plan for leader failover carefully, since promoting the wrong replica or having two nodes think they're leader can cause real data problems."

---

## Q17. [Mid] What is a hot shard, and give a concrete example of a sharding key choice that would create one?

**Answer:**

A hot shard (or hot partition) is a shard that receives a disproportionate amount of traffic or data relative to other shards, becoming a bottleneck even though the overall system has plenty of spare capacity elsewhere.

**Concrete example**

Sharding a social media platform's posts table by the first letter of the username. Usernames aren't evenly distributed across letters (a lot cluster around common starting letters), and worse, if you shard by

something correlated with popularity, like sharding celebrity accounts alongside regular ones by user ID range, one celebrity with millions of followers can generate far more read and write traffic than an entire shard's worth of regular users combined.

**Another common example**

Sharding by day (range sharding on a timestamp) for a high write-volume events table. All of today's writes go to today's shard, so that one shard absorbs the entire system's current write load while yesterday's and last week's shards sit idle.

**Why it matters**

**A hot shard defeats the purpose of sharding, you added machines to spread load, but load isn't actually spread, it's concentrated.** The rest of the cluster can be underutilized while the hot shard is maxed out on CPU, IO, or connections, degrading latency for anyone whose data happens to live there.

**Interview Answer:**

"A hot shard is a partition that gets far more traffic than the others, so it becomes a bottleneck even though the cluster overall has capacity to spare. A classic cause is sharding by something correlated with popularity or time, like sharding a celebrity's posts the same way as everyone else's, or range-sharding events by timestamp so all of today's writes hit one shard. The fix is usually choosing a shard key that distributes load more evenly, like hashing a user ID, or detecting outlier keys and giving them dedicated capacity."

---

## Q18. [Mid] What is multi-leader replication, and what new problem does it introduce that leader-follower doesn't have?

**Answer:**

**What it is**

Multi-leader replication allows more than one node to accept writes, with each leader replicating its changes to the other leaders (and often to followers beneath each leader too). It's common in setups spanning multiple data centers, where each region has its own local leader so writes don't have to cross the globe to be accepted.

**Why it's appealing**

- Writes can be accepted locally in each region, giving lower write latency for geographically distributed users.
- One region's leader going down doesn't stop writes in other regions.

**The new problem: write conflicts**

Because two leaders can accept writes to the same piece of data at nearly the same time, independently, before either knows about the other's change, you can end up with conflicting versions of the same record. **Leader-follower doesn't have this problem because there's only ever one place writes can happen, so there's a single, unambiguous order of operations.**

**Example**

A user updates their profile name in the US region while, at nearly the same moment, an admin updates the same profile's status in the EU region. Both leaders accept their write locally, then replicate to each other, at which point the system has to decide how to reconcile them.

**Interview Answer:**

"Multi-leader replication lets more than one node accept writes, typically one leader per data center, so writes don't have to travel across the globe to a single leader. That's great for write latency and regional availability, but it introduces write conflicts: two leaders can accept different writes to the same record at nearly the same time, with neither aware of the other, and now the system has to reconcile them after the fact. Leader-follower avoids this entirely because there's only one place writes happen, so ordering is unambiguous."

---

## Q19. [Senior] How do you resolve write conflicts in multi-leader replication? Walk through a couple of strategies.

**Answer:**

**Last-write-wins (LWW)**

- Each write gets a timestamp, and when a conflict is detected, the write with the later timestamp wins, the other is discarded.
- Pros: simple to implement, no application logic needed.
- Cons: silently loses data, and clock skew between nodes can make "later" unreliable unless you use something like a hybrid logical clock, a clock that blends real time with a counter so ordering stays correct even if machines disagree slightly on the time.

**Application-level merge**

- The conflict is surfaced to the application (or stored with both versions), and custom logic decides how to combine them, for example merging two edits to different fields of the same record, or taking the union of two sets.
- Pros: no silent data loss, can produce a semantically correct result.
- Cons: requires writing and maintaining conflict resolution logic for every data type, and can get complex fast.

**Conflict-free replicated data types (CRDTs)**

- Use data structures specifically designed so that concurrent updates can always be merged deterministically without conflict, like a grow-only counter or an observed-remove set.
- Pros: mathematically guaranteed to converge to the same result on all replicas without manual conflict resolution.
- Cons: only works for data types that have a CRDT formulation, doesn't generalize to arbitrary application state.

**Versioning with vector clocks**

- Track causal history per write, so the system can detect whether two writes were truly concurrent (real conflict) or one causally followed the other (no conflict, just apply in order), and surface true conflicts to the application or client.

**Best Choice**

For most business applications, application-level merge for critical fields, combined with last-write-wins for low-stakes fields, is the pragmatic default. Reach for CRDTs when the data naturally fits one, like counters or collaborative sets.

**Why?**

LWW alone silently loses data, which is unacceptable for anything users care about, but building custom merge logic for every field is overkill. **Mixing strategies by field importance gets you correctness where it matters and simplicity everywhere else.**

**Interview Answer:**
"There's a spectrum of approaches. Last-write-wins is simplest, timestamp every write and let the latest one win, but it silently discards data and is sensitive to clock skew. Application-level merge resolves conflicts with custom logic, like merging edits to different fields, which avoids data loss but takes real engineering effort per data type. CRDTs are data structures designed to merge automatically and deterministically, great for counters or sets, but they don't generalize to arbitrary records. In practice I'd use LWW for low-stakes fields and application-level merge or CRDTs for anything where silently losing a write would actually hurt the business."

---

## Q20. [Senior] Design a resharding plan for a hash-sharded database that's outgrown its current shard count. What would break if you did it naively?

**Answer:**

**What breaks with naive resharding**

**If you simply change the number of shards and recompute `hash(key) % new_shard_count` for every key, almost every key maps to a different shard than before.** That means you'd have to move nearly 100% of your data to new locations, causing a massive, disruptive migration and likely downtime or severe performance degradation while it happens.

**A better plan**

1. Adopt consistent hashing (or a similar scheme) before you need to reshard, if you haven't already, so that adding a shard only requires moving a small, predictable fraction of keys, not nearly all of them.
2. Pre-split into virtual shards: run many more logical shard partitions (say, 4096) than physical machines from day one, and map several virtual shards to each physical machine. Resharding then means moving whole virtual shards between machines, not recomputing a hash function.
3. Migrate incrementally: move one virtual shard (or a small batch) at a time, verify data integrity, then update the routing/directory layer to point at the new location, rather than cutting everything over at once.
4. Dual-write or backfill plus catch-up: for each shard being moved, backfill the new location from a snapshot, then stream ongoing writes (via change data capture) until the new copy is caught up, then atomically flip routing.
5. Keep the old location readable during a grace period, in case of routing bugs, before decommissioning it.
6. Monitor load per shard throughout, so you're moving virtual shards to genuinely balance load, not just splitting arbitrarily.

**Interview Answer:**
"Naively changing the shard count and recomputing hash of key mod new count reshuffles almost every key, forcing a massive all-at-once data migration. I'd avoid that by using consistent hashing or pre-splitting into many more virtual shards than physical machines up front, so resharding just means moving a subset of virtual shards to new machines. The migration itself would be incremental: backfill a shard to its new home, stream ongoing changes with change data capture until it's caught up, then atomically update the routing layer, one shard or small batch at a time, with monitoring to confirm load actually balances out."

## Q21. [Senior] Explain consistent hashing and why it matters for resharding.

**Answer:**

### The problem it solves

With simple modulo hashing (`hash(key) % N`), changing N, the number of shards, changes the mapping for nearly every key, because the modulo operation depends on the total count. **That makes adding or removing a shard extremely expensive.**

### How consistent hashing works

1. Map both shards and keys onto the same circular hash space (a ring), using a hash function.
2. Each key is assigned to the first shard found by walking clockwise around the ring from the key's position.
3. When you add a new shard, it takes a position on the ring and only "steals" the keys between itself and the previous shard going backward, it doesn't affect the mapping of keys anywhere else on the ring.
4. When a shard is removed, only its keys need to move, to the next shard clockwise, everyone else is unaffected.

### Why it matters for resharding

- Adding or removing a shard only requires moving roughly `1/N` of the keys (where N is the new shard count), not nearly all of them.
- This makes horizontal scaling incremental and low-disruption instead of an all-or-nothing migration event.

### A refinement: virtual nodes

Real systems assign each physical shard multiple positions on the ring (virtual nodes), which smooths out load distribution, since a single physical shard landing in an unlucky spot on the ring could otherwise absorb a disproportionate range of keys.

**Interview Answer:**

"Consistent hashing places both shards and keys on a hash ring, and each key belongs to the first shard clockwise from its position. The advantage over plain modulo hashing is that adding or removing a shard only reassigns the keys immediately adjacent to that change on the ring, not almost every key in the system. That's exactly what makes resharding practical: you move a small, bounded fraction of data instead of nearly everything. Real implementations also use virtual nodes, giving each physical shard several ring positions, so load stays balanced even with a small number of shards."

---

## Q22. [Senior] You have a covering index that includes every column a common query needs. Explain the trade-off versus a regular index, and when it's worth it.

**Answer:**

### What a covering index is

A covering index includes not just the column(s) used to filter or sort, but also every other column the query selects. This means the database can satisfy the entire query from the index alone, without a second lookup back to the actual table row (often called avoiding a "table lookup" or, in some engines, avoiding hitting the heap).

### Why it's faster for reads

- A regular index gets you to the right index entry, but then the database usually still has to fetch the full row from the table to get columns not in the index, an extra IO operation per matching row.
- A covering index skips that second fetch entirely, since everything needed is already sitting right there in the index.

**The trade-off**

- Storage: the index is larger, since it duplicates more column data, not just the key columns.
- Write cost: every insert or update to any of the included columns now has to update this wider index, which is more expensive per write than a narrow index.
- Maintenance: as query patterns evolve and new columns get selected, a covering index can silently stop covering and need to be redefined.

**When it's worth it**

For a specific, high-frequency, performance-critical query, especially read-heavy tables where the same narrow set of columns is repeatedly selected together, like a dashboard query that always selects the same five columns for a given date range.

**Options**

- Regular index: smaller, cheaper writes, extra table lookup per read.
- Covering index: larger, more expensive writes, no extra lookup, fastest possible read path.

**Best Choice**

**Covering index, but scoped tightly to one or two genuinely hot queries, not applied broadly.**

**Why?**

The write and storage cost only pays off when the read savings are realized often enough and matter enough (like a query hit on every page load), applying it everywhere would bloat storage and slow down writes for marginal benefit on rarely-run queries.

**Interview Answer:**

"A covering index includes every column a query needs, so the database can answer the query straight from the index without a second fetch back to the table row, which is normally required with a regular index. That makes reads noticeably faster, but the index is bigger and every write touching any included column now updates a wider structure, so it costs more storage and write throughput. I'd only add one for a specific, frequently run, performance-sensitive query, like a hot dashboard query, rather than broadly, since the write and storage overhead only pays for itself when the read savings happen often enough to matter."

---

## Q23. [Senior] A social media app shards users by hash of user ID, but one shard is consistently at 90% CPU while others sit at 20%. Walk me through how you'd diagnose and fix this.

**Answer:**

**Diagnose first**

1. Check whether the hot shard has more data than others (a size imbalance) or the same amount of data but more traffic (a load imbalance). These call for different fixes.
2. Look at the query and write patterns hitting that shard specifically: is it a small number of extremely active accounts (a celebrity or viral post), a specific query pattern that's inefficient (missing index,

causing scans), or a background job that happens to be scheduled against that shard's data disproportionately?

3. Check whether hashing is actually distributing keys evenly, a poor hash function, or hashing a low-cardinality field by mistake, can leave one shard with disproportionately more keys even though the intent was uniform distribution.
4. Rule out a purely mechanical cause: is that shard's underlying machine simply weaker or already handling other workloads?

**Likely root cause, given user ID hashing is already in place**

**Since users are hashed (not ranged), an even distribution of accounts is likely, so the more common cause is a small number of extremely hot keys, a celebrity account, a viral post, or a bot generating disproportionate traffic, landing on that shard.**

**Fix options**

- Cache aggressively for the hot keys specifically, so most reads never reach the shard at all.
- Give a small number of identified hot keys their own dedicated shard or replica set (this generally requires directory-based or hybrid sharding to express).
- Add read replicas specifically for that shard, so read traffic (often the bulk of celebrity account traffic) spreads across multiple machines even though writes still go to one place.
- Rate-limit or queue write-heavy abusive traffic if it's bot-driven.

**Interview Answer:**
"First I'd figure out whether it's a data size imbalance or a traffic imbalance, since hashing by user ID should distribute account count evenly, so my first suspicion is a small number of very hot keys, like a celebrity account or a viral post, concentrated on that one shard. I'd confirm by looking at the query and write patterns actually hitting it. If it's a hot key problem, I'd add aggressive caching in front of that data, add read replicas for that specific shard to spread read load, and consider giving genuinely oversized accounts a dedicated shard through a directory-based override, since pure hashing can't express that kind of exception."

---

## Q24. [Senior] Compare quorum-based replication, like Dynamo-style W plus R greater than N, to a fixed leader-follower model. What consistency guarantees does each give you?

**Answer:**

**Leader-follower**

- A single leader accepts all writes and defines a single, unambiguous order of operations.
- Consistency guarantee: strong consistency is achievable for reads against the leader itself, reads from followers may be stale depending on replication mode.
- Availability trade-off: if the leader is unreachable, writes stop (until failover completes), even if followers are healthy.

**Quorum-based (leaderless) replication**

- Any replica can accept a write. A write is considered successful once it's been acknowledged by W replicas out of N total, and a read queries R replicas and returns the most recent version among them.

- If W plus R is greater than N, every read set and every write set is guaranteed to overlap by at least one node, so a read will see at least one copy of the most recent write.
- Consistency guarantee: this gives you a form of strong-ish consistency for a single key (you'll always see the latest write, assuming correct conflict resolution via versioning), but it's not the same as serializability across multiple keys (the guarantee that transactions behave as if they ran one at a time in some order), and concurrent writes to the same key from different nodes still need conflict resolution.
- Availability trade-off: the system can keep accepting writes and serving reads even if some nodes are down, as long as enough replicas (W or R) are reachable, there's no single leader to become a bottleneck or single point of failure for writes.

**Options**

- Leader-follower: simpler consistency reasoning (single order of writes), but the leader is a write availability bottleneck and single point of failure until failover completes.
- Quorum-based: higher write availability and no single point of failure, but requires explicit conflict resolution (versioning, vector clocks, or CRDTs) since concurrent writes can genuinely race.

**Best Choice**

**Leader-follower for systems where a single consistent order of operations matters and brief write unavailability during failover is tolerable (most transactional systems).** Quorum-based for systems that must keep accepting writes even during network partitions or node failures, and where the application can handle eventual conflict resolution.

**Why?**

Leader-follower is easier to reason about and debug because there's one true order of writes, quorum systems trade that simplicity for higher availability under partition, which is the right trade for systems like a shopping cart or a distributed key-value store where "always available for writes" matters more than a strict global order.

**Interview Answer:**

"Leader-follower gives you a single leader defining one unambiguous order of writes, so consistency is easy to reason about, but the leader is a bottleneck and single point of failure for writes until a failover completes. Quorum-based replication, with W plus R greater than N, has any replica accept writes and guarantees read and write sets overlap, so reads see the latest acknowledged write for a given key, without needing a single leader. That buys you higher write availability during node failures or partitions, at the cost of needing explicit conflict resolution for concurrent writes to the same key. I'd pick leader-follower for most transactional systems, and quorum-based for systems where write availability during failures matters more than a strict global order."

---

## Q25. [Senior] Explain the difference between read-after-write consistency, monotonic reads, and consistent prefix reads. Give an example of a bug caused by violating each.

**Answer:**

**Read-after-write consistency**

Guarantees that a user always sees their own writes immediately after making them, even if other users might see stale data briefly.

- Bug example: a user updates their profile picture, refreshes the page, and the read hits a lagging replica, showing the old picture. The user assumes the upload silently failed and re-uploads, creating confusion and duplicate work.

**Monotonic reads**

Guarantees that once a user has seen a piece of data at a certain point in time, they never see an older version of it later, even across multiple reads. It doesn't require seeing the latest data immediately, just that things don't go backward.

- Bug example: a user refreshes a comment thread and sees a new comment. They refresh again moments later, but this time the request is routed to a different, more-lagged replica, and the comment has disappeared. The user thinks their own comment or someone else's got deleted, when really they're just seeing an older snapshot.

**Consistent prefix reads**

Guarantees that if a sequence of writes happened in a certain order, anyone reading those writes sees them in that same order, they don't see effects out of causal sequence, even if they're reading from different partitions or shards.

- Bug example: in a chat app, user A asks "are you free tonight?" and user B replies "yes!". Because the question and answer are stored on different shards with independent replication lag, another user reading the conversation sees B's "yes!" appear before A's question, making the conversation look nonsensical.

**Why this distinction matters**

**Each of these is a separate, independently violatable guarantee.** A system can have monotonic reads without read-after-write consistency (you never go backward, but you might not see your own recent write immediately), and a system can maintain per-key monotonic reads while still failing consistent prefix reads across keys that are causally related but stored separately.

**Interview Answer:**

"Read-after-write consistency means you always see your own writes right away, and violating it looks like a user's own update appearing to vanish after they just made it. Monotonic reads mean once you've seen a value, you never see an older version later, violating it looks like data flickering backward in time across repeated reads, like a comment disappearing on refresh. Consistent prefix reads mean causally related writes are seen in the same order they happened, violating it looks like an answer appearing before its question in a chat log, because the two were stored in different partitions with different lag. They're distinct guarantees, and a system can satisfy one while violating another."

---

## Q26. [Senior] You need to add a new column and backfill an index across a sharded database with 500 shards, live, with zero downtime. Walk me through the plan.

**Answer:**

**Phase 1: Schema change**

1. Add the new column as nullable (or with a default) on every shard. This is a metadata-only change on most databases and should be fast and non-blocking if done correctly, avoid a full table rewrite by using an online schema change tool if the database doesn't support instant column addition natively.
2. **Roll this out shard by shard, or in small batches of shards, monitoring for locking or replication issues on each batch before proceeding, rather than hitting all 500 shards simultaneously.**

**Phase 2: Backfill the data**

1. Write a backfill job that processes rows in small batches (by primary key range) per shard, computing and writing the new column's value, throttled to avoid saturating each shard's IO or replication bandwidth.
2. Run backfill jobs in parallel across shards, but bound the concurrency (for example 20 shards at a time) so you don't create a thundering herd on shared infrastructure like a connection pool proxy layer.
3. Make the backfill idempotent and resumable, since with 500 shards, some batches will fail transiently and need retrying without corrupting state or double-processing.
4. Have the application start dual-writing the new column for all new and updated rows as soon as the column exists, so the backfill only needs to cover historical data, not a moving target.

**Phase 3: Build the index**

1. Once backfill is complete and verified (spot-check row counts and null counts per shard), build the index using an online/concurrent index build feature (like `CREATE INDEX CONCURRENTLY` in Postgres) so it doesn't lock writes, again rolling out shard by shard or in small batches.
2. Monitor replication lag and shard CPU during index builds, since this is IO and CPU intensive, and pause the rollout if any shard shows signs of strain.

**Phase 4: Cutover**

1. Only switch the application to actually query using the new index (or rely on the new column) once every shard has confirmed the index is built and the column is fully backfilled.
2. Keep the ability to roll back at each stage, don't remove any old code path or column until the new one has been running cleanly in production for a reasonable soak period.

**Interview Answer:**

"I'd break it into stages: first add the nullable column shard by shard using an online schema change so it doesn't lock writes, and immediately have the application dual-write the new column going forward so backfill only needs to cover historical rows. Then run an idempotent, batched backfill job per shard, throttled and parallelized across a bounded number of shards at a time, so I don't overwhelm shared infrastructure. Once backfill is verified complete, build the index using an online, concurrent index build, again rolling out gradually and watching replication lag and CPU per shard. Only after every shard confirms the index is built do I cut the application over to actually use it, keeping a rollback path the whole way."

---

## Q27. [Senior] Compare leader-follower replication with failover to leaderless, Dynamo-style replication for an e-commerce shopping cart service. Which would you choose, and why?

**Answer:**

**Leader-follower with failover**

- Single leader accepts writes, followers replicate, and an automated or manual failover promotes a follower if the leader fails.
- Pros: simple consistency model, a clear source of truth, easier to reason about ordering of cart updates.
- Cons: during the failover window, writes to the cart are unavailable. If failover takes even a few seconds, a user trying to add an item during that window sees an error or timeout. Also, if a follower is promoted before fully catching up, recent writes can be lost.

**Leaderless, Dynamo-style replication**

- Any replica can accept a cart write, using a quorum (W and R) to balance consistency and availability, with versioning (vector clocks or similar) to detect concurrent writes to the same cart.
- Pros: no single point of failure for writes, the cart service keeps accepting adds and removes even if individual nodes fail or a network partition isolates part of the cluster.
- Cons: concurrent writes to the same cart, like adding an item from a phone and a laptop tab at nearly the same time, need conflict resolution, since two nodes might accept conflicting versions.

**Best Choice**

**Leaderless, Dynamo-style replication for the shopping cart specifically.**

**Why?**

A shopping cart is a great fit for eventual consistency with merge-based conflict resolution: if two devices add different items concurrently, the correct resolution is usually just the union of both carts, nobody wants "add to cart" to fail because of a leader election in progress. This is actually the classic motivating example from Amazon's own Dynamo paper: cart availability during writes matters more than strict ordering, and conflicts are naturally mergeable (union of item sets) rather than requiring complex reconciliation. A cart merge that occasionally needs the user to review a duplicate quantity is a far better failure mode than a checkout flow that can't add an item because the write path is temporarily down during failover.

**Interview Answer:**

"For a shopping cart, I'd lean toward leaderless, Dynamo-style replication instead of leader-follower with failover. A cart needs to always accept adds and removes, even during node failures or network issues, and leader-follower has an unavoidable unavailability window during failover. Leaderless replication avoids a single point of failure for writes, using quorum reads and writes and versioning to detect concurrent updates. Conflicts, like adding items from two devices at once, are naturally resolved by merging, usually just taking the union of items, which is a far better user experience than a failed add-to-cart request. This is essentially the original use case Amazon built Dynamo for."

---

## Q28. [Mid] This query filters on a column that has an index, but `EXPLAIN ANALYZE` shows a sequential scan instead of an index scan. What could be causing that, and how would you confirm and fix it?

```
CREATE INDEX idx_users_email ON users(email);

EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'jane.doe@example.com';
Seq Scan on users  (cost=0.00..98452.00 rows=1 width=142) (actual time=0.031..312.884
rows=1 loops=1)
  Filter: (email = 'jane.doe@example.com'::text)
  Rows Removed by Filter: 4999999
Planning Time: 0.098 ms
Execution Time: 312.921 ms
```

**Answer:**

An index existing doesn't guarantee the planner will use it, **the planner picks whichever plan it estimates will be cheapest, and several common mistakes make it estimate the index scan as more expensive than a full scan even when that's wrong.**

**A function wrapping the column**

If the actual query (or an ORM-generated one) looks like this instead, the index on the raw column can't help:

```
SELECT * FROM users WHERE LOWER(email) = 'jane.doe@example.com';
```

The index was built on `email`, not on `LOWER(email)`, so the planner has no way to use it for this predicate. The fix is an expression index: `CREATE INDEX idx_users_email_lower ON users(LOWER(email));`, or better, normalize the column on write so you can query it directly.

**Outdated statistics**

If the table was just bulk-loaded or heavily modified and `ANALYZE` hasn't run recently, the planner's row estimates can be badly wrong, causing it to misjudge the cost of an index scan versus a seq scan. Running `ANALYZE users;` and re-checking the plan is the first thing to try.

**Low selectivity**

If the filtered value matches a large fraction of the table (say a status column where 80% of rows are ``active''), a seq scan genuinely can be cheaper than jumping around the index and then fetching each row from the heap. This is correct planner behavior, not a bug.

**Small table**

On a table with only a few hundred or thousand rows, the entire table often fits in a handful of disk pages, so a seq scan and an index scan cost about the same, and the planner reasonably picks the simpler seq scan.

**How to confirm**

Run `EXPLAIN (ANALYZE, BUFFERS)` and compare the estimated row count against the actual row count, a large mismatch points at stale statistics. Check whether the predicate uses a function or type cast on the indexed column. Check the value's selectivity with a quick `SELECT COUNT(*)` for that value versus the total row count.

**Interview Answer:**

"I'd start by checking whether the query is filtering on the raw column or wrapping it in a function like `LOWER()` or a type cast, since that silently defeats a plain index and needs an expression index instead. Next I'd run `ANALYZE` on the table in case statistics are stale, and re-check the plan. If neither explains it, I'd check the selectivity of the filtered value: if it matches a large share of rows, or the table is small enough to fit in a few pages, a seq scan is genuinely cheaper and the planner is right to choose it, not a bug to fix."

---

## Q29. [Mid] You have this composite index, but the query below is still doing a sequential scan. Why doesn't the index help here, and what's the fix?

```
CREATE INDEX idx_orders_customer_status ON orders(customer_id, status);

EXPLAIN ANALYZE
SELECT * FROM orders WHERE status = 'pending';
Seq Scan on orders  (cost=0.00..45210.00 rows=8210 width=96) (actual time=0.045..198.332
rows=8143 loops=1)
  Filter: (status = 'pending'::text)
  Rows Removed by Filter: 1191857
Planning Time: 0.087 ms
Execution Time: 198.702 ms
```

**Answer:**

This is the leftmost prefix rule in action. **A composite index is sorted first by its leading column and only sorted by later columns within each value of the leading column, so it can only be searched efficiently starting from that first column.**

**Why `(customer_id, status)` doesn't help this query**

The index entries are ordered like `(customer_id=1, status=...), (customer_id=1, status=...), (customer_id=2, status=...)`. Within the whole index, `status` values are not sorted, they're only sorted inside each `customer_id` group. A query that filters on `status` alone with no `customer_id` predicate would have to scan the entire index in roughly the same order as scanning the table, so the planner correctly decides a seq scan is simpler and just as fast.

**What does work with this index**

```
SELECT * FROM orders WHERE customer_id = 42 AND status = 'pending';
SELECT * FROM orders WHERE customer_id = 42;
```

Both use the index efficiently because they anchor on the leading column.

**The fix**

- If queries filtering on `status` alone are common, add a separate index on `status`, or swap the column order to `(status, customer_id)` if that pattern is more common than filtering by customer.
- If both access patterns matter, keep two indexes: one for each leading column, understanding that each additional index adds write overhead.
- A composite index does not substitute for an index on the second column alone.

**Interview Answer:**

"A composite index is only sorted as a unit starting from its first column, so `(customer_id, status)` is sorted by customer_id first and status only within each customer_id group. A query that filters on status with no customer_id predicate can't jump to the right spot using that index, since status values are scattered across the whole structure, so the planner falls back to a seq scan. The fix is to add a standalone index on status, or reorder the composite index to `(status, customer_id)` if that's actually the more common query pattern, since column order has to match how you query, not just which columns you query."

---

## Q30. [Mid] Here's a piece of application code that loads a list of orders and then, for each order, fetches its customer separately. What's wrong with this, and how would you fix it?

```
List<Order> orders = db.query(
    "SELECT id, customer_id, total FROM orders WHERE created_at > ?", yesterday);

for (Order order : orders) {
    Customer customer = db.queryOne(
        "SELECT name, email FROM customers WHERE id = ?", order.getCustomerId());
    System.out.println(order.getId() + " " + customer.getName() + " " +
order.getTotal());
}
```

**Answer:**

This is the classic N+1 query problem. **One query fetches the list of N orders, and then the loop issues one additional query per order to fetch its customer, for a total of N+1 round trips to the database instead of two.**

**Why it's expensive**

- Each query, even a fast one, still pays a network round trip to the database. A thousand orders means a thousand and one round trips instead of two.
- The database also loses the chance to optimize this as a single join, it processes 1,000 tiny, disconnected queries instead of one query it can plan holistically.
- This is easy to miss in development with a handful of test rows, and only shows up as a real problem once the order list grows in production.

**How to spot it**

- Turn on query logging (or an ORM's query counter/APM tool) for a request and see an unexpectedly large, repetitive number of near-identical queries, that pattern is the signature of N+1.
- Most ORMs (Django, ActiveRecord, SQLAlchemy) have a lazy-loading default that produces this exact pattern whenever you access a related object inside a loop.

**The fix**

Fetch everything in one query with a join, or batch the second query:

```
SELECT o.id, o.total, c.name, c.email
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.created_at > $1;
```

Or, if a join isn't practical, batch-load: collect all `customer_id`s from the orders first, run a single `WHERE customer_id = ANY($1)` query, and join the results in application memory.

**Interview Answer:**

"This is N+1 queries: one query to get the orders, then one more query per order inside the loop to get its customer, so N orders means N+1 round trips instead of one or two. It's expensive because every query pays a network round trip and the database never gets to optimize this as a single operation, and it's easy to miss until the data volume grows. I'd catch it by looking at query logs or an APM tool for a suspiciously repetitive query pattern, and fix it by rewriting it as a single join, or batching the second query with a `WHERE customer_id = ANY(...)` if a join isn't practical."

---

## Q31. [Senior] Here are two EXPLAIN ANALYZE plans for the same join query, run against a small dataset and then again after the tables grew. What does each plan tell you about the planner's reasoning?

```
SELECT o.id, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'pending';
```

Small dataset (customers: 2,000 rows, orders: 5,000 rows, 40 pending):

```
Nested Loop  (cost=0.29..156.40 rows=40 width=24) (actual time=0.020..0.412 rows=40
loops=1)
  ->  Seq Scan on orders o  (cost=0.00..100.00 rows=40 width=12) (actual
time=0.012..0.198 rows=40 loops=1)
        Filter: (status = 'pending'::text)
  ->  Index Scan using customers_pkey on customers c  (cost=0.29..1.39 rows=1 width=20)
(actual time=0.004..0.004 rows=1 loops=40)
        Index Cond: (id = o.customer_id)
Planning Time: 0.145 ms
Execution Time: 0.461 ms
```

Same query, after both tables grew (customers: 2,000,000 rows, orders: 40,000,000 rows, 380,000 pending):

```
Hash Join  (cost=61245.00..912340.55 rows=380000 width=24) (actual time=402.112..3811.998
rows=380000 loops=1)
  Hash Cond: (o.customer_id = c.id)
  ->  Seq Scan on orders o  (cost=0.00..820000.00 rows=380000 width=12) (actual
time=0.031..2988.204 rows=380000 loops=1)
        Filter: (status = 'pending'::text)
  ->  Hash  (cost=35000.00..35000.00 rows=2000000 width=20) (actual time=398.221..398.221
rows=2000000 loops=1)
        Buckets: 2097152  Batches: 1  Memory Usage: 121344kB
Planning Time: 0.201 ms
Execution Time: 3902.556 ms
```

**Answer:**

Both plans are correct choices for their respective data sizes, the point is understanding why the planner switches.

**Nested Loop (small dataset)**

The planner scans the small set of matching orders (only 40 rows) and, for each one, does a fast indexed lookup into customers. **Nested Loop is cheap when the outer side of the join produces few rows, because the cost is roughly the outer row count multiplied by the cost of one inner lookup, and with only 40 outer rows that stays tiny even though each inner lookup happens 40 separate times.** Notice `loops=40` on the inner index scan, that repetition is exactly what makes Nested Loop expensive once the outer side gets big.

**Hash Join (large dataset)**

Once pending orders grew to 380,000 rows, repeating an indexed lookup 380,000 times would be far more expensive than doing it 40 times. Instead the planner builds an in-memory hash table from the smaller side of the join (customers, keyed by `id`) once, then does a single pass over orders, probing the hash table for each row. This turns what would be 380,000 repeated index lookups into one hash table build plus one sequential probe pass, which scales much better as row counts grow.

**What decides the switch**

- The planner estimates the cost of both strategies using row count estimates and picks whichever is cheaper, this is a cost-based decision, not a hard row-count threshold.
- Nested Loop wins when one side of the join is small (or well-filtered to a small row count) and there's a good index to support fast repeated lookups on the other side.
- Hash Join wins when both sides are large and there's no way to avoid touching most of both tables, building one hash table is cheaper than millions of individual index probes.
- If the build side (customers here) is too large to fit in `work_mem` (Postgres's per-query memory budget for operations like sorts and joins), the hash join spills to disk in batches, which shows up as `Batches > 1` and adds significant IO cost, worth watching for in the plan.

**Interview Answer:**

"The small-data plan uses Nested Loop because only 40 orders match, so doing 40 fast indexed lookups into customers is cheap, that's why the inner index scan shows `loops=40`. Once the data grew and 380,000 orders matched, repeating an indexed lookup that many times would be far more expensive than building a hash table from customers once and probing it in a single pass over orders, which is what the Hash Join plan does. The planner picks between these based on estimated cost from row counts, not a fixed threshold, so I'd read a Nested Loop as a sign of a well-filtered small outer side, and a Hash Join as a sign both sides are large enough that per-row index lookups stop being worth it."

---

## Q32. [Senior] This query used to run in a few milliseconds when the table had 10,000 rows. Now the table has 50 million rows and it takes several seconds, even though the index is still there. What would you investigate?

```
EXPLAIN ANALYZE
SELECT * FROM events
WHERE user_id = 4821
ORDER BY created_at DESC
LIMIT 20;
Limit  (cost=0.56..8452.11 rows=20 width=88) (actual time=1204.552..4890.331 rows=20
loops=1)
  ->  Index Scan Backward using idx_events_created_at on events  (cost=0.56..912004.20
rows=2140 width=88) (actual time=1204.548..4890.301 rows=20 loops=1)
        Filter: (user_id = 4821)
        Rows Removed by Filter: 3812940
```

```
Planning Time: 0.201 ms
Execution Time: 4890.402 ms
```

**Answer:**

The index exists, but it's the wrong index for this query, and that's masked by the `LIMIT` making it look like it should be fast. Here's what to check, roughly in order.

**Wrong index for the access pattern**

`idx_events_created_at` is on `created_at` alone, so the plan uses it to walk rows in date order, but then has to filter out every row that isn't `user_id = 4821` one at a time, it removed 3.8 million rows to find 20 matching ones. This worked fine at 10,000 rows because there was little to filter through, but at 50 million rows the filter has to walk deep into the index before it accumulates 20 matches. **The fix here is a composite index on `(user_id, created_at)`, so the database can jump directly to this user's rows already sorted by date, instead of scanning the global date order and filtering.**

**Missing or stale VACUUM/ANALYZE**

Check `pg_stat_user_tables` for `last_autovacuum` and `last_autoanalyze`. A table that grew 5,000x without regular vacuuming can accumulate significant bloat (dead tuples from updates/deletes still occupying index and heap pages), which makes both index and sequential scans slower than they should be, and stale statistics can also mislead the planner's row estimates.

**Index bloat**

Even with vacuuming, high churn (frequent updates or deletes) can leave a B-tree index with a lot of partially empty pages. Running `SELECT * FROM pgstatindex('idx_events_created_at');` or checking index size relative to expected size can reveal this. `REINDEX CONCURRENTLY` reclaims the wasted space.

**Growing result set assumptions**

Confirm the query's actual selectivity hasn't changed, if `user_id = 4821` used to match 5 rows out of 10,000 and now matches 50,000 out of 50 million, the fundamental shape of the query changed too, not just the table size, and no index change fixes a query that's now inherently returning a lot of data.

**How to confirm**

Compare `EXPLAIN ANALYZE` before and after adding the composite index, and check `pg_stat_user_tables` and index bloat stats before concluding it's purely a missing-index problem.

**Interview Answer:**

"The plan shows the query using an index on `created_at` alone and then filtering out 3.8 million rows to find 20 matching a specific user, that's fine at 10,000 rows but falls apart at 50 million because the filter has to walk much further into the index before it finds enough matches. I'd add a composite index on `(user_id, created_at)` so the lookup goes straight to that user's rows already in date order. Beyond that, I'd check `pg_stat_user_tables` for autovacuum and autoanalyze history, since a table that grew that much without regular maintenance can also suffer from bloat and stale statistics, both of which compound the problem."

---

## Q33. [Senior] There's an index on this column, but the query below still runs a full scan. Why can't the index help, and what would you actually do about it?

```
CREATE INDEX idx_products_description ON products(description);

EXPLAIN ANALYZE
SELECT * FROM products WHERE description LIKE '%bluetooth speaker%';
Seq Scan on products  (cost=0.00..184210.00 rows=812 width=210) (actual
time=0.089..1102.443 rows=794 loops=1)
```

```
  Filter: (description ~~ '%bluetooth speaker%'::text)
  Rows Removed by Filter: 3199206
Planning Time: 0.112 ms
Execution Time: 1102.671 ms
```

**Answer:**

A standard B-tree index is built on sorted values, and it can only help when the search can be anchored to a starting point in that sorted order. **A leading wildcard like `%bluetooth speaker%` means the match could start anywhere inside the string, so there's no prefix to anchor a sorted search on, and the database has no choice but to check every row.**

**Why a trailing wildcard is different**

`WHERE description LIKE 'bluetooth%'` can use a B-tree index just fine (with the right locale/opclass), because the index is sorted alphabetically and the database can jump straight to entries starting with "bluetooth" and scan forward until they stop matching. A leading wildcard destroys that anchor entirely.

**Alternatives that actually work**

- **Trigram index (`pg_trgm`)**: builds an index over three-character substrings of the text, so it can support arbitrary substring matches, including leading wildcards, reasonably efficiently.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_products_description_trgm ON products USING gin (description
gin_trgm_ops);
```

This lets the same `LIKE '%bluetooth speaker%'` query use an index scan instead of a seq scan.

- **Full-text search**: if the real goal is "does this description contain these words," rather than an exact substring, Postgres's built-in full-text search (`tsvector`/`tsquery`) with a GIN index, a different index structure built for values with many components, like words in a document, is usually a better fit than `LIKE`, since it handles word stemming, ranking, and multi-word queries naturally.

```
CREATE INDEX idx_products_description_fts ON products USING gin (to_tsvector('english',
description));
SELECT * FROM products WHERE to_tsvector('english', description) @@ to_tsquery('bluetooth
& speaker');
```

- **External search engine**: for large-scale, fuzzy, ranked, or faceted search (typo tolerance, relevance scoring, filtering across many fields at once), offloading to Elasticsearch or OpenSearch is the standard answer once Postgres text search stops being enough, at the cost of maintaining a separate system and keeping it in sync.

**Choosing between them**

Trigram indexes are the simplest fix and stay in Postgres, good for arbitrary substring or fuzzy matching on a moderate scale. Full-text search is better when the query is really about words and relevance, not raw substrings. A dedicated search engine is worth the operational cost only once search becomes a first-class product feature with real scale and relevance requirements.

**Interview Answer:**

"A B-tree index is a sorted structure, so it only helps when a query can anchor to a starting point, a prefix. `LIKE '%bluetooth speaker%'` has no fixed starting point, since the match could begin anywhere in the string, so the database can't use the index and falls back to checking every row. If this needs to stay in Postgres, I'd add a trigram index with `pg_trgm`, which indexes substrings and can support arbitrary `LIKE` patterns including leading wildcards. If the real need is word-based search rather than exact substrings, I'd use full-text search with a GIN index on a tsvector instead. And if this is becoming a real search feature with relevance ranking and scale, I'd move it out to something like Elasticsearch."

## Q34. [Mid] Gmail has to tell a user instantly whether a username they just typed is already taken, out of hundreds of millions of existing accounts. How do you make that check fast and correct at that scale?

**Answer:**

The naive query, a lookup against a single users table, is already fast thanks to a unique index on username, so correctness is not actually the hard part here. The hard part is volume: a signup form firing this check on every keystroke turns one signup into five to ten reads, and at Gmail scale that read volume dwarfs the actual signup rate many times over.

*Why a unique index alone gets you correctness, not speed*

A unique index on username is what actually prevents two accounts from ever colliding, this is a database-level guarantee enforced at insert time, not something the application has to coordinate itself. A single indexed point lookup is already sub-millisecond, so the index was never the bottleneck, the bottleneck is how many of these lookups arrive per second.

*The real bottleneck: read volume, not data volume*

Routing every keystroke's availability check straight to the primary database competes with real signups, logins, and profile writes for the same connections and the same lock manager. The fix is to keep this read traffic away from the primary entirely.

**Best Choice:** read replicas and a cache in front of them, not a cleverer index.

Availability checks are read-only and can tolerate **a few seconds of staleness** (a username freed up moments ago being briefly reported as taken is a non-event), which makes them a perfect fit for a cache backed by read replicas rather than the primary.

**Best Choice:** a Bloom filter in front of the cache for the common case.

A Bloom filter is a compact, in-memory probabilistic structure loaded with every existing username, refreshed incrementally as accounts are created. It answers "definitely not taken" with complete certainty and "possibly taken" with a small, tunable false-positive rate. Most usernames people try during signup are, in fact, available, so the Bloom filter answers the overwhelming majority of checks in microseconds, entirely in memory, without touching a cache or database. Only the rarer "possibly taken" case falls through to the cache, then the read replica.

**The trick is layering cheap, approximate checks in front of expensive, exact ones, and only paying for a real database lookup on the minority of cases that actually need one.**

*Handling the race condition at actual signup*

None of the above stops two people from racing to claim the same username at the exact same instant, it only makes the live preview fast. The real guarantee still comes entirely from the unique index on the primary at insert time: whichever signup transaction commits second gets a constraint violation and is told to pick another name, exactly as it would with no caching layer at all.

**Interview Answer:** "I'd split this into two different problems: making the live 'is this available' preview feel instant, and guaranteeing two accounts never actually collide. For speed, I'd layer a Bloom filter in memory in front of a cache, in front of read replicas, so the overwhelming majority of checks, which are for available usernames, get answered in microseconds without hitting a database. For correctness, none of that matters: the actual guarantee comes from a unique index on the primary users table, so if two people ever raced to grab the same name, the database rejects the second insert and that signup is asked to choose another username."

**If you remember nothing else from this chapter:** *the wrong shard key doesn't fail on day one, it fails a year later when fixing it is an order of magnitude more expensive.*

# Chapter 8: Caching Strategies

Caching is one of the highest-leverage topics in a backend interview, because it's also one of the easiest to get subtly wrong in production. If your instinct is to say "just add Redis" without explaining invalidation, staleness, or thundering herd risk, that's a sign you haven't yet had to operate a cache under real traffic, this chapter fixes that.

> **War Story:** *A cache that only warmed on deploy meant every single release caused a five-minute spike of database load as the cache emptied and every request became a miss at once, a self-inflicted thundering herd the team caused with every deploy.*

> **Avoid:** treating "just add a cache" as a complete answer, with no mention of invalidation or the stampede risk
>
> **Aim For:** raising cache stampede or thundering herd unprompted and proposing a mitigation, like locking or jittered TTLs

## Q1. [Junior] What is a cache, and why would you put one in front of a database for a read-heavy API?

**Answer:**

**A cache is a smaller, faster storage layer that keeps a copy of data so future requests for that same data can be served without redoing the expensive work, usually a database query or a computation.** For example, caching a product page's data in Redis means the second person who views that same product gets it back in under a millisecond from memory, instead of the 20 to 50 milliseconds a full database query might take.

**Why it helps on a read-heavy API**

- Databases are slow relative to memory. A disk-backed query can take tens of milliseconds, an in-memory cache lookup takes well under a millisecond.
- Most read-heavy workloads follow a pattern where a small set of records gets requested far more often than the rest, so caching those hot records removes a large share of database load.
- Reducing load on the database also protects the writes and the less-popular reads that still need to hit it directly.

**What you give up**

- The cached copy can go stale the moment the underlying row changes, so you now own a freshness problem you didn't have before.
- You add an extra component that can fail, run out of memory, or return wrong data if it's misconfigured.

**Interview Answer:**

"A cache is a fast, temporary store that sits between my application and a slower system of record, usually a database. I'd add one here because read-heavy traffic tends to repeatedly ask for the same popular data, and serving that from memory instead of running a database query every time cuts latency and takes a huge amount of load off the database. The trade-off is that I now have to manage staleness, since the cached copy can drift out of sync with the database."

## Q2. [Junior] If you had to know just four caching technologies cold for interviews, which four would you pick, and what's the one-line pitch for each?

**Answer:**

**These four cover almost every caching decision you'll be asked about, and the real skill is knowing which layer a given problem actually belongs to, not just knowing that "caching helps."**

**Redis, the versatile one**

- One-line pitch: an in-memory data store with rich data structures, strings, hashes, lists, sets, sorted sets, plus optional persistence, which makes it useful for far more than plain caching.
- Reach for it when you need more than get and set: rate limiters, leaderboards, session stores, or pub/sub, on top of caching.
- What it's bad at: it's single-threaded for command execution, so one extremely expensive command can briefly block everything else on that node.

**Memcached, the simple one**

- One-line pitch: a pure, multi-threaded, in-memory key-value cache with no persistence and no extra features, just fast get and set.
- Reach for it when you genuinely just want to cache blobs of data to cut database load, with nothing fancy, and want to take advantage of multiple CPU cores per node without extra work.
- What it's bad at: everything beyond plain caching. No data structures, no persistence, no built-in replication.

**CDN edge caching, the geography one**

- One-line pitch: a network of servers spread across the world that cache HTTP responses close to users, so requests don't have to cross the planet to your origin.
- Reach for it for static assets, images, and any response that's safe to share across many users.
- What it's bad at: personalized or highly dynamic per-user content, since there's little to share across users and cache hit rates collapse.

**Local in-process cache, the fast one**

- One-line pitch: a cache that lives in your application's own memory, so a lookup costs a few nanoseconds with zero network hop.
- Reach for it for extremely hot, small pieces of data read constantly by that one service, like a feature flag or a config value.
- What it's bad at: consistency across instances. Every instance has its own copy, so invalidating one doesn't touch the others.

**Interview Answer:**

"The four I'd want cold are Redis, Memcached, CDN edge caching, and a local in-process cache. Redis is my default when I need more than plain caching, like data structures or pub/sub. Memcached is what I'd reach for when I genuinely just want the simplest possible cache in front of a database. CDN caching lives at a completely different layer, geography, for static or shareable content. And a local in-process cache is for the rare case where even a sub-millisecond network hop to Redis is too slow for something read constantly by one service."

## Q3. [Mid] You need a session store plus a real-time leaderboard for a multiplayer game. Which caching technology fits, and why would a simpler option not cut it?

**Answer:**

**This needs more than plain key-value caching, sessions are simple, but a real-time leaderboard needs ranking operations, which is exactly what Redis's sorted set data structure is built for.**

**Why plain Memcached falls short**

- Memcached gives you get and set on opaque blobs, so ranking players by score would mean fetching every player's data and sorting it yourself on every leaderboard read, which doesn't scale.
- No persistence either, so a full node restart quietly loses both the sessions and the leaderboard, forcing every user to log in again and the leaderboard to rebuild from scratch.

**Why Redis fits**

- A sorted set keys each player by their score and keeps the whole set ordered automatically, so "get the top 10" or "get this player's rank" are both fast, native operations instead of application-level sorting.
- Session data fits naturally as a simple key holding a hash or a string, with a TTL so stale sessions expire automatically.
- Optional persistence (RDB or AOF) means a restart doesn't have to mean losing all active sessions or the current leaderboard state.

**Interview Answer:**
"Sessions alone would work fine on almost anything, but the leaderboard is the deciding factor. Redis's sorted set keeps every player ranked by score automatically, so getting the top 10 or a specific player's rank is a fast, built-in operation instead of something I'd have to compute myself on every read. Memcached would force me to fetch and sort everything by hand and offers no persistence, so I'd lose both the sessions and the leaderboard on a restart. Redis's data structures and optional persistence make it the clear fit here."

## Q4. [Mid] You just need a dead-simple cache in front of a database to cut read load: no persistence, no fancy data types, just get and set. Would you reach for Redis or Memcached, and why?

**Answer:**

**Both would technically work, but this is exactly the scenario Memcached was built for, and reaching for Redis here means carrying complexity you're not actually using.**

**The case for Memcached**

- It's multi-threaded, so a single node can use multiple CPU cores for pure get and set traffic without any extra configuration, which can mean better raw throughput per node for this exact workload.
- No persistence to configure or think about, which matches "just a cache, if it's lost we refetch from the database" perfectly.
- Its simplicity is a feature here: fewer moving parts, fewer configuration decisions, fewer ways to misuse it.

**The case for Redis anyway**

- Even if you don't need extra data structures today, most teams standardize on Redis because their next caching need (rate limiting, a leaderboard, pub/sub) will want it, and running two caching technologies in production is its own operational cost.
- Redis's single-threaded model is rarely the bottleneck in practice for typical cache workloads, so the "Memcached is faster" argument matters less than it used to.

**Best Choice**

For a genuinely standalone, simple cache with no foreseeable need for anything beyond get and set, Memcached is a perfectly reasonable, sometimes even better, choice. In most real organizations, Redis wins anyway because of what else it enables later.

**Interview Answer:**

"For exactly this workload, plain get and set with no persistence, Memcached is a completely valid, sometimes preferable choice, since it's simpler and its multi-threaded model can give better raw throughput for pure caching. That said, in practice I'd lean Redis unless I was confident this would never grow beyond simple caching, because the moment you need a rate limiter or a leaderboard next quarter, you either bolt on Redis anyway or contort Memcached to do something it wasn't built for. Running one caching technology instead of two is a real operational win."

---

## Q5. [Mid] You're serving product images and static JS and CSS bundles to a global user base. Where should the caching actually happen, and why would putting this in Redis be the wrong instinct?

**Answer:**

**This is a geography problem, not a data structure problem, so it belongs at the CDN edge, close to each user, not in a single Redis cluster sitting in one region.**

**Why Redis is the wrong instinct here**

- A Redis cluster lives in one place (or a few regions if you've built multi-region replication yourself), so a user on the other side of the world still crosses the same long network distance to reach it, even though the actual response never changes per user.
- You'd be solving a "many users share the same shareable content" problem with a tool designed for fast application-level data access, missing the actual lever that matters here: physical distance to the user.

**Why a CDN fits**

- A CDN's whole job is caching content at edge locations physically close to users, so a user in Singapore gets a cached image from a nearby edge server instead of crossing an ocean to your origin.
- Static assets are exactly the "safe to share across every user" content a CDN is built for, since the image or bundle doesn't change per request.
- The origin server barely gets touched once content is cached at the edge, which also protects it from traffic spikes.

**Interview Answer:**

"This is fundamentally a distance problem, not a data access problem, so it belongs at the CDN edge, not in Redis. Redis would still sit in one place, or a few regions at best, so a user far from that region pays the same long network trip even though the image never changes. A CDN caches that same static content at edge locations physically close to every user, so the response comes from nearby instead of crossing the globe.

Redis is the right tool when I need fast access to data that changes per request or per user, not for serving the same static file to millions of people worldwide."

---

## Q6. [Senior] A single config value or feature flag is read millions of times per second by every instance of a service. Even a sub-millisecond Redis lookup is now a bottleneck. What do you do?

**Answer:**

**At this read volume, even a fast network round trip to Redis adds up, so the fix is to stop making a network call at all and keep the value in the application's own memory.**

**Why Redis alone isn't enough here**

- A sub-millisecond Redis lookup still means a network hop, serialization, and Redis itself processing the command, and at millions of reads per second across many service instances, that adds real aggregate load and latency that a local memory read simply doesn't have.
- Redis is still a great source of truth for the value, the problem is calling it on every single read.

**The fix: a local in-process cache in front of Redis**

- Each service instance keeps its own copy of the config value in local memory, so a read costs nanoseconds with zero network involvement.
- The instance refreshes its local copy periodically (a short TTL) or, better, subscribes to a Redis pub/sub channel that Redis publishes to whenever the value changes, so updates propagate quickly without constant polling.

**What you give up**

- Every instance's local copy can be briefly stale between the moment the value changes and the moment that instance's cache refreshes or receives the pub/sub update. For a feature flag or config value, that's usually an acceptable trade for the latency win.
- More moving parts: you're now running a two-level cache and need to reason about propagation delay, not just a single Redis lookup.

**Interview Answer:**
"At millions of reads per second, even a fast Redis call adds up in aggregate network and processing cost, so I'd add a local in-process cache in front of it. Each instance keeps its own copy in memory, so a read is a nanosecond-scale local lookup with zero network hop. I'd keep Redis as the source of truth and either use a short TTL or, better, a pub/sub channel that pushes updates out immediately when the value changes, so every instance's local copy refreshes quickly instead of polling constantly. The trade-off is brief staleness between instances, which is a fine price to pay for a config value or feature flag."

---

## Q7. [Junior] Walk me through what happens on a cache-aside read, both on a cache hit and on a cache miss.

**Answer:**

Cache-aside, also called lazy loading, puts the application in charge of both reading from the cache and populating it. **The cache itself has no knowledge of the database.**

**On a cache hit**

1. The application receives a request for a piece of data.
2. It checks the cache first, using a key like `user:1234`.
3. The cache returns the value immediately, and the database is never touched.

**On a cache miss**

1. The application checks the cache and gets nothing back.
2. It queries the database directly for the data.
3. It writes the result into the cache, usually with a TTL, so the next request for that key is a hit.
4. It returns the data to the caller.

**Why this pattern is popular**

- Only data that's actually requested ends up in the cache, so you don't waste memory on cold data.
- If the cache goes down entirely, the application can still fall back to the database, just slower.

**The trade-off**

- The first request for any key always pays the full database cost, and if writes don't also update or invalidate the cache, reads can serve stale data until the TTL expires.

**Interview Answer:**
"With cache-aside, my application code checks the cache first. On a hit, I return the cached value directly. On a miss, I query the database, then write that result back into the cache with a TTL before returning it to the caller. The nice part is the cache only ever holds data that's actually being requested, and if the cache layer is unavailable, I can still serve traffic straight from the database, just at higher latency."

---

## Q8. [Junior] What is a TTL, and why does almost every cache entry need one?

**Answer:**

TTL stands for time to live. **It's the amount of time a cache entry is allowed to sit in the cache before it's considered expired and either removed or refreshed.**

**Why it matters**

- Without a TTL, a cache entry lives forever unless something explicitly deletes it. If the underlying data changes and nobody remembers to invalidate that key, the cache serves stale data indefinitely.
- A TTL puts a ceiling on how stale data can get. Even if invalidation logic has a bug or gets skipped, the entry will eventually expire and get refreshed from the source of truth.
- TTLs also help with memory management, since expired entries can be reclaimed automatically instead of accumulating forever.

**Choosing a TTL**

- Short TTLs (seconds) suit data that changes often or where staleness is costly, like inventory counts.
- Long TTLs (hours or more) suit data that rarely changes, like a user's display name or a product description.
- The TTL should reflect how tolerant the system is of showing slightly old data, not just a default value copied from another service.

**Interview Answer:**

"A TTL is the expiration window on a cache entry, after which it's treated as stale and removed or reloaded. Almost every cache entry needs one because it's a safety net against staleness. Even if my invalidation logic has a gap, a TTL guarantees the cache eventually self-corrects by falling back to the database. I'd tune the TTL based on how often the underlying data changes and how much staleness the product can tolerate, rather than using one default value everywhere."

---

## Q9. [Junior] Compare LRU and FIFO eviction policies. Which would you pick for a cache holding recently viewed products?

**Answer:**

When a cache is full and a new item needs to be stored, an eviction policy decides what gets removed.

**Options**

**FIFO (First In, First Out)**

- Evicts the oldest entry by insertion time, regardless of how often it's been accessed.
- Pros: simple to implement, cheap to maintain, predictable behavior.
- Cons: it can evict a very popular item just because it happened to be inserted early, which hurts hit ratio.

**LRU (Least Recently Used)**

- Evicts the entry that hasn't been accessed for the longest time.
- Pros: naturally keeps frequently and recently accessed items around, which matches real access patterns where recent activity predicts future activity.
- Cons: slightly more overhead to track access order, and it can be fooled by a one-time scan through lots of keys that pushes out genuinely hot data.

**Best Choice**

LRU.

**Why?**

For recently viewed products, the access pattern is exactly what LRU is built for: a user who viewed an item recently is likely to view it again soon, and popular products stay hot because they keep getting accessed, which keeps refreshing their position. **FIFO would evict based on age alone and could throw away a popular item simply because it was cached early, hurting the hit ratio for no good reason.**

**Interview Answer:**

"I'd pick LRU here. Recently viewed products are a textbook case for recency-based access, users tend to revisit things they just looked at, and popular products stay in the cache because repeated access keeps refreshing their position. FIFO only looks at insertion order, so it could evict something still being actively viewed just because it was cached first. LRU costs a little more to implement since it tracks access order, but for this workload the better hit ratio is worth it."

---

## Q10. [Junior] Where would you place a cache in a typical web application: in the application layer, or directly in front of the database?

**Answer:**

**Caches can live at several layers, and the right answer is usually more than one of them, depending on what you're trying to speed up.**

**Common layers**

- **Client side**: browser or mobile app caches responses locally, cutting out network round trips entirely for repeat views.
- **Application layer**: the app server checks an in-process or nearby cache before doing any real work, useful for computed results or session data.
- **Distributed cache**: a shared service like Redis or Memcached sits between the application and the database, shared across all app server instances.
- **Database layer**: the database itself has internal caches like buffer pools and query caches that reduce disk I/O.

**How to decide**

- If multiple app servers need to share cached data consistently, a distributed cache in front of the database is the natural fit, since an in-process cache would be duplicated and inconsistent across instances.
- If the data is truly local and doesn't need to be shared, an in-process cache is faster because it avoids a network hop.

**Interview Answer:**

"For a typical web application with multiple app server instances, I'd put a shared distributed cache like Redis between the application layer and the database. That way every instance sees the same cached data instead of each one keeping its own inconsistent copy. I might layer an in-process cache on top of that for very hot, rarely changing data to avoid even the network hop to Redis, but the distributed cache is the main line of defense in front of the database."

---

## Q11. [Junior] Your cache hit ratio is sitting at 95 percent. Is that good, and what does hit ratio actually tell you?

**Answer:**

**Hit ratio is the percentage of read requests that were served from the cache instead of falling through to the database.** It's calculated as hits divided by total requests, hits plus misses.

**Is 95 percent good?**

- It depends entirely on context. For a cache in front of a heavy, slow database query, 95 percent means only 5 percent of traffic ever touches the database, which is usually excellent.
- For a cache where the underlying data changes constantly, 95 percent might mean the TTL is too long and you're serving stale data most of the time to get that number.

**What hit ratio doesn't tell you**

- It doesn't tell you about correctness. A high hit ratio with an aggressive TTL can still mean users are seeing outdated data.
- It doesn't tell you about the cost of a miss. If a miss is cheap, a lower hit ratio might be fine. If a miss triggers an expensive fallback, even a small percentage of misses can matter a lot.
- It should always be read alongside database load, latency percentiles, and how stale the data is allowed to be.

**Interview Answer:**
"On its own, 95 percent sounds good, but I wouldn't call it good or bad without more context. I'd look at how expensive a miss is for this specific database query, and whether the TTL that's producing this hit ratio is also producing acceptable staleness. A high hit ratio achieved by an overly long TTL can hide a data freshness problem, so I'd pair hit ratio with latency and staleness metrics before calling it a success."

---

## Q12. [Mid] Walk me through cache-aside, write-through, and write-back. Which would you choose for a shopping cart service?

**Answer:**

These are the three main strategies for how writes interact with a cache.

**Options**

**Cache-aside**

- The application reads from the cache on lookups and writes directly to the database on updates, then either invalidates or updates the corresponding cache key.
- Pros: simple, resilient, cache only holds requested data.
- Cons: a small window exists between the database write and the cache update or invalidation where a read can return stale data.

**Write-through**

- Every write goes to the cache first, and the cache synchronously writes through to the database before acknowledging the write.
- Pros: the cache is always consistent with the database immediately after a write.
- Cons: every write pays the latency of both the cache write and the database write, so writes are slower.

**Write-back (write-behind)**

- Writes go to the cache and are acknowledged immediately, then flushed to the database asynchronously in the background.
- Pros: very low write latency, can batch and reduce database write load.
- Cons: risk of data loss if the cache crashes before the flush happens, and the database temporarily lags behind reality.

**Best Choice**

Cache-aside for reads, combined with a write-through style update or invalidation on the cart write path.

**Why?**

**A shopping cart needs the database to be the durable source of truth immediately, since losing a cart item or an order due to an unflushed write-back buffer is unacceptable.** Write-through's added write latency is a fair price for guaranteed consistency on something users directly interact with and expect to see reflected right away.

**Interview Answer:**
"For a shopping cart, I'd lean toward cache-aside reads with a write-through style update on the write path: the write goes to the database first, and I update or invalidate the cache in the same operation so the next read is correct. I'd avoid write-back here because losing a cart update if the cache crashes before flushing to

the database is a real user-facing problem, and cart writes aren't frequent enough that the extra write latency of touching the database directly is a real cost."

---

## Q13. [Mid] What is a cache stampede, and describe a scenario where it takes down your database.

**Answer:**

A cache stampede, also called a thundering herd, happens when a single cached value that many requests depend on disappears at once, and all of those requests fall through to the database simultaneously to regenerate it.

**How it happens**

1. A popular cache key, say the homepage's list of trending articles, has a TTL of 60 seconds.
2. At second 60, the key expires.
3. In the next instant, thousands of concurrent requests all check the cache, all get a miss on the same key, and all independently query the database to rebuild that same value.
4. The database, which normally only serves a trickle of traffic because the cache absorbs almost everything, suddenly gets hit with thousands of identical, redundant queries at once.
5. This spike can exhaust database connections or CPU, causing latency to spike or the database to fall over, which then causes even more requests to pile up and retry.

**Why it's dangerous**

**The database was sized assuming the cache would absorb the vast majority of load.** A stampede briefly removes that protection for exactly the moment when demand is highest, which is often the worst possible time for the database to be overwhelmed.

**Interview Answer:**
"A cache stampede happens when a hot cache key expires and every concurrent request for it misses at the same instant, so they all hit the database to recompute the same value simultaneously. A good example is a trending content list with a short TTL, where the expiration lines up with a burst of traffic, and suddenly the database gets thousands of duplicate queries it was never sized to handle directly, which can cascade into a full outage."

---

## Q14. [Mid] How would you prevent a thundering herd when a popular cache key expires?

**Answer:**

**The core idea is to make sure only one request rebuilds an expired key while everyone else either waits for that result or gets a slightly stale value.**

**Techniques**

1. **Locking or singleflight**: when a request misses the cache, it acquires a short lock (for example, using Redis's `SETNX`, which sets a key only if it doesn't already exist, a simple building block for a distributed lock) before querying the database. Other requests that see the lock held either wait briefly and retry the cache, or serve stale data instead of also querying the database.
2. **Request coalescing in the application**: within a single process, deduplicate concurrent requests for the same key so only one in-flight database call happens, and all waiters share its result.

3. **Early refresh (probabilistic early expiration)**: instead of waiting for a hard TTL expiry, let some requests before the deadline recompute the value early with a small probability that increases as expiration approaches, spreading the recompute out over time.
4. **Stale-while-revalidate**: serve the old, slightly expired value immediately while one background request refreshes it, so users never see the cache-miss latency at all.
5. **Staggered TTLs (jitter)**: avoid setting the exact same TTL on many keys created at the same time, so they don't all expire in the same instant.

**Interview Answer:**
"I'd combine a couple of these. First, a lock so only one request rebuilds the value while others wait or get a stale copy, using something like Redis SETNX as a mutex. Second, stale-while-revalidate, so I keep serving the last known good value while a single background request refreshes it, which means users never even see the miss. I'd also add jitter to TTLs so keys created in a batch don't all expire at the exact same moment and recreate the same problem."

---

## Q15. [Mid] Compare Redis and Memcached. Which would you pick for a session store, and why?

**Answer:**

Both are in-memory key-value stores commonly used as caches, but they differ in features and complexity.

**Options**

**Memcached**

- Pure key-value store, values are typically simple blobs like strings.
- Multithreaded, very efficient for pure caching of simple objects.
- No built-in persistence or replication, if a node restarts, the data is gone.
- Pros: simple, fast, easy to scale horizontally with consistent hashing.
- Cons: no data structures beyond simple values, no persistence, no native replication.

**Redis**

- Rich data structures: strings, hashes, lists, sets, sorted sets, streams.
- Supports persistence (snapshotting or append-only file), replication, and clustering.
- Supports pub/sub, transactions, and Lua scripting for atomic operations.
- Pros: much more versatile, can serve as a cache, a message broker, or a lightweight database.
- Cons: single-threaded for command execution (per core in cluster mode), so a large or blocking command can affect latency for others.

**Best Choice**

Redis.

**Why?**

Session data is a hash-like structure, often needs atomic updates (like refreshing an expiry or updating a single field without rewriting the whole session), and benefits from optional persistence so a Redis restart doesn't log every user out at once. **Redis's native TTL support per key and its hash data type map cleanly onto session storage needs in a way that Memcached's flat key-value model does not.**

**Interview Answer:**

"I'd pick Redis for a session store. Sessions map well onto Redis's hash data type, so I can update individual fields atomically, and Redis gives me per-key TTLs out of the box for session expiry. Persistence is also valuable here, since a Memcached restart wipes every session instantly and logs everyone out, while Redis can be configured to survive a restart without losing that state. Memcached would be simpler and slightly faster for pure flat-value caching, but it's missing features that sessions specifically need."

---

## Q16. [Mid] What is the hot key problem in a distributed cache, and how would you handle a single celebrity user's profile getting hammered?

**Answer:**

A distributed cache normally spreads load across many nodes using something like consistent hashing on the key. The hot key problem occurs when one specific key gets a disproportionate share of all traffic, which means the single node responsible for that key becomes a bottleneck no matter how many nodes are in the cluster. For example, a celebrity with 50 million followers posting once can cause that single profile key to receive far more reads per second than any one cache node was sized to handle, even though the other 10 million keys in the cluster are barely touched.

**Why sharding doesn't fix it**

Sharding distributes different keys to different nodes, but it can't split a single key's traffic across multiple nodes, since normally one key lives on exactly one node. **Adding more nodes to the cluster doesn't help a hot key at all.**

**How to handle it**

1. **Local caching layer**: cache the hot value in-process on each application server for a very short TTL, so most requests never even reach the distributed cache for that key.
2. **Key replication**: store copies of the hot key under several suffixed key names (like `celebrity:1`, `celebrity:2`, `celebrity:3`) spread across different nodes, and have clients pick one at random or round robin, spreading the load.
3. **Read replicas**: if the cache supports replicas, route reads for hot keys across replicas rather than a single primary.
4. **Request coalescing**: if the hot key is expensive to compute on miss, ensure concurrent misses don't all hit the backend at once.

**Interview Answer:**
"A hot key means all the traffic is concentrated on one cache key, and since that key lives on one node, sharding across more nodes doesn't help at all. For a celebrity profile, I'd add a short-lived local in-process cache on each app server first, since that removes most of the traffic before it even reaches the distributed cache. If it's still too hot, I'd replicate that key under multiple suffixed names across nodes and spread reads across those copies, or route reads through replicas instead of the primary."

---

## Q17. [Mid] How do you keep a cache consistent with the database when the underlying row changes?

**Answer:**

There's no way to guarantee perfect real-time consistency between a cache and a database without giving up most of the performance benefit of caching, so the goal is usually to bound staleness to an acceptable window.

**Approaches**

1. **TTL-based expiration**: every cached entry expires after a fixed window, guaranteeing staleness never exceeds that window even if invalidation is missed.
2. **Write-time invalidation**: whenever a write updates a row, the same code path deletes or updates the corresponding cache key, so the next read is forced to refetch fresh data.
3. **Delete over update**: on a write, prefer deleting the cache key rather than writing the new value into the cache directly. This avoids race conditions where a slower concurrent write could overwrite a newer write's cache entry with stale data.
4. **Change data capture (CDC)**: for larger systems, a stream of database changes (like from a write-ahead log) drives cache invalidation asynchronously, decoupling it from the application's write path.
5. **Versioning**: store a version number or timestamp alongside cached data so a stale write can be detected and discarded instead of blindly overwriting.

**What to accept**

**Some staleness window is almost always acceptable in practice.** The real design decision is choosing how small that window needs to be for the specific data, and then picking the cheapest mechanism that satisfies it.

**Interview Answer:**

"I treat perfect consistency as unrealistic for a cache and instead bound the staleness window. On every write, I invalidate the corresponding cache key rather than trying to update it in place, since deleting avoids race conditions where two concurrent writes could leave the cache holding an older value. I back that with a TTL as a safety net in case an invalidation gets missed. For stricter needs, I'd look at CDC off the database's change log to drive invalidation asynchronously and decouple it from the request path."

---

## Q18. [Mid] What is cache penetration, and how would you protect the database when requests keep asking for keys that don't exist?

**Answer:**

**Cache penetration happens when requests ask for data that doesn't exist in either the cache or the database.** Since there's nothing to cache, every one of these requests falls straight through to the database, bypassing the protection the cache is supposed to provide.

**Why it's a real risk**

- This is often exploited or triggered accidentally, for example a client repeatedly requesting a user ID that was deleted, or an attacker scanning random IDs that don't exist.
- Because there's no cached "not found" result, the cache offers zero protection against this pattern, no matter how well-tuned it is otherwise.

**How to fix it**

1. **Negative caching**: cache the fact that a key doesn't exist, using a short TTL, so repeated lookups for the same missing key are absorbed by the cache instead of hitting the database every time.
2. **Bloom filter**: maintain a probabilistic structure of all keys known to exist. Before querying the database, check the bloom filter, and if it says the key definitely doesn't exist, skip the database entirely. This is especially useful when the space of possible invalid keys is huge.
3. **Input validation**: reject obviously malformed or out-of-range keys before they ever reach the cache or database layer.

**Interview Answer:**

"Cache penetration is when requests hit keys that don't exist anywhere, so the cache can't help and every request goes straight to the database. I'd fix it with negative caching, storing a short-lived marker that a key doesn't exist so repeated lookups for it are absorbed by the cache. For a much larger space of possible invalid keys, like scanning random user IDs, I'd add a bloom filter in front of the database to cheaply reject lookups for keys that are guaranteed not to exist."

---

## Q19. [Mid] Explain LFU eviction versus LRU. When would LFU actually beat LRU?

**Answer:**

Both decide what to evict when the cache is full, but they use different signals.

**Options**

**LRU (Least Recently Used)**

- Evicts based on recency, the item that hasn't been touched in the longest time goes first.
- Pros: cheap to track, works well when recent access predicts future access.
- Cons: a single burst of one-time access, like a batch job scanning many keys once, can evict genuinely popular items just because they weren't touched during that scan.

**LFU (Least Frequently Used)**

- Evicts based on total access count, the item accessed the fewest times goes first.
- Pros: resistant to one-off scans, since a key with a long history of frequent access won't be evicted by a brief burst of unrelated traffic.
- Cons: more overhead to maintain accurate counts, and it can be slow to adapt when something genuinely was popular in the past but isn't anymore, since old counts linger unless decayed.

**Best Choice**

It depends on the access pattern, LFU wins specifically when access frequency is a better predictor of future demand than recency.

**Why?**

Picture a product catalog cache where a handful of best-selling items are looked up constantly, but a nightly analytics job scans through thousands of rarely viewed products once each. Under LRU, that scan would evict the best-sellers since they weren't the most recently touched, even though they're the items that matter most to keep cached. **LFU would correctly keep the best-sellers in place because their access counts dwarf the one-time scan traffic.**

**Interview Answer:**

"LRU evicts based on recency, LFU evicts based on how often something has been accessed overall. LFU beats LRU when you have a workload with a stable set of genuinely popular items alongside occasional bursts of one-time scans, because LRU would let that scan traffic wrongly evict the popular items just for not being the most recent, while LFU protects them based on their access history. The trade-off is LFU needs decay logic so old popularity doesn't stick around forever after demand shifts."

## Q20. [Mid] You update a database row, then evict the cache key, but a concurrent read repopulates the cache with stale data. Walk me through this race condition and how you'd close it.

**Answer:**

**This is a classic cache-aside race condition.**

**How it happens**

1. Request A reads key X, gets a cache miss, and starts querying the database for the old value.
2. Before A's query returns, Request B writes a new value for X to the database and deletes the cache key.
3. A's database query, which started before B's write, returns the old value.
4. A writes that old value into the cache.
5. The cache now holds stale data, and since there's no pending invalidation, it stays wrong until the TTL expires.

**Why it's tricky**

The order of operations that creates the bug depends on timing, so it won't show up in every request, only occasionally under concurrent load, which makes it easy to miss in testing.

**How to close it**

1. **Short TTL as a backstop**: guarantees the staleness window is bounded even if this race happens.
2. **Delayed double delete**: after updating the database, delete the cache key, then delete it again a few hundred milliseconds later, catching any stale value written by a race that happened in that window.
3. **Version or timestamp checks**: attach a version to cached values and to database rows, and only accept a cache write if its version is not older than what's already there.
4. **Write-through with locking**: on write, hold a lock on the key so a concurrent read's cache-populate write can't land after the invalidation, forcing it to wait or retry the read.

**Interview Answer:**

"This happens because a read that started before a write, based on old data, finishes and writes to the cache after the write's invalidation already ran, leaving stale data with nothing scheduled to clean it up. A TTL bounds the damage, but to actually close the gap I'd use a delayed double delete, where I delete the cache key right after the database write and again a few hundred milliseconds later, catching any race that slipped a stale value in during that window. For stricter cases, I'd attach a version number and reject cache writes that are older than what's already stored."

## Q21. [Mid] How would you design a distributed cache so that adding or removing nodes doesn't invalidate almost the entire cache?

**Answer:**

**The naive approach of hashing a key and taking the result modulo the number of nodes falls apart the moment the node count changes, because nearly every key's hash-modulo-count result changes too, meaning almost every key suddenly maps to a different node and is treated as a miss.**

**The fix: consistent hashing**

1. Map both nodes and keys onto a fixed hash ring, for example values from 0 to 2^32 minus 1.
2. Each key is assigned to the first node found going clockwise from the key's position on the ring.
3. When a node is added, it only takes ownership of the portion of the ring between itself and the next node, so only the keys in that slice move. Everything else stays put.
4. When a node is removed, only the keys it owned need to move, to the next node on the ring.

**Refinements**

- Use virtual nodes: give each physical node many points on the ring instead of one, which spreads load more evenly and avoids one node absorbing a disproportionate share when a neighbor leaves.
- Combine with replication, so each key lives on more than one node, protecting against a single node's data being lost entirely when it goes down.

**Interview Answer:**

"I'd use consistent hashing instead of a plain modulo scheme. With modulo, changing the node count reshuffles almost every key's target node, wiping out most of the cache. Consistent hashing places both nodes and keys on a ring, so adding or removing a node only affects the slice of keys between it and its neighbor, leaving the rest of the cache untouched. I'd also use virtual nodes per physical node to keep the load balanced, since a single point per node can create uneven hot spots when the cluster is small."

---

## Q22. [Mid] What's the difference between deleting a cache key and updating it in place on a write? Which do you prefer for write-through consistency?

**Answer:**

Both are ways of keeping a cache in sync after a database write, but they behave differently under concurrency. For example, if two updates to the same product's price arrive close together, updating the cache in place risks the slower write overwriting the cache with a value that's already stale by the time it lands, while deleting the key just forces the next read to fetch the current price fresh.

**Options**

**Update in place**

- On write, compute the new value and write it directly into the cache alongside the database write.
- Pros: the cache stays warm, no cache miss for the next read.
- Cons: if two writes happen close together, the slower one's cache write can land after the faster one's, leaving the cache holding an older value than the database, since there's no ordering guarantee between the two cache writes.

**Delete (invalidate)**

- On write, remove the cache key entirely instead of writing a new value.
- Pros: simpler and safer under concurrency, the next read is forced to fetch fresh data straight from the database, so it can never serve a stale value that a race condition slipped in.
- Cons: the next read after a write always pays a cache miss, adding a bit of latency and a small amount of extra database load right after writes.

**Best Choice**

Delete.

**Why?**

The race condition risk with update-in-place is subtle and hard to detect in testing, but it will happen under real concurrent load and silently serve wrong data. **Delete trades a small amount of guaranteed extra latency on the next read for eliminating an entire class of stale-data bugs, which is almost always the right trade for correctness-sensitive data.**

**Interview Answer:**

"Updating the cache in place keeps it warm, but it's vulnerable to a race where an older write's cache update lands after a newer one's, leaving stale data with no signal that anything's wrong. Deleting the key instead forces the next read to go to the database and repopulate the cache from the current source of truth, which avoids that race entirely. I'd default to delete for write-through consistency, since the cost is just one extra cache miss per write, and the correctness guarantee is worth it."

---

## Q23. [Senior] Design a caching layer for a product catalog that serves 200,000 reads per second and updates a few times a minute. Walk me through your approach end to end.

**Answer:**

**This workload is extremely read-heavy with infrequent writes, which makes it a great fit for aggressive caching.**

**Step by step**

1. **Estimate the working set**: figure out how many distinct products get most of the traffic. If the catalog has millions of SKUs but 95 percent of reads target a much smaller popular subset, size the cache around that hot set rather than the full catalog.
2. **Cache placement**: use cache-aside with a distributed cache like Redis in front of the database, so all app server instances share consistent cached data.
3. **Add a local in-process layer**: at 200,000 reads per second, even a distributed cache round trip adds up. An in-process cache with a short TTL (a few seconds) on the hottest products cuts a large share of that traffic before it ever reaches Redis.
4. **Choose eviction policy**: LRU or LFU depending on whether recency or long-term popularity better predicts demand, likely LFU given catalogs tend to have stable best-sellers.
5. **Write path**: since updates are infrequent, use write-through style invalidation, deleting the affected product's cache key on update rather than trying to keep it warm, since the write volume is low enough that the extra miss cost is negligible.
6. **Protect against stampede**: since traffic is so high, apply request coalescing or locking so a popular product's cache expiry doesn't cause a burst of duplicate database queries.
7. **TTL and jitter**: set a TTL as a staleness backstop, with jitter so keys don't expire in synchronized batches.
8. **Monitoring**: track hit ratio, p99 latency, and database load, and alert if hit ratio drops, which would signal either a cache sizing problem or a bug in invalidation.

**Interview Answer:**

"Given the read-to-write ratio here, I'd lean heavily on caching. I'd use cache-aside with Redis as the shared layer, sized around the actual hot working set rather than the whole catalog, and I'd add a short-TTL in-process cache on top since 200,000 reads a second makes even a Redis round trip add up. Writes are infrequent, so on update I'd just invalidate the affected key rather than keeping it warm. I'd also add stampede protection like request coalescing, since a hot product's cache expiring under this much traffic could otherwise spike the database, plus TTL jitter and hit ratio monitoring to catch regressions."

## Q24. [Senior] Your cache and database can diverge under write-back caching if the cache node crashes before flushing. How would you address durability there?

**Answer:**

Write-back (write-behind) caching acknowledges a write as soon as it's in the cache, then flushes to the database asynchronously later. **This gives great write latency but creates a durability gap, since any write sitting in the cache but not yet flushed is lost if that cache node crashes.**

**Ways to close the gap**

1. **Persist the write-back queue**: instead of holding pending writes only in memory, back them with a durable, append-only log (similar to a write-ahead log) so a crashed node can replay unflushed writes on recovery.
2. **Replicate before acknowledging**: require the write to be replicated to at least one other cache node before returning success to the client, so a single node crash doesn't lose data, similar to how replicated databases handle acknowledgment.
3. **Shorten the flush interval**: flushing more frequently, or after every N writes, shrinks the window of unflushed data at the cost of some of the batching efficiency that made write-back attractive in the first place.
4. **Only use write-back for tolerant data**: reserve write-back for data where losing the last few seconds of updates is acceptable, like view counters or non-critical analytics, and use write-through for anything that must never be lost, like financial transactions.
5. **Idempotent replay with monitoring**: make the flush operation idempotent so a crash-and-retry doesn't double apply a write, and alert if the queue backlog grows unexpectedly, which could signal the database is falling behind.

**Interview Answer:**
"The core issue is that write-back acknowledges before durably persisting, so I'd close that gap either by replicating the write to a second cache node before acknowledging, similar to how a replicated database confirms a write, or by backing the pending write queue with a durable log that can be replayed after a crash. I'd also make sure the flush to the database is idempotent so a replay after a crash can't double-apply a write. And I'd be selective, only using write-back for data where a small loss window is genuinely acceptable, not for anything like financial state."

---

## Q25. [Senior] Walk me through how you'd design a multi-level cache: local in-process cache plus a shared distributed cache. What goes wrong if you get the invalidation between levels wrong?

**Answer:**

**A multi-level cache puts a small, very fast local cache on each application instance in front of a larger shared distributed cache, which sits in front of the database.**

**Design**

1. **L1 (local, in-process)**: holds the hottest subset of data per instance, extremely low latency since there's no network hop, but limited in size and not shared across instances.
2. **L2 (distributed, shared)**: holds a much larger working set, shared consistently across all app instances, one network hop away.

3. **Read path**: check L1 first, then L2 on an L1 miss, then the database on an L2 miss, populating both L1 and L2 on the way back.
4. **TTLs**: L1's TTL should generally be shorter than L2's, since L1 is harder to invalidate promptly across many instances.

**What goes wrong with bad invalidation**

- If a write invalidates L2 but not L1, every app instance keeps serving its locally cached stale value from L1 until its own TTL expires, even though the shared cache is already correct. This is worse than a single-level cache miss because the staleness window is now determined by however many instances exist and how long each one's L1 TTL is.
- Broadcasting an invalidation to every instance's L1 (for example via pub/sub) fixes this but adds complexity and a new failure mode: if an instance misses the invalidation message, for example due to a dropped connection, its L1 silently stays stale until TTL expiry with no other safety net.
- The safest default is to keep L1 TTLs short enough that staleness is bounded and acceptable even without perfect invalidation propagation, since trying to guarantee perfect multi-instance invalidation adds a lot of complexity for marginal benefit in most cases.

**Interview Answer:**
"I'd put a small, short-TTL in-process cache in front of a larger shared distributed cache, checking L1 first, then L2, then the database, and populating both levels on the way back. The dangerous failure mode is invalidating L2 on a write but forgetting L1, since every app instance then keeps serving its own stale local copy independently, and the staleness window becomes however long each instance's L1 TTL is, which is worse than a plain cache miss. I'd either broadcast invalidations to all instances via pub/sub, or more simply just keep L1's TTL short enough that bounded staleness is acceptable without needing perfect propagation."

---

## Q26. [Senior] A single Redis key that holds a trending topic's count is being hit by 40 percent of your total traffic. Your Redis node's CPU is pegged. Walk me through how you'd fix this hot key without just resharding everything.

**Answer:**

**Resharding won't help here because the problem isn't distribution across keys, it's that one key is overwhelmingly popular, and a single key always lives on one node regardless of how many nodes exist in the cluster.**

**Diagnosis first**

1. Confirm it's genuinely a single-key hot spot using Redis's slow log or a monitoring tool that tracks per-key command rates, not just aggregate CPU.
2. Check whether the reads are simple GETs or something more expensive, like a sorted set operation, since the fix differs.

**Fixes**

1. **Local caching**: add a very short TTL, in-process cache on each application server for this specific key. If the count only needs to be accurate to within a second or two, this alone can eliminate the vast majority of the load reaching Redis.
2. **Key sharding (fan-out)**: split the single logical key into multiple physical keys, like `trend:count:0` through `trend:count:9`, have writes increment a randomly chosen shard, and have reads sum across all shards (or read a cached, periodically recomputed sum). This spreads the load for a single logical value across multiple nodes.

3. **Read replicas**: if the cache is set up with replication, route reads for this key across replicas instead of the primary, since reads don't need to go to the same node as writes.
4. **Client-side batching**: if many reads are coming from the same small set of callers, batch or debounce their requests instead of each one hitting Redis independently.

**Interview Answer:**

"Since it's one key overwhelming one node, adding more nodes to the cluster wouldn't help at all, the fix has to change how that specific key is accessed. My first move would be a short-TTL local cache on each app server, since trending counts usually tolerate a second or two of staleness, and that alone can absorb most of the load before it reaches Redis. If it's still too hot, I'd shard the counter itself into several physical keys that writes randomly distribute across, and either sum them on read or maintain a periodically recomputed total, which spreads a single logical value's load across multiple nodes."

---

## Q27. [Senior] How would you use cache warming to protect a service from cold-start thundering herd after a deploy or a cache cluster restart?

**Answer:**

Cache warming means proactively populating a cache with data before it starts serving live traffic, instead of letting the first wave of real requests discover an empty cache and stampede the database.

**Why cold starts are dangerous**

**When a fresh cache cluster comes online, or a deploy replaces cache nodes, every key starts as a miss.** If traffic resumes at full volume immediately, the database suddenly has to serve every request directly, exactly like a massive stampede across the entire key space rather than just one key.

**Approaches**

1. **Pre-load known hot keys**: before routing production traffic to a new cache instance, run a script that reads the most frequently accessed keys, based on historical access logs or a stored list of hot IDs, and populates them ahead of time.
2. **Replicate from an existing warm node**: if this is a rolling deploy rather than a full cluster wipe, replicate data from still-warm nodes into new ones before shifting traffic, rather than starting empty.
3. **Gradual traffic ramp-up**: use a load balancer to slowly increase the percentage of traffic routed to the new cache instance, giving it time to warm up organically under lighter load rather than being hit at full volume immediately.
4. **Fallback throttling on the database**: as a safety net, add a rate limiter or circuit breaker in front of the database so that even if warming isn't perfect, a request surge can't fully overwhelm it, degrading gracefully instead of failing entirely.

**Interview Answer:**

"I'd avoid ever routing full production traffic to a completely empty cache. Before cutting over, I'd pre-populate the new cache with the known hot keys from historical access patterns, and if it's a rolling deploy, I'd replicate from the still-warm nodes instead of starting from scratch. I'd also ramp traffic to the new instance gradually rather than all at once, so it can absorb some organic warming under lighter load. As a last line of defense, I'd put a circuit breaker or rate limiter in front of the database so an imperfect warm-up degrades gracefully instead of taking the database down."

## Q28. [Senior] Explain negative caching: caching the fact that something doesn't exist. When is that a good idea, and what risk does it introduce?

**Answer:**

**Negative caching means storing a marker for a lookup that returned "not found," so that repeated lookups for the same missing key are served from the cache instead of re-querying the database every time.**

**When it's a good idea**

- When clients repeatedly ask for keys that legitimately don't exist, such as a user checking whether a username is available, or a mobile app polling for a resource that hasn't been created yet.
- When the space of possible invalid keys is large enough that these lookups add real load, but not so large that a bloom filter would be a better fit.
- Anywhere the "not found" state doesn't need to be discovered instantly the moment it changes.

**The risk it introduces**

- If the entity is created shortly after a negative cache entry was written, the cache will keep saying "not found" for the remainder of the TTL, even though the data now exists. This can look like a real bug to users, for example a freshly created account appearing not to exist for a short window.
- The fix is to keep negative cache TTLs short, and to explicitly invalidate the negative cache entry at the moment the entity is actually created, the same way you'd invalidate any other cache entry on a write.

**Interview Answer:**
"Negative caching means storing the result 'this doesn't exist' so repeated lookups for the same missing key don't all hit the database. It's a good fit when legitimate traffic keeps asking about things that aren't there, like username availability checks. The risk is that if the entity gets created right after, the cache keeps insisting it doesn't exist until the TTL runs out, which looks like a bug to the user. I'd manage that by keeping the negative TTL short and explicitly clearing the negative cache entry as part of the creation flow, the same way I'd invalidate any other cache key on write."

---

## Q29. [Senior] You need a rate limiter or a leaderboard on top of a distributed cache. What consistency issues do you run into, and how would Redis's data structures help?

**Answer:**

**Both rate limiters and leaderboards need atomic read-modify-write operations under high concurrency, which is exactly where naive caching approaches break down.**

**The consistency problem**

- A rate limiter that reads a counter, checks it against a limit, then increments it in three separate steps has a race condition: two concurrent requests can both read the count before either increments it, both see it under the limit, and both proceed, letting more requests through than the limit allows.
- A leaderboard built as a plain sorted list has a similar problem: updating a score and re-sorting isn't atomic, so concurrent score updates can corrupt the ranking or drop updates.

**How Redis's data structures help**

1. **Atomic counters**: Redis's `INCR` command increments and returns the new value in a single atomic operation, eliminating the read-then-write race for simple rate limiting.
2. **Sliding window rate limiting**: Redis sorted sets can store request timestamps as scores, letting you atomically remove entries outside the window and count what's left using `ZREMRANGEBYSCORE` and `ZCARD`, giving an accurate sliding window without separate read and write steps racing each other.
3. **Sorted sets for leaderboards**: `ZADD` and `ZINCRBY` update a member's score atomically, and `ZRANGE` or `ZREVRANGE` retrieve ranked results directly from the data structure without needing to sort anything in the application.
4. **Lua scripting**: for logic that needs multiple commands to happen atomically together, like checking a limit and conditionally incrementing it, Redis can execute a Lua script as a single atomic unit, removing the race entirely.

**Interview Answer:**

"The core issue is that naive read-then-write logic for counters or rankings has race conditions under concurrency. Redis solves this by giving me atomic primitives instead of relying on my own read-modify-write logic. For rate limiting, INCR gives an atomic counter, and sorted sets let me implement an accurate sliding window using ZREMRANGEBYSCORE and ZCARD atomically. For leaderboards, sorted sets with ZADD and ZINCRBY handle atomic score updates and ranked retrieval natively. Where I need multiple steps to be atomic together, I'd push that logic into a Lua script so Redis executes it as a single unit."

---

## Q30. [Senior] Design the eviction and memory management strategy for a Redis cluster running near its memory limit, serving a mixed workload of large and small values.

**Answer:**

**Running near the memory limit requires deciding what Redis should do when it can't fit new data, and making sure the wrong things don't get evicted.**

**Steps**

1. **Choose an eviction policy deliberately**: Redis supports several `maxmemory-policy` settings. `allkeys-lru` evicts any key based on recency, `volatile-lru` only evicts keys that have a TTL set, `allkeys-lfu` and `volatile-lfu` use frequency instead. For a mixed cache workload, `allkeys-lru` or `allkeys-lfu` is usually right, since most cached data should be evictable.
2. **Separate critical data from evictable cache data**: if some keys must never be evicted (for example, a small amount of durable state accidentally stored in the same Redis instance), keep them without a TTL and use `volatile-lru`, so eviction only ever touches keys explicitly marked as cache.
3. **Address large value skew**: a workload mixing large and small values means a few large keys can dominate memory usage disproportionately. Identify oversized keys using Redis's memory usage command, and consider whether they should be split, compressed, or moved to a separate cache instance sized specifically for large objects, so evicting them doesn't unfairly starve small, frequently accessed keys.
4. **Right-size and shard**: if the working set genuinely doesn't fit in one node's memory even with eviction, shard the keyspace across more nodes with consistent hashing rather than relying on eviction to paper over insufficient total capacity.

5. **Monitor eviction rate**: a nonzero eviction rate isn't automatically bad, but a rising eviction rate alongside a falling hit ratio signals the cache is undersized for its working set and needs more memory or more nodes, not just a policy tweak.

**Interview Answer:**
"I'd start by picking an explicit eviction policy rather than relying on defaults, likely allkeys-lru or allkeys-lfu since this is a pure cache workload where most keys should be evictable. For the large-value skew, I'd check per-key memory usage and consider isolating oversized values into a separate instance so they don't crowd out small, frequently accessed keys through eviction. I'd also track eviction rate alongside hit ratio, since a rising eviction rate with a dropping hit ratio tells me the cluster is genuinely undersized and needs more capacity or sharding, not just a different policy."

---

## Q31. [Senior] Walk me through how you'd add jitter to TTLs across millions of cache keys, and why a naive fixed TTL for all keys set at the same time is dangerous.

**Answer:**

**A fixed TTL applied uniformly is dangerous specifically when many keys get created or refreshed at the same moment, because they then all expire at the same moment too, recreating a large-scale thundering herd across the entire batch instead of one key.**

**Why this happens in practice**

- A common trigger is a bulk cache warm-up or a deploy that repopulates a large chunk of the cache at once, all with the same TTL. Hours later, they all expire together, and if traffic is still high at that moment, the database gets hit with a synchronized wave of misses.

**How to add jitter**

1. Instead of setting `TTL = base_ttl` for every key, set `TTL = base_ttl + random(0, jitter_window)`, where the jitter window is some fraction of the base TTL, commonly 10 to 20 percent.
2. This spreads expirations out over a window instead of a single instant, so misses trickle in rather than arriving all at once.
3. For keys that are refreshed proactively before expiry (stale-while-revalidate style), the same jitter idea applies to the refresh trigger, not just the hard expiry, so background refreshes also don't cluster together.
4. Make sure the jitter is applied once, at write time, and stored as the actual TTL rather than recalculated randomly on every check, so behavior is deterministic per key.

**Interview Answer:**
"A fixed TTL is dangerous when a batch of keys gets written at the same time, because they'll all expire at the same instant later, turning what should be routine cache misses into a synchronized stampede across the whole batch. I'd fix this by adding randomized jitter on top of the base TTL when each key is written, something like base TTL plus a random amount up to 10 or 20 percent of it, so expirations spread out over a window instead of landing all at once. I'd apply the same idea to any proactive refresh logic, not just the hard expiry, so background refreshes don't cluster together either."

---

## Q32. [Senior] Compare strong consistency and eventual consistency for a distributed cache backing a financial balance display. Which would you choose, and what would you build around it?

**Answer:**

**This is a case where the usual caching instinct to favor availability and performance runs directly into a domain where correctness matters more than speed.**

**Options**

**Strong consistency**

- Every read reflects the most recent write, typically achieved by reading through to the database for anything that must be exact, or by using a cache invalidation scheme with strict synchronous guarantees.
- Pros: users never see an incorrect balance, which matters enormously for trust and for avoiding actions taken on wrong information, like overdrawing based on a stale higher balance.
- Cons: higher latency, since you can't fully rely on serving from cache without some verification step, and more load on the database.

**Eventual consistency**

- Reads may return a slightly stale value for a bounded period after a write, relying on TTLs or asynchronous invalidation to eventually catch up.
- Pros: lower latency, less database load, simpler to scale.
- Cons: a user could see an outdated balance, which for money is a serious trust and correctness problem, not just a minor inconvenience.

**Best Choice**

A hybrid: cache aggressively for display purposes but never let a stale cached value be the basis for anything that moves money.

**Why?**

For simply showing a balance on a dashboard, brief staleness is usually tolerable if it's on the order of a second or two and clearly not the system of record. But any action that actually debits, transfers, or authorizes based on that balance must read the current value directly from the database, or from a cache with a strict, synchronous invalidation guarantee, inside the same transaction boundary as the action. Treating "display" and "authorize" as different consistency requirements avoids over-engineering the whole system for strong consistency while still protecting the parts that actually need it.

**Interview Answer:**

"I wouldn't apply one consistency model to the whole feature. For just displaying a balance, I'd tolerate a very short staleness window, a second or two cached, since it's low risk and gives good performance. But for anything that actually acts on that balance, like authorizing a transfer or a withdrawal, I'd bypass the cache and read the current value directly from the database inside that transaction, because serving a stale balance for an action that moves money is a correctness and trust problem, not just a minor UX issue. So the cache serves the read-heavy display path, and the source of truth guards every write-adjacent decision."

---

## Q33. [Mid] Redis executes commands on a single thread. How does it still handle hundreds of thousands of operations per second without that thread becoming a bottleneck?

**Answer:**

Redis's single-threaded command execution is not the design flaw it sounds like, it is closely tied to why Redis is fast in the first place.

**I/O multiplexing**

Redis runs an event loop built on the operating system's I/O multiplexing facilities, epoll on Linux and kqueue on BSD-based systems like macOS. A single thread can register interest in thousands of client sockets at once and only gets woken up when a socket actually has data ready to read or write, instead of blocking on each connection one at a time. That means one thread can service a huge number of concurrent connections, since it is never sitting idle waiting on a slow socket, it is always working on whatever is actually ready.

**Everything lives in memory**

Since Redis operates on in-memory data structures rather than going to disk for each command, individual operations are extremely cheap, often just pointer chasing and simple arithmetic. **A single thread doing pure in-memory work with no disk seeks in the hot path can execute an enormous number of commands per second before it becomes the limiting factor.**

**No lock contention or context switching**

Because only one thread ever touches the data structures at a time, Redis needs no locks, no mutexes, and no coordination overhead between threads for command execution. There is also no context-switching cost, which is a real expense in multithreaded systems where the CPU has to save and restore state as it jumps between threads. This makes performance highly predictable, since two clients never race for the same lock.

**Offloading expensive work**

Newer versions of Redis push genuinely slow operations onto background threads so they do not block the main thread. Freeing the memory of a very large key, for example, can happen asynchronously instead of stalling every other client.

`UNLINK bigkey`
`UNLINK` behaves like `DEL` but reclaims the key's memory on a background thread instead of blocking the main event loop, which matters a lot when the key is large.

**Interview Answer:**

"Redis stays single-threaded for command execution, but that's a big part of why it's fast, not a limitation. It uses epoll or kqueue to multiplex thousands of socket connections through one event loop, so the thread is always doing useful work instead of blocking. Since everything lives in memory, each command is cheap, and with only one thread touching data there's no lock contention or context-switching overhead. For the few operations that are genuinely expensive, like freeing a huge key, newer Redis versions offload that work to background threads with commands like UNLINK so the main loop never stalls."

---

## Q34. [Mid] Walk me through Redis's core data structures, strings, hashes, lists, sets, and sorted sets, and give me a real scenario where you'd reach for a sorted set instead of a plain key or a hash.

**Answer:**

Redis is often described as a data structure server rather than a plain key-value store, because the value behind a key can be one of several native types, each suited to a different access pattern.

**The core types**

- **Strings**: the simplest type, holding text, numbers, or binary data. Good for simple counters, flags, or cached blobs like a rendered HTML fragment.
- **Hashes**: a map of field-value pairs under a single key, ideal for representing an object, like a user profile, where you want to read or update individual fields without touching the rest.
- **Lists**: ordered collections of values, well suited to queues or recent-activity feeds where you push and pop from either end.
- **Sets**: unordered collections of unique members, useful for membership checks and set operations like intersection or union, for example finding common tags between two items.
- **Sorted sets**: like a set, but every member also has a score, and Redis keeps members ordered by that score automatically, giving fast range and rank queries.

#### When a sorted set beats a hash or a plain key

Picture a game leaderboard with millions of players. A hash could store player ID mapped to score, but getting "the top 10 players" or "this player's rank" out of a hash means pulling every entry and sorting it in the application, which gets slower as the player base grows. **A sorted set keeps every member ordered by score internally, so fetching the top N players or a specific player's rank is a fast, near-logarithmic lookup instead of a full scan and sort.**

```
ZADD leaderboard 15420 "player42"
ZREVRANGE leaderboard 0 9 WITHSCORES
ZRANK leaderboard "player42"
```

#### Interview Answer:

"Redis gives me strings for simple values, hashes for objects with multiple fields, lists for ordered queues, sets for unique unordered membership, and sorted sets for anything that needs ranking. I'd reach for a sorted set over a hash for something like a leaderboard, where I constantly need the top N entries or a specific member's rank. A hash would force me to pull everything and sort it in application code every time, while a sorted set keeps members ordered by score internally, so top-N and rank lookups are fast, native operations instead of an expensive scan."

---

## Q35. [Mid] What problem does pipelining solve in Redis, and why does batching commands even matter when Redis is already so fast?

#### Answer:

Without pipelining, a client sends a command, waits for the full round trip to the server and back, then sends the next one. **That round-trip latency, not Redis's internal processing time, is usually the real bottleneck when a client needs to run many commands back to back.**

#### The problem it solves

Say a client needs to run 10,000 `SET` commands. Sent one at a time, the client pays the network round-trip cost 10,000 times, even though Redis itself might execute each command in microseconds. If the client and server are in the same data center, a round trip might be under a millisecond, but that still adds up to seconds of pure waiting for 10,000 sequential commands, dwarfing the actual command execution time.

#### How pipelining fixes it

The client writes multiple commands to the socket without waiting for each individual reply, then reads all the responses once the server has processed them.

```
PING
SET foo bar
SET baz qux
```

```
GET foo
```

Sent as a pipeline, all four commands go out in one write, and Redis processes them in order and streams the four replies back, so the client pays for one round trip instead of four. This does not make each command execute faster inside Redis, it eliminates network latency that was being paid over and over for no reason.

**What it doesn't fix**

- Pipelining does not make commands atomic, other clients' commands can still be interleaved between pipelined commands unless it's combined with a transaction using `MULTI` and `EXEC`.
- Very large pipelines can use a lot of memory buffering replies, so batching in reasonably sized chunks, thousands rather than millions of commands at a time, is a common practice.

**Interview Answer:**

"Pipelining solves a round-trip latency problem, not a processing speed problem. Redis executes any single command in microseconds, but sending commands one at a time and waiting for each reply means paying network round-trip cost over and over, which usually dwarfs the actual execution time. Pipelining lets me write a batch of commands to the socket without waiting for each reply, then read all the responses at once, collapsing many round trips into one. It's worth remembering pipelining doesn't make commands atomic, if I need that guarantee, I'd combine it with MULTI and EXEC."

---

## Q36. [Senior] Compare RDB snapshotting and AOF persistence in Redis. What's the durability versus performance trade-off, and when would you run both together?

**Answer:**

Redis offers two different persistence mechanisms, and they make very different trade-offs between how much data you can afford to lose and how expensive persistence is to maintain.

**RDB (Redis Database snapshot)**

Periodically writes the entire dataset to disk as a point-in-time binary snapshot, either on a schedule, like every 5 minutes if at least 100 keys changed, or on demand. It produces a compact single file that's fast to load on restart since Redis just reads it back into memory, and taking the snapshot has minimal impact on runtime performance since Redis forks a child process to write it. The downside is that any writes made after the last snapshot are lost if Redis crashes, so the durability window is only as good as the snapshot interval.

**AOF (append-only file)**

Logs every write operation to a file as it happens, and Redis can rebuild the entire dataset by replaying that log from the start. With `appendfsync everysec`, at most about a second of writes can be lost, and `appendfsync always` can approach zero data loss at a real cost to write throughput. The trade-off is that the log file grows continuously and needs periodic rewriting with `BGREWRITEAOF` to stay compact, and replaying a large AOF file on restart is slower than loading a single RDB snapshot.

**The trade-off**

**RDB optimizes for fast recovery and low runtime overhead at the cost of a wider data-loss window, while AOF optimizes for durability at the cost of a larger file and slower restart.**

**Running both together**

```
CONFIG SET appendonly yes
save 300 100
```

Since Redis 4.0, a mixed persistence mode lets the AOF file start with an RDB-format preamble, giving fast loading like RDB plus the fine-grained durability of AOF for everything written after that preamble. In

practice, most production setups keep RDB snapshots for fast full backups and disaster recovery, and enable AOF alongside it to minimize data loss between snapshots, accepting the modest extra disk and CPU cost.

**Interview Answer:**
"RDB takes periodic full snapshots, which load fast and have low runtime overhead but can lose everything written since the last snapshot. AOF logs every write, so with appendfsync everysec I only risk losing about a second of data, but the file is larger and slower to replay on restart. In production I'd usually run both: RDB for fast, compact backups and quick recovery, and AOF turned on for the tighter durability window, especially since Redis's mixed persistence format now lets AOF start from an RDB preamble, giving fast loading without giving up AOF's durability."

---

## Q37. [Senior] Walk me through Redis's eviction policies, noeviction, allkeys-lru, volatile-lru, allkeys-lfu, and so on. How do you pick the right maxmemory-policy for a given workload?

**Answer:**

When Redis hits its configured `maxmemory` limit, the `maxmemory-policy` setting decides what happens next, and picking the wrong one either breaks writes unexpectedly or evicts the wrong data.

**The policies**

- **noeviction**: Redis refuses new writes once memory is full, returning an error, while reads still work. Suited to Redis used as a primary data store where rejecting a write is safer than silently losing data.
- **allkeys-lru**: evicts the least recently used key across the entire keyspace, regardless of whether it has a TTL. Suited to a pure cache workload where every key is disposable.
- **volatile-lru**: evicts the least recently used key, but only considers keys that have a TTL set, leaving keys without a TTL untouched. Suited to mixed workloads where some keys are cache and some are meant to persist indefinitely.
- **allkeys-lfu / volatile-lfu**: same scope rules as their LRU counterparts, but evict based on access frequency instead of recency, which resists being fooled by a one-off scan through cold keys.
- **volatile-ttl**: evicts keys with a TTL, preferring the ones closest to expiring anyway, useful when proactively clearing near-expired data is preferable to picking by access pattern.
- **volatile-random / allkeys-random**: evicts a random key within the relevant scope, cheaper to compute than tracking recency or frequency, occasionally chosen when the access pattern is close to uniform.

**How to choose**

1. Decide first whether Redis is being used purely as a cache or is also holding data that must never be evicted. If it's mixed, an `allkeys-*` policy is dangerous, since it can evict durable, non-cache data.
2. For a pure cache, pick LRU if recency predicts future access, or LFU if there's stable long-term popularity that occasional scans shouldn't disturb.
3. For a mixed instance, use a `volatile-*` policy and make sure only genuinely evictable data carries a TTL, since only TTL'd keys are eligible for eviction under those policies.

```
CONFIG SET maxmemory 4gb
CONFIG SET maxmemory-policy allkeys-lfu
```

**Getting maxmemory-policy wrong on a mixed workload either silently evicts durable data under allkeys-lru, or the instance starts rejecting writes under noeviction when everyone assumed eviction was happening.**

**Interview Answer:**
"The first question is whether this Redis instance is a pure cache or holds a mix of durable and disposable data. For a pure cache, I'd pick allkeys-lru or allkeys-lfu depending on whether recency or long-term frequency better predicts what's worth keeping. For a mixed instance, I'd use a volatile-* policy and make sure only the genuinely disposable keys carry a TTL, since volatile policies only ever evict TTL'd keys. I'd avoid noeviction unless Redis is acting as a primary store where rejecting a write beats silently losing data."

---

## Q38. [Senior] How does Redis Cluster shard data across nodes using hash slots, and what actually happens during a resharding operation or when a node fails?

**Answer:**

Redis Cluster splits the entire keyspace into a fixed number of hash slots and distributes ownership of those slots across the master nodes in the cluster, rather than using a hash ring or consistent hashing.

### How sharding works

Redis Cluster always divides the keyspace into exactly 16,384 hash slots. **Every key is mapped to exactly one of those 16,384 slots by computing `CRC16(key) mod 16384`, and each master node owns a subset of the slots rather than an arbitrary hash range.** A client can ask any node which node currently owns a given slot, and modern clients cache that slot map so they can go directly to the right node on later requests.

- Keys that need to be manipulated together, or read in a single multi-key operation, can be forced onto the same slot using a hash tag: wrapping the part of the key that should determine the slot in curly braces, like `user:{1234}:profile` and `user:{1234}:orders`, so both land on the same node.

### Resharding

1. An operator, or a tool like `redis-cli --cluster reshard`, decides how many slots to move and from which node to which node.
2. The cluster marks the slots being moved as `MIGRATING` on the source node and `IMPORTING` on the destination node.
3. Keys within those slots are migrated one at a time, and during the transition, a client asking the source node for a key that has already moved gets an `ASK` redirect telling it to retry against the destination node.
4. Once every key in a slot has moved, ownership of that slot is reassigned entirely to the destination node, and clients get a `MOVED` redirect from then on.

```
redis-cli --cluster reshard 127.0.0.1:7000
```

### Node failure

Each master is typically paired with one or more replicas that asynchronously replicate its data. Nodes continuously gossip with each other, a gossip protocol has each node periodically exchange state with a few random peers so information spreads across the whole cluster without one central checker, to track cluster health, and if enough other nodes agree a master is unreachable past a timeout, the cluster considers it failed. One of that master's replicas is then automatically promoted to master, taking over ownership of its hash slots so the cluster keeps serving that portion of the keyspace. Any writes that hadn't yet replicated to the promoted replica at the moment of failure are lost, since replication in Redis Cluster is asynchronous by default, a real trade-off for write-heavy, failure-sensitive workloads.

**Interview Answer:**
"Redis Cluster divides the keyspace into 16,384 fixed hash slots, and each master owns a range of them, so a key's slot is just CRC16 of the key mod 16,384. During a reshard, slots being moved are marked MIGRATING

and IMPORTING while keys transfer one at a time, and clients get ASK redirects mid-move and MOVED redirects once a slot fully belongs to its new owner. On a node failure, the cluster's gossip protocol detects it and promotes one of that master's replicas to take over its slots automatically, though since replication is asynchronous, writes that hadn't replicated yet are lost in the failover."

---

**If you remember nothing else from this chapter:** *adding a cache is the easy ten percent, getting invalidation and stampede protection right is the hard ninety.*

# Chapter 9: CDNs, Proxies & DNS

These are the layers a request passes through before your own code ever runs, and they're easy to gloss over because they feel like "infrastructure, not my job." Use this chapter to make sure you actually understand what a CDN or proxy can and cannot fix, since a wrong assumption here quietly breaks designs for personalized or dynamic content.

> **War Story:** *A DNS TTL set to twenty-four hours "to reduce lookups" meant that during an emergency IP failover, a large share of traffic kept hitting the dead IP for most of a day.*

> **Avoid:** not knowing the difference between a forward and reverse proxy • treating TTL as a knob to just set high rather than a real trade-off
>
> **Aim For:** explaining TTL as an availability-versus-performance trade-off, not just a caching number

## Q1. [Junior] What is a CDN, and why would a website want one in front of it?

**Answer:**

A CDN (content delivery network) is a distributed set of servers spread across many physical locations that cache copies of your content and serve it to users from a location close to them, instead of every request traveling all the way to your origin server.

**Why it helps**

- Lower latency: a user in Tokyo requesting a site hosted in Virginia gets a response from a nearby edge server instead of crossing an ocean and back.
- Reduced origin load: cached responses are served from the edge, so far fewer requests ever reach your actual infrastructure.
- Better resilience: if your origin is slow or briefly unavailable, cached content at the edge can keep serving users.
- Basic traffic absorption: CDNs soak up traffic spikes and a lot of low-level DDoS noise before it reaches you.

**What it's good at**

CDNs are strongest for static or semi-static content: images, video, JavaScript and CSS bundles, and pages that look the same for every visitor. **They're not a magic layer that speeds up everything, which is a distinction that comes up a lot later in this chapter.**

**Interview Answer:**

"A CDN is a network of geographically distributed servers that cache your content close to users, so requests don't have to travel all the way back to your origin every time. I'd put one in front of a site mainly to cut latency for far away users, reduce load on my origin servers, and get some free resilience and DDoS absorption. It's most valuable for static assets like images, video, and JS or CSS bundles, less so for things that are unique per user."

## Q2. [Junior] Why do we need DNS at all? Why not just use IP addresses directly?

**Answer:**

**DNS (Domain Name System) exists because humans are bad at remembering numbers, and computers need numbers to route traffic.** DNS is the system that translates a human readable domain name like example.com into the IP address a computer actually needs to open a connection.

**What it buys you beyond memorability**

- Indirection: the IP address behind a domain can change (new servers, new datacenter, failover) without anyone needing to update a link or bookmark.
- Load distribution: a single domain name can resolve to multiple IP addresses, letting you spread traffic across servers or regions.
- Flexibility for CDNs and cloud providers: DNS is the mechanism that lets a CDN route different users to different edge servers, even though they're all typing the same domain name.

**Why this matters in interviews**

DNS is often treated as boring plumbing, but it's the layer that makes routing, failover, and geographic distribution possible without changing a single line of application code. Nearly every CDN, load balancer, and geo-routing scheme in this chapter is built on top of DNS indirection.

**Interview Answer:**
"DNS translates human friendly domain names into IP addresses because that's what machines actually need to route traffic. But the real value is indirection: since a domain can point to different IPs over time or based on where you're asking from, DNS becomes the mechanism behind load balancing, failover, and CDN routing. Without it, every IP change would require updating every client, bookmark, and hardcoded reference, which just isn't workable at internet scale."

---

## Q3. [Junior] What's the difference between a forward proxy and a reverse proxy?

**Answer:**

**Both proxies sit between a client and a server and relay traffic, but they represent different parties and solve different problems.** For example, a company's corporate network uses a forward proxy so every employee's outbound requests appear to come from one company IP, while a website uses a reverse proxy so every visitor's request appears to be handled by one address even though it's actually routed to one of many backend servers.

**Forward proxy**

- Sits in front of the client, representing the client to the outside world.
- The server the client is talking to often doesn't know, or doesn't care, that a proxy is involved.
- Common uses: corporate networks filtering or logging outbound traffic, bypassing geographic restrictions, anonymizing a client's IP address, or caching outbound requests for a group of users.

**Reverse proxy**

- Sits in front of the server, representing the server to the outside world.
- The client doesn't know, or doesn't care, which backend server actually handled the request.
- Common uses: load balancing across many backend instances, TLS termination, request routing, caching responses, and hiding internal topology from the internet.

**The simplest way to remember it**

A forward proxy protects and represents the client. A reverse proxy protects and represents the server. Both sit in the middle of the connection from a pure networking standpoint, just facing opposite directions, which is different from the malicious 'man-in-the-middle' attack that term usually brings to mind.

**Interview Answer:**
"A forward proxy sits in front of the client and represents the client to whatever server it's talking to, like a corporate proxy that filters or logs employee traffic. A reverse proxy sits in front of the server and represents the server to the outside world, like nginx or an ALB routing requests to backend instances while hiding them from the internet. The direction of who they represent is really the whole distinction."

---

## Q4. [Junior] What does TTL mean for a DNS record, and what's the trade-off between setting it high versus low?

**Answer:**

TTL (time to live) on a DNS record tells resolvers and clients how long they're allowed to cache that record's answer before they need to ask again. It's set in seconds on the record itself, at the authoritative nameserver.

**High TTL (for example, 24 hours)**

- Pros: fewer DNS lookups overall, faster average resolution for users since answers are already cached, less load on your nameservers.
- Cons: if you need to change the IP behind that domain (failover, migration, provider switch), it can take up to that full TTL for all cached resolvers worldwide to pick up the change.

**Low TTL (for example, 60 seconds)**

- Pros: changes propagate quickly, which matters a lot for failover and blue-green style cutovers.
- Cons: many more DNS queries hit your nameservers, and every client pays a small resolution cost more often.

**How to think about it in practice**

**Most stable production records use TTLs in the minutes to hours range.** Teams often deliberately lower the TTL ahead of a planned change, like a migration or failover drill, then raise it back afterward once things are stable.

**Interview Answer:**
"TTL controls how long a DNS answer can be cached before a resolver has to look it up again. A high TTL means fewer lookups and faster average resolution, but slow propagation if I need to change the underlying IP. A low TTL means fast propagation for things like failover, but more DNS query volume. In practice I'd run a moderate TTL day to day, and deliberately lower it in advance of any planned migration or failover."

---

## Q5. [Junior] What's a PoP, and why does a CDN operate many of them around the world instead of one big datacenter?

**Answer:**

A PoP (point of presence) is a physical location where a CDN, or any network provider, has edge servers, routers, and network equipment that connect to the broader internet. A CDN might operate hundreds of PoPs across cities and regions worldwide.

**Why many small PoPs beat one big datacenter**

- Latency is fundamentally limited by the speed of light and distance. A single centralized datacenter, no matter how powerful, can't beat a nearby PoP on round trip time for a user thousands of miles away.
- Distributing PoPs geographically means most users are only a short network hop away from cached content, rather than crossing continents.
- Redundancy: if one PoP has an outage or gets overwhelmed, traffic can be rerouted to the next nearest PoP instead of the whole service going down.
- Localized traffic absorption: spikes or attacks in one region get absorbed locally instead of overwhelming a single point of failure.

**The trade-off**

Running many PoPs is operationally more complex than running one datacenter: you need cache invalidation across all of them, consistent configuration, and routing logic to send users to the right one. **That complexity is the price of low latency at global scale.**

**Interview Answer:**

"A PoP is a physical edge location where a CDN keeps servers close to end users. CDNs run many of them, sometimes hundreds, because latency is bounded by physical distance, and no single datacenter can be close to everyone on earth. Spreading PoPs out also gives redundancy and localizes traffic spikes. The cost is operational complexity, since you now need to keep caches and configuration consistent across every one of those locations."

---

## Q6. [Junior] If a CDN is caching your homepage and you update the content, does the CDN pick up the change automatically? What actually happens?

**Answer:**

**Not automatically, and not immediately, unless you tell it to.** A CDN caches a response for as long as its cache rules say it should, and it has no built-in awareness that your origin content changed underneath it.

**What determines the behavior**

- Cache headers: your origin sets headers like Cache-Control and its max-age, which tell the CDN how long it's allowed to serve the cached copy before checking back with the origin.
- Until that TTL expires, the CDN will keep serving the old cached version to users, even though the origin has the new content ready.
- Once the TTL expires, the next request triggers a fresh fetch from the origin (or a conditional check), and the CDN updates its cached copy.

**How to make the update show up sooner**

- Invalidate or purge the specific cached object at the edge, forcing every PoP to drop it and fetch fresh content on the next request.
- Use versioned URLs or filenames for assets (like main.abc123.js) so a new deploy is a new URL, sidestepping the staleness question entirely for that asset.

**Interview Answer:**

"No, a CDN has no idea your content changed, it just knows what its cache headers told it. If the homepage was cached with a max-age of, say, one hour, users keep getting the old version until that TTL expires or

someone triggers a purge. For anything time sensitive, I'd either issue a cache invalidation on deploy or use versioned file names so new content gets a new URL and avoids the staleness question altogether."

---

## Q7. [Mid] Compare push CDNs and pull CDNs. Which would you choose for a video streaming platform launching a large new catalog, and why?

**Answer:**

**Push CDN**

- You (the origin) proactively upload content to the CDN's edge servers ahead of time, before any user requests it.
- Pros: content is ready and warm at every PoP before traffic arrives, no cold cache misses on first request, you control exactly what's distributed and when.
- Cons: you pay to store and distribute content that might never actually be requested in some regions, and you're responsible for managing uploads, updates, and expiration yourself.

**Pull CDN**

- The CDN fetches content from your origin lazily, on the first request for that object, then caches it for subsequent requests.
- Pros: much simpler to operate, you don't manage distribution, storage on the edge naturally reflects what's actually popular.
- Cons: the very first request for any object at any PoP is a cache miss, meaning that user pays the full origin round trip latency, and a spike of first-time requests can hit the origin hard.

**Best Choice**

**Push CDN for the video streaming catalog launch.**

**Why?**

A large catalog launch is a known, scheduled event with predictable content, exactly the situation where pre-warming pays off. Video files are large, and a cold cache miss on a multi-gigabyte file at launch time, multiplied across every PoP as users log in simultaneously, would create a painful thundering herd on the origin and slow first-play time badly. Pushing the catalog out ahead of the launch means every PoP is already warm when traffic actually arrives. For catalogs that update less predictably or where storage cost matters more than launch day experience, pull would be the more practical everyday default.

**Interview Answer:**
"Push CDNs pre-upload content to edge servers ahead of demand, so nothing is ever a cold cache miss, but you pay for storage and manage distribution yourself. Pull CDNs fetch lazily on first request, which is simpler to run but means the very first user anywhere pays a full origin round trip. For a scheduled catalog launch with known, large video files, I'd push the content out ahead of time so every PoP is warm the moment users start streaming, avoiding a thundering herd on the origin at launch."

---

## Q8. [Mid] How does a CDN decide which edge server or PoP handles a given user's request?

**Answer:**

**There are two main mechanisms, often layered together, and the exact answer depends on the CDN's architecture.**

**DNS based routing**

1. The user's resolver asks the CDN's authoritative nameserver to resolve the CDN's hostname.
2. The nameserver looks at where the query is coming from (the resolver's IP, sometimes with EDNS Client Subnet, a DNS extension that forwards a bit of the actual user's IP range to get closer to their real location) and returns the IP address of a nearby PoP.
3. The client connects directly to that PoP's IP for all subsequent requests until the DNS TTL expires.

**Anycast routing**

- Many CDNs advertise the same IP address from multiple PoPs simultaneously using BGP anycast (the internet's routing protocol automatically sends each user to whichever advertised location is network-wise closest to them).
- Internet routing itself (not DNS) naturally sends the user's packets to whichever PoP is "closest" in terms of network hops, without any special DNS logic needed.
- This tends to react faster to network conditions and outages than DNS TTL based approaches.

**What factors influence the choice at either layer**

- Geographic proximity and network latency.
- PoP health and current load, unhealthy or overloaded PoPs get avoided.
- Sometimes cost or contractual routing rules between the CDN and upstream ISPs.

**Interview Answer:**

"CDNs typically use either DNS based routing, where the authoritative nameserver looks at where a query originates and returns the IP of a nearby, healthy PoP, or anycast routing, where multiple PoPs advertise the same IP and internet routing itself sends users to the closest one by network hops. Anycast tends to react faster to outages since it doesn't depend on DNS TTLs expiring. Most large CDNs combine both, using load and health checks to keep steering traffic away from struggling PoPs."

---

## Q9. [Mid] Explain geo-DNS and latency-based routing. How do they differ, and when would you pick one over the other?

**Answer:**

**Geo-DNS**

- Routes based on the geographic location of the requester (usually inferred from their resolver's IP), returning the IP of a server in or near that geographic region.
- Simple to reason about and configure: "users in Europe go to the Frankfurt cluster, users in Asia go to the Singapore cluster."
- Doesn't account for actual network conditions, a geographically close server can still have worse real world latency than a farther one if the network path is bad or congested.

**Latency-based routing**

- Routes based on measured or historical latency data between the resolver's location and each candidate endpoint, not just geographic distance.
- Can route a user to a server that's technically farther away but faster to reach, because of better peering or a healthier network path.
- Requires the routing provider to continuously measure and maintain latency data across regions, more operationally complex to run.

**Best Choice**

Latency-based routing for most global, latency-sensitive production traffic; geo-DNS when you have hard requirements tied to geography itself, like data residency or content licensing by country.

**Why?**

**Latency based routing directly optimizes for the thing users actually feel, response time, rather than a proxy for it, geographic distance.** But geo-DNS remains the right tool when the constraint isn't about speed at all, for example legal requirements that EU user data must be served from EU infrastructure. In that case, geography is the actual requirement, and routing purely on measured latency could accidentally violate it.

**Interview Answer:**

"Geo-DNS routes users based on where they physically are, which is simple but assumes proximity equals speed, which isn't always true. Latency-based routing measures actual network performance between regions and endpoints and optimizes for that directly, so it can send someone to a farther but faster path. I'd default to latency-based routing for performance, but fall back to geo-DNS whenever there's a hard geographic requirement, like data residency rules that mean EU traffic must legally stay on EU infrastructure."

---

## Q10. [Mid] You ship a critical CSS fix, but users report seeing the old broken styles for another hour. Walk me through what's likely happening and what you'd do about it.

**Answer:**

**Most likely cause**

**The CSS file is being served through a CDN (or possibly a browser cache) with a Cache-Control max-age that hasn't expired yet.** The origin has the new file, but every PoP, and possibly every user's browser, is still confidently serving its own cached copy because nothing has told it to check again.

**How to confirm**

1. Check the response headers for the CSS file, both from the origin directly and through the CDN, looking at Cache-Control, Age, and any CDN specific cache status header.
2. Compare the Age header value against the max-age, if Age is smaller than max-age, the cached copy is still considered valid.
3. Check whether the browser is even reaching the CDN or serving straight from local disk cache.

**Immediate fix**

- Issue a cache purge or invalidation for that specific asset across all PoPs so the next request forces a fresh fetch from origin.
- If browser caching is also a factor, this won't clear existing browsers' local caches, only new fetches.

**Longer term fix**

- Move to versioned or fingerprinted filenames for static assets (style.a1b2c3.css) so every deploy is a brand new URL with its own fresh cache entry, and old URLs simply age out naturally.
- Reserve short TTLs, or must-revalidate, for assets you expect to change without a filename change, like an HTML entry point.

**Interview Answer:**

"That's almost certainly a caching TTL issue, either at the CDN edge or in users' browsers, where the old CSS is still within its max-age window so nothing has gone back to origin to fetch the new version. I'd confirm by checking the Cache-Control and Age headers, then do an immediate CDN purge to force a refresh. Longer term, I'd move static assets to fingerprinted filenames tied to content hash, so a new deploy is automatically a new URL and this class of bug stops happening."

---

## Q11. [Mid] Walk me through the full DNS resolution process, starting from a browser's cache miss, all the way to getting back an IP address.

**Answer:**

**Step by step**

1. **Browser cache check**: the browser first checks if it already has a cached answer for this hostname from a recent lookup. If found and not expired, resolution ends here.
2. **OS resolver cache**: if the browser doesn't have it, the operating system checks its own local DNS cache.
3. **Recursive resolver query**: if still no answer, the request goes to a configured recursive resolver, often the ISP's resolver or a public one like a well known third party resolver. This resolver does the heavy lifting on the client's behalf.
4. **Root nameserver**: the recursive resolver, if it has nothing cached, asks a root nameserver, which doesn't know the answer but points it to the correct TLD (top level domain) nameserver, for example the one responsible for ".com".
5. **TLD nameserver**: the resolver asks the TLD nameserver, which responds with the authoritative nameserver responsible for the specific domain.
6. **Authoritative nameserver**: the resolver asks that authoritative nameserver directly, which returns the actual DNS record, an A or AAAA record with the IP address (or a CNAME to follow further).
7. **Caching and response**: the recursive resolver caches this answer for the record's TTL, then returns the IP address back to the OS, which returns it to the browser.
8. **Connection**: the browser now opens a TCP (or QUIC) connection to that IP address and the actual HTTP request proceeds.

**Why this matters**

Every one of these hops can be a cache hit that shortcuts the chain, which is why most real world DNS lookups are fast, they rarely walk the full root to authoritative chain. **Understanding this chain also matters for reasoning about propagation delays and where a stale entry can be hiding.**

**Interview Answer:**

"On a full cache miss, the chain goes browser cache, then OS cache, then out to a recursive resolver, which if it has nothing cached asks a root nameserver for the right TLD server, asks that TLD server for the domain's authoritative nameserver, then asks that authoritative server for the actual record. The IP comes back up the chain, gets cached at the resolver for the TTL, and the browser finally opens a connection. In practice most lookups short circuit early because something along that chain already has it cached."

---

## Q12. [Mid] What is a reverse proxy actually doing on the request path in a production system, beyond "forwarding requests"?

**Answer:**

A reverse proxy sits in front of your application servers and does real work on every request, not just pass-through relaying.

**Core responsibilities**

- **Load balancing**: distributing incoming requests across a pool of backend instances using a strategy like round robin, least connections, or consistent hashing.
- **TLS termination**: handling the HTTPS handshake and decryption once at the proxy, so backend servers can speak plain HTTP internally, simplifying certificate management to one place.
- **Request routing**: directing traffic to different backend services based on path, hostname, or headers, useful for microservices and gradual migrations.
- **Caching**: storing responses for repeated identical requests so backends don't redo the same work.
- **Compression**: gzip or brotli compressing responses before they leave the datacenter.
- **Security filtering**: rate limiting, blocking malicious patterns, hiding internal topology and server versions from the outside world, and acting as a buffer against certain classes of attack.
- **Observability**: centralizing access logs, request tracing, and metrics in one layer instead of every backend service handling it independently.

**Why this matters**

**A reverse proxy is effectively a policy enforcement point sitting between the internet and your infrastructure.** Removing it usually means duplicating all of this logic (TLS handling, rate limiting, routing) into every individual backend service, which is both more work and more places for security gaps to appear.

**Interview Answer:**

"Beyond forwarding, a reverse proxy is doing load balancing across backend instances, terminating TLS in one place instead of every server managing its own certificates, routing based on path or host for things like microservices, caching repeated responses, and providing a security boundary with rate limiting and hidden internal topology. It's really a centralized policy enforcement layer sitting between the internet and my application fleet, and pulling it out would mean re-implementing all of that in every backend service."

---

## Q13. [Mid] Can a CDN speed up an API endpoint that returns personalized, per-user data? Why or why not?

**Answer:**

**Short answer**

**Generally no, at least not by caching the response itself, because the fundamental thing a CDN does is serve a shared cached copy to many requesters, and personalized data by definition isn't shareable across users.**

**Why it breaks down**

- If the response differs for every user (their name, their cart, their feed), caching it at the edge and serving it to the next requester would leak one user's data to another, which is a serious bug, not just a missed optimization.
- Even with a Vary header (which tells a cache to store a separate copy per value of a given request header) keyed on user ID, you'd end up with essentially one cache entry per user, giving you all the storage and invalidation overhead of a cache with almost none of the hit rate benefit, since each entry might only ever be requested once.

**Where a CDN can still help, even here**

- Terminating TLS and absorbing connection overhead at a nearby edge PoP, then using an optimized, persistent connection back to origin, which cuts network layer latency even for uncacheable responses.
- Serving the static shell of a page (HTML template, JS, CSS, images) from cache while the personalized data loads via a separate, uncached API call.
- Edge compute (like Cloudflare Workers or Lambda@Edge) can do lightweight personalization at the edge itself for simple cases, without a full round trip to origin.

**Interview Answer:**
"Not through response caching, no, because personalized data is by definition not shareable, and caching it at the edge risks serving one user's data to another. What a CDN can still do is speed up the connection layer, terminating TLS close to the user and using a fast, persistent link back to origin, and it can cache the static shell of the page separately from the personalized parts. But the actual personalized payload has to go back to origin every time."

---

## Q14. [Mid] What's the difference between a browser cache and a CDN edge cache, and how do their TTLs interact?

**Answer:**

**Browser cache**

- Lives on the individual user's device.
- Only that one user benefits from a cached entry, it's private to their browser.
- Controlled by the same Cache-Control headers the origin (or CDN) sends, plus the browser's own heuristics if headers are missing. For example, ten thousand different users hitting the same product image within a PoP's TTL window all get served from that one shared CDN edge cache entry, while each of those users also keeps their own private copy in their own browser cache alongside it.

**CDN edge cache**

- Lives at the CDN's PoPs, shared across every user who happens to be routed to that PoP.
- One cache miss and fetch benefits potentially thousands of subsequent users hitting that same PoP.
- Also controlled by Cache-Control headers, but the CDN may apply its own separate cache rules on top, sometimes distinguishing browser facing TTL from edge facing TTL using headers like s-maxage.

**How they interact**

1. **A response can carry both max-age (for browsers) and s-maxage (specifically for shared caches like CDNs), letting you set different lifetimes for each layer.**
2. Typically you'd want the CDN's cache to be at least as long lived as the browser's, since it serves many more users and refreshing it is relatively cheap for the CDN but still requires an origin round trip.
3. If a resource is purged at the CDN but a user's browser still has its own separate copy cached, that user keeps seeing the old version until their browser cache separately expires, which is a common source of "I purged it but it's still stale" confusion.

**Interview Answer:**
"A browser cache is private to one user's device, while a CDN edge cache is shared across every user hitting that PoP, so a single cache fill benefits far more people. Both respect Cache-Control, but you can differentiate them using max-age for browsers and s-maxage for shared caches like CDNs. The tricky part is that purging the CDN doesn't touch anyone's browser cache, so a user can still see stale content from their own local cache even after an edge purge."

## Q15. [Mid] Design a cache invalidation approach for a news site where published articles sometimes need urgent corrections.

**Answer:**

**Requirements to satisfy**

- Most articles are read heavily right after publishing and should be cached aggressively for performance.
- Occasionally, an article needs a correction that must show up everywhere quickly, sometimes within seconds for legal or accuracy reasons.

**Approach**

1. **Default caching**: cache each article page at the CDN with a moderate TTL, for example 5 to 15 minutes, since most articles don't change after publishing and this keeps origin load low.
2. **Short TTL plus revalidation for freshness**: use stale-while-revalidate so users still get fast responses from cache while a background refresh checks with origin, keeping content reasonably fresh without every request hitting origin.
3. **Explicit purge on correction**: **when an editor issues a correction, trigger an immediate CDN purge (or targeted invalidation) for that specific article's URL, forcing every PoP to drop it and fetch the corrected version on the next request.**
4. **Purge propagation monitoring**: track how long purges take to propagate across all PoPs, most CDNs offer an API response or webhook confirming completion, and alert if it exceeds an acceptable window.
5. **Versioned cache keys for critical urgency**: for extremely time sensitive corrections (legal takedowns), consider bumping a cache-busting query parameter or version tag on the URL itself so old cached copies become irrelevant instantly, rather than waiting on purge propagation.
6. **Editorial tooling**: give the CMS a "publish correction" action that automatically triggers the purge API call, so this isn't a manual, easy to forget step for editors.

**Interview Answer:**
"I'd cache article pages with a moderate TTL and stale-while-revalidate for normal freshness, since most articles never change after publishing. For corrections, I'd wire the CMS's publish action directly to the CDN's purge API, so an editor hitting 'publish correction' automatically invalidates that article across all PoPs. For anything extremely urgent, like a legal issue, I'd also consider cache-busting the URL itself so old cached copies become instantly irrelevant rather than depending on purge propagation timing."

---

## Q16. [Mid] What is a CNAME record, and why does a CDN provider ask you to point your domain to one instead of using an A record?

**Answer:**

**What a CNAME is**

**A CNAME (canonical name) record maps one domain name to another domain name, rather than directly to an IP address like an A record does.** When a resolver looks up a CNAME, it gets told "the real answer is over at this other hostname," and then continues resolution from there.

**Why CDNs want a CNAME, not an A record**

- CDN edge IP addresses change frequently as PoPs are added, removed, or rebalanced. If you pointed an A record directly at a specific CDN IP, that IP could become stale or invalid without warning.

- By pointing your CNAME at a hostname the CDN controls (like yourapp.cdnprovider.net), the CDN is free to change which underlying IPs that hostname resolves to at any time, and your DNS configuration never needs to change.
- This also lets the CDN return different IPs to different users based on their location or current PoP health, since the actual A record lookup for the CDN's hostname happens dynamically on their side, not yours.

**The catch**

CNAME records technically can't coexist with other records at the root/apex of a domain (like example.com itself) in the standard DNS spec, which is why many providers offer a workaround like "ALIAS" or "flattened CNAME" records for apex domains specifically.

**Interview Answer:**
"A CNAME points your domain to another domain name instead of directly to an IP, and CDNs want that because their edge IPs change constantly as PoPs scale up, fail over, or rebalance. If I hardcoded an A record to a specific CDN IP, it could go stale. Pointing a CNAME at a hostname the CDN controls lets them freely change the underlying IPs, and even return different answers per user, without me ever touching my own DNS configuration."

---

## Q17. [Mid] Walk me through what happens at the edge on a cache miss, from the moment the request lands at the PoP.

**Answer:**

1. **Request arrives at the PoP**: the user's request, routed there by DNS or anycast, hits the CDN's edge server.
2. **Cache key lookup**: the edge server computes a cache key from the request (typically URL, sometimes plus specific headers or query params) and checks its local cache for a match.
3. **Miss detected**: no valid entry exists, either because it was never cached, it expired, or it was purged.
4. **Origin fetch**: the edge server forwards the request to the origin server (or in some architectures, to a regional "shield" cache layer first, which sits between edge PoPs and origin to absorb repeated misses across many PoPs).
5. **Origin responds**: the origin returns the response along with cache control headers dictating how (and whether) it should be cached.
6. **Cache population**: the edge server stores the response locally, keyed by that cache key, respecting the TTL from the response headers.
7. **Response to user**: the edge server returns the response to the original requester, who experiences the full round trip latency to origin on this particular request.
8. **Future requests**: subsequent requests for the same object hitting this PoP now get served directly from cache until it expires or is purged again.

**Why "origin shield" matters**

Without a shield layer, a popular object with no cache in any PoP could trigger simultaneous origin fetches from hundreds of PoPs at once. **A shield layer consolidates those into far fewer actual origin hits.**

**Interview Answer:**
"On a cache miss, the edge PoP computes a cache key from the request, finds nothing valid, and forwards the request to origin, sometimes through an intermediate regional shield cache that consolidates misses from multiple PoPs. The origin's response comes back with cache headers telling the edge how long to hold

onto it, the edge stores it and serves it to the user, and every subsequent request to that PoP is now served straight from cache until it expires or gets purged."

---

## Q18. [Senior] You drop a DNS TTL from 24 hours to 60 seconds right before a planned failover. What are you trading off, and what could still go wrong during the cutover?

**Answer:**

**What you're trading off**

- Lowering the TTL in advance is the right move, it means resolvers won't be holding onto stale answers for up to a day when you actually flip the record. You're trading increased DNS query volume and load on your nameservers (since caches expire and re-query far more often) for much faster propagation of the eventual change.
- **This trade only works if you make the change with enough lead time, at least one full old TTL period, before the actual cutover.** If you drop the TTL and cut over immediately, resolvers that already cached the old, high TTL answer will keep it for up to 24 more hours regardless of your new setting, because the low TTL only applies once resolvers actually re-fetch the record.

**What can still go wrong**

- **TTL disrespecting resolvers**: some ISP resolvers and enterprise DNS servers don't strictly honor low TTLs, and cache longer than instructed for their own load reasons, so a meaningful slice of traffic can still land on the old endpoint well past 60 seconds.
- **Client side and application level caching**: some applications, SDKs, and even some operating systems cache resolved IPs independently of DNS TTLs, ignoring your DNS settings entirely.
- **Connection pooling**: clients (and load balancers, and even CDNs) that already hold open connections to the old IP will keep using them until the connection is closed or recycled, DNS TTL changes don't affect established connections.
- **Partial cutover state**: during the propagation window, some fraction of users hit the old system and some hit the new one simultaneously. If the two aren't fully compatible (data, sessions, schema), this window can produce real inconsistency.

**How to make it safer**

Keep the old system fully functional and in sync during the entire propagation window, monitor traffic split on both old and new endpoints, and don't decommission the old infrastructure until traffic to it has dropped to near zero for a sustained period, not just until the TTL theoretically expired.

**Interview Answer:**
"Dropping the TTL ahead of time trades higher query volume for faster propagation once I actually make the change, but it only works if I do it with enough lead time, at least one old TTL cycle, before cutover. Even then, some resolvers ignore TTLs and cache longer anyway, some clients cache DNS independently of TTL, and existing open connections to the old IP won't drop just because DNS changed. So I'd keep the old system fully live and in sync through the whole window and only decommission it once real traffic there has actually dropped to near zero."

---

## Q19. [Senior] Walk me through how you'd design routing so a partial outage in one region doesn't send users in Mumbai across the globe to a PoP in Virginia.

**Answer:**

**The failure mode being avoided**

Naive latency or geo based routing that doesn't account for health can make things worse during a partial outage: if the nearest PoP degrades, some routing schemes fail over blindly to the globally "next best" option without considering that a much closer, still healthy alternative exists.

**Design approach**

1. **Health checked routing, not just static geo/latency tables**: every PoP is continuously health checked (latency, error rate, saturation), and routing decisions incorporate current health, not just a fixed geographic or latency map computed once.
2. **Regional failover tiers**: define failover priority within a region first. Mumbai's primary might be a Mumbai PoP, its secondary a Singapore or Chennai PoP, and only as a last resort does it fail over to a different continent entirely. **This keeps failover "close" even when degraded.**
3. **Weighted, gradual shifting**: instead of an all or nothing cutover the moment a health check fails, shift traffic gradually (canary style) to the next tier, so a flapping or partially degraded PoP doesn't cause a full traffic stampede elsewhere.
4. **Anycast plus BGP awareness**: if using anycast, ensure BGP withdrawal happens cleanly for an unhealthy PoP so traffic naturally reroutes via network layer proximity to the next closest advertising PoP, not globally.
5. **Client side or resolver level EDNS Client Subnet**: make sure the actual end user's approximate location, not just their resolver's location, is used for routing decisions, since a Mumbai user using a distant public resolver could otherwise get routed as if they were wherever that resolver is.
6. **Continuous testing (chaos style)**: regularly simulate regional PoP failures to verify the actual failover behavior matches intent, this is exactly the kind of thing that looks correct on a whiteboard but breaks under real conditions.

**Interview Answer:**

"I'd build routing on continuous health checks, not a static latency table, and define regional failover tiers so Mumbai's fallback is a nearby PoP like Singapore before it ever considers something across the globe. I'd shift traffic gradually rather than an all or nothing cutover, use EDNS Client Subnet so routing reflects the actual user's location rather than their resolver's, and make sure anycast BGP withdrawal is clean so unhealthy PoPs get routed around locally. I'd also regularly chaos test regional failures to confirm the behavior actually matches the design."

---

## Q20. [Senior] Your reverse proxy terminates TLS in front of a fleet of app servers. Beyond load distribution, what security responsibilities does that proxy layer actually carry?

**Answer:**

**Certificate and encryption management**

- Centralizes TLS certificate storage, rotation, and renewal in one place instead of distributing private keys across every backend instance, dramatically shrinking the attack surface for key compromise.
- Enforces minimum TLS versions and cipher suites consistently, so you're not relying on every backend service to independently stay current on security config.

**Traffic shaping and abuse prevention**

- Rate limiting and connection throttling per client IP or API key, protecting backends from being overwhelmed by a single abusive client.
- Can absorb and mitigate a meaningful portion of low to mid volume DDoS traffic before it ever reaches application servers.

**Request sanitation and filtering**

- Can enforce header size limits, reject malformed requests, and apply WAF (web application firewall) rules to block common attack patterns like SQL injection or path traversal attempts before they reach application code.
- Strips or normalizes sensitive headers, and hides internal implementation details (server versions, internal hostnames, stack traces) from external responses.

**Network segmentation**

- Backend servers can live in a private network with no direct internet exposure at all, only reachable through the proxy, meaning even a misconfigured backend service isn't directly exploitable from the internet.

**What it does not do**

It doesn't replace application level authentication and authorization, input validation for business logic, or secure coding practices further down the stack. **A reverse proxy is a strong first line of defense, not a substitute for defense in depth.**

**Interview Answer:**

"Beyond load balancing, my reverse proxy centralizes TLS certificate management so private keys aren't scattered across every backend instance, and enforces consistent TLS versions and ciphers. It does rate limiting and can absorb a chunk of DDoS traffic before it reaches app servers, applies WAF rules to filter common attack patterns, and hides internal details like server versions and stack traces from the outside world. It also lets backend servers sit in a private network with no direct internet exposure. It's a strong first layer, but it doesn't replace application level auth or input validation."

---

## Q21. [Senior] A customer complains that purging the CDN cache after a deploy takes over ten minutes to propagate globally, and calls that broken. Explain what's actually happening and how you'd design faster invalidation.

**Answer:**

**Why this happens**

A CDN purge isn't a single instantaneous operation, it's a distributed command that has to reach every PoP, potentially hundreds of them worldwide, each of which needs to actually process the purge and drop or mark stale the relevant cache entries. That propagation involves internal control plane messaging between the CDN's central system and every edge location, and depending on the provider's architecture, this can genuinely take anywhere from seconds to several minutes, sometimes longer under load. Ten minutes suggests either a provider with a slower purge propagation model, a purge scope that's expensive to compute (like a broad wildcard or tag based purge touching many objects), or contention in the CDN's purge queue during a busy period.

**How to design faster invalidation**

1. **Prefer targeted, specific purges over broad wildcard purges.** Purging /images/* is typically far more expensive to propagate than purging ten specific known URLs.

2. **Use cache tagging if the provider supports it**, so a deploy can invalidate exactly the objects that changed, computed at build time, rather than guessing at URL patterns.
3. **Avoid needing purges altogether where possible**, by using content-hashed, versioned filenames for assets that change on every deploy. **A new deploy is a new URL, and the old cached URL simply becomes irrelevant rather than needing active invalidation.**
4. **Use short TTLs with stale-while-revalidate for content that changes unpredictably**, so the system self-heals within a bounded, known window without depending on a purge call succeeding at all.
5. **Set expectations with an SLA-backed provider** if instant purge really matters for the business, some CDNs offer faster (sub-second to a few seconds) purge tiers at a higher cost.

**Interview Answer:**
"Purging isn't instant because it's a distributed operation that has to reach every PoP and get processed there, and broad or wildcard purges are especially expensive to propagate. I'd reduce reliance on purge speed altogether by using content-hashed, versioned filenames so most deploys don't need a purge at all, the old URL just becomes irrelevant. For content that must be purged, I'd use targeted, specific invalidations or cache tags rather than wildcards, and pair short TTLs with stale-while-revalidate as a safety net so freshness doesn't depend entirely on the purge call succeeding quickly."

---

## Q22. [Senior] Compare full page caching at the CDN versus running logic at the edge, like Cloudflare Workers or Lambda@Edge. When do you actually reach for the latter?

**Answer:**

**Full page caching**

- Pros: extremely simple, extremely fast, the CDN just serves a stored byte-for-byte response, no compute involved.
- Cons: entirely static, the same response goes to everyone. Any personalization, A/B testing, or request-dependent logic can't happen this way, the moment content needs to vary per request, full page caching breaks down.

**Edge compute (Workers, Lambda@Edge, similar)**

- Pros: runs actual code at the PoP, close to the user, so you can do lightweight per-request logic (auth checks, header rewriting, A/B test bucketing, geolocation-based redirects, request or response modification) without a round trip all the way to origin.
- Cons: more operational complexity, cold start considerations for some platforms, limits on execution time and available libraries compared to a full application server, and it's still not a replacement for a real backend doing heavy business logic or database access.

**Best Choice**

Full page caching by default for genuinely static content, edge compute specifically for the narrow slice of logic that's simple enough to run at the edge but still needs to vary per request.

**Why?**

**The two aren't really competitors, they're complementary.** Use full page caching for content that's identical for everyone, since it's the cheapest and fastest option. Reach for edge compute specifically when you have a real need for per-request variation that's still lightweight, like redirecting based on country, rewriting a response header, running an A/B test bucketing decision, or validating a JWT before letting a request

through, all of which are far cheaper to do at the edge than sending every request all the way back to origin just to make that small decision.

**Interview Answer:**

"Full page caching serves an identical stored response to everyone, which is as fast and simple as it gets, but it can't handle anything that varies per request. Edge compute runs actual code at the PoP, so I can do lightweight per-request logic like geolocation redirects, header rewriting, or auth checks close to the user instead of sending every request back to origin. I'd default to full page caching for genuinely static content, and reach for edge compute specifically when I need cheap, simple, per-request decisions that don't justify a full origin round trip."

---

## Q23. [Senior] Explain what stale while revalidate buys you as a caching strategy at the edge, and where it can bite you.

**Answer:**

**What it does**

Stale-while-revalidate lets a cache serve an expired (stale) response immediately to the user, while simultaneously kicking off a background fetch to origin to refresh the cache for the next request. **The user never waits on the origin round trip, they get the fast, if slightly outdated, cached response right away.**

**Why it's valuable**

- Removes the latency spike that normally happens the instant a TTL expires and the next unlucky request has to pay the full origin round trip cost.
- Smooths out load on the origin, since revalidation happens as a background trickle rather than every PoP simultaneously blocking on a synchronous refetch the moment the same TTL expires everywhere.
- Great fit for content that's mostly fine to be a little stale, like a product listing page or a homepage.

**Where it can bite you**

- **Correctness-sensitive content**: if the content absolutely must be current (pricing, inventory counts, security-relevant configuration), serving stale-by-design data is actively wrong, not just a minor inconvenience.
- **Compounding staleness**: if the background revalidation itself fails or is slow (origin is degraded), the cache can keep serving increasingly stale content for longer than intended, since the mechanism is designed to tolerate exactly that kind of staleness silently.
- **Debugging confusion**: because it's serving stale by design, engineers investigating a "why am I seeing old data" report need to know this pattern is in play, otherwise it looks like a caching bug rather than intended behavior.
- **Needs a max staleness bound**: without a stale-if-error or hard staleness ceiling, an origin outage combined with stale-while-revalidate can result in indefinitely serving old data without anyone noticing, since users technically keep getting fast, successful responses.

**Interview Answer:**

"Stale-while-revalidate serves the expired cached response immediately while quietly refreshing it in the background, so users never eat the latency spike right when a TTL expires, and it smooths load on origin instead of everyone hammering it the instant the cache goes stale simultaneously. It's great for content that's fine being a little outdated, like a homepage, but it's the wrong choice for correctness-sensitive data

like pricing or inventory. I'd also pair it with a hard staleness ceiling, otherwise a struggling origin can silently keep serving old data indefinitely without anyone noticing."

---

## Q24. [Senior] A highly cached, popular asset's TTL expires at the same moment across thousands of edge nodes, and your origin gets hammered. What's happening, and how do you fix it?

**Answer:**

**What's happening**

**This is the classic thundering herd (also called cache stampede) problem.** Because the asset was cached with the same TTL everywhere at roughly the same time, every PoP's cache entry expires nearly simultaneously. The next request at each of those thousands of PoPs sees a cache miss at the same moment and independently forwards a request to origin, so instead of one origin request, the origin suddenly receives thousands of near-simultaneous requests for the exact same object.

**How to fix it**

1. **Request coalescing at the edge**: configure the CDN (most support this) so that if multiple requests for the same cache key arrive while a fetch to origin is already in flight, only the first one actually goes to origin, and the rest wait for that in-flight response and share it, rather than each independently hitting origin.
2. **Jittered TTLs**: instead of a fixed TTL like exactly 300 seconds for every object, add small random jitter (295 to 310 seconds) so expiration times spread out slightly across PoPs instead of aligning perfectly.
3. **Stale-while-revalidate**: let PoPs keep serving the stale copy while one background request refreshes it, instead of every PoP blocking on a synchronous refetch simultaneously.
4. **Origin shield layer**: introduce an intermediate regional cache between edge PoPs and origin, so misses across many edge PoPs first funnel through a shield cache, which itself only forwards a much smaller number of actual requests to origin.
5. **Pre-warming for predictable expirations**: for known, scheduled high traffic content, proactively refresh the cache slightly before expiration rather than waiting for a reactive miss storm.

**Interview Answer:**
"That's a classic thundering herd, the TTL for a popular object expires at nearly the same instant across every PoP, so they all independently miss and hit origin at once. I'd fix it with request coalescing so only one in-flight request per cache key actually reaches origin while others wait and share the result, add small random jitter to TTLs so expirations don't align perfectly, use stale-while-revalidate so PoPs serve the old copy during refresh instead of blocking, and put an origin shield layer in between to absorb and consolidate misses before they reach origin directly."

---

## Q25. [Senior] A client switched DNS providers, but some users kept hitting the old server for days afterward. Diagnose what's going on.

**Answer:**

**Most likely causes, in order of likelihood**

1. **Old TTL still in effect at the time of the switch**: if the previous DNS record had a high TTL (say 24 or 48 hours) and the switch happened without lowering that TTL well in advance, every resolver that had

already cached the old answer keeps serving it until that TTL naturally expires, completely independent of when the new provider's records went live.

2. **Resolvers ignoring TTL**: some ISP or corporate DNS resolvers cache longer than instructed, for their own performance or policy reasons, which can extend staleness well past what the TTL alone would predict.
3. **Application or OS level DNS caching**: some environments (certain Java versions historically, some mobile OS network stacks, some corporate proxy setups) cache resolved IPs at a layer that doesn't respect DNS TTL at all, sometimes until the process or device restarts.
4. **Registrar-level propagation delay**: if the change also involved switching nameservers at the registrar (not just records at the same DNS provider), that change itself has its own propagation characteristics and can take up to 24 to 48 hours to be seen everywhere, since it depends on TLD-level caching, not the record's own TTL.
5. **The old server is still up and technically working**: if the old server wasn't decommissioned, this failure mode is invisible operationally, since impacted users aren't getting errors, they're just quietly getting served by the old, wrong system.

**How to actually diagnose it**

Check what the TTL was on the old A/NS records before the switch, check how much lead time was given before cutover, and directly query multiple public resolvers worldwide to see which ones still return the old answer. **Keep the old server running and instrumented until traffic to it drops to zero, rather than assuming the migration is done the moment the new DNS is live.**

**Interview Answer:**

"The most likely explanation is that the old records had a high TTL and it wasn't lowered with enough lead time before the switch, so every resolver that already cached the old answer keeps serving it until that TTL naturally runs out, regardless of when the new provider went live. It gets worse if the switch also involved changing nameservers at the registrar, since that has its own separate propagation delay. I'd check the pre-switch TTL, query resolvers globally to see who's still stale, and keep the old server running and monitored until its traffic actually drops to zero."

---

## Q26. [Senior] Give me a concrete example of a reverse proxy misconfiguration that becomes a security hole, and a separate example for a forward proxy.

**Answer:**

**Reverse proxy example: header spoofing via missing trust boundary**

A reverse proxy adds an X-Forwarded-For header with the real client IP before passing the request to backend servers. If a backend application trusts this header blindly, without the proxy first stripping or overwriting any X-Forwarded-For value the client itself might have sent, an attacker can simply set that header themselves in their original request and spoof any IP address they want. If backend logic uses X-Forwarded-For for anything security relevant, like IP based rate limiting, access control lists, or geofencing, an attacker bypasses all of it by lying about their own IP in a header the proxy never sanitized.

**Fix**: **the reverse proxy must always overwrite (not just append to) the X-Forwarded-For header with the actual observed connection IP, so backend services can trust it as coming only from the proxy, never directly from the client.**

**Forward proxy example: open proxy misconfiguration**

A forward proxy meant to serve internal, authenticated corporate clients is accidentally left open to accept connections from any source, with no authentication or IP allowlisting. Attackers on the internet discover it and use it to relay their own traffic through the company's IP address and infrastructure, for anything from spamming to scanning other targets to laundering the origin of an attack, all of which now appears to come from the company's own network.

**Fix**: forward proxies should always require authentication or restrict access to known internal IP ranges, and should be regularly scanned externally to confirm they aren't inadvertently reachable from the open internet.

**Interview Answer:**
"On the reverse proxy side, a common one is trusting a client-supplied X-Forwarded-For header without the proxy overwriting it, so an attacker just sets that header themselves and spoofs any IP, bypassing IP based rate limits or access controls on the backend. On the forward proxy side, a classic mistake is leaving it open with no authentication or IP restriction, so it becomes an open proxy attackers can use to relay traffic through your company's IP, for spam, scanning, or hiding the true source of an attack. Both come down to the proxy failing to enforce a trust boundary it was supposed to own."

---

## Q27. [Senior] Your origin datacenter in one region just went down entirely. Using only DNS and CDN infrastructure, no code deploy, how do you keep serving users in that region?

**Answer:**

**Immediate mitigation using DNS**

1. **Health check based DNS failover**: if health checks are already configured on the DNS provider (most major providers support this), the failed origin's record automatically stops being returned once health checks detect it's down, and traffic shifts to a healthy origin in another region. If this wasn't pre-configured, manually update the DNS record now to point at a healthy region, accepting that propagation speed depends on the existing TTL.
2. **Multi-region failover priority**: if a secondary origin already exists in another region with data reasonably in sync, DNS failover (or the CDN's own origin failover feature) routes new origin fetches there instead.

**Using the CDN itself to buy time**

1. **Serve stale content from cache**: enable (or confirm you already have) stale-if-error behavior at the CDN, so if the origin is unreachable, the CDN keeps serving the last known good cached response instead of returning an error. This alone can keep a huge share of read-heavy traffic working even with the origin fully down.
2. **Extend cache TTLs temporarily**: for content currently in cache, consider temporarily instructing the CDN to treat existing cached objects as valid longer than their original TTL, buying time while origin recovery happens.
3. **Origin failover configured at the CDN layer**: many CDNs support defining a primary and backup origin directly in their configuration, so on origin timeout or 5xx responses, the CDN automatically retries against the backup origin without needing any DNS change at all.

**What this doesn't solve**

None of this helps with write traffic or genuinely dynamic, personalized functionality, since that data doesn't exist anywhere except the failed origin (or wherever its replica lives). **This buys time and preserves read**

**availability for cacheable content, it isn't a substitute for actual multi-region redundancy and data replication.**

**Interview Answer:**
"First, I'd rely on DNS health check based failover, or manually repoint the record, to shift new origin traffic to a healthy region, though propagation speed depends on the existing TTL. At the CDN layer, I'd lean on stale-if-error so cached content keeps serving even with the origin fully down, and configure or confirm origin failover at the CDN itself so it automatically retries a backup origin on failure. This preserves read availability for cacheable content, but it doesn't solve anything for write traffic or fully dynamic functionality, that still needs real multi-region data redundancy underneath it."

---

**If you remember nothing else from this chapter:** *DNS TTL is a dial between fast lookups and fast recovery, and most outages make you wish you had picked the other side.*

# Chapter 10: Asynchronous Systems: Message Queues & Event-Driven Architecture

Anyone can draw a queue between two boxes. The real signal in an interview is whether you've thought about what happens when a message is duplicated, delayed, or delivered out of order. This chapter is where you build the instincts of someone who has actually debugged an async system in production, not just used one when everything went smoothly.

**War Story:** *A consumer that wasn't idempotent double-charged a batch of customers after a routine deploy caused a brief network blip and a handful of at-least-once redeliveries.*

**Avoid:** assuming exactly-once delivery is free and not designing consumers to handle duplicate messages

**Aim For:** designing every consumer as if redelivery is guaranteed to happen eventually, and treating idempotency as non-negotiable

## Q1. [Junior] What is a message queue, and why would a service send work through one instead of calling the downstream service directly over HTTP?

**Answer:**

**A message queue is an intermediary that stores messages produced by one service until a consumer is ready to process them.** The producer writes a message and moves on, it does not wait for the consumer to finish the work.

**Why decouple with a queue**

- Availability: if the downstream service is slow or temporarily down, the message waits in the queue instead of the request failing outright.
- Load leveling: a burst of 10,000 requests can be smoothed out and processed at a steady rate instead of overwhelming the consumer.
- Independent scaling: producers and consumers can scale separately, and you can add more consumers to drain the queue faster without touching the producer.
- Loose coupling: the producer does not need to know how many consumers exist or where they are deployed.

**Trade-off to mention**

- You give up immediate confirmation of success. The caller no longer knows synchronously whether the work actually completed, so you need a way to check status later or notify the original caller asynchronously.

**Interview Answer:**

"A message queue decouples the producer from the consumer in time. Instead of the producer blocking on a direct call and failing if the downstream service is slow or down, it drops a message on the queue and moves on. This smooths out traffic spikes, lets consumers scale independently, and improves availability, at the cost of giving up an immediate synchronous response, so you need another way to tell the caller the work is done."

## Q2. [Junior] Explain the difference between point-to-point messaging and publish/subscribe.

**Answer:**

**Both patterns move messages from producers to consumers, but they differ in how many consumers receive each message.**

**Point-to-point (queue model)**

- A message is placed on a queue and delivered to exactly one consumer, even if multiple consumers are listening.
- Consumers typically compete for messages, this is used for work distribution, like processing a batch of image resize jobs across a worker pool.
- Example: a classic queue like RabbitMQ or SQS standard queue.

**Publish/subscribe (fan-out model)**

- A message is published to a topic, and every subscriber that is interested gets its own copy.
- Used when multiple independent systems need to react to the same event, for example an "order placed" event that both the shipping service and the analytics service need to see.
- Example: SNS, or a Kafka topic being read by multiple consumer groups.

**Key distinction to state clearly**

- Point-to-point is about splitting work among many workers.
- Pub/sub is about broadcasting one event to many independent listeners.

**Interview Answer:**
"Point-to-point is one message, one consumer, used to distribute work across a pool of workers, like a queue of jobs. Pub/sub is one message, many consumers, used to broadcast an event to multiple independent systems that each need to react to it in their own way. The practical difference shows up in delivery semantics: in a queue, a message is removed once one consumer processes it, in pub/sub every subscriber gets its own copy."

---

## Q3. [Junior] What does "at-least-once delivery" mean, and why can't most systems just default to exactly-once instead?

**Answer:**

**At-least-once delivery means the messaging system guarantees a message will be delivered one or more times, but never zero times.** If a consumer crashes after receiving a message but before acknowledging it, the broker redelivers it, which can produce duplicates.

**Why exactly-once is hard**

- Exactly-once requires coordinating the act of processing a message with the act of acknowledging it, across two independent systems (the broker and whatever the consumer is writing to, like a database).
- Network failures make it impossible to distinguish "the ack was lost" from "the message was never delivered," so the safe default is to redeliver, which risks duplicates.

- True exactly-once semantics (like Kafka's transactional producer/consumer) exist but only work end to end when every step, produce, process, and commit, participates in the same transactional boundary, which adds real complexity and throughput cost.

**Practical takeaway**

- Most production systems choose at-least-once delivery plus an idempotent consumer, because it is simpler and cheaper to make duplicate processing harmless than to prevent duplicates from ever happening.

**Interview Answer:**
"At-least-once means the broker guarantees a message arrives at least one time, but it might arrive more than once if an acknowledgment is lost after a crash. True exactly-once is hard because you can't cheaply tell the difference between a lost ack and a lost message across two independent systems. So instead of chasing exactly-once everywhere, most teams accept at-least-once delivery and make the consumer idempotent, so processing the same message twice has no extra effect."

---

## Q4. [Junior] What is a dead-letter queue, and why does a production messaging system need one?

**Answer:**

**A dead-letter queue, or DLQ, is a separate queue where messages get routed after they fail to be processed successfully a set number of times.** For example, an order-processing consumer that throws an exception every time because of a malformed payload would otherwise retry that same broken message forever, blocking every valid order behind it, until a DLQ pulls it aside after a few failed attempts.

**Why you need one**

- Without a DLQ, a message that keeps failing (a "poison message") gets redelivered forever, retried endlessly, and can block or slow down every other message behind it in the queue.
- A DLQ isolates the bad message so healthy messages keep flowing.
- It gives engineers a place to inspect what went wrong: bad payload, a bug in the consumer, a downstream dependency that's down, and decide whether to fix and replay it or discard it.

**How it's typically configured**

1. Set a maxReceiveCount or retry limit on the main queue.
2. Once a message exceeds that limit, the broker automatically moves it to the DLQ instead of redelivering it again.
3. Alert on DLQ depth, a growing DLQ usually means a systemic bug, not a one-off bad message.
4. Provide a replay mechanism to push messages back to the main queue once the underlying issue is fixed.

**Interview Answer:**
"A dead-letter queue catches messages that fail processing repeatedly, after a retry limit is hit, instead of letting them retry forever and clog the queue. It protects the healthy message flow and gives you a place to inspect and debug failures. I'd always pair it with alerting on DLQ depth, because a sudden spike usually means a bug shipped, not that one message happened to be malformed, and a replay tool to push fixed messages back through."

## Q5. [Junior] What's the practical difference between a traditional message queue like RabbitMQ and an event streaming log like Kafka?

**Answer:**

**Both move messages from producers to consumers, but they store and expose messages differently, which changes what you can do with them.**

**Traditional queue (RabbitMQ, SQS)**

- A message is typically deleted once it's successfully consumed and acknowledged.
- Built around delivering a message once to a consumer (or once per queue), it's a transient mailbox.
- Great for task distribution, request routing, and complex routing rules (RabbitMQ exchanges).

**Event log (Kafka)**

- Messages, called records, are appended to a durable, ordered log and retained for a configured period (or indefinitely), regardless of whether they've been read.
- Multiple independent consumer groups can each read the same topic from their own position, replaying history if needed.
- Built for high-throughput streaming, event sourcing, and cases where you want to reprocess old events.

**When it matters**

- If you need "replay the last 24 hours of events to rebuild a cache," you want a log, not a queue.
- If you need "distribute this job to exactly one worker and forget it," a queue is simpler and does not require you to manage offsets.

**Interview Answer:**
"A traditional queue like RabbitMQ treats a message as transient, once it's acknowledged it's gone. Kafka treats it as a durable, ordered log that's retained for a configurable window, so multiple consumer groups can read it independently and even replay history. I'd reach for a queue for simple task distribution and for Kafka when I need high-throughput event streaming, replay, or multiple systems consuming the same event stream at their own pace."

## Q6. [Junior] What does it mean for a message consumer to be idempotent, and why does that matter in an at-least-once delivery system?

**Answer:**

**An idempotent consumer produces the same end result no matter how many times it processes the same message.** If a "charge customer $10" message gets delivered twice due to a redelivery after a crash, an idempotent consumer charges the customer once, not twice.

**Why it matters**

- Almost every real-world message broker guarantees at-least-once delivery under failure, which means duplicates are not a rare edge case, they are an expected occurrence.
- Without idempotency, duplicate delivery translates directly into duplicate side effects: double charges, duplicate emails, double-counted inventory decrements.

**Common ways to achieve it**

1. Deduplication key: attach a unique message ID or idempotency key, and check a "processed" table or cache before acting.
2. Conditional writes: use database constraints like a unique index on (order_id, event_type) so a duplicate insert fails safely instead of double-applying.
3. Upserts instead of increments: writing "set balance = X" is naturally idempotent, "add 10 to balance" is not.
4. Natural idempotency: some operations, like setting a status field to "shipped," are idempotent by nature since repeating them has no additional effect.

**Interview Answer:**
"Idempotent means processing the same message once or five times produces the same result. It matters because at-least-once delivery is the norm, not the exception, so duplicates will happen after retries or consumer crashes. I'd implement it with a deduplication key stored alongside the processed record, checked before applying the side effect, or by designing the operation itself to be naturally idempotent, like an upsert instead of an increment, so replays are safe by construction."

---

## Q7. [Mid] Compare Kafka, RabbitMQ, and SQS/SNS. Walk me through how you'd choose between them for a given system.

**Answer:**

These three solve overlapping but distinct problems, and the right choice depends on throughput, ordering needs, routing complexity, and operational appetite.

**Kafka**

- Pros: extremely high throughput, durable ordered log with configurable retention, natural fit for event streaming, replay, and multiple independent consumer groups.
- Cons: operationally heavier to run yourself, ordering is only guaranteed within a partition, consumers manage offsets, no built in complex routing.

**RabbitMQ**

- Pros: flexible routing via exchanges (direct, topic, fanout), lower latency for smaller messages, mature support for priority queues, per-message TTL, and complex routing topologies.
- Cons: lower raw throughput ceiling than Kafka, message retention is push-and-delete oriented so you generally cannot replay history, clustering for scale is more manual.

**SQS/SNS (managed AWS)**

- Pros: fully managed, zero infrastructure to operate, SQS gives simple durable queues, SNS gives pub/sub fan-out, and together (SNS to multiple SQS queues) give a fan-out pattern with per-consumer durability. Pay-as-you-go, scales automatically.
- Cons: less control over ordering (only SQS FIFO guarantees strict order, at lower throughput), less flexible routing than RabbitMQ, tied to AWS.

**Best Choice**

**Depends on the scenario, there is no universal winner.** For high-throughput event streaming with replay needs (analytics pipelines, event sourcing), pick Kafka. For complex routing between internal services with moderate throughput, pick RabbitMQ. For a small team that wants managed infrastructure and simple fan-out without operating a cluster, pick SQS/SNS.

**Why?**

The decision hinges on what you're optimizing for: raw throughput and replay favor Kafka, routing flexibility favors RabbitMQ, and operational simplicity favors managed AWS services. Getting this trade-off right in an interview matters more than naming a "best" broker.

**Interview Answer:**

"I would not default to one broker. If I need high-throughput event streaming with replay and multiple consumer groups, I'd pick Kafka. If I need flexible routing logic between services and don't need to operate at Kafka's scale, RabbitMQ's exchanges give me more control with less throughput. If I want to avoid operating any broker infrastructure and my routing needs are simple, SNS fanning out to SQS queues gets me a managed pub/sub system with per-consumer durability. The choice comes down to throughput, ordering needs, and how much operational overhead the team can absorb."

---

## Q8. [Mid] Walk me through how Kafka partitions and consumer groups work together to let you scale message consumption.

**Answer:**

**Kafka splits a topic into partitions, each partition is an ordered, append-only log.** Producers write records to a partition, usually chosen by hashing a key, and consumers read from partitions in order.

**How consumer groups fit in**

1. A consumer group is a set of consumer instances that share the work of reading a topic.
2. Kafka assigns each partition to exactly one consumer within a group at a time, so if a topic has 6 partitions and a group has 3 consumers, each consumer reads 2 partitions.
3. Adding more consumers up to the number of partitions increases parallelism, adding more than the partition count leaves extra consumers idle.
4. Multiple consumer groups can independently read the same topic, each group tracks its own offsets, so group A and group B don't interfere with each other.

**Why partitioning by key matters**

- All records with the same key (say, the same user ID) land on the same partition, which preserves per-key ordering, this is critical when order matters, like a sequence of account balance updates for one user.
- Choosing too few partitions caps parallelism, choosing too many adds overhead and can hurt latency and increase the cost of leader elections during failover.

**Rebalancing**

- When a consumer joins or leaves the group, Kafka triggers a rebalance, reassigning partitions among the remaining consumers, which briefly pauses consumption.

**Interview Answer:**

"Kafka topics are split into partitions, each an ordered log, and a consumer group divides those partitions among its members so each partition is read by exactly one consumer in the group at a time. That's how you scale consumption horizontally, add more consumers up to the partition count. Keying messages by something like user ID keeps all events for that key in order on one partition. The trade-off is picking the partition count up front, since it caps parallelism and changing it later can disrupt ordering guarantees."

## Q9. [Mid] Design the messaging flow for an e-commerce order: an order service takes the order, and inventory, payment, and shipping services all need to react. Walk me through it.

**Answer:**

This is a textbook event-driven fan-out. **The order service should not call inventory, payment, and shipping directly and wait for all three, that couples it tightly to three services and one slow dependency blocks the whole flow.**

**Design**

1. The order service persists the order to its own database and publishes an "OrderPlaced" event to a topic or exchange.
2. Inventory, payment, and shipping each subscribe independently and react to the event at their own pace. This is pub/sub: one event, three independent consumers.
3. Each downstream service processes its part and, if needed, emits its own follow-up event, like "PaymentCharged" or "InventoryReserved," which other services can react to in turn.
4. The order service (or a dedicated orchestrator or saga coordinator) listens for these downstream events to update the order's overall status, and to trigger compensating actions if something fails downstream (this is the saga pattern, covered in detail later in this chapter).

**Reliability concerns to address**

- Use the outbox pattern so "save the order" and "publish the event" happen atomically, avoiding a state where the order is saved but the event is lost, or vice versa.
- Make every downstream consumer idempotent, since events may be redelivered.
- Decide what "done" means: the order is "placed" as soon as the event is published, but "fulfilled" only once inventory, payment, and shipping all succeed, which is eventual consistency, not immediate consistency.

**Interview Answer:**
"I'd have the order service publish a single OrderPlaced event rather than calling three services synchronously. Inventory, payment, and shipping each subscribe and react independently, which decouples them and lets any one of them be slow or temporarily down without blocking order creation. I'd use the outbox pattern so the order write and the event publish are atomic, make each downstream consumer idempotent since delivery is at-least-once, and treat the overall order state as eventually consistent, tracked by listening for follow-up events like PaymentCharged or InventoryReserved."

---

## Q10. [Mid] How do you guarantee message ordering in Kafka, and what can break that guarantee?

**Answer:**

**Kafka guarantees ordering only within a single partition, not across an entire topic.**

**How ordering is preserved**

- Records within one partition are strictly ordered by offset, appended and read in the sequence they were written.
- To keep related events in order (like all updates for one order ID), you key messages by that ID, Kafka's default partitioner hashes the key to consistently route it to the same partition.

**What breaks the guarantee**

- Changing the number of partitions on a topic reshuffles the key-to-partition mapping going forward, so events for the same key can suddenly land on a different partition, breaking continuity of order.
- Producer retries without idempotent producer settings enabled can reorder records if an earlier send fails and is retried after a later one succeeds.
- Multiple producers writing the same key from different processes without coordination can race and interleave in an order that doesn't reflect real-world causality.
- Reading with multiple consumer threads per partition (fetching a batch and processing it in parallel) can process records out of order even though they were read in order.

**How to protect it**

1. Enable the idempotent producer (`enable.idempotence=true`) to avoid retry-induced reordering.
2. Never repartition a topic that has an ordering dependency on existing keys, create a new topic instead if you must change partition count.
3. Process each partition's records sequentially in the consumer, don't parallelize within a partition if order matters.

**Interview Answer:**

"Kafka only orders records within a partition, so I key messages by whatever needs order preserved, like order ID, so they always land on the same partition. That guarantee breaks if you change the partition count, since the key-to-partition mapping shifts, if producer retries aren't idempotent and get reordered, or if a consumer parallelizes processing within a single partition. So I'd enable idempotent producers, avoid repartitioning topics with ordering dependencies, and keep partition-level processing strictly sequential."

---

## Q11. [Mid] What is backpressure in a message-driven system, and how would you handle a consumer that can't keep up with the producer?

**Answer:**

**Backpressure is what happens when a consumer processes messages slower than a producer generates them.** Without a plan for it, the gap grows unbounded: memory fills up, queue depth balloons, latency for every message climbs, and the system can eventually crash or start dropping data.

**Ways to handle it**

- Buffer with limits: let the queue absorb the burst, but cap its size or retention so it doesn't grow forever, and monitor queue depth or consumer lag as a leading indicator.
- Scale consumers horizontally: add more consumer instances (bounded by partition count in Kafka) to increase total processing throughput.
- Rate limit or throttle the producer: some systems push backpressure upstream, telling the producer to slow down (this works well with pull-based systems like Kafka, harder with push-based ones).
- Shed load deliberately: drop or sample lower-priority messages under sustained overload rather than let latency degrade for everyone.
- Batch processing: process messages in larger batches to amortize per-message overhead and increase throughput per consumer.
- Autoscale based on lag: use consumer lag (Kafka) or queue depth (SQS, RabbitMQ) as the metric driving autoscaling policies, not just CPU.

**Interview Answer:**

"Backpressure happens when the consumer can't keep up with the producer and the backlog keeps growing. I'd first make sure the buffer, the queue itself, has bounds and I'm alerting on depth or consumer lag rather than discovering the problem when memory runs out. Then I'd scale consumers horizontally to raise total throughput, and where possible push the slowdown signal back to the producer or shed lower-priority work. Pull-based systems like Kafka make this easier since the consumer controls its own read rate."

---

## Q12. [Mid] What's the outbox pattern, and what specific problem does it solve?

**Answer:**

**The outbox pattern solves the "dual write" problem: when a service needs to update its own database and publish an event about that update, but those two actions aren't naturally atomic across two different systems (a database and a message broker).**

**The problem without it**

- If you write to the database first and then publish the event, a crash between the two steps means the database is updated but no event ever goes out, other services silently miss the change.
- If you publish the event first, a crash before the database commit means downstream services react to something that never actually happened.

**How the outbox pattern fixes it**

1. Instead of publishing directly to the broker, the service writes the business record and an "event to be published" row into an outbox table, in the same local database transaction.
2. Because both writes are in one transaction, they either both commit or both roll back, there is no window where one happened without the other.
3. A separate process, either a polling publisher or a change data capture tool like Debezium reading the database's write-ahead log, reads new rows from the outbox table and publishes them to the message broker.
4. Once published (and acknowledged), the outbox row is marked as sent or deleted.

**Trade-offs**

- Adds a bit of latency between the write and the actual event publish (usually milliseconds to low seconds).
- Requires either a polling job or a CDC pipeline as extra infrastructure.
- Still delivers at-least-once, since the publisher might publish and then crash before marking the row as sent, so consumers must remain idempotent.

**Interview Answer:**
"The outbox pattern solves the dual write problem, where a service needs to atomically update its database and publish an event, but a database and a message broker can't share one transaction. Instead of writing to both, I write the business record and the event into an outbox table in the same local transaction, then a separate publisher, often a CDC tool like Debezium tailing the database log, reads that table and publishes to Kafka or a queue. It guarantees the event is never lost or published without the underlying write actually committing."

---

## Q13. [Mid] Compare RabbitMQ's queue-based model to Kafka's log-based model directly. When would you pick one over the other?

**Answer:**

Both are widely used, but they're built around different core abstractions, which shows up in what each is good at.

**RabbitMQ (queue based)**

- Pros: rich routing via exchanges (fanout, topic, direct, headers), supports priority queues, per-message TTL, delayed messages, and complex conditional delivery out of the box.
- Cons: messages are removed once consumed, so there's no built-in replay, throughput ceiling is lower than Kafka for very high volume streaming, scaling out requires more manual cluster management.

**Kafka (log based)**

- Pros: very high sustained throughput, records are retained and replayable, multiple independent consumer groups can read the same topic at their own pace, strong fit for event sourcing and streaming analytics.
- Cons: routing is simple (topic based), no built-in complex conditional routing, consumers must manage their own offsets and rebalancing behavior, heavier to operate correctly at scale (ZooKeeper or KRaft, partition management).

**Best Choice**

For internal task queues, RPC-style request routing, or systems that need conditional routing logic, RabbitMQ. For event streaming, audit logs, replay-driven architectures, or very high throughput pipelines, Kafka.

**Why?**

**RabbitMQ's exchange model is built for routing decisions, Kafka's log model is built for durable, replayable event history at scale.** Picking based on the actual need, routing complexity versus replay and throughput, avoids reaching for the fashionable tool instead of the right one.

**Interview Answer:**
"RabbitMQ is built around queues and exchanges, so it excels at conditional routing, priority messages, and typical task distribution. Kafka is built around a durable, partitioned log, so it excels at high-throughput streaming and letting multiple consumer groups replay history independently. If I need complex routing logic between services at moderate volume, I'd pick RabbitMQ. If I need to stream millions of events per second and let different teams consume the same stream independently, sometimes replaying it, I'd pick Kafka."

---

## Q14. [Mid] How would you design the retry strategy for a consumer that occasionally fails to process a message?

**Answer:**

**A naive retry loop, retry immediately forever, can make things worse: it hammers a struggling downstream dependency and can starve other messages behind it.**

**Design**

1. Classify the failure: transient (network blip, downstream service briefly unavailable) versus permanent (malformed message, business rule violation that will never succeed).
2. For transient failures, retry with exponential backoff and jitter, for example 1s, 2s, 4s, 8s, capped at some maximum, so retries space out instead of hammering the dependency in lockstep.
3. Cap the retry count, after N attempts (commonly 3 to 5), stop retrying on the main queue.

4. Route exhausted messages to a dead-letter queue rather than dropping them or retrying forever, so they can be inspected and replayed once the root cause is fixed.
5. For permanent failures detected early (bad schema, missing required field), skip straight to the DLQ, don't waste retry attempts on something that will never succeed.

**Implementation notes**

- In SQS, this is built in via `maxReceiveCount` and a redrive policy to a DLQ.
- In Kafka, this typically means the consumer catches the exception, and either commits the offset and pushes the message to a separate "retry" topic with a delay, or uses a retry topic pattern with increasing delay topics (retry-1s, retry-30s, retry-5m) before landing in a DLQ topic.
- Always log enough context (message ID, error, attempt count) alongside the DLQ entry to make debugging possible without replaying blind.

**Interview Answer:**
"I'd first distinguish transient failures from permanent ones. Transient failures get retried with exponential backoff and jitter, capped at a handful of attempts, so I'm not hammering a struggling dependency. Permanent failures, like a malformed payload, go straight to a dead-letter queue instead of wasting retries. Once the retry budget is exhausted for a transient failure, it also lands in the DLQ rather than retrying forever, with enough context logged to debug and replay it later once the underlying issue is fixed."

---

## Q15. [Mid] What happens if a Kafka consumer crashes after processing a message but before committing its offset?

**Answer:**

**The offset is Kafka's bookmark for what a consumer group has already read.** If the consumer processes a message, say it writes a row to a database, and crashes before committing the offset, Kafka has no record that message was ever read.

**What happens next**

1. Kafka detects the consumer is gone (missed heartbeats) and triggers a rebalance, reassigning that partition to another consumer in the group, or the same consumer once it restarts.
2. Whoever picks up the partition resumes from the last committed offset, not from where the crashed consumer actually got to.
3. That means the message that was already processed gets redelivered and processed again. This is exactly why Kafka is described as at-least-once by default: the gap between "processed" and "committed" is where duplicates come from.

**How to manage it**

- Make the processing idempotent, so reprocessing that message causes no harm, this is the standard fix rather than trying to eliminate the gap entirely.
- Commit offsets only after the side effect is durably applied (commit after processing, not before), committing before processing risks the opposite problem, losing a message if the consumer crashes mid-processing (that would be at-most-once behavior).
- For stronger guarantees, use Kafka's transactional API to atomically write the processing result and the offset commit together (common in Kafka Streams "exactly-once" processing), though this adds throughput cost and complexity.

**Interview Answer:**

"If a consumer processes a message but crashes before committing the offset, Kafka doesn't know that message was handled. After a rebalance, whoever gets that partition resumes from the last committed offset and reprocesses that message, which is a duplicate. That's the mechanism behind Kafka's at-least-once default. The fix isn't to try to prevent it, it's to make the consumer idempotent and to commit offsets only after the processing side effect is durably applied, so a duplicate replay causes no harm."

---

## Q16. [Mid] Explain event-driven architecture versus request/response. When would you actually choose one over the other?

**Answer:**

Request/response is synchronous: a caller sends a request and blocks waiting for a direct answer. Event-driven architecture is asynchronous: a service publishes a fact about something that happened, and interested parties react whenever they're ready, with no expectation of an immediate reply.

**Request/response**

- Good fit: the caller needs an answer right now to proceed, like "is this username available" or "what's the current price."
- Simpler to reason about, easier to trace a single call end to end, but tightly couples caller and callee's availability and latency.

**Event-driven**

- Good fit: multiple systems need to react to something that happened, and the producer doesn't need or want to know who's listening, like "user signed up" triggering welcome email, analytics tracking, and a fraud check, independently.
- Naturally resilient to a slow or temporarily unavailable consumer, since events wait in the broker.
- Harder to trace end to end (needs correlation IDs and distributed tracing), and introduces eventual consistency, the "current state" of the system may lag briefly behind reality.

**Best Choice**

Use request/response for anything the caller must know synchronously to proceed. Use events for side effects, fan-out, and cross-service reactions where immediate confirmation isn't required.

**Why?**

**Most real systems use both: a synchronous API to accept the request and give the user immediate feedback, with events fired underneath to drive everything that doesn't need to block the response.**

**Interview Answer:**

"Request/response is for when the caller genuinely needs an answer before it can continue, it's synchronous and tightly coupled by nature. Event-driven is for broadcasting that something happened and letting independent systems react on their own schedule, which is more resilient and scales better for fan-out, but trades away immediate confirmation and adds eventual consistency. In practice I use both together, a synchronous endpoint that does the minimal necessary work and returns fast, publishing an event underneath to drive everything else asynchronously."

---

## Q17. [Mid] You're building a payment processing consumer where double-charging a customer is completely unacceptable. How do you handle duplicate messages?

**Answer:**

**Since the messaging layer will deliver at-least-once, duplicate messages are expected, not a rare bug.** The fix has to live in the consumer's processing logic, not in trying to make the broker perfectly exactly-once.

**Design**

1. Require every payment event to carry a unique idempotency key, generated by the original request (not by the retry), often the order ID or a client-generated UUID.
2. Before charging, check a durable store, a dedicated "processed_payments" table keyed on that idempotency key, or use the payment gateway's own idempotency key support (Stripe, for example, has this built in).
3. Perform the check-and-charge atomically: insert the idempotency key with a unique constraint as part of the same transaction as the charge attempt, so a duplicate insert fails cleanly instead of racing.
4. If the key already exists and the prior attempt succeeded, return the prior result rather than charging again. If the prior attempt is still in progress, reject or queue the duplicate rather than double-processing concurrently.
5. Log and alert on duplicate detections, since a high duplicate rate can signal an upstream retry storm or a broker misconfiguration worth investigating.

**Interview Answer:**

"I wouldn't try to prevent duplicates from ever arriving, at-least-once delivery makes that unrealistic. Instead I'd require an idempotency key on every payment event, tied to the original request, and check it against a processed_payments table with a unique constraint before charging, so the charge and the dedup check happen atomically. If the key's already been processed, I return the previous result instead of charging again. Most payment gateways, like Stripe, also support idempotency keys natively, which I'd use as a second layer of protection."

---

## Q18. [Senior] Design a notification system that fans out one event, like "order shipped", to email, SMS, and push notification services, each with very different throughput and failure characteristics.

**Answer:**

**The core challenge is that a single slow or failing channel, say the SMS provider having an outage, shouldn't block or slow down the others.**

**Design**

1. The order service publishes a single "OrderShipped" event to a topic (Kafka) or an SNS topic.
2. Each channel, email, SMS, push, has its own independent subscription: an SQS queue subscribed to the SNS topic, or its own Kafka consumer group. This gives each channel its own backlog, its own retry policy, and its own scaling profile, one channel's backlog never blocks another's.
3. Each channel's consumer talks to its own third-party provider (SES for email, Twilio for SMS, FCM/APNs for push) with provider-specific rate limiting and retry logic, since SMS providers often have far stricter rate limits than email providers.
4. Each channel has its own DLQ, since "email bounced because the address is invalid" and "SMS failed because the carrier rejected it" require different remediation.
5. For channels with strict rate limits (SMS), add a token bucket rate limiter in front of the consumer, or throttle the SQS visibility timeout and concurrency, so you don't get throttled or banned by the provider.

**Edge cases to raise**

- Idempotency per channel: a redelivered event should not send a second SMS, dedupe on (order_id, channel).
- User preferences: some users opt out of SMS, filter that at the consumer level or via a message attribute filter policy on the SNS subscription itself, so unwanted messages are dropped before even reaching that channel's queue.
- Partial failure visibility: if push succeeds but SMS fails, that should show up per channel in monitoring, not as a single opaque "notification failed" metric.

**Interview Answer:**
"I'd publish a single event and fan it out through SNS or a Kafka topic to three independent subscriptions, one per channel, each with its own queue, retry policy, and DLQ. That isolates failures, an SMS provider outage doesn't back up email or push. Each consumer respects its provider's own rate limits, SMS in particular needs a rate limiter since carriers throttle aggressively. I'd dedupe per channel using order ID plus channel as the key, and filter opted-out users before they even reach a channel's queue, using subscription filter policies where possible."

---

## Q19. [Senior] Walk me through what it would actually take to achieve exactly-once processing end to end, and where the guarantee tends to break down in practice.

**Answer:**

"Exactly-once" is really the combination of three things all being true together: no message is lost, no message is processed more than once, and side effects are applied consistently with that guarantee. **Achieving all three end to end is genuinely hard, and most systems approximate it instead.**

**What a real exactly-once pipeline requires**

1. **Produce side**: an idempotent producer (Kafka's `enable.idempotence=true`) to prevent duplicate writes from network retries, and transactional writes if producing to multiple partitions atomically.
2. **Process side**: Kafka Streams or a consumer using the transactional API can atomically commit the processing output and the input offset together, so either both happen or neither does, this is Kafka's "exactly-once semantics" (EOS) within the Kafka ecosystem.
3. **External side effects**: this is where it usually breaks. If the consumer's job is to write to an external database or call a third-party API, that write is not part of Kafka's transaction. A crash between "commit the Kafka offset" and "write to the database" (or the reverse order) can still produce a duplicate or a loss, unless the external system also participates via something like the outbox pattern or a two-phase commit, which most external systems don't support.

**Where it breaks down**

- Any time a message triggers a side effect outside the messaging system's own transactional boundary, such as calling a third-party payment API or sending an email, true exactly-once is not achievable, because the broker cannot know if that call succeeded if it crashes right after making it.
- Multi-system pipelines (Kafka to database to another Kafka topic) only get exactly-once if every hop is transactional and idempotent, one weak link degrades the whole pipeline to at-least-once.

**Practical stance**

- Design for at-least-once delivery with idempotent consumers as the default, and reserve true EOS machinery for cases fully contained within one system, like Kafka Streams aggregations, where it's actually achievable and worth the throughput cost.

**Interview Answer:**

"True exactly-once requires the produce, process, and offset-commit steps to be atomic together, which Kafka's transactional API can give you within its own ecosystem, like Kafka Streams. It breaks down the moment a side effect crosses outside that boundary, calling a third-party API or writing to an external database, because you can't atomically commit an offset and an external call together. In practice I design for at-least-once with idempotent consumers everywhere, and only reach for EOS machinery in fully Kafka-native pipelines where it's achievable and the throughput cost is worth it."

---

## Q20. [Senior] Your Kafka consumer group's lag keeps growing and isn't recovering. Walk me through how you'd diagnose it and fix it.

**Answer:**

**Growing, non-recovering lag means the consumer group is processing slower than the topic is being produced to, sustained over time, not just a momentary blip.**

**Diagnosis steps**

1. Check lag per partition, not just the aggregate, uneven lag across partitions often points to a hot key or skewed partitioning, one partition getting much more traffic than others.
2. Check consumer count versus partition count, if there are fewer active consumers than partitions, some partitions have no one reading them or a single consumer is overloaded covering multiple.
3. Look at per-message processing time on the consumer, a slow downstream dependency (a database, a third-party API call inside the consumer loop) is the most common root cause.
4. Check for frequent rebalances, if consumers are crashing or exceeding `max.poll.interval.ms` because processing takes too long per batch, the group keeps rebalancing, which pauses consumption entirely during each rebalance.
5. Check broker-side health, under-replicated partitions or a broker under heavy load can slow fetches even if consumers are healthy.

**Fixes depending on root cause**

- Hot partition: rekey to spread load more evenly, or split the hot key's traffic across a synthetic sub-key.
- Too few consumers: scale up to match partition count, this is the ceiling, more consumers than partitions won't help further without repartitioning.
- Slow per-message processing: move slow, blocking work (like a synchronous third-party call) out of the hot path, batch writes, or scale consumers.
- Frequent rebalances from slow processing: increase `max.poll.interval.ms`, reduce `max.poll.records` per batch, or move heavy work to an async pipeline so the poll loop returns quickly.
- If the topic simply outgrew its partition count long term, plan a partition increase (accepting the ordering caveat from key remapping).

**Interview Answer:**

"First I'd look at lag per partition, not the aggregate, to catch a hot key skewing load onto one partition. Then I'd compare consumer count to partition count, since that's the hard ceiling on parallelism. If those look fine, I'd check per-message processing time, a slow downstream call inside the consumer loop is the most common cause, and check whether frequent rebalances are pausing consumption because processing is exceeding max.poll.interval.ms. The fix depends on what I find: rekey a hot partition, scale consumers up

to the partition count, move blocking calls out of the hot path, or tune poll settings so the consumer doesn't keep getting kicked out of the group."

---

## Q21. [Senior] Design the migration of a monolith that writes directly to its database into an event-driven architecture using the outbox pattern, with zero data loss and no downtime.

**Answer:**

**This is a strangler-fig style migration: you introduce the new event-driven path alongside the old direct-write path, validate it, then cut over, rather than flipping a switch.**

**Phased plan**

1. **Add the outbox table**: create the outbox table in the existing monolith's database, and start writing to it in the same transaction as every relevant business write, without changing any existing behavior yet. This alone is safe, it's additive.
2. **Stand up the publisher**: deploy a CDC connector (like Debezium reading the database's write-ahead log) or a polling publisher that reads new outbox rows and publishes them to Kafka. Run this in "shadow mode" first, publish events but have no consumers acting on them yet, just verify volume and content look correct.
3. **Build and validate new consumers**: build the new event-driven consumers (say, a new inventory service) that will eventually replace direct database calls or triggers in the monolith. Run them in parallel with the existing logic, compare outputs (dual-write validation) without letting the new path affect production behavior yet.
4. **Cut over incrementally**: once validated, switch the source of truth for one workflow at a time to the event-driven path, monitoring closely, and keep the old path available to roll back to quickly if something's wrong.
5. **Backfill history**: for any workflow needing historical events (not just new ones going forward), run a one-time backfill job that reads existing rows and publishes synthetic "created" events into the same topic, clearly tagged as backfill (via a header or event type) so consumers can decide whether to treat them differently.
6. **Decommission the old path**: once the new path has run cleanly in production for a sufficient soak period, remove the old direct-write or trigger-based logic.

**Ensuring zero data loss**

- The outbox write is atomic with the business write from day one, so no event can be missed once the outbox table exists.
- CDC reading the write-ahead log (rather than polling with potential gaps) captures every committed transaction, including ones that happened before the publisher process was even running, since it reads from the log's position, not from application-triggered polls.
- Keep both old and new paths active during validation so any discrepancy is caught by comparison, not discovered after cutover.

**Interview Answer:**

"I'd treat this as a strangler migration, not a cutover. First, add an outbox table and start writing to it atomically with existing business writes, which is safe on its own. Then stand up a CDC-based publisher, Debezium reading the write-ahead log, in shadow mode to validate event content and volume without anything consuming them yet. I'd build new consumers and run them in parallel with the old logic,

comparing outputs before trusting them. Only after that validation would I cut over one workflow at a time, backfilling historical events as a separate labeled batch, and keep the old path around to roll back to until the new path has proven itself in production."

---

## Q22. [Senior] Design a system where two related event streams must be processed in strict order relative to each other, but the overall system still needs to scale horizontally. How do you reconcile those two requirements?

**Answer:**

Strict ordering and horizontal scaling are in tension by nature: parallelism is what breaks ordering, since concurrent workers can finish out of sequence. **The way to reconcile them is to scope ordering to the smallest unit that actually needs it, and parallelize across those units, not within them.**

**Design**

1. Identify the actual ordering scope: rarely does the entire system need one global order, usually order only matters within some entity, like all events for a given account ID, or a given order ID. Define that key precisely.
2. Partition by that key: route every event for the same key to the same partition (Kafka) or the same queue shard, using the key as the partition key, so Kafka's per-partition ordering guarantee lines up exactly with the ordering the business actually needs.
3. Scale horizontally across keys, not within a key: run as many partitions and consumer instances as there are independent keys' worth of parallel work, up to the partition count, each partition still processes sequentially, but many partitions run concurrently.
4. If two separate streams need to be ordered relative to each other (say, "PaymentAuthorized" must always be processed before "InventoryReserved" for the same order), consider merging them into a single topic with a shared partition key, so both event types for the same order land on the same partition and are naturally read in production order, rather than trying to coordinate two independently partitioned topics after the fact.
5. Within a single partition's consumer, process strictly sequentially, don't multi-thread processing of one partition's records if order matters, that reintroduces the exact problem partitioning solved.

**Edge cases**

- If ordering must be global (rare, and usually a sign the requirement should be revisited), you fundamentally cannot horizontally scale that specific stream, you'd run a single partition and accept it as a throughput ceiling, possibly a separate low-volume "sequencer" stream distinct from the bulk of traffic.
- Cross-partition causality (an event in partition A must be seen before a related event in partition B) usually requires a coordination point, like a saga or a small stateful service that buffers and sequences those cross-stream dependencies explicitly, since the broker alone won't guarantee it.

**Interview Answer:**

"The key insight is that ordering and scaling aren't actually opposed if you scope ordering correctly, most systems only need order within an entity, like an order ID, not globally. I'd partition by that key so Kafka's per-partition ordering lines up exactly with what the business needs, then scale by adding more partitions and consumers across different keys, while keeping processing within one partition strictly sequential. If two related event types need relative order, I'd put them in the same topic keyed the same way so they land on the same partition together, rather than trying to reconcile ordering across two independently partitioned streams after the fact."

## Q23. [Senior] A single malformed message is stuck looping through your consumer, retrying and failing over and over, and it's blocking everything behind it. Walk me through what's happening and how you'd prevent it.

**Answer:**

**This is the classic "poison message" scenario, and what makes it dangerous depends heavily on the ordering guarantee of the underlying system.**

**Why it blocks everything**

- In an ordered system like a single Kafka partition, the consumer cannot commit an offset past the failing message without skipping it, and if it keeps retrying that same offset without advancing, every message behind it in that partition sits unprocessed too, lag climbs, and downstream consumers relying on timely processing fall behind indefinitely.
- In a queue system like SQS or RabbitMQ, without ordering constraints, a poison message being retried usually doesn't block other messages the way it does in a Kafka partition, but it does keep consuming worker capacity and cluttering the visible queue.

**How to prevent and fix it**

1. Set a firm retry limit and route to a DLQ after it's exceeded, this is the single most important guardrail, discussed earlier for retry design.
2. For ordered systems specifically, the consumer needs explicit logic: after N failed attempts on a given offset, log it, publish it to a DLQ topic, and manually advance (commit) past that offset so the partition keeps flowing, don't let a single offset become a permanent blocker.
3. Add schema validation and defensive parsing at the very start of the consumer, before any business logic, so malformed payloads are caught immediately and routed to the DLQ on the first attempt instead of burning through retries.
4. Add schema enforcement upstream too, using a schema registry (Confluent Schema Registry, AWS Glue Schema Registry) so producers can't even publish a message that violates the expected schema in the first place, this prevents the problem rather than just handling it gracefully.
5. Monitor and alert on DLQ growth and on "same offset retried N times" as a specific signal, distinct from general lag, since it points to a different class of problem, a bad message, not a slow consumer.

**Interview Answer:**
"A poison message loops because the consumer keeps failing to process it and, especially in an ordered system like a Kafka partition, can't skip past it without deliberately advancing the offset, so everything behind it backs up. I'd cap retries, and once exceeded, explicitly log the message to a DLQ and commit past that offset so the partition keeps moving rather than stalling forever. I'd also add strict schema validation right at the start of the consumer so malformed messages get caught on the first attempt, and ideally enforce schema at the producer level with a schema registry so invalid messages can't be published in the first place."

---

## Q24. [Senior] Compare SQS standard, SQS FIFO, and Kafka partitions purely on ordering and throughput trade-offs. When would you actually use each?

**Answer:**

**All three can be used to build ordered processing, but they make very different trade-offs between strict ordering, throughput, and operational complexity.**

**SQS Standard**

- Pros: nearly unlimited throughput, simplest to operate, fully managed, automatic scaling.
- Cons: no ordering guarantee at all, messages can be delivered out of order and, rarely, more than once even outside normal retry behavior. Best-effort ordering only.

**SQS FIFO**

- Pros: strict ordering guaranteed within a message group (a group ID you assign, similar in spirit to a Kafka partition key), exactly-once processing support within a 5-minute deduplication window, simpler mental model than Kafka.
- Cons: significantly lower throughput ceiling (up to 3,000 messages per second with batching per API call, versus Kafka's much higher ceilings), ordering is still only per message group, not globally, across the whole queue.

**Kafka partitions**

- Pros: ordering guaranteed per partition, very high throughput per partition and overall (can scale to millions of messages per second across a cluster), long retention and replay, multiple independent consumer groups.
- Cons: requires you to size and manage partition count up front, changing it later disrupts the ordering guarantee for existing keys, meaningfully more operational overhead to run and tune well (unless using a managed offering like MSK or Confluent Cloud).

**Best Choice**

For simple task queues with no ordering need, SQS standard, for its simplicity and virtually unlimited throughput. For ordered processing at moderate volume without wanting to operate Kafka, SQS FIFO. For high-throughput ordered event streaming with replay and multiple independent consumers, Kafka.

**Why?**

The real axis of decision is throughput needed versus operational tolerance: SQS FIFO gives you ordering with almost no operational burden but a real throughput ceiling, Kafka removes that ceiling at the cost of running (or paying for) a more complex system.

**Interview Answer:**

"SQS standard has no ordering guarantee but scales almost without limit and requires no tuning. SQS FIFO gives strict ordering per message group with a real throughput ceiling, a few thousand messages per second, but it's fully managed and simple to reason about. Kafka partitions give ordering per partition too, but at far higher throughput and with replay and multiple independent consumer groups, at the cost of operational complexity, partition planning, and the fact that changing partition count later can break ordering for existing keys. I'd pick based on how much throughput and replay I actually need versus how much operational complexity the team can take on."

---

## Q25. [Senior] Design an event-driven saga to handle a distributed transaction across order, payment, and inventory services, without using a two-phase commit (a protocol that first asks every participant to prepare, then only commits for real once all of them confirm they can).

**Answer:**

A saga breaks a distributed transaction into a sequence of local transactions, each in its own service, coordinated through events, with explicit compensating actions to undo prior steps if a later step fails. **Two-**

**phase commit doesn't scale well across independently owned services and creates blocking dependencies, so sagas are the standard alternative.**

**Choreography-based design (services react to each other's events)**

1. Order service creates the order in a "pending" state and publishes `OrderCreated`.
2. Payment service consumes `OrderCreated`, attempts to charge the customer, and publishes either `PaymentSucceeded` or `PaymentFailed`.
3. Inventory service consumes `PaymentSucceeded`, attempts to reserve stock, and publishes `InventoryReserved` or `InventoryFailed`.
4. Order service consumes the terminal events: on `InventoryReserved`, mark the order "confirmed", on `InventoryFailed`, publish a compensating `RefundRequested` event that the payment service listens for to reverse the charge, and mark the order "failed".
5. Each compensating action must itself be idempotent and safe to redeliver, since the saga's messages carry the same at-least-once delivery guarantees as any other event.

**Orchestration-based alternative**

- Instead of services reacting to each other directly, a central saga orchestrator issues commands to each service in sequence and listens for their responses, deciding what to do next, including which compensations to trigger on failure.
- Trade-off: choreography avoids a central point of coordination logic but makes the overall flow harder to see and debug, since it's implicit across many services. Orchestration is easier to reason about and monitor, but introduces a central coordinator that needs its own reliability and becomes a service everyone depends on.

**Key correctness concerns**

- Every step needs a well-defined compensating action, some things (sending a confirmation email) can't be truly "undone", the compensation there might be sending a cancellation email instead.
- The saga must handle partial failure of the saga coordinator or a lost event mid-flow, usually via a timeout mechanism, if a service doesn't respond within an expected window, treat it as a failure and trigger compensation rather than waiting forever.
- The end state of a saga is only eventually consistent, the order might briefly show as "pending" while the saga plays out.

**Interview Answer:**
"I'd model this as a saga, a sequence of local transactions with compensating actions, since two-phase commit doesn't work well across independently owned services. In a choreography style, order publishes OrderCreated, payment reacts and publishes PaymentSucceeded or PaymentFailed, inventory reacts to success and publishes its own result, and order listens for the terminal events to either confirm or trigger a compensating refund. Every step needs an idempotent compensating action, and I'd add timeouts so a stuck or lost event triggers compensation rather than leaving the saga hanging forever. For more visibility I'd consider an orchestrator instead of pure choreography, trading a central coordination point for much easier debugging."

---

## Q26. [Senior] How do you handle schema evolution for events in Kafka when producers and consumers are deployed independently and can't all be upgraded at once?

**Answer:**

In an event-driven system, producers and consumers are decoupled in time and deployment, which means a producer can start emitting a new event shape while some consumers are still running old code expecting the old shape. **Schema evolution has to be handled deliberately, or you get silent breakage.**

**Approach**

1. **Use a schema registry**: adopt a schema format like Avro, Protobuf, or JSON Schema with a schema registry (Confluent Schema Registry or AWS Glue Schema Registry) that tracks every version of every event schema and enforces compatibility rules at publish time.
2. **Enforce a compatibility mode**: set the registry to backward compatible (new schema can read data written with the old schema) at minimum, so old consumers don't break when a new field appears. Backward and forward compatible together is the safest choice when you can't control consumer upgrade order at all.
3. **Only make safe changes**: adding a new optional field with a default value is safe. Removing a field, renaming a field, or changing a field's type are breaking changes and need a new schema version and, often, a new topic or a transition period where both old and new fields are populated.
4. **Never repurpose a field's meaning**: changing what an existing field means without changing its name is the most dangerous kind of "silent" breaking change, since it passes schema validation but corrupts business meaning.
5. **Version consumers defensively**: consumers should ignore unknown fields rather than failing on them, and handle missing optional fields with sensible defaults, this is what backward compatibility buys you in practice.
6. **Coordinate breaking changes explicitly**: for a genuinely breaking change (removing a required field, restructuring the event), introduce a new topic version (`order-events-v2`), dual-publish to both old and new during a transition window, and give consumer teams a deprecation timeline before retiring the old topic.

**Interview Answer:**

"I'd put a schema registry in front of the topic, using Avro or Protobuf, with backward compatibility enforced so a new schema can still be read correctly by consumers running old code. That means only additive, optional changes are allowed without a version bump, adding a new field with a default is fine, removing or renaming a field is not. For genuinely breaking changes, I'd version the topic itself, like order-events-v2, dual-publish during a transition window, and give consumer teams a clear deprecation timeline for the old version, rather than trying to force everyone to upgrade in lockstep."

---

## Q27. [Senior] Walk me through the trade-offs of SNS fanning out to SQS queues versus using Kafka, for a system where roughly 50 downstream services all need to consume the same event.

**Answer:**

**Fifty independent consumers of one event is a genuine fan-out problem, and the two approaches scale to it very differently.**

**SNS to SQS fan-out**

- Pros: each of the 50 consumers gets its own SQS queue subscribed to the SNS topic, fully managed, effectively zero infrastructure to run, each queue is independently durable and can retry, DLQ, and scale on its own without any coordination between teams. Adding a 51st consumer is a five-minute change, just subscribe a new queue, with no impact on the other 50.

- Cons: no replay, once a message is consumed and deleted from a given SQS queue it's gone from that queue, a newly added consumer only sees events from the moment it subscribes onward, not history. At high message volume, running 50 separate SQS queues each incurs its own cost and adds operational surface area to monitor (50 sets of DLQs, alarms, and metrics).

**Kafka**

- Pros: all 50 consumer groups read from the same topic and its retained log, a new consumer added later can replay recent history rather than only seeing events going forward. Very high throughput on a single topic can support many consumer groups without duplicating storage, since Kafka stores the log once regardless of how many groups read it.
- Cons: someone has to run and tune the Kafka cluster (or pay for a managed offering), and 50 independent consumer groups all reading the same topic means the topic's retention, partition count, and throughput have to be sized generously enough for the union of everyone's needs, a noisy or slow consumer group can still cause its own lag independently, but a Kafka outage is a single shared point of failure for all 50 consumers at once, versus SNS/SQS's independently isolated queues.

**Best Choice**

If most of the 50 consumers only need "events going forward" and value operational simplicity and strong isolation, SNS to SQS fan-out. If several of those consumers need to replay history, or the event volume is very high and storing 50 copies of it would be wasteful, Kafka.

**Why?**

The deciding factor is replay and storage efficiency versus operational simplicity and failure isolation: Kafka stores the log once for everyone and allows replay, but centralizes risk and operational burden, SNS/SQS isolates every consumer completely and needs no cluster management, but each queue is transient and 50 queues means 50 things to operate.

**Interview Answer:**

"With SNS fanning out to 50 SQS queues, each consumer gets full isolation, its own retries and DLQ, and it's fully managed, adding a 51st consumer doesn't touch the other 50. The trade-off is no replay, a new consumer only sees events from when it subscribes, and running 50 queues means 50 sets of alarms and DLQs to operate. Kafka stores the log once and lets any of the 50 consumer groups replay history independently, and scales better if volume is very high, but it centralizes risk, a cluster outage affects everyone at once, and someone has to run and tune it. I'd lean toward Kafka if replay matters to enough consumers or if volume is high enough that storing 50 copies would be wasteful, otherwise SNS to SQS is simpler and safer to operate."

---

## Q28. [Mid] What exactly is a Kafka partition, and why does the number of partitions you pick affect both ordering guarantees and how much you can parallelize consumption?

**Answer:**

**A partition is an ordered, append-only, immutable log, and it is the single unit Kafka uses both to guarantee order and to hand out work across consumers, which is why the partition count is one of the most consequential decisions in a Kafka design.** A topic is really just a named collection of one or more partitions, each partition lives on a broker, gets its own sequence of offsets, and is written to and read from independently of every other partition in the topic.

**Why partition count matters for ordering**

- Kafka only guarantees order within a partition, records in different partitions have no ordering relationship to each other at all.
- Since a producer's partitioner (by default, a hash of the record key) decides which partition a record lands on, everything with the same key always goes to the same partition, which is how you get ordering for a logical entity like a single user or order.

**Why partition count matters for parallelism**

- A partition can only be consumed by one consumer instance within a given consumer group at a time, so the partition count is a hard ceiling on how many consumer instances in that group can do useful work in parallel.
- Too few partitions means you cannot scale consumption further no matter how many machines you throw at it, extra consumers just sit idle.
- Too many partitions adds overhead: more open file handles and memory on the brokers, longer leader election during failover, and higher end-to-end latency for small messages.

**Practical guidance**

1. Size partitions for your expected peak consumer parallelism, not today's traffic, since increasing partition count later reshuffles the key-to-partition mapping and breaks ordering continuity for existing keys.
2. Treat partition count as a capacity planning decision made up front, closer to choosing a shard count for a database than a runtime tuning knob.

**Interview Answer:**
"A partition is an ordered, append-only log, and a topic is just one or more partitions spread across brokers. It matters for ordering because Kafka only guarantees sequence within a single partition, so I key records by whatever entity needs order, like a user or order ID, to keep them on the same partition. It matters for parallelism because only one consumer per group can read a given partition at a time, so partition count is the hard ceiling on how many consumers can do useful work concurrently. I size it for peak parallelism up front, since raising it later reshuffles keys and breaks ordering."

---

## Q29. [Mid] How do Kafka consumer groups work, and how does Kafka know which messages a given group has already processed?

**Answer:**

A consumer group is a named set of consumer instances that cooperate to read a topic, with Kafka guaranteeing that each partition is assigned to exactly one consumer within the group at any moment. **Kafka tracks progress not by deleting messages once they're read, but by having each consumer group record an offset, the position of the last record it has successfully processed, per partition.**

**How the mechanics work**

1. Every partition is a sequence of records numbered by offset, starting at 0 and increasing forever as new records are appended.
2. A consumer reads records starting just after its group's last committed offset for that partition.
3. After processing, the consumer commits the new offset, either automatically on a timer (`enable.auto.commit=true`) or manually after the side effect is durably applied, back to a special internal topic called `__consumer_offsets`.

4. Because offsets are stored per consumer group, multiple independent groups can read the exact same topic at completely different positions without interfering with each other, one group might be near the tail processing in real time, another might be replaying from an hour ago.

**Why this design matters**

- Records are never deleted just because they were read, retention is time or size based, which is what makes replay possible: reset a group's offset backward and it reprocesses history.
- Offset commit timing is the actual lever for delivery semantics: commit before processing risks losing a message if the consumer crashes mid-work, commit after processing risks reprocessing a message if the consumer crashes after finishing but before the commit lands.

**Interview Answer:**

"A consumer group is a set of consumers sharing the work of reading a topic, and Kafka assigns each partition to exactly one member of the group at a time. Progress is tracked with offsets, a per-partition, per-group pointer to the last record successfully consumed, stored in an internal topic called __consumer_offsets rather than by deleting messages. Because offsets live per group, different groups can read the same topic independently, one near real time, another replaying from earlier. Whether I commit the offset before or after processing directly determines whether I lean toward losing messages or duplicating them on a crash."

---

## Q30. [Mid] Explain the difference between at-least-once, at-most-once, and exactly-once delivery in Kafka, and what it actually takes to get exactly-once.

**Answer:**

These three describe what happens to a message relative to a consumer's processing when something fails partway through, a crash, a dropped connection, a slow network. **The difference comes down entirely to the order of two steps: doing the work and committing the offset that marks it done.**

**At-most-once**

- The consumer commits the offset before or immediately upon receiving the message, then processes it.
- If it crashes during processing, the offset is already committed, so on restart it moves on to the next message, the failed one is simply gone, never retried.
- Rarely what anyone actually wants, it trades data loss for simplicity, and is mostly seen as a byproduct of naive auto-commit configuration rather than a deliberate design choice.

**At-least-once**

- The consumer processes the message first, then commits the offset afterward.
- If it crashes after processing but before the commit lands, the message gets redelivered and processed again after a rebalance, since the last committed offset still points before it.
- This is Kafka's practical default, and the standard fix for the resulting duplicates is an idempotent consumer, not trying to eliminate the gap.

**Exactly-once**

- Requires that the act of processing and the act of committing the offset are atomic, either both happen or neither does, no window where one occurs without the other.

- Kafka provides this natively only within its own ecosystem: the transactional producer plus consumer API (the basis of Kafka Streams' exactly-once semantics) lets you atomically write output records and commit input offsets together in one transaction.
- The moment a side effect crosses outside Kafka, writing to an external database, calling a third-party API, that external write is not part of the transaction, so true exactly-once degrades back to at-least-once unless the external system also participates through something like the outbox pattern.

**Interview Answer:**

"At-most-once commits the offset before processing, so a crash mid-work loses the message for good. At-least-once processes first and commits after, so a crash between finishing the work and the commit lands causes a redelivery and a duplicate, this is Kafka's practical default. Exactly-once needs processing and the offset commit to be atomic, which Kafka's transactional API gives you inside its own ecosystem, like Kafka Streams. It breaks the moment a side effect touches something outside Kafka's transaction, an external database or API call, so in practice I design for at-least-once with idempotent consumers everywhere else."

---

## Q31. [Senior] Walk me through what actually happens inside a Kafka consumer group when a consumer joins or crashes, and why do frequent rebalances hurt throughput so much?

**Answer:**

**A rebalance is Kafka redistributing partition ownership across the live members of a consumer group, and while it's happening, every consumer in that group stops consuming, not just the one that joined or left.**

**What triggers a rebalance**

- A new consumer instance joins the group, say during a deploy or a scale-up.
- An existing consumer leaves cleanly, shuts down, or is considered dead because it missed too many heartbeats or failed to call `poll()` within `max.poll.interval.ms`.
- The topic's partition count changes.

**What happens step by step**

1. One broker acts as the group coordinator for that consumer group, tracking membership through periodic heartbeats sent on a background thread.
2. When membership changes, the coordinator triggers a rebalance, and in the classic (eager) protocol, every consumer in the group revokes all of its currently assigned partitions, even the ones staying with the same consumer.
3. Consumers rejoin the group via a `JoinGroup` request, the coordinator picks one as the group leader, which computes a new partition assignment across all members using the group's assignment strategy.
4. The new assignment is distributed via `SyncGroup`, and consumers resume fetching only once they have their new assignment, resuming from each partition's last committed offset.
5. Newer cooperative rebalancing protocols (`CooperativeStickyAssignor`) only revoke the specific partitions that need to move, letting unaffected consumers keep processing during the rebalance, which is a meaningfully cheaper version of the same event.

**Why frequent rebalances hurt throughput**

- Under the classic eager protocol, the entire group pauses consumption for the duration of the rebalance, no partition in that group is being read by anyone.

- If a consumer's processing loop is slow enough that it exceeds `max.poll.interval.ms` between calls to `poll()`, the coordinator assumes it's dead and kicks it out, triggering yet another rebalance, which can spiral into a loop where the group barely makes progress between constant reshuffles.
- Frequent rebalances are usually a symptom, not the root problem, common causes are consumers crashing due to unhandled exceptions, deploys that restart many consumers at once instead of rolling gradually, or per-message processing taking too long relative to the poll interval.

**Interview Answer:**

"When a consumer joins or is deemed dead by the group coordinator, missed heartbeats or a poll interval timeout, Kafka triggers a rebalance: partitions get revoked, consumers rejoin, a new assignment is computed and handed out, and only then does consumption resume. Under the classic eager protocol, the whole group stops during that window, not just the consumer that changed, so frequent rebalances mean the group spends more time reorganizing than actually processing. That's usually a symptom of consumers crashing, slow per-message processing blowing past max.poll.interval.ms, or deploys restarting too many consumers at once, and cooperative rebalancing helps but doesn't remove the root cause."

---

## Q32. [Senior] How does Kafka replication work, what do the leader and follower brokers for a partition actually do, and what does the in-sync replica set tell you about your durability guarantees?

**Answer:**

Every partition has a configurable number of replicas, copies of that partition's log spread across different brokers, and exactly one of those replicas is the leader at any given time. **Producers and consumers only ever talk to the leader for a partition, followers exist purely to replicate the leader's log and stand ready to take over if the leader fails.**

**How replication works**

1. The leader broker for a partition accepts all writes and reads for it.
2. Follower brokers continuously fetch new records from the leader, the same way a consumer would, and append them to their own local copy of the log.
3. A follower is considered "in sync" if it has fetched and replicated up to (within a bounded lag of) the leader's latest offset, the set of such replicas, including the leader itself, is the in-sync replica set, or ISR.
4. If the leader broker fails, Kafka's controller elects a new leader, but only from among the current ISR, this is what prevents data loss on failover, a replica that had fallen behind and wasn't in sync is not eligible to become leader and silently drop the tail of the log.

**Why ISR is the durability lever**

- `acks=all` on the producer means the leader waits for the write to be acknowledged by all current members of the ISR, not just written to the leader's own disk, before confirming the write to the producer.
- `min.insync.replicas` sets a floor, if the ISR shrinks below that number (because followers fell behind or brokers went down), the leader rejects writes rather than accepting them with a false sense of durability.
- Together, `acks=all` and a `min.insync.replicas` of at least 2 (with a replication factor of 3) is the standard combination for not losing acknowledged writes even if the leader broker dies immediately after acknowledging them.

- Choosing `acks=1` (leader only) or `acks=0` (fire and forget) trades this durability for lower latency and higher throughput, a legitimate choice for less critical data, but it means an acknowledged write can still be lost if the leader fails before followers catch up.

**Interview Answer:**
"Each partition has a leader broker that handles all reads and writes, and follower brokers that continuously replicate the leader's log and can take over if it fails. The in-sync replica set is whichever replicas, including the leader, are caught up within Kafka's allowed lag, and leader election on failover only picks from that set, which is what prevents a stale replica from silently losing data. The real durability lever is combining acks=all, so the leader waits for the whole ISR to acknowledge a write, with a min.insync.replicas floor, so if the ISR shrinks too far the leader refuses writes instead of pretending they're safely durable."

---

## Q33. [Senior] Why can Kafka sustain such high write throughput compared to a lot of traditional messaging systems? Walk me through the underlying mechanics.

**Answer:**

Kafka's throughput advantage comes less from any single clever trick and more from a set of design decisions that all point the same direction: avoid random I/O, avoid unnecessary copying, and let data move in bulk. **The single biggest factor is that Kafka only ever appends to the end of a partition's log and never does random-access writes or in-place updates, which turns every write into a fast sequential disk operation.**

**Sequential writes**

- Sequential disk I/O, even on spinning disks, and especially with modern SSDs, is dramatically faster than random I/O, because there's no seek time and the OS can read or write large contiguous blocks efficiently.
- Since a partition is append-only, a producer's write is always just "append to the end," Kafka never has to seek to some arbitrary offset and rewrite existing data.

**Leaning on the OS page cache**

- Kafka deliberately writes to the filesystem and relies on the operating system's page cache rather than managing its own in-process cache of recent records.
- Since most reads are near the tail of the log (consumers reading close to real time), those recent records are almost always still sitting in the page cache, so a "read" is often served straight from RAM without touching disk at all.

**Zero-copy sends**

- When serving data to a consumer, Kafka uses the `sendfile` system call to transfer bytes directly from the page cache to the network socket, without copying that data into the Java process's user-space buffers first.
- This avoids several redundant copies and context switches that a naive read-then-write implementation would incur, meaningfully reducing CPU overhead per byte shipped.

**Batching and compression**

- Producers batch multiple records destined for the same partition into a single request instead of sending one record per network round trip, amortizing per-request overhead across many records.
- Batches are typically compressed (like with `lz4` or `zstd`) before being sent, reducing network bandwidth and the amount of data actually written to and read from disk.

- Consumers fetch in similarly sized batches, so the efficiency gain applies symmetrically on the read side too.

**Interview Answer:**
"Kafka's throughput comes from stacking several cheap wins on top of each other. It only appends to the end of a log, so every write is sequential disk I/O instead of random access, which is the biggest single factor. It relies on the OS page cache instead of a custom in-process cache, so reads near the tail of the log are usually served straight from RAM. Sending data to consumers uses a zero-copy sendfile call straight from the page cache to the network socket, skipping extra copies. On top of that, producers and consumers batch and compress records instead of handling them one at a time, which amortizes overhead across many records at once."

---

**If you remember nothing else from this chapter:** *if a consumer isn't idempotent, at-least-once delivery is just a delayed way of saying it will process this twice.*

# Chapter 11: API Design & Communication Protocols

How you choose between REST, gRPC, GraphQL, and WebSockets, and whether you can defend that choice with concrete trade-offs instead of familiarity, tells an interviewer a lot about how you'll make similar calls on the job. This chapter builds that judgment.

> **War Story:** *A "backwards compatible" API change that just added one new required field broke every mobile client that hadn't updated yet, since backwards compatible meant nothing to an app already installed on a million phones.*

> **Avoid:** not considering client versions already in the wild when discussing a breaking change
> **Aim For:** raising client version skew as a real constraint on any change that isn't purely additive

## Q1. [Junior] What makes an API RESTful? Walk me through the core principles.

**Answer:**

**REST (Representational State Transfer) is an architectural style for designing networked APIs around resources, not actions.**

**Core principles**

- **Resource based URLs**: nouns, not verbs. `/orders/123`, not `/getOrder?id=123`.
- **Statelessness**: each request carries everything the server needs to process it. The server holds no session state about the client between requests.
- **Standard HTTP methods**: GET to read, POST to create, PUT/PATCH to update, DELETE to remove. The method conveys the intent, not the URL.
- **Uniform interface**: consistent conventions for status codes, headers, and response shapes across every endpoint.
- **Cacheability**: responses declare whether they can be cached, using headers like `Cache-Control` and `ETag`.
- **Representations**: a resource is returned as a representation, usually JSON, and the client manipulates the resource through that representation.

A lot of what people call "REST" in practice is really "HTTP plus JSON with resource shaped URLs," and that is fine, full HATEOAS style REST with hypermedia links is rare in real systems.

**Interview Answer:**
"REST models an API around resources identified by URLs, manipulated with standard HTTP verbs like GET, POST, PUT, and DELETE. The big architectural principle is statelessness, every request has to carry all the context the server needs, so any server instance can handle any request. In practice that means predictable, cacheable, resource oriented endpoints like `/users/42/orders` instead of RPC style `/getUserOrders`."

## Q2. [Junior] What's the difference between PUT and PATCH, when would you use each?

**Answer:**

Both update an existing resource, but they differ in how much of the resource they expect you to send.

**PUT**

- Replaces the entire resource with what you send in the request body.
- If you omit a field, many implementations treat that as clearing it to null or default, because PUT means "this is now the full state of the resource."
- Idempotent: sending the same PUT twice leaves the resource in the same state.

**PATCH**

- Applies a partial update, only the fields you include change.
- Better fit for large resources where you only want to touch one or two fields, like updating just a user's email.
- Idempotency depends on how the patch is expressed. A patch like `{"email": "new@x.com"}` is idempotent since replaying it sets the same value. A patch like `{"increment": 1}` is not, since replaying it changes the result each time.

**Best Choice**

Use PUT when the client legitimately owns and sends the full resource representation on every update. **Use PATCH when clients only ever change a subset of fields, which is the common case for most CRUD APIs.**

**Why?**

PATCH avoids accidental data loss from clients that only know about part of the resource, and it reduces payload size. PUT is simpler to reason about when the full object is already in memory client side, like a form save.

**Interview Answer:**
"PUT replaces the whole resource, so the body should represent the complete new state, and it's naturally idempotent. PATCH applies a partial update to specific fields. I default to PATCH for most update endpoints because clients usually only want to change one or two fields and I don't want a client with a stale copy of the object to accidentally wipe out fields it didn't know about."

---

## Q3. [Junior] Is DELETE idempotent? Walk me through why or why not.

**Answer:**

**Yes, DELETE is defined as idempotent by the HTTP specification, and it should behave that way in a well designed API.**

**Why it's idempotent**

- The end state after calling DELETE once or calling it five times in a row is the same: the resource no longer exists.
- The first DELETE removes the resource and typically returns 200 or 204.
- Subsequent DELETE calls on the same resource should return 404, since it's already gone, but the state of the system, "resource does not exist," has not changed further.

**Where teams get this wrong**

- Returning a 500 error on the second DELETE call because the code assumes the row must exist. That breaks idempotency and causes retry storms from clients that assume DELETE is safe to retry.

- Soft deletes complicate this a bit, if a second DELETE call throws because the record is already marked deleted, that violates the idempotency contract even though technically nothing bad happens to the data.

**Interview Answer:**
"Yes. Idempotent means calling it once or many times leaves the system in the same state, and after a DELETE the resource is gone whether you call it once or five times. The only nuance is the HTTP status code: the first call might return 204, later calls should return 404 rather than error out, since throwing an error on a repeat DELETE would break the idempotency guarantee that clients rely on for safe retries."

---

## Q4. [Junior] What does idempotency mean in the context of an API, and why does it matter for POST requests?

**Answer:**

**Idempotency means that making the same request multiple times produces the same result as making it once, with no unintended side effects from the repeats.**

**Why it matters**

- Networks are unreliable. A client sends a request, the server processes it successfully, but the response is lost before the client sees it. The client, not knowing whether it succeeded, retries.
- GET, PUT, and DELETE are idempotent by design under the HTTP spec.
- POST is explicitly not idempotent by default, because POST usually means "create a new thing." Retrying a POST for "charge this card $50" or "place this order" can create duplicates: double charges, duplicate orders.

**How to fix it for POST**

- Require the client to generate a unique idempotency key, usually a UUID, and send it in a header like `Idempotency-Key`.
- The server stores the key alongside the result of the first successful request.
- If the same key arrives again, the server returns the stored result instead of re-executing the operation.

**Interview Answer:**
"Idempotency means repeating the same request doesn't change the outcome beyond the first call. GET, PUT, and DELETE get this for free by design, but POST usually creates something new, so retrying a POST after a timeout can create duplicates. That's why payment and order creation APIs ask clients to pass an idempotency key, so the server can recognize a retry and return the original result instead of processing the request twice."

---

## Q5. [Junior] Explain offset based pagination. How would you implement page and limit query parameters?

**Answer:**

**Offset pagination splits a large result set into pages by telling the database how many rows to skip and how many to return.**

**How it works**

1. Client requests `GET /products?page=3&limit=20`.

2. Server translates that into `OFFSET 40 LIMIT 20` (skip the first 40 rows, return the next 20).
3. Response includes the page of results plus metadata like total count and total pages.

**Why it's popular**

- Simple to implement and simple for clients to understand.
- Supports jumping to an arbitrary page, "take me to page 7," which cursor pagination doesn't support well.
- Works naturally with UI page number controls.

**Downsides to mention**

- `OFFSET` gets slower as the offset grows, the database still has to scan and discard all the skipped rows.
- If rows are inserted or deleted between page requests, items can shift, causing duplicates or skipped items across pages.

**Interview Answer:**
"Offset pagination uses page and limit parameters, which the server converts into SQL OFFSET and LIMIT. It's easy to build and lets users jump to any page number, which is great for admin dashboards or search results with page numbers. The trade off is performance on large offsets, since the database still has to walk through and discard all the skipped rows, and results can shift if data changes between page loads."

---

## Q6. [Junior] What is a webhook, and how is it different from a client polling an API?

**Answer:**

**A webhook is a way for a server to push a notification to a client when an event happens, instead of the client repeatedly asking "did anything change yet?"**

**Polling**

- The client calls the API on a schedule, say every 10 seconds, to check for new data.
- Simple to build, but wasteful, most of those calls return "nothing new," and there's a delay between the event happening and the client noticing it.

**Webhooks**

- The client registers a URL with the server ahead of time.
- When the event occurs, for example "order shipped," the server makes an HTTP POST request to that URL with the event payload.
- The client's endpoint receives the event immediately without asking for it.

**Trade offs**

- Webhooks reduce wasted traffic and give near real time delivery.
- They require the client to run a publicly reachable endpoint, handle retries if their endpoint is briefly down, and verify the request actually came from the real sender.

**Interview Answer:**
"Polling means the client keeps asking the server if anything changed, which wastes requests and adds delay. A webhook flips that around, the client gives the server a callback URL up front, and the server pushes an HTTP request to that URL the moment the event happens. It's more efficient and near real time, but it

puts more responsibility on the client, they need a reachable endpoint, retry handling, and a way to verify the payload is authentic."

---

## Q7. [Mid] Compare REST and GraphQL. Which would you pick for a mobile app with limited bandwidth?

**Answer:**

**Both are ways to expose data over HTTP, but they differ in how the client asks for data.**

**REST**

- Fixed endpoints, each returning a fixed shape of data, `/users/42` returns the full user object.
- Pros: simple mental model, plays well with HTTP caching, easy to version and monitor per endpoint.
- Cons: overfetching, you get fields you don't need, and underfetching, you need multiple round trips to assemble a screen, for example fetching a user then separately fetching their orders.

**GraphQL**

- Single endpoint, the client sends a query describing exactly the fields it wants across multiple related resources in one request.
- Pros: no overfetching, no underfetching, the client controls the shape of the response, which is great for mobile where every byte and round trip costs battery and latency.
- Cons: harder to cache at the HTTP layer since every query can be shaped differently, more complex server implementation, risk of expensive nested queries if not guarded with depth limits and complexity analysis.

**Best Choice**

GraphQL for the mobile app described here.

**Why?**

Mobile clients on variable networks benefit most from GraphQL's ability to fetch exactly the fields needed for a screen in a single round trip, avoiding both wasted bytes and the latency of multiple sequential REST calls. The added server complexity is worth it because the win, fewer round trips and smaller payloads, directly targets the constraint in the question, limited bandwidth.

**Interview Answer:**

"For a bandwidth constrained mobile app, I'd lean GraphQL. It lets the client ask for exactly the fields it needs across related resources in a single request, which cuts both payload size and round trips compared to REST, where you'd either overfetch a fixed endpoint or make several calls to stitch together a screen. The cost is more server side complexity, like query depth limiting and different caching strategy, but that trade off favors the client experience here."

---

## Q8. [Mid] What's the overfetching and underfetching problem in REST, and how does GraphQL address it?

**Answer:**

**This is one of the main motivations behind GraphQL's design.**

**Overfetching**

- A REST endpoint returns a fixed shape, so a mobile screen that only needs a user's name and avatar still gets the full user object: address, preferences, account metadata, all of it.
- Wastes bandwidth and parsing time, especially painful on mobile.

**Underfetching**

- The opposite problem: a single REST endpoint doesn't return everything a screen needs, so the client has to make several calls. For example, load the user, then load their recent orders, then load product details for each order, three or more round trips for one screen.
- Each round trip adds latency, and it compounds on slow networks.

**How GraphQL addresses both**

- The client sends a single query specifying exactly which fields and nested relationships it wants.
- The server resolves that query in one request, returning exactly that shape, nothing more, nothing less.
- This collapses what would be several REST round trips into one GraphQL request, and eliminates unused fields in the response.

**The trade off to mention**

GraphQL pushes complexity to the server: resolvers need to avoid the N+1 query problem when fetching nested fields, and the server needs query cost analysis to stop a client from requesting deeply nested, expensive data in one shot.

**Interview Answer:**
"Overfetching is getting more data than you need because a REST endpoint returns a fixed shape. Underfetching is the opposite, needing multiple REST calls because no single endpoint returns everything a screen needs. GraphQL fixes both by letting the client specify exactly the fields and relationships it wants in one query, resolved in a single round trip. The cost shifts to the server, which now needs to guard against expensive nested queries and avoid N+1 database calls when resolving those nested fields."

---

## Q9. [Mid] Walk me through how you'd version a public REST API. Compare URI versioning, header versioning, and query param versioning.

**Answer:**

**Versioning lets you evolve an API with breaking changes without breaking existing clients who haven't upgraded yet.**

**URI versioning**

- Example: `/v1/orders`, `/v2/orders`.
- Pros: explicit and visible, easy to route at the load balancer or gateway level, easy for developers to test in a browser or curl.
- Cons: technically "the same resource" now lives at two different URLs, which is a bit at odds with REST's idea that a URL identifies a resource.

**Header versioning**

- Example: `Accept: application/vnd.myapi.v2+json` or a custom header like `Api-Version: 2`.
- Pros: keeps URLs clean and stable, resource identity doesn't change across versions.
- Cons: less discoverable, harder to test casually, caching proxies need to vary on the header correctly or they'll serve the wrong version.

**Query param versioning**

- Example: `/orders?version=2`.
- Pros: simple to add, easy to default when omitted.
- Cons: mixes versioning concerns into query strings used for filtering and pagination, and it's easy for clients to forget to pass it.

**Best Choice**

URI versioning for public APIs consumed by many external teams.

**Why?**

It is the most explicit and discoverable option, it's trivial to route different versions to different backend deployments at the gateway, and external developers can see and test the version directly in the URL without needing to know about custom headers. Most large public APIs, Stripe and GitHub included, use a version identifier that's either in the URL or extremely visible in documentation for this reason.

**Interview Answer:**
"I'd use URI versioning, something like `/v1/` and `/v2/`, for a public API. It's the most explicit option, it's trivial to route at the gateway level to different backend versions, and external developers can test it directly without knowing about custom headers. Header based versioning is more RESTfully pure since the URL stays stable, but it's less discoverable and complicates caching. I reserve header versioning for internal APIs where I control both sides of every call."

---

## Q10. [Mid] Compare offset pagination and cursor based pagination. When does offset pagination start to break down?

**Answer:**

Both paginate a large result set, but they anchor to different things.

**Offset pagination**

- Anchors to a position: "skip N rows, take M rows."
- Breaks down as N grows: the database still has to scan and discard all N skipped rows, so page 5,000 is dramatically slower than page 1.
- Also breaks down under concurrent writes: if a row is inserted before the current offset while a user is paging, an item can shift and either get duplicated across pages or skipped entirely.

**Cursor based pagination**

- Anchors to a value: "give me the next M rows after this specific row," typically using an indexed column like `id` or `created_at` plus `id` as a tiebreaker.
- Query looks like `WHERE (created_at, id) < (:last_created_at, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20`.
- Performance stays constant regardless of how deep you page, since the database uses the index to seek directly to the cursor position instead of scanning from the start.
- Stable under concurrent writes, new rows inserted don't shift the position of a cursor that already points at a specific row.
- Downside: no jumping to an arbitrary page number, you can only go forward or backward from where you are.

**Best Choice**

Cursor pagination for infinite scroll feeds, activity logs, or any large or frequently changing dataset. Offset pagination for admin style UIs with numbered pages over smaller, fairly static datasets.

**Why?**

**The deciding factor is dataset size and mutation rate.** Offset pagination's cost grows with depth and its correctness suffers under concurrent writes, both of which get worse exactly when the dataset is large, which is when pagination matters most.

**Interview Answer:**

"Offset pagination skips N rows and takes M, which is simple but gets slower the deeper you page, since the database still scans past every skipped row, and it can show duplicate or missing items if rows are inserted while someone is paging. Cursor pagination anchors to the last seen row's sort key and asks for rows after it, so it stays fast and stable regardless of depth because the database can seek directly using an index. I'd use cursor pagination for any large or frequently updated dataset, and reserve offset pagination for small, mostly static lists where users want to jump to page 7."

---

## Q11. [Mid] What's the difference between rate limiting and throttling?

**Answer:**

People often use these interchangeably, but they solve related, slightly different problems.

**Rate limiting**

- Enforces a hard cap on how many requests a client can make in a given time window, for example 1000 requests per hour.
- Once the cap is hit, further requests are rejected outright, typically with an HTTP 429 status and a `Retry-After` header.
- Purpose: protect the system from abuse, enforce fair usage across tenants, and enforce pricing tiers.

**Throttling**

- Controls the rate at which requests are processed rather than outright rejecting them, often by slowing down or queueing requests instead of dropping them.
- Can also refer to dynamically reducing a client's allowed rate under system load, distinct from a fixed, pre-agreed quota.
- Purpose: protect system stability under load, smooth out bursts, and give the system breathing room without necessarily denying service.

**Practical distinction**

**Rate limiting is usually a hard, quota style contract, "you get X requests per period, no more."** Throttling is more about pacing, "I will process your requests but might slow you down when the system is under pressure," which may or may not reject anything.

**Interview Answer:**

"Rate limiting is a hard quota, a client gets a fixed number of requests per time window, and anything past that gets rejected with a 429. Throttling is more about controlling the pace of processing, slowing down or queuing requests under load rather than necessarily rejecting them outright. In practice they're often combined: rate limiting enforces the contract with each client, and throttling protects the overall system when it's under stress, even from clients within their quota."

## Q12. [Mid] How would you design idempotency keys for a POST /payments endpoint so retries don't double charge a customer?

**Answer:**

**The goal is to make a naturally non-idempotent operation, "charge this card," safe to retry.**

**Design**

1. The client generates a unique key, typically a UUID, for each logical payment attempt, and sends it in an `Idempotency-Key` header.
2. On receiving a request, the server checks a store, a database table or a fast key value store like Redis, for that key.
3. If the key hasn't been seen: the server locks the key, processes the payment, stores the result, status code and response body, keyed by that idempotency key, then returns the response.
4. If the key has been seen and the original request completed: return the stored response immediately without charging again.
5. If the key has been seen but the original request is still in flight, for example a previous attempt is mid processing: return a 409 or make the second request wait briefly, to avoid two concurrent charges racing each other.

**Key details worth mentioning**

- Store the request payload hash alongside the key, and reject reuse of the same key with a different payload, since that usually indicates a client bug.
- Expire idempotency keys after a reasonable window, like 24 hours, since clients won't retry indefinitely.
- The uniqueness check and the payment charge need to happen atomically, using a database unique constraint on the idempotency key column is a common way to prevent a race where two requests with the same key both pass the "not seen yet" check simultaneously.

**Interview Answer:**
"I'd have the client generate a UUID per payment attempt and send it as an Idempotency-Key header. The server stores that key with a unique constraint alongside the eventual response. On a retry with the same key, it looks up the stored result and returns it instead of charging the card again. If a request with that key is still processing, I'd return a conflict or make the retry wait, so two concurrent attempts with the same key can't both go through. I'd also expire keys after 24 hours and reject reuse of a key with a different payload."

---

## Q13. [Mid] When would you reach for gRPC instead of REST for a service to service call?

**Answer:**

**gRPC is a binary, contract first RPC (Remote Procedure Call: calling a function on another service as if it were local) framework built on HTTP/2, and it shines in specific scenarios rather than being a universal upgrade over REST.**

**When gRPC makes sense**

- **Internal microservice to microservice calls**: you control both the client and server code, so the tight coupling to a shared `.proto` schema isn't a problem, it's actually a benefit.

- **High throughput, low latency requirements**: Protocol Buffers serialize to a compact binary format that's faster to encode and decode than JSON, and HTTP/2 multiplexing avoids head of line blocking across multiple calls on one connection.
- **Streaming needs**: gRPC natively supports client streaming, server streaming, and bidirectional streaming, useful for things like a live feed of updates between two services.
- **Polyglot environments**: the `.proto` file generates strongly typed client and server code in many languages, which keeps contracts consistent across teams using different stacks.

**When REST is still the better fit**

- Public APIs consumed by external developers or browsers, since gRPC needs special proxying (gRPC-Web) to work from a browser, and REST/JSON is far more universally understood and debuggable with just curl.
- APIs where human readability and easy debugging with basic tools matter more than raw performance.

**Best Choice**

gRPC for internal, high volume service to service communication. REST for public facing or browser facing APIs.

**Why?**

The performance and streaming benefits of gRPC matter most on the hot path between your own services, while REST's simplicity, wide tooling support, and human readability matter most when the consumer is external or unpredictable.

**Interview Answer:**

"I'd reach for gRPC for internal service to service calls where I control both ends and care about throughput and latency, since Protocol Buffers are more compact than JSON and HTTP/2 gives multiplexing and native streaming. For anything public facing, browser facing, or consumed by external partners, I'd stick with REST, since it's easier to debug, doesn't need special browser proxying, and is the format most third party developers already expect."

---

## Q14. [Mid] You're building a webhook system to notify customers when an order ships. Walk me through the design. What happens if their endpoint is down?

**Answer:**

**The core design has three parts: registration, delivery, and failure handling.**

**Registration**

- Customers register a callback URL and select which events they want, "order.shipped," "order.cancelled," and so on.
- Generate a shared signing secret per customer at registration time, used later to sign payloads.

**Delivery**

1. When "order.shipped" happens, publish an internal event.
2. A delivery worker builds the payload, signs it with HMAC using the customer's secret, and sends an HTTP POST to their registered URL with the signature in a header.
3. The customer's endpoint should respond quickly with a 2xx status to acknowledge receipt.

**What happens if their endpoint is down**

- Treat any non-2xx response, timeout, or connection failure as a delivery failure.
- Retry with exponential backoff, for example 1 minute, 5 minutes, 30 minutes, a few hours, spread over 24 to 72 hours.
- After exhausting retries, mark the event as failed and expose it in a dashboard or dead letter log the customer can inspect, and ideally notify them out of band, like an email.
- Give customers a way to replay a failed webhook manually.

**Other things worth mentioning**

- Include an event ID in the payload, and expect the customer's endpoint to be idempotent, since retries can result in duplicate deliveries if the first attempt actually succeeded but the acknowledgment was lost.
- Verify the payload signature on their side to make sure the request genuinely came from you.

**Interview Answer:**
"I'd let customers register a URL and event types, and sign each payload with a per customer secret so they can verify authenticity. Delivery is a worker that POSTs the event and expects a fast 2xx back. If their endpoint is down or times out, I retry with exponential backoff over a day or two, then mark it failed and surface it in a dashboard with a manual replay option. I'd also include a unique event ID and document that their endpoint needs to be idempotent, since retries can occasionally cause duplicate deliveries."

---

## Q15. [Mid] Design pagination for an infinite scroll social media feed. Which strategy fits best and why?

**Answer:**

**Infinite scroll has specific characteristics that point strongly toward one pagination style.**

**Characteristics of the problem**

- The feed is large and constantly growing, new posts are inserted at the top continuously.
- Users only scroll forward, they rarely jump to "page 500."
- Consistency matters less than a smooth, non-duplicating experience, users don't notice or care if a post they haven't seen yet appears slightly later.

**Options**

- **Offset pagination**: breaks down here specifically because new posts inserted at the top shift everything below, causing duplicate or skipped posts as the user scrolls, on top of the general performance problem with deep offsets.
- **Cursor based pagination**: anchor the cursor to something stable per item, like `(created_at, id)` of the last item the client has seen. Request the next batch with `WHERE (created_at, id) < (:cursor)`, which is unaffected by new inserts at the top.

**Design**

1. Return a page of posts plus an opaque `next_cursor` token encoding the last post's sort key.
2. Client stores that cursor and sends it back on the next scroll triggered fetch.
3. Encode the cursor as an opaque, ideally signed or encrypted, token so clients don't build assumptions around its internal structure, which gives you freedom to change the underlying sort key later.

**Best Choice**

Cursor based pagination.

**Why?**

It's the only approach that's both fast regardless of depth and immune to the "shifting list" problem caused by new posts being inserted at the top while the user scrolls, which is exactly what happens continuously in a social feed.

**Interview Answer:**

"I'd use cursor based pagination anchored on something like created_at plus post id as a tiebreaker. Offset pagination breaks here because new posts get inserted at the top constantly, which shifts every page below and causes duplicates or gaps as the user scrolls. A cursor pointing at the last post the client has seen avoids that entirely, and it stays fast at any scroll depth since the database can seek using an index instead of scanning and discarding rows."

---

## Q16. [Mid] Walk me through which HTTP status codes you'd return for a validation error, a missing resource, and a duplicate resource.

**Answer:**

**Status codes communicate the category of outcome to the client, and picking the right one lets clients handle errors programmatically instead of parsing error message text.**

**Validation error**

- Use `400 Bad Request` for malformed or missing required fields, wrong data types, or violated business rules on the input itself.
- Some APIs use `422 Unprocessable Entity` specifically when the request is well formed JSON but fails semantic validation, for example a valid email format for a field that must be unique and isn't. Either is defensible, the important part is being consistent across your API.

**Missing resource**

- Use `404 Not Found` when the requested resource doesn't exist, for example `GET /orders/999` where order 999 was never created or was deleted.

**Duplicate resource**

- Use `409 Conflict` when the request would create a resource that violates a uniqueness constraint, for example creating a user with an email that already exists.

**General principle**

- `4xx` codes mean the client did something the server considers invalid, fix the request and it might work.
- `5xx` codes mean the server failed to do something it should have been able to do, the client's request was fine.
- Pair every error status with a structured JSON error body containing a machine readable error code and a human readable message, so clients can branch on the code without string matching.

**Interview Answer:**

"Validation errors get a 400, or 422 if I want to distinguish malformed input from semantically invalid input, that's a call I'd make once and keep consistent across the API. A missing resource gets a 404. A duplicate, like trying to create a user with an email that's already taken, gets a 409 Conflict. I always pair the status code

with a structured error body that includes a stable error code, so clients can branch on that instead of parsing an error message string."

---

## Q17. [Junior] A DELETE request succeeds and the resource is gone. Should the API return 200 or 204, and does the response body matter?

**Answer:**

**204 means "success, and there is deliberately no body," so the moment you return 204 alongside a JSON body, you've violated the meaning of the status code itself, even though plenty of APIs get this wrong.**

**204 No Content**

- Signals success with nothing further to say. The response must not include a body.
- Correct choice when the client already knows what it deleted and doesn't need anything echoed back.
- A common bug: returning 204 but still attaching a JSON body like `{"message": "deleted"}`. Some HTTP clients and proxies will strip it, others will pass it through inconsistently, so behavior becomes unpredictable across clients.

**200 OK**

- Signals success and comes with a body. Appropriate if you want to return something useful, like the final state of a soft-deleted resource, or a summary of what was removed.
- If you choose 200, you must actually include a meaningful body. Returning 200 with an empty body just confuses clients that expect content.

**Best Choice**

204 with a genuinely empty body for a simple hard delete. 200 with a real, useful body only if you have something worth returning.

**Why?**

Consistency matters more than which one you pick: a client integrating against your API should never have to special case "sometimes DELETE has a body, sometimes it doesn't" for the same endpoint.

**Interview Answer:**
"For a simple delete, I'd return 204 with no body at all, since that's exactly what 204 means and it keeps the contract simple. If I return 200 instead, I make sure there's an actual meaningful body, like the deleted resource's final state, otherwise I'm sending a status code that implies content with none attached. The bug I watch for in reviews is 204 responses that still carry a JSON body, that's a contract violation that behaves inconsistently across HTTP clients."

---

## Q18. [Junior] Your API returns 401 Unauthorized when a logged in user tries to access someone else's private order. Is that the right status code, and why might an API deliberately return 404 instead of 403 here?

**Answer:**

**401 is wrong here since the user is authenticated, the actual problem is authorization, not identity, so this should be 403, though many APIs deliberately use 404 instead for a security reason.**

**Why 401 is the wrong code**

- 401 Unauthorized actually means "you are not authenticated," missing, invalid, or expired credentials. If the user is logged in with a valid token, they are authenticated.
- Returning 401 for an authorization failure is a common mislabel that confuses client error handling, since the client might think their session expired and try to re-log in, when the real problem is a permissions check.

**403 Forbidden: the technically correct answer**

- Says "I know who you are, and you're not allowed to do this."
- The honest response for "this order belongs to someone else."

**Why some APIs return 404 instead of 403**

- Returning 403 confirms the resource exists, just that you can't see it, which can leak information: an attacker probing order IDs learns which ones are real even without seeing their contents.
- Returning 404 for both "doesn't exist" and "exists but not yours" avoids that leak, at the cost of being technically less precise about what actually went wrong.

**Best Choice**

403 for internal tools and lower-risk resources where information disclosure isn't a real concern. 404 for anything where confirming existence itself is sensitive, like other users' financial records or private messages.

**Interview Answer:**

"401 is wrong here because the user is authenticated, the issue is authorization, so the technically correct code is 403. That said, I'd think about whether returning 403 leaks anything, since it confirms the resource exists even if you can't access it. For sensitive resources, like someone else's financial data, I'd deliberately return 404 for both cases, not found and not yours, to avoid letting an attacker enumerate valid IDs, even though that's a bit less precise than a textbook-correct 403."

---

## Q19. [Mid] A client gets rate limited. Should the API return 429 or 503, and what header should always come with that response?

**Answer:**

**429 and 503 mean fundamentally different things: 429 says "you personally are sending too many requests," while 503 says "the service itself is temporarily unavailable to everyone," and mixing them up gives the client the wrong signal about what to do next.**

**429 Too Many Requests**

- The correct code for rate limiting a specific client. It tells that one client to back off, while other clients might be completely unaffected.
- Should always be paired with a `Retry-After` header, in seconds or a timestamp, telling the client exactly when it's safe to try again instead of guessing.

**503 Service Unavailable**

- Signals the whole service is degraded or down for everyone, not just one client, typically due to overload, maintenance, or a dependency failure.
- Also benefits from a `Retry-After` header, but the meaning is "the system is struggling," not "you specifically are the problem."

**Why the distinction matters to a client**

- A well behaved client sees 429 and backs off just its own request rate. A client that sees 503 might reasonably assume the whole service is down and could even alert on-call, so returning 503 for routine per-client rate limiting causes false alarms and unnecessary panic.

**Interview Answer:**

"429 is for rate limiting a specific client, and it should always come with a Retry-After header so the client knows exactly when to try again instead of guessing with its own backoff. 503 means the whole service is struggling, not just one client's request rate. If I return 503 for normal per-client rate limiting, I'm giving a false signal that the whole system is down, which can trigger unnecessary alerts or client-side circuit breakers that shouldn't be tripping yet."

---

## Q20. [Mid] A POST /orders endpoint returns a 302 Found to redirect the client to the new order's URL after creation. What can actually break here, and what should you return instead?

**Answer:**

**302 leaves the HTTP method ambiguous on redirect, which means some clients replay the redirect as a GET and others might replay it as a POST, so a client that "correctly" follows a 302 after your POST can end up resubmitting the order a second time.**

**Why 302 is the trap**

- The original HTTP spec was vague about whether a client should preserve the original method (POST) or switch to GET when following a 302, and different browsers and HTTP libraries picked different defaults over the years.
- In the worst case, a client faithfully re-issues the POST against the redirect target, creating a duplicate order, exactly the kind of bug that's painful to reproduce because it depends on which HTTP client the caller happens to use.

**What to use instead**

- `201 Created` with a `Location` header pointing at the new resource is the standard, unambiguous way to say "I made this, here's where it lives," with zero redirect and zero method confusion.
- If you genuinely need a redirect after a POST, like a traditional web form submission, `303 See Other` explicitly tells every client to switch to GET, which is exactly the "post then redirect to a GET page" pattern browsers expect.
- `307 Temporary Redirect` and `308 Permanent Redirect` explicitly preserve the original method, which is the opposite behavior, useful when you genuinely want the POST replayed at a new URL.

**Interview Answer:**

"302 doesn't clearly say whether the client should keep using POST or switch to GET, and that ambiguity has caused real bugs where a client resubmits the same POST against the redirect target, creating a duplicate order. For a resource creation endpoint, I'd just return 201 Created with a Location header instead of redirecting at all. If I actually need a redirect after a POST, I'd use 303 See Other to force a GET, since that's the one status code that's unambiguous about switching methods."

---

## Q21. [Senior] What's the actual difference between 400 and 422, and is a client ever justified in automatically retrying either one?

**Answer:**

**400 means the request itself is malformed or unparseable, while 422 means the request was perfectly well formed and understood but violates a business rule, and neither one should ever be retried automatically without the client changing something first.**

**400 Bad Request**

- The server couldn't even properly understand the request: broken JSON, a missing required field, a value in the wrong data type.
- The problem lives at the syntax and structure level, before any business logic runs.

**422 Unprocessable Entity**

- The request was syntactically fine and fully understood, but fails a semantic or business rule once the server actually looks at the meaning: an end date before a start date, a discount code that's expired, an age field that's technically a valid number but violates a minimum age rule.
- Distinguishing this from 400 lets a client's error handling branch differently: a 400 usually means a client-side bug in how the request was built, a 422 usually means the client needs to show the user a specific validation message.

**Why retrying either one automatically is wrong**

- Both are 4xx client errors, meaning the server is telling you the request itself is the problem, not a transient server or network issue.
- Retrying the exact same request produces the exact same failure every time, since nothing about the situation has changed, the client needs to fix the input first.
- This is the opposite of a 429 or a 5xx, where the same request might genuinely succeed on a later attempt because the transient condition (overload, a dependency being briefly down) has passed.

**Interview Answer:**
"400 means the request is malformed or unparseable at a structural level, 422 means it's well formed but violates a business rule once the server actually evaluates it, like an end date before a start date. I use that distinction so client error handling can react differently: a 400 usually points at a bug in how the client built the request, a 422 usually means show the user a specific validation message. Neither should ever be retried automatically, since retrying the identical request against the identical bug just produces the identical failure, unlike a 429 or a 5xx where the same request might genuinely succeed once the transient condition clears."

---

## Q22. [Mid] How do you evolve an API without breaking existing clients? Give me concrete rules you'd enforce for backward compatibility.

**Answer:**

**The goal is to let the API grow while guaranteeing that clients built against an older contract keep working.**

**Safe, backward compatible changes**

- Adding a new optional field to a response.
- Adding a new endpoint.
- Adding a new optional request parameter with a sensible default.
- Relaxing a validation rule, accepting input that used to be rejected.

**Breaking changes that require a new version or careful handling**

- Removing or renaming a field.
- Changing a field's type, like turning a string ID into a numeric ID.
- Changing the meaning of an existing field.
- Making a previously optional field required.
- Changing default behavior that clients silently depend on.

**Concrete rules to enforce**

1. Never remove a field, deprecate it instead: mark it in docs, keep returning it, and give a sunset timeline.
2. Additive only for a given major version, anything else goes into a new version.
3. Use a schema or contract testing tool, like a JSON schema or OpenAPI spec checked in CI, to catch accidental breaking changes before merge.
4. Communicate deprecations through response headers, like a `Deprecation` or `Sunset` header, in addition to changelogs.
5. Keep old versions running in parallel with new ones for a defined support window rather than a hard cutover.

**Interview Answer:**
"I treat additive changes, new optional fields, new endpoints, new optional params, as safe to ship without a version bump. Anything that removes a field, changes a type, or changes existing behavior is a breaking change and either goes into a new version or gets a deprecation period with clear communication, like a Sunset header and changelog entry, before removal. I'd also back that with contract tests against the API schema in CI so a breaking change gets caught before it ships instead of being discovered by an angry partner."

---

## Q23. [Senior] Compare REST, GraphQL, and gRPC across latency, tooling, and caching. Which would you choose for an internal microservices mesh versus a public partner API?

**Answer:**

**Each protocol optimizes for a different point in the design space, and the right choice depends heavily on who the consumer is.**

**REST**

- Latency: fine for typical CRUD, but multiple related resources often mean multiple round trips.
- Tooling: enormous ecosystem, works from a browser natively, testable with curl, universally understood.
- Caching: excellent, HTTP caching semantics like `ETag` and `Cache-Control` work out of the box at the CDN, proxy, and browser level.

**GraphQL**

- Latency: strong for client driven, composed views since one query replaces several REST round trips, but a single complex query can itself be slow if resolvers aren't optimized.
- Tooling: strong client tooling, like generated types and query validation, but heavier server side setup, schema stitching, resolver design, cost limiting.

- Caching: weak at the HTTP layer since responses vary per query shape, usually needs application level caching, like normalized client side caches or persisted queries.

**gRPC**

- Latency: lowest of the three for service to service calls, thanks to binary Protocol Buffers and HTTP/2 multiplexing.
- Tooling: excellent for polyglot internal teams via generated stubs from a shared `.proto`, poor for browsers without a gRPC-Web proxy layer.
- Caching: essentially none built in, RPC style calls aren't cache friendly by nature.

**Best Choice**

gRPC for the internal microservices mesh. REST, possibly with a GraphQL layer for aggregation, for the public partner API.

**Why?**

Inside the mesh, you control both ends of every call, so gRPC's performance and strong typing outweigh its lack of caching and browser support. For the public partner API, partners need something universally accessible, debuggable, and cacheable without needing your `.proto` files or specialized client libraries, which is exactly what REST provides. If partners frequently need to compose data across many resources, a GraphQL layer on top of the REST/gRPC backend is a reasonable middle ground.

**Interview Answer:**
"Inside the mesh I'd use gRPC, since I control both sides, and the binary protocol plus HTTP/2 multiplexing gives the best latency and throughput for service to service traffic, with generated stubs keeping contracts consistent across languages. For the public partner API, I'd use REST, because partners need something they can hit with curl, cache at the HTTP layer, and understand without needing my proto definitions. If partners need to compose a lot of related data in custom ways, I'd consider adding a GraphQL layer in front of those REST or gRPC services rather than replacing REST outright."

---

## Q24. [Senior] A product table has 500 million rows and users keep paging deeper. Offset pagination is timing out past page 10,000. Walk me through keyset pagination and how you'd implement it.

**Answer:**

This is the classic case where offset pagination's cost profile becomes untenable, and keyset (cursor) pagination is the fix.

**Why offset fails here**

`OFFSET 200000 LIMIT 20` forces the database to scan and discard 200,000 rows before returning the next 20, even with an index, because offset is a row count, not a seekable position. **At 500 million rows, deep pages become full table scans in practice.**

**How keyset pagination works**

1. Pick a sort key that is unique and indexed, commonly `(created_at, id)` where `id` breaks ties for rows with identical timestamps.
2. First page: `SELECT * FROM products ORDER BY created_at DESC, id DESC LIMIT 20`.
3. Client remembers the `(created_at, id)` of the last row returned.

4. Next page: `SELECT * FROM products WHERE (created_at, id) < (:last_created_at, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20`.
5. Because there's a composite index on `(created_at, id)`, the database seeks directly to that position using the index instead of scanning from the start, so every page costs roughly the same regardless of depth.

**Implementation details**

- Encode the cursor as an opaque base64 token, don't expose raw column values, so you can change the underlying sort key later without breaking clients.
- Handle ties correctly, the tuple comparison `(created_at, id) < (...)` is what makes this correct when multiple rows share a timestamp, a naive `created_at < :last_created_at` alone would skip rows with the exact same timestamp.
- Give up the ability to jump to an arbitrary page number, this is the fundamental trade off, and communicate that clearly, since some UIs, like "jump to page 50," aren't compatible with pure keyset pagination.
- If a "total count" is still needed for UI purposes, compute it separately and cache it, don't compute it on every page request from a 500 million row table.

**Interview Answer:**

"The problem is that OFFSET forces the database to scan and discard everything before the offset, so page 10,000 means scanning 200,000 rows just to throw them away. I'd switch to keyset pagination: order by an indexed, unique composite key like created_at plus id, and instead of an offset, the client sends the last row it saw, and the query becomes a WHERE clause like created_at, id less than the last seen values, with the same limit. With a matching composite index, the database can seek directly to that position, so page 10,000 costs the same as page 1. The trade off is you lose the ability to jump to an arbitrary page number, which I'd flag to the product team."

---

## Q25. [Senior] Design idempotency for a distributed payments API where requests can retry across multiple app server instances behind a load balancer.

**Answer:**

**The distributed part is what makes this harder than a single server idempotency check, since multiple instances might process retries concurrently, and no instance has an in memory view of what another instance is doing.**

**Design**

1. **Shared idempotency store**: use a database table or a distributed store like Redis, not in memory state on an app server, since the retry might land on a different instance behind the load balancer.
2. **Atomic reservation**: on receiving a request with an `Idempotency-Key`, attempt an atomic insert of that key with a status of "processing" and a request payload hash. Use a unique constraint on the key column so only one instance can win this insert if two requests race in at the same time.
3. **Handle the three outcomes of that insert**:
   - Insert succeeds: this instance owns processing the payment. Proceed, then update the row with the final result, status "completed" plus the response body.
   - Insert fails because the key already exists and status is "completed": return the stored response immediately, no charge happens again.

- Insert fails because the key already exists and status is "processing": another instance is actively handling it right now. Return a 409, or poll briefly with backoff for the result to appear, depending on the client's tolerance for waiting.

1. **Payload consistency check**: store a hash of the request body with the key, and reject reuse of the same key with a different payload as a client error, since that indicates a bug in the caller.
2. **TTL and cleanup**: expire idempotency records after a window like 24 to 72 hours, since clients don't retry indefinitely, and this keeps the store from growing unbounded.
3. **Downstream idempotency too**: if the payment call goes out to a third party processor, pass the same idempotency key through to them if they support it, so a retry after a timeout to the processor also doesn't double charge upstream.

**Edge case worth raising**

If the app server crashes after charging the card but before writing "completed" to the idempotency store, a retry would see "processing" forever, or after a stale timeout, might re-attempt the charge. Mitigate by having the payment processor call also be idempotent using the same key, so even a retried charge attempt against the processor is safe, and by using a reasonably short "processing" timeout after which a retry with the same key can be reattempted rather than blocked indefinitely.

**Interview Answer:**

"Because the retry can hit a different instance, idempotency state has to live in a shared store, not in memory. I'd do an atomic insert of the idempotency key with a unique constraint, so only one instance wins the race and proceeds to charge, while the loser either returns the stored result if it's done, or a 409 or a short poll if it's still processing. I'd store a hash of the payload to catch key reuse with different data, expire records after a day or two, and importantly, pass the same idempotency key down to the payment processor itself, so that even a retried outbound charge call is protected, not just the inbound API call."

---

## Q26. [Senior] Compare token bucket, leaky bucket, fixed window, and sliding window rate limiting algorithms. Which would you implement for an API gateway serving thousands of tenants?

**Answer:**

**Each algorithm makes a different trade off between burst tolerance, smoothness, and implementation cost.**

**Fixed window**

- Counts requests in a fixed time block, like "100 requests per minute, reset at the top of each minute."
- Pros: trivial to implement, cheap to compute.
- Cons: allows a burst of up to double the limit right at the window boundary, 100 requests at 0:59 and another 100 at 1:00 is 200 requests in two seconds.

**Sliding window**

- Tracks requests over a rolling time frame rather than a fixed reset point, either by keeping a log of timestamps (sliding log) or by weighting the previous and current fixed windows (sliding window counter).
- Pros: smooths out the boundary burst problem of fixed windows.

- Cons: sliding log is memory heavy at scale, sliding window counter is an approximation, slightly more complex to implement than fixed window.

**Token bucket**

- A bucket holds tokens, refilled at a steady rate up to a max capacity. Each request consumes a token, and requests are rejected when the bucket is empty.
- Pros: naturally allows short bursts up to the bucket size while still enforcing an average rate over time, matches how real traffic behaves, occasional bursts from a client are fine as long as the average rate holds.
- Cons: slightly more state to track per client than fixed window, needs careful tuning of bucket size versus refill rate.

**Leaky bucket**

- Requests enter a queue, processed, "leak out," at a constant rate, smoothing bursts into a steady output rate.
- Pros: enforces a very steady outbound rate, good for protecting a downstream system that can't handle bursts at all.
- Cons: bursty clients experience added latency or drops rather than being smoothly absorbed, less generous to legitimate bursty traffic than token bucket.

**Best Choice**

Token bucket per tenant, implemented in a shared store like Redis using its atomic increment and expiry primitives, for the API gateway.

**Why?**

An API gateway serving thousands of tenants needs to tolerate normal bursty client behavior, a tenant firing off a batch of requests briefly is not necessarily abuse, while still enforcing a fair average rate per tenant over time and protecting the gateway from sustained overload. Token bucket is the standard choice precisely because it balances burst tolerance with a hard long term average, and it's cheap to implement per tenant key in Redis with an atomic script to keep the check and decrement race free across gateway instances.

**Interview Answer:**

"Fixed window is simplest but allows a boundary burst of up to twice the limit. Sliding window fixes that but costs more to compute or approximate. Leaky bucket enforces a very steady, smoothed output rate, which is great for protecting a fragile downstream system but punishes legitimate bursty clients. Token bucket allows short bursts up to the bucket size while still holding to an average rate over time, which matches real traffic patterns best. For a multi-tenant gateway I'd implement token bucket per tenant in Redis, using an atomic Lua script so the check and decrement stay race free across gateway instances."

---

## Q27. [Senior] Design a webhook delivery system that guarantees at least once delivery to thousands of subscriber endpoints. Cover ordering, deduplication, retries, and security.

**Answer:**

**At scale, webhook delivery is really a distributed messaging problem wearing an HTTP costume.**

**Architecture**

1. Internal event occurs, published to an internal event stream or queue.
2. A fan out worker looks up all subscribers for that event type and enqueues one delivery task per subscriber.
3. A pool of delivery workers pulls tasks and makes the HTTP POST to each subscriber's URL.

**At least once delivery and retries**

- Only mark a delivery as complete when the subscriber's endpoint responds with a 2xx status within a timeout.
- Anything else, timeout, 5xx, connection refused, gets retried with exponential backoff and jitter, for example 30s, 2m, 10m, 1h, 6h, spread across 24 to 72 hours, then moved to a dead letter queue with alerting and a manual replay option.
- At least once inherently means duplicates are possible, if the subscriber processed the event but their acknowledgment was lost in transit, a retry fires again.

**Deduplication**

- Every event gets a unique, stable event ID, included in the payload.
- Document clearly that subscriber endpoints must be idempotent on event ID, store processed IDs on their side and ignore repeats. This is the standard and correct place to solve deduplication, since only the receiver truly knows whether it already acted on a given event.

**Ordering**

- Strict global ordering across all events for a subscriber is expensive to guarantee and often unnecessary.
- A pragmatic middle ground: guarantee ordering only within a partition key, like per resource ID, for example all events for order 123 are delivered in order relative to each other, by routing them through the same queue partition and using a single worker per partition key, while events for different orders can be delivered out of order relative to each other.
- Include a sequence number or timestamp in the payload so subscribers can detect and handle any out of order delivery themselves as a final safety net.

**Security**

- Sign every payload with HMAC using a secret unique to each subscriber, sent in a header like `X-Signature`, so subscribers can verify the request genuinely came from you and wasn't tampered with.
- Rotate secrets on demand and support a grace period with two active secrets during rotation.
- Reject and don't retry delivery to non-HTTPS URLs.

**Interview Answer:**

"I'd fan events out to a per subscriber delivery queue, with workers that only mark delivery complete on a 2xx response, retrying everything else with exponential backoff and jitter before landing in a dead letter queue with manual replay. Since at least once delivery means duplicates can happen, every event carries a unique ID, and I document that subscribers must dedupe on that ID themselves, since only they know if they already processed it. For ordering, I'd guarantee it only within a partition key, like per resource, routing same key events through a single worker, rather than paying for expensive global ordering. Every payload gets HMAC signed with a per subscriber secret so receivers can verify authenticity."

## Q28. [Senior] Walk me through deprecating and sunsetting v1 of a public API used by hundreds of external partners, without breaking them.

**Answer:**

**This is a change management problem as much as a technical one, since you don't control when, or if, partners upgrade.**

**Steps**

1. **Ship v2 alongside v1**: never turn off v1 the moment v2 exists. Run both in parallel so partners can migrate on their own schedule.
2. **Announce with a concrete timeline**: publish a deprecation notice with a specific sunset date, typically 6 to 12 months out for a public API with many partners, giving enough runway for partners' own release cycles.
3. **Signal deprecation in every response**: add a `Deprecation` header and a `Sunset` header, per the relevant HTTP standards draft, to every v1 response, so automated tooling on the partner side, and not just humans reading a changelog, can detect it.
4. **Instrument usage**: track which partners are still calling v1, by API key or client ID, so you know who to actively reach out to as the deadline approaches, rather than guessing.
5. **Directly contact remaining laggards**: as the sunset date nears, email or account manager outreach to the specific partners still on v1, since a subset will always miss a changelog announcement.
6. **Provide a migration guide**: a clear field by field, endpoint by endpoint mapping from v1 to v2, plus code samples, reduces the friction that causes partners to delay migrating.
7. **Consider a soft cutover before hard shutdown**: for a period, return a 410 Gone with a clear message pointing to v2 docs, or intentionally degrade v1, like lower rate limits, before fully removing it, so the failure mode is loud and informative rather than a silent outage on the sunset date.
8. **Hard shutdown**: only after the above steps and after usage has dropped near zero or the deadline has firmly passed, turn off v1 entirely.

**Interview Answer:**
"I'd never cut v1 off the moment v2 ships, I'd run them in parallel and publish a firm sunset date with enough runway, six to twelve months for a public API with hundreds of partners. Every v1 response would carry Deprecation and Sunset headers so tooling can catch it automatically, not just humans reading a changelog. I'd track usage per partner so I know exactly who's still on v1 as the deadline nears and reach out directly to the stragglers, since some will always miss the announcement. Right before the hard shutdown, I'd rather return a clear 410 with a pointer to v2 than have partners hit a silent, confusing failure."

---

## Q29. [Senior] Design an API to let a customer export 50 million rows of their data as CSV. Walk me through pagination, rate limits, and failure handling.

**Answer:**

**A synchronous request response API is the wrong shape for this entirely, 50 million rows will never fit comfortably in a single HTTP request/response cycle, so the design needs to be asynchronous.**

**High level design**

1. Client calls `POST /exports` with filters describing what to export. The server responds immediately with `202 Accepted` and an export job ID, it does not try to stream 50 million rows back synchronously.

2. A background worker picks up the job, queries the data in bounded batches using keyset pagination internally, ordering by an indexed key, to avoid a single massive query or one giant offset scan.
3. The worker streams rows into a CSV file in chunks, writing to object storage, like S3, as it goes, rather than building the whole file in memory.
4. The client polls `GET /exports/{id}` for status, or better, gets notified via a webhook when the export is ready, which avoids polling overhead entirely.
5. Once complete, the response includes a time limited, signed download URL pointing directly at the file in object storage, so the download itself doesn't route back through the application servers.

**Handling scale within the job**

- Process in fixed size batches, for example 50,000 rows at a time, using a keyset cursor on an indexed column, committing progress after each batch so a crash mid job can resume from the last completed batch instead of restarting from zero.
- If the export needs to reflect a consistent snapshot, capture a fixed point in time, like a max transaction ID or timestamp, at job start, and filter all batches against that boundary, so new writes during the multi hour export don't cause inconsistent or duplicate rows.

**Rate limits**

- Exports are heavy, resource intensive jobs, so rate limit the number of concurrent export jobs per customer, for example one active export at a time, rather than a per request rate limit, which doesn't map well to a job that might take hours.
- Separately, general read API rate limits should stay unaffected by background export jobs, since they run on a different worker pool from the main request serving path, isolating them so an export doesn't degrade the customer's regular API traffic.

**Failure handling**

- Persist progress per batch so a worker crash resumes rather than restarts.
- Set a retry policy per batch with backoff for transient failures, like a database timeout, distinct from marking the whole job failed.
- After repeated batch failures or a stuck job, mark the job failed with a clear reason, and notify the customer, rather than leaving it silently stuck.
- Expire and clean up the generated file and signed URL after a reasonable window, like 7 days, to avoid unbounded storage growth from abandoned exports.

**Interview Answer:**

"This has to be asynchronous, not a single request response cycle. The client kicks off a job and gets a job ID back immediately, a background worker streams the data out in batches using keyset pagination against an indexed column, writing directly to object storage rather than building the file in memory, and commits progress per batch so a crash can resume instead of restarting. I'd snapshot a fixed point in time at job start so a multi hour export stays consistent despite ongoing writes. For rate limiting, I'd cap concurrent export jobs per customer rather than applying a request based limit, and run exports on a separate worker pool so they don't compete with normal API traffic. Once done, the customer gets a signed, time limited download URL."

---

## Q30. [Senior] GraphQL is causing N+1 query storms against the database in production. Walk me through diagnosing and fixing it.

**Answer:**

**The N+1 problem in GraphQL comes from how resolvers execute independently for each item in a list.**

**Why it happens**

A query like "give me 50 posts and each post's author" resolves in two levels: one query fetches the 50 posts, then, naively, the resolver for "author" runs once per post, firing 50 separate `SELECT * FROM users WHERE id = ?` queries, one query for the list plus N queries for the related field, hence N+1.

**Diagnosing it**

1. Turn on database query logging or APM tracing scoped to a single GraphQL request, and look for repeated near identical queries differing only by an ID parameter.
2. Use GraphQL specific tracing, like Apollo's tracing extension, which reports per resolver timing and call counts, making a resolver that fires 50 times for one request obvious.
3. Correlate spikes in database connection usage or query volume with specific GraphQL operation names in your monitoring, if the API is only exposed through GraphQL, this narrows it down fast.

**Fixing it**

- **Batching with a data loader pattern**: introduce a per request batching and caching layer, like the DataLoader library, that collects all the individual `author` lookups requested during a single tick of the event loop and issues one batched query, `SELECT * FROM users WHERE id IN (...)`, instead of 50 separate ones.
- **Per request caching**: the same data loader also caches results within the lifetime of a single request, so if two different fields in the same query both need user 42, it's fetched once.
- **Schema and query design**: for extremely hot paths, consider denormalizing or precomputing frequently joined data so the resolver doesn't need a separate lookup at all.
- **Query complexity limiting**: independently, add depth and complexity limits to the schema so a client can't request pathologically nested data that would cause N+1 style storms even after batching is in place, since batching helps per level but deeply nested queries still multiply work across levels.

**Interview Answer:**

"N+1 happens because a resolver for a nested field, like a post's author, runs once per item in the parent list, so fetching 50 posts triggers 50 separate author lookups instead of one. I'd confirm it by tracing a single request, either through GraphQL resolver tracing or by watching for repeated near identical queries in the database logs. The standard fix is a data loader: a per request batching layer that collects all the individual lookups queued during one tick and issues a single IN query instead, while also caching within that request so duplicate lookups for the same ID are free. I'd pair that with query depth and complexity limits so deeply nested queries can't cause a similar storm even with batching in place."

---

## Q31. [Senior] Compare gRPC streaming and REST or webhooks for a real time stock ticker feed to thousands of clients. Which would you pick and why?

**Answer:**

**The core requirement, continuous, low latency updates to many clients, rules out a plain request response model and points toward a persistent connection.**

**REST, request response**

- Fundamentally the wrong shape, REST is a pull model, the client asks and the server answers once. A stock ticker needs continuous pushes, so this would require constant polling, which adds latency

proportional to the poll interval and wastes huge amounts of traffic on empty responses when prices haven't changed.

**Webhooks**

- Wrong direction for this use case, webhooks push server to client, which fits per event notifications like "order shipped," but they're not built for a continuous, high frequency stream, thousands of clients each maintaining a webhook receiver for tick by tick updates is an awkward, heavyweight fit for what's really a subscription to a live feed.

**gRPC server streaming**

- The client opens a single request, and the server keeps the stream open, pushing new price updates as they occur, over one long lived HTTP/2 connection.
- Efficient: one persistent connection per client instead of repeated request overhead, binary encoding keeps each tick's payload small, and HTTP/2 multiplexing means a client can subscribe to many symbols without opening many connections.
- Natural fit for the access pattern: "keep sending me updates for these symbols until I disconnect."

**WebSockets, worth mentioning as the browser facing alternative**

- If the consumer is a browser, gRPC needs a gRPC-Web proxy layer and doesn't map as cleanly, so a WebSocket based pub/sub feed is often the more practical choice for browser clients specifically, while gRPC streaming fits better for backend to backend or native app consumers.

**Best Choice**

gRPC server streaming for backend and native app consumers, WebSockets for browser consumers, both instead of REST polling or webhooks.

**Why?**

The requirement is a continuous push feed to a large number of concurrently connected clients, and both gRPC streaming and WebSockets are built exactly for that persistent, low overhead, server initiated push pattern, whereas REST polling wastes resources and adds latency, and webhooks are designed for discrete events, not a continuous tick stream.

**Interview Answer:**

"Neither REST nor webhooks fit well here. REST is pull based, so you'd be polling constantly and either wasting requests when nothing's changed or adding latency between polls. Webhooks push server to client, but they're built for discrete events, not a continuous tick by tick stream. I'd use gRPC server streaming for backend or native app consumers, since the client opens one long lived connection and the server pushes updates as they happen with compact binary encoding, and use WebSockets for browser clients specifically, since gRPC needs an extra proxy layer to work in a browser."

---

## Q32. [Senior] Design a rate limiting layer for an API gateway that needs to enforce different limits per tenant, per endpoint, and a global limit, while running across multiple gateway instances.

**Answer:**

**This needs a shared, low latency counting mechanism, since state living on any single gateway instance won't see traffic hitting the other instances.**

**Requirements breakdown**

- **Per tenant limit**: overall requests per tenant across all endpoints, for example 10,000 requests per minute for a paid tier tenant.
- **Per endpoint limit**: some endpoints are more expensive than others, for example a search endpoint might be capped tighter than a simple read, independent of which tenant is calling it.
- **Global limit**: a system wide ceiling to protect the backend regardless of how traffic is distributed across tenants, guarding against the case where many tenants under their individual limits still collectively overwhelm a downstream dependency.

**Architecture**

1. **Shared counting store**: use Redis, since gateway instances are stateless with respect to rate limiting and need a common source of truth. Redis's single threaded command execution and support for atomic Lua scripts make check and increment operations race free even under concurrent access from many gateway instances.
2. **Key design**: maintain separate counters per scope, for example `ratelimit:tenant:{tenant_id}`, `ratelimit:endpoint:{endpoint}`, `ratelimit:global`, each implemented as a token bucket with its own capacity and refill rate.
3. **Request flow**: on each incoming request, the gateway checks all applicable buckets, tenant, endpoint, and global, in a single atomic Lua script executed in Redis, decrementing tokens only if all three checks pass, and rejecting with 429 if any one of them fails, including which limit was hit in the response so clients understand why.
4. **Local caching for the common case**: to avoid a Redis round trip on every single request at very high volume, some gateways use an approximate local counter that syncs periodically with Redis, trading a small amount of precision for lower latency, this is a reasonable mention as an optimization for extreme scale, with an explicit note that it makes limits slightly soft rather than exact.
5. **Response headers**: return `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `Retry-After` so clients can back off intelligently instead of hammering the endpoint after a rejection.

**Failure mode to address**

If Redis itself becomes unavailable, decide explicitly whether to fail open, allow all requests through, risking overload, or fail closed, reject all requests, risking a full outage for a rate limiter dependency issue. For most systems, failing open for a short window with aggressive alerting is preferable to a full self inflicted outage, since a brief period of unenforced limits is usually less damaging than blocking all traffic.

**Interview Answer:**

"I'd centralize counting in Redis since gateway instances are stateless and need one shared source of truth, using token buckets per scope, tenant, endpoint, and a global bucket, each checked and decremented atomically in a single Lua script so concurrent requests across gateway instances don't race. A request only succeeds if it passes all three applicable checks. I'd return rate limit headers so clients can back off intelligently, and for very high traffic, consider approximate local counters that sync periodically with Redis to cut round trips, accepting slightly soft limits in exchange for latency. I'd also explicitly decide the Redis outage behavior ahead of time, and lean toward failing open briefly with strong alerting rather than taking down the whole API over a rate limiter dependency."

---

**If you remember nothing else from this chapter:** *on the backend you control the deploy; on mobile, the deploy schedule belongs to app stores and users who never update, design accordingly.*

# Chapter 12: Distributed Systems Concepts: CAP Theorem, Consistency & Consensus

This is usually where a system design interview separates senior candidates from everyone else. Being able to recite "CAP theorem" isn't enough, you need to apply it to a specific trade-off, like a bank balance versus a like counter, to show you've actually internalized why distributed systems are hard.

**War Story:** *A "just add leader election" fix for split-brain made things worse, because the quorum size chosen could still produce two leaders under one specific partition pattern nobody had tested for.*

**Avoid:** treating "eventual consistency" as a magic phrase rather than something with a specific, describable staleness window

**Aim For:** being able to describe what actually happens to one specific request during one specific network partition, not just citing CAP theorem by name

## Q1. [Junior] Can you explain the CAP theorem in your own words, with a simple example?

**Answer:**

**CAP theorem says that a distributed data store can only guarantee two out of three properties at the same time: Consistency, Availability, and Partition tolerance.**

### The three properties

- Consistency: every read gets the most recent write, or an error. All nodes see the same data at the same time.
- Availability: every request gets a response, even if it isn't the latest data.
- Partition tolerance: the system keeps working even when network messages between nodes are lost or delayed.

### Why you only get two

Networks fail, so partition tolerance isn't really optional for any real distributed system, it's going to happen. That means the actual choice in practice is between consistency and availability when a partition occurs. If two nodes can't talk to each other, you either stop serving requests until they reconcile (favoring consistency) or let both keep serving and risk them disagreeing (favoring availability).

### A simple example

Imagine two data centers holding a copy of a user's shopping cart, and the network link between them drops. If you want consistency, one or both data centers stop accepting cart updates until the link is restored. If you want availability, both keep accepting updates, and you might end up with two different versions of the cart to merge later.

### Interview Answer:

"CAP theorem says that when a network partition happens, a distributed system has to choose between consistency and availability, it can't have both. Consistency means every node sees the same, most up to date data. Availability means every request still gets a response. Partition tolerance is basically mandatory

because networks fail. So in practice, CAP is about what you do when nodes can't talk to each other: block and stay consistent, or keep serving and risk disagreement."

---

## Q2. [Junior] What does "eventual consistency" mean, and can you give a real world example of it?

**Answer:**

Eventual consistency means that if no new updates are made to a piece of data, all replicas will eventually converge to the same value, but there's no guarantee about how long "eventually" takes.

**Key characteristics**

- Reads right after a write might return stale data.
- There's no lock or coordination step slowing writes down.
- The system prioritizes availability and low latency over immediate correctness across replicas.

**Real world example**

DNS is a classic example. When you update a DNS record, it doesn't propagate to every resolver on the planet instantly. Some users will see the new IP address right away, others might see the old one for minutes or hours until caches expire. Eventually, every resolver converges on the correct value, but there's a window of inconsistency.

Another common example is a social media "like" count. If you like a post, your friend on the other side of the world might see the old count for a second or two before it updates. **Nobody is harmed by that delay, which is exactly why eventual consistency is an acceptable trade there.**

**Interview Answer:**
"Eventual consistency means replicas don't have to agree instantly, they just have to converge over time if writes stop. DNS is a good example: when you change a record, some resolvers see it immediately and others see the stale value until their cache expires, but eventually everyone agrees. It's the right model when a short window of staleness is harmless, and in exchange you get lower latency and higher availability than waiting for every replica to confirm a write."

---

## Q3. [Junior] Say your service has three database replicas across two regions, and the network link between the regions drops for two minutes. Walk me through what happens if the system is designed to favor consistency versus if it's designed to favor availability.

**Answer:**

**This is the CAP trade-off playing out in a concrete scenario.**

**If the system favors consistency (CP)**

1. The two regions can no longer confirm they agree with each other.
2. To avoid serving stale or conflicting data, the side that can't reach a majority of replicas (or the designated leader) stops accepting writes, and possibly reads too.
3. Users on the minority side see errors or timeouts for those two minutes.
4. Once the link is restored, the regions resync and resume normal operation with no data conflicts to resolve.

**If the system favors availability (AP)**

1. Both regions keep accepting reads and writes independently during the outage.
2. Each region's replica set drifts, since it doesn't know what the other side is doing.
3. Users get fast responses the whole time, but some might read different values in different regions.
4. Once the link is restored, the system has to reconcile any conflicting writes, using something like last-write-wins, version vectors, or application level merge logic.

**Interview Answer:**

"With a consistency first design, the minority side of the partition stops serving writes, maybe reads too, until it can confirm it's in sync, so users there see errors for those two minutes but never see wrong data. With an availability first design, both sides keep serving traffic independently, so users always get a response, but the two regions can drift apart and you need a reconciliation strategy, like version vectors or last-write-wins, once the partition heals."

---

## Q4. [Junior] What's the difference between availability and partition tolerance? People sometimes mix these up.

**Answer:**

**They sound similar but describe different failure conditions.**

**Availability**

- Means the system responds to every request it receives, successfully or with a graceful degraded response, without unbounded delay.
- Is about the system staying responsive under normal or degraded conditions.

**Partition tolerance**

- Means the system continues operating even when the network between its nodes is broken or dropping messages.
- Is about surviving a specific kind of failure: nodes being unable to talk to each other, even though each individual node is healthy.

**How they relate**

A system could be available but not partition tolerant, imagine a single node system that never has a network to partition, it's always available by default. In a real distributed system, once you accept that partitions will happen, availability becomes a question of what you do during that partition: keep answering requests (available) or refuse to answer until you're sure you're consistent (not available during the partition).

**Interview Answer:**

"Availability is about the system always giving you a response. Partition tolerance is about the system surviving a broken network link between its own nodes. They're related because once a partition happens, you have to decide whether to stay available and risk inconsistency, or sacrifice availability to protect consistency. A single node system can be available without ever worrying about partition tolerance, since there's nothing for it to partition from."

---

## Q5. [Junior] What's a leader node in a distributed database, and why do most systems need one?

**Answer:**

**A leader, also called a primary or master, is the single node in a replica set that's responsible for accepting writes and coordinating how those writes propagate to the other nodes, called followers or replicas.**

**Why systems use a leader**

- Ordering: someone has to decide the order in which writes happen, otherwise two conflicting writes could be applied in different orders on different nodes.
- Simplicity: clients always know where to send writes, they don't have to coordinate with each other.
- Conflict avoidance: if only one node accepts writes, you avoid the problem of two nodes accepting conflicting writes at the same time.

**What followers do**

Followers receive a stream of changes from the leader and apply them locally, so they can serve reads, often called read replicas, and are ready to take over if the leader fails.

**The trade-off**

A single leader is a single point of coordination, and if it goes down, the system needs a leader election process to pick a new one, which takes some time and can cause a brief write unavailability window.

**Interview Answer:**
"A leader is the one node that accepts writes and decides the order they happen in, then streams those changes to followers. It exists because someone needs to be the single source of truth for write ordering, otherwise you'd get conflicting updates applied in different orders on different replicas. The downside is the leader becomes a coordination bottleneck and a single point of failure, which is why systems need a leader election mechanism for when it goes down."

---

## Q6. [Junior] What's a network partition, and can you describe a real scenario where one might happen in production?

**Answer:**

A network partition is when nodes in a distributed system that normally can communicate lose the ability to reach each other, while each node individually is still running fine.

**Common causes in production**

- A switch or router fails between two racks in the same data center.
- A fiber cut or BGP misconfiguration cuts connectivity between two cloud regions.
- A firewall rule change accidentally blocks traffic between services.
- Overloaded network links cause packets to drop or time out, which looks like a partition even if the link isn't fully down.

**Why it's tricky**

From a single node's point of view, it can't tell the difference between "the other node crashed" and "the other node is fine but I can't reach it." Both look like a timeout. **This ambiguity is exactly what makes distributed systems hard: you have to design for the case where you genuinely don't know if the other side is alive.**

**A concrete example**

Say your app servers in us-east and us-west both talk to a database cluster, and a transient BGP issue cuts cross region traffic for 90 seconds. Nodes in us-east can't reach nodes in us-west, but both sides are still up and serving their own local traffic. That's a partition.

**Interview Answer:**
"A network partition is when otherwise healthy nodes can't communicate with each other, because of a switch failure, a fiber cut, a bad firewall rule, or just an overloaded link dropping packets. The hard part is that from one node's perspective, an unreachable peer looks identical to a crashed peer, you get a timeout either way. That ambiguity is why distributed systems have to assume partitions will happen and decide in advance how to behave when they do."

---

## Q7. [Mid] Can you explain PACELC and how it goes further than CAP?

**Answer:**

**PACELC extends CAP by pointing out that the consistency versus availability trade-off isn't only about what happens during a partition, it also applies during completely normal operation.**

**Breaking down the acronym**

- P: if there's a Partition,
- A: choose Availability
- C: or Consistency,
- E: Else, meaning when the system is running normally with no partition,
- L: choose Latency
- C: or Consistency.

**Why this matters**

CAP only describes behavior during a network partition, which is relatively rare. But even when everything is healthy, a system with synchronous replication has to wait for acknowledgments from other nodes before confirming a write, which adds latency, in exchange for stronger consistency. A system that returns success after writing to just the local node has lower latency, but a follower read might be stale.

**Practical examples**

- DynamoDB and Cassandra are PA/EL systems: available during partitions, and low latency over strict consistency during normal operation.
- Traditional relational databases with synchronous replication, or systems like Spanner, lean toward PC/EC: consistency both during partitions and in normal operation, accepting higher latency.

**Interview Answer:**
"PACELC says CAP doesn't tell the whole story, because it only describes partition scenarios. PACELC adds that even without a partition, you're trading latency against consistency, since synchronous replication for strong consistency costs you round trip time. So a system like Cassandra chooses availability during a partition and low latency during normal operation, PA/EL, while something like Spanner chooses consistency in both cases, PC/EC, at the cost of higher latency."

---

## Q8. [Mid] Compare strong consistency and eventual consistency for a bank account balance versus a social media like counter. Which would you use for each, and why?

**Answer:**

**The right consistency model depends entirely on how much the application cares about correctness at the exact moment of read versus how much it cares about speed and availability.**

**Options**

**Strong consistency**

- Every read reflects the most recent committed write, guaranteed, usually by routing reads through a leader or using quorum reads.
- Pros: no risk of acting on stale data, simpler reasoning for the application.
- Cons: higher latency, lower availability during failures, and reduced throughput since coordination is required.

**Eventual consistency**

- Reads might return slightly stale data, but replicas converge over time.
- Pros: low latency, high availability, scales well.
- Cons: application has to tolerate or reconcile temporary disagreement between replicas.

**Best Choice**

Bank account balance: strong consistency. Like counter: eventual consistency.

**Why?**

A bank balance directly controls decisions with real financial consequences, if a user sees a stale balance and overdraws, or if two concurrent withdrawals aren't serialized correctly, that's real money lost and a compliance problem. The cost of coordination is worth it. A like counter, on the other hand, has zero real world consequence if it's off by a few for a couple of seconds. Forcing every like to go through a strongly consistent write path would tank throughput on a high traffic feature for no meaningful user benefit.

**Interview Answer:**
"I'd use strong consistency for the bank balance, because incorrect reads there can cause real financial harm, like a double spend or an incorrect overdraft decision, so it's worth paying the latency and availability cost of coordinating writes. For a like counter, I'd use eventual consistency, since being off by a few for a second or two has no real consequence, and I'd rather have that feature be fast and highly available than slow and perfectly accurate."

---

## Q9. [Mid] I want you to design the data layer for a chat app's "online presence" feature, showing whether a user is online, offline, or away. Would you make this system AP or CP, and why?

**Answer:**

**I'd design presence as an AP system, favoring availability over strict consistency.**

**Why availability wins here**

- Presence is inherently a soft, real time signal, it's already slightly stale the moment it's displayed, since a user could go offline the instant after their status was fetched.
- Users expect presence to be "mostly right," not perfectly synchronized. Nobody is harmed if a friend's status is a second or two out of date.

- Blocking a presence read or write because of a network hiccup would be a much worse user experience than just showing slightly stale data.

**How I'd build it**

1. Each user's client sends periodic heartbeats to a presence service, with a short TTL, like 30 seconds.
2. Presence state is stored in an in memory, replicated store like Redis, or a wide column store with fast, eventually consistent replication.
3. If a heartbeat isn't renewed within the TTL, the user is marked offline automatically, which also solves the "did the client crash without telling us" problem.
4. Reads are served from the nearest replica, no cross region coordination needed, and clients can also use gossip or pub sub to update presence in near real time for people they're actively chatting with.

**What I'd explicitly avoid**

I would not put presence behind a strongly consistent quorum write or a single leader for the whole system, since that adds latency and creates unnecessary unavailability risk for a feature where the data is short lived and low stakes anyway.

**Interview Answer:**
"I'd make presence AP. Presence is already a fuzzy, real time signal, a status is stale the instant it's displayed, so there's no value in paying for strong consistency there. I'd use short lived heartbeats with a TTL stored in a fast, eventually consistent store like Redis, and auto expire anyone who stops heartbeating. Reads come from the nearest replica for low latency. The failure mode of a stale 'online' badge for a second is completely acceptable, but blocking the feature during a network blip would not be."

---

## Q10. [Mid] Explain how quorum reads and writes work, and what the W plus R greater than N rule actually guarantees.

**Answer:**

Quorum based replication is a way to get tunable consistency in a system with N replicas, without requiring every single replica to acknowledge every operation.

**The basic setup**

- N: total number of replicas holding a copy of the data.
- W: number of replicas that must acknowledge a write before it's considered successful.
- R: number of replicas that must respond to a read before the read is considered successful.

**The rule: W + R > N**

**If the write quorum and read quorum overlap by at least one node, then any read is guaranteed to include at least one replica that has the latest write.** That's because there's no way to pick W nodes for the write and R nodes for the read, out of N total, without at least one node appearing in both groups.

**Practical configurations**

- N=3, W=2, R=2: a common default, tolerates one node being down for either reads or writes, and guarantees overlap.
- N=3, W=3, R=1: writes are slow and need everyone, but reads are fast since any single replica has the data.
- N=3, W=1, R=3: writes are fast, but reads have to check all replicas and pick the newest version, since only one node might have the latest write.

**What it doesn't guarantee**

Quorum overlap guarantees you'll see the latest write among the responses, but you still need a way to identify which response is "latest," usually via version numbers or timestamps, and concurrent writes without a leader can still conflict and need resolution.

**Interview Answer:**

"With N replicas, W is how many must ack a write and R is how many must respond to a read. If W plus R is greater than N, the read set and write set are guaranteed to overlap by at least one node, so every read sees at least one copy with the latest write. It's a knob for trading off latency and availability: high W means slower, more durable writes, high R means slower, more consistent reads, and you tune N, W, and R based on whether your workload is more read heavy or write heavy."

---

## Q11. [Mid] What is causal consistency, and can you give an example of where it matters?

**Answer:**

**Causal consistency guarantees that operations which are causally related, meaning one happens because of or after another, are seen by everyone in that same order.** Operations that are unrelated, called concurrent, don't need a global order.

**Why it's a useful middle ground**

- It's stronger than plain eventual consistency, because it preserves cause and effect relationships.
- It's weaker, and therefore cheaper and more available, than strong or linearizable consistency (where every read reflects the most recent write and all operations appear to happen in one single, real-time order), because it doesn't require a single global order for everything.

**Classic example: a social media comment thread**

Say user A posts "What movie should we watch tonight?" and user B replies "Let's watch the new sci-fi one." If a third user, C, sees B's reply, they must also be able to see A's original question, otherwise B's comment appears completely out of context. But if user D independently posts an unrelated comment on the same thread, there's no requirement that everyone sees D's comment in any particular order relative to A and B's exchange, since D's comment is causally unrelated to theirs.

**How systems implement it**

Causal consistency is usually implemented using vector clocks or similar mechanisms attached to writes, which track dependencies, so a replica can hold off showing a reply until the message it depends on has arrived.

**Interview Answer:**

"Causal consistency means if one event depends on another, like a reply depending on the original post, everyone has to see them in that order. But independent, unrelated events don't need a global order, so the system doesn't have to pay for full strong consistency everywhere. The classic example is comment threads: a reply should never show up before the comment it's replying to, but two unrelated comments posted around the same time can appear in any order without confusing anyone."

---

## Q12. [Mid] Walk me through what happens, step by step, when the leader in a replicated database cluster crashes and a new leader needs to be elected.

**Answer:**

**Leader election is the process a cluster uses to detect that the current leader is gone and agree on a replacement, without ending up with two leaders at once.**

**Step by step**

1. Followers monitor the leader through heartbeats. Each follower expects a heartbeat within some timeout window, often randomized slightly per node to avoid every follower reacting at exactly the same time.
2. When a follower's timeout expires without a heartbeat, it assumes the leader is gone and transitions to a candidate state.
3. The candidate increments a term or epoch number, a monotonically increasing counter, and requests votes from the other nodes.
4. Other nodes vote for at most one candidate per term, usually granting their vote to whichever candidate asks first and has an up to date enough log.
5. Once a candidate gets votes from a majority of nodes, it becomes the new leader for that term and starts sending heartbeats itself.
6. If no candidate gets a majority, for example due to a split vote, the term times out and a new election round starts with a new term number.

**Why the majority and term number matter**

Requiring a majority prevents two leaders existing at once, since two disjoint groups can't both contain a majority of the same cluster. The term number lets nodes recognize and reject messages from a stale, previous leader that might come back online after being partitioned away.

**Interview Answer:**

"Followers expect regular heartbeats from the leader. If a follower doesn't hear one within a randomized timeout, it becomes a candidate, bumps a term number, and asks the rest of the cluster to vote for it. Whoever gets votes from a majority of nodes becomes the new leader for that term. The majority requirement guarantees only one leader can win per term, and the term number lets everyone reject stale messages from an old leader that reconnects after being partitioned away."

---

## Q13. [Mid] What problems can clock skew cause in a distributed system, and how do engineers typically deal with it?

**Answer:**

**Clock skew is the difference between the wall clock time on different machines, which happens because hardware clocks drift and NTP synchronization isn't perfect or instantaneous.**

**Problems it causes**

- Incorrect event ordering: if you timestamp events with local wall clock time and later sort by timestamp, events from a machine with a fast clock can appear to happen before events that actually happened first on a machine with a slow clock.
- Expired or premature TTLs: a cache entry or lock might expire earlier or later than intended if the issuing and checking machines disagree on the time.
- Broken "last write wins" conflict resolution: if two writes to the same key happen close together on different machines, the one with the more advanced clock wins, even if it was actually the earlier write in real time.

- Certificate and token validation failures: if a machine's clock is far enough off, TLS certificates or JWTs can appear expired or not yet valid.

**How engineers deal with it**

- Use NTP or better, PTP, to keep clocks reasonably synchronized, and monitor for drift.
- Avoid relying on wall clock time for ordering, use logical clocks instead, like Lamport timestamps or vector clocks, which capture causal order without depending on physical time.
- For systems that truly need tight time bounds, like Google Spanner, invest in specialized infrastructure, TrueTime uses atomic clocks and GPS receivers plus an explicit uncertainty interval, and the system waits out that uncertainty before committing, rather than pretending clocks are perfectly synced.
- Design conflict resolution to not solely depend on timestamp comparison when correctness matters, use version vectors or application level merge logic instead.

**Interview Answer:**
"Clock skew means different machines disagree slightly on what time it is, and that breaks anything that depends on wall clock ordering, like sorting events by timestamp, TTL expiration, or last write wins conflict resolution, because the machine with the faster clock can incorrectly look like it wrote more recently. The fix is usually to avoid depending on physical clocks for ordering at all, use logical clocks like Lamport timestamps or vector clocks instead, keep NTP synchronization tight, and for systems that truly need bounded time guarantees, invest in something like TrueTime that models and waits out clock uncertainty explicitly."

---

## Q14. [Mid] What's the difference between a Lamport timestamp and a vector clock, and when would you reach for one over the other?

**Answer:**

**Both are ways to order events in a distributed system without relying on synchronized wall clocks, but they capture different amounts of information.**

**Options**

**Lamport timestamps**

- Each node keeps a single counter. On a local event, it increments the counter. On sending a message, it attaches the counter. On receiving a message, it sets its counter to max of its own and the received one, then increments.
- Gives a total order, every event gets a single number, and you can always compare any two events.
- Limitation: if Lamport timestamp of A is less than B, you cannot conclude A happened before B causally, only that they're consistently ordered. It can't distinguish "A caused B" from "A and B were concurrent but happened to get ordered this way."

**Vector clocks**

- Each node keeps a counter per node in the system, an array or vector, not just one number.
- When comparing two vector clocks, you can determine whether one happened strictly before the other, or whether they were concurrent, meaning neither caused the other.
- Limitation: the vector grows with the number of nodes, so it doesn't scale cleanly to systems with a very large or dynamic number of nodes.

**Best Choice**

Use Lamport timestamps when you just need a consistent, cheap way to order events for something like a total order broadcast. Use vector clocks when you actually need to detect conflicts, meaning you need to know whether two writes were truly concurrent, such as resolving conflicting updates to the same object in a system like Dynamo.

**Why?**

Lamport timestamps are cheap, a single integer, but they hide whether events are truly causally related. Vector clocks cost more, one integer per node, but they answer the specific question multi-writer, conflict resolving systems actually need: did these two writes know about each other, or not.

**Interview Answer:**

"Lamport timestamps give every event a single number that provides a consistent total order, cheap to maintain, but they can't tell you whether two events were truly causally related or just happened to land in that order. Vector clocks keep a counter per node, so comparing two vector clocks tells you if one causally preceded the other or if they were concurrent. I'd use Lamport timestamps for simple event ordering, and vector clocks when I actually need to detect and resolve real write conflicts, like Dynamo does for sibling values."

---

## Q15. [Mid] Two-phase commit is used to make a transaction atomic across multiple nodes. Walk me through the protocol, and tell me what goes wrong if the coordinator crashes right after the prepare phase.

**Answer:**

**Two-phase commit, 2PC, coordinates a single atomic transaction across multiple independent nodes, called participants, using a coordinator.**

**The two phases**

1. Prepare phase: the coordinator asks every participant "can you commit this transaction?" Each participant does whatever work is needed to guarantee it can commit, writes to its own log, acquires locks, and then replies "yes, I'm prepared" or "no."
2. Commit phase: if all participants said yes, the coordinator tells everyone to commit. If any participant said no, or timed out, the coordinator tells everyone to abort.

**What goes wrong if the coordinator crashes after prepare**

1. Every participant has said "yes, I'm prepared," which means each of them has taken locks and written data to a durable log, but they can't unilaterally decide to commit or abort, because they don't know what the other participants said, only the coordinator knows that.
2. The participants are now stuck holding their locks, unable to proceed, waiting for the coordinator to come back and tell them the outcome. This is often called the "blocking problem" of 2PC.
3. If the coordinator's crash is permanent or takes a long time to recover, those locks stay held for a long time, which can cascade into unrelated transactions getting blocked too.
4. When the coordinator does recover, it reads its own durable log to see what decision it had made, if any, and re-sends the commit or abort message to finish the protocol.

**Why this matters**

This blocking behavior is the core weakness of 2PC, it's why systems that need higher availability often move to three phase commit, or more commonly today, to consensus based protocols or Sagas that avoid holding cross service locks for extended periods.

**Interview Answer:**

"In 2PC, the coordinator first asks every participant to prepare, meaning lock the resources and durably log that they're ready to commit. If everyone says yes, the coordinator tells them all to commit, if anyone says no, it tells them all to abort. The dangerous case is the coordinator crashing right after everyone says yes, because participants are now holding locks and can't decide on their own whether to commit or abort, they have to wait for the coordinator to recover. That blocking behavior is 2PC's biggest weakness, and it's why a lot of systems prefer Sagas or consensus based coordination instead."

---

## Q16. [Mid] Compare two-phase commit and the Saga pattern for handling a distributed transaction, like booking a flight and a hotel together. Which would you pick?

**Answer:**

**Both approaches try to keep multiple services consistent when a single logical operation spans them, but they make very different trade-offs.**

**Options**

**Two-phase commit**

- Provides real atomicity: either both the flight and hotel booking succeed, or neither does, with no visible in-between state.
- Requires all participants to support the protocol and hold locks during the prepare phase.
- Pros: strong guarantees, simpler mental model of "all or nothing."
- Cons: blocking if the coordinator fails, doesn't scale well across services owned by different teams or vendors, held locks reduce throughput and availability.

**Saga pattern**

- Breaks the transaction into a sequence of local transactions, each with a corresponding compensating action if something later fails.
- Example: book the flight, then book the hotel, if the hotel booking fails, run a compensating action that cancels the flight.
- Pros: no long held cross service locks, each service commits its own local transaction immediately, much better availability and scalability, works well across independently owned services.
- Cons: no true atomicity, there's a real window where the flight is booked but the hotel isn't, and compensating actions have to be carefully designed, since some things, like an email already sent, can't be perfectly undone.

**Best Choice**

Saga pattern.

**Why?**

In a real travel booking system, the flight and hotel are typically owned by separate services, possibly separate companies entirely, so a synchronous, locking protocol like 2PC isn't realistic to implement across that boundary, and it would create a fragile availability dependency between two systems that should be able to fail independently. Sagas accept a brief window of partial completion in exchange for much better availability, and the compensating cancel action is a completely normal, well understood operation in travel booking anyway.

**Interview Answer:**
"I'd use the Saga pattern here. 2PC gives you true atomicity, but it requires holding locks across every participant until the coordinator finishes, which doesn't scale across independently owned services like a flight provider and a hotel provider, and it creates a blocking failure mode if the coordinator crashes. A Saga books the flight, then books the hotel, and if the hotel booking fails it runs a compensating action to cancel the flight. You give up strict atomicity for a short window, but you get much better availability and you're not tightly coupling two systems that should fail independently."

---

## Q17. [Mid] What is split brain, and how would you prevent it in a system that uses leader election?

**Answer:**

**Split brain happens when a network partition causes two nodes to each believe they are the legitimate leader at the same time, and both start accepting writes independently.** For example, during a network partition a five-node cluster could split into a group of three and a group of two, and if the group of two doesn't realize it has lost majority, both groups might believe they are the leader and accept conflicting writes at the same time.

**Why it's dangerous**

- Both "leaders" can accept conflicting writes to the same data, which corrupts state and creates conflicts that might not be cleanly resolvable later.
- Clients on different sides of the partition get inconsistent answers depending on which leader they happen to reach.
- It's especially bad for things like distributed locks, where split brain could mean two clients both believe they hold an exclusive lock at the same time.

**How to prevent it**

1. Require a majority quorum for leadership. A node can only become or remain leader if it can reach more than half the cluster, this guarantees at most one side of any partition can have a majority, so at most one leader can exist.
2. Use fencing tokens. When a new leader is elected, it gets a monotonically increasing token number, and downstream systems, like storage, reject any write from a leader presenting an older token than one they've already seen, even if the old leader doesn't realize it's been deposed yet.
3. Use a term or epoch number, similar to Raft, so nodes can recognize and reject messages from a stale leader once a newer term has started.
4. Avoid pure heartbeat timeout based failover without quorum, since a node isolated by a partition, but still alive, might keep acting as leader unless it's specifically required to confirm it still has majority support.

**Interview Answer:**
"Split brain is when a network partition leaves two nodes each thinking they're the leader, so both start accepting writes independently, which can corrupt data or cause double actions, like two clients both holding what they think is an exclusive lock. The main defense is requiring a majority quorum to become or stay leader, since only one side of a partition can ever have a majority. On top of that, I'd use fencing tokens, a monotonically increasing number attached to each leader term, so storage systems can reject writes from a stale leader even if that leader hasn't realized yet that it's been replaced."

## Q18. [Senior] Walk me through Raft's leader election in detail. What are the failure modes, and how does Raft avoid a split vote from stalling the cluster indefinitely?

**Answer:**

**Raft models the cluster as nodes in one of three states: follower, candidate, or leader, and organizes time into terms, each with at most one leader.**

**Normal election flow**

1. All nodes start as followers and expect heartbeats from a leader.
2. A follower whose election timeout expires becomes a candidate, increments the current term, votes for itself, and sends RequestVote RPCs to every other node.
3. A node grants its vote to at most one candidate per term, and only if the candidate's log is at least as up to date as its own, this last check prevents electing a leader with stale data.
4. If the candidate collects votes from a majority of the cluster, it becomes leader for that term and immediately starts sending heartbeats to establish authority and reset everyone's election timers.

**Handling split votes**

If two followers time out at nearly the same moment, they can both become candidates for the same term and split the vote so neither reaches a majority.

- Raft's fix is randomized election timeouts: each node picks a random timeout within a range, so it's statistically unlikely for multiple nodes to time out simultaneously on every retry.
- If a split vote does happen, the election simply times out with no winner, terms increment, and another round starts, the randomization makes it very likely one node will time out first on a subsequent attempt and win cleanly.

**Other failure modes**

- Old leader reappears after a partition: it will have a stale term number, and once it receives a message with a higher term from any current node, it immediately steps down to follower, so it can't cause split brain.
- Candidate with an out of date log: even if it requests votes, no up to date node will grant it a vote, because the log up to date check in RequestVote rejects it, this protects against electing a leader who would lose committed data.
- Network partition where no side has a majority: no leader can be elected at all until connectivity is restored, this is Raft intentionally sacrificing availability to protect consistency.

**Interview Answer:**

"Raft nodes are followers, candidates, or leaders, organized into terms. A follower that stops hearing heartbeats becomes a candidate, bumps the term, and requests votes, and it wins if it gets a majority, then starts sending heartbeats. Split votes happen when two followers time out together, but Raft randomizes election timeouts so that's rare and self-resolving, a stalled election just times out and retries with a new term. A stale leader that reappears after a partition sees a higher term from the current leader and immediately steps down, so you never get two leaders active at once, and a candidate with an out of date log can't get elected because voters check log freshness before granting a vote."

## Q19. [Senior] Conceptually, how does Paxos achieve consensus among a set of nodes, and why does it have a reputation for being hard to understand and implement correctly?

**Answer:**

**Paxos is a protocol for getting a group of nodes to agree on a single value, even if some nodes fail or messages are delayed, arbitrarily, but not maliciously.**

**The roles**

- Proposers: suggest values to be agreed on.
- Acceptors: vote on proposed values.
- Learners: find out what value was chosen.

A single node often plays more than one role.

**The two phase flow, conceptually**

1. Prepare or Promise phase: a proposer picks a proposal number higher than any it has used before and asks a majority of acceptors to promise not to accept any proposal with a lower number. Acceptors reply with a promise, and tell the proposer about the highest numbered proposal they've already accepted, if any.
2. Accept phase: if the proposer gets promises from a majority, it sends an Accept request with a value, either its own proposed value, or, critically, the value from the highest numbered proposal it learned about from acceptors in step one, if one exists. Acceptors accept the request unless they've since promised a higher numbered proposal. Once a majority accept the same value, that value is chosen.

**Why the "adopt the highest previously accepted value" rule matters**

This is the subtle heart of Paxos: it guarantees safety, meaning once a value is chosen, no other value can ever be chosen, even under network delays, retries, and competing proposers, because any future proposer with a higher number is forced to discover and re-propose the already chosen value rather than overwrite it.

**Why it has a reputation for being hard**

- The basic single value protocol, often called Basic Paxos, doesn't even specify how to run it repeatedly for a stream of decisions, that's Multi-Paxos, which the original paper leaves fairly informal.
- Competing proposers can cause livelock, where proposals keep leapfrogging each other's proposal numbers without anything actually getting chosen, requiring a leader election style optimization in practice to avoid this.
- The safety proof is subtle, edge cases around partial failures and message reordering are easy to get wrong, and there's no single canonical, complete specification for a production ready version, which is part of why Raft was explicitly designed to be more understandable while providing similar guarantees.

**Interview Answer:**

"Paxos has proposers suggest a numbered proposal to a majority of acceptors, and if a majority promise to accept it, the proposer sends the actual value, adopting the highest numbered value any acceptor already accepted, if one exists, rather than overwriting it. That adoption rule is what guarantees safety, once a majority accept a value, no different value can ever be chosen later. It has a reputation for being hard because the paper describes the single decision case cleanly, but running it repeatedly in production, Multi-Paxos, handling competing proposers without livelock, and getting all the edge cases right, is only loosely

specified, which is a big part of why Raft exists, to make the same guarantees easier to implement correctly."

---

## Q20. [Senior] I want you to design the core of a strongly consistent configuration store, similar to etcd or ZooKeeper, used by other services to store leader election locks and small pieces of critical config. Walk me through your approach.

**Answer:**

This is fundamentally a consensus based key value store problem, where correctness matters far more than throughput, since it's a small amount of data that other systems' correctness depends on.

**Core requirements**

- Linearizable reads and writes, since services will use this store for things like distributed locks and leader election, where stale reads can cause real bugs.
- High availability for reads, tolerable brief unavailability for writes during leader failover.
- Small data volumes, strong durability, not a general purpose database.
- Watch or notification support, so clients can react to changes without polling.

**Design**

1. Use a small, odd numbered cluster, typically 5 nodes, running a consensus protocol like Raft. Odd sized clusters avoid wasted nodes in majority calculations and tolerate floor(N/2) failures.
2. All writes go through the elected leader, which replicates each write as a log entry to followers, a write is only acknowledged once a majority of nodes have durably persisted it, per Raft's commit rule.
3. Reads can be served two ways: linearizable reads go through the leader and confirm it's still the leader, via a quorum check, before responding, to avoid a stale reply from a leader that's been silently deposed. Lower guarantee, faster reads can be served from any follower if the client accepts slightly stale data.
4. Implement leases and TTL backed keys, so clients can build distributed locks: acquiring a lock is a conditional write, "put this key only if it doesn't exist," and the lock auto releases if the holder stops renewing its lease, protecting against a crashed lock holder blocking everyone forever.
5. Support watch streams, clients subscribe to a key or prefix and get pushed notifications on changes, this is how services detect leader changes or config updates without polling the store repeatedly.
6. Attach a monotonically increasing revision number to every write, so clients can build fencing tokens on top of locks, this prevents the split brain scenario where a paused or slow client thinks it still holds a lock after it's expired.

**Trade-offs I'd call out explicitly**

**This system deliberately sacrifices some availability, if the cluster can't reach a majority, it stops accepting writes entirely, rather than risking split brain.** That's the correct choice here because the whole point of the system is to be a source of truth other services trust blindly.

**Interview Answer:**

"I'd build this on a small, odd sized Raft cluster, five nodes, since consensus based replication is what gives you linearizable, majority acknowledged writes. Reads can go through the leader with a quorum check for strict consistency, or from followers if the caller accepts staleness. On top I'd add TTL backed leases so clients can build safe distributed locks with automatic release if a holder crashes, watch streams so services get pushed config and leadership change notifications instead of polling, and a monotonic revision number

on every write so clients can use fencing tokens to prevent stale lock holders from causing damage. The system intentionally refuses writes if it can't reach a majority, because its whole value is being a source of truth other services can trust unconditionally."

---

## Q21. [Senior] Google Spanner claims to offer external consistency across globally distributed data centers. How does TrueTime make that possible, and what's the actual cost of that guarantee?

**Answer:**

Spanner's central problem is that in a globally distributed database, you can't just use each machine's local clock to decide transaction ordering, because clocks drift and are never perfectly synchronized, which normally makes it impossible to guarantee that a transaction which commits after another, in real time, is assigned a later timestamp.

**What TrueTime actually is**

TrueTime is an API, backed by GPS receivers and atomic clocks in Google's data centers, that doesn't return a single timestamp, it returns an interval, TT.now, which returns earliest and latest, guaranteed to contain the true current time. **The width of that interval, called epsilon, represents the system's honest uncertainty about the current time, typically a few milliseconds.**

**How Spanner uses it to guarantee external consistency**

1. When a transaction commits, Spanner assigns it a timestamp based on TT.now.
2. Critically, before actually releasing the locks and letting the transaction's effects become visible, Spanner waits out the remaining uncertainty interval, this is called "commit wait."
3. By waiting until the interval has definitely passed, Spanner guarantees that any transaction which starts after this one commits, in real, external time, will be assigned a strictly later timestamp, that's what "external consistency" means: commit order matches real world time order globally, not just per node.

**The actual cost**

- Commit wait adds real latency to every commit, roughly equal to epsilon, the uncertainty bound, typically single digit milliseconds, but it's paid on every write.
- It requires genuinely specialized infrastructure, atomic clocks and GPS antennas in every data center, which most companies cannot replicate, this is why TrueTime is a defining, hard to copy piece of Spanner's design, not just a software algorithm.
- It's a deliberate, explicit trade of latency for a much stronger consistency guarantee than most distributed databases offer, most systems either don't offer external consistency at all, or fake it with vector clocks or application logic instead of hardware backed guarantees.

**Interview Answer:**

"TrueTime is Spanner's API for getting the current time as an interval, not a single value, backed by atomic clocks and GPS in every data center, so it can express how uncertain it actually is about 'now,' typically a few milliseconds. Spanner uses that by waiting out the uncertainty window, commit wait, before making a transaction's effects visible, which guarantees that any transaction that truly starts later in real time gets a later timestamp, everywhere, not just locally. The cost is real: every commit pays that uncertainty window in added latency, and it depends on specialized hardware most companies don't have, which is exactly why this level of consistency at global scale is so hard to replicate outside Google."

## Q22. [Senior] For a quorum based system with N equals 5 replicas, walk me through the trade-offs of W equals 1, R equals 5 versus W equals 5, R equals 1 versus W equals 3, R equals 3. When would you pick each?

**Answer:**

**All three satisfy W plus R greater than N, meaning 6 in every case, so all three guarantee overlap between the write set and read set, but they distribute latency and fault tolerance very differently.**

**Options**

**W=1, R=5**

- Writes only need one node to acknowledge, so writes are extremely fast and available even with four nodes down.
- Reads must contact all five nodes, so a read is only as fast as the slowest reachable node, and reads fail entirely if even a single node is unreachable.
- Best for extremely write heavy, read light workloads where read latency and read availability aren't critical, like a firehose ingestion pipeline that's rarely read directly.

**W=5, R=1**

- Writes need every single node to acknowledge, so writes are slow, and a write fails entirely if even one node is down, terrible write availability.
- Reads only need one node, so reads are extremely fast and tolerate up to four node failures.
- Best for read heavy, write light workloads where you want the fastest, most available reads possible and can tolerate slower, less available writes, like a rarely updated reference dataset read constantly.

**W=3, R=3**

- Balanced: both reads and writes tolerate up to two node failures, since a majority, three of five, is always reachable as long as three nodes are up.
- Neither reads nor writes are as fast as their single node counterparts, but neither is fragile either.
- Best for general purpose workloads with a mix of reads and writes, and no strong reason to bias heavily toward one or the other.

**Best Choice**

W=3, R=3 for most general purpose systems, with the extreme configurations reserved for workloads with a clearly lopsided read to write ratio.

**Why?**

Extreme configurations, like W=1 R=5 or W=5 R=1, each make one operation fast at the cost of making the other operation both slow and fragile, a single node outage completely blocks the "expensive" side. A balanced quorum spreads that fault tolerance evenly, so the system degrades gracefully, tolerating the same number of failures, two, for both reads and writes, which is usually what you want unless you have very strong evidence your workload is heavily skewed one way.

**Interview Answer:**
"All three satisfy the overlap rule since W plus R is six against five replicas, but they distribute cost differently. W=1, R=5 makes writes fast and highly available but makes reads slow and fragile, since a read needs every node. W=5, R=1 flips that, fast reads but slow, fragile writes needing every node to ack. W=3,

R=3 is balanced, both operations tolerate two node failures since three is always a majority. I'd default to W=3, R=3 for general workloads, and only skew toward one of the extremes if I had clear evidence the workload is heavily read heavy or write heavy and could tolerate the fragility on the other side."

---

## Q23. [Senior] Your five node cluster gets partitioned into a group of three nodes and a group of two nodes, and the partition lasts twenty minutes. Which side should be allowed to keep serving writes, and what would you actually implement to enforce that?

**Answer:**

**The group of three should serve writes, the group of two should not, this is the majority quorum rule in action.**

**Why the three node side is correct**

- Three out of five is a majority, so that side can independently confirm it hasn't been outvoted or superseded, it can safely elect or retain a leader and continue committing writes.
- The two node side cannot form a majority on its own, three votes are needed, so it cannot safely elect a new leader, and if its existing leader happens to be on that side, it should recognize it's lost majority support and step down rather than keep serving writes.

**What I'd actually implement**

1. Consensus protocol, like Raft, where every write must be acknowledged by a majority of the full cluster's voting members, five, not a majority of whatever nodes happen to be currently reachable. This is the mechanism that structurally prevents the minority side from committing anything.
2. Leader lease or heartbeat based self-demotion: if a leader on the minority side stops receiving acknowledgments from a majority within a bounded time, it proactively steps down to follower and stops serving writes, rather than waiting to be told externally that it's been replaced.
3. Fencing tokens on any external resource the leader writes to, like shared storage, so even in the rare case a stale leader on the minority side attempts a write, the downstream system rejects it because it's presenting an outdated term or token.
4. Client side leader discovery with retry and redirect, so clients talking to the isolated minority side get told, or eventually time out and rediscover, that the majority side's leader is the one to talk to.

**What happens on the minority side in the meantime**

Those two nodes should reject writes, and ideally reject strongly consistent reads too, since they can't confirm they're not stale, though they could optionally serve explicitly labeled "possibly stale" reads if the application accepts that trade-off.

**Interview Answer:**

"The three node side keeps serving writes, because three is a majority of five and can safely elect or keep a leader, the two node side cannot form a majority and must stop serving writes. I'd enforce this with a Raft style protocol where every write needs acknowledgment from a majority of the full cluster's members, not just whoever's reachable, plus a leader lease so a leader that loses majority contact proactively steps down instead of continuing to act, and fencing tokens so any stale write from a demoted leader gets rejected downstream. The minority side just returns errors or unavailable for writes until the partition heals."

## Q24. [Senior] I need a distributed lock service so only one worker processes a given job at a time. Walk me through what you'd build, and what could go wrong with a naive implementation.

**Answer:**

**A distributed lock needs to guarantee mutual exclusion even across process pauses, crashes, and network delays, which is harder than it sounds, a naive implementation looks correct until it hits an edge case.**

### A naive first attempt

Store a key in something like Redis, "lock:job123", with a TTL, set only if it doesn't already exist. A worker acquires the lock by successfully setting that key, and releases it by deleting the key when done.

### What goes wrong with the naive version

1. Process pauses: if worker A acquires the lock, then experiences a long garbage collection pause or gets descheduled by the OS for longer than the TTL, the lock expires while A still thinks it holds it. Worker B then acquires the lock and starts processing the same job. When A resumes, it has no idea time has passed, and it might still write results as if it holds the lock, now you have two workers acting on the same job concurrently, exactly what the lock was supposed to prevent.
2. Clock or TTL mismatches: if the lock store and the worker disagree even slightly on elapsed time, similar double acquisition scenarios can occur.
3. Network partition: worker A can't reach the lock store to renew the lease, but is still alive and still working, from A's point of view it looks like a failure, but it's actually still legitimately processing.

### A safer design

1. Attach a monotonically increasing fencing token to every successful lock acquisition, issued by the lock service itself, incrementing on every grant.
2. The worker must pass this fencing token along with every write it makes to the actual resource being protected, for example the job's result store.
3. That downstream resource rejects any write carrying a fencing token lower than the highest one it has already seen, this means even if worker A wakes up from its pause and tries to write, believing it still holds the lock, its write carries an old token and gets rejected, because worker B's newer token has already been used.
4. Use a proper consensus backed lock service, like one built on ZooKeeper, etcd, or Chubby, rather than a single Redis node, so the lock service itself doesn't become a single point of failure or split brain risk.

**Interview Answer:**

"A naive lock, set a key with a TTL, breaks under process pauses, if a worker holding the lock gets stuck in a long GC pause past the TTL, another worker acquires the lock while the first one has no idea time passed, and now two workers touch the same job. The fix is fencing tokens, every lock grant gets a monotonically increasing number, and the resource being protected rejects any write carrying an older token than one it's already seen, so even a confused, paused worker that wakes up and tries to act gets rejected. I'd also build the lock service itself on a consensus backed store like etcd or ZooKeeper rather than a single node, so the lock service can't split brain either."

## Q25. [Senior] People often say a message queue gives you "exactly once" delivery. Is that actually achievable in a distributed system? Walk me through what's really going on.

**Answer:**

**True exactly once delivery, meaning a message is guaranteed to arrive and be processed precisely one time, no more, no less, without any coordination on the receiving side, is not achievable in an asynchronous distributed system, and understanding why is the point of this question.**

**Why it's not achievable**

- The sender can't know for certain whether a message was received and processed, or lost, when it doesn't get an acknowledgment, because a lost ack looks identical to a lost message, or a slow receiver, from the sender's perspective.
- If the sender retries to be safe, in case the message or ack was lost, it risks delivering the same message twice.
- If the sender doesn't retry, to avoid duplicates, it risks losing messages that actually did fail to arrive.
- You cannot resolve this ambiguity with more retries or timeouts alone, it's a fundamental consequence of communicating over an unreliable network, this is closely related to the Two Generals Problem, a classic thought experiment showing that two parties can never reach perfect certainty over an unreliable channel that might silently drop messages.

**What you actually get, and how**

1. At most once: send without retry, messages can be lost, but are never duplicated. Rarely acceptable on its own.
2. At least once: retry until acknowledged, messages might be delivered more than once, but are never silently lost. This is what most real messaging systems, like Kafka or SQS, actually guarantee at the transport level.
3. Effectively exactly once: build at least once delivery, then add idempotency on the consumer side, so processing the same message multiple times produces the same result as processing it once. Typically implemented with a unique message ID and a dedupe table or idempotency key check before applying the message's effect.

**How to make idempotency practical**

- Design consumer side operations to be naturally idempotent where possible, for example "set balance to X" instead of "add X to balance."
- Where that's not possible, track processed message IDs, in the database transaction that applies the effect, so the id check and the effect commit atomically together.

**Interview Answer:**

"True exactly once delivery isn't achievable in an asynchronous distributed system, because the sender can never distinguish a lost acknowledgment from a lost message, so it either retries and risks duplicates, or doesn't retry and risks silently losing messages. What real systems actually provide is at least once delivery, retrying until acked, which can create duplicates, and then they achieve effectively exactly once processing by making the consumer idempotent, tracking a unique message ID and skipping or safely reapplying anything already processed. So 'exactly once' as marketed is really 'at least once delivery plus idempotent processing,' not a magic guarantee against duplicates at the transport layer."

## Q26. [Senior] Compare a single leader replication model with a multi leader, multi region setup where each region can accept writes locally. What breaks in the multi leader model, and how would you handle it?

**Answer:**

**The core trade-off is write latency and regional availability versus the complexity of handling conflicting writes.**

**Options**

**Single leader**

- All writes go to one leader, regardless of which region the client is in.
- Pros: simple conflict story, writes are naturally ordered since there's only one place they happen.
- Cons: clients far from the leader pay real cross region latency on every write, and if the leader's region goes down or is partitioned away, the whole system loses write availability until failover completes.

**Multi leader**

- Each region has its own leader accepting local writes, which then propagate asynchronously to other regions.
- Pros: low write latency for local users, each region keeps accepting writes even if it's cut off from the others.
- Cons: two regions can accept conflicting writes to the same record at nearly the same time, and now the system has to detect and resolve that conflict after the fact, since there was no coordination preventing it up front.

**What actually breaks in multi leader**

- Write conflicts: user updates their email in region A, and updates their shipping address in region B, milliseconds apart, before either write has replicated, now both regions have a version the other doesn't know about.
- Non-commutative operations are especially dangerous, like two regions both decrementing the same inventory count independently, if you just apply both, you can oversell stock that only existed once.
- Ordering and causality can be violated across regions if you're not careful, a reply might replicate before the comment it's responding to.

**How I'd handle it**

1. Use vector clocks or version vectors to detect when writes are truly concurrent versus one causally following the other.
2. Apply conflict resolution appropriate to the data: last write wins for low stakes fields, CRDTs, conflict free replicated data types, for things like counters or sets where merges can be defined mathematically to always converge safely, or explicit application level merge logic for anything where losing data silently is unacceptable, like showing both addresses and asking the user to pick.
3. For genuinely non-commutative, high stakes operations like inventory decrements, avoid multi leader for that specific data, route it through a single leader or a reservation service, even if the rest of the system is multi leader.

4. Use causal consistency and dependency tracking for anything display order sensitive, like comment threads, so replication respects "happened before" relationships even without a single global order.

**Interview Answer:**
"Single leader keeps writes simply ordered but costs cross region latency and centralizes write availability risk in one region. Multi leader gives every region fast, locally available writes, but opens the door to real conflicts, two regions can both write to the same record before they've synced, and for things like inventory counts that can mean overselling stock that never existed. I'd handle it with vector clocks to detect true concurrency, CRDTs or well-defined merge logic for data where automatic resolution is safe, and for anything non-commutative and high stakes, like inventory, I'd route that specific data through a single leader or reservation service even if the rest of the system stays multi leader."

---

## Q27. [Senior] What's the difference between crash fault tolerance and Byzantine fault tolerance, and when would a real system actually need the Byzantine kind?

**Answer:**

**Both describe what kinds of node misbehavior a consensus protocol can survive, but they assume very different threat models.** For example, a public blockchain network has to assume some nodes might be run by malicious actors deliberately sending false information, which is why it needs Byzantine fault tolerance, while a company's own internal Raft-based configuration store only has to survive machines crashing or losing network connectivity, not lying to each other.

**Crash fault tolerance, CFT**

- Assumes nodes either work correctly, or stop entirely, crash, hang, get partitioned away, but never send incorrect or misleading information while still participating.
- Protocols like Raft and Paxos are crash fault tolerant, they need a simple majority, more than half the nodes, to make progress and stay safe.
- This assumption is reasonable when you control all the hardware and software, your own data centers, your own deployed binaries, so a node acting maliciously or sending intentionally corrupted messages isn't a realistic threat, only crashes and network issues are.

**Byzantine fault tolerance, BFT**

- Assumes nodes can behave arbitrarily badly, sending incorrect, inconsistent, or contradictory information to different peers, whether due to bugs, corruption, or actual malicious intent, and still needs the system to reach correct consensus.
- BFT protocols typically need more than two thirds of nodes to be honest, a stricter threshold than CFT's simple majority, since the protocol has to tolerate a fraction of nodes actively lying, not just going silent.
- Requires substantially more message overhead, since nodes often need to cross verify what others claim to have received, rather than simply trusting a majority vote at face value.

**When you actually need BFT**

- Public blockchains and cryptocurrency networks, where anyone can run a node and there's a direct financial incentive to cheat, this is the textbook case.
- Systems that must tolerate compromised or actively malicious participants, not just failed ones, such as coordination between mutually distrusting organizations with no shared operator.
- You generally do not need BFT for a typical internal distributed database or microservice cluster, where you fully control and trust the operators of every node, the realistic failure mode there is a

crash or a network blip, not a node deliberately lying, so CFT protocols like Raft are the appropriate, much cheaper choice.

**Interview Answer:**

"Crash fault tolerance assumes nodes either work correctly or stop entirely, they never lie while still participating, that's the model Raft and Paxos use, and it only needs a simple majority to make progress. Byzantine fault tolerance assumes nodes can actively send false or contradictory information, whether from bugs or malicious intent, and needs a much stricter threshold, typically more than two thirds honest, plus a lot more message overhead to cross check claims. You need BFT in environments like public blockchains, where anyone can run a node and has a financial incentive to cheat. For an internal cluster you fully control and operate yourself, the realistic failure mode is a crash or network partition, not a lying node, so CFT protocols like Raft are the right, far cheaper choice."

---

**If you remember nothing else from this chapter:** *citing CAP theorem by name means nothing if you can't describe what happens to a real request during a real partition.*

# Chapter 13: Reliability & Fault Tolerance

In production, things fail constantly, and how you design for that reality determines whether a single dependency going down takes out the whole system. Use this chapter to practice reaching for the right pattern, timeout, retry, or circuit breaker, for the failure in front of you, rather than listing every pattern you've heard of.

> **War Story:** *A retry-without-backoff bug turned a two-minute blip in one downstream dependency into a four-hour, self-inflicted outage, as retries piled on top of retries across every service that called it.*

> **Avoid:** proposing retries without mentioning backoff or a circuit breaker, as if retrying is always safe
>
> **Aim For:** raising the thundering-herd risk of naive retries unprompted, and proposing backoff and circuit breaking together

## Q1. [Junior] Why would you set a timeout on an HTTP call to a downstream service instead of letting it wait indefinitely?

**Answer:**

**Without a timeout, a slow or hung downstream dependency can tie up resources in the calling service indefinitely, turning one slow dependency into an outage for everyone.**

**What happens without a timeout**

- The calling thread, connection, or worker stays blocked waiting for a response that may never come.
- Under load, all available threads or connections in the pool get consumed waiting on the same stuck dependency.
- New requests queue up behind the stuck ones, and the calling service itself becomes unresponsive even though its own code is fine.

**What a timeout gives you**

- A hard upper bound on how long you'll wait, so resources get freed and returned to the pool.
- The ability to fail fast and return an error, retry, or fall back to a default, instead of hanging.
- A signal you can alert on: rising timeout rates tell you a dependency is degrading before it fully dies.

**Practical guidance**

- Set the timeout based on the p99 latency of the downstream call plus a margin, not an arbitrary round number.
- Pair it with retries and circuit breakers, a timeout alone just converts a hang into a fast failure, it doesn't fix the underlying problem.

**Interview Answer:**
"A timeout puts a hard limit on how long I'll wait for a downstream response. Without one, a hung dependency can block my threads or connections indefinitely, and that exhaustion cascades into my own service becoming unavailable even though my code is healthy. I set timeouts based on observed p99 latency plus a margin, so I fail fast, free up resources, and can retry or degrade instead of hanging forever."

## Q2. [Junior] What's a health check endpoint, and what should it actually verify?

**Answer:**

**A health check is an endpoint, usually something like `/health`, that load balancers, orchestrators, or monitoring systems call to decide whether an instance should keep receiving traffic.**

**What it should check**

- That the process is up and able to handle requests, not just that the port is open.
- Optionally, that critical dependencies the service cannot function without (its primary database connection, for example) are reachable.

**What it should not do**

- Perform expensive operations like full end to end transactions on every check, that adds load and can make the check itself unreliable.
- Check dependencies that are not strictly required to serve requests, a health check that fails because a non critical downstream cache is slow will cause unnecessary restarts or traffic pulls.

**Two common flavors**

- A shallow check that just confirms the process responds.
- A deep check that also verifies key dependencies, used carefully since a false failure here can take a healthy instance out of rotation.

**Interview Answer:**
"A health check is a lightweight endpoint that tells the load balancer or orchestrator whether an instance is ready to serve traffic. It should verify the process is actually responsive and, for a deep check, that essential dependencies like the primary database are reachable. It shouldn't run expensive operations or depend on non critical systems, because a flaky health check that fails for the wrong reasons can pull healthy instances out of rotation or cause restart loops."

---

## Q3. [Junior] What's the difference between encryption in transit and encryption at rest? Where would you use each?

**Answer:**

**Both protect data from being read by someone who shouldn't see it, but they protect it at different points in its lifecycle.** For example, a payment form submits a credit card number over TLS so it can't be sniffed on the way to the server, encryption in transit, and that same card number is stored encrypted on disk so a stolen database backup doesn't expose raw numbers, encryption at rest.

**Encryption in transit**

- Protects data while it moves across a network, between a client and server or between two internal services.
- Implemented with TLS, so anyone intercepting the traffic on the wire sees only ciphertext.
- Applies to every hop: client to load balancer, load balancer to service, service to database.

**Encryption at rest**

- Protects data while it's stored, on disk, in a database, in a backup, or in object storage.

- Implemented with disk level or database level encryption, using keys managed by a key management service.
- Protects against someone gaining access to the physical storage medium, a stolen backup, or a misconfigured storage bucket.

**Where you need both**

- A payment system needs TLS on every network hop so card data isn't sniffed in transit, and encryption at rest so a stolen disk or leaked backup doesn't expose raw card numbers.
- Neither one alone is sufficient, they cover different attack surfaces.

**Interview Answer:**

"Encryption in transit protects data while it's moving across the network, that's TLS between clients and servers or between internal services. Encryption at rest protects data while it's stored, on disk, in a database, or in backups, typically using a key management service to handle the keys. A real system needs both: TLS stops someone sniffing traffic, at rest encryption stops someone who gets a stolen disk or backup from reading raw data."

---

## Q4. [Junior] Your service calls another service that fails intermittently. Why is retrying the request not always a safe fix?

**Answer:**

**Retrying feels like the obvious answer to a transient failure, but it introduces its own risks if you don't think through what the failed operation actually did.**

**When retries are safe**

- The operation is idempotent, meaning running it twice has the same effect as running it once, like a GET request or a PUT that sets a value to a fixed state.
- The failure was clearly transient, like a network blip or a timeout, not a validation error or a 4xx response.

**When retries cause problems**

- The operation is not idempotent, like "charge this card" or "increment this counter", a blind retry can double charge a customer or corrupt a count.
- You don't know if the original request actually succeeded on the server but the response was lost, retrying in that case duplicates the effect.
- Retrying aggressively during an outage adds more load to an already struggling service, making the outage worse, sometimes called a retry storm.

**How to make retries safe**

- Use idempotency keys so the server can recognize and deduplicate a retried request.
- Only retry on specific error classes (timeouts, 5xx, connection errors), never on 4xx client errors.
- Cap the number of retries and use backoff so you're not hammering a struggling dependency.

**Interview Answer:**

"Retries are only safe if the operation is idempotent or if I have a way to deduplicate it, like an idempotency key. If I blindly retry something like a payment charge, I risk double charging the customer because I don't actually know if the first attempt succeeded before the response was lost. I'd also limit retries to specific

error types like timeouts and 5xx errors, and add backoff, so I'm not adding more load onto a dependency that's already struggling."

---

## Q5. [Junior] What is a circuit breaker, and what problem does it solve that a timeout alone doesn't?

**Answer:**

**A circuit breaker sits in front of calls to a downstream dependency and tracks whether those calls are succeeding or failing, so it can stop sending traffic to a dependency that's clearly broken.** For example, if a payment service goes down, a circuit breaker can notice the failure rate spike, open the circuit, and immediately return an error for the next batch of requests instead of letting thousands of callers each wait out a five second timeout against a dependency that's already dead.

**The problem with timeouts alone**

- A timeout limits how long you wait per call, but if the dependency is down, every single call still has to wait out the full timeout before failing.
- Under load, that means thousands of requests all burning their timeout duration against a dead dependency, wasting resources and adding latency to every caller.

**What a circuit breaker adds**

- It tracks failure rate over a rolling window.
- Once failures cross a threshold, it "opens" the circuit and starts failing calls immediately without even attempting the network call, avoiding the wasted wait.
- After a cooldown period, it goes "half open" and allows a small number of test requests through to see if the dependency has recovered.
- If those succeed, the circuit "closes" and normal traffic resumes, if they fail, it reopens and waits again.

**Interview Answer:**
"A timeout limits how long any single call waits, but if a dependency is fully down, every request still pays that full timeout cost before failing. A circuit breaker tracks the failure rate and, once it crosses a threshold, opens and fails calls immediately without even attempting the network request, which saves resources and latency across the board. After a cooldown it goes half open, sends a few test requests, and closes again if the dependency has recovered."

---

## Q6. [Junior] What's rate limiting, and why would a public API need it even if the backend can technically handle the load?

**Answer:**

**Rate limiting caps how many requests a client, user, or API key can make in a given time window, and it protects against more than just raw overload.**

**Reasons beyond raw capacity**

- Fair usage: without limits, one heavy client can consume disproportionate capacity and degrade the experience for everyone else sharing the system.
- Abuse prevention: rate limits slow down brute force login attempts, scraping, and credential stuffing attacks, even if the backend could technically serve the requests.

- Cost control: many APIs meter usage tied to real infrastructure or third party costs, and limits prevent runaway bills from a bug or malicious actor.
- Protecting downstream dependencies: your backend might scale fine, but a dependency further down the chain, like a third party payment provider, might not.

**How it's typically enforced**

- Per API key or per user, so one identity can't starve others.
- Different tiers, generous limits for paying customers, tighter limits for anonymous or free tier traffic.
- Returned with clear signals, like a 429 status code and headers telling the client how long to wait before retrying.

**Interview Answer:**
"Rate limiting isn't just about protecting raw backend capacity, it's about fairness, abuse prevention, and cost control. Even if my servers could handle a flood of requests, I still want limits so one client can't starve everyone else, so brute force login attempts get slowed down, and so a bug or bad actor can't run up a huge bill against a metered third party dependency. I'd enforce it per API key or per user, return a 429 with retry guidance, and tier limits by customer type."

---

## Q7. [Mid] Walk me through how exponential backoff works, and why you'd add jitter to it.

**Answer:**

**Exponential backoff increases the wait time between retries so that repeated failures don't hammer a struggling dependency at a constant rate.**

**How it works**

1. On the first failure, wait a base delay, say 100ms, before retrying.
2. On each subsequent failure, double the delay: 100ms, 200ms, 400ms, 800ms, and so on.
3. Cap the delay at some maximum, like 30 seconds, so waits don't grow unbounded.
4. Give up after a maximum number of attempts and surface the failure.

**Why jitter matters**

- If many clients fail at the same moment, like during a brief outage, they'll all compute the exact same backoff schedule and retry in synchronized waves.
- Those synchronized waves hit the recovering service at the same instant, over and over, which can prevent it from ever fully recovering, sometimes called the thundering herd problem.
- Jitter adds randomness to the delay, for example picking a random value between 0 and the computed backoff, so retries from different clients spread out over time instead of arriving in a synchronized spike.

**Common jitter strategies**

- Full jitter: pick a random delay between 0 and the exponential value.
- Equal jitter: take half the exponential delay as a fixed floor, and add a random amount on top for the rest.

**Interview Answer:**
"Exponential backoff doubles the wait between retries after each failure, so I'm not hammering a struggling dependency at a constant rate, and I cap it at a max delay with a max retry count. The problem is that

without jitter, every client that failed at the same time computes the same backoff schedule and they all retry in synchronized waves, which can keep a recovering service from ever stabilizing. Adding randomness, jitter, to the delay spreads those retries out over time instead of piling up at the same instant."

---

## Q8. [Mid] Design a circuit breaker for a service that calls a flaky payment provider. Walk me through its states and the transitions.

**Answer:**

**A circuit breaker for this payment provider needs three states, and the transitions between them should be based on real failure signal, not a single bad call.**

**Closed (normal operation)**

- All calls pass through to the payment provider normally.
- The breaker tracks a rolling window of successes and failures, for example the last 20 calls or the last 10 seconds.
- If the failure rate in that window crosses a threshold, say 50 percent, the breaker trips to open.

**Open (failing fast)**

- Calls are rejected immediately without hitting the network, returning a fast error or triggering a fallback.
- This protects both the payment provider from more load while it's struggling, and the calling service from wasting resources on calls likely to fail.
- After a cooldown period, say 30 seconds, the breaker moves to half open.

**Half open (testing recovery)**

- A small number of requests are allowed through to test if the provider has recovered.
- If those succeed above the threshold, the breaker closes and normal traffic resumes.
- If they fail, the breaker reopens and the cooldown timer resets, possibly with a longer cooldown on repeated failures.

**Payment specific considerations**

- Distinguish between error types: a timeout or 5xx should count toward tripping the breaker, a declined card (a legitimate business response) should not.
- Combine the breaker with an idempotency key so retried or fallback attempts don't double charge.
- Have a fallback plan for the open state, like queuing the payment for later processing or switching to a secondary payment provider.

**Interview Answer:**
"I'd track a rolling window of call outcomes and trip the breaker to open once the failure rate crosses a threshold, at which point calls fail immediately instead of hitting the flaky provider. After a cooldown, it goes half open and lets a few test requests through, closing again if they succeed or reopening if they don't. The important detail for payments is only counting real failures, timeouts and 5xx errors, toward the threshold, not legitimate declines, and pairing this with idempotency keys so any fallback or retry doesn't double charge the customer."

## Q9. [Mid] What's a bulkhead pattern, and how would you apply it to protect a service that talks to multiple downstream dependencies?

**Answer:**

**A bulkhead isolates resources, like thread pools or connection pools, per dependency, so that a failure in one dependency can't exhaust resources needed to talk to the others.**

**The problem without bulkheads**

- If all downstream calls share one thread pool or connection pool, one slow or hung dependency can consume every available thread waiting on it.
- Once that pool is exhausted, calls to healthy dependencies also start failing, because there's nothing left to serve them, turning one bad dependency into a total outage.

**How bulkheads fix this**

- Give each downstream dependency its own dedicated pool of threads or connections, sized to what that dependency actually needs.
- If the payment provider hangs, only the payment pool gets exhausted, calls to inventory or shipping keep flowing normally through their own pools.
- This mirrors the ship design the pattern is named after: separate watertight compartments so a hole in one doesn't sink the whole vessel.

**Applying it in practice**

- Use separate connection pools per downstream service in your HTTP client configuration.
- Size each pool based on that dependency's expected concurrency and latency, not a single global number.
- Combine with circuit breakers per dependency, the bulkhead limits blast radius, the breaker stops sending doomed calls once a dependency is clearly down.

**Interview Answer:**
"A bulkhead gives each downstream dependency its own isolated pool of threads or connections instead of sharing one global pool. Without that isolation, one hung dependency can exhaust the shared pool and take down calls to every other dependency too, even healthy ones. By partitioning resources per dependency, a failure in the payment provider only exhausts the payment pool, calls to inventory and shipping keep working through their own separate pools. I'd pair this with per dependency circuit breakers so failing calls also get cut off quickly."

---

## Q10. [Mid] Your recommendation service depends on a third-party API that's currently down. Users are still trying to load the homepage. How do you keep the site usable?

**Answer:**

**This is a graceful degradation problem: the recommendation feature is not essential to loading the page, so its failure shouldn't block or crash the whole request.**

**Immediate steps**

1. Wrap the call to the recommendation API in a timeout and circuit breaker so it fails fast instead of hanging the whole page load.

2. Catch the failure and fall back to a default experience, such as a generic "popular items" list computed ahead of time, or simply omitting the recommendations section.
3. Make sure the homepage template renders correctly with recommendations absent, this needs to be tested as a real code path, not just an afterthought.

**Longer term resilience**

- Cache the last known good recommendations per user or segment, and serve stale cached results during an outage rather than nothing at all.
- Load non critical sections like recommendations asynchronously on the client side, so a slow or failed call doesn't block the rest of the page from rendering.
- Track this dependency's health separately and alert on it, but don't page anyone at 3am for a non critical outage, only for full outages of critical paths.

**Why this matters**

- Treating every dependency as equally critical means a non essential feature failing takes down your whole product, which is a bad trade off for the actual business value of that feature.

**Interview Answer:**

"I'd treat recommendations as a non critical dependency and design for its absence. That means a timeout and circuit breaker around the call so it fails fast, a fallback like cached or generic recommendations instead of a blank error, and rendering the homepage in a way that works fine with that section simply missing. Ideally the recommendations load asynchronously on the client so a slow third party API never blocks the rest of the page from rendering in the first place."

---

## Q11. [Mid] Compare TLS termination at the load balancer versus end-to-end TLS all the way to your application servers. Which would you pick for a payments system?

**Answer:**

Both protect data in transit, they differ in where encryption stops and what's exposed on the internal network.

**Option: TLS termination at the load balancer**

- The load balancer decrypts incoming TLS traffic and forwards plain HTTP to backend servers over the internal network.
- Pros: simpler certificate management (one place to renew certs), offloads CPU heavy TLS handshakes from application servers, easier to inspect traffic for debugging or security tooling.
- Cons: traffic between the load balancer and application servers is unencrypted, relying entirely on the internal network being trusted and isolated.

**Option: End to end TLS to the application servers**

- The load balancer passes through or re-encrypts traffic so it stays encrypted all the way to the application server.
- Pros: no plaintext anywhere on the wire, even inside your own data center or VPC, which matters if the internal network is ever compromised or if you operate in a zero trust model.
- Cons: more certificate management overhead across many servers, added CPU cost at each hop, more complexity in debugging since traffic can't be inspected in plaintext.

**Best Choice**

**End to end TLS, terminating at the application server or at a sidecar proxy next to it.**

**Why?**

For a payments system, compliance standards like PCI DSS (the Payment Card Industry Data Security Standard) generally expect cardholder data to be encrypted end to end, and you shouldn't assume the internal network is safe from a compromised host or an insider. The added operational cost of managing certificates across the fleet is worth it given what's at stake, and this can be simplified with a service mesh that automates certificate issuance and rotation between services.

**Interview Answer:**
"Terminating TLS at the load balancer is simpler to operate but leaves traffic in plaintext on the internal network, which assumes that network is fully trusted. For a payments system I wouldn't make that assumption, I'd want end to end TLS all the way to the application server, ideally handled by a service mesh sidecar so I'm not manually managing certificates on every host. It costs more in CPU and operational complexity, but it matches what compliance standards like PCI DSS expect and protects against a compromised internal host."

---

## Q12. [Mid] Walk me through what an idempotency key is and why you'd require one on a payment retry endpoint.

**Answer:**

**An idempotency key lets the client tell the server "this is the same logical request as before," so retries don't cause the operation to happen more than once.**

**How it works**

1. The client generates a unique key, often a UUID, before making the request, and sends it in a header.
2. The server checks if it has already processed a request with that key.
3. If it has, the server returns the original result without re-executing the operation.
4. If it hasn't, the server processes the request normally and stores the key along with the result, keyed for some retention window.

**Why it's essential for payment retries**

- A client might time out waiting for a response even though the charge actually succeeded on the server side.
- Without an idempotency key, retrying that request creates a second charge, because the server has no way to know it's a retry rather than a new, separate charge request.
- With the key, the retry is recognized as the same logical operation, and the server simply replays the original success response instead of charging again.

**Implementation details worth mentioning**

- The key should be generated once per logical operation on the client, and reused across retries of that same attempt, not regenerated each time.
- Store the key and result atomically with the payment record itself, in the same transaction, so there's no race where two concurrent retries both see "not found" and both charge the card.
- Expire keys after a reasonable window, you don't need to keep them forever.

**Interview Answer:**

"An idempotency key is a unique identifier the client attaches to a request so the server can recognize a retry as the same logical operation instead of a brand new one. For payments this matters because a client can time out even after the charge succeeded, and without the key a naive retry would charge the card twice. The server stores the key with the result atomically with the transaction, so if it sees that key again, it just returns the original result instead of processing the payment a second time."

---

## Q13. [Mid] Design a disaster recovery strategy for a database that must survive the loss of an entire cloud region. What's your RTO and RPO trade-off?

**Answer:**

Surviving a full region loss means the database, and enough of the application stack to use it, needs to exist in at least one other region, with a clear plan for how fast you can fail over and how much data you can afford to lose.

**Key concepts**

- RTO (recovery time objective): how long you can be down before service is restored.
- RPO (recovery point objective): how much data, measured in time, you can afford to lose in the failover.

**Options**

- **Backup and restore to another region**: periodic backups shipped to a second region, restored on demand. Cheapest, but RPO is as bad as your backup interval (hours, potentially), and RTO is slow since you're restoring from scratch.
- **Warm standby with asynchronous replication**: a replica database running in a second region, continuously replicated but with some lag. RPO is small (seconds to low minutes of lag), RTO is fast since the replica just needs to be promoted to primary.
- **Active-active across regions**: both regions actively serving reads and writes, with conflict resolution or partitioning to keep them consistent. Near zero RTO and RPO, but significantly more complex and costly to build and operate correctly.

**Best Choice**

Warm standby with asynchronous replication for most systems, active-active only when the business truly requires near zero downtime and the team can support the added complexity.

**Why?**

Backup and restore is too slow to be a real disaster recovery plan for a region loss, most businesses can't tolerate hours of downtime and hours of data loss. Active-active solves both objectives but requires solving hard consistency and conflict resolution problems, and multiplies operational cost, so it's overkill unless the SLA genuinely demands it. **Warm standby hits a strong middle ground: fast failover, small data loss window, and manageable complexity.**

**Interview Answer:**

"I'd set up a warm standby: a replica database in a second region kept current through asynchronous replication, so RPO is just the replication lag, typically seconds, and RTO is however long it takes to promote that replica and redirect traffic, usually minutes. I'd reserve active-active for cases where the business genuinely can't tolerate any downtime, because it adds real complexity around conflict resolution and consistency. Plain backup and restore isn't fast enough or fresh enough to count as a real disaster recovery plan for losing a whole region."

## Q14. [Mid] Your service starts throwing errors under load, but only during a downstream partial outage. How do you decide between retrying, failing fast, or degrading gracefully?

**Answer:**

**The right choice depends on what the downstream dependency does, whether the operation is idempotent, and whether the feature is essential to the request.**

**When to retry**

- The failure looks transient, like a connection reset or a single timeout, not a sustained pattern.
- The operation is idempotent or protected by an idempotency key.
- Use a small number of attempts with exponential backoff and jitter, never unlimited retries.

**When to fail fast**

- The dependency is clearly and consistently failing, a circuit breaker has already tripped open.
- Retrying would just add more load to an already struggling dependency without any real chance of success.
- The operation is critical and there's no reasonable fallback, better to surface a clear error immediately than to make the user wait through a doomed retry sequence.

**When to degrade gracefully**

- The failing dependency is not essential to completing the request, like a personalization or recommendation feature.
- There's a reasonable fallback: cached data, a default value, or simply omitting the feature.
- The user's core task can still complete without it.

**Putting it together**

1. Classify the dependency as critical or non critical to this request.
2. For non critical dependencies, fail fast internally and degrade gracefully for the user, don't retry something the user doesn't need to see succeed.
3. For critical dependencies, retry a bounded number of times with backoff if the failure looks transient, otherwise fail fast and surface a real error, respecting a circuit breaker if one has already tripped.

**Interview Answer:**
"First I'd figure out whether this dependency is essential to the request. If it's not, like a recommendations widget, I degrade gracefully and skip it rather than retrying something the user doesn't actually need. If it's essential, I check whether the failure looks transient, in which case I retry a small bounded number of times with backoff and jitter, respecting any circuit breaker that's already open. If the dependency is clearly and consistently down, I fail fast rather than retrying into a wall, since that just adds load without any real chance of success."

## Q15. [Senior] Design the full resilience strategy (timeouts, retries, circuit breaker, bulkhead) for a checkout service that calls inventory, payment, and shipping services, each with different failure characteristics.

**Answer:**

**Checkout has three dependencies with very different risk profiles, so a single one size fits all resilience policy is the wrong approach, each needs its own tuning.**

**Per dependency design**

**Inventory service**

- Fast, high volume, called to confirm stock before charging.
- Timeout: short, maybe 200ms, since it's an internal, usually fast service.
- Retries: 2 attempts with small backoff, since checking stock is a read and safely retryable.
- Bulkhead: its own connection pool, sized generously since it's called on every checkout.
- Circuit breaker: trips quickly on sustained failure, since without inventory data checkout can't safely proceed.

**Payment service**

- Slower, external or semi external, and definitely not safely retryable without care.
- Timeout: longer, maybe 3 to 5 seconds, since payment providers can be slow.
- Retries: only with an idempotency key, capped at 1 or 2 attempts, since a blind retry risks double charging.
- Bulkhead: dedicated, smaller pool, since payment calls hold resources longer.
- Circuit breaker: trips on real failures (timeouts, 5xx), never on business declines.

**Shipping service**

- Least critical to complete the immediate transaction, shipping options can be estimated or finalized slightly later.
- Timeout: moderate, and can degrade to a default shipping estimate if it doesn't respond in time.
- Retries: light, since correctness here is less urgent.
- Bulkhead: smallest allocation, since it's the least critical path.
- Circuit breaker: trips into a fallback of default shipping rates rather than blocking checkout.

**Orchestration across all three**

1. Reserve inventory first, fail fast if it's unavailable, no point charging for something out of stock.
2. Charge payment with an idempotency key, only after inventory is confirmed.
3. Attempt shipping quote and booking, but if it's slow or down, fall back to a default rate and finalize the order asynchronously once shipping recovers.
4. Wrap the whole checkout flow with an overall request timeout budget so no single slow step can make the user wait indefinitely.

**Interview Answer:**

"I'd treat each dependency differently based on its role. Inventory is fast and safely retryable, so short timeout, a couple of quick retries, and its own connection pool, and I fail the checkout fast if it's down since I can't sell what I can't confirm is in stock. Payment gets a longer timeout, at most one or two retries guarded by an idempotency key so I never double charge, and its own bulkhead and circuit breaker tuned to trip only on real failures, not declines. Shipping is least critical to the immediate transaction, so I give it a smaller resource allocation and fall back to a default rate if it's slow, finalizing the real rate asynchronously later rather than blocking the whole checkout."

## Q16. [Senior] Walk me through a retry storm, how it happens, and how you'd prevent it across a service mesh with many hops.

**Answer:**

**A retry storm happens when retries, meant to improve reliability for a single caller, compound across a chain of services and multiply the effective load on an already struggling system, making the outage worse instead of better.**

**How it happens**

1. Service A calls Service B, which calls Service C, which calls Service D.
2. Service D starts failing under load, maybe it's just a bit slow.
3. Service C's calls to D start timing out, so Service C retries, say 3 times, tripling the load it sends to D.
4. Service B's calls to C are now slower because C is busy retrying, so B's calls to C also start timing out, and B retries too, say 3 times.
5. Each hop's retries multiply against the next, so by the time you reach the origin of the failure, the effective request volume hitting D can be an order of magnitude higher than normal, exactly when it can least handle it.
6. This can turn a minor slowdown at D into a full outage that takes down the whole chain, and can prevent D from ever recovering since the retry pressure never lets up.

**How to prevent it**

- Cap total retries per request across the whole call chain, not per hop independently, for example by passing a "retries remaining" budget in request metadata.
- Only retry at one layer of the stack for a given failure, typically the layer closest to the user, and have inner services fail fast rather than also retrying.
- Use circuit breakers at every hop so a hop stops retrying against a dependency that's clearly down, rather than retrying blindly.
- Use exponential backoff with jitter at whatever layer does retry, so retries don't arrive in synchronized bursts.
- Set a global request deadline that propagates through the call chain, so downstream services know how much time budget is left and can skip retries entirely if there's no time left anyway.

**Interview Answer:**
"A retry storm happens when retries at each hop in a call chain compound multiplicatively, so a service that's just a bit slow ends up getting hit with retry traffic from every layer above it, turning a minor degradation into a full outage. I'd prevent it by not retrying at every layer, generally only the outermost layer retries and inner services fail fast, by using circuit breakers at each hop so a clearly failing dependency stops receiving retried traffic, and by propagating a request deadline through the whole chain so services downstream know not to bother retrying if there's no time budget left."

## Q17. [Senior] Compare active-active and active-passive failover for a multi-region system. Which would you pick for a system that can't tolerate more than a few seconds of downtime?

**Answer:**

**Both approaches run infrastructure in multiple regions, they differ in whether the secondary region is actively serving traffic before a failure happens.**

**Option: Active-passive**

- One region serves all live traffic, a second region stands by with replicated data but isn't serving requests.
- On failure, traffic is redirected to the passive region, which must be promoted (for example, its database replica becomes primary) before it can serve.
- Pros: simpler to reason about, no need to handle conflicting writes from two active regions.
- Cons: failover takes real time, typically tens of seconds to a few minutes, to detect the failure, promote the standby, and redirect traffic through DNS or a load balancer, that gap is downtime.

**Option: Active-active**

- Both (or all) regions actively serve live traffic simultaneously, with data replicated and reconciled between them.
- Pros: if one region fails, traffic simply shifts to the remaining region or regions that are already warm and already serving traffic, so the failover gap is dramatically smaller.
- Cons: much harder to build correctly, requires solving conflict resolution for concurrent writes to the same data in different regions, or partitioning data so each region owns a distinct subset, and testing this thoroughly is a real engineering investment.

**Best Choice**

Active-active.

**Why?**

A requirement of a few seconds of downtime rules out active-passive, because detecting a failure and promoting a standby reliably takes longer than that in almost every real system, DNS propagation and health check intervals alone often exceed a few seconds. Active-active is the only approach where the failover path is "route away from the dead region," which can happen at the load balancer or DNS layer nearly instantly, since the other region is already live and already has current data.

**Interview Answer:**

"Active-passive is simpler, but failover means detecting the failure, promoting a standby, and redirecting traffic, and that sequence realistically takes tens of seconds to minutes, which blows past a few second downtime budget. Active-active keeps every region live and serving real traffic continuously, so failover is just rerouting away from the dead region at the load balancer or DNS layer, which can happen almost instantly since the healthy region is already warm and current. Given the tight requirement here, I'd choose active-active and invest in solving the harder problem of data conflict resolution or clean partitioning between regions."

---

## Q18. [Senior] How would you design chaos engineering experiments for a production payment system without risking real customer harm?

**Answer:**

**Chaos engineering deliberately injects failure to verify a system actually behaves the way you assume it does, but for a payment system the blast radius has to be tightly controlled since real money and real customers are involved.**

**Principles to follow**

1. Start with a clear hypothesis, like "if the payment provider times out, checkout falls back to a queued retry instead of erroring out to the customer," and design the experiment to test exactly that.
2. Run first in staging or a shadow environment that mirrors production traffic patterns, before ever touching production.
3. In production, scope the blast radius tightly: target a small percentage of traffic, a single non critical region, or synthetic transactions rather than real customer transactions.
4. Have an automated, fast abort mechanism, if key business metrics like payment success rate or checkout completion drop below a threshold during the experiment, the experiment stops immediately.
5. Run during low traffic windows initially, and only expand scope once confidence is built.

**Concrete experiments for this system**

- Inject latency into calls to the payment provider and confirm the timeout and circuit breaker trip as designed, and that checkout degrades or queues rather than hanging.
- Kill a percentage of instances of the checkout service and confirm the load balancer routes around them without dropped requests.
- Simulate a full region outage using synthetic traffic and confirm failover to the standby region actually completes within the target RTO.
- Introduce packet loss between checkout and inventory, and confirm retries and bulkheads behave as designed rather than cascading.

**Guardrails specific to payments**

- Never inject failure into real customer initiated payment transactions, use synthetic transactions or shadow traffic that mirrors real patterns without touching real money.
- Coordinate with on call and have a rollback plan ready before starting, chaos experiments should be scheduled, monitored events, not surprises.

**Interview Answer:**
"I'd start with a specific hypothesis, like confirming a payment provider timeout triggers our circuit breaker and fallback instead of hanging checkout, and test it first in staging against realistic traffic. In production, I'd scope the blast radius tightly, using synthetic transactions rather than real customer payments, targeting a small percentage of traffic or a single region, and I'd have an automated abort that halts the experiment if real business metrics like payment success rate dip. The goal is proving our resilience mechanisms work as designed, without ever risking an actual customer's money or a real transaction."

---

## Q19. [Senior] Walk me through how you'd design a rate limiter that has to be enforced consistently across a fleet of stateless API servers.

**Answer:**

**The core challenge is that stateless servers don't share memory, so a rate limit tracked only in local memory on each server lets a client exceed the intended global limit by spreading requests across servers behind a load balancer.**

**Why local, in memory limiting fails**

- If the limit is 100 requests per minute and you have 10 servers behind the load balancer, a client that gets spread evenly across all 10 could send 1000 requests per minute in total, each server only sees 100 and thinks it's compliant.

**Design with shared state**

1. Use a centralized, fast data store, typically Redis, that every API server reads from and writes to when checking a client's rate limit.
2. Track counts per client identifier (API key, user ID, or IP) with a time window, using an atomic increment operation so concurrent requests from different servers don't race and both pass a check that should have failed.
3. Use a sliding window or token bucket algorithm implemented via Redis operations (like a Lua script for atomicity) rather than fixed windows, to avoid burst issues at window boundaries.
4. Set a short TTL on the counter keys so they expire automatically and don't need manual cleanup.

**Handling scale and failure**

- Redis becomes a critical dependency for every request, so it needs to be highly available itself, typically clustered, and the rate limiter should have a defined fallback (fail open and allow the request, or fail closed and reject it) if Redis is unreachable, chosen based on whether abuse prevention or availability matters more for this system.
- To avoid the shared store becoming a bottleneck at very high scale, some systems use local counters with periodic synchronization to a central store, trading a small amount of limit precision for lower latency and reduced load on the central store.

**Interview Answer:**
"Stateless servers can't just keep the rate limit counter in local memory, because a client can be spread across the fleet and blow past the intended limit while each server individually thinks it's fine. I'd use a shared, fast store like Redis, with atomic increment operations, ideally via a Lua script, so concurrent checks from different servers don't race past each other. I'd implement it as a sliding window or token bucket to avoid burst problems at fixed window boundaries, and decide up front whether to fail open or fail closed if Redis itself becomes unavailable, based on whether availability or strict abuse prevention matters more for that particular API."

---

## Q20. [Senior] Encryption at rest is enabled everywhere in this system, but I still think you have a real security gap. Where would you look?

**Answer:**

**Encryption at rest protects data if someone steals the physical storage or gains unauthorized filesystem access, but it doesn't protect against several other realistic attack paths that are easy to overlook.**

**Places a gap commonly hides**

- **Key management**: if encryption keys are stored alongside the encrypted data, or a compromised application has access to both the data and the key, encryption at rest provides no real protection, since the attacker who compromises the app can just read decrypted data through the application anyway.
- **Data in use**: encryption at rest doesn't protect data while it's decrypted in memory and being processed, if an attacker gets code execution on the server, they can read plaintext directly from memory regardless of how the data was stored.
- **Application layer access controls**: encryption at rest doesn't stop an attacker who has valid, if overprivileged, application credentials, from simply querying the data through the normal application interface, which decrypts it transparently for legitimate use.

- **Logs and backups**: sensitive data often leaks into application logs, error messages, or third party monitoring tools in plaintext, completely bypassing storage level encryption, and backups may be encrypted with weaker or older keys, or stored in a less protected location than production.
- **Encryption in transit**: if internal service to service traffic isn't also encrypted, data is exposed on the wire even though it's protected at rest, closing one door while leaving another open.
- **Access control granularity**: overly broad IAM roles or database permissions mean encryption at rest is a formality, since the legitimate decrypt path is available to far more people or services than it should be.

**What good coverage looks like**

- Keys managed by a dedicated key management service, separate from the data store, with tight access controls and rotation.
- Least privilege access to both keys and data, audited regularly.
- Sensitive fields scrubbed from logs and monitoring tooling.
- Encryption in transit for all internal and external traffic, not just at rest.

**Interview Answer:**

"Encryption at rest only protects against someone stealing the physical storage, it doesn't protect against a compromised application reading decrypted data in memory, against overprivileged accounts querying the data normally through the app, or against sensitive fields leaking into logs and monitoring tools in plaintext. I'd look hardest at key management, are keys separate from the data and tightly access controlled, at whether internal traffic is also encrypted in transit, and at whether access to both the data and the keys follows least privilege. Encryption at rest is one layer, not a substitute for access control and secure handling everywhere else."

---

## Q21. [Senior] Design a zero trust access model for a system where you can no longer assume the internal network is trusted. What changes compared to a traditional perimeter model?

**Answer:**

**A traditional perimeter model assumes that once you're inside the corporate network or VPC, you're mostly trusted, zero trust assumes the opposite: no request is trusted by default, regardless of where it originates, and every request must prove who it is and what it's allowed to do.**

**What changes from a perimeter model**

**Identity replaces network location as the trust boundary**

- Instead of "this request came from inside the VPC, so it's probably fine," every service to service call must present verifiable identity, typically via mutual TLS certificates issued to each service.
- Human access follows the same logic: strong authentication (with multi factor) plus continuous verification, not just "you're on the VPN so you're in."

**Every request is authenticated and authorized, every time**

- No implicit trust from being on the internal network, a service calling another service still needs to authenticate and be authorized for that specific action, even if both are inside the same data center.
- Authorization decisions consider more than just identity, factors like device posture, request context, and the sensitivity of the resource can all factor in.

**Micro segmentation instead of a flat internal network**

- Instead of one flat trusted internal network where any service can reach any other, network policies restrict which services can even talk to which other services, so a compromised host can't freely move laterally.

**Practical building blocks**

1. Mutual TLS between all services, using short lived certificates issued and rotated automatically, often via a service mesh.
2. A centralized policy engine that every service consults (or that's enforced via a sidecar) to authorize each request based on identity, action, and resource.
3. Short lived credentials everywhere, long lived static secrets and API keys are minimized since they're a bigger liability if leaked.
4. Continuous logging and monitoring of access decisions, since zero trust relies on being able to detect anomalies rather than assuming the perimeter blocked the bad actor already.

**Interview Answer:**

"In a perimeter model, being inside the network is treated as mostly trustworthy. Zero trust removes that assumption entirely, every request, whether it's a person or a service, has to authenticate and be authorized for that specific action regardless of where it's coming from. Practically that means mutual TLS between all services with short lived, automatically rotated certificates, a centralized policy engine that authorizes each request based on identity and context, and micro segmentation so a compromised host can't just freely reach everything else on the network. It's a shift from trusting location to trusting verified identity on every single request."

---

## Q22. [Senior] Two services in your architecture use mutual retries and a slow shared database that starts to saturate connections. Walk me through how this cascades into a full outage and how you'd have prevented it.

**Answer:**

**This is a classic cascading failure where a small initial slowdown gets amplified by well intentioned resilience mechanisms until it takes down the whole system.**

**How the cascade unfolds**

1. The shared database starts responding slowly, maybe due to a slow query, a lock, or a spike in load.
2. Service A's queries to the database start taking longer, its connection pool fills up with in flight queries that haven't returned yet.
3. Service A starts timing out on some queries and retries them, but the retries go right back to the same overloaded database, adding more concurrent connections rather than fewer.
4. Meanwhile, Service B, which calls Service A, sees Service A getting slower (because A itself is now blocked on its exhausted connection pool), times out, and also retries.
5. Now both services are retrying against an already saturated resource, connection pool exhaustion spreads to Service A itself, since its pool is fully consumed by slow queries and retries, so even unrelated requests to Service A start failing.
6. What started as one slow database query becomes total unavailability of both services, because the retries multiplied load at exactly the moment the database most needed load to decrease.

**How to have prevented it**

- **Bulkheads**: separate connection pools per query type or per calling service, so a slow query pattern from one workload doesn't exhaust the pool needed by unrelated workloads.
- **Bounded retries with backoff and jitter**: cap retry attempts and back off, so retries decrease pressure over time instead of arriving as an immediate second wave of the same load.
- **Circuit breakers**: once queries are failing at a high rate, stop sending new queries to the database from that path entirely for a cooldown period, rather than continuing to attempt them.
- **Load shedding**: when the database is clearly saturated, deliberately reject a portion of incoming requests early, before they consume a connection, prioritizing what you keep serving.
- **Query level timeouts tuned tightly**: a query timeout much shorter than the connection pool exhaustion point ensures slow queries get killed and their connections freed before the pool fully saturates.
- **Monitoring connection pool utilization directly**: alerting on pool saturation before it hits 100 percent gives you a chance to intervene before the cascade is already underway.

**Interview Answer:**

"A slow query fills up Service A's connection pool with in flight requests, Service A starts timing out and retrying against the same overloaded database, which adds more load exactly when the database needs less. Service B then sees Service A get slow, times out, and also retries, so now two services are adding retry pressure onto an already saturated resource, and it cascades into full unavailability. I'd prevent this with bulkheads so one query pattern can't exhaust a shared pool, bounded retries with backoff and jitter so retries don't compound the load, circuit breakers that stop sending queries once failures spike, and tight query timeouts plus pool utilization alerts so the problem gets caught and contained before it cascades this far."

---

## Q23. [Senior] How would you decide the timeout budget across a request that fans out to five microservices, each with their own timeouts, and still needs to return within a 500ms SLA?

**Answer:**

**The overall SLA has to be broken down into a budget that accounts for both the fan out pattern and a safety margin, otherwise individual service timeouts can add up to something that blows past the SLA even when every service is technically within its own limit.**

**Key questions to answer first**

- Are the five calls made in parallel or sequentially? This changes the math completely: parallel calls only need to fit the slowest one within budget, sequential calls need their timeouts to sum within budget.
- Are all five calls required for a valid response, or can some be treated as optional and skipped if slow?

**Approach for parallel fan out (the common case)**

1. Reserve a margin off the top for overhead you don't control: network hops to and from the client, serialization, and the gateway or load balancer itself, say 50 to 100ms.
2. Distribute the remaining budget, roughly 400ms, across the five services, but not necessarily evenly, faster or less critical services can get shorter timeouts, slower critical ones get more.

3. Set each service's timeout so that even the slowest one still leaves room for the response to be assembled and returned within the remaining budget, for example capping each at 350ms so there's still time to aggregate results after the slowest one returns or times out.
4. For non critical services in the fan out, set an aggressive timeout and treat a timeout there as "proceed without this data" rather than failing the whole request.

**Handling sequential dependencies within the fan out**

- If any of the five calls depend on each other's results, their timeouts need to be budgeted as a sum, not treated independently, which usually means those calls need much tighter individual timeouts to leave room for the others.

**Propagating the deadline**

- Pass the remaining time budget along with the request (a deadline, not just a fixed timeout), so each downstream service knows exactly how much time is actually left and doesn't waste effort on work that will exceed the caller's deadline anyway.

**Interview Answer:**

"First I'd confirm whether the five calls are parallel or sequential, since that completely changes the math, parallel calls only need to fit the slowest one in budget, sequential calls need their timeouts to sum within it. Assuming parallel, I'd reserve some margin off the top for network and aggregation overhead, then set each service's timeout so even the slowest still leaves room to assemble the response, treating non critical services more aggressively so a slow, optional call doesn't hold up the whole response. I'd also propagate an actual deadline rather than independent fixed timeouts, so every downstream service knows exactly how much time budget is genuinely left."

---

**If you remember nothing else from this chapter:** *a retry with no backoff isn't resilience, it's a denial-of-service attack you launch against yourself.*

# Chapter 14: Object-Oriented Design & SOLID Principles (LLD Foundations)

SOLID principles are easy to recite and hard to actually apply. The code-snippet questions in this chapter test whether you can spot a real violation in front of you and fix it, a much stronger skill than being able to define a principle from memory.

> **War Story:** *A single "do everything" order class that also handled payments and notifications meant a one-line change to notification formatting required re-testing the entire checkout flow, every single time.*

> **Avoid:** adding a new responsibility to an existing class because it's convenient, without noticing it now has two reasons to change
>
> **Aim For:** naming the specific SOLID principle a piece of code violates, rather than just calling it messy

## Q1. [Junior] Here's a `BankAccount` class with a public `balance` field that any code in the codebase can read and write directly. What's wrong with this, and how would you fix it?

**Answer:**

A public mutable field means any part of the program can set the balance to an invalid value, like a negative number, with no validation and no record of why it changed. That is a direct violation of encapsulation, the idea that an object should control access to its own internal state.

**What's wrong**

- No validation: nothing stops `account.balance = -500`.
- No control over how the value changes: you cannot log, audit, or trigger side effects like sending a low balance alert.
- Tight coupling: if you ever change how balance is stored internally, every piece of code touching the field directly breaks.

**How to fix it**

1. Make the field private.
2. Expose a `deposit(amount)` and `withdraw(amount)` method instead of a raw setter.
3. Validate inputs inside those methods, reject negative deposits, reject withdrawals that overdraw.
4. Expose a read only `getBalance()` if external code needs to read the value.

**This turns the balance from a value anyone can corrupt into a value the object itself guarantees is always valid.**

**Interview Answer:**

"The public field breaks encapsulation because nothing stops external code from setting an invalid balance or bypassing business rules. I'd make the field private and only allow changes through `deposit` and `withdraw` methods that validate the amount, for example rejecting a withdrawal that would overdraw the account. I'd expose a read only getter for the balance itself. That way the class, not the caller, is responsible for keeping its own state valid at all times."

## Q2. [Junior] What's the difference between abstraction and encapsulation? Give me an example of each.

**Answer:**

**These two are often confused because they work together, but they solve different problems.**

**Encapsulation**

- Bundling data and the methods that operate on it into one unit, and restricting direct access to the internal state.
- Example: a `Car` class hides its `fuelLevel` and `engineTemperature` fields behind private access, only exposing `start()` and `drive()`.

**Abstraction**

- Hiding implementation complexity behind a simple, well defined interface, so the caller only needs to know what an object does, not how it does it.
- Example: calling `car.start()` starts the ignition sequence, fuel injection, and engine turnover, but the driver only sees one simple method call.

**How they relate**

- Encapsulation is about access control and data protection.
- Abstraction is about reducing complexity by exposing only what's necessary.
- You typically achieve abstraction through encapsulation: hiding fields and internal steps is what lets you present a simple interface.

**Interview Answer:**
"Encapsulation is about protecting an object's internal state by making fields private and only exposing controlled methods to change them. Abstraction is about hiding complexity, letting the caller work with a simple interface without knowing the implementation details underneath. A `Car` class encapsulates its fuel and engine fields as private, and it provides abstraction through a simple `start()` method that hides the multi step ignition process. In practice, encapsulation is often the mechanism you use to achieve abstraction."

---

## Q3. [Junior] I want a `Shape` base with `Circle` and `Rectangle` subclasses, each needing to compute its own area. How would polymorphism help here, and what would the code look like?

**Answer:**

Polymorphism lets you call the same method name on different object types and have each type execute its own version of that behavior, without the caller needing to know which concrete type it's dealing with.

**Design**

1. Define an abstract `Shape` class or interface with an abstract method `area()`.
2. `Circle` implements `area()` as `pi * radius * radius`.
3. `Rectangle` implements `area()` as `width * height`.
4. Client code holds a list of `Shape` references and calls `shape.area()` on each, without an if/else chain checking the type.

**Why it matters**

- Without polymorphism, you'd write something like `if (shape instanceof Circle) { ... } else if (shape instanceof Rectangle) { ... }` everywhere you compute area, and every new shape type means editing that logic in every location.
- With polymorphism, adding a new shape, say `Triangle`, means writing one new class. No existing code changes.

**This is also the foundation for the Open/Closed Principle discussed later in this chapter: polymorphism is the mechanism that lets you extend behavior without modifying existing code.**

**Interview Answer:**
"I'd make `Shape` an abstract class or interface with an abstract `area()` method, then have `Circle` and `Rectangle` each provide their own implementation. Calling code just holds a `Shape` reference and calls `area()`, and the correct version runs based on the actual object type at runtime. That avoids type checking branches scattered through the codebase, and it means adding a new shape later is just a new class, not a change to existing logic."

---

## Q4. [Junior] What's the difference between inheritance and composition? Walk me through an example of each.

**Answer:**

**Both let you reuse code and build relationships between classes, but they model different kinds of relationships.**

**Inheritance, an "is-a" relationship**

- A `Car` is a `Vehicle`. A `SavingsAccount` is an `Account`.
- The subclass inherits fields and methods from the parent and can override behavior.
- Example: `class Car extends Vehicle`.

**Composition, a "has-a" relationship**

- A `Car` has an `Engine`. A `Car` has a `GPS`.
- One class holds a reference to another and delegates work to it, instead of inheriting its behavior.
- Example: `class Car { private Engine engine; }`, and `Car.start()` calls `engine.ignite()`.

**When to use which**

- Use inheritance when the relationship is genuinely a specialization, and the subclass should always be substitutable for the parent.
- Use composition when you want to reuse behavior without committing to a rigid type hierarchy, or when the relationship might change over time.

**Interview Answer:**
"Inheritance models an is-a relationship, like a `Car` being a kind of `Vehicle`, and the subclass gets the parent's behavior automatically. Composition models a has-a relationship, like a `Car` containing an `Engine` object and delegating to it. I generally lean toward composition by default because it's more flexible, you can swap the `Engine` implementation without touching `Car`, whereas swapping out inherited behavior often means restructuring the whole class hierarchy."

---

## Q5. [Junior] When would you reach for an interface versus an abstract class?

**Answer:**

**Both let you define a contract that other classes implement, but they differ in what they can contain and how many a class can use.**

**Interface**

- Defines a contract, method signatures, with no shared implementation, in most languages, though modern languages allow default methods.
- A class can implement multiple interfaces.
- Use it when unrelated classes need to guarantee the same capability, like `Comparable` or `Serializable`.

**Abstract class**

- Can hold shared state and partial implementation alongside abstract methods that subclasses must fill in.
- A class can only extend one abstract class in most languages.
- Use it when a group of related classes share common fields or logic, and you want to avoid duplicating that logic in every subclass.

**How to decide**

- Ask: is this a capability unrelated classes can plug into, or a shared identity with common code between related classes?
- Capability, no shared code: interface.
- Shared identity, some shared code: abstract class.

**Interview Answer:**
"I use an interface when I want to define a capability that unrelated classes can implement, like making both a `File` and a `NetworkStream` `Closeable`. I use an abstract class when there's a family of related classes that share actual state or logic, not just a method signature, since it lets me put the common implementation in one place. A quick test I use is whether I need multiple inheritance of the contract, since a class can implement many interfaces but extend only one abstract class."

---

## Q6. [Junior] I've got a class where every private field has a getter and a setter that just return or assign the field directly, no logic at all. Is that good encapsulation?

**Answer:**

No, this is a common smell sometimes called an anemic class or a "getter and setter bag." **Technically the fields are private, but wrapping every field in a pass through getter and setter gives you none of the benefits of encapsulation while adding boilerplate.**

**Why it's not real encapsulation**

- The setter accepts any value with no validation, so it's functionally identical to a public field.
- The object exposes its entire internal representation, so callers still reach in and manipulate state directly instead of asking the object to do something.
- Behavior that should live inside the object, like validation or derived calculations, ends up scattered across every caller instead.

**What better encapsulation looks like**

- Only expose setters for fields that genuinely need external mutation, and validate inside them.
- Replace pure data getters with behavior, instead of `getBalance()` then subtracting elsewhere, expose a `withdraw(amount)` method that does the subtraction and validation internally.
- Ask "tell, don't ask": tell the object what to do, rather than asking for its data and doing the work outside.

**Interview Answer:**
"Wrapping every field in a getter and setter that just passes the value through isn't real encapsulation, it's a public field with extra steps. The point of encapsulation is that the object protects its own invariants, so I'd only add setters where external mutation is actually needed, and I'd put validation or business rules inside them. Where possible I'd replace data getters with real behavior methods, so instead of exposing raw balance and letting callers do math on it, I'd expose a `withdraw` method that the object controls."

---

## Q7. [Mid] Here's an `Invoice` class. It has methods to calculate the total, save the invoice to the database, and format and print a receipt. What SOLID principle is this violating, and how would you refactor it?

**Answer:**

**This violates the Single Responsibility Principle, which says a class should have only one reason to change.** This `Invoice` class has at least three: a change to tax calculation logic, a change to how invoices are persisted, and a change to receipt formatting all touch the same class.

**Why this is a problem**

- A change to the database schema now risks breaking calculation logic that has nothing to do with persistence.
- The class is hard to test in isolation, testing calculation logic means dragging in database and printing dependencies too.
- Multiple developers working on unrelated features end up editing the same file and colliding.

**Refactor**

1. Keep `Invoice` as a data holder with calculation logic that's intrinsic to what an invoice is, like `calculateTotal()`.
2. Extract `InvoiceRepository` to handle saving and loading invoices from the database.
3. Extract `ReceiptPrinter` or `InvoiceFormatter` to handle turning an invoice into a printable or displayable format.
4. Each class now has one reason to change: `Invoice` changes when the pricing model changes, `InvoiceRepository` changes when the storage mechanism changes, `InvoiceFormatter` changes when the receipt layout changes.

**Interview Answer:**
"This is a Single Responsibility Principle violation, the `Invoice` class is mixing business logic, persistence, and presentation, three unrelated reasons to change living in one place. I'd split it into the `Invoice` class itself holding calculation logic, an `InvoiceRepository` responsible for saving and loading from the database, and an `InvoiceFormatter` or `ReceiptPrinter` responsible for turning the invoice into output. Each piece can now change independently, and each is much easier to unit test without pulling in unrelated dependencies like a database connection."

## Q8. [Mid] I've got a `DiscountCalculator` with a big if/else chain checking `customerType`, regular, premium, VIP, each with different discount math. Marketing wants a new tier every quarter. What's the problem here, and how would you redesign it?

**Answer:**

**This violates the Open/Closed Principle, a class should be open for extension but closed for modification.** Every new customer tier means editing this method directly, risking breaking existing tiers and requiring a redeploy and retest of code that already worked.

**What's wrong with the if/else chain**

- Adding VIP-plus next quarter means touching code that already handles regular, premium, and VIP, increasing the blast radius of a small change.
- The method tends to grow indefinitely and becomes harder to read and test as tiers accumulate.
- It's easy to introduce a bug in an unrelated branch while adding a new one.

**Redesign**

1. Define a `DiscountStrategy` interface with a single method, `applyDiscount(order)`.
2. Create one implementation per tier: `RegularDiscount`, `PremiumDiscount`, `VipDiscount`.
3. Store the mapping from customer type to strategy in a lookup, like a map or a registry, rather than a conditional.
4. Adding a new tier means writing a new class and registering it, with zero changes to existing discount classes.

**Interview Answer:**
"The if/else chain on customer type violates Open/Closed, every new tier forces a modification to code that already works, which risks regressions. I'd introduce a `DiscountStrategy` interface with one implementation per tier, register them in a map keyed by customer type, and have the calculator just look up the right strategy and call it. Adding a new tier next quarter becomes writing one new class and registering it, without touching the existing regular, premium, or VIP logic at all."

---

## Q9. [Mid] Here's a `Rectangle` class with `setWidth` and `setHeight`, and a `Square` that extends it and overrides both setters to keep width and height equal. What breaks, and what does this tell you about inheritance?

**Answer:**

This is the classic Liskov Substitution Principle violation. **LSP says subclasses should be substitutable for their base class without breaking the correctness of the program.** `Square` looks like a valid `Rectangle` on paper, but it silently breaks any code that relies on `Rectangle`'s contract.

**Where it breaks**

- Code that does `rect.setWidth(5); rect.setHeight(10); assert(rect.area() == 50)` passes for a real `Rectangle` but fails for a `Square`, since setting height also changes width.
- Any test or logic written against the `Rectangle` contract can now produce wrong results when handed a `Square`, without a type error anywhere, the bug is silent.

- This shows that "is-a" in the English sense, a square is a rectangle mathematically, does not automatically mean "is-a" in the code sense, where behavior and invariants matter more than the label.

**How to fix it**

1. Do not model `Square` as a subclass of `Rectangle`.
2. Make both implement a common `Shape` interface with an `area()` method, and let them be independent, unrelated classes.
3. More generally, before creating an inheritance relationship, check whether every method inherited from the parent still holds its original contract in the child. If overriding a method changes its meaning or breaks a precondition or postcondition, the relationship probably shouldn't be modeled as inheritance.

**Interview Answer:**

"This breaks Liskov Substitution because `Square` changes the meaning of `setWidth` and `setHeight`, setting one also changes the other, which violates the behavioral contract that code relying on `Rectangle` expects. Any test that sets width and height independently and checks the resulting area breaks silently when given a `Square`. I'd stop modeling `Square` as a `Rectangle` subclass entirely, and instead have both implement a shared `Shape` interface as independent classes, since the real lesson is that inheritance should be based on behavioral compatibility, not just a category label."

---

## Q10. [Mid] Here's a `Worker` interface with `work()` and `eat()`. A `Robot` class implements `Worker` but has to override `eat()` with an empty body or throw an exception, since robots don't eat. What's the issue, and how do you fix the design?

**Answer:**

This is an Interface Segregation Principle violation. **ISP says clients shouldn't be forced to depend on methods they don't use.** `Worker` bundles two unrelated capabilities into one interface, forcing every implementer to support both even when one makes no sense for them.

**Why this hurts**

- `Robot.eat()` either does nothing, which is misleading and confusing to anyone reading the code, or throws an exception, which risks breaking any code that calls `eat()` polymorphically on a list of `Worker` objects.
- Any change to the `eat()` method's contract now potentially affects `Robot`, even though `Robot` doesn't logically care about eating at all.
- It signals that the interface is grouping things by convenience rather than by genuine shared capability.

**How to fix it**

1. Split `Worker` into smaller, focused interfaces: `Workable` with `work()`, and `Eatable` with `eat()`.
2. `Human` implements both `Workable` and `Eatable`.
3. `Robot` implements only `Workable`.
4. No implementer is ever forced to provide a fake or empty implementation of a method that doesn't apply to it.

**Interview Answer:**

"This is an Interface Segregation Principle violation, `Worker` forces every implementer to support both working and eating even though those are unrelated capabilities. I'd split it into two smaller interfaces, `Workable` and `Eatable`. `Human` implements both, `Robot` only implements `Workable`, and neither class is stuck faking a method that doesn't make sense for it. The general rule I follow is that an interface should represent one coherent capability, not a bundle of things that happen to apply to some implementers and not others."

---

## Q11. [Mid] Your `OrderService` directly creates a `new MySQLDatabase()` inside its constructor to save orders. What's wrong with that, and how would you restructure it?

**Answer:**

**This violates the Dependency Inversion Principle, high level modules shouldn't depend on low level modules directly, both should depend on abstractions.** `OrderService`, which contains important business logic, is now tightly coupled to a specific database technology.

**Why this is a problem**

- You cannot swap MySQL for PostgreSQL, or a mock database for testing, without editing `OrderService` itself.
- Unit testing `OrderService` means either standing up a real MySQL instance or accepting that your "unit" test is really an integration test.
- Business logic and infrastructure concerns are now entangled, a schema change or connection detail change can ripple into logic that has nothing to do with storage mechanics.

**Restructure**

1. Define an `OrderRepository` interface with methods like `save(order)` and `findById(id)`.
2. Implement `MySQLOrderRepository` that fulfills that interface using MySQL specifics.
3. `OrderService` takes an `OrderRepository` in its constructor, dependency injection, rather than constructing one itself.
4. In tests, pass in an `InMemoryOrderRepository` or a mock, no real database needed.
5. Swapping databases later means writing a new implementation of `OrderRepository`, with zero changes to `OrderService`.

**Interview Answer:**
"Constructing `MySQLDatabase` directly inside `OrderService` violates Dependency Inversion, the high level business logic is now hard wired to a specific low level implementation. I'd introduce an `OrderRepository` interface, have `MySQLOrderRepository` implement it, and inject the repository into `OrderService` through its constructor instead of having it build its own dependency. That makes `OrderService` testable with an in-memory or mock repository, and it means changing databases later is isolated to a new repository implementation rather than a change across the service layer."

---

## Q12. [Mid] When would you choose composition over inheritance in a real design? Give me a concrete example where you'd pick one over the other.

**Answer:**

The choice usually comes down to how stable and behaviorally consistent the "is-a" relationship really is, versus how much flexibility you need later.

**Options**

**Inheritance**

- Pros: less boilerplate for genuinely shared behavior, natural fit when subtypes truly extend a stable contract, useful for framework level hierarchies like exceptions.
- Cons: creates tight coupling between parent and child, changes to the parent ripple into every subclass, fixed at compile time, a class can't easily change its "kind" at runtime, deep hierarchies become fragile and hard to reason about.

**Composition**

- Pros: flexible, you can swap out the contained object's behavior at runtime, avoids fragile base class problems, each piece can be tested independently, aligns with "favor composition over inheritance" guidance from most modern design literature.
- Cons: slightly more boilerplate, you write delegation methods, can obscure the relationship if overused without clear interfaces.

**Concrete example**

- A `Car` needing different engine types, electric, gas, hybrid, over its lifetime or across product lines, should use composition: `Car` holds an `Engine` interface reference, and you inject `ElectricEngine` or `GasEngine`. Modeling this as `ElectricCar extends Car` and `GasCar extends Car` locks the type in at compile time and gets awkward the moment you need a hybrid, which is now both.

**Best Choice**

Composition, for most real world relationships involving behavior that varies or might need to change.

**Why?**

**Composition keeps classes decoupled and lets you change behavior by swapping a reference rather than restructuring a class hierarchy, which is almost always cheaper as a system evolves.**

**Interview Answer:**

"I default to composition unless the relationship is a genuinely stable specialization. For a `Car` that needs interchangeable engine types, electric, gas, hybrid, I'd have `Car` hold an `Engine` interface and inject the concrete engine, rather than creating `ElectricCar` and `GasCar` subclasses. That way swapping behavior is just swapping the injected object, and I avoid the fragile base class problem where a change to a shared parent ripples unpredictably into every subclass. I still use inheritance when the hierarchy is genuinely fixed and the contract is stable, like exception types."

---

## Q13. [Mid] Let's design a parking lot system. Walk me through how you'd identify the classes and relationships before writing any code.

**Answer:**

**The goal before coding is to extract nouns as candidate classes, verbs as candidate methods, and then figure out how those classes relate.**

**Step by step approach**

1. **Restate the requirements**: multiple levels, multiple spot sizes, multiple vehicle types, entry and exit gates, a ticketing or payment mechanism.
2. **Extract nouns as candidate classes**: `ParkingLot`, `Level`, `ParkingSpot`, `Vehicle`, `Ticket`, `Gate`, `Payment`.
3. **Extract verbs as candidate methods**: park a vehicle, find an available spot, calculate fee, issue a ticket, process payment.
4. **Identify hierarchies where real specialization exists**: `Vehicle` as an abstract base with `Car`, `Motorcycle`, `Truck` subclasses, since they share attributes like license plate but differ in size requirements.
5. **Identify composition relationships**: `ParkingLot` has many `Level` objects, each `Level` has many `ParkingSpot` objects, that's composition, spots don't exist independently of a level.
6. **Identify associations**: a `Ticket` references a `Vehicle` and a `ParkingSpot`, but doesn't own them.
7. **Assign responsibilities**: `ParkingLot` coordinates finding a spot and issuing a ticket, `ParkingSpot` just tracks its own occupied state and size, `Ticket` tracks entry time and computes duration.
8. **Sanity check with SOLID**: does any one class do too much, does the spot assignment logic need to be swappable, for example nearest spot versus size matching, which suggests a strategy interface.

**Interview Answer:**

"I start by restating requirements, then pull out the nouns as candidate classes, `ParkingLot`, `Level`, `ParkingSpot`, `Vehicle`, `Ticket`, and the verbs as candidate behaviors, like parking a vehicle or calculating a fee. I look for genuine specialization, `Car` and `Truck` extending `Vehicle`, and for containment, a `Level` owning its `ParkingSpot` objects through composition. Then I assign one clear responsibility per class, so `ParkingLot` coordinates the overall flow while `ParkingSpot` just tracks its own state, and I check whether anything like spot assignment strategy should be pulled out as an interface for future flexibility."

---

## Q14. [Mid] Explain the difference between association, aggregation, and composition, with an example of each.

**Answer:**

**These describe increasingly strong relationships between two classes, mostly about lifecycle ownership.**

**Association**

- The weakest relationship, two classes interact but neither owns the other.
- Example: a `Student` and a `Teacher`, a student takes a class from a teacher, but they exist independently and neither's lifecycle depends on the other.

**Aggregation**

- A "has-a" relationship where one object contains a reference to another, but the contained object can exist independently.
- Example: a `Department` has `Professor` objects, but if the `Department` is dissolved, the `Professor` objects still exist and can move to another department.

**Composition**

- The strongest "has-a" relationship, the contained object's lifecycle is tied to the container. If the container is destroyed, the contained object is destroyed too.
- Example: a `House` has `Room` objects. If the `House` is torn down, the rooms cease to exist as well, they have no independent existence.

**Why this distinction matters in practice**

- It tells you how to manage object creation and destruction, and whether a reference should be shared or owned.
- It affects design decisions like whether to cascade deletes in a database, or whether an object should be passed in from outside versus constructed internally.

**Interview Answer:**

"Association is the loosest relationship, two objects interact without either owning the other, like a `Student` and a `Teacher`. Aggregation is a has-a relationship where the contained object can outlive the container, like a `Professor` still existing if their `Department` is dissolved. Composition is the strongest, the contained object's lifecycle depends entirely on the container, like `Room` objects that stop existing if the `House` is torn down. I use this distinction to decide how objects should be created, owned, and cleaned up in the design."

---

## Q15. [Mid] You're looking at a class hierarchy that's five levels deep, `Animal` to `Mammal` to `Carnivore` to `Feline` to `HouseCat`. Adding a small behavior change at the top level breaks something two levels down. What's going on here, and how would you address it?

**Answer:**

**This is the fragile base class problem, a well known consequence of deep inheritance hierarchies.** Because subclasses depend on the internal behavior and assumptions of their parent, a change anywhere up the chain can ripple down and break behavior far removed from the original change, often in ways that are hard to predict just by reading the changed class.

**Why deep hierarchies get fragile**

- Each level adds implicit assumptions about how the levels above behave, and those assumptions aren't always visible from reading a single class.
- A change to `Animal` might change behavior that `Carnivore` silently relied on, but the person changing `Animal` has no way to know that without reading every subclass.
- Testing becomes harder, verifying a change is safe means retesting the entire subtree, not just the class that changed.

**How to address it**

1. Favor shallow hierarchies, generally no more than two or three levels, and prefer composition for the rest.
2. Extract behaviors that vary, like diet or movement type, into separate interfaces or strategy objects that are composed in, rather than encoded as another layer of inheritance.
3. Use the "favor composition over inheritance" guideline concretely here: `HouseCat` could compose a `DietBehavior` and a `MovementBehavior` rather than inheriting them from four levels of ancestors.
4. If you must keep the hierarchy, document and test the contract each level exposes, so changes are validated against those contracts, not just against the immediate subclass.

**Interview Answer:**

"This is the fragile base class problem, deep hierarchies mean subclasses implicitly depend on assumptions baked in several levels up, so a change at the top can break behavior far down the chain in ways that aren't obvious from reading the changed class alone. I'd flatten the hierarchy where possible, generally keeping it

to two or three levels, and pull out behaviors that vary, like diet or movement, into composed strategy objects instead of additional inheritance layers. That isolates the blast radius of a change to just the piece of behavior that actually changed."

---

## Q16. [Mid] Multiple inheritance, a class inheriting from two parent classes that both define the same method, causes the diamond problem. How do modern languages handle this?

**Answer:**

The diamond problem happens when a class inherits from two classes that both define a method with the same signature, and it becomes ambiguous which version the child should use. **Most modern object oriented languages sidestep this by disallowing multiple inheritance of implementation while still allowing multiple inheritance of type.**

**How different approaches handle it**

- Java and C#: a class can extend only one class, but can implement multiple interfaces. Java allows default methods on interfaces, and if two interfaces provide conflicting default methods, the compiler forces the implementing class to explicitly override and resolve the conflict rather than guessing.
- C++: allows true multiple inheritance of implementation, and the diamond problem is real there, solved using virtual inheritance, where the compiler ensures only one shared instance of the common base exists, but this adds real complexity.
- Python: uses the C3 linearization algorithm, the Method Resolution Order, MRO, to deterministically decide which parent's method wins when there's ambiguity, following a well defined left to right, depth first with overrides rule.

**Why this matters for design**

- It's a strong argument for favoring composition and interfaces over deep or multiple implementation inheritance, since ambiguity resolution rules are often subtle and can surprise you.
- When you do need to combine multiple behaviors, prefer composing multiple single purpose objects rather than inheriting from multiple base classes with implementation.

**Interview Answer:**
"The diamond problem comes up when a class inherits conflicting implementations of the same method from two parents. Most modern languages avoid this by restricting multiple inheritance to interfaces, which only define contracts, while allowing only single inheritance of actual implementation, that's the Java and C# approach. Languages like Python that allow it resolve ambiguity with a deterministic method resolution order. My takeaway for design is to avoid relying on multiple inheritance of implementation altogether and use composition to combine behaviors instead, since it sidesteps the ambiguity entirely."

---

## Q17. [Mid] Here's a `User` class with boolean flags, `isPremiumUser`, `isAdmin`, `isGuest`, and every method has if statements branching on these flags to decide behavior. What's the smell here, and how would you redesign it?

**Answer:**

**This is a type checking smell, using flags or booleans to simulate different types instead of actually modeling them as different types.** It's effectively hand rolled, fragile polymorphism, and it tends to violate both the Single Responsibility Principle and the Open/Closed Principle as the flags multiply.

**Why this is a problem**

- Every new user category means adding another flag and updating every method's conditional logic, which is exactly the kind of change Open/Closed is meant to prevent.
- Flag combinations that shouldn't be possible, like `isAdmin` and `isGuest` both true, can occur because nothing enforces mutual exclusivity.
- The `User` class accumulates unrelated behavior for every role, growing indefinitely and becoming hard to test in isolation.

**Redesign**

1. Model roles as actual types: define a `UserRole` interface or abstract class, with `PremiumUser`, `AdminUser`, `GuestUser` as separate implementations.
2. Move role specific behavior, like what a user can access or what discount they receive, into the corresponding role class as an overridden method.
3. `User` holds a `UserRole` reference, composition, rather than a pile of booleans, and delegates role specific decisions to it.
4. Now adding a new role means writing a new class, and it's impossible to accidentally end up with a self contradictory user, since the role is a single, well defined object.

**Interview Answer:**

"Boolean flags like `isPremiumUser` and `isAdmin` are really trying to encode different types, and branching on them everywhere is a substitute for real polymorphism that gets more fragile as roles multiply. I'd introduce a `UserRole` interface with `PremiumUser`, `AdminUser`, and `GuestUser` implementations, move the role specific logic into those classes, and have `User` hold a `UserRole` reference instead of a pile of booleans. That removes the risk of contradictory flag combinations and means adding a new role is a new class, not a change scattered across every conditional in `User`."

---

## Q18. [Senior] Classic example: a `Bird` base class has a `fly()` method. `Ostrich` and `Penguin` extend `Bird` but can't actually fly, so they override `fly()` to throw an exception. Redesign this properly.

**Answer:**

Throwing an exception from an overridden method is a strong signal of a Liskov Substitution violation combined with poor abstraction: the base class made a promise, every bird can fly, that isn't true for all birds, and the subclass breaks the contract instead of the modeling being fixed.

**Why the naive fix doesn't work**

- Making `fly()` a no-op instead of an exception is not better, it silently does nothing when called, misleading any code that expects flight to actually happen.
- Adding an `isFlightless` boolean and checking it before calling `fly()` everywhere reintroduces the type checking smell and defeats the purpose of polymorphism.

**Proper redesign**

1. Recognize the real abstraction: not all birds fly, so `fly()` doesn't belong on the base `Bird` class at all.

2. Define `Bird` with only truly universal behavior: `eat()`, `layEggs()`, `makeSound()`.
3. Define a separate `FlyingBird` abstraction, an interface or abstract subclass, with the `fly()` method.
4. `Sparrow` and `Eagle` implement or extend `FlyingBird`. `Ostrich` and `Penguin` extend `Bird` directly and never inherit a `fly()` method they can't honor.
5. If flightless birds have their own shared locomotion, like `Penguin` swimming, model that as a separate `SwimmingBird` interface, composed in just like flying.

**Why this holds up**

**Any code written against `FlyingBird` can now safely call `fly()` on every instance it holds, because the type system itself guarantees only flying birds are in that reference, restoring the substitutability that LSP requires.**

**Interview Answer:**
"Throwing from an overridden method means the base class contract was wrong in the first place, not every bird can fly, so `fly()` shouldn't live on `Bird` at all. I'd pull `fly()` out into a separate `FlyingBird` interface that only `Sparrow` and `Eagle` implement, and keep `Bird` itself limited to behavior every bird actually shares, like eating and laying eggs. That way code holding a `FlyingBird` reference can call `fly()` with total confidence, and `Ostrich` and `Penguin` never inherit a method they can't honestly support."

---

## Q19. [Senior] You inherit a codebase where several subclasses override a base class method just to throw `UnsupportedOperationException`. How do you catch this pattern in review, and what's your process for fixing it without a full rewrite?

**Answer:**

This pattern, sometimes called a refused bequest, means a subclass is inheriting a method it fundamentally cannot support, and it's one of the clearest signals of a Liskov Substitution violation. **Catching it and fixing it incrementally, rather than rewriting everything at once, is the realistic path in a live codebase.**

**How to spot it in review**

- Search for method bodies that consist entirely of a thrown exception, or a comment like "not supported for this type."
- Look for base classes with a large surface area, since the more methods a base class exposes, the more likely some subclass legitimately can't support all of them.
- Check whether calling code has defensive try/catch blocks specifically around calls to that method, that's a sign callers have already learned not to trust the base type's contract.

**Fixing it incrementally**

1. Identify the minimal actual contract, the methods every subclass can genuinely support, and keep those on the base class.
2. Extract the problematic methods into a separate interface that only the subclasses which truly support them implement.
3. Update calling code gradually to check for the narrower interface, using a pattern like optional casting or a capability check, rather than assuming every base type instance supports everything.
4. Add new subclasses against the corrected, narrower hierarchy going forward, and migrate old callers opportunistically rather than all at once.
5. Add a static analysis rule or code review checklist item flagging any override whose body is just a thrown exception, to prevent regressions.

**Interview Answer:**
"An override that just throws is a refused bequest, a subclass inheriting a method it can't honor, and it's a clear Liskov violation. In review I look for method bodies that are just an exception throw, and for defensive try/catch blocks around calls to that method elsewhere, since that tells me callers already don't trust the contract. To fix it without a full rewrite, I extract the problematic method into a narrower interface that only capable subclasses implement, migrate callers to check for that interface over time, and stop new subclasses from inheriting the bad contract going forward."

---

## Q20. [Senior] Design a notification system that needs to support email today, but SMS and push notifications are coming next quarter, and the team keeps adding new channels every few months. How do you structure this to respect the Open/Closed Principle?

**Answer:**

**The goal is to make adding a new channel purely additive, a new class, with zero changes to existing code, dispatch logic, or tests for existing channels.**

**Design**

1. Define a `NotificationChannel` interface with a single method, `send(message, recipient)`.
2. Implement `EmailNotification`, `SmsNotification`, `PushNotification` as separate classes, each handling its own delivery mechanism and formatting requirements.
3. Introduce a `NotificationService` that holds a collection of `NotificationChannel` implementations, registered at startup, for example through dependency injection or a configuration driven registry.
4. If a message needs to go out through multiple channels, `NotificationService` iterates over the registered channels and calls `send()` on each, without knowing or caring what channel it is.
5. For channel specific behavior differences, like SMS having a character limit, keep that logic inside the specific channel's class, not in shared dispatch logic.

**Handling channel specific configuration**

- Use a small factory or a dependency injection container to wire up each channel with its specific configuration, API keys, provider SDKs, so `NotificationService` itself stays entirely provider agnostic.
- If channels need to be enabled or disabled dynamically, drive that from configuration that feeds the registry, rather than conditional logic in the service.

**Why this satisfies Open/Closed**

Adding push notifications next quarter means writing `PushNotification implementing NotificationChannel` and registering it. No existing channel's code changes, and no existing tests for email or SMS need to be touched or rerun to validate the new addition.

**Interview Answer:**
"I'd define a `NotificationChannel` interface with a single `send` method, then implement `EmailNotification`, `SmsNotification`, and later `PushNotification` as separate classes behind that interface. A `NotificationService` holds a registered collection of channels and dispatches to each without knowing their concrete types. Adding a new channel next quarter is purely additive, write the new class, register it, and nothing about email or SMS needs to change or be retested. Channel specific quirks, like SMS character limits, stay encapsulated inside that channel's own class rather than leaking into shared dispatch logic."

## Q21. [Senior] Walk me through how you'd structure a layered application, controller, service, repository, so the business logic layer doesn't end up depending on database or framework specifics.

**Answer:**

**This is Dependency Inversion applied across an entire application's layers: the business logic, the high level policy, should define the interfaces it needs, and the infrastructure, the low level detail, should implement those interfaces, not the other way around.**

**Layered structure**

1. **Domain and service layer**: contains the core business logic, like `OrderService.placeOrder()`. This layer defines interfaces it depends on, like `OrderRepository` and `PaymentGateway`, but has zero knowledge of what implements them.
2. **Repository layer**: implements the persistence interfaces defined by the domain layer, for example `PostgresOrderRepository implements OrderRepository`. This is where SQL, ORMs, and connection details live.
3. **Controller layer**: handles HTTP specifics, request parsing, response formatting, and calls into the service layer. It also has no knowledge of persistence details.
4. **Composition root**: a single place, often at application startup, where concrete implementations get wired to the interfaces they satisfy, through a dependency injection container or manual construction.

**Why this matters in practice**

- The service layer can be unit tested with in-memory or mock repositories, no database needed.
- Swapping Postgres for DynamoDB later means writing a new repository implementation, the service layer code doesn't change at all.
- The direction of source code dependency is inverted from the natural call flow: the repository package depends on interfaces defined in the domain package, not the reverse, which is the literal meaning of "dependency inversion."

**Common pitfall to avoid**

Placing repository interfaces in the same package as their implementations rather than alongside the domain logic that needs them defeats the purpose, the domain layer should own the interface definitions it depends on.

**Interview Answer:**
"I'd have the service layer define the interfaces it needs, like `OrderRepository`, without knowing anything about the concrete database. The repository layer then implements those interfaces with the actual persistence technology, and a composition root at startup wires the concrete implementation to the interface. This means the service layer can be unit tested with an in-memory repository, swapping databases later only touches the repository layer, and critically, the interface definitions live with the domain logic that needs them, not with the infrastructure that implements them, which is what actually inverts the dependency direction."

## Q22. [Senior] You've published a public interface that a dozen teams implement. Now you need to add a new method to it. What are your options, and which would you pick?

**Answer:**

Adding a method to a widely implemented interface is risky because every existing implementer now fails to compile or fails at runtime until they add the new method, and you often don't control all of those implementers directly.

**Options**

**Add the method directly to the interface, breaking change**

- Pros: keeps the interface clean and minimal, no versioning complexity.
- Cons: breaks every implementer immediately, requires coordinated releases across a dozen teams, risky if any implementer is external or on a different release cadence.

**Add a default implementation, in languages that support default methods**

- Pros: existing implementers keep compiling and working without any changes, since they inherit the default behavior automatically.
- Cons: the default behavior might not be correct for every implementer, it's a stopgap, not a substitute for implementers eventually providing their own real implementation, and it only works in languages with default method support like Java or Kotlin.

**Introduce a new, extended interface instead of modifying the existing one**

- Pros: fully backward compatible, existing implementers are completely unaffected, callers who need the new capability can check for or require the extended interface.
- Cons: leads to interface proliferation over time, callers may need to handle both the old and new interface for a transition period.

**Best Choice**

Default implementation if the language supports it and a sensible default exists, otherwise a new extended interface.

**Why?**

**Backward compatibility for a dozen independently deployed implementers matters more than interface purity.** A default method gives you the new capability without forcing synchronized releases, and where no sensible default exists, a new interface avoids breaking anyone while still letting capable implementers opt in.

**Interview Answer:**

"I'd avoid adding the method directly to the existing interface since that's a breaking change for every one of the dozen implementers, and coordinating a dozen synchronized releases is a real cost. If the language supports default methods and there's a sensible default behavior, I'd add it there so existing implementers keep working unchanged while new implementers can override it. If no sensible default exists, I'd introduce a new extended interface instead, so callers needing the new capability can opt in without breaking anyone still on the original contract."

---

## Q23. [Senior] Here's a `UserManager` class. It handles authentication, writes audit logs, sends welcome emails, and persists user records to the database, all in one class with over a thousand lines. Refactor this properly, applying more than one SOLID principle.

**Answer:**

This is a god object, and it typically requires applying several SOLID principles together rather than just one, since the responsibilities are tangled with each other as well as being individually too broad.

**Step by step refactor**

1. **Apply Single Responsibility first**: split into `AuthenticationService` (verifying credentials and issuing sessions or tokens), `AuditLogger` (recording user actions), `WelcomeEmailSender` (sending onboarding emails), and `UserRepository` (persistence).
2. **Apply Dependency Inversion for each collaborator**: `AuthenticationService` shouldn't construct a concrete `UserRepository` or `AuditLogger` itself, it should depend on interfaces for both, injected in.
3. **Apply Open/Closed for extensibility**: if welcome emails might later become welcome emails plus SMS plus in-app notification, define a `WelcomeNotifier` interface rather than hardcoding email specifically, similar to the notification system design.
4. **Introduce a coordinating class carefully**: something like `UserRegistrationService` can orchestrate the flow, create the user through `UserRepository`, log it through `AuditLogger`, notify through `WelcomeNotifier`, but this coordinator should only orchestrate, not contain the actual logic of any of those steps, otherwise you've just renamed the god object.
5. **Verify with tests**: each extracted class should be independently unit testable without needing the others, if you still need to mock four collaborators to test one class's core logic, the split probably isn't fine grained enough yet.

**Result**

**`UserManager` no longer exists as a single class.** `AuthenticationService`, `AuditLogger`, `WelcomeNotifier`, `UserRepository`, and a thin `UserRegistrationService` coordinator each have one clear reason to change, and each depends on abstractions rather than concrete implementations of its collaborators.

**Interview Answer:**
"This is a god object, so I'd apply Single Responsibility first and split it into `AuthenticationService`, `AuditLogger`, `WelcomeNotifier`, and `UserRepository`, each with one clear job. Then I'd apply Dependency Inversion so each of those depends on interfaces for its collaborators rather than constructing concrete instances directly, which makes each piece independently testable. I'd add a thin `UserRegistrationService` to orchestrate the overall flow, but keep it limited to sequencing calls, not containing logic itself, otherwise it just becomes a smaller god object. The test for success is whether each piece can be unit tested without pulling in the others."

---

## Q24. [Senior] Let's design a `Money` class to represent currency amounts in a financial system. Why would you make it immutable, and what does that buy you around thread safety?

**Answer:**

**An immutable object's state cannot change after construction, every operation that would "modify" it instead returns a new instance.** For a value like money, this isn't just a style preference, it directly prevents an entire category of bugs.

**Why immutability fits `Money` specifically**

- Money represents a value, not an identity, two `Money` instances representing 10 dollars should be treated as equal and interchangeable, which is a natural fit for value objects, and value objects are almost always best made immutable.
- Arithmetic operations like `add()` or `multiply()` should return a new `Money` instance rather than mutating the original, preventing a shared reference from being silently changed underneath code that didn't expect it.

**Design**

1. All fields, `amount` and `currency`, are `final` or read only, set only in the constructor.
2. `add(Money other)` validates the currencies match, then returns `new Money(this.amount + other.amount, currency)`, it never mutates `this`.
3. No setters exist at all.
4. Validate invariants once, at construction time, for example rejecting a negative amount if that's a business rule, so every existing instance is guaranteed valid for its entire lifetime.

**Thread safety implications**

- An immutable object is automatically thread safe for reads, since no thread can ever observe a partially updated or torn state, there's nothing to synchronize because nothing ever changes after construction.
- You can freely share `Money` instances across threads, cache them, and use them as map keys without defensive copying or locking.
- This eliminates a whole class of concurrency bugs where one thread mutates a shared object while another thread is reading it mid update.

**Interview Answer:**
"I'd make `Money` immutable because it represents a value rather than an identity, so operations like `add` should return a new instance rather than mutating the original, which also prevents a shared reference from changing unexpectedly under another part of the code. The thread safety benefit is significant, an immutable object can be freely shared across threads without any locking, because there's no mutable state that could ever be read in a partially updated form. All fields are set once in the constructor, there are no setters, and invariants like non-negative amounts are validated once at creation."

---

## Q25. [Senior] You're designing a repository layer with `save`, `findById`, `findAll`, `delete`, `findByEmail`, `findByStatus`, and a dozen other query methods. Some implementations only need a handful of these. How do you balance interface segregation against not ending up with fifteen tiny interfaces?

**Answer:**

This is a genuine tension: taking Interface Segregation to its extreme produces an unmanageable number of tiny interfaces, while ignoring it produces the original fat interface problem where every implementer is forced to support methods it doesn't need.

**Options**

**One fat `Repository` interface with everything**

- Pros: simple to reference, one type for callers to depend on.

- Cons: implementers that only need basic CRUD are forced to implement or stub out a dozen query methods they don't use, exactly the ISP problem.

**One tiny interface per method**

- Pros: theoretically perfect segregation, no implementer depends on a method it doesn't use.
- Cons: impractical in real codebases, callers now need to depend on a combination of many interfaces just to get common functionality, and the sheer number of types adds cognitive overhead that outweighs the benefit.

**Segregate by cohesive capability, not by individual method**

- Group methods that are genuinely used together into a small number of interfaces: `CrudRepository` with `save`, `findById`, `delete`, and a separate `QueryableRepository` or entity specific interface like `UserQueryRepository` with `findByEmail`, `findByStatus`, only for repositories that actually need those queries.
- Pros: implementers only depend on the capability groupings they need, without an explosion of single method interfaces.
- Cons: requires judgment about where the natural groupings are, which can shift as usage patterns evolve.

**Best Choice**

Segregate by cohesive capability groupings.

**Why?**

**ISP's actual goal is that no client is forced to depend on methods it doesn't use, not that every method must live in its own interface.** Grouping by real usage patterns, basic CRUD as one interface, specialized queries as another, satisfies the spirit of the principle without generating interface sprawl that makes the codebase harder to navigate.

**Interview Answer:**

"I wouldn't go to either extreme, one giant interface forces every implementer to support methods it doesn't need, but a separate interface per method creates unmanageable sprawl. I'd group methods by cohesive capability, a `CrudRepository` with save, findById, and delete for basic persistence, and a separate interface like `UserQueryRepository` for entity specific queries like findByEmail, so implementers only depend on the capability groupings they actually use. The goal of Interface Segregation is avoiding forced dependencies on unused methods, not minimizing interface size for its own sake, and grouping by real usage achieves that without excessive interface proliferation."

---

## Q26. [Senior] How do you decide the right level of abstraction in a design, avoiding both over-engineering with speculative interfaces nobody needs yet, and under-engineering with a rigid design that can't adapt?

**Answer:**

**This is a judgment call every experienced designer has to make repeatedly, and the useful heuristics are less about a fixed rule and more about signals to watch for on both sides.**

**Signals you're over-engineering**

- You're adding an interface or an abstract base class for a concept that currently has exactly one implementation and no concrete plan for a second.

- You're introducing a strategy pattern or plugin architecture for behavior that has never actually changed and shows no sign of needing to.
- Reviewers or new team members need several extra minutes just to trace through the layers of indirection to find where actual logic lives.

**Signals you're under-engineering**

- The same conditional logic checking type or state is duplicated in multiple places and has already changed more than once.
- Adding a genuinely new, foreseeable variant, like a new payment method in a payments system, requires touching code across several unrelated classes rather than just adding one new class.
- You find yourself writing a comment like "if we ever need to support X" right before hardcoding a solution that doesn't support X.

**A practical rule of thumb**

1. Build the concrete, direct solution first for the current, known requirement.
2. Introduce an abstraction, an interface or a strategy, at the point you're implementing the second real variant, not in anticipation of a hypothetical one. This is sometimes called the "rule of two" or "wait for the second use case."
3. Exception: if the domain has an extremely well known and stable pattern of variation, like payment methods or notification channels, it's reasonable to design for extensibility from the start, since the second variant is a near certainty, not a guess.
4. Keep abstractions narrow and grounded in the actual variation observed, not in every theoretically possible variation.

**Interview Answer:**

"I look for concrete signals rather than trying to judge this in the abstract. If I'm adding an interface for something with exactly one implementation and no real second use case in sight, that's over-engineering. If the same conditional logic keeps getting duplicated and has already changed more than once, that's under-engineering. My default is to build the direct, concrete solution for the current requirement first, and introduce an abstraction when I'm implementing the second real variant, not in anticipation of a hypothetical one, unless the domain is one where variation is already a near certainty, like payment methods, in which case designing for extensibility from day one is reasonable."

---

## Q27. [Senior] How do you actually determine what counts as "one responsibility" for the Single Responsibility Principle? Walk me through it with a `Report` class that formats data, and applies business rules to decide what data to include.

**Answer:**

"One responsibility" is one of the most debated parts of SOLID because taken too literally, every method could be argued into its own class. **The more useful formulation, from Robert Martin, is "a class should have only one reason to change," meaning one axis of change, tied to one actor or stakeholder who would request that change.**

**Applying it to `Report`**

- Ask: who would request a change to the logic that decides what data goes into the report, business rules like "exclude cancelled orders"? That's likely a business analyst or product owner, reacting to policy changes.

- Ask: who would request a change to how the report is formatted, PDF layout, column ordering, styling? That's likely a design or presentation concern, possibly a different stakeholder entirely.
- If those two categories of change come from different sources and happen on different timelines, they are two separate responsibilities, even though both are, in a loose sense, "about the report."

**Practical process**

1. List the plausible reasons this class might need to change over the next year.
2. Group those reasons by who would be requesting them, and how often each is likely to happen.
3. If two groups are requested by different people for different reasons, on different schedules, split them into separate classes: `ReportDataSelector` for the business rules, `ReportFormatter` for the presentation.
4. If the groups always change together for the same reason, they may genuinely be one responsibility, and splitting them further would just add indirection without benefit.

**Why this matters more than counting methods**

A class with ten methods that all serve one clear purpose, all changing for the same reason, single responsibility. A class with two methods that get modified by two entirely different teams for two entirely different reasons, that's already violating SRP, method count is a poor proxy for actual responsibility.

**Interview Answer:**

"I don't count methods, I ask who would request a change and why. For `Report`, deciding what data qualifies for inclusion is a business rules concern that a product owner would drive, while formatting and layout is a presentation concern that might come from a completely different stakeholder on a different schedule. Since those are different reasons to change, driven by different people, I'd split them into a `ReportDataSelector` and a `ReportFormatter`. If two pieces of logic always change together for the same underlying reason, I leave them combined, since splitting purely by method count just adds indirection without actually isolating any real axis of change."

---

## Q28. [Mid] Look at this `Employee` class. Which SOLID principle does it violate, and how would you fix it?

```
class Employee {
    private String name;
    private double hoursWorked;
    private double hourlyRate;

    public Employee(String name, double hoursWorked, double hourlyRate) {
        this.name = name;
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    public double calculatePay() {
        return hoursWorked * hourlyRate;
    }

    public void printPaystub() {
        double pay = calculatePay();
        System.out.println("Paystub for " + name + ": $" + pay);
    }

    public void saveToDatabase() {
        // opens a connection and writes this employee's row
    }
}
```

**Answer:**

This violates the Single Responsibility Principle. The `Employee` class is doing three unrelated jobs: computing pay, formatting and printing a paystub, and persisting the record to a database. **Each of those is a separate reason to change, a new pay rule, a new paystub layout, and a new storage mechanism, and right now all three are tangled inside one class.**

**Why it's a problem**

- A change to how paystubs are printed risks a typo or bug that breaks pay calculation, since both live in the same file.
- Testing `calculatePay()` in isolation is harder than it should be, since the class also drags in printing and database concerns.
- Two different teams, payroll and infrastructure, might need to edit the same class for completely unrelated reasons.

**How to fix it**

1. Keep `Employee` as a plain data holder with `calculatePay()`, since that's intrinsic to what an employee's pay actually is.
2. Extract `PaystubPrinter` with a `printPaystub(employee)` method.
3. Extract `EmployeeRepository` with a `save(employee)` method.

```
class Employee {
    private String name;
    private double hoursWorked;
    private double hourlyRate;

    public Employee(String name, double hoursWorked, double hourlyRate) {
        this.name = name;
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    public double calculatePay() {
        return hoursWorked * hourlyRate;
    }

    public String getName() {
        return name;
    }
}

class PaystubPrinter {
    public void printPaystub(Employee employee) {
        double pay = employee.calculatePay();
        System.out.println("Paystub for " + employee.getName() + ": $" + pay);
    }
}

class EmployeeRepository {
    public void save(Employee employee) {
        // opens a connection and writes this employee's row
    }
}
```

**Interview Answer:**

"This is a Single Responsibility violation, `Employee` is mixing pay calculation, paystub formatting, and persistence into one class, three unrelated reasons to change. I'd keep `calculatePay` on `Employee` itself since that's core to what an employee's pay is, then pull printing out into a `PaystubPrinter` and saving out

into an `EmployeeRepository`. Now a change to the paystub layout can't accidentally break pay math, and I can unit test pay calculation without touching a database or a print statement."

---

## Q29. [Mid] Look at this `ShippingCostCalculator`. Which SOLID principle does it violate, and how would you fix it?

```
class ShippingCostCalculator {
    public double calculate(Order order, String shippingMethod) {
        switch (shippingMethod) {
            case "standard":
                return order.getWeight() * 0.5;
            case "express":
                return order.getWeight() * 1.5 + 5;
            case "overnight":
                return order.getWeight() * 3 + 15;
            default:
                throw new IllegalArgumentException("Unknown shipping method");
        }
    }
}
```

**Answer:**

This violates the Open/Closed Principle, a class should be open for extension but closed for modification. Every time the business adds a new shipping method, say "same_day", someone has to open this method and add another branch. **That means code that already works for standard, express, and overnight has to be touched and re-tested just to support one new option.**

**Why the if/elif chain hurts**

- The method grows without bound as shipping options increase, and it becomes harder to read and reason about.
- A typo or mistake in the new branch can accidentally break an existing branch above or below it.
- There's no way to plug in a shipping method defined outside this class, like one added by a partner integration, without editing this file directly.

**How to fix it**

1. Define a `ShippingStrategy` interface with a single method, `calculate(order)`.
2. Implement one class per method: `StandardShipping`, `ExpressShipping`, `OvernightShipping`.
3. Look up the right strategy from a registry instead of branching on a string.

```
interface ShippingStrategy {
    double calculate(Order order);
}

class StandardShipping implements ShippingStrategy {
    public double calculate(Order order) {
        return order.getWeight() * 0.5;
    }
}

class ExpressShipping implements ShippingStrategy {
    public double calculate(Order order) {
        return order.getWeight() * 1.5 + 5;
    }
}

class OvernightShipping implements ShippingStrategy {
    public double calculate(Order order) {
        return order.getWeight() * 3 + 15;
```

```
    }
}

class ShippingCostCalculator {
    private Map<String, ShippingStrategy> strategies;

    public ShippingCostCalculator(Map<String, ShippingStrategy> strategies) {
        this.strategies = strategies; // e.g. {"standard": new StandardShipping()}
    }

    public double calculate(Order order, String shippingMethod) {
        return strategies.get(shippingMethod).calculate(order);
    }
}
```

**Interview Answer:**

"The if/elif chain on shipping method violates Open/Closed, adding a new method means editing code that already works for the existing ones, which risks regressions. I'd introduce a `ShippingStrategy` interface with one implementation per method, standard, express, overnight, and have the calculator look up the right strategy from a registry keyed by method name instead of branching. Adding same day shipping later is just a new class and a new registry entry, with zero changes to the existing shipping classes."

---

## Q30. [Senior] Look at this `Thermostat` and its subclass. Which SOLID principle does it violate, and how would you fix it?

```
class Thermostat {
    protected int temperature = 68;

    public void setTemperature(int temperature) {
        this.temperature = temperature;
    }

    public int getTemperature() {
        return temperature;
    }
}

class SmartLockedThermostat extends Thermostat {
    private static final int MIN_ALLOWED = 65;

    @Override
    public void setTemperature(int temperature) {
        if (temperature < MIN_ALLOWED) {
            return; // silently ignore, "protecting" the tenant
        }
        this.temperature = temperature;
    }
}
```

**Answer:**

This is a Liskov Substitution Principle violation, and it's a subtler one than the usual "override throws an exception" example, because `SmartLockedThermostat` doesn't throw or crash at all, it just quietly does nothing. **Any code written against `Thermostat` reasonably assumes that after calling `setTemperature(x)`, `getTemperature()` returns `x`, and `SmartLockedThermostat` breaks that postcondition without any signal that something went wrong.**

**Where it breaks**

- `thermostat.setTemperature(60); assert thermostat.getTemperature() == 60` passes for a base `Thermostat` but silently fails for `SmartLockedThermostat`, no exception, no return value, just a value that never changed.
- A scheduling system that lowers the temperature overnight to save energy will appear to work in testing against the base class, then silently fail to actually lower the temperature once a `SmartLockedThermostat` is substituted in.
- The bug is worse than a thrown exception precisely because nothing crashes, it just produces quietly wrong behavior that's hard to trace back to this override.

**How to fix it**

1. Don't hide the rejection inside a silent override, make the contract itself account for the possibility of rejection.
2. Have `setTemperature` return whether the change was applied, or raise a specific exception, so callers can react instead of assuming success.
3. If the locking behavior is fundamentally a different concept, not just a variant of `Thermostat`, model it as composition instead, a `Thermostat` wrapped by a `TemperatureLockPolicy`, rather than inheritance.

```
class Thermostat {
    private int temperature = 68;

    public boolean setTemperature(int temperature) {
        this.temperature = temperature;
        return true;
    }

    public int getTemperature() {
        return temperature;
    }
}

class LockedTemperaturePolicy {
    private Thermostat thermostat;
    private int minAllowed;

    public LockedTemperaturePolicy(Thermostat thermostat, int minAllowed) {
        this.thermostat = thermostat;
        this.minAllowed = minAllowed;
    }

    public boolean setTemperature(int temperature) {
        if (temperature < minAllowed) {
            return false;
        }
        return thermostat.setTemperature(temperature);
    }

    public int getTemperature() {
        return thermostat.getTemperature();
    }
}
```

**Interview Answer:**

"This is a Liskov violation, and a sneaky one, `SmartLockedThermostat` doesn't throw, it just silently ignores the request, which breaks the postcondition that calling `setTemperature` and then `getTemperature` returns the value you set. Any scheduling or automation code written against the base class will look correct in testing and then quietly fail to change the temperature once this subclass is substituted in. I'd fix it by either having `setTemperature` return a boolean so callers know whether the change applied, or by not

inheriting at all and instead wrapping a `Thermostat` in a separate `LockedTemperaturePolicy` that composes the locking behavior explicitly."

---

## Q31. [Senior] Look at this `MultiFunctionDevice` interface and its implementation. Which SOLID principle does it violate, and how would you fix it?

```
interface MultiFunctionDevice {
    void printDocument(String doc);
    void scanDocument(String doc);
    void faxDocument(String doc);
}

class BasicPrinter implements MultiFunctionDevice {
    public void printDocument(String doc) {
        System.out.println("Printing " + doc);
    }

    public void scanDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot scan");
    }

    public void faxDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot fax");
    }
}
```

**Answer:**

This is an Interface Segregation Principle violation. `MultiFunctionDevice` bundles three capabilities, printing, scanning, faxing, into a single contract, but `BasicPrinter` only genuinely supports one of them. **Forcing `BasicPrinter` to implement `scanDocument` and `faxDocument` just to satisfy the interface means it has to fake support for behavior it can't actually provide.**

**Why this hurts**

- Any code that holds a `MultiFunctionDevice` reference and calls `scanDocument()` polymorphically will crash at runtime when handed a `BasicPrinter`, and the type system gave no warning.
- The interface grew by convenience, whoever needed fax support last added it to the shared interface, rather than by asking what capability every implementer genuinely has.
- Every new low end device added to the codebase inherits the same problem and has to stub out methods it can't support.

**How to fix it**

1. Split `MultiFunctionDevice` into focused interfaces: `Printable`, `Scannable`, `Faxable`.
2. `BasicPrinter` implements only `Printable`.
3. `AllInOnePrinter` implements all three, since it genuinely supports all three.

```
interface Printable {
    void printDocument(String doc);
}

interface Scannable {
    void scanDocument(String doc);
}

interface Faxable {
    void faxDocument(String doc);
}
```

```
class BasicPrinter implements Printable {
    public void printDocument(String doc) {
        System.out.println("Printing " + doc);
    }
}

class AllInOnePrinter implements Printable, Scannable, Faxable {
    public void printDocument(String doc) {
        System.out.println("Printing " + doc);
    }

    public void scanDocument(String doc) {
        System.out.println("Scanning " + doc);
    }

    public void faxDocument(String doc) {
        System.out.println("Faxing " + doc);
    }
}
```

**Interview Answer:**

"This is an Interface Segregation violation, `MultiFunctionDevice` forces every implementer to support printing, scanning, and faxing even though a basic printer only genuinely does one of those. I'd split it into `Printable`, `Scannable`, and `Faxable`, so `BasicPrinter` only implements `Printable`, and `AllInOnePrinter` implements all three because it actually supports all three. That removes the fake stubbed methods that throw at runtime, and it means code that only needs printing can depend on `Printable` alone instead of the whole bloated interface."

---

## Q32. [Senior] Look at this `InventoryManager`. Which SOLID principle does it violate, and how would you fix it?

```
class EmailAlerts {
    public static void sendLowStockEmail(String itemName, int quantity) {
        // connects to an SMTP server and sends an email
    }
}

class InventoryManager {
    private Map<String, Integer> items;

    public InventoryManager(Map<String, Integer> items) {
        this.items = items;
    }

    public void removeStock(String itemName, int quantity) {
        items.put(itemName, items.get(itemName) - quantity);
        if (items.get(itemName) < 10) {
            EmailAlerts.sendLowStockEmail(itemName, items.get(itemName));
        }
    }
}
```

**Answer:**

This is a Dependency Inversion Principle violation, and it's easy to miss because there's no `new` keyword or constructor injection anywhere to spot, it's a direct static call buried inside a business method. `InventoryManager` contains the actual business rule, reorder when stock drops below ten, but it reaches directly into a concrete, low level detail, SMTP email delivery, to act on that rule. **High level policy and low level mechanism are still tightly coupled here, just through a static method call instead of a constructor, which makes it just as hard to test or swap out.**

**Why this is a problem**

- Unit testing `removeStock()` now means either mocking a module level static method or accidentally sending real emails during a test run.
- Switching from email alerts to Slack or SMS notifications means editing `InventoryManager` itself, even though inventory logic has nothing to do with how alerts are delivered.
- Because the coupling is a static call rather than an obvious constructor dependency, it's easy for this violation to slip past code review.

**How to fix it**

1. Define a small `LowStockNotifier` interface with a `notify(itemName, quantity)` method.
2. Implement `EmailLowStockNotifier` as one concrete implementation of that interface.
3. Inject the notifier into `InventoryManager` through its constructor, rather than calling a concrete class directly from inside business logic.

```
interface LowStockNotifier {
    void notify(String itemName, int quantity);
}

class EmailLowStockNotifier implements LowStockNotifier {
    public void notify(String itemName, int quantity) {
        // connects to an SMTP server and sends an email
    }
}

class InventoryManager {
    private Map<String, Integer> items;
    private LowStockNotifier notifier;

    public InventoryManager(Map<String, Integer> items, LowStockNotifier notifier) {
        this.items = items;
        this.notifier = notifier;
    }

    public void removeStock(String itemName, int quantity) {
        items.put(itemName, items.get(itemName) - quantity);
        if (items.get(itemName) < 10) {
            notifier.notify(itemName, items.get(itemName));
        }
    }
}
```

**Interview Answer:**

"This is Dependency Inversion, just disguised as a static call instead of the usual constructor example. `InventoryManager` holds the actual reorder rule, but it reaches straight into a concrete `EmailAlerts` class to act on it, so the business logic is still hard wired to a specific delivery mechanism. I'd define a `LowStockNotifier` interface, implement `EmailLowStockNotifier` against it, and inject the notifier into `InventoryManager`'s constructor. Now I can test the reorder logic with a fake notifier, and switching to Slack or SMS alerts later only means writing a new notifier implementation, not touching inventory logic at all."

---

**If you remember nothing else from this chapter:** *most "this code is hard to change" complaints trace back to one class quietly doing three unrelated jobs.*

# Chapter 15: Design Patterns for LLD

The trap to avoid here is reaching for a pattern because you memorized its name, not because it fits the problem in front of you. These questions, several with real code to evaluate, push you to justify a pattern choice against the next closest alternative.

> **War Story:** *A codebase with fourteen different ad-hoc "if type equals X" branches for payment methods finally got replaced by a strategy pattern, and the next new payment method took an afternoon instead of a week.*

> **Avoid:** reaching for a design pattern because it sounds impressive, without explaining the specific problem it solves here
>
> **Aim For:** explaining what would go wrong without the pattern before introducing it, rather than just naming it

## Q1. [Junior] Your application needs exactly one object that manages the connection pool to the database, and every part of the codebase should share it. How would you design this?

**Answer:**

**This is the textbook case for the Singleton pattern: a class that guarantees only one instance exists for the lifetime of the application, with a single global point of access to it.**

**How to build it**

1. Make the constructor private (or otherwise inaccessible) so no other code can call `new` directly.
2. Store the one instance in a static field on the class itself.
3. Expose a static method, commonly `getInstance()`, that creates the instance the first time it's called and returns the same one on every later call.

**Why it fits here**

- A connection pool is expensive to create and holds live resources (sockets, threads), so you want exactly one, not one per caller.
- Every part of the app needs to reach the same pool, not a copy of it, otherwise you'd exhaust the database's connection limit.

**What to be careful about**

- Lazy versus eager initialization: lazy creates the instance on first use, eager creates it at class load time. Lazy is more common but needs care in multithreaded environments (covered in a later question).
- Don't let Singleton become an excuse to store unrelated global state in it later.

**Interview Answer:**

"I'd use the Singleton pattern here since a connection pool is expensive to create and must be shared, not duplicated. I'd make the constructor private, hold the instance in a static field, and expose it through a `getInstance()` method that lazily creates it on first call. The main thing I'd watch for is that this only works

cleanly if the pool itself is thread safe internally, and if I ever need to swap implementations for testing, I'd inject the instance rather than call the static accessor everywhere."

---

## Q2. [Junior] You're building a notification service that needs to send emails, SMS messages, and push notifications, and you expect more channels to be added later. How would you structure the code that creates these notification objects?

**Answer:**

**This calls for the Factory Method pattern: instead of the calling code deciding which concrete class to instantiate with a big if/else or switch on type, that decision is centralized in one place.**

**Structure**

- Define a common `Notification` interface with a `send()` method.
- Create concrete classes: `EmailNotification`, `SmsNotification`, `PushNotification`, each implementing `send()` differently.
- Create a factory, for example `NotificationFactory.create(type)`, that returns the right concrete object based on the type passed in.

**Why this helps**

- The rest of the codebase depends only on the `Notification` interface, never on concrete classes.
- Adding a new channel, say WhatsApp, means adding one new class and one new branch in the factory, without touching any code that already sends notifications.
- It keeps object creation logic out of business logic, so a checkout flow that needs to notify a customer doesn't need to know how a `PushNotification` object gets built.

**Interview Answer:**
"I'd define a `Notification` interface with a `send()` method, implement `EmailNotification`, `SmsNotification`, and `PushNotification` against it, and put the decision of which one to build behind a factory method that takes a channel type and returns the right object. Client code only ever talks to the interface. When a new channel shows up later, I add one class and one line in the factory, and nothing that already sends notifications has to change, which is the whole point of pulling creation out of business logic."

---

## Q3. [Junior] You need to construct a complex HTTP request object that can have a URL, headers, query params, a body, a timeout, and authentication, most of which are optional. What's a clean way to construct this instead of a constructor with ten parameters?

**Answer:**

**This is a Builder pattern problem.** A constructor with ten parameters, many optional, forces callers to pass `null` or default values for everything they don't need, and it's easy to mix up the order of two adjacent string or int parameters.

**How the Builder works**

1. Create a `HttpRequestBuilder` with fluent setter methods, `withUrl()`, `withHeader()`, `withBody()`, `withTimeout()`, each returning `this`.

2. Callers chain only the methods relevant to their request: `builder.withUrl(x).withHeader("Auth", token).build()`.
3. `build()` validates required fields and returns an immutable `HttpRequest` object.

**Why it's better than the alternatives**

- Telescoping constructors (multiple overloaded constructors for different combinations of parameters) grow unmanageable past three or four optional fields.
- A single constructor with everything as parameters is unreadable at the call site and error prone with same-typed adjacent parameters.
- The Builder makes intent explicit at the call site since every field is named.

**Interview Answer:**
"With that many optional fields I'd reach for a Builder rather than a giant constructor. I'd give `HttpRequestBuilder` chainable methods like `withUrl`, `withHeader`, and `withTimeout`, let the caller set only what they need, and have `build()` validate anything required, like the URL, before returning an immutable `HttpRequest`. It reads clearly at the call site because every value is named, and it avoids the bug-prone telescoping constructor problem where two string parameters can silently get swapped."

---

## Q4. [Junior] Your app's payment code expects a `processPayment(amount, currency)` method, but the new third-party payment gateway SDK exposes `submitTransaction(TransactionRequest)` with a totally different signature and object model. How do you integrate this without rewriting your existing payment code?

**Answer:**

**This is exactly what the Adapter pattern is for: it lets two incompatible interfaces work together by introducing a translation layer, without changing either side.**

**How to build the adapter**

1. Create a class, say `GatewayPaymentAdapter`, that implements the interface your existing code already expects, `processPayment(amount, currency)`.
2. Inside that method, build the `TransactionRequest` object the SDK needs from the `amount` and `currency` arguments, and call `submitTransaction()` on the SDK.
3. Translate the SDK's response or exceptions back into whatever shape your existing code expects.

**Why this is the right call**

- Your existing payment code, and anything that calls it, doesn't change at all.
- If you swap the payment gateway again later, you only replace the adapter, not the calling code.
- It isolates third party API churn behind one seam in your codebase.

**Interview Answer:**
"I'd write an Adapter class that implements the `processPayment(amount, currency)` interface my code already relies on, and internally builds a `TransactionRequest` and calls `submitTransaction()` on the new SDK, translating the response back to whatever shape my code expects. That way none of the existing call sites change, and if this gateway gets replaced again in a year, I only touch the adapter, which is the whole point of isolating third party integration churn behind one seam."

## Q5. [Junior] You're building a stock price ticker where multiple parts of the UI, a chart, a table, and an alert panel, all need to update whenever the price changes. How would you wire this so the price source doesn't need to know about every consumer?

**Answer:**

**This is the Observer pattern: a subject maintains a list of observers and notifies all of them when its state changes, without knowing what any specific observer does with that notification.**

**Structure**

1. The price source (the subject) exposes `subscribe(observer)` and `unsubscribe(observer)` methods, and keeps an internal list of subscribers.
2. Whenever the price updates, the subject loops through its subscriber list and calls `update(newPrice)` on each one.
3. The chart, table, and alert panel each implement an `Observer` interface with `update()`, and register themselves with the price source at startup.

**Why it fits**

- The price source stays completely decoupled from the UI components: it has no idea a chart or alert panel exists, it just calls `update()` on whatever is registered.
- New consumers, like a future mobile widget, can subscribe without any change to the price source.

**Interview Answer:**
"I'd model the price source as a subject that keeps a list of subscribed observers and calls `update()` on each one whenever the price changes. The chart, table, and alert panel would each implement a small `Observer` interface and register themselves at startup. This keeps the price source completely ignorant of who's listening, so adding a new UI panel later is just a matter of subscribing it, with zero changes to the code that actually publishes price updates."

---

## Q6. [Junior] You're modeling a coffee ordering system where a customer can order a plain coffee and then add any combination of milk, sugar, and whipped cream, each affecting the price. Subclassing every combination would explode into dozens of classes. What pattern solves this?

**Answer:**

**This is the Decorator pattern.** Instead of creating a subclass for every combination of add-ons, you wrap a base object in layers, each layer adding one piece of behavior or cost.

**How it works**

1. Define a `Beverage` interface with `cost()` and `description()`.
2. Implement a base `Coffee` class.
3. Implement decorators, `MilkDecorator`, `SugarDecorator`, `WhippedCreamDecorator`, each of which wraps a `Beverage` and implements the same interface, adding its own cost on top of the wrapped object's cost.
4. To build an order, wrap layers as needed: `new WhippedCreamDecorator(new MilkDecorator(new Coffee()))`.

**Why this beats subclassing**

- With three optional add-ons you'd need up to eight subclasses to cover every combination, and that grows exponentially as you add more options.
- Decorators let you compose behavior at runtime, in any combination, without creating a class for each one.
- Each decorator only knows about adding its own piece, keeping responsibilities isolated.

**Interview Answer:**

"I'd use the Decorator pattern here. A base `Coffee` class and small decorator classes for milk, sugar, and whipped cream, each implementing the same `Beverage` interface and wrapping another `Beverage` to add its own cost and description. Building an order just means wrapping the layers a customer wants. This avoids the combinatorial explosion of subclasses you'd get trying to model every possible combination directly, and it lets you add a new topping later as one new decorator class."

---

## Q7. [Mid] Two threads call getInstance() on your Singleton connection pool manager at the exact same time, before it's been created. Walk me through what could go wrong and how you'd fix it.

**Answer:**

**With a naive lazy Singleton, both threads can see the instance field as `null` at the same time, both proceed to construct a new object, and you end up with two connection pools instead of one, defeating the entire purpose of the pattern.**

**Fixes, from simplest to most efficient**

- **Synchronize the whole method**: mark `getInstance()` as `synchronized`. Correct, but every call pays a locking cost even long after the instance already exists.
- **Eager initialization**: create the instance at class load time instead of lazily. Simple and thread safe by construction, at the cost of creating it even if it's never used.
- **Double checked locking**: check if the instance is `null`, and only then enter a synchronized block and check again before creating it. The field must be declared `volatile` so partially constructed objects aren't visible to other threads due to instruction reordering.
- **Static holder / language level guarantees**: in Java, a static inner holder class is only loaded on first access and class loading is thread safe by the JVM spec, or use an `enum` singleton which Java guarantees is instantiated once. In Python, module level singletons are safe since imports are cached and effectively single threaded.

**Interview Answer:**

"Without protection, two threads can both see a null instance and each construct their own, so I end up with two pools. I'd fix it either with eager initialization if the cost of always creating it is acceptable, or with double checked locking using a volatile field if I need lazy creation, checking null once outside the lock and again inside before constructing. In Java I'd actually prefer a static holder class or enum singleton since the JVM already guarantees thread safe class loading, which is simpler than hand rolling the lock."

---

## Q8. [Mid] You're building a UI toolkit that needs to render buttons, checkboxes, and text fields consistently, but must support both a Windows look and a Mac look,

## and the two families of widgets should never be mixed on screen. How would you design the object creation for this?

**Answer:**

**This needs the Abstract Factory pattern, which produces a whole family of related objects through one factory, guaranteeing they're consistent with each other.**

**Structure**

1. Define an abstract factory interface, `GUIFactory`, with methods `createButton()`, `createCheckbox()`, `createTextField()`.
2. Implement `WindowsFactory` and `MacFactory`, each returning the widget variants for their platform.
3. The client code receives a single `GUIFactory` instance, chosen once based on the current OS, and calls its creation methods without ever knowing or caring which concrete factory it holds.

**Why plain Factory Method isn't enough**

- Factory Method abstracts creation of a single product type, for example just buttons.
- Here you have three related product types that must all match each other, a Mac button next to a Windows checkbox would look and behave inconsistently.
- Abstract Factory solves that by making "which family" a single decision made once, at factory selection time, rather than a decision repeated independently for every widget.

**Interview Answer:**
"I'd use Abstract Factory here since I have several related products, buttons, checkboxes, text fields, that need to stay consistent as a family. I'd define a `GUIFactory` interface with a creation method for each widget type, then a `WindowsFactory` and `MacFactory` implementing it. The client picks one factory instance based on the platform once, and every widget it creates afterward automatically matches. That's the key difference from Factory Method, which only handles a single product type and wouldn't stop you from accidentally mixing families."

---

## Q9. [Mid] When would you introduce a separate Director class alongside a Builder, versus just calling builder methods directly and chaining them?

**Answer:**

**Both are valid uses of the Builder pattern, the difference is where the "recipe" for construction lives.**

**Fluent chaining without a Director**

- The caller decides, at the call site, exactly which methods to call and in what combination.
- Good for ad hoc, one-off construction where every caller's needs genuinely differ, like our earlier HTTP request example.

**Director**

- Encapsulates a specific, reusable sequence of builder calls behind a named method, for example `MealDirector.buildVeganMeal(builder)` or `buildKidsMeal(builder)`.
- Useful when the same construction recipe needs to be reproduced identically in multiple places in the codebase, so you don't duplicate the sequence of builder calls at every call site.
- Keeps the "what steps make a valid vegan meal" knowledge in one place instead of scattered across callers.

**When to skip it**

- If there's only one caller, or every caller's construction is genuinely unique, a Director is unnecessary ceremony. Most modern fluent builder APIs, like a lot of Java or Kotlin builder usage, skip the Director entirely and just chain methods at the call site.

**Interview Answer:**

"I'd add a Director when the same construction sequence needs to be repeated identically from multiple places, so that recipe lives in one method instead of being copy pasted at every call site. If it's a one-off, or every caller genuinely needs a different combination of fields set, I'd just chain builder methods directly and skip the Director, since it would just be ceremony wrapping a sequence nobody else reuses."

---

## Q10. [Mid] Checking out an order in your e-commerce system involves validating inventory, calculating tax, charging a payment gateway, and creating a shipment record, each handled by a different subsystem with its own interface. How do you keep the checkout controller from becoming tangled with all four subsystems directly?

**Answer:**

**This is what the Facade pattern is for: providing one simplified interface in front of a set of subsystems, so the client doesn't need to know how many subsystems exist or how they interact.**

**Design**

1. Create a `CheckoutFacade` class with one method, `placeOrder(cart, customer)`.
2. Inside that method, the facade calls `InventoryService.reserve()`, `TaxCalculator.calculate()`, `PaymentGateway.charge()`, and `ShippingService.createShipment()` in the right order, handling any coordination between them, like rolling back an inventory reservation if payment fails.
3. The checkout controller calls `facade.placeOrder()` and doesn't need to know these four subsystems even exist.

**What the Facade does and doesn't do**

- It doesn't remove the subsystems or their complexity, it hides that complexity from callers who don't need to deal with it directly.
- Code that genuinely needs fine grained control over one subsystem, for example an admin tool adjusting inventory directly, can still bypass the facade and call `InventoryService` on its own.

**Interview Answer:**

"I'd wrap the four subsystems behind a `CheckoutFacade` with a single `placeOrder()` method that internally coordinates inventory, tax, payment, and shipping in the right order, including any rollback logic if one step fails. The checkout controller just calls that one method and stays completely unaware that four separate subsystems exist behind it. It doesn't eliminate the complexity, it just gives most callers a single simple entry point instead of forcing everyone to learn four different subsystem interfaces."

---

## Q11. [Mid] Your app displays a gallery of high resolution images, but loading all of them upfront makes the page slow. You want images to load only when they actually scroll into view, while the rest of the code still treats them like normal image objects. What pattern helps here?

**Answer:**

**This is a virtual proxy, a specific use of the Proxy pattern where a lightweight stand in object controls access to a more expensive real object, creating it only when actually needed.**

**Design**

1. Define an `Image` interface with a `display()` method.
2. `RealImage` implements it and does the expensive work, loading the full resolution file, in its constructor.
3. `ImageProxy` also implements `Image`, holds only the file path initially, and does not create a `RealImage` until `display()` is actually called for the first time. After that it caches the reference and delegates directly.

**Why the rest of the code doesn't need to change**

- Both `RealImage` and `ImageProxy` satisfy the same `Image` interface, so the gallery code just calls `display()` on whatever it's holding, unaware whether the expensive load has happened yet.

**Other proxy variants worth knowing**

- Protection proxy: checks permissions before delegating to the real object.
- Remote proxy: represents an object living in a different process or machine, hiding the network call behind a local looking interface.

**Interview Answer:**
"I'd use a virtual proxy. `ImageProxy` implements the same `Image` interface as `RealImage`, but only stores the file path at construction and defers actually loading the full resolution image until `display()` is called for the first time, caching it after that. The gallery code doesn't need to know the difference since both classes satisfy the same interface. It's the same core idea as protection proxies for access control or remote proxies for network calls, just applied to deferring expensive work instead."

---

## Q12. [Mid] You need to represent a file system where a folder can contain files or other folders, and you want to compute the total size of any folder by calling one method that works the same whether it's a single file or a deeply nested folder. How would you model this?

**Answer:**

**This is the Composite pattern, which lets you treat individual objects and compositions of objects through the same interface.**

**Structure**

1. Define a common `FileSystemComponent` interface with a `getSize()` method.
2. `File` is a leaf: it implements `getSize()` by returning its own size directly.
3. `Folder` is a composite: it holds a list of child `FileSystemComponent` objects, which can be files or other folders, and implements `getSize()` by summing the sizes of all its children recursively.

**Why this is the right shape**

- Calling code doesn't need to check "is this a file or a folder" anywhere, it just calls `getSize()` on whatever it has, and the recursion naturally handles arbitrary nesting depth.

- Adding a new component type, like a symbolic link, only means implementing the same interface, no existing code needs to branch on type.

**Interview Answer:**
"I'd use Composite here. A common `FileSystemComponent` interface with `getSize()`, `File` implementing it as a leaf that returns its own size, and `Folder` implementing it as a composite that sums `getSize()` over all its children, which can themselves be files or folders. That recursive structure means calling code never has to check whether it's holding a file or a folder, it just calls `getSize()` and the composition handles arbitrary depth on its own."

---

## Q13. [Mid] Your checkout flow needs to support multiple payment methods, credit card, PayPal, and wallet balance, and the customer picks one at checkout time. New payment methods get added every few months. How do you avoid a giant if-else chain in your checkout code?

**Answer:**

**This is the Strategy pattern: encapsulate each algorithm, here each payment method, behind a common interface, and let the client hold a reference to whichever one it needs at runtime.**

**Design**

1. Define a `PaymentStrategy` interface with a `pay(amount)` method.
2. Implement `CreditCardStrategy`, `PayPalStrategy`, `WalletStrategy`, each with its own logic for actually moving money.
3. The checkout class holds a `PaymentStrategy` reference, set based on what the customer picked, and calls `strategy.pay(amount)` without caring which concrete class it is.

**Why this beats a big if-else**

- Without Strategy, checkout code accumulates a growing `if (method == "creditcard") ... else if (method == "paypal") ...` block that has to change every time a payment method is added, violating the open/closed principle.
- With Strategy, adding "Apple Pay" next quarter means writing one new class implementing `PaymentStrategy`, checkout code doesn't change at all.
- It also makes each payment method independently testable in isolation.

**Interview Answer:**
"I'd use Strategy. Each payment method, credit card, PayPal, wallet, becomes a class implementing a common `PaymentStrategy` interface with a `pay()` method, and checkout just holds whichever strategy the customer selected and calls `pay()` on it. That keeps checkout code stable as new payment methods get added, since each one is just a new class implementing the interface rather than another branch in a growing conditional, and it also makes each payment method trivially testable on its own."

---

## Q14. [Mid] You're building a text editor that needs undo and redo for actions like typing, deleting, and formatting. How would you structure the code so any action can be undone in the order it was performed?

**Answer:**

**This is a natural fit for the Command pattern, which turns a request or action into a standalone object with its own `execute()` and `undo()` methods.**

**Design**

1. Define a `Command` interface with `execute()` and `undo()`.
2. Implement concrete commands, `TypeCommand`, `DeleteCommand`, `FormatCommand`, each holding whatever state it needs to reverse itself, for example `DeleteCommand` stores the text it removed so `undo()` can reinsert it.
3. The editor keeps a history stack. Every time a command executes, push it onto the stack. Undo pops the most recent command and calls its `undo()`. Redo needs a second stack to hold undone commands so they can be replayed with `execute()` again.

**Why this is better than tracking state manually**

- The editor doesn't need special case logic per action type to know how to reverse it, each command knows how to reverse itself.
- Commands can also be logged, queued, or composed into a macro, execute a list of commands as one unit, which a plain function call approach doesn't give you for free.

**Interview Answer:**
"I'd encapsulate every editable action as a `Command` object with `execute()` and `undo()`, where each command stores whatever data it needs to reverse itself, like the text a delete command removed. The editor keeps a history stack of executed commands and a redo stack of undone ones. Undo just pops and calls `undo()`, redo pops from the other stack and calls `execute()` again. This keeps the editor itself free of special case logic for how to reverse each action type."

---

## Q15. [Mid] An order in your system moves through states like Placed, Paid, Shipped, Delivered, and Cancelled, and the valid transitions and allowed actions differ per state. You currently have a bunch of if or switch statements checking order.status all over the codebase. What's a cleaner design?

**Answer:**

**This is a textbook case for the State pattern, which moves state specific behavior out of scattered conditionals and into dedicated classes, one per state.**

**Design**

1. Define an `OrderState` interface with methods for every action an order can receive, `pay()`, `ship()`, `cancel()`.
2. Implement one class per state, `PlacedState`, `PaidState`, `ShippedState`, and so on. Each implements only the transitions that are valid from that state, for example `PlacedState.pay()` transitions the order to `PaidState`, while `PlacedState.ship()` might throw since you can't ship before payment.
3. The `Order` object holds a reference to its current `OrderState` and delegates every action to it, swapping the reference when a transition happens.

**Why this beats scattered conditionals**

- Every place in the codebase that checked `if (status == "paid")` before doing something now just calls a method on the current state object, and the state itself decides what's legal.

- Adding a new state, like `Refunded`, means adding one new class, existing state classes don't need to change.
- Invalid transitions become a compile time or single point runtime concern instead of a bug waiting to happen anywhere `order.status` is checked.

**Interview Answer:**
"I'd replace the status checks with the State pattern. An `OrderState` interface with methods like `pay`, `ship`, and `cancel`, and one class per concrete state that only implements the transitions valid from that state. The `Order` object holds the current state and delegates every action to it, updating the reference when a transition happens. That way the logic for what's legal from a given state lives in one place per state, instead of being re-derived every time someone writes a new `if status ==` check somewhere in the codebase."

---

## Q16. [Mid] An incoming support ticket needs to be checked for spam, then checked against blocked keywords, then routed to the right team based on category, and any of these steps might reject the ticket outright. How would you structure this pipeline so new checks can be added or reordered easily?

**Answer:**

**This is the Chain of Responsibility pattern: a series of handler objects, each holding a reference to the next, where any handler can process the request, reject it, or pass it along.**

**Design**

1. Define a `TicketHandler` interface with a `handle(ticket)` method and a way to set the next handler in the chain.
2. Implement `SpamCheckHandler`, `BlockedKeywordHandler`, `RoutingHandler`, each doing its own check, and either calling `next.handle(ticket)` to continue, or stopping the chain and returning a rejection if the ticket fails its check.
3. Assemble the chain once, `spamCheck.setNext(keywordCheck); keywordCheck.setNext(routing);`, and hand the ticket to the first handler.

**Why this beats a hardcoded sequence**

- Reordering checks means changing the wiring in one place, not touching the logic of any handler.
- Adding a new check, like a profanity filter, means writing one new handler and inserting it into the chain, none of the existing handlers need to know it exists.
- Each handler stays focused on exactly one concern, which also makes each one independently testable.

**Interview Answer:**
"I'd model this as a Chain of Responsibility. Each check, spam, blocked keywords, routing, becomes its own handler implementing a common interface with a `handle()` method and a reference to the next handler. Any handler can stop the chain by not calling next if the ticket fails its check. Wiring the chain together happens in one place, so reordering or inserting a new check, like a profanity filter, doesn't require touching the logic inside any existing handler, just the assembly line that connects them."

---

## Q17. [Mid] You're generating different kinds of reports, PDF, CSV, and HTML, and all of them follow the same basic steps: fetch data, format it, add a header and

## footer, then write it out. Only the formatting step really differs between report types. How do you avoid duplicating the fetch, header, and footer logic in every report class?

**Answer:**

**This is the Template Method pattern: define the overall algorithm's skeleton once in a base class, and let subclasses override only the steps that actually vary.**

**Design**

1. Create an abstract `ReportGenerator` class with a `generate()` method that calls, in order, `fetchData()`, `formatBody()`, `addHeaderFooter()`, `writeOutput()`.
2. Implement `fetchData()`, `addHeaderFooter()`, and `writeOutput()` once, in the base class, since they're identical across report types.
3. Declare `formatBody()` as abstract, and implement it separately in `PdfReportGenerator`, `CsvReportGenerator`, and `HtmlReportGenerator`, each producing output in its own format.
4. Mark `generate()` as `final` (or otherwise non overridable) so subclasses can't accidentally change the overall sequence of steps.

**Why this is a good fit**

- The shared steps are written and tested exactly once, in the base class, instead of being copy pasted into every report subclass.
- Adding a new report format, like Excel, means implementing one method, `formatBody()`, on a new subclass.
- Optional steps can be modeled as "hooks", methods with a default no-op implementation that a subclass can override only if it needs to.

**Interview Answer:**
"I'd use Template Method. A base `ReportGenerator` class defines `generate()` as a fixed sequence, fetch data, format the body, add header and footer, write output, and implements everything except `formatBody()` directly. Each report type subclasses it and only overrides `formatBody()`. That keeps the fetch, header, and footer logic written once instead of duplicated across PDF, CSV, and HTML classes, and adding a new report format later is just one new subclass implementing one method, not a full rewrite of the pipeline."

---

## Q18. [Senior] I have a document object with different states, Draft, InReview, and Published, and its behavior changes a lot depending on which state it's in. I also have a shipping cost calculator that picks between three different algorithms based on user selection. Would you model both of these with the same pattern? Compare Strategy and State and tell me which fits where.

**Answer:**

Structurally, Strategy and State look almost identical: an object holds a reference to an interface and delegates behavior to whichever implementation it currently has. **The difference is in intent and who controls the swap.**

**Strategy**

- The algorithm is chosen externally, by the client, based on some criterion outside the object's own lifecycle, like a user picking a shipping method.
- The strategies are typically unaware of each other, and there's no notion of one strategy "transitioning" into another.
- Good fit: interchangeable algorithms selected by a caller, like the shipping cost calculator or the earlier payment method example.

**State**

- The transitions are part of the object's own lifecycle, and the current state often decides what the next valid state is, sometimes even performing the transition itself inside a handler method.
- States are aware of, and often reference, other states as valid transition targets.
- Good fit: an object like a document or an order whose allowed behavior genuinely changes as it moves through a lifecycle with defined transitions.

**Best Choice**

Shipping calculator: Strategy. Document lifecycle: State.

**Why?**

The shipping calculator has no lifecycle, an algorithm is picked once per calculation and there's no transition between algorithms. The document has an actual lifecycle where `Draft.submit()` transitions to `InReview`, and `InReview.approve()` transitions to `Published`, and those transitions and their legality are intrinsic to the object's own life story, not an external choice made by a caller each time.

**Interview Answer:**

"They're structurally similar, both are an object delegating to an interchangeable implementation behind an interface, but the intent differs. Strategy is for algorithms a caller picks externally with no notion of transitioning between them, which fits the shipping calculator. State is for an object with an actual lifecycle, where the current state can decide and even trigger the next valid state, which fits the document going from Draft to InReview to Published. I'd use Strategy for the calculator and State for the document."

---

## Q19. [Senior] I've seen teams ban Singleton outright as an anti-pattern. What's the argument against it, and if you need one shared instance of something, what would you do instead in a codebase that takes testing seriously?

**Answer:**

**The Singleton pattern itself isn't inherently broken, but the way it's typically accessed causes real problems at scale.**

**The core arguments against it**

- **Hidden dependencies**: any method can call `Logger.getInstance()` or `ConfigManager.getInstance()` internally without that dependency showing up anywhere in its signature, so you can't tell what a class actually depends on just by reading its constructor.
- **Testing friction**: because the dependency is hardcoded to a global accessor, you can't easily substitute a fake or mock version in a unit test without reaching for tricks like reflection or static mocking libraries.
- **Global mutable state**: if the Singleton holds mutable state, tests that run in the same process can leak state into each other, causing order dependent test failures.

- **Tight coupling to a concrete class**: code depends on the concrete Singleton class directly rather than an interface, making it harder to swap implementations.

**What to do instead**

- Use dependency injection: a container or composition root creates a single instance and passes it explicitly into whatever constructor needs it. The dependency is visible in the class's signature, and tests can pass in a fake implementation instead.
- The "one instance" property is now enforced by how the container is configured, not by the class itself refusing to be instantiated twice.

**When Singleton is still fine**

For genuinely stateless, side effect free utility functions, being a singleton or not doesn't really matter either way, since there's no shared state to worry about.

**Interview Answer:**
"The problem isn't having one shared instance, it's accessing it through a global static call. That hides the dependency from a class's signature, makes it hard to substitute a fake in tests, and risks state leaking between tests if it holds mutable data. In a codebase that takes testing seriously I'd use dependency injection instead: let a container create the single instance and pass it explicitly into constructors that need it. The singleton property is enforced by container configuration rather than by a private constructor, and now every dependency is visible and mockable."

---

## Q20. [Senior] You've built a pub sub notification system using the Observer pattern, and now it's used by hundreds of subscribers across services, some of which are slow or occasionally throw exceptions during notify. What problems do you now need to handle that a textbook Observer implementation ignores?

**Answer:**

The textbook Observer assumes a small number of well behaved, fast, in process observers. **None of those assumptions hold once you're at this scale.**

**Problems that show up**

- **Memory leaks**: observers that never call `unsubscribe()`, for example a UI component that gets destroyed without deregistering, stay referenced by the subject forever, preventing garbage collection. Weak references, or requiring explicit lifecycle hooks that force unsubscription, help here.
- **Blocking on slow observers**: if `notify()` calls each observer synchronously and one of them is slow, every other observer, and the publisher itself, waits behind it. Moving to asynchronous dispatch, a thread pool, or a message queue per observer avoids one slow subscriber stalling everyone.
- **One observer's exception affecting others**: a naive loop calling `observer.update()` for each subscriber will stop notifying the rest if one throws. Wrapping each call in its own try/catch, or dispatching independently, isolates failures.
- **Delivery guarantees**: at this scale you often need to reason about at least once delivery, retries, and backpressure if a downstream consumer can't keep up, which a plain in memory Observer implementation has no concept of.

**Where this actually leads**

In practice, once you hit these problems, teams often replace the in process Observer with an actual message broker, Kafka, SNS, RabbitMQ, since a single process struggles to manage retries, durability, and subscriber lifecycle at this scale. The Observer pattern is still conceptually what's happening, subject notifies subscribers, but the implementation moves out of process.

**Interview Answer:**

"At this scale the textbook Observer's assumptions break down. I'd worry about observers that never unsubscribe causing memory leaks, so I'd either use weak references or enforce explicit lifecycle cleanup. I'd make dispatch asynchronous so one slow subscriber doesn't block the publisher or other subscribers, and I'd isolate each observer's exception so one failing subscriber doesn't stop notification to the rest. Past a certain scale, I'd honestly consider moving this out of process entirely into a real message broker that already handles retries and durability."

---

## Q21. [Senior] You want to build a background job processing system where tasks like resize image or send email can be queued, retried on failure, and logged for auditing. How does the Command pattern help you here, and what has to change compared to the simple undo and redo example?

**Answer:**

**The Command pattern still applies, each job is an object encapsulating an action and the data needed to perform it, but a persistent job queue changes several assumptions from the in memory undo example.**

**What stays the same**

- Each job type, `ResizeImageCommand`, `SendEmailCommand`, implements a common interface with `execute()`.
- Decoupling the invoker, the worker pulling jobs off the queue, from the receiver, the actual image resizing or email sending logic, is exactly what Command gives you.

**What has to change**

- **Serializability**: an in memory undo command can hold live object references, but a queued job needs to survive a process restart, so it must be serializable to JSON or another format and reconstructed later, meaning a `CommandFactory` that can rebuild the right command object from serialized data by type.
- **Idempotency**: retries mean the same command might execute more than once if a worker crashes after doing the work but before acknowledging it, so `execute()` needs to be safe to run twice, for example checking if the email was already sent before sending again.
- **No live undo stack**: undo and redo assumed an in memory history of recently executed commands tied to a live session. A job queue instead needs persistent state about job status, pending, running, succeeded, failed, and retry count, tracked outside the command object itself, typically in a database.
- **Failure handling as a first class concern**: `execute()` needs to signal failure in a way the worker pool can act on, triggering a retry with backoff or moving the job to a dead letter queue after too many failures.

**Interview Answer:**

"The core idea holds, each job is a Command object with an `execute()` method, decoupling the worker pulling jobs from the logic that actually performs them. What changes is that commands now need to be serializable so they survive a restart, which means a factory that can rebuild the right command from

persisted data. I also need `execute()` to be idempotent since retries can run it more than once, and I need job status and retry count tracked persistently outside the command, rather than relying on an in memory history stack like the undo example."

---

## Q22. [Senior] You've modeled a company's org chart as a Composite tree so you can call getTotalHeadcount() at any node. In production the tree is thousands of nodes deep and that call has become a hot path. What can go wrong with a naive Composite implementation here, and how would you fix it?

**Answer:**

**A naive Composite recomputes its aggregate recursively on every single call, which gets expensive fast once the tree is large and the method is called often.**

**What goes wrong**

- **Repeated recomputation**: every call to `getTotalHeadcount()` on the root walks the entire subtree again, even if nothing changed since the last call, an O(n) cost paid repeatedly for no reason.
- **Deep recursion risk**: a tree "thousands of nodes deep" (as opposed to wide) can risk a stack overflow with a purely recursive implementation, depending on the language's stack size and whether tail call optimization applies.
- **Broken tree invariant**: Composite assumes a tree, if a bug or bad data ever introduces a cycle, for example a department accidentally listed as its own descendant, naive recursion produces infinite recursion and a crash.

**Fixes**

- **Cache the aggregate at each node**, and invalidate the cache upward from any node whose subtree changed, rather than recomputing from scratch on every read. This turns most calls into an O(1) cache hit.
- **Consider an iterative traversal** using an explicit stack or queue instead of language level recursion for very deep trees, to avoid stack overflow risk.
- **Guard against cycles** when nodes are added, refusing to attach a node as a child of one of its own descendants.
- **Combine with Visitor** (a pattern that lets you add a new operation over a tree of objects without changing the objects themselves) if you need to compute several different aggregates, headcount, budget, average tenure, in a single traversal instead of walking the tree separately for each metric.

**Interview Answer:**
"The naive version recomputes the sum recursively on every call, which is fine for small trees but becomes a real cost at thousands of nodes on a hot path. I'd cache the aggregate at each composite node and invalidate it upward whenever a subtree changes, so most reads become a cache hit instead of a full traversal. I'd also watch for stack overflow risk on very deep trees and consider an iterative traversal, and I'd guard against a bug ever introducing a cycle, since Composite assumes a tree, not a graph."

---

## Q23. [Senior] I need to build a request pipeline for an API gateway: authentication, rate limiting, logging, then the actual handler. Would you build this with Chain of Responsibility or Decorator? They look pretty similar structurally, walk me through the difference and which you'd pick.

**Answer:**

**Chain of Responsibility**

- Each handler independently decides whether to pass the request along at all, and can stop the chain entirely, for example authentication returning a 401 without ever reaching rate limiting or the handler.
- Typically only one link ultimately "resolves" the request, the rest either pass it through or reject it early.
- Handlers are usually unaware of what comes before them, only what comes next.

**Decorator**

- Every layer wraps the call and, in the classic form, always calls through to the next layer, adding behavior before or after, but the emphasis is on layering additional behavior onto every call, not on selectively terminating the flow.
- All decorators typically participate in every call, since the point is composing behavior, not gatekeeping.

**Best Choice**

**Chain of Responsibility, conceptually.**

**Why?**

An API gateway pipeline needs each stage to be able to reject a request outright, expired token means stop, don't run rate limiting or the handler at all, and that "stop here, don't proceed" behavior is the defining feature of Chain of Responsibility, not Decorator. That said, it's worth being honest that in most real web frameworks, Express, Koa, ASP.NET middleware, this is implemented as nested functions each wrapping a `next()` call, which looks structurally like Decorator, and middleware achieves the same short circuiting simply by choosing not to call `next()`. The two patterns blur together in practice at the implementation level, what matters in the interview is recognizing that the requirement, "any stage can reject and stop the pipeline", is the Chain of Responsibility intent, whichever way it ends up coded.

**Interview Answer:**
"I'd frame this as Chain of Responsibility because the defining requirement is that any stage, expired auth token or an exceeded rate limit, needs to be able to stop the pipeline outright rather than always passing through to the next stage, which is the core difference from Decorator, where every layer typically participates in every call. In practice, most frameworks implement this as nested middleware functions that look a lot like Decorator, and short circuit simply by not calling `next()`, so the two blur at the code level, but the intent here is clearly Chain of Responsibility."

---

## Q24. [Senior] You're designing a Builder for an immutable Order object that must also validate that required fields, like customer ID and at least one line item, are present before construction succeeds. How do you enforce that, and does this design have to worry about thread safety?

**Answer:**

**Enforcing required fields**

- **Fail fast in build()**: run validation inside the `build()` method itself and throw an exception if something required is missing, so an invalid `Order` can never be returned, rather than surfacing a

null pointer exception later when the object is actually used somewhere far from where it was constructed.

- **Push some requirements to compile time**: take genuinely mandatory fields, like `customerId`, as constructor arguments of the Builder itself, so you can't even create a builder without them, and reserve fluent setter methods for fields that are truly optional, like a discount code.
- **At least one line item**: this kind of cross field or collection level rule generally can't be enforced at compile time in most languages, so it belongs in `build()`'s runtime validation.

**Making the resulting object immutable**

- All fields on `Order` should be final or read only, set only in its constructor, which the Builder calls internally from `build()`.
- Mutable fields like a list of line items need a defensive copy taken during construction, so mutating the original list the caller passed in afterward doesn't silently change the built `Order`.

**Thread safety**

- The Builder instance itself is typically used by a single thread during the construction sequence, it's a short lived, local object, so it usually doesn't need synchronization.
- If a Builder instance were somehow shared and mutated across multiple threads concurrently, that would be unsafe and need external synchronization, but that's an unusual and generally avoidable usage pattern.
- **The payoff is on the other side: once `build()` produces an immutable `Order`, that object is inherently thread safe and can be freely shared and read by multiple threads without any locking, since nothing can mutate it after construction.**

**Interview Answer:**

"I'd validate required fields inside `build()` so an invalid Order can never be returned, and I'd push the genuinely mandatory fields, like customer ID, into the Builder's own constructor so you can't even start building without them, keeping the fluent setters for optional fields. The Order itself would have all final fields and a defensive copy of the line items list. Thread safety isn't really a concern for the Builder itself since it's a short lived object used by one thread, but the payoff is that the resulting immutable Order is safely shareable across threads with zero locking."

---

## Q25. [Senior] Beyond lazy loading, how would you use the Proxy pattern to add caching and access logging to an existing service class without modifying its source code, and how is this different from just wrapping it manually?

**Answer:**

**Hand written proxy**

1. Create `CachingLoggingServiceProxy` implementing the same interface as `RealService`.
2. Before delegating a call, check a cache for a previous result and return it directly if present, skipping the real service entirely, and log that the call happened.
3. If not cached, call through to the real `RealService`, log the result, store it in the cache, and return it.

That hand written wrapper class is itself an instance of the Proxy pattern, there's no difference in concept from what was described in the lazy loading example, it's just applying the same "stand in object controlling access" idea to caching and logging instead of deferred creation.

**Dynamic proxies**

- Instead of writing one proxy class per interface by hand, tools like Java's `java.lang.reflect.Proxy`, or interception frameworks like Spring AOP, generate a proxy implementation at runtime that intercepts every method call and routes it through your caching or logging logic before delegating to the real object.
- This saves writing near identical boilerplate proxy classes for every interface you want this behavior applied to, which matters once you have dozens of services needing the same cross cutting treatment.
- Trade off: a dynamically generated proxy is harder to step through in a debugger since there's no source file to look at, and reflection based dispatch carries a small performance overhead compared to a directly written class.

**Proxy versus Decorator here**

The intent still differs from Decorator even though the code can look similar: a caching proxy might not call the real object at all on a cache hit, which is about controlling and potentially avoiding access to the real object. **A Decorator's purpose is to always add behavior around a call, not to potentially skip the underlying call entirely.**

**Interview Answer:**

"I'd write a proxy class implementing the same interface as the real service, checking a cache before delegating and logging around the call, which is really just a hand written Proxy applied to caching and logging instead of lazy loading. At scale, instead of writing one proxy class per interface, I'd consider a dynamic proxy, like Java's reflection based proxies or Spring AOP, which generates that interception logic at runtime so I'm not duplicating near identical boilerplate for every service. The trade off is losing some debuggability and a small reflection overhead."

---

## Q26. [Senior] Both Factory Method and Abstract Factory are about creating objects without the client knowing the concrete class. When would picking one over the other actually matter, and can you give an example where you'd upgrade from Factory Method to Abstract Factory as requirements grow?

**Answer:**

**Factory Method**

- A single method, often overridden in a subclass, that creates one product type and lets the caller or a subclass decide which concrete class gets instantiated.
- Fits when you only need to abstract the creation of one kind of object, for example just `Button` objects that vary by platform.

**Abstract Factory**

- An interface exposing several creation methods that together produce a whole family of related products meant to be used together.
- Fits when multiple related product types must stay consistent with each other, and mixing products from different families would be a bug, not just a style choice.

**Best Choice**

Depends entirely on whether there's one product type or a family that must stay consistent, this isn't a case where one is generally better.

**Why?**

Concretely: imagine starting with a Factory Method that creates just `Button` objects for Windows or Mac. That's the right level of abstraction while buttons are the only thing that varies by platform. Later, a requirement lands that checkboxes and text fields also need platform specific look and feel, and a Mac styled button next to a Windows styled checkbox on the same screen would look visibly broken. **At that point, a single Factory Method per product type no longer guarantees the three types picked for a given screen actually match.** Upgrading to an Abstract Factory, one `GUIFactory` instance producing buttons, checkboxes, and text fields together, closes that gap, since the whole family is now selected and created through one consistent factory instance.

**Interview Answer:**

"Factory Method is right when you're abstracting the creation of a single product type. Abstract Factory earns its extra complexity once you have several related products that need to stay consistent as a family. A concrete example is starting with a Factory Method for just Button objects varying by platform, then later needing checkboxes and text fields to also vary by platform without ever getting mismatched with the button. At that point I'd upgrade to an Abstract Factory so one factory instance guarantees the whole family stays consistent, rather than each product type being decided independently."

---

## Q27. [Senior] Facade and Adapter both put a class in front of other code, and both simplify a client's life. What's actually different in their purpose, and can you give me a scenario where you'd combine them?

**Answer:**

**Adapter**

- Converts one interface into a different interface that the client already expects.
- Used because two interfaces are structurally incompatible, not because the underlying functionality is complex, often wraps just one class or interface.

**Facade**

- Provides a new, simpler interface in front of a set of subsystems that are individually usable but collectively cumbersome to coordinate.
- Used to reduce complexity and coupling for the common case, not because interfaces are incompatible, the subsystems it wraps are typically already usable on their own, just numerous.

**Best Choice / Scenario for combining them**

Take the checkout example from earlier: inventory, tax, payment, and shipping, orchestrated behind a `CheckoutFacade`. Now suppose the shipping subsystem is actually a third party carrier SDK with a foreign interface that doesn't match your internal `ShippingProvider` interface used by the rest of the checkout code.

1. Write a `CarrierSdkAdapter` implementing your internal `ShippingProvider` interface, translating calls to the third party SDK's method signatures underneath.
2. Have `CheckoutFacade` use that adapted `ShippingProvider`, alongside inventory, tax, and payment, exactly as if it were any other internal subsystem, with no awareness that adaptation is happening underneath.

**Why?**

**Adapter and Facade solve different problems, interface translation versus complexity reduction, and it's common for them to appear together: an adapter making an external, mismatched dependency look like**

**an internal one, feeding into a facade that then coordinates it alongside other, already compatible subsystems.**

**Interview Answer:**
"Adapter exists to translate between two interfaces that don't match, usually wrapping one dependency. Facade exists to simplify a client's interaction with several subsystems that are already individually usable but tedious to coordinate together. They solve different problems but combine naturally: in the checkout example, I'd wrap a third party shipping SDK in an Adapter so it matches my internal ShippingProvider interface, and then have the CheckoutFacade use that adapted provider alongside inventory, tax, and payment, without the facade ever knowing adaptation is happening underneath."

---

## Q28. [Senior] You've built a text processing pipeline using Decorator, stacking encryption, compression, and buffering around a base file writer. A teammate wraps them in a different order than you did and gets corrupted output. Walk me through why order matters here, and would you worry about thread safety if multiple decorators are shared across requests?

**Answer:**

**Why order matters**

- Each decorator transforms the data on its way through, before or after passing it to the next layer, so the order of transformations changes the actual bytes produced.
- Compressing then encrypting produces different output than encrypting then compressing, since encrypted data typically doesn't compress well and the two operations aren't commutative.
- To read the data back correctly, it has to be unwound in exactly the reverse order it was wrapped in. If the write path wraps as encrypt then compress, but the read path unwraps assuming compress then encrypt, you get exactly the kind of silent corruption described here.
- Not every decorator combination is order sensitive, stacking a logging decorator and a metrics decorator might not semantically matter which comes first, but encryption and compression very much do.

**How to prevent this class of bug**

- **Don't leave decorator assembly to whoever happens to write the call site.** Centralize construction of the correct stack behind a factory function or builder, `FileWriterFactory.createSecureCompressedWriter()`, so the order is defined once and can't be assembled incorrectly by a teammate guessing at it.
- Document the required order explicitly if a factory isn't practical for some reason.

**Thread safety**

- If a single decorator instance holds mutable internal state, for example a buffering decorator with an internal byte buffer, and that same instance is shared across concurrent requests, you have a real race condition and need either synchronization around access to that state or, more simply, a fresh decorator chain instance per request.
- Decorators that are purely stateless pass through, for example a stateless encryption call that takes a key and data and returns a result with no internal mutable fields, are safe to share across threads since there's no shared mutable state to corrupt.

**Interview Answer:**

"Order matters because each decorator transforms the bytes on the way through, so compressing then encrypting produces genuinely different output than the reverse, and reading it back requires unwinding in the exact opposite order it was wrapped. I'd fix the root cause by not leaving decorator assembly to individual call sites, instead centralizing it behind a factory method that always builds the stack in the correct order. On thread safety, it depends on whether a decorator holds mutable state, like an internal buffer, in which case sharing one instance across requests needs synchronization or a fresh instance per request, while stateless decorators are safe to share as is."

---

## Q29. [Mid] You have this code. Every call re-reads and re-parses the config file from disk, and there's no guarantee two callers ever end up holding the same settings if the file changes mid-run. What pattern would you use, and how would you restructure it?

```
class ConfigLoader {
    Map<String, Object> settings;

    public ConfigLoader() {
        this.settings = readConfigFile();
    }

    private Map<String, Object> readConfigFile() {
        String content = Files.readString(Paths.get("app_config.yaml"));
        return parseYaml(content);
    }
}

Map<String, Object> getSettings() {
    ConfigLoader loader = new ConfigLoader();
    return loader.settings;
}

Map<String, Object> priceSettings = getSettings();
Map<String, Object> discountSettings = getSettings();
```

**Answer:**

**This is a Singleton problem: the app needs exactly one ConfigLoader instance that reads the file once and hands the same cached settings to every caller afterward.**

### Why the naive version is broken

- Every call to `getSettings()` constructs a brand new `ConfigLoader`, so the file gets opened and parsed again each time, which is wasted I/O for data that never changes during the process's lifetime.
- Two callers invoked back to back have no guarantee they're looking at the same values, if the file happens to change on disk between the two reads, `priceSettings` and `discountSettings` could silently disagree with each other.

### Refactor to Singleton

```
class ConfigLoader {
    private static ConfigLoader instance;
    private final Map<String, Object> settings;

    private ConfigLoader() {
        this.settings = readConfigFile();
    }

    public static ConfigLoader getInstance() {
```

```
        if (instance == null) {
            instance = new ConfigLoader();
        }
        return instance;
    }

    private Map<String, Object> readConfigFile() {
        String content = Files.readString(Paths.get("app_config.yaml"));
        return parseYaml(content);
    }

    public Map<String, Object> getSettings() {
        return settings;
    }
}

Map<String, Object> priceSettings = ConfigLoader.getInstance().getSettings();
Map<String, Object> discountSettings = ConfigLoader.getInstance().getSettings();
```

**Why this is enough here**

- The private constructor combined with the static `getInstance()` method guarantees the file is read exactly once, the first time `getInstance()` is called, and every later call returns the same object holding the already loaded settings.
- If this could be constructed concurrently by multiple threads at startup, I'd add the double checked locking treatment covered earlier in this chapter rather than relying on this simple form.

**Interview Answer:**

"This is Singleton. Every call to getSettings creates a brand new ConfigLoader, so the file gets parsed repeatedly, and there's no guarantee two callers ever see the same instance if it changes mid-run. I'd make the constructor private and add a static getInstance() method that holds the one instance in a static field, reads and parses the file only the first time it's constructed, and returns that cached instance and its settings on every subsequent call. If this could be called concurrently from multiple threads at startup, I'd add locking around the first creation to avoid a race."

---

## Q30. [Mid] You have this code, and it's copy-pasted with small variations into the API layer, the admin CLI, and a nightly batch job. One of those copies is missing the "xml" branch entirely, which nobody noticed until a customer's export silently failed. What pattern would you introduce, and how?

```
void handleExportRequest(String fileFormat, List<Record> records) {
    Exporter exporter;
    if (fileFormat.equals("csv")) {
        exporter = new CsvExporter();
    } else if (fileFormat.equals("json")) {
        exporter = new JsonExporter();
    } else {
        throw new IllegalArgumentException("Unsupported format: " + fileFormat);
    }
    exporter.export(records);
}
```

**Answer:**

**This calls for the Factory Method pattern, the real bug here isn't the if/elif chain itself, it's that the same branching logic has been duplicated in three places and allowed to drift out of sync.**

**Structure**

- Define a single `ExporterFactory` that owns the mapping from a format string to a concrete exporter class.
- Give it one `create(fileFormat)` method that every call site uses instead of writing its own if/else if.
- Concrete exporters, `CsvExporter`, `JsonExporter`, `XmlExporter`, still implement a common `export()` method, so the factory just needs to know which class to instantiate.

**Refactored code**

```
class ExporterFactory {
    private static final Map<String, Supplier<Exporter>> EXPORTERS = Map.of(
        "csv", CsvExporter::new,
        "json", JsonExporter::new,
        "xml", XmlExporter::new
    );

    public static Exporter create(String fileFormat) {
        Supplier<Exporter> supplier = EXPORTERS.get(fileFormat);
        if (supplier == null) {
            throw new IllegalArgumentException("Unsupported format: " + fileFormat);
        }
        return supplier.get();
    }
}

void handleExportRequest(String fileFormat, List<Record> records) {
    Exporter exporter = ExporterFactory.create(fileFormat);
    exporter.export(records);
}
```

**Why this fixes the actual problem**

- The API layer, CLI, and batch job all call `ExporterFactory.create()` instead of maintaining their own copy of the mapping, so there's exactly one place that can drift.
- Adding a new format, say YAML, means adding one map entry in the factory, not hunting down and updating three separate call sites.

**Interview Answer:**

"This is Factory Method. The real problem isn't the if/elif itself, it's that the same branching logic is duplicated across the API layer, the CLI, and the batch job, and each copy can drift out of sync, which is exactly how the XML branch went missing in one of them. I'd pull the mapping from format string to exporter class into a single ExporterFactory with a create() method, have all three call sites use it, and adding a new format later becomes one entry in the factory's mapping instead of three separate edits that might not all happen."

---

## Q31. [Mid] You have this code. Marketing now wants a fourth thing to happen on every stock change, logging to a data warehouse, and every new requirement so far has meant opening this class and adding one more direct call. What pattern would decouple this, and how?

```
class InventoryItem {
    private final String sku;
    private int quantity;

    public InventoryItem(String sku) {
        this.sku = sku;
        this.quantity = 0;
    }
```

```
    public void setQuantity(int newQuantity) {
        this.quantity = newQuantity;
        if (newQuantity < 10) {
            new EmailService().sendLowStockAlert(sku);
            new SmsService().notifyWarehouse(sku);
        }
        new AnalyticsService().recordStockChange(sku, newQuantity);
    }
}
```

**Answer:**

**This is the Observer pattern: InventoryItem should just announce that its quantity changed, without knowing or caring which services react to that fact.**

**Structure**

- Turn `InventoryItem` into a subject that keeps a list of subscribed observers, with a `subscribe(observer)` method.
- Give observers a common interface, `onStockChanged(sku, quantity)`, and move the email, SMS, and analytics logic each into its own observer class.
- `setQuantity()` just updates state and loops over its observers calling `onStockChanged()`, it no longer references any concrete service.

**Refactored code**

```
interface StockObserver {
    void onStockChanged(String sku, int quantity);
}

class InventoryItem {
    private final String sku;
    private int quantity;
    private final List<StockObserver> observers = new ArrayList<>();

    public InventoryItem(String sku) {
        this.sku = sku;
        this.quantity = 0;
    }

    public void subscribe(StockObserver observer) {
        observers.add(observer);
    }

    public void setQuantity(int newQuantity) {
        this.quantity = newQuantity;
        for (StockObserver observer : observers) {
            observer.onStockChanged(sku, newQuantity);
        }
    }
}

class LowStockAlertObserver implements StockObserver {
    public void onStockChanged(String sku, int quantity) {
        if (quantity < 10) {
            new EmailService().sendLowStockAlert(sku);
            new SmsService().notifyWarehouse(sku);
        }
    }
}

class AnalyticsObserver implements StockObserver {
    public void onStockChanged(String sku, int quantity) {
        new AnalyticsService().recordStockChange(sku, quantity);
    }
```

```
}

InventoryItem item = new InventoryItem("SKU-123");
item.subscribe(new LowStockAlertObserver());
item.subscribe(new AnalyticsObserver());
```

**Why this solves the actual pain**

- Adding the data warehouse logging requirement is now one new `WarehouseLogObserver` class plus a `subscribe()` call, `InventoryItem` doesn't change at all.
- `InventoryItem` becomes trivially testable, since a test can subscribe a fake observer and assert it received the right notification, instead of mocking three concrete services.

**Interview Answer:**

"This is Observer. InventoryItem is directly wired to email, SMS, and analytics services, so every new requirement, like a data warehouse log, means opening this class again and adding one more line, and it also makes InventoryItem hard to test in isolation. I'd turn InventoryItem into a subject that keeps a list of observers and calls a single onStockChanged() on each one, then move the alerting and analytics logic into their own observer classes that subscribe at startup. Adding the warehouse logging need is just one new observer class and a subscribe call, with zero changes to InventoryItem itself."

---

## Q32. [Senior] You have this code, called from five different places in the codebase. Marketing now wants seasonal promotions that can override or stack with these tier discounts. What pattern would let you change discount logic without touching those five call sites, and how?

```
double calculateDiscount(String customerTier, double orderTotal) {
    if (customerTier.equals("gold")) {
        return orderTotal * 0.20;
    } else if (customerTier.equals("silver")) {
        return orderTotal * 0.10;
    } else if (customerTier.equals("bronze")) {
        return orderTotal * 0.05;
    } else {
        return 0;
    }
}
```

**Answer:**

**This is Strategy: each discount rule should be its own interchangeable object behind a common interface, so the five call sites depend on the interface, not on a growing tier check.**

**Structure**

- Define a `DiscountStrategy` interface with a `calculate(orderTotal)` method.
- Implement one class per rule, `GoldDiscount`, `SilverDiscount`, `HolidayPromoDiscount`, each free to encode whatever logic it needs, a flat percentage, a stacked bonus, a cap.
- Give a `PricingEngine` a `discountStrategy` reference, set based on whatever rule currently applies to an order, and have it call `strategy.calculate()` instead of branching on tier itself.

**Refactored code**

```
interface DiscountStrategy {
    double calculate(double orderTotal);
}

class GoldDiscount implements DiscountStrategy {
```

```
    public double calculate(double orderTotal) {
        return orderTotal * 0.20;
    }
}

class SilverDiscount implements DiscountStrategy {
    public double calculate(double orderTotal) {
        return orderTotal * 0.10;
    }
}

class HolidayPromoDiscount implements DiscountStrategy {
    public double calculate(double orderTotal) {
        return orderTotal * 0.25;
    }
}

class PricingEngine {
    private final DiscountStrategy discountStrategy;

    public PricingEngine(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public double totalAfterDiscount(double orderTotal) {
        return orderTotal - discountStrategy.calculate(orderTotal);
    }
}
```

**Why this solves the real problem**

- The five call sites now just build or receive a `PricingEngine` with the right strategy and call `totalAfterDiscount()`, they stop needing to know how any given discount is computed.
- Launching a holiday promotion, or letting it stack with a tier discount, becomes a matter of composing or swapping strategy objects, not editing a shared function that five other things depend on.

**Interview Answer:**

"This is Strategy. The tier check works fine until marketing wants promotions that stack or override it, and by then the function is called from five places that would all need to change together. I'd extract a DiscountStrategy interface with a calculate() method, implement one class per discount rule, gold, silver, holiday promo, and have PricingEngine hold whichever strategy applies to a given order rather than branching on tier internally. The five call sites stop caring how the discount is computed, they just ask the engine for a total, and a new promotion becomes one new class."

---

## Q33. [Senior] You have this code. Every time a new cross-cutting requirement shows up, compression, encryption, base64 encoding, someone adds another constructor flag and another if-block to write(). What pattern would let you compose these behaviors without growing this class every time, and how?

```
class FileWriter {
    private final String path;
    private final boolean compress;
    private final boolean encrypt;
    private final boolean encodeBase64;

    public FileWriter(String path, boolean compress, boolean encrypt, boolean
encodeBase64) {
        this.path = path;
        this.compress = compress;
```

```
        this.encrypt = encrypt;
        this.encodeBase64 = encodeBase64;
    }

    public void write(byte[] data) throws IOException {
        if (compress) {
            data = Gzip.compress(data);
        }
        if (encrypt) {
            data = encryptBytes(data);
        }
        if (encodeBase64) {
            data = Base64.getEncoder().encode(data);
        }
        try (FileOutputStream f = new FileOutputStream(path)) {
            f.write(data);
        }
    }
}
```

**Answer:**

**This is the Decorator pattern: each optional behavior should be its own object wrapping another writer, instead of another boolean flag branching inside one increasingly bloated class.**

**Structure**

- Define a common `Writer` interface with a `write(data)` method.
- Implement a base `RawFileWriter` that just writes bytes to disk.
- Implement one decorator per behavior, `CompressingWriter`, `EncryptingWriter`, `Base64EncodingWriter`, each wrapping another `Writer` and transforming the data before passing it along.
- Assemble the stack by nesting decorators around the base writer, in whatever order the behavior actually requires.

**Refactored code**

```
interface Writer {
    void write(byte[] data) throws IOException;
}

class RawFileWriter implements Writer {
    private final String path;

    public RawFileWriter(String path) {
        this.path = path;
    }

    public void write(byte[] data) throws IOException {
        try (FileOutputStream f = new FileOutputStream(path)) {
            f.write(data);
        }
    }
}

class CompressingWriter implements Writer {
    private final Writer wrapped;

    public CompressingWriter(Writer wrapped) {
        this.wrapped = wrapped;
    }

    public void write(byte[] data) throws IOException {
        wrapped.write(Gzip.compress(data));
```

```
    }
}

class EncryptingWriter implements Writer {
    private final Writer wrapped;

    public EncryptingWriter(Writer wrapped) {
        this.wrapped = wrapped;
    }

    public void write(byte[] data) throws IOException {
        wrapped.write(encryptBytes(data));
    }
}

Writer writer = new EncryptingWriter(new CompressingWriter(new
RawFileWriter("out.bin")));
writer.write(payload);
```

**Why this beats the flag-based version**

- `FileWriter` never needs to change again to support a new behavior, a new requirement is a new decorator class, not another constructor parameter and another branch.
- Combinations that used to require testing every flag permutation inside one method now come from composing small, independently testable decorators.

**Interview Answer:**

"This is Decorator. Every new cross-cutting need, compression, encryption, base64 encoding, has been turned into another boolean flag and another if-block inside FileWriter, which means the class keeps growing and every combination has to be tested inside one method. I'd define a common Writer interface and pull each behavior into its own decorator, CompressingWriter, EncryptingWriter, each wrapping another Writer and calling through to it. Composing behavior becomes wrapping objects in whatever order is needed at the call site, and adding a new behavior later is one new decorator class, not another flag on FileWriter."

---

## Q34. [Mid] Your vendor just replaced the analytics SDK your app has always used with the one below, and every one of your existing analytics.track() calls across the codebase, dozens of them, assumes a completely different shape. Rewriting all of them isn't realistic before the deprecation deadline. What pattern lets the existing calls keep working unchanged?

```
class LegacyAnalyticsSDK {
    void sendEvent(String evtType, String payloadJsonString, long timestamp) {
        // ...
    }
}

// Scattered across the app:
analytics.track("checkout_completed", Map.of("amount", 49.99, "currency", "USD"));
analytics.track("signup_completed", Map.of("plan", "pro"));
```

**Answer:**

**This is the Adapter pattern: it lets the new SDK's incompatible method signature sit behind the exact interface your existing calls already expect, with no rewrite of those call sites.**

**Structure**

- Create an `AnalyticsAdapter` class that exposes the same `track(eventName, properties)` method every call site already calls.
- Inside `track()`, serialize the `properties` map into the JSON string the new SDK wants, compute a timestamp, and call `sendEvent()` on the wrapped SDK.
- Construct one `AnalyticsAdapter` wrapping a `LegacyAnalyticsSDK` instance, and swap it in wherever `analytics` was previously the old client.

**Refactored code**

```
class AnalyticsAdapter {
    private final LegacyAnalyticsSDK sdk;

    public AnalyticsAdapter(LegacyAnalyticsSDK sdk) {
        this.sdk = sdk;
    }

    public void track(String eventName, Map<String, Object> properties) {
        String payloadJsonString = new Gson().toJson(properties);
        long timestamp = System.currentTimeMillis();
        sdk.sendEvent(eventName, payloadJsonString, timestamp);
    }
}

AnalyticsAdapter analytics = new AnalyticsAdapter(new LegacyAnalyticsSDK());
analytics.track("checkout_completed", Map.of("amount", 49.99, "currency", "USD"));
```

**Why this is the right call**

- Every existing `analytics.track(...)` call site keeps working exactly as written, since the adapter presents the same interface they were already coded against.
- If the vendor changes the SDK again next year, only the adapter's internals change, not the dozens of call sites that depend on the stable `track()` interface.

**Interview Answer:**

"This is Adapter. Every analytics.track() call across the app already assumes an (eventName, properties map) shape, and rewriting all of them before the deprecation deadline isn't realistic just because the vendor swapped in an SDK with a different method name and a different argument shape. I'd write an AnalyticsAdapter that exposes the same track(eventName, properties) interface the codebase already calls, and internally serializes the map to JSON, computes the timestamp, and calls sendEvent() on the new SDK. Every existing call site keeps working unchanged, and if the SDK changes again, only the adapter needs to be touched."

---

**If you remember nothing else from this chapter:** *a design pattern used because it solves a real problem is elegant; the same pattern used because it sounds senior is over-engineering*.

# Chapter 16: Putting It Together: HLD Case Study Walkthroughs

These eight full walkthroughs are useful when you want to rehearse an entire forty-five-minute design round end to end, from requirements through the hardest trade-off, not just practice a single isolated question.

## Design a URL Shortener

**Interviewer Pacing:** budget roughly forty-five minutes end to end, five to ten on requirements and estimation, ten to fifteen on the core architecture, fifteen to twenty on the hardest trade-off, and keep five in reserve for a curveball edge case. A junior pass gets the boxes right; a senior pass defends the one decision that actually mattered once you push on it.

---

## Q1. [Junior–Mid] Design a URL shortener like bit.ly or TinyURL, from clarifying requirements through handling a viral traffic spike.

**Answer:**

This is the canonical opening case study for a reason: it looks trivial ("just store a mapping in a table"), but it forces you to touch every layer of system design, requirements gathering, capacity math, ID generation, caching, sharding, and hot-key handling, without needing any domain-specific knowledge. Jumping straight to "I'll have a `urls` table with `id`, `long_url`, `short_code`" without asking a single question is the biggest red flag you can raise here; at a mid level and above, you're expected to reason cleanly about scale and defend at least two alternatives at each key decision point.

#### 1. Clarify Requirements

Before touching a whiteboard, ask the interviewer (playing product manager) questions like: Do we need custom aliases? Do short URLs expire? Do we need click analytics? Is this a global product or single-region? Do we need to support editing/deleting a mapping after creation? Is uniqueness of the short code the only constraint, or do we also need to prevent someone from shortening the same long URL twice (dedup)? Skipping this step and diving straight into schema design is the single most common mistake, it signals that you are designing in a vacuum rather than negotiating scope.

*Functional Requirements*

- Given a long URL, generate a short URL
- Given a short URL, redirect to the original long URL
- Optional custom aliases (e.g. `bit.ly/my-brand`)
- Optional expiration date per link
- Basic click analytics (count, timestamp, referrer)
- REST APIs for creation and redirection

*Non-Functional Requirements*

- Redirect latency under 50ms (this is on the critical path of someone's click)
- High availability - a redirect service going down breaks every link ever shared
- Horizontally scalable to billions of URLs
- No two long URLs ever resolve to a colliding short code

- Read-heavy system - far more people click links than create them

**2. Capacity Estimation**

Assuming the interviewer gives us:

- 100 million new URLs created per month
- A 100:1 read-to-write ratio (100 redirects for every 1 URL created)

*Write throughput*

100,000,000 URLs / month ÷ (30 days × 24 hours × 3,600 seconds) = 100,000,000 / 2,592,000 seconds ≈ **38.6 writes/sec average**.

Traffic isn't flat - assume a 3x peak-to-average multiplier for daytime/regional bursts → **~115 writes/sec at peak**. This is a very low number; write throughput is never the bottleneck in this system.

*Read throughput*

With a 100:1 read/write ratio, monthly redirects = 100,000,000 × 100 = **10 billion redirects/month**.

Average read QPS = 10,000,000,000 / 2,592,000 ≈ **3,858 reads/sec**, and at the same 3x peak multiplier, **~11,600 reads/sec at peak**. **This confirms the system must be architected around cheap, fast reads - writes are almost a rounding error by comparison.**

*Storage*

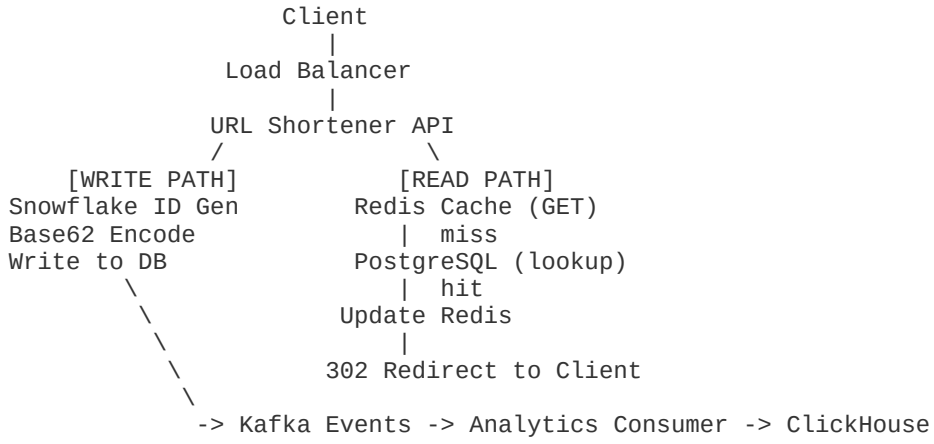
Each URL record needs: the long URL (often under a few hundred bytes, but budget up to ~500 bytes to be safe), a 7-character short code, and metadata (`created_at`, optional `expires_at`, `user_id`, `click_count`). Budget **~500 bytes/record** all-in.

- Storage/month: 100M × 500 bytes = **50 GB/month**
- Storage/year: 50 GB × 12 = **~600 GB/year**
- Over 5 years: ~3 TB of raw data; with a 3x replication factor for durability, budget **~9 TB** of actual disk footprint.

None of these numbers are large by modern infrastructure standards - the real engineering challenge in this system isn't storage volume, it's **read latency at massive fan-out**, which is why nearly every design decision below optimizes for reads.

**3. High-Level Architecture**

A write (shorten) and a read (redirect) take very different paths through the system:




**Write flow:** client submits a long URL → API generates a unique ID → encodes it to a short code → persists the mapping in the database → returns the short URL to the client. A click event is *not* generated here.

**Read flow:** client hits `bit.ly/{code}` → load balancer routes to an API node → node checks Redis first → on a cache hit, issue a 301/302 redirect immediately (sub-millisecond); on a miss, fall back to PostgreSQL, populate Redis, then redirect. Every redirect also asynchronously emits a click event onto Kafka for the analytics pipeline, so the hot path is never blocked by analytics writes.

**4. Database Choice**

*SQL* - Pros: ACID transactions, strong consistency, unique constraints (guarantees no duplicate short codes), mature tooling. Cons: harder to scale horizontally past a point.

*NoSQL* - Pros: horizontal scaling, high write throughput, easy sharding. Cons: eventual consistency, more operational complexity, weaker uniqueness guarantees.

**Best Choice:** PostgreSQL initially, Cassandra at very large scale.

**Why?** The URL mapping is a simple key-value lookup, but correctness (no colliding short codes) matters more than raw write throughput - and we already established writes are a non-issue at ~115 QPS peak. PostgreSQL gives us a unique constraint on `short_code` essentially for free, plus simple backups and operational maturity. Only once the dataset grows into the billions of rows with multi-region traffic does it make sense to migrate the mapping table to Cassandra, which trades some consistency for horizontal write scaling and multi-region replication.

**Interview Answer:** "I'd start with PostgreSQL because it gives strong consistency and a unique constraint on the short code essentially for free, and our write volume - even at peak - is only about 100 QPS, so a single well-provisioned instance handles it comfortably. If we later need multi-region writes at billions of rows, I'd migrate the mapping table to Cassandra, which is purpose-built for that kind of horizontal scale."

**5. Primary Key / Short Code Generation**

This is the core design decision in the whole problem, and the interviewer is explicitly listening for at least two approaches compared against each other.

*Approach A - Random string + collision check:* generate a random 7-character Base62 string, check the database for a collision, retry if one exists. Simple to reason about, but as the table grows toward billions of rows, collision probability rises and every write potentially costs multiple round trips to the database to retry.

*Approach B - Auto-increment counter, then Base62-encode it:* a single global counter guarantees uniqueness by construction (no collision checking needed at all), but a single counter is a single point of contention and doesn't scale across multiple database instances/regions.

*Approach C - UUID:* globally unique with zero coordination, but a 128-bit UUID is far too long once encoded - defeats the purpose of a *short* URL - and has poor index locality (random inserts fragment B-tree indexes).

*Approach D - Snowflake-style distributed ID:* each ID encodes a timestamp + machine/worker ID + local sequence number, so every application server can mint globally unique, time-ordered IDs independently, with no database round trip and no coordination between nodes.

**Best Choice:** Snowflake IDs, encoded with Base62.

**Why?** It combines the best of both worlds - Approach B's guarantee of uniqueness by construction (so we never need collision retries like Approach A) with Approach D's horizontal scalability (unlike Approach B's single counter). Every API server generates IDs locally and independently, which removes the database as a bottleneck entirely and keeps write latency flat as we add more app servers.

**Interview Answer:** "I'd compare a random-string-plus-collision-check approach against a Snowflake-style generator. Random strings are simple but the collision-retry cost grows as the table fills up, and a plain auto-increment counter doesn't scale across multiple write nodes. I'd go with Snowflake IDs - each server embeds

a timestamp, a machine ID, and a local sequence number, so IDs are globally unique and time-ordered with zero coordination and zero database round-trips, which keeps write latency flat as we scale out."

**6. Base62 Encoding**

A raw Snowflake ID is a large number like `9,223,372,036,854,775,807` - not something anyone wants in a URL. We convert it into a compact string using the 62-character alphabet `0-9A-Za-z`, producing something like `Ab9XzP`.

**Best Choice:** Base62 encoding.

**Why?** It's URL-safe (no special characters that need escaping), fully reversible (we can decode the short code straight back to the numeric ID if we ever want to skip the DB lookup entirely), compact (a 64-bit ID encodes to about 11 characters, and a 38-bit counter fits comfortably in 7), and it's the same scheme every major URL shortener actually uses.

**Interview Answer:** "Once I have the numeric Snowflake ID, I Base62-encode it to get a short, URL-safe string. It's reversible, compact, and avoids any characters that would need URL-escaping."

**7. Read Optimization**

Given the 100:1 read/write ratio and ~11.6K peak read QPS we calculated earlier, hitting PostgreSQL on every single redirect would be wasteful and slow. The fix is a cache-aside pattern in front of the database: check Redis first; on a miss, fall back to PostgreSQL, populate Redis with the result, and return it.

**Best Choice:** Redis, cache-aside.

**Why?** Sub-millisecond lookups keep us well inside the 50ms latency budget, hot links (which dominate real-world traffic - a small fraction of URLs receive the vast majority of clicks) stay resident in memory, and the database is shielded from the ~11.6K QPS peak, only ever seeing cache-miss traffic for long-tail, rarely-clicked links.

**Interview Answer:** "Since redirects outnumber writes 100 to 1, I'd put Redis in front of PostgreSQL using cache-aside. Popular links stay hot in memory and get served in sub-millisecond time, and the database only has to handle the much smaller trickle of cache misses for less popular links."

**8. Database Sharding**

Once a single PostgreSQL instance can no longer hold the dataset or absorb the query volume, we need to partition it.

*Range-based* - e.g. shard by ID ranges. Simple, but creates hotspots since new writes (and their associated reads, while a link is fresh and popular) all land on the newest shard.

*Hash-based* - `hash(short_code) % N`. Distributes both storage and traffic evenly across shards, but makes range scans and resharding slightly more involved.

*Directory-based* - a lookup service maps each key to its shard. Very flexible (easy to rebalance), but adds an extra network hop and a new single point of failure to every request.

**Best Choice:** Hash-based sharding on `short_code`.

**Why?** It gives uniform distribution of both data and query load with no additional lookup hop, which matters a lot on the read path where we're trying to stay under 50ms. The main cost - resharding being harder than with a directory - is an acceptable trade given how rarely we'd need to add shards compared to how often we serve reads.

**Interview Answer:** "Once one database can't keep up, I'd shard by hashing the short code. It spreads both storage and query load evenly across shards without adding an extra lookup hop on the critical read path, which a directory-based approach would."

**9. Handling a Viral Spike (edge case)**

Now the interviewer pushes on seniority: one long URL suddenly gets 50,000 redirect requests per second - five times our entire system's previously estimated peak. What breaks first?

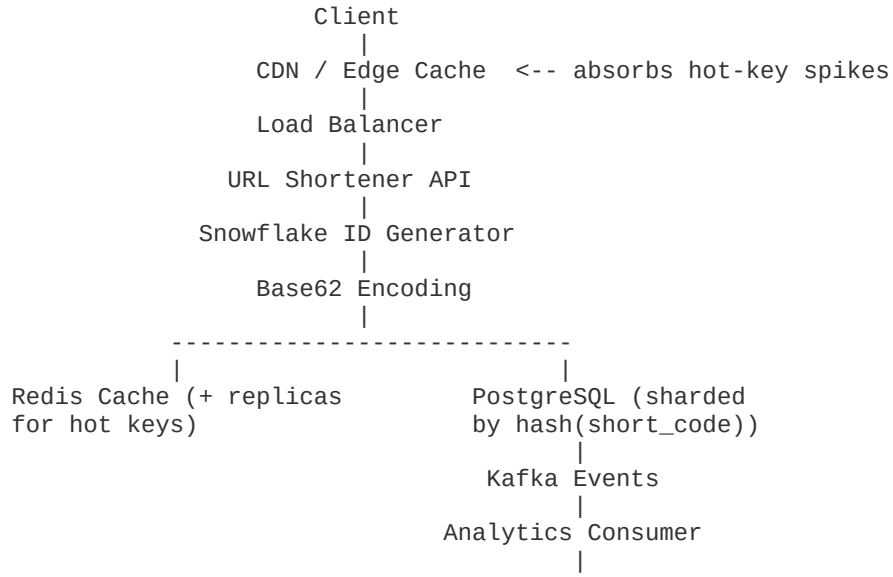
The first thing to break is **a single Redis node or a single database shard becoming a hot key**. Hash-based sharding evenly distributes *different* keys across shards, but it does nothing to help when *one* key alone is receiving 50K req/sec - that traffic is still pinned to one shard and, if evicted, would hammer one database partition on every miss.

Mitigations, roughly in the order they'd actually kick in:

- **CDN/edge caching:** the redirect response itself (a 301/302 with a `Location` header) can be cached at the edge with a short TTL (a few seconds is enough). Because the mapping essentially never changes once created, this is a safe trade of a few seconds of staleness for eliminating the vast majority of requests before they ever reach our infrastructure.
- **Local in-process cache on each API server:** as a second line of defense, each app server can hold an extremely short-lived (1-5 second) in-memory cache of the hottest keys, so even if the CDN layer is bypassed, repeated requests for the same hot key don't all hit Redis.
- **Redis read replicas for the specific hot key:** if the key is still getting through to Redis at damaging volume, replicate reads for that key (or the whole hot shard) across several replicas so no single Redis node saturates its network or CPU.
- **The database is largely protected by construction:** because Redis (plus edge caching) absorbs the overwhelming majority of traffic for a hot key, PostgreSQL/Cassandra rarely sees this spike directly - the real fire to fight is at the cache and edge layers, not the database.

**Interview Answer:** "A viral spike is really a hot-key problem, not a capacity problem - hashing already spreads different keys evenly, but it can't split traffic for a *single* key across shards. Since the mapping for that link never changes, I'd let edge/CDN caching with a short TTL absorb almost all of the 50K requests per second before they even reach us, back that with per-node in-process caches as a second buffer, and add Redis read replicas for that hot key so no single cache node or database shard becomes the bottleneck."

**10. Final Architecture**



```
ClickHouse
```

**11. Follow-up Questions**

*How do you generate unique short codes without a database bottleneck?*

Best Answer: I use a Snowflake-style ID generator - each server independently mints a globally unique, time-ordered ID from a timestamp, machine ID, and sequence number, then Base62-encodes it. No coordination or collision-checking is needed, unlike a random-string approach.

*Why Redis specifically, and why cache-aside rather than write-through?*

Best Answer: Redirects outnumber writes 100 to 1, so the read path is what needs to be fast. Cache-aside only populates the cache on demand (on a read miss), which keeps the cache full of actually-requested links rather than pre-warming it with links nobody clicks - a better fit than write-through here since most newly created links are never clicked at all.

*What actually breaks during a viral traffic spike, and why doesn't sharding alone fix it?*

Best Answer: Hash-based sharding distributes different keys evenly, but a single URL going viral is still one key, pinned to one shard/cache node - so a 50K req/sec spike on one link is a hot-key problem, not a distribution problem. The fix is edge/CDN caching with a short TTL plus Redis replicas for that specific key, not more shards.

*How do you support billions of URLs end to end?*

Best Answer: Snowflake IDs for distributed, coordination-free ID generation; Base62 encoding for compact, URL-safe codes; Redis cache-aside for sub-millisecond reads; hash-based sharding once a single database can't hold the dataset; Kafka to decouple analytics writes from the redirect hot path; and a migration from PostgreSQL to Cassandra for the mapping store once we outgrow a single-region relational database.

## Design a Rate Limiter

---

## Q1. [Junior–Senior] Design a rate limiter service that can be used to protect multiple APIs from abuse and overload.

**Answer:**

**1. Clarify Requirements**

*Functional Requirements*

- Limit the number of requests a client can make in a given time window
- Support multiple limiting dimensions: per user, per IP address, per API key, or a combination
- Return a clear signal when the limit is exceeded (HTTP 429 Too Many Requests) plus metadata (e.g. `Retry-After`, remaining quota)
- Support different limit tiers/policies per API or per customer plan (e.g. free tier: 100 req/min, paid tier: 10,000 req/min)
- Usable as a shared service in front of many independent APIs/microservices, not baked into each one individually

*Non-Functional Requirements*

- Very low added latency (the limiter sits on the critical path of every request, so it must add low single-digit milliseconds at most)
- High availability - if the limiter goes down, it should fail open or degrade gracefully rather than take down all APIs behind it

- Horizontally scalable to handle very high request volume
- Accurate enough to prevent abuse, but does not need to be perfectly exact (approximate counting is an acceptable trade-off for speed)
- Low memory footprint per tracked entity, since there can be millions of unique users/IPs/keys

**2. Capacity Estimation**

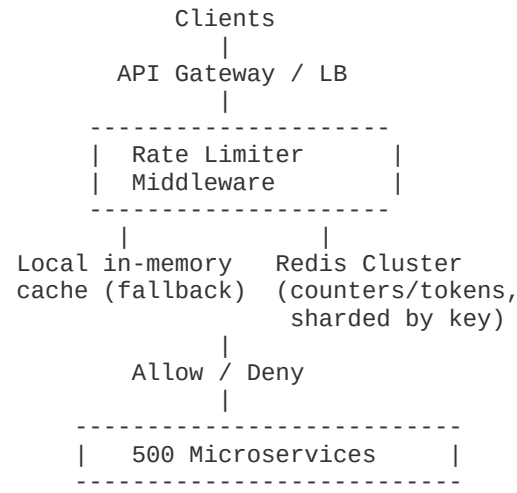
Assume the interviewer gives:

- 500 microservices sit behind this shared rate limiter
- 1,000,000 requests/second combined across all of them (average)
- Peak traffic runs ~3x average during bursts → ~3,000,000 requests/second at peak

What that implies for the rate limiter itself:

- **Throughput:** every single request needs a check-and-increment operation, so the limiter's backing store must sustain ~1M ops/sec sustained, ~3M ops/sec at peak. A single Redis node realistically tops out around 100K–200K ops/sec for this kind of workload, so the store must be **sharded across roughly 15–30 nodes** (via Redis Cluster or similar) to have headroom.
- **Latency budget:** since the check happens synchronously in the request path, it must add well under the microservices' own latency budgets - target **p99 < 5 ms**, ideally < 1 ms, which rules out any design that does synchronous cross-region calls per request.
- **Unique keys tracked:** assume ~50M active users + ~500 API keys + a long tail of IP addresses ≈ **~100M unique counters** at any time.
- **Memory:** each counter (key + count + timestamp/bucket metadata) is roughly 50–100 bytes → 100M × 100 bytes ≈ **10 GB** of hot data. This comfortably fits in memory across a modest Redis cluster, which rules out anything that requires storing a full log of every request (see algorithm comparison below - this number is what kills sliding window log at this scale).
- **Failure impact:** at 1M req/sec, even a few seconds of limiter unavailability means millions of unprotected requests - the design must **fail open with local fallback** rather than block traffic entirely if the store is unreachable.

**3. High-Level Architecture**




**4. Where the Rate Limiter Lives in the Request Path**

*Options*

- **Embedded in each microservice** - simplest to reason about locally, but means reimplementing and re-tuning limiting logic 500 times, with no shared view of a client's overall usage across services.
- **Centralized rate-limiting service called synchronously per request** - one shared source of truth, but every request pays a network round trip to that service.
- **Middleware/sidecar at the API Gateway edge, backed by a shared, sharded store** - the check happens once, at the front door, before a request is allowed to consume any downstream compute.

**Best Choice:** A rate-limiting middleware layer embedded in (or as a sidecar to) the API Gateway/load balancer, backed by a shared, horizontally-sharded Redis cluster, with an atomic Lua script doing the check-and-increment in a single round trip.

**Why? Placing the check at the edge means abusive or over-quota traffic is rejected before it ever reaches any of the 500 microservices, saving downstream compute entirely.** Doing the check-and-increment as a single atomic Redis Lua script avoids the classic race condition of separate GET then INCR calls under high concurrency, and keeps the operation to one network hop instead of two. A local in-memory cache in the gateway process holds a short-lived, slightly-stale copy of each key's remaining quota so that the common case (client well under its limit) can be answered without even hitting Redis, cutting real traffic to the store by an order of magnitude.

**Interview Answer:** "I'd place the rate limiter as middleware in the API Gateway, in front of all 500 services, so blocked requests never consume downstream compute. The check-and-increment against the counter store needs to be a single atomic operation - I'd use a Redis Lua script - to avoid race conditions and keep it to one round trip. On top of that, I'd add a short-TTL local cache in the gateway itself so that the vast majority of requests, which are nowhere near their limit, can be approved without even talking to Redis."

**5. Rate-Limiting Algorithm Comparison**

*Options*

- **Token Bucket** - each key has a bucket that holds up to N tokens and refills at a fixed rate; a request is allowed if a token is available and consumes one. Memory cost is tiny (just token count + last-refill timestamp per key). Naturally allows short bursts up to the bucket size, which matches how real client traffic behaves.
- **Sliding Window Log** - stores the timestamp of every individual request for a key (e.g. in a Redis sorted set), and counts entries within the trailing window to decide whether to allow the next one. Perfectly accurate, but memory grows with request volume per key - at this scale (100M keys, some doing hundreds of requests/window) this could balloon into tens of GB and requires expensive trimming of old entries on every check.
- **Sliding Window Counter** - keeps just two fixed-window counters per key (current window and previous window) and computes a weighted estimate of the sliding window count using the fraction of the current window elapsed. Much cheaper than the log (2 counters vs. a full timestamp list) and smooths out the "burst at window boundary" problem that plain fixed windows have, at the cost of being an approximation rather than exact.

**Best Choice:** Token Bucket as the default algorithm, with Sliding Window Counter as the alternative for APIs that need smoother, burst-free enforcement (e.g. billing-sensitive quotas).

**Why?** Token bucket is the cheapest option memory-wise (critical given the ~100M unique keys estimated above), is trivial to implement atomically in a single Redis Lua script, and its ability to absorb short bursts matches real client behavior - a mobile app catching up after being offline, or a batch job firing a handful of requests at once, shouldn't be punished the way a hard fixed window would punish it. Sliding window log is ruled out primarily on memory grounds at this scale. Sliding window counter is a good secondary choice when burst tolerance is undesirable, such as for a hard billing quota where allowing bursts could mean under-charging or violating a contractual cap.

**Interview Answer:** "I'd default to token bucket - it's memory-cheap, easy to make atomic with a Lua script, and its natural burst tolerance matches how real clients actually send traffic. I'd rule out sliding window log at this scale because storing every request timestamp for ~100 million keys is far too much memory and CPU for trimming. If a specific API needed stricter, burst-free enforcement - like a hard billing quota - I'd switch that one to a sliding window counter instead, since it approximates a true sliding window with just two counters per key."

**6. Enforcing Limits Without the Limiter Becoming a Bottleneck**

*Options*

- **Synchronous read-then-write against a single database row per key** - simple, but two round trips per check plus lock contention on hot keys (e.g. a popular API key) makes this fall over quickly at 1M+ checks/sec.
- **Atomic increment via a single Lua script in Redis** - one round trip, no race condition, and Redis's single-threaded execution model makes the script itself atomic without external locking.
- **Local, in-process approximate counting with periodic sync to the shared store** - each gateway instance keeps a local token bucket and only reconciles with Redis every N milliseconds or every N requests, trading a small amount of accuracy for a large reduction in store traffic.

**Best Choice:** Atomic Redis Lua script as the source of truth, combined with local approximate counting in front of it for the common case.

**Why?** The Lua script approach turns "read count, check, increment" into one atomic server-side operation, eliminating the classic race condition where two concurrent requests both read the same count before either increments it. Layering a local cache in front of Redis means that for a client sitting comfortably under its limit, the gateway can approve the request from memory and only asynchronously true-up the count in Redis, which is what actually lets the system absorb 1M–3M checks/sec without every single one hitting the shared cluster.

**Interview Answer:** "The core enforcement primitive is a Redis Lua script that does the read, check, and increment atomically in one round trip - that removes race conditions without needing external locks. To keep the shared Redis cluster from being hammered by every single request, each gateway node also keeps a small local cache of recent counts and only needs to consult Redis precisely, or resync periodically, when a client is getting close to its limit. That combination is what lets the design scale to millions of checks per second."

**7. What Gets Limited, and What Happens When the Limit Is Hit**

*Options*

- **Limit by IP address only** - works without authentication, but is unreliable behind NAT/shared IPs and easy to spoof or rotate.
- **Limit by authenticated user ID or API key** - much more accurate since it ties directly to an identity and a specific quota/plan.
- **Limit by a combination (e.g. API key AND IP, or API key AND endpoint)** - most flexible, supports different policies per API, and can catch abuse patterns a single dimension would miss (e.g. one compromised API key suddenly hitting from thousands of IPs).

**Best Choice:** Multi-dimensional limiting keyed primarily by API key or authenticated user ID, with IP-based limiting as a secondary, coarser layer for unauthenticated or pre-auth endpoints (like login).

**Why?** API key/user-based limiting maps directly to business policy - a free-tier key gets 100 req/min, a paid key gets 10,000 req/min - and can't be trivially bypassed the way an IP can (by rotating through a proxy pool). IP-based limiting stays valuable as a first line of defense on endpoints that don't require auth, such as login or signup, where there's no identity yet to key on. When any dimension's limit is exceeded, the service

returns **HTTP 429 Too Many Requests** with a `Retry-After` header (and ideally `X-RateLimit-Remaining` / `X-RateLimit-Reset` headers) so well-behaved clients can back off correctly instead of hammering the API in a retry loop.

**Interview Answer:** "I'd key the primary limit off the authenticated identity - user ID or API key - since that's what maps to an actual pricing tier or abuse policy and can't be evaded by rotating IPs. For unauthenticated endpoints like login, I'd add a secondary, coarser IP-based limit as a first line of defense. Either way, when a limit is hit, I return a 429 with a `Retry-After` header so clients know exactly when to try again instead of hammering us in a tight retry loop."

**8. Enforcing a Global Limit Across Multiple Data Centers (edge case)**

The hardest version of this problem: the rate limiter is now deployed across **5 data centers**, each with its own local Redis cluster, and a single user's requests can land in any of them depending on which region they're routed to. How do you enforce one global limit without paying a huge cross-region latency cost on every request?

*Options*

- **Synchronous cross-DC check on every request** (e.g. always ask a single "home" Redis cluster for that user) - perfectly accurate, but adds 50–150ms of cross-region latency to every single request, which blows the < 5ms latency budget from the capacity estimate.
- **Fully independent local limits** - split the global limit evenly across data centers (global_limit / 5) and enforce purely locally, no cross-DC communication at all. Fast, but wrong if a user's traffic is skewed toward one or two regions - they'd get throttled early in a "hot" DC while budget sits unused elsewhere.
- **Local enforcement + asynchronous global aggregation** - each DC enforces against a local budget in real time (fast path, no cross-region call), while a background process periodically (every 1–2 seconds) shares each DC's actual usage with the others, or with a lightweight global aggregator, so local budgets can be rebalanced. This accepts a small, bounded amount of overshoot during the sync window in exchange for near-local latency.

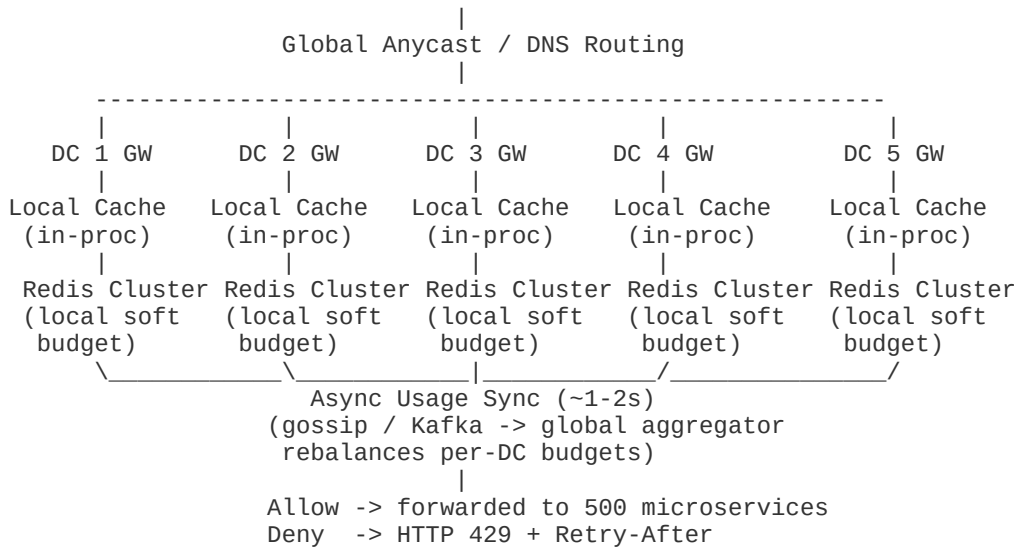
**Best Choice:** Local-first enforcement against a per-DC soft budget, with asynchronous cross-DC/global aggregation to true-up and rebalance those budgets - not a synchronous global check.

**Why?** A hard synchronous global check would mean every request pays cross-region round-trip latency, which is a non-starter at this scale and directly contradicts the sub-5ms budget established earlier. By letting each DC's Redis cluster enforce its own slice of the limit instantly, and only reconciling actual usage across DCs on a short interval (e.g. via a lightweight gossip protocol between the 5 clusters, or Kafka-based usage events feeding a global aggregator that periodically pushes updated per-DC allowances), the system stays fast on the hot path. The trade-off is that a user could briefly exceed the true global limit by a bounded amount - at most roughly one sync interval's worth of extra requests - which is an acceptable trade for real-world rate limiters (this is effectively what Cloudflare and Stripe do in practice).

**Interview Answer:** "I wouldn't make every request pay a cross-datacenter round trip just to check a global counter - at 1M+ requests/second that latency cost is far too high. Instead, I'd give each data center a local soft budget, enforced purely against its own Redis cluster so the fast path stays local. In the background, the five data centers asynchronously share their actual usage - every second or two - so the aggregate view can rebalance each DC's budget if one region is getting more of a given user's traffic than the others. That means a user could very briefly exceed the exact global limit by a small, bounded amount during the sync window, but in exchange we keep per-request latency low, which for a rate limiter - whose whole job is to not become the bottleneck it's trying to prevent - is the right trade-off."

**9. Final Architecture**

```
Clients
```




**10. Follow-up Questions**

*What happens to a request when the rate limit is exceeded?*

Best Answer: The gateway returns HTTP 429 Too Many Requests immediately, without forwarding the request downstream, along with a `Retry-After` header and remaining-quota headers so well-behaved clients back off correctly instead of retrying in a tight loop.

*Why not just use a fixed window counter - it's the simplest option?*

Best Answer: A fixed window (e.g. "max 100 requests per clock-minute") lets a client send 100 requests in the last second of one window and another 100 in the first second of the next, i.e. 200 requests in ~2 seconds against a "100/min" limit. Sliding window counter fixes this boundary-burst problem for roughly the same memory cost, which is why it's preferred over plain fixed windows whenever burst-at-the-edge matters.

*What if the Redis cluster the rate limiter depends on goes down?*

Best Answer: Fail open with a locally cached, slightly conservative limit rather than blocking all traffic - a rate limiter that itself causes a full outage is worse than temporarily under-enforcing. Each gateway node keeps enough local state (from its recent cache) to keep applying an approximate limit for a short period until the store recovers.

*How would you support different rate limit tiers (free vs. paid customers) in this design?*

Best Answer: Store the policy (limit + window + algorithm) as configuration keyed by API key/user ID rather than hardcoding it - the gateway middleware looks up the applicable policy for the identity on each request before running the token bucket or sliding window counter check, so tiers can be changed without redeploying the limiter itself.

## Design a Parking Lot System

*Note: this prompt is intentionally more LLD than HLD - lean into class/object design, state modeling, and concurrency at the object level. It's a useful palate-cleanser before the harder distributed-systems case studies.*

---

## Q1. [Junior–Senior] Design a parking lot system for a multi-level garage, from core class design through to scaling it into a real-time, multi-city chain.

**Answer:**

### 1. Clarify Requirements

*Functional Requirements*

- Support multiple vehicle types: motorcycle, compact car, regular car, bus/truck
- Support multiple spot types: motorcycle, compact, large, handicapped-accessible, electric-vehicle (with charger)
- A vehicle entering the garage is directed to (or reserved) an available spot matching its size class
- Track occupancy per level and per garage in real time
- Issue a ticket on entry, compute a fee on exit, and support payment
- Support multiple pricing schemes: flat-rate, hourly/tiered, dynamic/peak pricing
- Display availability per level (and eventually per garage) on entry-gate boards and a mobile app

*Non-Functional Requirements*

- Correctness above all: two vehicles must never be assigned the same physical spot at the same time
- Low-latency spot lookup at entry (drivers are waiting at a gate arm)
- The system should be extensible - new spot types, vehicle types, and pricing models should not require touching core allocation logic
- Should scale from a single 5-level garage to a chain of hundreds of garages without a redesign
- Graceful degradation: a garage should keep functioning (in a "dumb"/local mode) even if it briefly loses connectivity to any central service

**Framing:** if you jump straight to classes here, it is worth pausing yourself, "What kinds of vehicles and spot types do we need to support, and how is pricing determined?" is a fair question to ask at this stage, and the answer (a small fixed enum of vehicle/spot sizes, pluggable pricing) shapes everything that follows.

### 2. Capacity Estimation

Assuming the interviewer says:

- 1 garage: 5 levels × 200 spots/level = 1,000 spots (mixed motorcycle/compact/large/handicap/EV)
- Peak turnover: ~3,000 entry/exit events/day per garage → roughly 1 event every ~15–20 seconds at peak, occasionally bursts of several cars arriving within the same second at different gates
- A chain scenario (raised at the senior stage): 500 garages × ~1,000 spots average = ~500,000 spots citywide, ~5–10 million app users checking availability, with spikes around events/holidays

For a single garage, this is a small, mostly in-memory problem - the interesting part is not throughput, it's correctness under concurrent access at a handful of gates. That's why this case study leans on object-level design and state modeling rather than horizontal scaling from the start; the scaling discussion is deliberately saved for the last section.

### 3. Core Classes & Relationships

```
ParkingLot (chain-level, one per city/company - singleton per garage in the simple case)
   |
   |--- has many ---> Level
   |                    |
   |                    |--- has many ---> Spot  <-------- SpotType (enum: MOTORCYCLE,
COMPACT, LARGE, HANDICAPPED, EV)
   |                    |                    |
   |--- has many ---> Gate (ENTRY / EXIT)     |--- references ---> Vehicle (0..1, when
occupied)
   |                                          |
   |--- uses ---> SpotAllocator (finds a free Spot for a Vehicle)
   |--- uses ---> PricingStrategy (computes fee for a Ticket)
```

```
   |
Vehicle (id, licensePlate, VehicleType: MOTORCYCLE/COMPACT/REGULAR/BUS)
   |
   '--- associated with ---> Ticket (issued on entry, closed on exit)

Ticket (id, vehicle, spot, entryTime, exitTime, amountDue, TicketStatus:
ACTIVE/PENDING_PAYMENT/PAID/LOST_TICKET)
   |
   '--- has ---> Payment (amount, method, PaymentStatus)
```

- **ParkingLot** - top-level facade/coordinator. Owns Levels and Gates, exposes `parkVehicle(vehicle)` and `unparkVehicle(ticket)`, and holds references to the pluggable `SpotAllocator` and `PricingStrategy`.
- **Level** - one physical floor; owns its list of `Spot`s and keeps a free-count per `SpotType` for fast availability display.
- **Spot** - smallest unit of state: `id`, `SpotType`, `SpotStatus` (FREE / RESERVED / OCCUPIED / OUT_OF_SERVICE), and an optional reference to the `Vehicle` currently parked there.
- **Vehicle** - a value-ish entity representing what enters the gate; carries the `VehicleType` used to match against compatible `SpotType`s (e.g., a `BUS` can only use `LARGE` spots, a `MOTORCYCLE` fits any spot but should prefer `MOTORCYCLE` spots to conserve capacity).
- **Ticket** - the transactional record binding a `Vehicle` to a `Spot` for a duration; it's the object most systems get wrong if they don't model its state machine explicitly (see next section).
- **Gate** - entry or exit hardware endpoint; entry gates call the allocator, exit gates call the pricing strategy and close the ticket.
- **SpotAllocator** and **PricingStrategy** are intentionally pulled *out* of `ParkingLot` as injected strategies rather than baked-in methods - this is what makes sections 5 and 6 possible without modifying `ParkingLot` itself.

This is a fairly standard composition tree (`ParkingLot` → `Level` → `Spot`) plus two cross-cutting strategy interfaces. **Keeping `Spot` "dumb" (just state, no allocation logic) and pushing decision-making into `SpotAllocator` is the key design choice - it keeps a single object's job small, which is exactly what makes the concurrency story in section 5 tractable.**

#### 4. State Modeling: Spot and Ticket Lifecycles

A `Spot` has a small, well-defined lifecycle:

```
FREE ---reserve()---> RESERVED ---confirm()---> OCCUPIED ---vacate()---> FREE
  \                                                                       ^
   \-------------------------- cancel() ----------------------------/
FREE ---markOutOfService()---> OUT_OF_SERVICE ---markInService()---> FREE
```

A `Ticket` mirrors it on the transaction side: `ACTIVE` (car parked, clock running) → `PENDING_PAYMENT` (car at exit gate, fee computed) → `PAID` (gate opens) - with a `LOST_TICKET` branch that falls back to a max-stay flat fee.

*Options for modeling this:*

- **Boolean flags / status enum checked via if-else** - quick to write, but logic for "what transitions are legal from this state" ends up scattered across the codebase, and it's easy to leave a `Spot` in an inconsistent state (e.g., `OCCUPIED` with no `Vehicle` reference) after a bug.
- **Formal State design pattern** (a `SpotState` interface with `FreeState`, `ReservedState`, `OccupiedState` classes, each implementing legal transitions and rejecting illegal ones) - more classes up front, but illegal transitions become compile-time-checked or throw immediately, and new states (e.g., a `CLEANING` state for EV spots) don't require touching existing state classes.

**Best Choice:** Enum-backed status field with transition validation centralized in one method (`Spot.transitionTo(newStatus)`), escalating to the full State pattern only if the state machine grows complex (e.g., adding reservation holds with expiry, multi-step EV charging states).

**Why?** For a spot's lifecycle - 4-5 states, a handful of legal transitions - a full State-pattern class hierarchy is often over-engineering; a `switch` on legal-transition pairs inside one guarded method gets the same safety with far less ceremony. It's the right call to *know about* the State pattern and reach for it the moment the transition rules start branching on more than the current state (e.g., transitions that depend on vehicle type or time of day).

**Interview Answer:** "I'd model `Spot` and `Ticket` status as enums, but I would never let calling code set `spot.status = OCCUPIED` directly - all transitions go through a single `transitionTo()` method that has a table of legal from→to moves and throws on anything else. That gives me the safety of a formal state machine without the class explosion of the full State pattern, and I can graduate to that pattern later if the transition rules get more complex."

**5. Concurrency: Preventing Double-Booking a Spot**

This is the sharpest object-level question in the whole system: two cars enter through two different gates at the same instant and are both directed to the same open spot. Where exactly does the race condition live, and how is it closed?

The race is a classic check-then-act: Gate A calls `allocator.findSpot(vehicle)`, which reads `spot.status == FREE`, and *before* it writes `RESERVED` back, Gate B's thread does the same read and gets the same answer.

*Options:*

- **No synchronization, "last write wins"** - simplest, but this is exactly the double-booking bug the question is testing for.
- **Application-level lock per Spot (`synchronized` / mutex) held across the check-and-set** - correct within a single process, but doesn't help if the garage runs multiple allocator instances (e.g., one process per gate) or the state lives in a shared database.
- **Optimistic concurrency (compare-and-swap on a version number / `status` column: `UPDATE spots SET status='OCCUPIED', version=version+1 WHERE id=? AND status='FREE' AND version=?`)** - no lock is held while "thinking"; the loser of the race simply gets zero rows updated and retries with the next candidate spot.
- **Pessimistic row-level lock (`SELECT ... FOR UPDATE` on the spot row)** - guarantees exclusivity but serializes all allocation through the database and can deadlock if spots are locked in inconsistent order across transactions.
- **External distributed lock (Redis `SETNX`/Redlock on `spot:{id}`)** - useful once allocation is spread across many machines, but adds an extra moving part and failure mode (lock TTL vs. slow allocator) for what is fundamentally a single-row update.

**Best Choice:** Atomic compare-and-swap at the persistence layer (optimistic concurrency via a conditional update or a `status` column with a unique partial index / version check), with the allocator retrying against the next candidate spot on a CAS failure.

**Why?** The operation is a single-row state transition (`FREE → RESERVED`) - this is exactly what atomic compare-and-swap primitives (whether a DB's conditional `UPDATE`, or an in-memory `AtomicReference.compareAndSet` for a single-process design) are built for. It avoids holding any lock while the "slow" part of the operation (e.g., printing a ticket, notifying a display board) runs, it scales to multiple allocator processes without a separate lock service, and the failure mode is simple and safe: the losing gate just tries the next free spot in its candidate list instead of blocking or erroring out for the driver.

**Interview Answer:** "The race is in the check-then-act between reading `FREE` and writing `RESERVED`. I'd close it with an atomic compare-and-swap - a conditional update like `UPDATE spots SET status='RESERVED' WHERE id=? AND status='FREE'` - rather than a lock held across the whole allocation flow. Whichever gate's update actually changes a row wins the spot; the other gate sees zero rows affected, treats that as 'someone beat me to it,' and immediately retries against the next candidate spot from the allocator. That keeps the exclusive-access window as short as a single row update and works the same whether I have one allocator process or ten."

**6. Extensible Pricing via Strategy Pattern**

The garage needs flat-rate pricing for some tenants, hourly/tiered pricing for transient parkers, and dynamic/peak pricing for event days - and the requirement explicitly is: adding a new scheme shouldn't mean touching `ParkingLot` or `Ticket`.

*Options:*

- **`if/else` or `switch` on a `pricingType` field inside `Ticket.calculateFee()`** - works for two pricing types, becomes an unmaintainable branch tree by the third or fourth, and every new scheme risks regressing the existing ones since they all live in one method.
- **Subclassing `Ticket` per pricing type** (`HourlyTicket`, `FlatRateTicket`) - mixes "what happened during this parking session" with "how do we price it," and a ticket can't change pricing scheme (e.g., switch to peak pricing mid-stay) without changing its class.
- **Strategy pattern** - extract a `PricingStrategy` interface (`calculateFee(entryTime, exitTime, spotType, vehicleType) -> Money`) with concrete implementations `FlatRateStrategy`, `HourlyStrategy`, `DynamicPeakStrategy`; `Ticket`/`ParkingLot` hold a reference to whichever strategy applies and delegate to it.

**Best Choice:** Strategy pattern, with the concrete strategy chosen per-garage (or even per-ticket, for promo pricing) via configuration, not a code branch.

**Why?** It isolates the one piece of the system guaranteed to change often - pricing rules are a business/product decision, not an engineering one - behind a stable interface. A `DynamicPeakStrategy` can internally call out to an occupancy signal ("if level utilization > 80%, apply a 1.5x multiplier") or an external demand service, and none of that logic ever needs to be known by `ParkingLot`, `Ticket`, or `Gate`. New pricing models are added by writing a new class and wiring it in, with zero risk to existing strategies since they're untouched.

**Interview Answer:** "I'd pull fee calculation out of `Ticket` entirely and behind a `PricingStrategy` interface with one method, `calculateFee`. `FlatRateStrategy`, `HourlyStrategy`, and `DynamicPeakStrategy` each implement it independently - the dynamic one might look at current occupancy or a time-of-day table internally, but that's invisible to the rest of the system. `ParkingLot` is just handed a strategy at configuration time and calls it at exit; adding peak pricing next quarter means adding one new class, not touching the ticket or gate logic at all."

**7. Spot Assignment Strategy**

A closely related design question: given a `Vehicle`, how does `SpotAllocator` pick *which* free spot to hand out - nearest to the entrance, nearest to an elevator, lowest level first, or dedicated EV/handicap spots reserved regardless of general demand?

*Options:*

- **Linear scan across all spots on all levels** - trivial to implement, but O(n) per allocation and doesn't naturally support "prefer this level" heuristics.
- **Per-level, per-type free-spot list (or min-heap ordered by proximity to entrance)** - each `Level` maintains a small free-list per `SpotType`; the allocator asks each level in a preferred order ("try

ground floor first for compact cars") and pops the first candidate, falling back to the next level on a CAS failure (from section 5).

- **Fully centralized index (e.g., a single sorted structure across the whole garage)** - best global optimality (truly nearest spot), but becomes a single contention point and doesn't decompose well once the garage becomes a multi-garage chain.

**Best Choice:** Per-level free-lists keyed by `SpotType`, with the allocator trying levels in a configurable preference order and retrying the next candidate on CAS failure.

**Why?** It keeps the hot path (find *a* free spot fast) cheap - O(1) pop from a free-list - while keeping contention localized to one level's data structure instead of one lock over the whole garage. It also composes cleanly: this is the same data structure a `Level` needs anyway to answer "how many spots of type X are free right now" for the entry-board display and, later, the mobile app.

**Interview Answer:** "Each `Level` keeps a small in-memory free-list per spot type, so `findSpot` is just 'pop from the first non-empty list among the levels I prefer.' If the CAS from the concurrency design fails because someone else grabbed that spot first, the allocator just pops the next candidate - the free-list and the CAS retry loop work together naturally. This also happens to be the exact data I need to show live availability counts, which becomes important once we add the mobile app."

**8. Scaling to a Multi-City Chain with a Real-Time Availability App**

Now the scope changes from one garage to 500 garages across a city, with a shared mobile app showing real-time availability and letting users reserve a spot remotely. Several assumptions from the single-garage design stop holding, and it's worth being explicit about what breaks and what survives:

- **What survives:** every class in section 3 stays the same shape. `ParkingLot`, `Level`, `Spot`, `Ticket`, and the `SpotAllocator`/`PricingStrategy` interfaces don't change - a garage is still, locally, exactly the LLD problem already solved. This is the payoff of having pulled allocation and pricing out as interfaces rather than hardcoding them.
- **What breaks:** the free-list allocator was in-process, in-memory, and assumed a handful of gates talking to one garage's state. At chain scale there are now three distinct concerns that used to be conflated: (1) the *authoritative* state of each garage (still owned locally, for latency and availability), (2) an *aggregated*, *eventually-consistent* availability feed for the app (hundreds of thousands of spots, read by millions of users, updates don't need to be instant), and (3) a *remote reservation* flow, which introduces a new state - `RESERVED_REMOTE` with an expiry - because a reservation now has to survive the gap between "user taps reserve on their phone" and "car physically arrives."
- **Local-first, sync-out architecture:** each garage keeps running its own `ParkingLot` instance as the source of truth for its own spots - a driver at the gate arm cannot be blocked on a citywide service being up. Each garage publishes occupancy deltas (spot X went FREE→OCCUPIED) to a central event stream; a chain-level `AvailabilityAggregator` consumes these to maintain the citywide, slightly-stale (seconds, not milliseconds) view that powers the app's map and search. This is the same cache-aside/eventual-consistency trade-off as any read-heavy system: the app doesn't need line-of-sight to the literal current second, it needs "roughly how full is this garage right now."
- **Remote reservation as a new spot state, not a new class:** a reservation from the app calls the same `SpotAllocator.reserve()` path used at a physical entry gate - CAS still applies, so an app user and a walk-up driver can't both grab the same spot - but the resulting state is `RESERVED_REMOTE` with a TTL (e.g., 15 minutes). A background sweeper releases spots whose reservation expired unclaimed, feeding them back into the free-list. This reuses the state-machine and concurrency design from sections 4–5 rather than inventing a parallel system for app-originated bookings.
- **Chain-level identity, not garage-level:** `Ticket` and `Payment` need a chain-wide customer/account concept (for loyalty pricing, saved payment methods, cross-garage history) - this pushes

`PricingStrategy` to sometimes need chain-level context (e.g., a subscriber's flat monthly rate that applies at any of the 500 garages), which is exactly why pricing was designed as an injected strategy and not a `Ticket` method: a `SubscriptionStrategy` slots in without touching `Ticket`.

- **Failure isolation:** if the central `AvailabilityAggregator` or the app's backend goes down, individual garages keep admitting and billing walk-up traffic normally - they just stop being visible on the app until the aggregator recovers. Local correctness (no double-booking within a garage) never depends on the chain-level service being healthy.

**Interview Answer:** "The single-garage classes don't change - that's the benefit of keeping `SpotAllocator` and `PricingStrategy` as interfaces from day one. What changes is that each garage stays authoritative for its own spots and keeps working offline from the chain's point of view; it just streams occupancy deltas outward to a chain-level aggregator that powers the app's citywide availability view, which is allowed to be a few seconds stale. Remote reservations reuse the exact same compare-and-swap allocation path as a physical gate, just landing in a new `RESERVED_REMOTE` state with a TTL so an unclaimed reservation expires back into the free pool. The one thing that has to be added deliberately is a chain-wide account/pricing context, which is why pricing was a pluggable strategy rather than hardcoded logic in the first place."

**9. Final Design**

```
                        Mobile App (millions of users)
                                    |
                        Chain API / AvailabilityAggregator
                        (eventually-consistent, cached)
                                    ^
                                    | occupancy deltas (async)
    ------------------------------------------------------------------
    |                             |                                  |
ParkingLot (Garage 1)     ParkingLot (Garage 2)   ...       ParkingLot (Garage 500)
    |
    |-- Level[] -- Spot[] (FREE/RESERVED/RESERVED_REMOTE/OCCUPIED/OUT_OF_SERVICE)
    |-- Gate[] (ENTRY/EXIT) --> SpotAllocator.reserve()  [atomic CAS per spot]
    |-- Ticket --> PricingStrategy (Flat / Hourly / DynamicPeak / Subscription)
    |-- Payment
```

**10. Follow-up Questions**

*How exactly do you stop two gates from assigning the same spot?*

Best Answer: I close the check-then-act race with an atomic compare-and-swap at the point of state change - a conditional update that only succeeds if the spot is still `FREE` - rather than holding a lock across the whole allocation flow. The losing request just retries against the next candidate spot instead of failing.

*Why pull pricing out into a Strategy interface instead of a field on Ticket?*

Best Answer: Pricing rules change far more often, and far more independently, than the core parking/ticketing logic. A `PricingStrategy` interface means `FlatRateStrategy`, `HourlyStrategy`, `DynamicPeakStrategy`, or a future `SubscriptionStrategy` can be added without touching `Ticket`, `ParkingLot`, or any existing strategy - new pricing is purely additive.

*What's the single biggest change going from one garage to a 500-garage chain?*

Best Answer: Splitting "authoritative local state" from "aggregated global view." Each garage must keep functioning and remain double-booking-safe with zero dependency on any citywide service; the chain only needs an eventually-consistent, async-updated feed of availability to power the app, and remote reservations are just the existing allocation path plus one new time-bounded spot state.

## Design a Chat/Messaging App

## Q1. [Mid–Senior] Design a messaging app like WhatsApp that supports 1:1 chats and group chats with near real-time delivery.

**Answer:**

### 1. Clarify Requirements

*Functional Requirements*

- 1:1 chat between two users
- Group chat with up to ~500 members
- Near real-time message delivery
- Messages delivered even if recipient is offline (store-and-forward)
- Delivery/read receipts (sent, delivered, read)
- Message ordering preserved per conversation
- Multi-device support (phone + web/desktop simultaneously)
- Media messages (images, video, files) - handled via separate blob storage, not in the hot path
- Push notifications when app is backgrounded

*Non-Functional Requirements*

- Low latency delivery (sub-second on good networks)
- High availability - messaging is core, outages are unacceptable
- At-least-once delivery - no message ever silently dropped
- Ordering guarantee: per-conversation causal ordering, not global ordering
- Horizontally scalable to hundreds of millions of concurrent connections
- Durable storage of messages for offline delivery and history
- Eventually consistent across a user's multiple devices

*Guarantees around delivery and ordering (the probe question)*

Users expect **at-least-once delivery** (a message should never vanish), **per-conversation ordering** (messages in one chat appear in the order they were sent, even if they arrive at the server out of order due to network races), and **eventual delivery** (if the recipient is offline, the message is queued and delivered once they reconnect - not lost). They do NOT need strict global ordering across different chats, and they tolerate small delivery delays as long as ordering within a conversation is preserved and duplicates are deduplicated (idempotent client-side rendering keyed by message ID).

### 2. Capacity Estimation

Assumptions:

- 500M daily active users (DAU)
- 40 messages/day/user average

Messaging volume:

- Total messages/day = 500M × 40 = 20 billion messages/day
- Average QPS = 20B / 86,400s ≈ **231,000 messages/sec**
- Peak QPS (3-4x average for evening/holiday peaks) ≈ **700K–900K messages/sec**

Storage:

- Average message size ≈ 100 bytes (text + metadata: sender, recipient/group ID, timestamp, message ID, status flags)

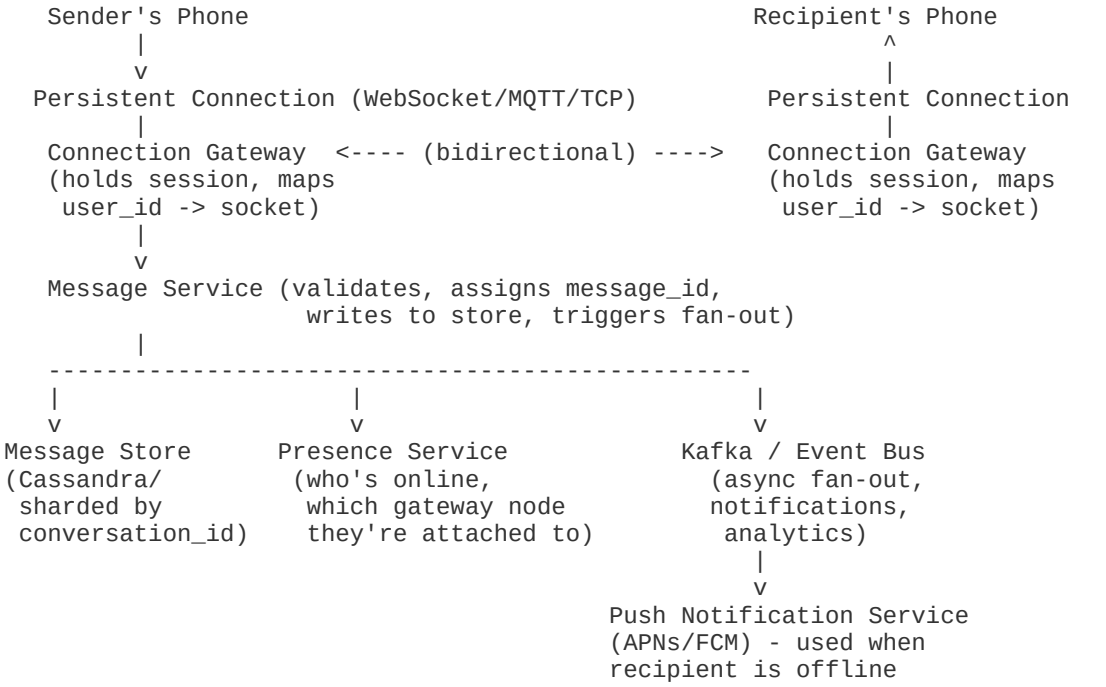
- Daily storage (text only) = 20B × 100 bytes ≈ **2 TB/day** ≈ **~730 TB/year** for raw message text
- With replication factor 3 for durability: ≈ **6 TB/day**, ≈ **2.2 PB/year**
- Media (images/video) is far larger in bytes but far lower in count - typically stored in object storage (S3-like) with only a URL/reference stored in the message row, so it does not dominate the message-store estimate above.

Connections:

- If ~40% of 500M DAU are concurrently connected at peak ≈ **200M concurrent persistent connections** to maintain across the fleet - this is the number that drives connection-server (gateway) sizing, not just message throughput.

**Takeaway: this is a write-heavy, latency-sensitive, small-payload system dominated by fan-out and connection management, not by raw storage volume (2 TB/day is trivial for modern storage - the hard part is real-time delivery to hundreds of millions of live sockets).**

### 3. High-Level Architecture




**Flow (sender to recipient, online case):**

1. Sender's client sends the message over its persistent connection to its connection gateway node.
2. Gateway forwards to the Message Service, which generates a unique, time-ordered message ID (Snowflake-style), persists the message durably, and marks it "sent."
3. Message Service asks Presence Service which gateway node the recipient is attached to (if any).
4. If the recipient is online, the message is routed directly to that gateway node and pushed down the recipient's open socket - near-instant delivery, and the sender gets a "delivered" ack once the recipient's client confirms receipt.
5. If the recipient is offline, the message is durably queued (already persisted in the Message Store); when the recipient reconnects, their client syncs unread messages since their last-seen cursor. In parallel, a push notification (APNs/FCM) is sent so the OS wakes the app or shows a banner.
6. Read receipts flow back the same way, in reverse, updating message status (sent → delivered → read).

### 4. Connection Management at Scale (persistent connections)

*Long Polling* - simple, works everywhere, but high latency and wasteful (repeated HTTP overhead, hard to scale to hundreds of millions of open requests).

*WebSockets* - full-duplex, low latency, one persistent TCP connection per client, widely supported on mobile and web.

*MQTT* - lightweight pub/sub protocol built for exactly this (mobile, battery-constrained, unreliable networks), used by real messaging systems (e.g., Facebook Messenger uses MQTT).

**Best Choice:** WebSocket (or MQTT for mobile-optimized battery/bandwidth) terminated at a horizontally-scaled fleet of stateful "connection gateway" servers, with a separate Presence Service tracking `user_id → gateway_node` mappings in a fast key-value store (Redis).

**Why?** Persistent connections avoid the overhead of establishing a new connection per message and let the server push data instantly instead of the client polling. Because a single machine can only hold a bounded number of open sockets (typically 50K-500K depending on tuning), the gateway tier must be horizontally scalable and stateless about routing logic - the actual "who is connected where" state lives in a shared, fast-lookup Presence Service (Redis), not on any single gateway box, so any gateway node can ask "where is this user" and any message service can route through any gateway.

**Interview Answer:** "I'd use WebSocket connections terminated by a fleet of connection gateway servers, with Redis-backed presence tracking mapping each user to the specific gateway node holding their socket. This lets the system scale horizontally - connection gateways scale independently of message processing, and a message can be routed to the correct node regardless of which gateway server the message service happens to be talking to."

#### 5. Offline Delivery

If a message arrives for a user with no active connection, the naive approach of "just push it" fails silently. The message is first durably written to the Message Store (this is the source of truth, not the socket push) with a status of "sent, not yet delivered." A push notification is fired via APNs/FCM to wake the app. When the client reconnects, it doesn't wait for the server to re-push individual missed messages - instead it does a **pull-based sync**: "give me everything since my last-seen message ID/timestamp per conversation," which is efficient, resumable, and idempotent even across app crashes or long offline periods.

**Best Choice:** Durable-write-first + pull-based catch-up sync on reconnect, backed by push notifications as a wake-up signal only (not as the delivery mechanism itself).

**Why?** Treating persistence as the delivery guarantee (not the live socket push) decouples "the message exists and will eventually be seen" from "the message was pushed live" - this is what makes at-least-once delivery achievable even through arbitrary offline durations, app kills, and device changes.

**Interview Answer:** "I never treat a live socket push as the delivery mechanism of record - the message is durably persisted first, and delivery over an open socket is just an optimization for the common online case. For offline users, I send a push notification purely to wake the app, and the client then pulls anything it missed since its last sync cursor. This makes offline delivery a special case of the same sync mechanism used for reconnects, rather than a separate code path."

#### 6. Group Chat Fan-Out at 500 Members

In a 1:1 chat, sending a message means routing to exactly one recipient - cheap, one lookup, one push. In a group of 500, the same "send" action must reach 500 recipients, and this is where the architecture must change.

*Fan-out on write (push model)* - when a message is sent, immediately write a copy of it (or a reference) into each of the 500 recipients' inboxes/queues, and push to each of their active gateway connections. Read is then fast (each user just reads their own inbox).

*Fan-out on read (pull model)* - store the message once against the group/conversation ID; each member's client queries "give me new messages for this group" on connect/poll. Write is cheap (one write), but reads are more complex and less "instant."

**Best Choice:** Hybrid - store the message once in the group's conversation log (single durable write, like a shared append-only log keyed by `group_id`), then **fan out asynchronously via a message queue (Kafka)** to determine, for each of the 500 members, whether they're online (push directly through their gateway) or offline (rely on pull-sync + push notification). Do not physically duplicate the message payload 500 times into 500 separate per-user tables.

**Why?** Writing the message 500 times (true fan-out-on-write duplication) wastes storage and creates a write-amplification problem that gets worse as group size grows (a 500-person group is 500x the writes of a 1:1 message). Instead, storing once against `group_id` and fanning out just the "notify/deliver" work (via Kafka consumers that push to whichever gateway node each online member is attached to) keeps the expensive part - durable storage - O(1) per message, while the delivery fan-out becomes an asynchronous, parallelizable, retryable background job. This is exactly where 1:1 and group chat diverge: 1:1 delivery is synchronous and inline with the send path; group delivery for large groups is deliberately pushed to an async fan-out worker pool so one slow/offline recipient never blocks the other 499, and so a burst of large-group traffic doesn't overload the message service's request path.

**Interview Answer:** "For groups, I store the message once against the group's conversation log rather than duplicating it per member - duplicating writes doesn't scale as group size grows. Delivery is handled by an asynchronous fan-out step: a Kafka consumer looks up presence for each of the group's members and either pushes directly to their gateway if they're online, or leaves it for pull-sync and a push notification if they're offline. This decouples the cost of durability (one write) from the cost of delivery (up to 500 pushes), and it means one unreachable member never blocks delivery to the rest of the group."

**7. Multi-Device Sync (hardest edge case)**

The user has WhatsApp open on both their phone and a laptop web client at the same time. Both need to show the same conversation, in the same order, with consistent read state - but each is an independent persistent connection to (possibly) different gateway nodes.

The core problem: if each device tracked delivery/read state independently, a message read on the laptop wouldn't be reflected as read on the phone, and if the two devices raced to update read receipts, "last read" could bounce backwards.

**Best Choice:** Treat devices as independent socket subscribers to the same underlying per-user/per-conversation event stream, not as independent state owners. Messages and status changes (delivered, read) are written once to the durable per-conversation log with a monotonically increasing sequence number; every device registered to that account subscribes to the same stream and fans out the update to whichever gateway sockets that user currently holds (one push per active device). Read state is stored server-side as a single logical "last read message ID/sequence number per conversation per user" (not per device); when any device marks messages read, that watermark is written once and broadcast to all of the user's other devices, so they all converge to the same read position. Ordering is preserved because every device renders messages by the shared server-assigned sequence number, never by local receipt order.

**Why?** If ordering or read-state were device-local, the two devices would drift and show contradictory unread counts or message orders. By making the server-assigned per-conversation sequence number and the single "last read" watermark the sources of truth, and treating each device merely as a subscriber that renders whatever the server stream says, both devices are guaranteed to converge to the same ordering and the same read state, regardless of which device the user is actively using or which device is intermittently offline.

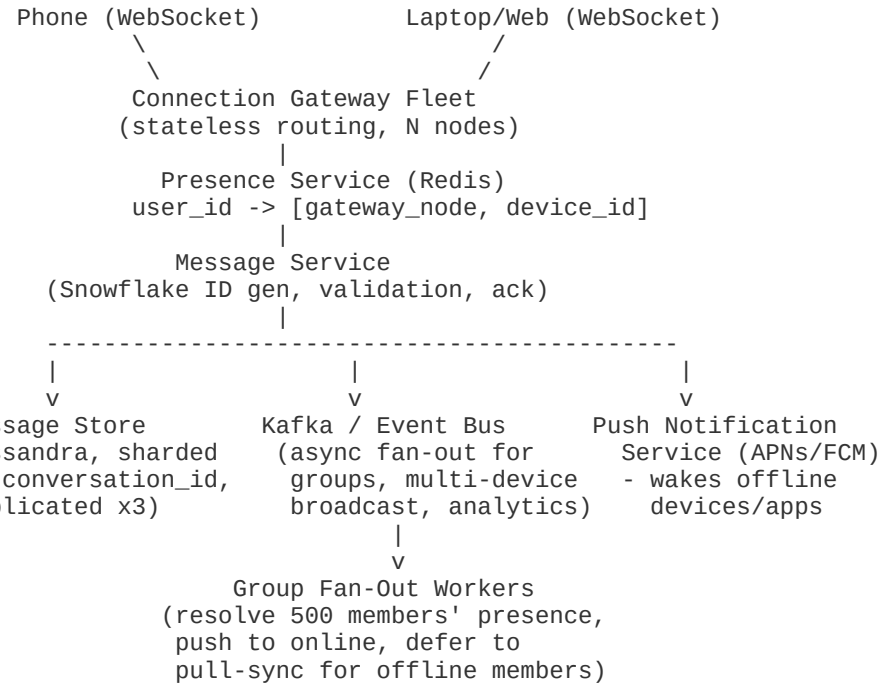
**Interview Answer:** "I'd model each of a user's devices as a separate socket subscribed to the same server-side event stream for that user, rather than giving each device its own notion of state. Every message gets one server-assigned sequence number per conversation, so all devices render the same order. Read receipts are a single 'last read' watermark per conversation per user, stored server-side - when one device marks a conversation read, that watermark update is broadcast to the user's other active devices, so a laptop and phone converge to the same unread count instead of drifting independently."

### 8. Handling the Hardest Edge Case in Production (message ordering races)

If a user sends two messages back-to-back and, due to network retries or multiple gateway nodes, they could theoretically be persisted out of order, the server must be the single point that assigns order - never trust client-side timestamps for ordering, since clocks drift and retries can reorder network delivery. This is why message IDs are generated server-side (Snowflake-style: time-ordered, globally unique, no coordination needed) at the moment the Message Service durably accepts the write, and clients render strictly by that ID/sequence number, buffering and reordering locally if a lower-numbered message arrives after a higher-numbered one (which can happen on flaky networks).

**Interview Answer:** "I never rely on client timestamps for ordering - clocks drift and retries can reorder delivery. Instead, the server assigns a monotonic, time-ordered ID the moment it durably accepts a write, and clients always render by that ID, buffering briefly to reorder if a network hiccup delivers messages out of sequence."

### 9. Final Architecture




### 10. Follow-up Questions

*What delivery guarantee do you actually provide - exactly-once or at-least-once?*

Best Answer: At-least-once, with client-side deduplication by message ID. Exactly-once delivery across an unreliable network is impractical to guarantee end-to-end; instead I make redelivery safe by giving every message a unique ID and having clients ignore duplicates, which achieves the same user-visible effect.

*Why not just duplicate the message into every group member's inbox at write time?*

Best Answer: That turns one write into up to 500 writes per message, and write amplification scales linearly with group size - a very active 500-person group would multiply storage and I/O costs 500x versus a 1:1 chat

of the same message volume. Storing once and fanning out delivery asynchronously via Kafka keeps the durable write O(1) and makes the fan-out step independently scalable and retryable.

*How do you keep message ordering correct if the sender's two messages hit different servers?*

Best Answer: Ordering is never decided by which server received the message or by client timestamps - the Message Service assigns a single monotonic, time-ordered ID (Snowflake-style) at write-acceptance time per conversation, and all clients render strictly in that order, buffering briefly on the client if a network hiccup causes messages to arrive out of sequence.

*How does read state stay consistent across a phone and a laptop open at the same time?*

Best Answer: Read state is stored as one server-side "last read" watermark per user per conversation, not per device. Any device updating that watermark broadcasts the change to the user's other active device sockets, so unread counts and read receipts converge across devices instead of drifting independently.

*Does a single connection gateway server actually hold a million WebSocket connections open at once? How does that scale?*

Best Answer: A single, well-tuned server can hold hundreds of thousands to around a million idle WebSocket connections open at once, since an idle persistent connection mostly costs a file descriptor and a small in-memory buffer, not CPU. The real scaling technique is not making one machine hold more sockets, it is running a horizontally scaled, stateless fleet of connection gateway nodes behind a load balancer, with a Redis-backed presence service tracking which specific node each user is currently attached to. Reaching hundreds of millions of concurrent users then just means adding more gateway nodes, not pushing any single node past what its OS and network stack can comfortably hold.

## Design a Twitter-like News Feed

---

## Q1. [Mid–Senior] Design the core feed feature of a Twitter-like service, where users see a feed of tweets from accounts they follow.

**Answer:**

### 1. Clarify Requirements

*Functional Requirements*

- Users can post short text tweets (optionally with media)
- Users can follow/unfollow other accounts
- Users can view a home feed showing tweets from accounts they follow, most-recent-first (chronological baseline)
- Users can view a single user's own timeline (their posted tweets)
- Users can delete a tweet
- REST/gRPC APIs for post, follow, and feed-fetch

*Non-Functional Requirements*

- Home feed loads fast (p99 well under 500 ms)
- Highly available - feed should render even if some backend is degraded (serve slightly stale data over an error)
- Read-heavy system, but with extreme write skew for certain accounts ("celebrity problem")
- Eventually consistent is acceptable (a new tweet appearing a few seconds late is fine)
- Horizontally scalable to hundreds of millions of users and a highly skewed social graph

*The social graph, and why it matters*

"Following" is a one-directional, asymmetric relationship: A can follow B without B following back. This is the key thing that breaks a naive design - some accounts (celebrities, news orgs) have tens of millions of followers, while the median account has a few hundred. That power-law skew means we cannot treat every write (a new tweet) the same way; a small number of accounts drive a disproportionate share of fan-out work. Any workable design has to explicitly handle this skew rather than assume a roughly uniform follower count.

**2. Capacity Estimation**

Assumptions (stated explicitly, as the interviewer gave them):

- 200M daily active users (DAU)
- Each user posts an average of 2 tweets/day
- Each user follows an average of 200 accounts

*Write load (tweets)*

- Tweets/day = 200M × 2 = 400M tweets/day
- Tweets/sec (avg) = 400M / 86,400 ≈ 4,630 writes/sec
- Peak (assume 3-5x average for daily peak hours) ≈ 15,000-23,000 writes/sec

*Fan-out multiplier*

- Average followers per account ≈ average follows per account = 200 (in a closed graph, average in-degree = average out-degree)
- Naive fan-out-on-write: every tweet gets copied into the home-feed cache of every follower
- Fan-out writes/day = 400M tweets × 200 avg followers = 80B feed-insertion events/day
- Fan-out writes/sec (avg) = 80B / 86,400 ≈ 926,000/sec - roughly a 200x amplification over the raw tweet write rate

This 200x multiplier is the headline number: a system that looks like ~4,600 writes/sec at the tweet layer actually needs to sustain ~900K+ writes/sec at the feed-fanout layer if done naively. **That gap is why fan-out strategy is the central design decision.**

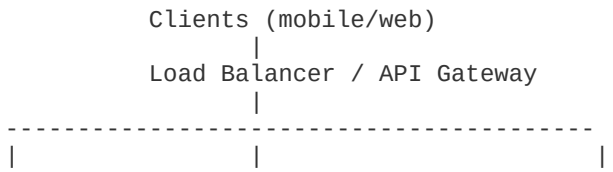
*Read load (home feed fetches)*

- Assume each DAU refreshes their feed ~10 times/day (open app, pull-to-refresh, scroll sessions)
- Feed reads/day = 200M × 10 = 2B reads/day
- Reads/sec (avg) = 2B / 86,400 ≈ 23,000 reads/sec
- Read:write ratio at the tweet-post layer ≈ 23,000 : 4,630 ≈ 5:1 - read-heavy, so caching the assembled feed matters a lot

*Storage*

- Tweet size (text + metadata) ≈ 300 bytes; 400M tweets/day × 300 bytes ≈ 120 GB/day ≈ 44 TB/year for raw tweets (media stored separately in blob storage/CDN, referenced by URL)
- Feed cache: storing ~800 tweet IDs (a few days of scrollback) per active user × 200M users × 8 bytes/ID ≈ 1.3 TB of hot cache data - comfortably shardable across a Redis/Memcached fleet

**3. High-Level Architecture**



```
Tweet Service     Follow/Graph Service     Feed Service
      |                 |                      |
Tweet Store       Graph Store             Feed Cache (Redis)
(sharded DB)      (sharded DB)            (per-user sorted list
      |                                     of tweet IDs)
      |
Fan-out Service  <----- Message Queue (Kafka) ----- new tweet events
      |
writes tweet ID into follower feed caches
(skips celebrities -> handled by hybrid path, see Deep Dive)
```

**4. Deep Dive: Fan-out-on-Write vs. Fan-out-on-Read**

This is the core trade-off in feed generation: when a user posts a tweet, do we push it into every follower's feed immediately (write time), or do we pull it together when a follower asks for their feed (read time)?

*Fan-out-on-write (push model)* - On tweet creation, look up all followers and insert the new tweet ID into each follower's precomputed feed (a sorted list/cache, keyed by user ID). Reading the feed becomes a cheap O(1) lookup: just fetch the precomputed list.

- Pros: Feed reads are extremely fast (single cache lookup), which matters given the 5:1 read:write ratio.
- Cons: Write amplification is severe (the 200x multiplier from the estimation above) and gets catastrophic for high-follower accounts - one tweet from a 50M-follower account means 50M writes.

*Fan-out-on-read (pull model)* - On tweet creation, just store the tweet once. When a follower requests their feed, fetch the list of accounts they follow, pull recent tweets from each, merge and sort by time on the fly.

- Pros: Writes stay cheap and constant-cost regardless of follower count - no amplification, no celebrity problem.
- Cons: Reads become expensive - fetching and merging tweets from 200 followees on every feed load, for every one of the ~23,000 reads/sec, is a heavy fan-in operation and hard to make consistently fast.

**Best Choice:** A hybrid model - fan-out-on-write by default for normal accounts, fan-out-on-read for celebrity/high-follower accounts, merged at read time.

**Why?** The two models optimize opposite bottlenecks, and the follower distribution is heavily skewed (power law), so a single global strategy is wrong for both ends of the distribution. Most accounts have modest follower counts, so eagerly pushing their tweets into followers' caches is cheap and keeps reads fast, which matters because reads outnumber writes 5:1. But the same eager push becomes disastrous for the small number of celebrity accounts, so those need to be excluded from push fan-out and instead pulled in at read time. Using the hybrid model means the system's cost scales with actual behavior (how many "normal" follows a user has) rather than being dominated by worst-case celebrity fan-out.

**Interview Answer:** "I'd default to fan-out-on-write because our read:write ratio is roughly 5:1 and the majority of accounts have modest follower counts, so precomputing feeds keeps reads cheap. But I would not apply that uniformly - the follower distribution is a power law, so I'd carve out an exception for high-follower 'celebrity' accounts and fetch their tweets at read time instead, merging them into the feed on the fly. That hybrid approach avoids paying the worst-case fan-out cost while keeping the common case fast."

**5. Deep Dive: The Celebrity Problem**

Walking through the hybrid design for a celebrity with 50M followers:

1. Every account is tagged with a follower-count threshold (e.g., > 1M followers = "celebrity"), maintained by the Graph Service and cached.

2. When a celebrity tweets, the Fan-out Service checks this flag and **skips push fan-out entirely** - the tweet is written once to the Tweet Store and published to a Kafka topic, but no per-follower feed-cache writes happen. This avoids 50M individual writes.
3. When any user requests their home feed, the Feed Service does two things in parallel:
   - Reads the user's precomputed feed from the Feed Cache (built from fan-out-on-write, covering all the non-celebrity accounts they follow).
   - Separately checks the small list of celebrity accounts the user follows (this list is short - most users follow only a handful of celebrities even if they follow hundreds of regular accounts) and fetches their few most recent tweets directly from the Tweet Store/cache.
1. The Feed Service merges both streams by timestamp (or rank score) at read time and returns the combined feed to the client.
2. Because the celebrity's recent tweets are hot, they're kept in a shared, heavily-replicated read cache (not per-follower, just one cached copy of the tweet), so the read-time fan-in is cheap - it's one cache read per celebrity followed, not per follower of the celebrity.

**Best Choice:** Threshold-based hybrid fan-out with read-time merge for celebrity accounts, backed by a shared (not per-user) cache entry for celebrity tweets.

**Why?** The problem with naive push fan-out isn't fan-out itself, it's that cost scales with the celebrity's follower count (50M) rather than with the reader's follow count (~200). By flipping to pull-based delivery for celebrities, the cost of a celebrity tweet becomes O(1) at write time (one row written) and the cost per reader becomes O(small number of celebrities they follow) at read time, which is bounded and cheap because the tweet data itself is shared and cacheable, not duplicated per follower.

**Interview Answer:** "The moment I see a celebrity is about to fan out to 50M people, I treat that write as a signal to switch strategies. Instead of fanning the tweet out to 50M feed caches, I write it once and flag the account as high-fan-out. At read time, each follower's feed service does a cheap merge: their precomputed feed for regular follows, plus a quick lookup of the handful of celebrity accounts they follow, whose tweets are served from one shared cache entry. That turns an O(followers) write into an O(1) write and a small, bounded read-time cost per follower."

#### 6. Edge Cases: Deleting an Already-Fanned-Out Tweet, and Introducing Ranking

*Deleting a tweet already pushed to 2M feeds*

Physically deleting the tweet ID from 2 million feed-cache entries synchronously is just as expensive as the original fan-out and would block the delete API on millions of writes. Instead:

1. Perform a **soft delete**: mark the tweet as deleted (tombstoned) in the Tweet Store immediately - this is a single, fast write and is the source of truth.
2. Publish a `tweet_deleted` event to Kafka.
3. The Feed Service checks tweet status at **render time**: when assembling a feed, any tweet ID that resolves to a tombstoned tweet is silently filtered out of the response. This makes the delete "invisible" to users instantly, even before cache cleanup finishes, because the Tweet Store is always consulted (or its cached status) to hydrate tweet IDs into full tweet content.
4. Asynchronously, a background consumer of the `tweet_deleted` event lazily evicts the tweet ID from the 2M feed caches over time (or just lets it expire naturally via TTL/LRU eviction, since it will never render anyway once filtered). No synchronous fan-out-of-deletion is required.

This mirrors the original fan-out-on-write vs. read-time-filter trade-off: we make the *write* (delete) cheap and push the cost to a cheap, already-necessary read-time check.

*Introducing ranking without breaking the cache*

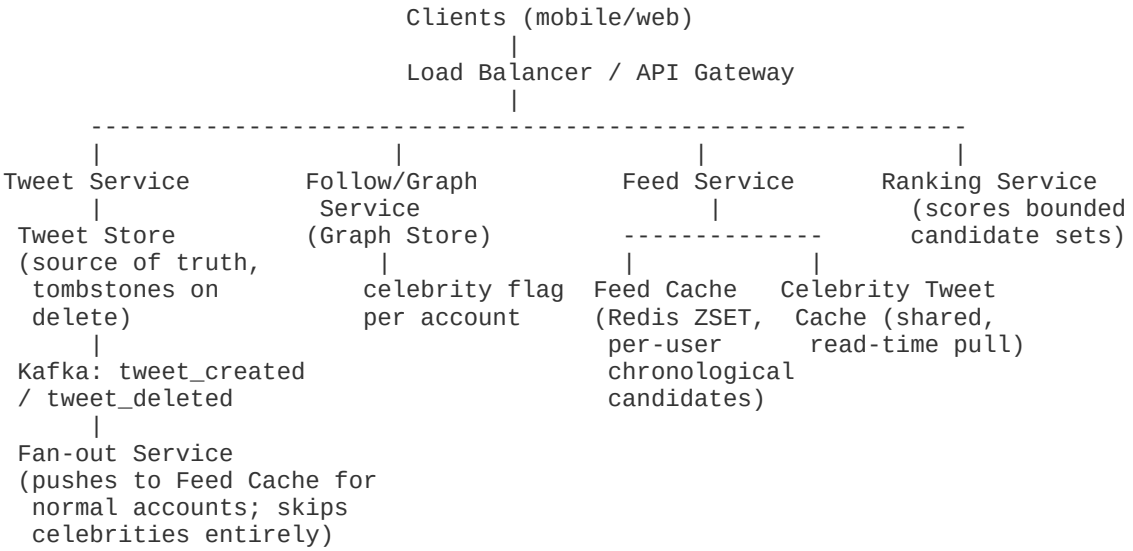
Naive chronological feeds are just a sorted list of tweet IDs, which is exactly what makes the fan-out cache cheap (append to a sorted set, e.g., Redis ZSET by timestamp). Ranking threatens this because "relevance" scores change over time (a tweet's engagement grows, then decays) and depend on the viewer, not just the poster, so you can't precompute one global order.

The way to introduce ranking without discarding the caching strategy:

- Keep the fan-out cache exactly as-is: a cheap, chronologically-sorted candidate list of tweet IDs per user (this is now a **candidate generation** step, not the final feed).
- Add a separate, **lightweight re-ranking step at read time**, applied only to the (bounded) set of candidates already pulled from cache - e.g., the most recent 200-500 tweet IDs, not the full history. A ranking service scores this small candidate set using signals like author affinity, recency decay, and engagement, then returns a re-ordered page.
- Because re-ranking only operates on a small, already-fetched candidate window, it doesn't require re-fanning-out anything or invalidating the cache - the cache still does the expensive part (assembling "which tweets are even eligible") cheaply, and ranking is a cheap in-memory sort/score pass on top.
- Cache the ranking model's inputs/signals (e.g., engagement counters) separately with short TTLs, decoupled from the feed candidate cache, so hot-tweet engagement updates don't churn the main feed cache.

**Interview Answer:** "For deletion, I don't try to reach into 2 million feed caches synchronously - I tombstone the tweet in the source of truth and filter it out at render time, then clean up the caches lazily in the background. For ranking, I keep the fan-out cache as a cheap candidate generator - a chronological list of tweet IDs - and add a separate re-ranking pass at read time that only reorders that bounded candidate set using fresher signals like engagement and affinity. That way ranking logic can evolve independently without touching or invalidating the underlying fan-out cache."

### 7. Final Architecture




### 8. Follow-up Questions

*Why not just use fan-out-on-read for everyone and skip the complexity of a hybrid model?*

Best Answer: Because our read:write ratio is roughly 5:1, and most accounts have modest follower counts - precomputing their feeds via push is cheap and makes the common-case read O(1). Pure pull would make every single feed load do a fan-in across ~200 followees, which doesn't scale at 23,000 reads/sec.

*How do you define the celebrity threshold, and does it need to be static?*

Best Answer: I'd use a follower-count threshold (e.g., 1M+) re-evaluated periodically rather than hardcoded, so growing accounts transition into the pull path automatically. The transition itself can be done lazily - stop pushing new tweets once flagged, old fanned-out tweets simply age out of feed caches naturally.

*What happens if the Fan-out Service crashes mid-fan-out for a large (but non-celebrity) account?*

Best Answer: Fan-out consumes from Kafka, so it's replayable - if a worker crashes partway through fanning a tweet out to followers, another consumer picks up the same tweet-created event and resumes/retries. Idempotent writes (checking if the tweet ID is already in a follower's feed set before adding) make retries safe.

*How would you keep the feed fresh if a user follows someone new - do they see that person's old tweets?*

Best Answer: On a new follow, I'd backfill the follower's feed cache with that account's last N tweets (a bounded, one-time fan-out), rather than waiting for the followee's next tweet. This keeps the "just followed" experience from feeling empty without requiring unbounded historical fan-out.

## Design a Ride-Sharing Service (like Uber)

---

## Q1. [Senior] Design the core of a ride-sharing service like Uber: a rider requests a ride and gets matched with a nearby available driver in near real time, at global scale.

**Answer:**

**1. Clarify Requirements**

*Functional Requirements*

- Riders can request a ride by specifying pickup and destination
- Drivers continuously broadcast their live location while online
- The system finds available drivers near the rider and assigns one
- Both parties see live trip status: matched, driver en route, arrived, in progress, completed
- Trip pricing (including surge/dynamic pricing) and payment on completion
- Cancellation handling on both sides
- Trip history and receipts

*Non-Functional Requirements*

- Match latency: rider should see a matched driver within a few seconds (target < 3–5 s end to end from request to match)
- High availability - the matching path cannot go down; a stalled system means no rides anywhere
- Low-latency, high-throughput ingestion of continuously moving driver locations
- Global scale - millions of concurrent drivers/riders across hundreds of cities with very different traffic patterns
- Strong consistency where money and trip state are involved (no double-booking a driver, no double-charging a rider)
- Eventual consistency is acceptable for raw location data (a few seconds of staleness doesn't matter for a driver who is already moving)
- Horizontally scalable, regionally isolated (an outage in one city should not take down another)

If the interviewer hasn't raised it yet, this is the moment to ask: *"What latency do riders expect between requesting and getting matched?"* - the answer (typically low single-digit seconds) is what forces the geospatial index and matching design in this write-up rather than a simpler polling/batch approach.

**2. Capacity Estimation**

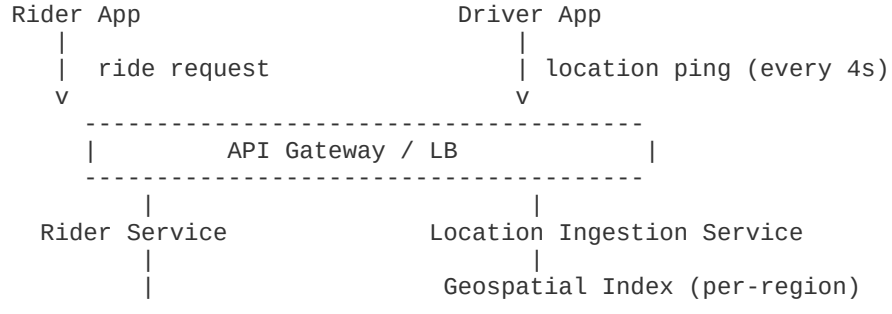
Assume the interviewer gives a global-scale prompt: **1M active drivers worldwide, each sending a location update every 4 seconds.**

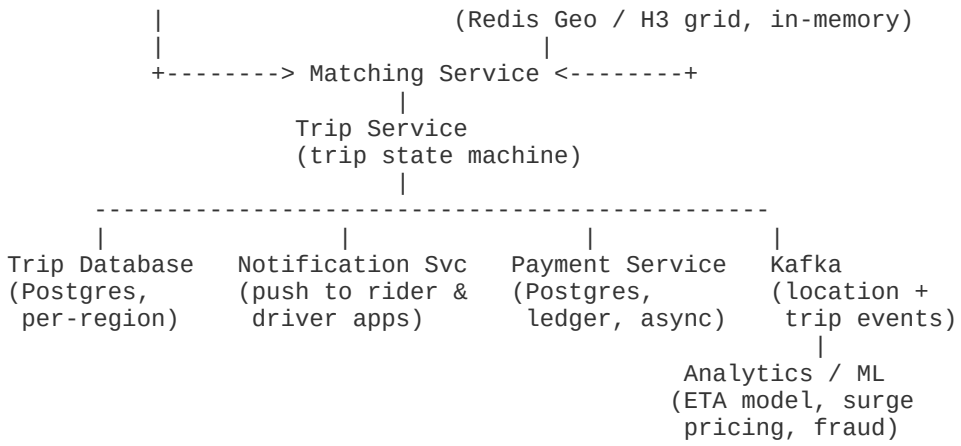
- Location-update ingestion rate: 1,000,000 drivers ÷ 4 s ≈ **250,000 updates/second** sustained, globally.
- Daily volume: 250,000/s × 86,400 s ≈ **~21.6 billion location events/day**.
- Payload size: driver_id, lat/lng, heading, speed, timestamp ≈ 150 bytes/update.
- Ingest bandwidth: 250,000 × 150 bytes ≈ 37.5 MB/s ≈ **~300 Mbps sustained inbound**, before compression.
- If every ping is also logged for analytics/ML (ETA models, surge prediction) via Kafka: 21.6B events/day × 150 bytes ≈ **~3.2 TB/day raw**, which is fine for a log-structured pipeline but would be wasteful to store in a hot geospatial index.
- Ride requests are a much smaller stream by comparison - even at 20M rides/day globally, that's ~230 requests/second average with sharp peaks (rush hour, weekend nights) of 5–10x that.

**Why global scale changes the approach vs. a single-city estimate:** a single-city design could plausibly get away with one geospatial index and one matching service, because a rider in downtown Chicago will only ever be matched with a driver also in Chicago - the search radius is a few kilometers. At global scale, the same locality property becomes the key sharding strategy rather than a simplifying assumption: **a rider in Chicago must never be considered against a driver in Mumbai**, so there is no reason for a single global data structure to hold both. Splitting the world into independent geo-shards (by city or by a hex/quadtree cell region) means:

- Each shard only holds the ~thousands-to-tens-of-thousands of drivers active in that metro at once, not the full 1M - the working set per index node stays small and fast.
- Traffic is naturally partitioned: NYC rush hour and Tokyo rush hour hit different shards at different times, so capacity can be provisioned and autoscaled per region instead of over-provisioning one monolithic global cluster for the sum of all peaks.
- Regional failure isolation - a data center or index outage in one region degrades only that region's rides.
- Regulatory/data-residency requirements (common in ride-sharing) are satisfied for free if a region's rider/driver/trip data never leaves that region.
- Cross-shard traffic effectively becomes zero on the hot path (matching), so 250K updates/sec globally decomposes into, e.g., a few thousand updates/sec per major metro shard - a problem any single Redis/in-memory node handles comfortably.

**3. High-Level Architecture**






**4. Deep Dive - Efficient Geospatial Lookup (finding drivers near a rider)**

The core query is: *given hundreds of thousands of constantly moving drivers, find all available ones within ~2 km of a point, fast, repeatedly*. A naive scan (compute distance to every driver) is O(n) per request and falls apart well before six figures of drivers.

*Options:*

- **Geohashing** - encode lat/lng into a base32 string where shared prefixes mean spatial proximity. Nearby drivers can be found with prefix matching / range queries. Simple, and Redis has this built in (`GEOADD`, `GEOSEARCH`). Weakness: geohash cells are rectangular and their size varies with latitude, and points near a cell boundary can be geographically close but hash very differently, requiring the classic "search neighboring cells too" workaround.
- **Quadtree** - recursively subdivide space into quadrants until each leaf holds a small number of points. Adapts naturally to density: dense downtown areas get finer subdivision than sparse suburbs. Weakness: more complex to keep balanced under constant insert/update/delete of moving drivers (essentially every 4 seconds per driver), and harder to shard/distribute cleanly than a flat hash scheme.
- **S2 / H3 hexagonal hierarchical index** - tessellate the sphere into cells (S2 uses a projected cube, Uber's own open-sourced H3 uses hexagons). Hexagons give more uniform neighbor distances than square geohash cells (every neighbor is equidistant, no "closer diagonal" distortion), support multiple resolutions for zooming in/out of search radius, and compress well to fixed-width integer cell IDs that are trivial to shard by.

**Best Choice:** Start with **Redis GEO (geohash-backed sorted sets) per regional shard** for the MVP, and note that this is exactly the kind of problem that motivates evolving toward a **hexagonal hierarchical index like Uber's own H3** as the system matures.

**Why?** Redis GEO gives `O(log n + m)` radius queries out of the box, keeps the entire regional driver set in memory (a few tens of thousands of drivers per city easily fits), and updating a driver's position is a cheap `GEOADD` overwrite - which matters because every driver rewrites their entry every 4 seconds. It's also operationally simple: no custom tree-balancing code to maintain. The upgrade path to H3 exists because hex cells give more uniform distance semantics (useful for consistent "expand my search ring outward" logic) and let the matching service reason about drivers using fixed integer cell IDs, which shard and cache trivially and support pre-computed multi-resolution lookups (e.g., "check the rider's cell, then its immediate ring, then the next ring" instead of an arbitrary radius query).

**Interview Answer:** "I'd back the live driver-search with an in-memory geospatial index - Redis's geo commands are a pragmatic starting point since they give sub-millisecond radius queries and cheap position updates, which matters because every one of a few hundred thousand drivers is rewriting its location every few seconds. I'd shard this index by region so no single node ever holds more than one metro's worth of

drivers. As the system matures I'd move toward a hexagonal hierarchical grid like Uber's H3, because uniform-distance hex cells and fixed-width cell IDs make ring-expansion search and cross-service sharding cleaner than raw geohash strings."

**5. Deep Dive - Driver Location Storage: Hot Path vs. Durable Store**

*Options:*

- **Write every ping straight to a durable relational/columnar store** - simple mental model, but 250K writes/sec of transient data to a durable, consistent store is expensive and unnecessary: the value of a location ping decays to zero within seconds of the next one arriving.
- **In-memory store only (Redis), no durability** - extremely fast, but a node restart loses all current positions; acceptable *if* drivers re-report within 4 seconds anyway, since state self-heals almost immediately.
- **Hybrid: Redis for current position, Kafka stream for the full historical feed, cold storage/analytics DB for anything needing history (ETA model training, trip replay, fraud/audit)** - keeps the hot path fast while not discarding the data entirely.

**Best Choice:** Hybrid - Redis (or equivalent in-memory geo index) as the *source of truth for "where is this driver right now,"* with every ping also fanned out to Kafka for asynchronous, best-effort persistence and downstream analytics.

**Why?** This is a direct CAP trade-off applied deliberately: driver location is a field where we choose **availability and low latency over strict consistency**. If a Redis node in a region briefly partitions or loses a few updates, the cost is a driver's shown position being a few seconds stale - self-correcting on the next ping. That's a far better trade than making location writes wait on a consistent, replicated durable store and risking ingestion backlog or increased match latency during exactly the traffic spikes (surge periods) when the system is under the most pressure.

**Interview Answer:** "Driver location is high-volume, high-churn, and self-correcting, so I treat it as an availability-favoring dataset - an in-memory geo store is authoritative for 'right now,' and I don't pay durability costs on the hot path. I still stream every ping through Kafka asynchronously so we don't lose the data entirely; it feeds ETA models, surge detection, and auditing without being on the critical path of a match."

**6. Deep Dive - Matching Algorithm: Beyond "Always Pick the Closest Driver"**

Once the geospatial index returns a candidate set of nearby available drivers, the naive rule - assign the physically closest one - sounds obviously correct but breaks down quickly:

- "Closest" by straight-line distance ignores road network and traffic; a driver 400 m away across a river with no bridge can have a longer ETA than one 1.5 km away on a straight road.
- It ignores the driver's heading - a driver 300 m away but driving away from the rider is a worse match than one 600 m away driving toward them.
- It's short-sighted for the marketplace as a whole: greedily grabbing the closest driver for every request one at a time can leave a cluster of riders in one area all matched to distant drivers because the nearby ones got claimed a half-second earlier by requests that arrived first, even if a globally better assignment existed.
- It ignores driver-side fairness/utilization - always sending rides to the same "closest" pocket of drivers burns them out while others nearby sit idle, hurting long-term driver retention.
- It ignores acceptance probability - a driver who has declined the last three requests is a worse bet than a slightly farther one who reliably accepts, since a decline costs a full extra round trip of latency.

*Options:*

- **Naive closest-driver, one request at a time** - simplest, low latency per request, but marketplace-inefficient and prone to the failure modes above, especially at scale.
- **Score-based single-request matching** - instead of raw distance, rank candidates by a weighted score combining estimated ETA (via a routing/traffic-aware service), driver heading alignment, driver idle time (fairness), and historical acceptance rate. Still assigns one rider at a time but picks a materially better driver.
- **Batch/windowed matching** - collect ride requests and available drivers over a short window (e.g., every 2–4 seconds) within a geo-cell, then solve a bipartite assignment, matching two separate groups (requests and drivers) to each other optimally, using a Hungarian-style algorithm or a faster greedy approximation, that optimizes total system ETA rather than each request in isolation. Higher quality matches, at the cost of adding up to one window's worth of latency to every request.

**Best Choice:** Score-based matching as the default, with short-window batching layered on top in dense, high-request-rate cells (e.g., downtown during rush hour), falling back to immediate score-based single matching in sparse areas where waiting for a batch window would just add latency for no benefit.

**Why?** This optimizes for the metric that actually matters to the business and the user - expected wait time and overall trip reliability - rather than a proxy (raw distance) that correlates with it poorly under real road networks and marketplace dynamics. Batching only pays off where there's enough simultaneous supply and demand in a cell to produce a meaningfully better joint assignment; in low-density areas it's pure added latency, so the system should switch strategies based on local request density rather than using one global rule.

**Interview Answer:** "I wouldn't pick by raw distance - I'd score nearby drivers using traffic-aware ETA, heading, idle-time fairness, and historical acceptance rate, and assign the highest-scoring driver. In dense areas with many simultaneous requests, I'd batch matching over a very short window - a couple of seconds - and solve it as an assignment problem so the whole cluster of riders and drivers gets a better joint outcome than greedily matching one at a time. In low-density areas I'd skip batching entirely since there's nothing to optimize against and it would only add latency."

**7. Deep Dive - Regional Sharding for Global Scale**

Tying back to the capacity estimate: with 1M drivers worldwide, the system is architected from day one as a set of largely independent **regional deployments** (by metro area or country), not a single global cluster.

*Options:*

- **Single global deployment** - simplest to operate as "one system," but every component (geo index, matching, trip DB) has to handle the sum of world peak traffic, cross-region latency hurts the match-latency SLA for anyone far from the primary data center, and one bad deploy or outage is global.
- **Fully independent per-region stacks** - best isolation and lowest latency (a rider's request never leaves their region), but duplicates operational overhead and complicates any feature that's inherently cross-region (e.g., a driver's trip history when they travel, global fraud detection, corporate billing across countries).
- **Regional shards for the hot path (location, matching, trip execution) + a shared global control plane for account data, payments ledger rollups, and analytics** - the pragmatic middle ground.

**Best Choice:** Regional shards for everything on the real-time matching/trip path, with a thin global layer for identity, payment records, and cross-region reporting.

**Why?** Matching is inherently local - nothing is lost by keeping it region-scoped - so put the sharding boundary exactly there and get independent scaling, independent failure domains, and lower latency for free. Reserve the cost of cross-region coordination for the few things that truly need a global view (a rider's

account should work if they fly to another city; a company should get one consolidated invoice), where consistency matters more than latency anyway.

**Interview Answer:** "I'd shard the real-time path - location ingestion, the geo index, and matching - by region, since a ride is never matched across cities and there's no reason to pay cross-region latency or share blast radius on that path. I'd keep a smaller global layer for things that are inherently cross-region, like a rider's identity and payment history, where I'd accept extra coordination cost because correctness matters more there than shaving milliseconds."

**8. Deep Dive - Trip and Payment Data: Why This Half of the System Needs Strong Consistency**

In contrast to location data, trip creation and payment records cannot tolerate the same looseness.

*Options:*

- **NoSQL/eventually-consistent store for trips and payments** - great write throughput, but risks a rider being charged twice, a trip being marked complete on one replica and still active on another, or a payout being double-counted - all directly costs money and trust.
- **Relational database (e.g., PostgreSQL) with ACID transactions for trip state and the payment ledger, per region** - slower writes than a pure KV/wide-column store, but guarantees exactly the invariants that matter here: a trip has exactly one terminal state, a charge is applied exactly once.

**Best Choice:** PostgreSQL (or an equivalent strongly consistent relational store) per region for trip state transitions and the payment ledger; Kafka + a downstream analytics store (e.g., a columnar warehouse) for everything that's append-only and doesn't need transactional guarantees (ping history, event logs, aggregated metrics).

**Why?** Trip and payment volume (tens of thousands of transitions per second at global peak, not hundreds of thousands) is well within what a well-sharded relational database can handle, and the cost of a consistency bug here - double-charging a rider or losing a completed trip's record - is far higher than the cost of the extra write latency. **This is the deliberate mirror image of the location-data decision: same system, opposite point on the CAP spectrum, chosen because the data has opposite tolerance for staleness or loss.**

**Interview Answer:** "Trip state and payments are where I want strong consistency even at the cost of latency - I'd use a relational database with real transactions per region so a trip can't end up in two terminal states and a rider can't be double-charged. That's the opposite trade-off from the location data, and that's intentional: the two halves of this system have very different tolerance for staleness, so they shouldn't share a consistency model just because they share a product."

**9. Edge Cases: Double-Matching Race Condition and Driver Connectivity Loss**

*Two riders simultaneously matched to the same driver.* This happens when two matching workers (or two threads within one) read the same driver as "available" from the geo index at nearly the same instant and both attempt to assign them, before either write reflects the other's decision. The fix is to make **claiming a driver an atomic, single-writer operation**, not a read-then-write:

- The instant a driver is selected as a match candidate, the matching service performs a **conditional/atomic claim** against the driver's state - e.g., a Redis `SETNX`/compare-and-swap on `driver:{id}:status`, or a single-row conditional `UPDATE drivers SET status='PENDING_ASSIGNMENT' WHERE id=? AND status='AVAILABLE'` in the durable trip store, checking the affected row count.
- Whichever request's claim succeeds proceeds to create the trip and notify that driver; the loser immediately falls back to the next-best candidate from its original scored candidate list (no need to redo the full geo query).
- The driver's status only moves out of `PENDING_ASSIGNMENT` once they accept (→ `ON_TRIP`) or a timeout/decline returns them to `AVAILABLE`. This turns "who gets the driver" into a strictly

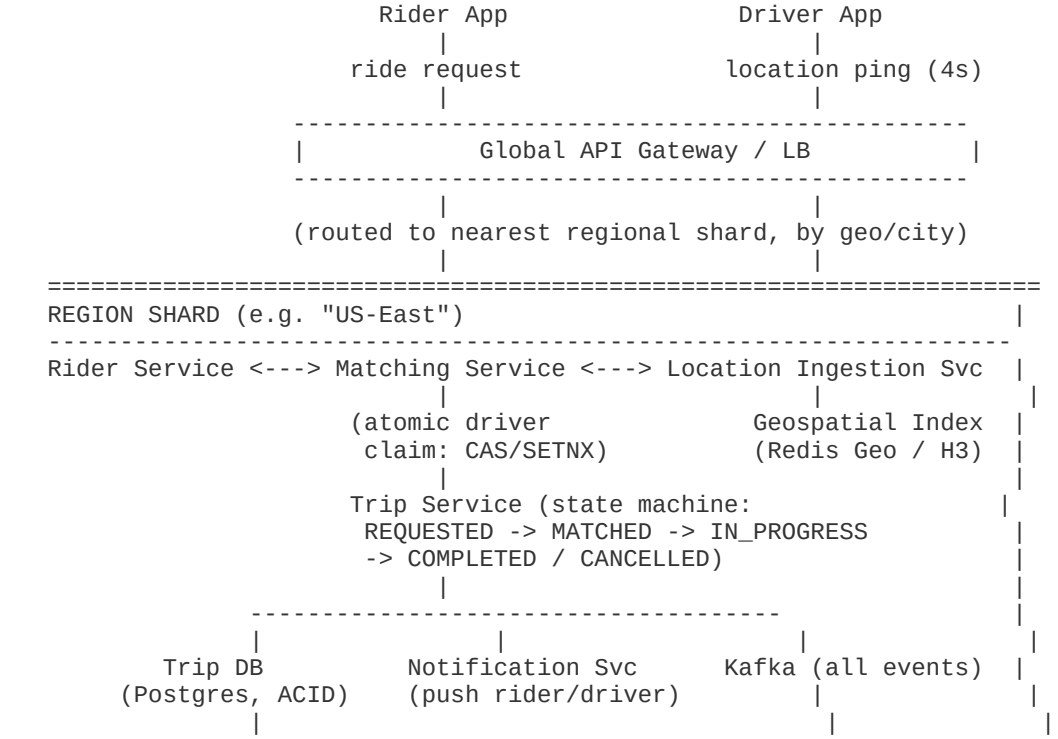
serialized decision even though the search that produced the candidate list was done in parallel by multiple matching workers.

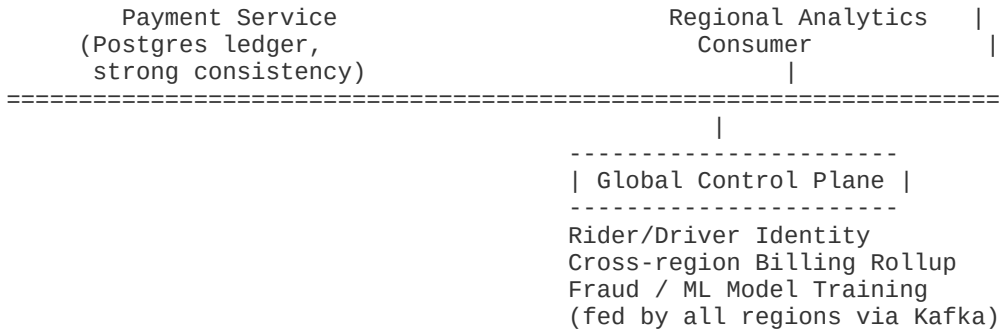
- This is exactly why driver status needs a stronger consistency guarantee than raw location - it's a small, low-volume field (one write per assignment, not one every 4 seconds), so paying for a conditional write here is cheap, while it would be wasteful to demand the same guarantee for the high-volume location stream.

*Driver's phone loses connectivity mid-ride.* The trip state machine has to distinguish "driver went briefly offline" from "trip needs intervention," using time-based thresholds rather than a hard binary:

- The last known location and trip state are retained (not discarded) the moment pings stop - the trip stays `IN_PROGRESS` with a "stale location" flag rather than being cancelled outright, since momentary connectivity loss (a tunnel, a dead zone) is common and self-resolving.
- A grace-period timer (e.g., 30–60 seconds without a ping) escalates the trip to a degraded-but-active state: the rider app shows "driver's location is temporarily unavailable" instead of a frozen or misleading live position.
- If connectivity resumes before the trip naturally completes, the driver's app resyncs its buffered pings/trip events and the system reconciles - no data is lost because pings are timestamped at generation, not at ingestion, so a burst of "late" pings on reconnect still lands in the correct order.
- If connectivity does not resume by a longer timeout (e.g., several minutes), the system falls back to safety and support workflows: attempt to reach the rider and driver via SMS/phone as an out-of-band channel, and if the trip must be force-completed for billing purposes, compute the fare from the last confirmed route/ETA estimate rather than leaving the trip in limbo indefinitely, flagging it for support review rather than silently closing it.
- Because trip state lives in the strongly consistent store (Section 8) rather than only in memory on a mobile device, the backend's view of "this trip is in progress, last confirmed at location X, at time T" survives the disconnection - the durable trip record is the source of truth that the flaky client eventually resyncs against, not the other way around.

**10. Final Architecture**






**11. Follow-up Questions**

*Why not use a single global geospatial index instead of sharding by region?*

Best Answer: Matching is inherently local - a rider is never matched to a driver in another city - so a global index only adds size and latency without adding any value. Sharding by region keeps each index's working set small, lets regions scale and fail independently, and often satisfies data-residency requirements as a side effect.

*Why is Redis acceptable for something as important as driver location, given it's not durable by default?*

Best Answer: Because location data is inherently self-healing - every driver overwrites their position every 4 seconds regardless of what happened to the last write. Losing a few seconds of position data to a transient node issue costs almost nothing, so it's a deliberate choice to favor availability and speed over durability for this one dataset, while keeping trip and payment data in a strongly consistent store.

*How would you extend the matching algorithm to account for surge pricing?*

Best Answer: Surge is a signal into the same scoring function - computed per geo-cell from the live ratio of open ride requests to available drivers in that cell (fed by the same Kafka stream used for analytics). It affects price shown to the rider and can also nudge idle drivers in adjacent cells toward the surging cell, but it doesn't change the core claim-and-match mechanics described above.

*How do you test that the double-matching fix actually prevents duplicate assignments under load?*

Best Answer: Load-test the matching service with many concurrent workers racing to claim the same small pool of drivers, and assert on the atomic claim's return value (rows affected / CAS success) rather than trusting application-level locking; also add a database-level uniqueness constraint (e.g., a partial unique index ensuring a driver can have at most one active trip) as a second line of defense in case the application logic has a bug.

*If you had to say in one sentence how CAP theorem shapes this whole design, what would you say?*

Best Answer: **This isn't one CAP decision, it's several - the same system deliberately picks different points on the CAP spectrum for different data: driver location favors availability and tolerates staleness because it's high-volume and self-correcting every few seconds, while trip assignment and payment/billing data favor consistency because a double-booked driver or a double-charged rider is a correctness failure, not a rounding error, so it's worth paying the extra latency there.**

## Design a Movie Ticket Booking System (like BookMyShow), Guaranteeing No Two People Get the Same Seat

---

## Q1. [Mid–Senior] Design a movie or event ticket booking platform where thousands of people can try to book seats for the same popular show at once, and no two people are ever sold the same seat.

**Answer:**

This looks like a simple booking form, but the entire interview lives in one narrow, brutal race condition: a blockbuster's opening show might have 200 to 500 seats, and the moment booking opens, thousands of people hit "book" on the same handful of good seats within the same second. Get the concurrency control wrong and two paying customers are both confirmed for seat G12, a product-breaking, refund-generating failure. Get it right, and everything else, search, payments, notifications, is comparatively easy.

**1. Clarify Requirements**

*Functional Requirements*

- A user can search for a show by movie, city, date, and time.
- A user can view a real-time seating map for a specific show.
- A user can select one or more seats and hold them temporarily while completing payment.
- A booking is confirmed only after payment succeeds.
- A held-but-unpaid seat is released automatically after a short timeout.
- A user can cancel a confirmed booking within a cancellation window.

*Non-Functional Requirements*

- Absolute correctness: two users must never both be confirmed for the same seat, under any load.
- Low-latency seat-map reads, since thousands of users watch the same seating grid at once.
- High availability at the exact moment a blockbuster's booking window opens, the system's worst-case traffic spike.
- Graceful handling of payment failures and timeouts, without silently losing or double-allocating a seat.

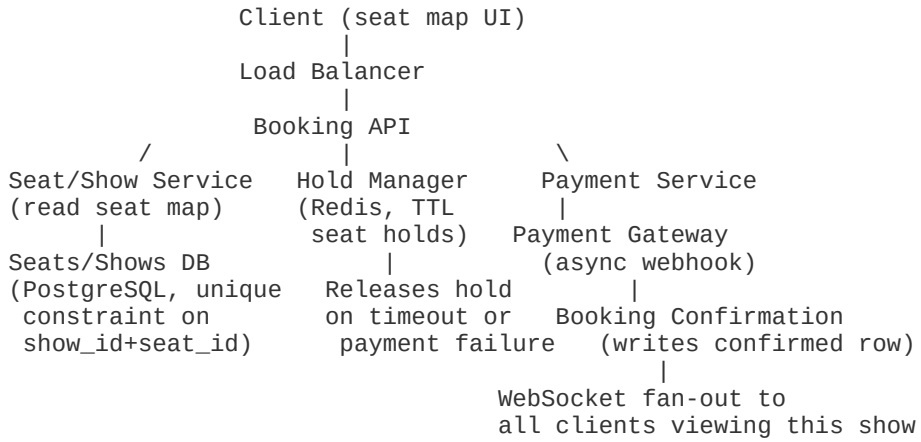
**2. Capacity Estimation**

*Assumptions*

- A single popular show has 200 to 500 seats.
- 5,000 concurrent users refreshing or booking against that one show when tickets go live.
- 10,000 shows/day nationally across a chain, each with its own independent seat map.

The interesting number here is not total data volume, a database can hold ten thousand shows' worth of seat rows without strain, it is contention density: potentially thousands of write attempts landing on the same 200 to 500 rows within the same few seconds. That is the opposite profile from a URL shortener's huge scale with low contention per key; this is low data volume with **extreme contention per key**.

**3. High-Level Architecture**



A seat-map read goes to the Seat/Show Service (cached, WebSocket-updated); a seat selection goes to the Hold Manager, which is where the entire correctness story lives; a completed payment converts a hold into a row in the database, which is the actual source of truth.

**4. Preventing Double-Booking (the core design decision)**

Approach A - Optimistic locking with a version column: read the seat's current version, then update it only if the version hasn't changed. Pros: no long-held locks. Cons: under this exact kind of extreme contention, most attempts fail and retry at once, a thundering herd at the worst possible moment.

Approach B - Pessimistic row-level locking (SELECT ... FOR UPDATE): a transaction locks the seat row before deciding to allocate it; competing transactions block until it commits or rolls back. Pros: simple, DB-guaranteed correctness. Cons: a slow or stalled client can hold that lock and stall out everyone else waiting on the same seat.

Approach C - A short-lived distributed hold in Redis (an atomic SETNX-style claim with a TTL): the moment a user selects a seat, they atomically claim a hold key such as seat:show123:G12 for a few minutes. If it already exists, the seat is unavailable to everyone else. Success converts the hold into a confirmed, database-backed booking; expiry or payment failure releases it automatically.

**Best Choice:** a short-lived Redis hold (Approach C), backed by a final database-level uniqueness guarantee at confirmation time.

It moves the high-contention "who gets dibs on this seat right now" decision into an in-memory store built for exactly this kind of atomic, low-latency claim, instead of pushing thousands of competing transactions onto the relational database's lock manager. The TTL solves the "user closed the tab mid-payment" problem automatically, with no cleanup job needed. Crucially, the Redis layer does not replace the database: the final confirmed booking still goes through a unique constraint on (show_id, seat_id), so even if the hold layer had a bug or a race, the database **physically cannot store two confirmed bookings for the same seat**.

**Interview Answer:** "I'd use a short-lived Redis hold, keyed by show and seat ID, that a user acquires atomically the moment they select a seat, with a TTL of a few minutes so an abandoned booking releases itself. If payment completes, I write the confirmed booking into the database behind a unique constraint on show ID and seat ID, so even if something raced or the Redis layer misbehaved, the database physically cannot end up with two confirmed bookings for the same seat."

**5. Why Not Just Rely on the Database Alone?**

A pessimistic SELECT FOR UPDATE would technically also guarantee correctness. The reason to still add a hold in front of it is user experience and load, not correctness: without one, a seat looks available right up until someone actually tries to pay, so ten different users could all reach the payment page for the same seat before the database rejects nine of them, a terrible experience at the exact moment you want to convert a sale. A visible hold lets the seat map grey out for everyone else the instant one person selects it, not just at the moment of final confirmation.

**The database is what guarantees correctness; the hold is what makes that correctness visible to users in real time instead of surfacing as a payment-page rejection.**

**6. Handling Payment Failure and Timeout (edge case)**

Two failure paths need to release a seat: the TTL expiring on its own when a user walks away, and an explicit payment failure or webhook. Both should route through the same idempotent release-hold function, rather than two separately-implemented paths that can drift out of sync.

**Best Choice:** one idempotent release-hold function, called from both the TTL expiry path and the payment-failure webhook.

Two different failure sources hitting two different pieces of release logic is exactly how seats end up stuck in limbo, neither booked nor bookable, after an incident.

**7. Scaling the Seat Map Read**

Thousands of users watching the same 200 to 500 seat show is a read-heavy, highly repeated query pattern.

**Best Choice:** push seat-state deltas to connected clients over WebSocket, instead of having every client poll.

Polling five thousand clients against the database or cache every couple of seconds multiplies load pointlessly, since almost nothing changed between polls; pushing only the seats that actually changed state keeps update volume proportional to real bookings, not to the number of people watching.

**Interview Answer:** "Rather than have every client repeatedly ask what's still open, I'd hold seat state in Redis and push incremental updates over a WebSocket connection whenever a seat's state actually changes, so five thousand people watching the same seat map costs one write and one fan-out, not five thousand polling queries."

**8. Follow-up Questions**

*What happens if the Redis hold succeeds but the process crashes before the database booking write completes?*

Best Answer: The hold's TTL still expires normally, since Redis doesn't know or care that the client crashed, so the seat becomes bookable again within minutes and the user simply tries again. Because the database write is the only source of truth for a confirmed sale, a crash between the two steps never produces a booking that looks confirmed but wasn't actually paid, at worst it produces a temporarily stuck hold that self-heals.

*Could you use the database's own row lock instead of Redis and skip the extra moving part?*

Best Answer: Yes for correctness, no for user experience and load. A database lock only protects the moment of the transaction, not the multi-minute window between a seat looking available and payment completing, so without a separate hold you would need to keep a live database transaction open for the entire payment flow, which does not scale and ties up connections exactly when load is highest.

*How would this change for a general-admission event with no assigned seats, just a total capacity number?*

Best Answer: That becomes a pure counter problem, not a per-seat lock problem. An atomic decrement, such as Redis DECR or a database row updated with UPDATE ... WHERE remaining > 0, reserves one unit of capacity, and the same hold-then-confirm pattern still applies during payment.

## Design "Continue Watching": Resume Playback From the Same Scene Across Devices

---

## Q1. [Mid] Design the feature that lets a user pause a show on their TV and resume from the exact same scene on their phone a few minutes later, for hundreds of millions of users across many devices each.

**Answer:**

This looks like "store a timestamp per video," and at its core it is, but the interview is really about two things: making an extremely high-frequency, low-value write, a playback position updated every few seconds for every actively-watching device on earth, cheap enough to sustain, and making the read consistent enough that switching devices mid-show never rewinds or skips the viewer.

**1. Clarify Requirements**

*Functional Requirements*

- Track playback position per user, per video/episode.
- Resume from the last saved position when the user reopens that title on any device.
- Support pause, resume, and seeking, and mark a title complete when finished.
- Show a "Continue Watching" list of in-progress titles per user across all devices.

*Non-Functional Requirements*

- Extremely high write volume: potentially one position update every 5 to 10 seconds per actively streaming session, across hundreds of millions of concurrent streams globally at peak.
- Reads must feel consistent: switching devices seconds after pausing must show the real position, not a stale one.
- Writes must be cheap and loss-tolerant: losing the last few seconds of a position update on a crash is fine, losing hours of it is not.
- Durable enough to survive a restart, but not durable enough to need financial-ledger-grade guarantees.

**2. Capacity Estimation**

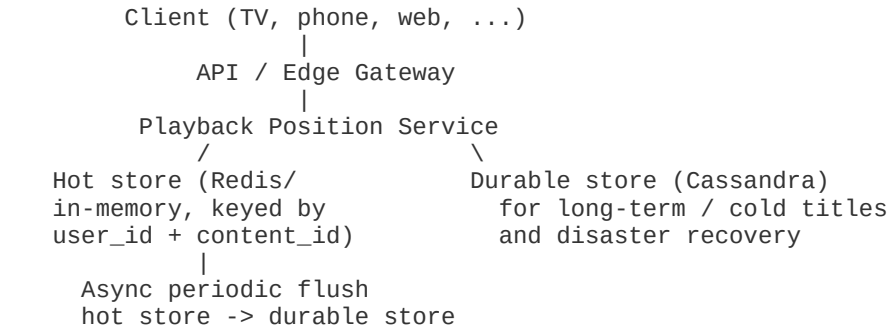
*Assumptions*

- 250 million subscribers, 20% concurrently streaming at peak = 50 million concurrent streams.
- One position update every 10 seconds per stream.

Write QPS = 50,000,000 concurrent streams divided by a 10-second update interval is approximately **5,000,000 writes/sec** at global peak, for a single integer field.

That number alone **rules out a traditional relational database as the primary store** for this data: 5 million small writes per second is squarely in the territory of an in-memory or log-structured store, not a database doing full ACID transactions on every write.

**3. High-Level Architecture**




Writes land in the hot, in-memory-first store; reads on device switch check the hot store first and fall back to the durable store, which is refreshed asynchronously and serves as the long-term backstop, not the primary write target.

**4. Where to Store the Position (the core decision)**

Approach A - Write straight to a relational database on every position update. Pros: simple, strongly consistent. Cons: 5 million writes/sec of a single integer would need a database fleet dedicated to nothing else, wildly disproportionate to the value of the data.

Approach B - Write only on explicit pause, stop, or exit events, not continuously. Pros: orders of magnitude fewer writes. Cons: a crash, force-quit, or dead battery between the last explicit event and now loses the position entirely, resuming from minutes or hours earlier.

Approach C - Buffer position updates client-side and batch-flush periodically, every 10 to 30 seconds, to a fast key-value store, plus a final flush on any explicit stop or app-close event. Pros: bounds both write volume and the worst-case data loss window. Cons: needs a store built for very high write throughput of small values.

**Best Choice:** Approach C, periodic client-side batching flushed to an in-memory-first store like Redis or a wide-column store like Cassandra, keyed by (user_id, content_id).

This caps the blast radius of any lost update to a few seconds of video, imperceptible to a viewer, while cutting write volume by roughly the batching factor. Redis and Cassandra are both built to absorb millions of small, mostly-overwrite writes per second, which a relational database, optimized for transactional integrity, is not.

**Interview Answer:** "I would not write on every second of playback. I'd have the client batch position updates and flush every ten to thirty seconds, plus immediately on pause or app close, into a store built for extremely high write throughput of small key-value pairs, like Redis or Cassandra, keyed by user and content ID. Worst case, a crash loses a few seconds nobody notices; best case, I've cut write volume by an order of magnitude versus writing every second."

**5. Reading the Position on a Different Device (the consistency question)**

The tricky part isn't the write, it's the read: the user pauses on the TV and opens the same show on their phone twenty seconds later. If the phone reads a stale replica that hasn't caught up, it resumes from the wrong spot, which is immediately and visibly broken in a way a stale follower count never is.

**Best Choice:** read the position from the primary/leader copy for this specific lookup, not an eventually-consistent replica, even though the write path elsewhere favors throughput over strict consistency.

This read happens rarely, only on app open or device switch, and is latency-insensitive enough that paying a small extra cost for a strongly consistent read is worth it, since this is one of the few reads in the whole design where staleness is immediately user-visible.

**Writes are optimized purely for throughput because there are millions of them a second; the rare, low-frequency read on device switch can afford to be the one place that pays for strong consistency.**

**Interview Answer:** "Writes are optimized for throughput because there are millions of them a second. Reads are rare, only on app open or device switch, so for that specific read I'd rather pay a small latency cost and read from the primary copy, to guarantee the user actually sees their real last position instead of risking a stale replica read that visibly rewinds or skips their show."

**6. Handling Rapid Scrubbing (edge case)**

A user dragging the progress bar can fire many position-change events within a couple of seconds.

**Best Choice:** debounce client-side, sending the position a short delay after scrubbing stops, plus a final flush on pause.

Sending every intermediate scrub position wastes writes on values immediately superseded by the next one; only the position where the user actually stops matters.

**7. The "Continue Watching" Row (a different read pattern)**

Beyond resuming one title, the home screen needs a list of all in-progress titles for a user, sorted by recency.

**Best Choice:** maintain a separate, small, explicitly-updated list, such as a capped sorted set per user updated on meaningful events like pause or stop, rather than deriving it by scanning every position record that user has.

The high-frequency position store is optimized for a single point lookup by user and content ID; "give me this user's twenty most recent in-progress titles" is a different access pattern that deserves its own purpose-built structure instead of forcing one store to serve two very different query shapes well.

**8. Follow-up Questions**

*What happens if the same account is watching two different episodes on two devices at once?*

Best Answer: Position is keyed by user ID and content ID together, not just user ID, so two different episodes get two independent position records with no conflict. If it's genuinely the same episode on two devices at the exact same time, last-write-wins on timestamp is an acceptable simplification, since there's no meaningful correct answer to two simultaneous positions for one viewer.

*How would you avoid this store becoming a single point of failure for the entire streaming product?*

Best Answer: Losing the position service should degrade gracefully, not break streaming. If the read fails, resume from the beginning or from a stale cached value rather than blocking playback, since watching the video is the core product and remembering exactly where you left off is a valuable but non-blocking enhancement on top of it.

*Would this design change for a live-streaming or live-sports feature instead of on-demand video?*

Best Answer: There's no position to resume for a live stream, you're either watching live or you've missed it, so this entire feature is specific to on-demand video and simply would not apply there.

---

**If you remember nothing else from this chapter:** *the case study that goes well isn't the one with zero mistakes, it's the one where you catch and fix your own mistake the moment it's pointed out.*

# Chapter 17: Security & Authentication

Security questions are easy to let slide in a system design interview because they don't come up unless someone asks directly, and that's exactly the risk: you can design a technically elegant system that's quietly insecure. This chapter starts from fundamentals like JWTs and OAuth2 so you can build a solid baseline before tackling harder trade-offs.

**War Story:** *A JWT signing secret committed to a public repository three years ago and never rotated meant one stale git history search was all a security researcher needed to mint valid admin tokens.*

**Avoid:** treating a token, secret, or credential as safe forever once issued, with no rotation story at all

**Aim For:** raising secret rotation and revocation as a first-class design concern, not an afterthought

## Q1. [Junior] What's the difference between authentication and authorization, and why do interviewers care that you keep them straight?

**Answer:**

**Authentication** answers "who are you," proving identity, typically with a username and password, a fingerprint, or a token that was issued after a successful login. For example, logging into an app with a password proves you are really Priya, that's authentication, while the app then checking whether Priya is allowed to view a specific patient's medical record is authorization, and mixing these up is how one user ends up able to see another user's private data.

**Authorization** answers "what are you allowed to do." Once the system knows who you are, it decides whether you can read a file, call an API, or see another user's data.

Why it matters: many production bugs and real vulnerabilities are exactly this confusion, checking that a request has a valid token (authentication) but forgetting to check whether this token's owner is allowed to touch this specific resource (authorization). **Almost every real-world "broken access control" vulnerability is really an authentication check standing in for a missing authorization check.**

- Authentication happens once per session or token: "prove you are Sumit."
- Authorization happens on every sensitive action: "is Sumit allowed to delete this invoice?"

**Interview Answer:**
I think of authentication as proving identity, like showing an ID badge at the door, and authorization as what that badge lets you do once inside, like which floors it unlocks. In interviews I bring this up early because a lot of real security bugs come from teams verifying that a token is valid but never checking whether the token's owner is actually allowed to perform the specific action or touch the specific resource being requested, which is an authorization gap disguised as an authentication success.

## Q2. [Junior] What is a JWT, and what actually lives inside the header, payload, and signature?

**Answer:**

A JWT (JSON Web Token) is a compact, URL-safe string used to pass identity and claims between two parties, most often from an authentication server to a client, and then from the client to an API. It looks like three dot-separated segments, `header.payload.signature`.

**Header**: a small JSON object saying which algorithm was used to sign the token, for example `{"alg":"HS256","typ":"JWT"}`.

**Payload**: a JSON object of "claims", things like user id, role, expiry time (`exp`), and issuer (`iss`).

**Signature**: a cryptographic value computed over the header and payload using a secret or private key, which is what lets a verifier trust the claims.

The header and payload are only Base64URL-encoded, not encrypted. Encoding is not encryption, it is a reversible text transformation with no key involved, so anyone with the token can decode and read it in seconds on a site like jwt.io. **The only thing standing between a user and a forged token is the signature, not the encoding of the payload.** This is why you should never put secrets like passwords inside a JWT payload. As a concrete example, a token starting with `eyJhbGciOiJIUzI1NiJ9...` decodes its middle segment into plain JSON like `{"sub":"1234","role":"user"}`, readable by anyone who copies the token.

**Interview Answer:**
A JWT is just three Base64URL-encoded pieces joined by dots: a header describing the signing algorithm, a payload of claims like user id and role, and a signature. The key thing I always clarify is that Base64 is encoding, not encryption, so anyone can decode the payload and read it, they just can't change it without invalidating the signature, since they don't have the signing key. That distinction between "readable" and "forgeable" trips up a lot of people early in their security learning, so I like to state it explicitly.

---

## Q3. [Junior] If anyone can read and decode a JWT's payload, how does the system stop someone from just editing their role to "admin" and sending it back?

**Answer:**

The signature is what prevents tampering, not the encoding. When the authorization server issues a token, it computes the signature over the header and payload using a secret (HS256) or a private key (RS256), and that signature travels as the third part of the token.

When a request comes in, the server:

1. Splits the token into header, payload, and signature.
2. Recomputes the signature over the received header and payload using its own key.
3. Compares the recomputed signature to the one attached to the token.

If someone changes even one character of the payload, for example `"role":"user"` to `"role":"admin"`, the recomputed signature will not match, because the signature is a mathematical function of the exact bytes of the header and payload. **Changing the payload without also having the signing key always breaks the signature check, which is the entire security guarantee of a JWT.** A well-written verifier rejects the whole request at that point, before any business logic runs.

A minimal Spring-style filter making this explicit:

```
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtVerifier jwtVerifier;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
```

```
                                    FilterChain chain) throws IOException,
ServletException {
        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }
        String token = header.substring(7);
        try {
            Claims claims = jwtVerifier.verifySignatureAndExpiry(token);
            SecurityContext context = new SecurityContextImpl(
                    new JwtAuthentication(claims.getSubject(), claims.get("role",
String.class)));
            SecurityContextHolder.setContext(context);
        } catch (SignatureException | ExpiredJwtException e) {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }
        chain.doFilter(request, response);
    }
}
```

**Interview Answer:**

The trick is that the signature is a function of the signing key plus the exact bytes of the header and payload, so if I edit my role from user to admin, the server recomputes the signature with its own key and gets a different value than what's attached to my tampered token, and rejects it. I'd point out that this only works if the server actually verifies the signature and expiry on every request; a surprising number of real vulnerabilities come from code that decodes a JWT and trusts the claims without calling the verify step at all.

---

## Q4. [Junior] What's the difference between HS256 and RS256 for signing a JWT, and how would you decide which one to use?

**Answer:**

**HS256 (symmetric)** uses a single shared secret key. The same key signs the token and verifies it. It's fast and simple, but anyone who has the key can both issue and verify tokens. For example, a single monolith that both issues and verifies its own tokens can use HS256 safely since only that one service holds the key, but a system where five separate microservices need to verify tokens without being trusted to issue new ones should use RS256, handing each of those five services only the public key.

**RS256 (asymmetric)** uses a private and public key pair. The private key signs the token, and only the matching public key is needed to verify it. The public key can be shared freely, even published, since it can't be used to create valid signatures, only to check them.

Trade-offs:

- HS256: simpler operations, one key to manage, but every service that can verify tokens can also mint them, which is risky once more than one service is involved.
- RS256: slightly heavier compute and key management, rotating key pairs, publishing the public key, but cleanly separates who can issue from who can verify.

**Best Choice**: RS256 (or its elliptic-curve cousin ES256) for any system with multiple services or multiple companies verifying tokens issued by one central authority, such as a microservices architecture or an OAuth provider like Google. HS256 is fine for a single monolith where the same process issues and checks its own tokens.

**Why? In a microservices setup you want ten services to be able to verify a token but only the one auth service to be able to issue one, and that separation is only possible with asymmetric signing.** If you used

HS256 and gave the shared secret to all ten services, any compromised service could forge tokens for any user.

**Interview Answer:**
HS256 signs and verifies with the same shared secret, so it's simple but means every verifier can also be an issuer. RS256 signs with a private key and verifies with a public key, so I can hand the public key to as many services as I want without ever letting them mint tokens. I'd default to RS256 in any multi-service architecture specifically because it lets one auth server be the sole issuer while every downstream API just verifies, and I'd only reach for HS256 in a small single-process system where that separation doesn't matter.

---

## Q5. [Junior] In plain terms, what is OAuth 2.0, and who are the four parties involved?

**Answer:**

OAuth 2.0 is an authorization framework, not an authentication protocol. It lets a user grant a third-party application limited access to their data on another service, without ever handing that application their password. The common confusion: OAuth answers "what is this app allowed to do on my behalf," not "who is this user," even though many apps use it, through OIDC covered later, to also establish identity.

The four roles:

- **Resource Owner**: the end user, the person who owns the data, for example your Google Drive files.
- **Client**: the third-party application asking for access, for example a photo printing app that wants to read your Drive.
- **Authorization Server**: the system that authenticates the resource owner and issues access tokens, for example Google's login and consent screens.
- **Resource Server**: the API that actually holds the protected data and accepts access tokens, for example the Google Drive API.

**The single most important mental model is that the client never sees the user's password, it only ever receives a scoped, time-limited token from the authorization server.** This is what makes OAuth safer than the old pattern of apps asking users to type their Gmail password directly into a third-party form.

**Interview Answer:**
OAuth 2.0 is about authorization, letting an app do something on your behalf, not about proving who you are. There are four roles: the resource owner is the user, the client is the app requesting access, the authorization server issues tokens after the user consents, and the resource server is the API that checks those tokens. The core value proposition I always mention is that the client app never touches the user's actual password, it only gets a scoped token, which limits the blast radius if that app is ever compromised.

---

## Q6. [Mid] Walk me through what actually happens, step by step, when I click "Login with Google" on a website.

**Answer:**

Concretely, using the authorization code flow:

1. I click "Login with Google" on Site X. Site X, the client, redirects my browser to Google's, the authorization server's, login page, including its `client_id`, a `redirect_uri`, and the `scope` it wants, for example email and profile.

2. I log into Google, if not already, and see a consent screen: "Site X wants to access your email and profile."
3. I approve. Google redirects my browser back to Site X's `redirect_uri` with a short-lived, single-use authorization code in the URL query string.
4. Site X's backend takes that code and calls Google's token endpoint directly, server to server, not through my browser, sending the code plus its own `client_secret` to prove it's really Site X.
5. Google validates the code and secret, then returns an access token, and often an ID token, to Site X's backend.
6. Site X's backend uses the access token to call Google's API for my profile and email, creates or looks up my account, and starts a session for me on Site X.

**Notice that my Google password never touches Site X at any point in this flow, only a one-time code and then a token do.** The `redirect_uri` is registered with Google in advance specifically so an attacker can't redirect the code somewhere else.

**Interview Answer:**

When I click login with Google, the site redirects me to Google, I authenticate and consent there, and Google redirects me back with a temporary authorization code in the URL. The site's backend then exchanges that code for an access token by calling Google directly, server to server, along with a client secret that proves it's the legitimate registered app. Google returns the token, the site fetches my profile, and creates a session. The key property I highlight is that my actual Google password never passes through the third-party site at any point.

---

## Q7. [Mid] What is PKCE, and why does the authorization code flow need it for mobile and single-page apps?

**Answer:**

PKCE, Proof Key for Code Exchange, pronounced "pixy", is an extension to the authorization code flow designed for public clients, meaning apps that cannot safely keep a secret. A mobile app's binary can be decompiled and a single-page app's JavaScript is fully visible in the browser, so a hardcoded `client_secret` in either one isn't actually secret.

The problem PKCE solves: without it, if an attacker intercepts the authorization code, for example via a malicious app registering the same custom URL scheme on a phone, they could exchange that code for a token themselves, since a public client has no secret to distinguish a legitimate app from an attacker.

How it works:

1. Before starting the flow, the app generates a random string called a `code_verifier`, and derives a `code_challenge` from it, a hashed and encoded version.
2. It sends the `code_challenge` in the initial authorization request.
3. When exchanging the authorization code for a token, the app sends the original `code_verifier`.
4. The authorization server hashes the verifier and checks it matches the challenge from step 2.

**Even if an attacker steals the authorization code in transit, they can't redeem it for a token without also having the original random code verifier, which never left the legitimate app.**

**Best Choice**: Authorization code flow with PKCE for any client that can't hold a secret: mobile apps, single-page apps, desktop apps.

**Why?** It keeps the strong security properties of the code exchange step without relying on a secret that would be trivially extractable.

**Interview Answer:**
PKCE fixes the authorization code flow for apps that can't keep a client secret, like mobile apps or single-page apps, since anything shipped to the client can be extracted. The app creates a random code verifier, sends a hashed version called the code challenge upfront, and later proves it's the same app by presenting the original verifier when exchanging the code for a token. That way, even if the authorization code itself leaks in transit, an attacker can't complete the exchange without the verifier, which never left the original app. I'd treat PKCE as mandatory for any public client today.

---

## Q8. [Mid] When would you use the client credentials grant instead of the authorization code flow?

**Answer:**

The client credentials flow is for machine-to-machine communication where there is no human user involved at all, for example a backend batch job calling a partner's API, or one internal microservice calling another. There's no resource owner to redirect for consent, no browser, no login screen.

How it works:

1. The client, service A, sends its `client_id` and `client_secret` directly to the authorization server's token endpoint.
2. The authorization server validates the credentials and returns an access token scoped to that service, not to any particular user.
3. Service A calls the resource server, service B, with that token.

Contrast with the authorization code flow, which always involves a human resource owner logging in and consenting; client credentials skips that entirely because the client is acting as itself, not on behalf of a user.

**Best Choice**: Client credentials for service-to-service or scheduled-job authentication; authorization code, with PKCE if public, for anything acting on behalf of a specific logged-in person.

**Why? Using the wrong grant type here either forces a nonexistent human through a login screen, or, worse, has a service impersonate a "user" that doesn't really exist just to get a token.**

**Interview Answer:**
Client credentials is the grant type for pure machine-to-machine calls, no human, no browser, no consent screen. A service authenticates directly with its own client id and secret and gets back a token that represents the service itself. I'd reach for this for things like a nightly reconciliation job calling a payments API, or one internal microservice calling another, and I'd reserve the authorization code flow for anything where a real user needs to log in and approve access to their own data.

---

## Q9. [Mid] Why is the OAuth2 implicit flow now discouraged in favor of authorization code with PKCE?

**Answer:**

The implicit flow was originally designed for browser-based apps that couldn't do a server-side code exchange. Instead of returning a short-lived code, the authorization server returned the access token directly in the URL fragment after login.

Problems with this:

- The token lands directly in the browser's address bar and history, and can leak through referrer headers, browser extensions, or shared or logged proxies.
- There is no code exchange step, so there's no opportunity to bind the token request to a client via PKCE.
- Access tokens issued this way typically can't be tied to a refresh token safely, since there's no secure back-channel exchange.
- The URL fragment is easy to accidentally log or expose via history manipulation or malicious scripts.

**Because the token is exposed directly in a place designed to be visible, like the URL, the implicit flow hands an attacker a much easier target than a short-lived, single-use authorization code ever would.**

**Best Choice**: Authorization code flow with PKCE, even for single-page apps that used to be the classic implicit flow use case.

**Why?** Modern browsers and CORS-enabled token endpoints make it practical for a single-page app to do the code exchange itself, getting the security benefits of a short-lived code and PKCE binding without needing a traditional confidential backend.

**Interview Answer:**
The implicit flow put the access token directly in the URL fragment, which meant it could show up in browser history, referrer headers, or logs, and it skipped the code exchange step entirely, so there was no way to bind the request to the legitimate app with something like PKCE. Once PKCE matured, there was no good reason left to accept that exposure, so the current guidance is to use authorization code with PKCE everywhere, including single-page apps, and treat implicit as a legacy flow you migrate away from rather than adopt.

---

## Q10. [Mid] OAuth2 is about authorization. What does OpenID Connect (OIDC) add, and why couldn't we just reuse OAuth2 access tokens for login?

**Answer:**

OIDC is a thin identity layer built on top of OAuth2. It standardizes how a client learns who logged in, which plain OAuth2 deliberately doesn't define, since OAuth2 access tokens are opaque by design and meant only for calling a resource server, not for the client itself to interpret.

What OIDC adds:

- **ID Token**: a JWT, specifically, containing identity claims about the user, such as a subject id, email, name, issuer, and audience, meant to be read by the client application itself, not sent to a resource server.
- A standard userinfo endpoint to fetch more profile details.
- Standard scopes like `openid`, `profile`, and `email` that a client requests to signal it wants identity info, not just resource access.
- A well-defined discovery document so clients can find the authorization server's endpoints and public keys automatically.

Why not just reuse the access token for login: access tokens are meant for the resource server and their format is an implementation detail, some are opaque strings, not even JWTs, so relying on their contents for identity would be fragile and was never part of the OAuth2 spec's guarantee. **OIDC exists precisely to give clients a standardized, guaranteed-JWT identity token instead of asking them to reverse-engineer meaning out of an access token that was never meant to be read by the client.**

**Interview Answer:**
OAuth2 never actually defines a standard way to know who the user is, it just hands out access tokens meant for resource servers. OIDC adds an ID token, always a JWT, specifically meant for the client app to read, containing claims like the user's subject id and email, plus a userinfo endpoint and a discovery document for finding the provider's keys and endpoints. So when people say "Login with Google" today, technically they're using OIDC layered on OAuth2, the OAuth2 part gets the access token for API calls, and the OIDC part gets the ID token that actually tells you who logged in.

---

## Q11. [Mid] Why do we have both access tokens and refresh tokens instead of just one long-lived token?

**Answer:**

**Access token**: short-lived, commonly five to fifteen minutes, sent with every API request, and used directly by resource servers to authorize calls. Because it's short-lived, if it leaks, the exposure window is small.

**Refresh token**: long-lived, hours to weeks, stored more carefully, and used only to get a new access token from the authorization server, never sent to a resource server directly.

Why not just issue one long-lived token: a long-lived token sent on every request to every API multiplies the number of places it could leak, logs, proxies, browser extensions, a compromised downstream service, while also being valid for a long time if it does leak. **Splitting the token in two means the thing that travels everywhere is short-lived, and the thing that's long-lived travels rarely and only to the one trusted authorization server.**

- If an access token leaks, the damage is capped by its short expiry.
- If a refresh token leaks, the authorization server can revoke it explicitly, since it usually tracks refresh tokens, unlike stateless access tokens.
- Refresh tokens also avoid re-prompting the user to log in every few minutes, which would be unusable.

**Interview Answer:**
Access tokens are short-lived and sent on every API call, so if one leaks, the blast radius is small because it expires quickly. Refresh tokens are long-lived but only ever go to the authorization server to mint a new access token, so they travel far less often and can usually be revoked server-side since the auth server tracks them. Splitting responsibility this way gives you both good usability, since the user doesn't need to re-authenticate every few minutes, and good security, since the widely-used token is the one with the short lifetime.

---

## Q12. [Mid] What is refresh token rotation, and what should happen if a refresh token gets stolen?

**Answer:**

Refresh token rotation means every time a refresh token is used to get a new access token, the authorization server also issues a brand new refresh token and invalidates the old one. The client must swap in the new refresh token for the next round.

Why rotate: without rotation, a single stolen refresh token stays valid for its entire long lifetime, giving an attacker a long window to keep minting access tokens silently. With rotation, each refresh token is single-use, so at most one refresh cycle's worth of access is at risk from a raw theft.

Reuse detection is the other half: if the authorization server ever sees an old, already-rotated-out refresh token being used again, that's a strong signal of theft, since the legitimate client should already have the newer one. **When reuse of an old refresh token is detected, the correct response is to revoke the entire token family immediately, not just the one token, because the attacker and the legitimate user may now both be racing to use stale credentials.**

Pseudocode for the rotation and reuse-detection logic on the server:

```
public TokenPair refresh(String incomingRefreshToken) {
    RefreshTokenRecord record = store.find(incomingRefreshToken);

    if (record == null) {
        throw new SecurityException("Unknown refresh token");
    }
    if (record.isAlreadyRotated()) {
        store.revokeFamily(record.getFamilyId());
        throw new SecurityException("Refresh token reuse detected, family revoked");
    }

    String newAccessToken = tokenIssuer.issueAccessToken(record.getUserId());
    String newRefreshToken = tokenIssuer.issueRefreshToken(record.getUserId(),
record.getFamilyId());
    store.markRotated(incomingRefreshToken, newRefreshToken);

    return new TokenPair(newAccessToken, newRefreshToken);
}
```

**Interview Answer:**

Refresh token rotation issues a new refresh token every time the old one is redeemed, and invalidates the one just used, so each refresh token is effectively single-use. That limits how much damage a stolen refresh token can do on its own. The more important piece is reuse detection: if the server ever sees a refresh token that was already rotated out being presented again, that means two parties have a copy of the same credential, so the right move is to revoke the whole token family and force the user to log in again, rather than quietly issuing more tokens.

---

## Q13. [Mid] What's the best pattern for handling access token expiry on the client: refreshing ahead of time, or reacting to a 401?

**Answer:**

**Reactive refresh**: the client keeps calling APIs normally, and only when it gets a 401 Unauthorized does it realize the access token expired, then calls the refresh endpoint and retries the original request. Simple to implement, but every request has a chance of hitting an extra round trip right when the token happens to expire, and the user may briefly see a failed request flicker.

**Silent, proactive refresh**: the client tracks the access token's expiry time and refreshes it a little before it actually expires, for example with sixty seconds left on a fifteen-minute token, so ideally no API request ever hits a 401 due to expiry at all.

**Best Choice**: a hybrid, primarily silent refresh with reactive refresh as a safety net for clock skew or a missed timer.

**Why?** Silent refresh gives a smoother experience and fewer failed requests, while keeping reactive refresh as a fallback avoids ever getting permanently stuck if the proactive timer is missed, for example when a laptop wakes from sleep.

The stampede problem: in an app making several API calls in parallel, common in a UI loading a dashboard with five widgets at once, if the access token has just expired, all five calls can independently notice this and

independently call the refresh endpoint at the same time. This wastes calls, can trip rate limits on the auth server, and, worse, can race with refresh token rotation, causing some calls to use an already-rotated-out refresh token and get incorrectly flagged as reuse. **The fix is to make the refresh call idempotent from the client's perspective by using a single in-flight refresh promise or lock that all waiting requests share instead of each one triggering its own refresh.**

A simple Java pattern using a single in-flight future to prevent duplicate refresh calls:

```
public class TokenRefresher {

    private final Object lock = new Object();
    private volatile CompletableFuture<String> inFlightRefresh;

    public CompletableFuture<String> getFreshAccessToken(AuthClient authClient,
TokenStore store) {
        if (!store.isExpiringSoon()) {
            return CompletableFuture.completedFuture(store.getAccessToken());
        }
        synchronized (lock) {
            if (inFlightRefresh == null) {
                inFlightRefresh = authClient.refreshAsync(store.getRefreshToken())
                        .thenApply(pair -> {
                            store.save(pair);
                            return pair.getAccessToken();
                        })
                        .whenComplete((r, e) -> inFlightRefresh = null);
            }
            return inFlightRefresh;
        }
    }
}
```

**Interview Answer:**

I'd default to silent refresh, proactively renewing the access token a bit before it expires based on its known lifetime, and keep reactive refresh on a 401 purely as a fallback for clock skew or a missed timer. The trap I'd call out explicitly is the refresh stampede: if several parallel API calls all notice an expired token at once, they can each fire their own refresh call, which wastes requests and can actually break refresh token rotation by racing multiple calls against a single-use token. The fix is a shared in-flight refresh so concurrent callers wait on one refresh instead of triggering many.

---

## Q14. [Mid] Where should you store access and refresh tokens on the client: cookies, localStorage, or in-memory? What are the real trade-offs?

**Answer:**

**localStorage**: persists across tabs and browser restarts, easy to use from JavaScript. The risk: it's fully readable by any JavaScript running on the page, so if the site has an XSS, cross-site scripting, vulnerability, meaning an attacker manages to inject and run their own script on your page, that script can simply read localStorage and steal the token directly. No special trick needed.

**Cookies, especially HttpOnly**: an HttpOnly cookie cannot be read by JavaScript at all, which closes off the XSS-reads-the-token path. The risk shifts to CSRF, cross-site request forgery, since the browser will automatically attach cookies to requests even when they're triggered by a different, malicious site, unless mitigated, as covered in a later question.

**In-memory**, a variable in the running app: not persisted anywhere, so it disappears on refresh or restart, meaning you need silent refresh via a refresh token stored more durably elsewhere, and it's not directly

exposed to storage-scanning attacks, but sophisticated in-page XSS can still potentially read it while the page is live, since it's still accessible to running JavaScript in that page's context.

**Best Choice**: access token in memory, or a short-lived HttpOnly cookie, for a web app, refresh token in an HttpOnly, Secure, SameSite cookie; avoid localStorage for anything sensitive.

**Why? The realistic threat model for a web app is XSS, not CSRF, once modern SameSite cookie defaults exist, so keeping tokens out of any JavaScript-readable storage closes the highest-impact hole.** CSRF is also comparatively easy to defend against with SameSite plus a CSRF token, whereas an XSS-exposed token in localStorage can be exfiltrated wholesale with a single injected script.

**Interview Answer:**

localStorage is convenient but fully readable by JavaScript, so any XSS vulnerability on the page means an attacker can just read the token straight out of storage. HttpOnly cookies close that hole since JavaScript can't touch them at all, but they reopen a CSRF risk, which is well understood and mitigated with SameSite plus CSRF tokens. My default for a web app is access tokens kept in memory and refresh tokens in an HttpOnly, Secure, SameSite cookie, because I'd rather defend against CSRF, which has mature mitigations, than leave a token sitting in a place a single injected script can exfiltrate.

---

## Q15. [Mid] Explain HttpOnly, Secure, and SameSite cookie attributes: what does each one actually protect against?

**Answer:**

**HttpOnly**: tells the browser not to expose this cookie to JavaScript. It protects against XSS reading the cookie's value; even if an attacker injects a script into your page, that script cannot read or exfiltrate an HttpOnly cookie.

**Secure**: tells the browser to only ever send this cookie over HTTPS, never plain HTTP. It protects against a network attacker on an insecure connection, for example public WiFi, intercepting the cookie in transit.

**SameSite**: controls whether the cookie is sent along with requests that originate from a different site than the one that set it, which is exactly the mechanism CSRF attacks exploit.

- `Strict`: cookie is never sent on cross-site requests, even when a user clicks a link from another site into yours. Safest, but can break legitimate cross-site navigation flows.
- `Lax`, the modern browser default: cookie is sent on top-level, "safe" navigations like clicking a link, but not on cross-site POSTs, image loads, or iframes. A reasonable middle ground for most session cookies.
- `None`: cookie is sent on all cross-site requests regardless of origin; required for legitimate cross-site use cases like a third-party embedded widget, but must be paired with Secure, and it removes SameSite's CSRF protection entirely.

**SameSite is the single attribute that most directly blocks CSRF, since it decides whether the browser will auto-attach your session cookie to a request that a malicious third-party page tricked you into making.**

Setting these in Java, Spring or Servlet style:

```
ResponseCookie refreshCookie = ResponseCookie.from("refreshToken", token)
        .httpOnly(true)
        .secure(true)
        .sameSite("Strict")
        .path("/auth/refresh")
        .maxAge(Duration.ofDays(7))
        .build();
```

```
response.addHeader(HttpHeaders.SET_COOKIE, refreshCookie.toString());
```

**Interview Answer:**

HttpOnly stops JavaScript, and therefore XSS, from reading the cookie. Secure stops the cookie from ever going over plain HTTP, protecting it in transit on untrusted networks. SameSite controls whether the cookie rides along on cross-site requests, which is the exact mechanism CSRF depends on, so setting it to Lax or Strict is one of the most effective, low-effort CSRF mitigations available. In practice for an auth cookie I'd set all three: HttpOnly and Secure always, and SameSite to Strict or Lax depending on whether the app needs to support incoming cross-site link navigation while logged in.

---

## Q16. [Mid] From a pure security angle, what are the trade-offs between session-based auth and token-based (JWT) auth?

**Answer:**

**Session-based auth**: the server generates a random session ID at login, stores the actual session data, user id, roles, expiry, server-side, in memory, Redis, or a database, and gives the client only the opaque session ID, usually in a cookie.

- Revocation is trivial: delete the session record server-side, and the ID is instantly useless everywhere.
- It's stateful: every request needs a lookup against the session store, and that store needs to be shared or replicated across all servers in a horizontally scaled deployment.

**Token-based auth (JWT)**: the server issues a self-contained, signed token holding the claims directly. Any server with the public key or shared secret can verify it without a database lookup.

- It's stateless and scales horizontally trivially, no shared session store needed, which is attractive in distributed or microservices systems.
- Revocation is the hard part: because the token is self-contained and valid until its expiry claim says otherwise, there is no built-in way to invalidate one early. **The core security trade-off is that sessions give you instant, precise revocation at the cost of needing centralized state, while JWTs give you statelessness at the cost of losing easy revocation.**

**Best Choice**: Sessions, or hybrid approaches, when instant revocation matters a lot, for example banking or admin consoles; short-lived JWTs, often paired with a lightweight denylist or a "token valid since" timestamp, covered in a later question, when horizontal scalability and cross-service verification matter more.

**Why?** Neither is universally more secure; it's a direct trade between revocation control and statelessness, and the right choice depends on how quickly you need to be able to kill a compromised credential.

**Interview Answer:**

Sessions store the real state server-side and hand the client just an opaque id, so revocation is instant, you just delete the record, but it requires a shared session store across every server handling requests. JWTs are self-contained and signed, so any service can verify one without a database call, which scales beautifully, but that same self-containment means there's no built-in way to revoke one before it expires. I'd pick sessions when instant revocation is a hard requirement, like an admin panel, and short-lived JWTs with some revocation escape hatch when statelessness and cross-service verification matter more.

---

## Q17. [Mid] Walk me through a concrete CSRF attack, and explain how SameSite cookies and CSRF tokens each stop it.

**Answer:**

CSRF, cross-site request forgery, tricks a logged-in user's browser into sending a request to a site they're authenticated on, without the user meaning to.

Concrete walkthrough:

1. I log into mybank.com; my browser now holds a session cookie for it.
2. Without logging out, I visit evil.com, which contains a hidden, auto-submitting form pointed at `mybank.com/transfer`, with an amount and destination account pre-filled to the attacker's account.
3. My browser, on loading evil.com, submits that form to mybank.com. Because cookies are attached automatically by the browser based on the destination domain, not the page that triggered the request, my mybank.com session cookie rides along.
4. mybank.com's server sees a validly authenticated request, a valid session cookie, and processes the transfer, having no way to know the request was actually initiated from evil.com rather than from mybank's own UI.

How SameSite helps: with `SameSite=Lax` or `Strict`, the browser simply won't attach the mybank.com cookie to this cross-site POST originating from evil.com in the first place, so the request arrives with no session at all and gets rejected.

How CSRF tokens help: the server embeds a random, unpredictable token in each legitimate form or page it serves and requires that same token to be resubmitted with any state-changing request. evil.com has no way to know or read that token, since same-origin policy stops it from reading mybank.com's page content, so its forged request is missing a valid CSRF token and gets rejected even if cookies were attached.

**CSRF works specifically because browsers attach cookies based on the destination site regardless of which page initiated the request, and both defenses exist to break that automatic trust.**

**Interview Answer:**
CSRF exploits the fact that browsers attach cookies to a request based on where it's going, not where it came from. So if I'm logged into my bank and I visit a malicious page with a hidden auto-submitting form pointed at the bank's transfer endpoint, my browser happily attaches my real session cookie, and the bank has no way to tell the request wasn't initiated by its own UI. SameSite cookies stop the cookie from being attached at all on that cross-site request, and CSRF tokens add a second layer, a random value the attacker's page has no way to read or guess, so even a request that does get through is rejected.

---

## Q18. [Mid] What is CORS, what is a preflight request, and why is it a misconception that CORS protects your server?

**Answer:**

CORS, cross-origin resource sharing, is a browser mechanism that lets a server explicitly tell browsers which other origins, meaning a domain, scheme, and port combination, are allowed to read the responses of requests made to it from JavaScript running on a page from a different origin.

By default, the browser's same-origin policy blocks a script on siteA.com from reading responses from api.siteB.com unless siteB.com's server responds with headers explicitly permitting that origin.

Preflight, OPTIONS, request: for "non-simple" requests, meaning ones using methods like PUT or DELETE, or custom headers like Authorization, the browser first sends an automatic OPTIONS request to ask the server "would you allow an actual request like this one, from this origin?" before sending the real request. The server responds with allowed origins, methods, and headers, and only if that preflight succeeds does the browser send the real request.

The misconception: CORS is entirely a browser-enforced rule. It does nothing to stop a non-browser client, like a Java HTTP client, curl, or a server-to-server call, from calling your API directly with any origin header it wants, or no origin header at all, since only browsers apply and check CORS headers. **CORS protects users of browsers from a malicious website secretly reading data from another site on their behalf, it does not protect your API from a direct, deliberate attacker who simply isn't using a browser.**

**Best Choice**: use CORS to control which frontends are allowed to call your API from a browser context, but never rely on it as an authentication or authorization control; still enforce tokens, sessions, and authorization checks on the server for every request.

**Why?** Anyone can bypass CORS entirely just by not using a browser, so real access control has to live in your actual auth layer, not in the allowed-origin headers.

**Interview Answer:**

CORS is a browser-only rule where a server tells browsers which other origins are allowed to read its responses when called from client-side JavaScript, and for requests using things like custom headers or non-GET/POST methods, the browser first sends an automatic OPTIONS preflight to check permission before sending the real request. The misconception I always correct is that CORS secures the server: it doesn't, since it's entirely enforced by browsers, and any non-browser client like curl or a backend service can call your API directly regardless of your CORS headers. Real protection still has to come from proper authentication and authorization on every request.

---

## Q19. [Mid] Why isn't hashing passwords with plain SHA-256 good enough, and what do bcrypt or Argon2 actually add?

**Answer:**

Plain SHA-256 is a fast, general-purpose cryptographic hash, and that speed is exactly the problem for password storage. If an attacker gets a database dump of SHA-256 password hashes, modern GPUs can compute billions of SHA-256 hashes per second, so they can brute-force or dictionary-attack the whole leaked table very quickly, especially against common or weak passwords.

Plain hashing also doesn't stop rainbow table attacks, precomputed tables mapping common password hashes back to their plaintext, since the same password always produces the same SHA-256 hash for every user.

bcrypt and Argon2 are purpose-built password hashing algorithms that add two things:

- **Salting**: a random value unique per password is mixed in before hashing, so two users with the same password get completely different stored hashes, and precomputed rainbow tables become useless.
- **Deliberate slowness, a work factor**: these algorithms are intentionally slow and tunable to get slower as hardware improves, bcrypt's cost factor, Argon2's memory, time, and parallelism parameters, so brute-forcing even a single password becomes computationally expensive, unlike a single fast SHA-256 hash.

**The entire point of bcrypt and Argon2 is to make each individual guess expensive, turning a brute-force attack that would take hours against SHA-256 into one that would take years against a properly tuned Argon2 hash.**

Why never write your own crypto: hand-rolled hashing or encryption schemes almost always miss subtle issues, like not being constant-time, opening timing side-channel attacks, weak randomness for salts, or an insufficient work factor, that took the cryptography community years of peer review to get right in libraries like bcrypt and Argon2. Using a well-vetted, widely-audited library is strictly better than reinventing it.

**Interview Answer:**

SHA-256 is fast by design, which is great for checksums but terrible for passwords, since an attacker with a stolen hash table can try billions of guesses per second on modern hardware. bcrypt and Argon2 fix this two ways: they salt each password uniquely so identical passwords don't produce identical hashes, killing rainbow tables, and they're deliberately slow and tunable, so each individual guess costs meaningful compute time. I'd never write custom hashing logic myself, because subtle mistakes, like non-constant-time comparisons or weak salting, are exactly the kind of thing peer-reviewed libraries have already solved correctly.

---

## Q20. [Junior] What is salting in password hashing, and why do you need it even if you are already using a slow algorithm like bcrypt or Argon2?

**Answer:**

A salt is a random value generated separately for each user, stored alongside their password hash, and mixed into the input before hashing, so the same password produces a different hash for every single user. It solves a different problem than a slow hash algorithm does, which is why you need both, not one instead of the other.

*What a salt actually prevents*

Without a salt, two users who happen to pick the same password end up with the identical hash stored in the database. An attacker only has to crack that hash once to instantly identify every other account sharing it, and it makes rainbow tables, precomputed lookup tables mapping common passwords to their hashes, devastatingly effective, since one table works against every user in every database that used the same hash function.

*How it changes the math for an attacker*

With a unique random salt per user, the same password hashes to a completely different value for every account. An attacker can no longer precompute anything useful in advance, they have to attack each salted hash individually, which **multiplies the cost of cracking an entire breached database by the number of users in it**.

*Why a slow hash function is not a substitute for a salt*

bcrypt and Argon2 build salting in automatically as part of the algorithm, which is why application code rarely handles it separately today, but the two properties defend against different attacks. Deliberate slowness, a tunable work factor, makes each individual guess expensive, defending against brute-forcing one specific hash. A salt stops an attacker from reusing precomputed work across many hashes at once, defending against attacking the whole database in bulk. Drop the salt and even a slow hash lets identical passwords be spotted instantly across accounts.

**A salt defeats reuse of precomputed work across accounts, a slow hash defeats speed within a single guess, and a modern password hash needs both at the same time.**

**Interview Answer:**

"A salt is a random value stored per user and mixed into the password before hashing, so identical passwords never produce identical hashes. Without it, one rainbow table or one cracked hash exposes every user who shares that password. bcrypt and Argon2 handle salting for you automatically, but that is a separate property from their deliberate slowness: slowness fights brute-forcing one account, the salt fights reusing that cracking work across the whole database, and a secure system needs both."

## Q21. [Mid] From a security angle, how would you prevent brute-force login attempts and credential stuffing, without re-deriving rate limiting algorithms?

**Answer:**

Brute force is repeatedly guessing passwords for one account. Credential stuffing is trying username and password pairs leaked from other breaches against your login, banking on password reuse across sites.

Layered defenses, the algorithms themselves, like token bucket or sliding window, are just the mechanism, covered elsewhere:

- Per-account and per-IP rate limits on login attempts, so both "one attacker hammering one account" and "one attacker spraying many accounts from one IP" get slowed down.
- Exponential backoff or temporary lockouts after repeated failures, rather than a permanent lockout, to avoid a trivial denial-of-service where an attacker locks a victim out just by failing their password a few times on purpose.
- CAPTCHA or proof-of-work challenges triggered after a few failed attempts, to make scripted, high-volume guessing expensive without blocking real users on their first try.
- Detecting credential stuffing signatures: many attempts across many different accounts from the same IP or device fingerprint in a short window, distinct from brute force's "many attempts, one account" pattern.
- Checking new passwords against known-breached password lists at signup or change time, so credential stuffing has fewer reused passwords to succeed against in the first place.
- Multi-factor authentication as the strongest single mitigation, since even a correctly guessed or stuffed password is useless without the second factor.

**The most important operational nuance is that lockout policies must be designed so an attacker can't weaponize them to lock a legitimate user out of their own account just by deliberately failing that user's login a few times.**

**Interview Answer:**
I'd separate brute force, many guesses against one account, from credential stuffing, reused breached credentials tried across many accounts, since the detection signatures differ. Defenses I'd layer in: per-account and per-IP rate limiting, exponential backoff instead of hard lockouts to avoid attackers weaponizing lockouts against victims, CAPTCHAs after a few failures, checking new passwords against known-breach lists, and MFA as the strongest backstop since a correct password alone stops being enough. I'd leave the actual rate limiting algorithm, like token bucket, out of this answer since that's a separate topic, and focus on the security-specific layering instead.

---

## Q22. [Senior] JWTs are stateless by design, so how do you actually revoke one before it expires, say, when a user's account is compromised or they're fired?

**Answer:**

There's a genuine tension here: the whole appeal of a stateless JWT is that no database lookup is needed to verify it, but "no lookup" also means "no way to say no" once it's issued. Two practical strategies:

**1. Short expiry plus a denylist (also called a blocklist)**. Keep access tokens very short-lived, minutes, and maintain a small, fast-lookup store, commonly Redis, of token identifiers that have been explicitly revoked before their natural expiry. Every verification does the usual signature and expiry check, then a quick denylist lookup.

- Pro: precise, can revoke one specific token immediately.
- Con: reintroduces a stateful lookup on every request, partially undoing the "no database call" benefit of JWTs, though a fast in-memory-store lookup is far cheaper than a full session table hit.
- Since access tokens are short-lived anyway, the denylist only needs to retain entries until the token's natural expiry passes, keeping it small.

**2. Versioned "token valid since" timestamp per user**. Store a single timestamp or version number per user. Include the version the token was issued under as a claim. On verification, compare the token's version claim against the user's current stored version; if the token is older, reject it. To revoke all of a user's tokens at once, a compromised account, a password change, an offboarding, just bump that one timestamp or version.

- Pro: constant storage per user regardless of how many tokens they have out, cheap to check, often already cached alongside user data.
- Con: coarse-grained, revokes everything for that user at once, can't selectively kill just one device's token without extending the model, for example a per-device version.

**Neither approach makes a JWT truly instantly revocable for free, they both trade back a small amount of statefulness specifically to buy back the one capability pure stateless JWTs give up.**

**Best Choice**: short expiry, five to fifteen minutes, as the baseline everywhere, since it bounds the damage regardless of strategy, plus the versioned timestamp approach for "kill all sessions for this user" events, and a denylist reserved for the rarer case of needing to kill one specific token precisely.

**Why?** Combining both covers the common case cheaply, a per-user version bump, and the precise case correctly, a denylist lookup, without paying full session-store overhead on every request.

**Interview Answer:**
The honest answer is that pure stateless JWTs can't be revoked early without giving up some statelessness. I'd keep access tokens short-lived as the baseline defense regardless. For events like a password reset or firing someone, I'd use a per-user version or "valid since" timestamp checked on every verification, a cheap single lookup that revokes every outstanding token for that user at once. For the rarer case of revoking one specific token, I'd use a denylist keyed by the token's id in something fast like Redis, kept small since entries only need to live until the token's natural expiry anyway.

---

## Q23. [Senior] Design the authentication flow for a native mobile app that calls your backend API. Walk through token issuance, storage, and refresh.

**Answer:**

Assume a typical setup: native iOS or Android app, backend REST API, users log in with email and password or "Sign in with Google or Apple."

1. **Login**: the app collects credentials, or runs the OAuth2 authorization code with PKCE flow against Google or Apple, since a mobile app is a public client with no safe place for a secret. The backend authenticates the user and issues a short-lived access token, RS256, so the backend's multiple services can verify it independently, and a longer-lived refresh token.
2. **Storage on device**: the access token can be kept in app memory; the refresh token goes into the platform's secure storage, iOS Keychain or Android Keystore, never plain shared preferences, since those are more easily extracted on a rooted or jailbroken device.
3. **API calls**: the access token is sent as a bearer token on every request; the backend's filter verifies signature, expiry, and issuer on every call.

4. **Refresh**: mostly silent, proactive refresh a bit before expiry, tracked client-side against the token's expiry, with reactive refresh on a 401 as a fallback, the same pattern as the web case. Refresh token rotation happens on every use, with reuse detection revoking the whole token family if an old, already-rotated token is presented again.
5. **Logout and revocation**: the app calls a logout endpoint that revokes the current refresh token, and for a "log out everywhere" option, bumps the user's version so all outstanding access tokens stop being accepted quickly too.
6. **Extra mobile-specific hardening**: certificate pinning to reduce man-in-the-middle risk on hostile networks, biometric re-authentication gating access to the stored refresh token for sensitive actions, and detecting jailbroken or rooted devices to decide whether to reduce trust, for example a shorter token lifetime.

**The core design principle carried over from the web case is the same: keep the token that travels on every request short-lived, and keep the token that's long-lived confined to secure, OS-provided storage and a narrow refresh-only purpose.**

**Interview Answer:**

I'd have the app authenticate via email and password or OAuth2 with PKCE for social login, since a mobile app can't hold a client secret safely. The backend issues a short-lived RS256 access token and a longer-lived refresh token; the access token lives in memory and the refresh token goes into Keychain or Keystore, never plain shared preferences. Refresh happens proactively before expiry with reactive fallback on 401, using rotation with reuse detection so a stolen refresh token gets caught. For logout-everywhere or a compromised account, I'd bump a per-user token version so all outstanding tokens are rejected immediately, and I'd add certificate pinning as extra hardening against hostile networks.

---

## Q24. [Senior] You have one auth service issuing RS256 JWTs and a dozen microservices verifying them. How do those services get the public key, and how do you rotate keys without an outage?

**Answer:**

Each verifying service needs the auth server's current public key to check signatures, but hardcoding a key into every service's config is an operational trap: rotating it means redeploying every single service in lockstep.

JWKS, a JSON Web Key Set: the auth server exposes its public keys at a well-known endpoint as a set of keys, each tagged with a key id. Every issued JWT's header includes the key id of the key that signed it. Verifying services fetch and cache the key-set document, and when checking a token, look up the specific key matching that token's key id from the cached set.

Rotation without downtime:

1. Generate a new key pair; add the new public key to the key-set endpoint alongside the old one, so now two keys are published.
2. Start signing new tokens with the new private key, and its new key id.
3. Existing tokens signed with the old key are still verifiable, since the old public key is still published, until they naturally expire, which is quick, since access tokens are short-lived.
4. After enough time has passed that no valid token could still reference the old key, a bit longer than the maximum access token lifetime, remove the old key from the key-set document.

**Publishing both the old and new public key simultaneously during the rotation window is what avoids an outage, since it lets already-issued tokens keep verifying while new tokens are already signed with the**

**new key.** Verifying services should cache the key-set response but respect its cache headers or poll periodically, and gracefully re-fetch if they see an unfamiliar key id, rather than treating a cache miss as an automatic failure.

**Interview Answer:**
Instead of hardcoding the public key everywhere, the auth server publishes a JWKS endpoint listing its current public keys, each with a key id, and every token's header names which key id signed it. Verifying services fetch and cache that document and look up the right key by id per token. To rotate safely, I'd publish the new key alongside the old one, start signing new tokens with the new key, and only remove the old public key once enough time has passed that every token signed with it has naturally expired. That overlap window is what avoids breaking in-flight tokens during rotation.

---

## Q25. [Senior] Suppose your single-page app has an XSS vulnerability and an attacker's injected script can run in a logged-in user's browser. Design your token handling so the damage is minimized even in that scenario.

**Answer:**

Assume the worst case: arbitrary attacker JavaScript is running in the page for some window of time. The goal shifts from "prevent all token theft," which is impossible once arbitrary JavaScript runs, to "limit the value and lifetime of anything that could be stolen."

Layered mitigations:

- **Never put the refresh token where JavaScript can reach it.** Store it in an HttpOnly cookie; even a fully XSS-compromised page cannot read an HttpOnly cookie's value, so the long-lived credential is out of reach regardless of what the injected script does.
- **Keep the access token in memory, not localStorage**, and keep its lifetime very short, minutes. An attacker's script running at that moment can likely still use the in-memory token while the page is open, there's no perfect defense against arbitrary code running in the same context, but it can't walk away with something durable, since it's gone once the tab is reloaded or closed and the short expiry limits its life anyway.
- **A Content Security Policy** as a preventative layer: restrict which script sources can execute at all, which reduces the chance the XSS payload runs in the first place.
- **Refresh token rotation with reuse detection**, so if the attacker's script does manage to trigger a refresh and somehow relay that refresh token out, harder, since it's HttpOnly and the script can't read it directly, but it could still trigger a same-origin fetch that uses it, any subsequent reuse from the legitimate user's next refresh gets flagged and the whole family revoked.
- **Binding tokens to context** where possible: checking IP range or device fingerprint consistency as a secondary signal, not a hard gate, that can flag anomalous use of a token from an unexpected context for additional scrutiny or step-up authentication.

**The realistic goal against XSS isn't making tokens unstealable, since a script running inside the page can always ride along on requests made while it's active, it's making sure nothing durable and long-lived is ever exposed to that script in the first place.**

**Interview Answer:**
Once arbitrary script is running in the page, I can't stop it from riding along on requests made while it's active, so I'd design around limiting what's durable. The refresh token goes in an HttpOnly cookie so it's simply never readable by any script, XSS or not. The access token stays in memory with a short lifetime, so even if it's used during the attack window, it expires quickly and isn't sitting in a file the attacker can

exfiltrate wholesale. I'd add a Content Security Policy to reduce the odds the XSS payload runs at all, and refresh token rotation with reuse detection as a backstop.

---

## Q26. [Senior] At scale, with many auth server instances behind a load balancer, how do you implement refresh token reuse detection correctly without race conditions?

**Answer:**

The naive version, check if a token is marked rotated, then mark it rotated, has a classic race condition: two near-simultaneous refresh requests using the same refresh token could both read "not yet rotated" before either writes "now rotated," and both would succeed, defeating the single-use guarantee.

Correct approach: make the check-and-mark step atomic, not two separate operations.

- Use a datastore that supports atomic compare-and-swap or conditional writes, for example a fast key-value store's "set only if not already set" command, or a relational database's conditional update where the row's affected-count tells you whether you won the race.
- Whichever request's atomic update actually succeeds is the legitimate one that gets the new token pair; any request that loses the race, an update that affected zero rows because someone else already flipped the status, is treated as reuse or replay and triggers family revocation.

This needs to work correctly across multiple auth server instances, which is why the atomicity has to live in the shared datastore, not in each instance's local memory or a lock that only protects one process.

**Reuse detection is only meaningful if the check-then-mark step is a single atomic operation in shared storage, otherwise concurrent requests across different server instances can both slip through the same race the whole mechanism was built to catch.**

Sketch of the atomic path in Java against a relational store:

```
int rowsUpdated = jdbcTemplate.update(
    "UPDATE refresh_tokens SET status = 'ROTATED', rotated_at = ? " +
    "WHERE token_id = ? AND status = 'ACTIVE'",
    Instant.now(), tokenId);

if (rowsUpdated == 0) {
    refreshTokenService.revokeFamily(familyId);
    throw new SecurityException("Refresh token reuse detected");
}
```

**Interview Answer:**

The trap at scale is that a naive check-then-update for reuse detection has a race condition, two simultaneous refresh calls with the same token can both read "not rotated yet" before either writes the update. I'd fix that by making the check-and-mark step a single atomic operation in the shared datastore, like a conditional update that only succeeds if the row's status is still active. Whichever request's atomic write actually succeeds proceeds normally; any request that gets zero rows affected lost the race and is treated as reuse, triggering full family revocation. That atomicity has to live in shared storage since multiple auth server instances can't rely on in-process locks.

---

## Q27. [Senior] A partner wants to embed your login widget in an iframe on their site, which requires SameSite=None on your auth cookie. What are the security implications, and how do you mitigate them?

**Answer:**

Setting `SameSite=None`, required, paired with Secure, for a cookie to be sent in a third-party or embedded context like an iframe, removes the CSRF protection SameSite normally provides, since the whole point of None is "send this cookie on cross-site requests too."

Implications:

- Any site, not just the intended partner, that can trigger a request to your endpoints will have this cookie attached, since SameSite=None doesn't let you allow-list which cross-site origins get it, it's all-or-nothing at the cookie level.
- This reopens the classic CSRF risk the SameSite attribute was closing, so it now has to be handled by other means.
- Being embedded in an iframe on a third-party page also raises clickjacking risk, tricking a user into clicking something in your hidden or disguised iframe, a related but distinct concern.

Mitigations:

- Reintroduce explicit CSRF tokens on all state-changing requests protected by this cookie, since SameSite is no longer doing that job.
- Validate the origin or referrer header server-side against an explicit allow-list of partner domains for any sensitive action, as a defense-in-depth check beyond the cookie itself.
- Use a frame-ancestors content security policy directive, the modern replacement for the older X-Frame-Options header, to restrict which sites are allowed to embed your page in an iframe at all, addressing clickjacking and limiting exactly who can even trigger the SameSite=None cookie flow.
- Scope this cookie as narrowly as possible, a dedicated widget-auth cookie, not your main session cookie, so the weakened protection only applies to the embedded widget's functionality, not your whole account.

**Loosening SameSite for one legitimate embedding use case shouldn't mean giving up CSRF protection everywhere, it means consciously re-implementing that protection through other layers for just the narrow surface that needs it.**

**Interview Answer:**
SameSite=None turns off the exact protection SameSite normally provides, so the cookie now rides along on requests from any site, not just the partner's, since the attribute doesn't support an allow-list. I'd treat this as reopening CSRF and clickjacking risk that needs to be closed elsewhere: explicit CSRF tokens on state-changing requests, server-side origin or referrer validation against a partner allow-list, and a frame-ancestors policy directive to control who can embed the page at all. I'd also scope this to a narrow, dedicated cookie for the widget rather than reusing the main session cookie, so the weakened guarantee is contained to the smallest possible surface.

---

## Q28. [Senior] Design the end-to-end auth architecture for a new product: a single-page web app talking to a REST API, supporting both email/password and "Login with Google," with reasonable security defaults. What do you pick and why?

**Answer:**

This ties together most of the chapter, so I'll go layer by layer.

**Identity**: use OIDC on top of OAuth2 for the Google option, authorization code flow with PKCE, since the single-page app is a public client, and a standard email and password flow, bcrypt or Argon2 hashed, never plain SHA-256, for direct signup, both converging on the same internal user record and the same token issuance path afterward.

**Tokens**: the backend issues short-lived, ten to fifteen minute, RS256 access tokens, since the API is expected to grow into multiple backend services that all need to verify tokens independently without being able to issue them, plus a longer-lived refresh token with rotation and reuse detection.

**Client-side storage**: the access token is kept in memory in the app; the refresh token sits in an HttpOnly, Secure, SameSite cookie scoped to the refresh endpoint's path, keeping it out of reach of any XSS-injected script while cookies handle the SameSite-based CSRF baseline.

**CSRF**: since a cookie is involved for refresh, add an explicit CSRF token on the refresh endpoint and any other cookie-authenticated state-changing endpoint, layered on top of SameSite rather than relying on SameSite alone.

**CORS**: the API is configured with an explicit allow-list of the app's own origin, understanding this is a browser-enforced convenience, not a substitute for the actual auth checks the API performs on every request regardless of origin.

**Token expiry handling**: silent refresh proactively before expiry, reactive refresh on 401 as a fallback, with a single shared in-flight refresh to avoid a stampede across concurrent API calls.

**Revocation**: a per-user "valid since" version for logout-everywhere, password reset, or account compromise events, plus a small denylist keyed by token id for revoking one specific session precisely, for example logging out one device.

**Password and brute-force defense**: per-account and per-IP rate limiting with exponential backoff, not permanent lockouts, on login, breach-list checking at signup, and multi-factor authentication offered as an option, ideally required for higher-privilege accounts.

**No single mechanism here is doing all the work: short-lived signed tokens, HttpOnly cookies, SameSite, CSRF tokens, and rotation with reuse detection are each covering a different failure mode, and removing any one of them reopens exactly the gap it was added to close.**

**Interview Answer:**

I'd use OIDC with PKCE for Google login and bcrypt-hashed passwords for direct signup, both landing on the same token issuance path: short-lived RS256 access tokens so multiple future backend services can verify without being able to issue, plus rotating refresh tokens with reuse detection. On the client, the access token stays in memory and the refresh token sits in an HttpOnly, Secure, SameSite cookie, with an explicit CSRF token layered on top since a cookie is involved. I'd handle expiry with proactive silent refresh and a shared in-flight refresh to avoid a stampede, and cover revocation with a per-user version bump plus a small denylist for killing one session precisely.

---

**If you remember nothing else from this chapter:** *a secret that has never been rotated isn't a secret, it's a permanent liability with a countdown nobody's watching.*

# Chapter 18: Production & Operations Knowledge

This chapter is deliberately short, but it's often the fastest way to show the difference between having actually been on call and having only designed systems on a whiteboard. Being able to talk concretely about logging, deployment strategy, and rollback plans is what signals you've carried real operational weight.

**War Story:** *An alert that fired correctly for two straight years trained the entire on-call rotation to ignore it, right up until the one week it was actually real and nobody looked.*

**Avoid:** treating "we have alerting" as equivalent to "we'll notice when it breaks," with no mention of alert fatigue

**Aim For:** discussing alert signal-to-noise ratio unprompted and distinguishing paging alerts from purely informational ones

## Q1. [Junior] Walk me through the standard log levels, DEBUG, INFO, WARN, and ERROR, and tell me when you'd reach for each one in a running service.

**Answer:**

Log levels let a developer control how much noise a service produces without changing any code, just a configuration value.

**The Four Levels**

- DEBUG: fine-grained detail useful only while actively developing or troubleshooting, like the exact payload of a request. Too noisy to leave on in production all the time.
- INFO: normal, expected events worth recording for context, like "order 4521 placed" or "cache warmed on startup."
- WARN: something unexpected happened but the request still succeeded, like a retry that eventually worked or a fallback path being used.
- ERROR: something failed and needs a human's attention, like a failed payment call or an unhandled exception.

**Why the Split Matters**

In production you typically run at INFO or WARN by default, since DEBUG at scale generates enormous, expensive log volume. **The real value of levels is that you can temporarily raise verbosity to DEBUG on a single service while chasing a bug, then drop it back down, without a redeploy.** A service that only ever logs at ERROR gives you no context when something breaks, since there's no trail of what led up to it.

**Interview Answer:**

I think of log levels as a volume knob, not a strict rulebook. DEBUG is for development detail I don't want in production noise, INFO is for normal business events that help me reconstruct what happened, WARN flags something recoverable that I should keep an eye on, and ERROR means a human needs to look at this now. In practice I default services to INFO, wire log levels to be changeable at runtime through configuration, and bump a specific service to DEBUG temporarily when I'm hunting a hard-to-reproduce bug.

## Q2. [Junior] Why do most production systems log in structured JSON instead of plain text sentences?

**Answer:**

Plain text logs like `"User 123 failed to login at 10:02"` are easy for a human to read one line at a time, but they're painful to search, filter, or aggregate once you have millions of lines across dozens of services.

### Structured Logging

Structured logging means each log line is emitted as a JSON object with consistent fields, for example:

```
log.info("{\"event\":\"login_failed\",\"userId\":123,\"reason\":\"bad_password\",
\"timestamp\":\"2026-08-20T10:02:00Z\"}");
```

Most teams use a logging library (SLF4J with Logback or Log4j2 in Java, wired to a JSON encoder) so they don't hand-build JSON strings themselves.

### Why It's Worth the Extra Setup

- Every log tool downstream, like a log aggregator or dashboard, can parse fields reliably instead of relying on fragile regular expressions.
- You can query "show me every failed login for userId 123" instantly instead of grepping and guessing at a text pattern.
- Fields stay consistent across services even when written by different teams, as long as everyone agrees on a schema.

**The core trade-off is that structured logs are slightly harder to eyeball on a terminal but dramatically easier for machines to search, filter, and alert on at scale, which is the trade production systems almost always take.**

**Interview Answer:**

I default to structured JSON logging in any service with more than a couple of instances, because plain text only works when a human is tailing one file on one box. With JSON, every log line has consistent fields like event, userId, and timestamp, so a log aggregator can index them and I can query "all failed payments for merchant X in the last hour" in seconds instead of grepping blindly. The one-time cost is wiring up a JSON encoder in the logging library, which is trivial compared to the searchability you get back.

---

## Q3. [Junior] Why should you never log passwords, tokens, or other sensitive personal data, and how do you actually prevent it from slipping in accidentally?

**Answer:**

Logs are copied, shipped to third-party tools, retained for months, and read by a wide set of engineers, so anything written to a log has effectively left your control the moment it's written.

### What Counts as Sensitive

- Passwords and API keys or tokens, even ones that are expired, since patterns can reveal how secrets are generated.
- Personally identifiable information (PII) like social security numbers, full credit card numbers, or home addresses.
- Anything covered by regulation, like health records under HIPAA or personal data under GDPR.

### Why the Risk Is Bigger Than It Looks

Logs are usually stored with much weaker access controls than a production database, often readable by anyone with basic engineering access, and they get forwarded into analytics tools, ticket attachments, and support screenshots. **A single accidental log line containing a plaintext password can turn a minor bug into a full security incident and a compliance violation.**

**Practical Guardrails**

- Mask or redact known sensitive fields at the logging library level (a filter that replaces `password` field values with `*`) so it doesn't depend on every developer remembering.
- Log identifiers, not the sensitive value itself, for example log `userId` instead of email or name where possible.
- Treat request and response body logging as opt-in and reviewed, since entire payloads are the most common accidental leak.

**Interview Answer:**

Logs travel further and live longer than most people assume, they get shipped to third-party aggregators, sit in retention for months, and are readable by a wide set of engineers. So anything written to a log is effectively no longer private. I bake redaction into the logging framework itself, masking fields like password or token automatically, rather than trusting every developer to remember by hand. I also push toward logging identifiers like a userId instead of raw PII, so I still have enough context to debug without exposing the actual sensitive value.

---

## Q4. [Junior] What's the difference between a liveness probe and a readiness probe, and why does mixing them up cause outages?

**Answer:**

Both are health checks an orchestrator like Kubernetes calls periodically, but they answer different questions and trigger different actions.

**Liveness vs Readiness**

- A liveness probe answers "is this process still alive, or is it stuck and needs to be restarted?" Failing it causes the container to be killed and restarted.
- A readiness probe answers "is this instance currently able to handle traffic?" Failing it just removes the instance from the load balancer's rotation, it keeps running.

**A Simple Java Example**

```
@GetMapping("/health/ready")
public ResponseEntity<String> readiness() {
    if (!dbConnectionPool.isHealthy()) {
        return ResponseEntity.status(503).body("NOT_READY");
    }
    return ResponseEntity.ok("READY");
}
```

**Why Conflating Them Causes Outages**

If a pod (Kubernetes' term for one or more containers deployed together) loses its database connection temporarily but that check is wired to the liveness probe instead of readiness, Kubernetes (covered in more detail later in this chapter) will restart the pod over and over, even though restarting does nothing to fix a downstream database outage. **The failure mode to avoid is a pod that's technically alive but can't serve real traffic still receiving requests, or a pod being killed and restarted in a loop for a problem a restart can**

**never fix.** Readiness should check dependencies that traffic actually needs, like a database or cache connection, while liveness should check something much simpler, like "is the main thread responsive."

**Interview Answer:**

Liveness asks "should this container be restarted," readiness asks "should this container receive traffic right now." I keep liveness checks minimal, basically "is the process responsive," so a downstream outage doesn't cause a restart storm that makes things worse. Readiness is where I put dependency checks like database connectivity, so during startup or a brief database blip, the pod stays alive but gets pulled out of the load balancer until it's actually ready again. Mixing the two up is one of the most common causes of self-inflicted outages I've seen in Kubernetes deployments.

---

## Q5. [Mid] A single user request touches five different microservices. How would you trace that one request end to end, and what does a tool like Jaeger or Zipkin actually give you?

**Answer:**

Without help, tracing one request across five services means manually correlating timestamps across five separate log streams, which doesn't scale and is often just guesswork.

**Correlation IDs**

The fix is a correlation ID (also called a trace ID): a unique identifier generated when the request first enters the system, then passed along on every downstream call, usually as an HTTP header like `X-Correlation-Id`.

```
String correlationId = request.getHeader("X-Correlation-Id");
if (correlationId == null) {
    correlationId = UUID.randomUUID().toString();
}
MDC.put("correlationId", correlationId);
log.info("Processing order request");
```

Using SLF4J's MDC (Mapped Diagnostic Context) means every log line emitted during that request automatically includes the correlation ID, so you can pull every log line across every service for one request with a single search.

**What Distributed Tracing Tools Add**

Tools like Jaeger or Zipkin build on the same idea but go further: each service reports a "span," a timed unit of work with a start and end time, tagged with the trace ID. The tracing tool stitches these spans into a single visual timeline, showing exactly how long each service took and where the time actually went. **A distributed trace turns "the request was slow somewhere in the system" into a concrete waterfall diagram showing that, say, service C's database call ate 800 of the 900 total milliseconds.**

**Interview Answer:**

I generate a correlation ID at the edge, the first service the request hits, and propagate it as a header to every downstream call, and every service logs it via something like MDC so all the logs for one request can be pulled together. That solves "find the logs." For "find the slowness," I'd use a distributed tracing tool like Jaeger, where each service reports timed spans tagged with the same trace ID, and the tool assembles them into a single timeline. That's the difference between guessing which service is slow and seeing exactly which call in which service ate the time.

## Q6. [Mid] Once you have more than a couple of instances of a service, why can't you just SSH into a box and grep the log file, and what does a stack like ELK actually give you instead?

**Answer:**

Grepping a single log file works fine for one server. It breaks down almost immediately once you're running multiple instances behind a load balancer or in an auto-scaling group.

**Why SSH-and-grep Stops Working**

- A single request could have landed on any one of dozens of instances, so you'd need to guess which box to check, or check all of them one by one.
- Instances get replaced constantly in modern deployments (auto-scaling, container restarts), and the logs on a terminated instance are usually just gone.
- There's no way to search across time ranges or correlate events across services from a raw file on a single box.

**What Centralized Logging Gives You**

A stack like ELK (Elasticsearch for storage and search, Logstash or a lighter shipper like Filebeat for collecting and forwarding logs, Kibana for the search and dashboard UI), or a managed equivalent like Datadog or CloudWatch Logs Insights, collects logs from every instance into one searchable place.

- Every instance ships its logs to a central store as they're written, so logs survive even after the instance itself is terminated.
- You can search across the entire fleet at once, for example "show me every ERROR log across all 40 instances of the payments service in the last 15 minutes."
- Dashboards and alerts can be built on top, so a spike in error rate triggers a page instead of waiting for someone to notice.

**Centralized logging is what turns "logs exist somewhere on some machine" into "logs are a searchable, queryable dataset," which is a requirement, not a nice-to-have, once you're running more than a handful of instances.**

**Interview Answer:**

SSH-and-grep works when you have one server and you know exactly where to look. It falls apart with auto-scaling and containers, since a request could have hit any of dozens of instances, and terminated instances take their logs with them. I ship logs off every instance in real time into a central store, an ELK stack or a managed option like Datadog, so logs survive instance churn and I can search the whole fleet at once instead of guessing which box to check. That's also what makes alerting possible, since you can't alert on a log file nobody's watching.

## Q7. [Mid] Explain the three pillars of observability, metrics, logs, and traces, with a concrete example of each.

**Answer:**

The three pillars answer different questions about the same system, and a mature production setup needs all three because none of them alone tells the whole story.

**Metrics**

Metrics are numeric measurements aggregated over time, cheap to store and great for dashboards and alerting. Example: "requests per second," "average response time," or "CPU utilization at 85%." Metrics tell you *that* something is wrong, like an error rate spiking from 0.1% to 8%.

**Logs**

Logs are discrete, timestamped records of individual events, richer in detail but more expensive to store and search at volume. Example: `"event":"payment_failed","orderId":4521,"reason":"card_declined"`. Logs tell you the specifics of *what* happened in one particular case.

**Traces**

Traces show the path and timing of a single request as it flows through multiple services, built from the spans described in distributed tracing. Example: a trace showing a checkout request spent 20ms in the API gateway, 15ms in the inventory service, and 400ms in the payment service. Traces tell you *where* time was spent across a distributed call chain.

**How They Work Together**

A typical flow: a metrics dashboard shows error rate spiking (metrics), you pull the logs for that time window to see which requests failed and why (logs), then you follow a trace for one of those failed requests to see exactly which downstream service caused the slowdown or error (traces). **Metrics tell you something is wrong, logs tell you what happened, and traces tell you where in the system it happened, and losing any one of the three leaves a real blind spot.**

**Interview Answer:**

I think of it as zooming in progressively. Metrics are the smoke alarm, cheap numeric time series like error rate or latency that tell me something's wrong right now and are what I alert on. Logs are the detailed incident report, individual events with context I can search once I know roughly where to look. Traces are the map, showing exactly how one request moved through multiple services and where the time went. In an incident I usually start at a metrics dashboard, drop into logs for specifics, then pull a trace to pinpoint which service in the chain is actually the bottleneck.

---

## Q8. [Mid] What's the actual difference between an SLI, an SLO, and an SLA? Give me a concrete example.

**Answer:**

These three terms are related but answer different questions, and they build on each other.

**The Three Terms**

- SLI (Service Level Indicator): the actual measurement, a number you collect. Example: "the percentage of requests that complete in under 200ms."
- SLO (Service Level Objective): the internal target you set for that measurement. Example: "99.9% of requests should complete in under 200ms, measured over a rolling 30 days."
- SLA (Service Level Agreement): an external, often contractual, promise to customers, usually with a defined consequence if it's missed, like a service credit. Example: "we guarantee 99.9% uptime per month, or you receive a service credit."

**How They Relate**

The SLI is the raw ingredient, the SLO is the internal goal built from it, and the SLA is the external commitment built on top of the SLO, usually set a bit looser than the internal SLO to leave a safety margin.

For instance, a team might target a 99.95% SLO internally while only promising 99.9% in the customer-facing SLA, so there's buffer before they'd actually breach the external commitment.

**Why This Distinction Matters Operationally**

**SLOs are what actually drive engineering decisions day to day, like how much error budget is left before you should stop shipping risky changes and focus purely on reliability.** Teams that only think in terms of the external SLA tend to react only after a customer-facing breach, instead of proactively managing toward an internal target with room to spare.

**Interview Answer:**

SLI is the raw metric, like the percentage of requests under 200ms. SLO is the internal target for that metric, like 99.9% over a rolling month. SLA is the external, often contractual promise to customers, usually set a bit looser than the internal SLO to leave a safety margin. In practice I care most about the SLO day to day, since it's what tells my team whether we have error budget left to ship riskier changes or whether we need to slow down and focus on stability before we breach what we've promised customers externally.

---

## Q9. [Mid] Explain continuous integration and continuous deployment in plain terms, and walk me through the typical stages of a pipeline.

**Answer:**

CI and CD are often said together but solve two different problems.

**Continuous Integration (CI)**

CI means every developer merges their code changes into a shared branch frequently, and each merge automatically triggers a build and test run. The goal is catching integration problems, like two people's changes conflicting, within minutes instead of discovering them weeks later during a big merge.

**Continuous Deployment (CD)**

CD means code that passes all checks is automatically deployed toward production, without a person manually clicking "deploy" for every change. (Continuous *delivery* is a related, slightly weaker version, where the code is automatically packaged and ready to deploy, but a human still approves the final push to production.)

**Typical Pipeline Stages**

1. Build: compile the code and package it, for example into a JAR file or a Docker image.
2. Test: run automated unit tests, then integration tests, failing the pipeline immediately if anything breaks.
3. Static analysis and security scan: check code quality and scan dependencies for known vulnerabilities.
4. Deploy to staging: push the build to a staging environment that mirrors production for further testing.
5. Deploy to production: release the build to production, often using one of the deployment strategies like canary or rolling (both covered in detail later in this chapter).

A minimal pipeline stage definition might look like:

```
stages:
  - build
  - test
  - deploy

test:
  stage: test
```

```
  script:
    - mvn test
```

**The entire point of a CI/CD pipeline is to make shipping small changes routine and low-risk instead of a rare, high-stress event, since small frequent changes are far easier to test and roll back than large infrequent ones.**

**Interview Answer:**

CI is about merging code frequently and automatically building and testing every merge, so integration problems surface in minutes, not weeks. CD is about automatically pushing code that passes those checks toward production without a manual gate for every change. A typical pipeline builds the artifact, runs unit and integration tests, runs a security or dependency scan, deploys to staging for a final check, then deploys to production, usually through a rolling or canary strategy. The real payoff is that shipping becomes a routine, low-risk, frequent event instead of a rare, high-stress, all-hands deployment.

---

## Q10. [Mid] In plain words, what problem does Docker actually solve? Explain it like you're talking to someone who has never used containers.

**Answer:**

The classic problem Docker solves is "it works on my machine," where code runs fine for the developer but breaks in staging or production because of some difference in the environment.

### Environment Inconsistency Without Containers

Without containers, a service depends on a specific version of Java, specific system libraries, specific environment variables, and specific configuration files all being present and correctly set up on whatever machine runs it. Any mismatch between a developer's laptop, a QA server, and a production server can cause bugs that are painful to reproduce, because the code itself hasn't changed, only its surroundings.

### What a Container Actually Packages

A container bundles the application together with everything it needs to run: the exact runtime, dependencies, and configuration, into a single portable image.

```
FROM eclipse-temurin:17-jre
COPY app.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

That image runs the same way on a laptop, a test server, or a production cluster, because it's not relying on whatever happens to already be installed on the host machine.

### Why This Matters in Production

**Docker's real value isn't the technology itself, it's that it eliminates an entire category of bugs caused by environment drift between where code is written and where it runs.** It also makes scaling simpler, since starting a tenth identical instance of a service is just running the same image again, not manually configuring another server by hand.

**Interview Answer:**

Before containers, a huge source of production bugs was environment drift, code worked on a developer's laptop but broke in production because of a different Java version, a missing library, or a different config file. Docker packages the application with its entire runtime and dependencies into one portable image, so the same thing runs identically everywhere, laptop, staging, and production. That kills "works on my machine" as an excuse, and it also makes scaling trivial, since spinning up another identical instance is just running the same image again rather than manually setting up a new server.

## Q11. [Mid] At a high level, what does Kubernetes actually do for you, without going too deep into its internals?

**Answer:**

Kubernetes is a container orchestrator: once you have your application packaged as containers, Kubernetes is the system that decides where those containers run and keeps them running.

**What It Actually Handles**

- Scheduling: it decides which physical machine (node) each container should run on, based on available CPU and memory, so you don't manually assign containers to servers.
- Self-healing: if a container crashes or a health check fails, Kubernetes restarts it automatically, without a human noticing and intervening.
- Scaling: it can add more copies of a container when load increases and remove them when load drops, either automatically based on CPU or manually.
- Service discovery and load balancing: it gives a stable way for one service to find and call another, even as individual container instances come and go.

**Why This Matters Operationally**

Running containers manually across many servers would mean someone has to track which server has capacity, notice when a container dies and restart it, and manually update routing when instances change, all error-prone at scale. **Kubernetes' core value is that it turns "keep N healthy copies of this service running somewhere across this fleet of machines" into a declarative statement instead of a manual, ongoing chore.** You describe the desired state, like "run 5 copies of this container with these resource limits," and Kubernetes continuously works to make reality match that description.

**Interview Answer:**
I think of Kubernetes as the thing that keeps a promise for you. You tell it "I want 5 healthy copies of this container running," and it figures out which machines to put them on, restarts any that crash or fail a health check, scales the count up or down based on load, and routes traffic to whichever instances are currently healthy. Without it, a team would be manually tracking server capacity and restarting dead processes by hand. Kubernetes turns that from a manual operational chore into a declarative statement the system continuously enforces.

## Q12. [Senior] Compare rolling deployments, blue-green deployments, and canary releases. When would you actually choose each one?

**Answer:**

All three strategies exist to answer the same question, how do you replace an old version of a service with a new one without an outage, but they trade off speed, cost, and blast radius differently.

**Rolling Deployment**

Instances are updated gradually, a few at a time, with the old and new versions running side by side during the rollout. Low infrastructure cost since you don't need double the capacity, but a bad deploy affects a growing slice of traffic before anyone notices, and rollback means rolling the update back the same gradual way.

**Blue-Green Deployment**

Two full, identical environments exist, "blue" (current production) and "green" (the new version). Traffic is switched from blue to green all at once, typically at the load balancer or router level. Rollback is nearly instant, just switch traffic back to blue, but it requires running double the infrastructure during the switch.

**Canary Release**

The new version is released to a small slice of real traffic, say 5%, while it's closely monitored, then gradually expanded to 25%, 50%, and 100% if metrics stay healthy. This limits the blast radius of a bad release to a small fraction of users, but it requires good metrics and monitoring to actually detect a problem in that small slice before expanding further.

**Best Choice**

For a typical high-traffic production service where a bad release could impact real customers, a canary release is usually the best choice.

**Why?**

**A canary limits exposure to a small percentage of real users while giving you live production signal, which catches the class of bugs that only show up under real traffic and real data, something staging environments routinely miss.** Blue-green is worth the extra infrastructure cost when you need near-instant rollback and can't tolerate any gradual exposure, and rolling deployments are fine for lower-risk internal services where a bit of gradual exposure and slower rollback is an acceptable trade for lower cost.

**Interview Answer:**
Rolling deployments are cheap since you don't double your infrastructure, but a bad version is exposed to a growing share of traffic before rollback catches up. Blue-green gives near-instant rollback by keeping two full environments and flipping traffic at once, at the cost of running double capacity temporarily. Canary is my default for anything customer-facing and high-risk, since it exposes a new version to a small slice of real traffic first, catching production-only bugs that staging never surfaces, before gradually expanding. I'd reach for blue-green when instant rollback matters more than gradual exposure, and rolling for lower-risk internal services where cost matters more than blast radius.

---

## Q13. [Senior] How do feature flags change the relationship between deploying code and releasing a feature to users?

**Answer:**

Without feature flags, deploying code and releasing a feature are the same event, the moment new code reaches production, users start hitting it. Feature flags break that coupling.

**Decoupling Deploy from Release**

A feature flag is a runtime condition, usually backed by a configuration service, that decides whether a piece of code path executes for a given user or not.

```
if (featureFlags.isEnabled("new-checkout-flow", userId)) {
    return newCheckoutService.process(order);
}
return legacyCheckoutService.process(order);
```

The new checkout code can be deployed to production, fully live on every server, while the flag stays off for everyone. That means deployment (getting code onto servers) and release (turning a feature on for users) become two separate, independently controllable events.

**Why This Matters for Rollback**

**If a feature causes problems after being turned on, the fastest possible rollback is flipping the flag off, which takes effect in seconds through a configuration change, instead of triggering a full redeploy of the previous code version.** This also enables gradual rollout, turning a flag on for 1% of users, then internal employees, then 10%, then everyone, all without touching the deployment pipeline again, and it enables instant kill switches for risky new features during high-traffic events.

**Interview Answer:**

Feature flags separate "the code is on the server" from "users are actually hitting that code," which used to be the same moment. I wrap risky new functionality in a flag, deploy it fully dark, off for everyone, then turn it on gradually for a percentage of users while watching metrics. If something goes wrong, I flip the flag off, which is a configuration change that takes effect in seconds, rather than needing a full redeploy of the previous version. It also lets product and engineering decouple their timelines, code can ship on the normal schedule while the actual release date is a business decision made later.

---

## Q14. [Senior] What does a good rollback plan actually look like, and why isn't "just redeploy the old version" always fast enough or even safe?

**Answer:**

"Just redeploy the previous version" sounds simple, but in a real system it often isn't the fast, safe option people assume it is.

**Why Simple Redeploy Can Fail**

The biggest risk is database schema changes. If the new version added a database migration, for example a new required column, and the old version's code doesn't know that column exists or expects it to be nullable, rolling back the application code while the database schema has already moved forward can break the old version just as badly as the bug you were rolling back from. Rollback isn't just "revert the code," it's "revert the entire system state," and the database usually can't be un-migrated instantly.

**What a Good Rollback Plan Includes**

- Backward-compatible migrations: design database changes so both the old and new application version can run against the same schema, for example adding a new column as nullable first, backfilling data, and only making it required in a later, separate deployment.
- A tested rollback path: rollback should be rehearsed, not assumed to work the first time it's actually needed during an incident.
- Feature flags for risky logic, so the fastest rollback is a flag flip, not a full redeploy.
- Clear rollback ownership and a time limit: someone on-call should know that if a deploy looks bad after N minutes, the decision is roll back now, not "wait and see."

**The core discipline is designing every deployment, especially database migrations, to be backward compatible with the previous version, so that rolling back the application code is always safe regardless of what the database currently looks like.**

**Interview Answer:**

The trap in "just redeploy the old version" is database migrations. If the new version added a schema change the old code doesn't understand, rolling the application back while the database has already moved forward can break things just as badly. My rule is that migrations should be backward compatible, adding a column as nullable and backfilling it in one deploy, then making it required in a later, separate deploy, so either app version can run against the schema at any point. I also lean on feature flags for risky logic specifically because flipping a flag is faster and safer than a full redeploy when something goes wrong.

## Q15. [Senior] What problem do Infrastructure as Code tools like Terraform actually solve compared to manually clicking around a cloud console?

**Answer:**

Manually creating cloud resources, like a virtual machine, a database, or a load balancer, by clicking through a console works for a quick experiment, but it breaks down fast in a real production environment.

**Problems With Manual Console Changes**

- No record of exactly what was created or why, since clicks don't leave a readable history.
- No easy way to recreate the same environment for staging, disaster recovery, or a new region, since it depends on someone remembering every setting they clicked.
- Configuration drift: over time, small manual tweaks accumulate and no two environments match, making bugs that only happen in one environment nearly impossible to reproduce.
- No review process, since anyone with console access can change production infrastructure directly, with no equivalent of a code review catching a mistake before it happens.

**What Infrastructure as Code Gives You**

Terraform-style tools let you describe infrastructure in a text file, and the tool figures out what to create, change, or destroy to match that description.

```
resource "aws_instance" "app_server" {
  ami           = "ami-0123456789"
  instance_type = "t3.medium"
}
```

That file can be checked into version control, reviewed like any other code change through a pull request, and reused to spin up an identical environment for staging or a disaster recovery region. **Infrastructure as Code turns infrastructure changes from an untracked, unreviewed manual action into a versioned, reviewable, repeatable process, the same discipline teams already apply to application code.**

**Interview Answer:**

Clicking through a cloud console works for a one-off experiment, but it leaves no record of what was done, invites configuration drift as small manual tweaks pile up, and skips any kind of review before a production change happens. Terraform-style tools describe infrastructure as text, checked into version control and reviewed through the same pull request process as application code, and the tool handles actually creating or updating the real resources to match. That means I can recreate an identical environment for staging or disaster recovery on demand, and I get an audit trail of exactly what infrastructure changed and why.

## Q16. [Senior] Why shouldn't secrets like database passwords or API keys live in code or plain configuration files, and what does a secrets manager actually do about it?

**Answer:**

A secret hardcoded in source code or sitting in a plain text config file is exposed to everyone who can read that repository or that file, which in most companies is a large and growing group of people, plus every fork, backup, and CI log that ever touched it.

**Why This Is Riskier Than It Seems**

- Source control history is permanent by default, so a secret committed once and later "removed" is still sitting in the git history unless it's actively purged.

- Config files often get copied into CI logs, container images, or debugging screenshots without anyone intending to leak them.
- A single leaked secret, like a database password, often grants broad access, not just to one narrow piece of functionality.

**What a Secrets Manager Actually Does**

A tool like HashiCorp Vault or a cloud provider's secrets manager stores secrets in an encrypted, access-controlled service, separate from both code and configuration files. The application authenticates to the secrets manager at startup or runtime, using an identity the platform already trusts (like its cloud IAM role), and pulls the actual secret value into memory only, never writing it to disk or checking it into anything.

```
String dbPassword = secretsClient.getSecretValue("prod/db/password");
```

This also enables automatic secret rotation, changing a database password on a schedule without redeploying code, since the application always fetches the current value rather than a value baked in at build time. **The core shift is that access to a secret becomes something you can audit and revoke per system, instead of something baked permanently into a file that's copied everywhere the code goes.**

**Interview Answer:**

A secret in code or a config file gets copied everywhere that code goes, into every clone, every backup, every CI log, and it's still sitting in git history even after you "remove" it. A secrets manager like Vault or a cloud provider's equivalent stores secrets encrypted and access-controlled, separate from the codebase, and the application fetches the current value at runtime using an identity the platform already trusts, rather than having it baked in at build time. That also makes rotation realistic, since you can rotate a database password on a schedule and every service just picks up the new value, instead of needing a coordinated redeploy.

---

## Q17. [Senior] What is a runbook, and what makes a postmortem "blameless"? Why does that actually matter for reliability over time?

**Answer:**

Both are tools for turning a stressful incident into a system that gets more reliable over time, rather than just a bad memory.

**Runbooks**

A runbook is a written, step-by-step guide for handling a known type of problem, for example "database connection pool exhausted" or "payment provider returning timeouts." It typically includes how to confirm the symptom, the specific commands or dashboards to check, and the known mitigation steps to try, in order. The value of a runbook is that during a 3am incident, an on-call engineer who has never seen this specific failure before can follow a known-good process instead of improvising under pressure and stress.

**Blameless Postmortems**

After an incident is resolved, a postmortem documents what happened, why it happened, and what will change to prevent it or catch it faster next time. "Blameless" means the document focuses on the systems and processes that allowed the failure, not on which individual made the mistake, since the honest premise is that any capable engineer, given the same information and pressure at that moment, could have made the same choice.

**Why Blamelessness Matters Practically**

**If postmortems assign blame to individuals, people quickly learn to hide mistakes, downplay their role in an incident, or avoid raising early warning signs, which makes the exact same failure more likely to recur,**

**not less.** A blameless culture encourages engineers to report near-misses and honestly describe what happened, which is the raw material that actually improves the system's design over time, not just the specific person's behavior.

**Interview Answer:**
A runbook is a written playbook for a known failure mode, the steps to confirm it, check it, and mitigate it, so an on-call engineer isn't improvising from scratch during a stressful 3am page. A blameless postmortem documents what happened and why, focused on the systems and process gaps that allowed the failure rather than which person made a mistake in the moment. The reason blamelessness matters is practical, not just cultural, if people fear blame they hide details and downplay near-misses, and you lose the honest information that would have actually prevented the next incident.

---

## Q18. [Senior] Walk me through what actually happens, end to end, from a developer merging code to that code running safely in production.

**Answer:**

This ties together most of what a production-minded engineer thinks about across a single change.

**The Path From Merge to Production**

1. A developer merges code into the main branch after code review.
2. CI triggers automatically: the code is built, unit and integration tests run, and a security or dependency scan checks for known vulnerabilities. Any failure stops the pipeline here.
3. The build is packaged, typically as a Docker image, and pushed to an image registry.
4. The image deploys to a staging environment that mirrors production, where further automated or manual checks run.
5. Production deployment begins using a strategy like canary release, sending a small percentage of real traffic to the new version first.
6. Health checks (liveness and readiness) confirm the new instances are actually up and able to serve traffic before they receive load.
7. Metrics, logs, and traces are watched closely during the rollout, error rate, latency, and any new ERROR-level log spikes, to catch a problem in the small canary slice before it reaches everyone.
8. If the canary looks healthy, the rollout gradually expands to 100% of traffic. If it doesn't, the deploy is rolled back or a feature flag is flipped off, ideally without needing a full redeploy.
9. If something does go wrong despite all this, an on-call engineer follows a runbook to mitigate, and afterward the team writes a blameless postmortem to find the underlying gap.

**Why This Whole Chain Matters**

**No single practice in this chain, tests, canaries, health checks, or observability, is sufficient by itself, it's the combination that lets a team ship changes constantly while keeping the blast radius of any single bad change small and recoverable.** A team missing even one link, say deploying without health checks, or without metrics to detect a bad canary, ends up either shipping much more cautiously than necessary or getting burned by outages that better tooling would have caught early.

**Interview Answer:**
Code gets reviewed and merged, which triggers CI to build, test, and scan it automatically, stopping the pipeline on any failure. The build gets packaged into an image, deployed to staging for a final check, then rolled into production behind a canary, a small percentage of real traffic first. Readiness checks make sure

new instances are actually ready before they take load, and I watch metrics, logs, and traces during the rollout to catch problems in that small slice before expanding further. If something goes wrong, a feature flag flip or rollback handles the immediate fix, an on-call runbook guides the response, and a blameless postmortem afterward makes sure the same gap doesn't cause the next incident.

---

**If you remember nothing else from this chapter:** *an alert nobody trusts is worse than no alert, because it still wakes someone up, just never for the incident that matters.*

# Chapter 19: Microservices Architecture & Distributed Systems Patterns

Most teams run microservices, but plenty of engineers have only worked inside one and never had to reason about how several of them coordinate. This chapter builds that muscle: the real coordination costs, data consistency across services, network calls, and cascading failure, not just the buzzwords.

> **War Story:** *A "microservices migration" split one monolith into thirty services, twenty-eight of which were still deployed together every single time, because the team had decoupled their code but never their release process.*

> **Avoid:** treating splitting into microservices as an automatic win, without discussing the operational and coordination cost it adds
>
> **Aim For:** asking what specific problem microservices are solving here before proposing the split

## Q1. [Junior] What actually makes a service a "microservice," instead of just a monolith that got split into a few files or folders?

**Answer:**

**A microservice is independently deployable and owns its own data, splitting files inside one codebase and one deployment does not give you either of those things.**

**What a real microservice has**

- Its own codebase and its own deployment pipeline, so it can be built, tested, and released without redeploying anything else.
- Its own data store, or at least its own schema, that no other service reaches into directly.
- A clear, versioned API that other services call instead of sharing internal code or tables.

**What splitting files into folders does not give you**

- Independent deployability. If everything still ships together in one build, you have a modular monolith, not microservices.
- Failure isolation. If one module crashing takes down the whole process, you haven't gained anything from the split.
- Independent scaling. You cannot give the checkout module more CPU than the reporting module if they live in the same running process.

**Why this distinction matters in an interview**

- Saying "we split it into modules" without deployment and data independence describes organization, not architecture.
- The real cost of microservices, network calls, distributed data, operational overhead, is only worth paying once you actually get independent deployability in return.

**Interview Answer:**

"A microservice needs to be independently deployable and own its own data. Just splitting a monolith's code into separate files or packages inside one deployment gives you better organization, but not the actual

benefits of microservices, like deploying one service without touching the others, or scaling one piece independently. If a crash in one module still takes down the whole process, it's still a monolith under the hood."

---

## Q2. [Junior] What goes wrong when multiple microservices share a single database, and what's the usual alternative?

**Answer:**

**A shared database quietly re-couples services that are supposed to be independent, because now nobody can change a table without checking every service that reads it.**

**Problems with a shared database**

- Any schema change is now a cross-team negotiation, since five services might read the same table.
- One service can accidentally overload the database with a heavy query, degrading every other service that shares it.
- You lose the ability to reason about which service actually owns a piece of data, since anyone can write to any table.

**The usual alternative: database per service**

- Each service owns its own database or schema, and no other service is allowed to query it directly.
- Other services get the data they need through the owning service's API, or through events it publishes.
- This does introduce new problems, like needing to keep data consistent across services, which is exactly what patterns like Saga (covered later in this chapter) exist to solve.

**A pragmatic middle ground**

- Early-stage teams sometimes start with one database and separate schemas per service inside it, then split the databases physically once operational needs (independent scaling, independent backups) demand it.

**Interview Answer:**
"Sharing one database across services quietly undoes the independence microservices are supposed to give you, since any schema change now has to consider every service reading that table, and one service's heavy query can hurt everyone else. The standard fix is database per service: each service owns its own data, and everyone else gets it through an API or an event, never a direct table read. That does mean you need patterns like Saga to keep data consistent across services, but it's a trade worth making for real independence."

---

## Q3. [Junior] What is an API gateway, and why shouldn't clients just call each microservice directly?

**Answer:**

**An API gateway is a single entry point that sits in front of your microservices, so clients talk to one address instead of needing to know about, and trust, every internal service.** For example, a mobile app that just wants to call one address, api.example.com, can hit the gateway for both the orders service and the inventory service, without ever needing to know those live at two different internal addresses that might change next month.

**What it does**

- Routes each incoming request to the right backend service based on the path or the request content.
- Handles cross-cutting concerns once, instead of in every service: authentication, rate limiting, request logging, and TLS termination.
- Can combine or reshape responses from multiple services into one response for the client, which is closer to a Backend for Frontend pattern (covered later in this chapter).

**Why direct client-to-service calls are a problem**

- Clients would need to know the internal network addresses of every service, which breaks the moment you rename, split, or move one.
- Every service would need to independently implement authentication and rate limiting, instead of getting it for free at one layer.
- Your internal architecture becomes a public contract, since a mobile app now depends directly on internal service boundaries that should be free to change.

**Interview Answer:**
"An API gateway is the single front door to your system. Clients hit one address, and the gateway routes the request to the right service behind the scenes, while also handling authentication, rate limiting, and logging in one place instead of duplicating that logic in every service. Without it, clients would need to know about every internal service directly, which locks you into your current internal structure and makes it much harder to reorganize services later without breaking clients."

---

## Q4. [Junior] When should two microservices talk to each other synchronously, and when should they use asynchronous messaging instead?

**Answer:**

**Use synchronous calls when the caller genuinely needs an immediate answer to continue, and asynchronous messaging when the caller just needs to know the work will eventually happen.**

**When synchronous (like REST or gRPC) makes sense**

- The caller cannot proceed without a result, like checking whether an item is in stock before showing an "add to cart" button.
- The interaction is naturally request and response, and the caller is willing to wait milliseconds for an answer.
- The downside: if the called service is slow or down, the caller is blocked too, so this couples the availability of both services together.

**When asynchronous messaging makes sense**

- The caller just needs to trigger work that doesn't need to finish before the caller's own request completes, like "send a confirmation email after checkout."
- You want to decouple the two services in time, so the receiving service can be temporarily down or slow without failing the caller's request.
- It naturally supports multiple independent consumers reacting to the same event, like both "send email" and "update analytics" reacting to "order placed."

**A rule of thumb**

- If a human is waiting on the screen for the result, lean synchronous. If the result is "this will happen soon" rather than "here is your answer," lean asynchronous.

**Interview Answer:**
"I look at whether the caller actually needs the answer to keep going. If a user is waiting to see whether checkout succeeded, that's a synchronous call. If checkout succeeding should also trigger a confirmation email and an analytics update, those don't need to block the response, so I'd publish an event and let those services react asynchronously. That way, if the email service is briefly down, it doesn't fail the customer's checkout, it just processes the event a bit later."

---

## Q5. [Mid] What is service discovery, and why do you need it once you have more than one instance of a service running?

**Answer:**

**Service discovery is how a caller finds a currently healthy address for a service, without hardcoding IP addresses that constantly change as instances scale up, scale down, or restart.**

**Why static addresses stop working**

- Instances get added and removed constantly, by auto-scaling, deployments, and crash recovery.
- A hardcoded IP list goes stale within minutes in any real cloud environment, especially with containers that get a new address every time they restart.

**How it typically works**

- Each service instance registers itself, or gets registered automatically, with a discovery mechanism when it starts, and deregisters when it stops.
- Callers query that registry (or a client-side library backed by it) to get a current, healthy address before making a call.

**Two common flavors**

- Client-side discovery: the calling service queries a registry (like Consul or Eureka) directly and picks an instance itself.
- Server-side discovery: the caller just talks to a fixed address (like a load balancer or a service mesh sidecar) that handles finding a healthy instance behind the scenes.

**Interview Answer:**
"Service discovery solves the problem of instances constantly appearing and disappearing as you scale, deploy, and recover from crashes. Instead of hardcoding IP addresses that go stale in minutes, each instance registers itself somewhere, and callers look up a currently healthy address before calling. Some systems do this on the client side, querying a registry like Consul directly, and others push it into infrastructure, like a service mesh sidecar, so the calling code doesn't have to think about it at all."

---

## Q6. [Mid] Explain the Strangler Fig pattern. Why would a team migrating off a legacy monolith use it instead of doing a full rewrite?

**Answer:**

**The Strangler Fig pattern replaces a legacy system piece by piece behind a routing layer, so the old system keeps running and taking traffic until every piece has been migrated and it can finally be turned off.**

**How it works**

- You put a routing layer, often the API gateway, in front of both the old monolith and the new services.
- You pick one capability, like "user profile," build it as a new service, and route just that traffic to the new service while everything else still goes to the monolith.
- You repeat this, capability by capability, until the monolith has nothing left to do, and you decommission it.

**Why not just do a full rewrite**

- Full rewrites are high risk: you're building a new system for months without shipping value, and by the time it's done, requirements have often changed.
- A full rewrite also has a terrifying single cutover moment, where you switch everyone to the new system at once and hope nothing breaks.
- Strangler Fig spreads the risk across many small cutovers instead of one big one, and the business keeps getting value from both systems the whole time.

**The trade-off**

- You do have to run and maintain two systems at once for a while, and the routing layer adds complexity during the migration.

**Interview Answer:**

"Strangler Fig means migrating a legacy monolith one capability at a time behind a routing layer, instead of doing a risky big-bang rewrite. You pick a piece, like user profiles, build it as a new service, and route just that traffic over to it while the monolith still handles everything else. You repeat that until the monolith is empty and can be shut down. It costs you running two systems at once for a while, but it avoids the classic failure mode of a rewrite that takes a year and then fails on one all-or-nothing cutover."

---

## Q7. [Mid] What is a Backend for Frontend (BFF), and why would you build one instead of pointing every client at the same shared API?

**Answer:**

**A BFF is a thin backend layer built specifically for one type of client, so each client gets exactly the shape of data it needs instead of everyone sharing one generic, compromise-shaped API.** For example, a mobile app's home screen might need a combined summary of orders, recommendations, and notifications in one lightweight response, while a web dashboard showing the same data might want the full, richer version of each, which is why a mobile BFF and a web BFF can call the same underlying services but shape the response differently.

**The problem a BFF solves**

- A mobile app usually wants small, combined payloads to save bandwidth and battery, while a web dashboard might want larger, more detailed responses.
- A single shared API ends up either bloated with fields most clients don't use, or making every client do multiple round trips to assemble what they need.

**What a BFF actually does**

- Sits between the client and the underlying microservices, and calls several of them internally to assemble exactly the response that one specific client needs.

- Often owned by the team that owns that client, so frontend developers can shape the API without waiting on a shared backend team.

**When it's worth the extra service**

- You have genuinely different client types (mobile, web, partner API) with different data shape and performance needs.
- Not worth it for a single client type, that's just an unnecessary extra hop.

**Interview Answer:**
"A BFF is a backend built for one specific client, like a mobile app, that calls into the underlying microservices and assembles exactly the response that client needs, instead of forcing every client to use one generic shared API. It's useful when a mobile app and a web dashboard genuinely need different shaped data, so each client gets an API tailored to it, often owned by the team building that client. If you only have one type of client, a BFF is just an extra hop with no real benefit."

---

## Q8. [Mid] Placing an order needs to update the order service's database and the inventory service's database as one logical operation. Why can't you just wrap that in a normal database transaction?

**Answer:**

**A normal database transaction only works within a single database connection, and since each microservice owns its own database, there is no single transaction that can span both of them.**

**Why a local transaction can't help here**

- The order service and the inventory service each have their own database, which is exactly the isolation you want for independent deployability.
- A traditional ACID transaction with commit and rollback only coordinates changes inside one database engine, it has no way to also roll back a change that already committed in a completely different database.

**What people try, and why it's usually avoided**

- Two-phase commit (2PC) can technically coordinate a transaction across databases, but it requires all participants to be available and holds locks across the network while waiting on every participant, which hurts availability and doesn't scale well across many services.
- Most systems in this situation don't try to force strong distributed transactions at all, they use the Saga pattern instead, which is the next question.

**The mental model shift**

- Instead of "all or nothing right now," you get eventual consistency: each local step commits on its own, and if a later step fails, you run compensating actions to undo the earlier ones.

**Interview Answer:**
"Each microservice owns its own database on purpose, for independent deployability, which means there's no single database connection that could wrap both updates in one ACID transaction. Two-phase commit exists to coordinate across databases, but it needs every participant online and holds locks while waiting on the slowest one, which hurts availability at scale. In practice, most teams don't try to force a single distributed transaction, they use the Saga pattern: each step commits locally, and if a later step fails, compensating actions undo the earlier ones instead of a real rollback."

## Q9. [Mid] Explain the Saga pattern for distributed transactions. What's the difference between orchestration-based and choreography-based sagas?

**Answer:**

**A saga breaks one business transaction into a sequence of local transactions across services, where every step has a matching compensating action to undo it if a later step fails.**

**The core idea**

- "Place order" might become: reserve inventory, charge payment, then confirm shipment, each a local transaction in a different service.
- If charging payment fails after inventory was reserved, the saga runs a compensating action, releasing the reserved inventory, instead of a real database rollback across services.

**Orchestration-based saga**

- A central orchestrator explicitly tells each service what step to run next, and calls the right compensating actions in order if something fails.
- Easier to understand and debug, since the whole flow lives in one place, but that orchestrator becomes a critical piece of infrastructure.

**Choreography-based saga**

- There is no central coordinator. Each service reacts to events from other services and publishes its own events when it finishes, like a chain reaction.
- More decoupled, since no service needs to know about the whole flow, but harder to trace and debug, since the "flow" only exists implicitly across many services' event handlers.

**Which to choose**

- Orchestration tends to win for complex flows with many steps and branching failure handling, since you want that logic centralized and testable.
- Choreography tends to work well for simpler, shorter flows where the decoupling benefit outweighs the debugging cost.

**Interview Answer:**
"A saga splits one business transaction into local transactions in each service, and pairs every step with a compensating action so you can undo earlier steps if a later one fails, instead of a real cross-service rollback. In an orchestrated saga, a central coordinator explicitly calls each step and runs compensations in order, which is easier to follow but adds a central dependency. In a choreographed saga, services just react to each other's events with no central coordinator, which is more decoupled but much harder to trace when something goes wrong. I'd reach for orchestration once the flow has more than a few steps or complex failure handling."

## Q10. [Mid] What is a compensating transaction, and why does every step in a saga need one?

**Answer:**

**A compensating transaction is the "undo" action for a step that already committed, and every step needs one because a saga has no real rollback once a step has succeeded.** For example, if a saga has already reserved ten units of inventory and the payment step then fails, the compensating action for that

reservation step is to release those ten units back into available stock, not to try to undo a database commit that already happened.

**Why it's not just "rollback"**

- In a normal database transaction, an uncommitted change can simply be discarded. In a saga, each step is its own committed local transaction, so by the time you know something later failed, the earlier step has already happened for real.
- Undoing it means performing a new, opposite action, releasing reserved inventory, refunding a charge, cancelling a shipment, not un-committing the old one.

**What makes a good compensating action**

- It should be idempotent, since the saga might retry the compensation itself if it fails partway.
- It should be semantically reasonable even if some time has passed, releasing inventory an hour later should still be correct, not just technically possible.
- Some steps genuinely can't be perfectly undone, like an email that already got sent, in which case the compensation might be a follow-up notice rather than a true undo.

**Interview Answer:**
"Once a step in a saga commits, there's no rollback the way there is in a single database transaction, that step already happened for real. A compensating transaction is the explicit undo: release the inventory you reserved, refund the payment you charged, cancel the shipment you created. Every step needs one because the saga has to be able to unwind cleanly no matter which later step fails, and that compensation itself needs to be idempotent since it might get retried."

---

## Q11. [Mid] What is a service mesh, and what does it give you that you couldn't already get by writing retry, timeout, and circuit breaker logic directly into each service?

**Answer:**

**A service mesh moves networking concerns like retries, timeouts, circuit breakers, and mTLS out of application code and into a sidecar proxy that runs alongside every service instance, so every team gets consistent behavior without writing or maintaining that logic themselves.** For example, in a mesh, an Envoy sidecar sitting next to the checkout service can retry a failed call to the inventory service and open a circuit breaker if inventory keeps failing, all without a single line of retry code inside the checkout service itself.

**The problem without a mesh**

- Every team implements their own retry and circuit breaker logic, in whatever language and library they prefer, so behavior is inconsistent across services.
- Upgrading that logic, say, fixing a bug in retry backoff, means touching and redeploying every single service.

**What the mesh actually does**

- A small proxy (commonly Envoy) runs next to every service instance and intercepts all network traffic in and out.
- That proxy handles retries, timeouts, circuit breaking, load balancing between instances, mutual TLS between services, and detailed traffic metrics, uniformly, regardless of what language the service is written in.

- A control plane (like Istio) lets you configure this behavior for the whole system in one place instead of per service.

**The trade-off**

- You add real operational complexity and latency overhead from the extra network hop through the sidecar, so a service mesh is usually only worth it once you have enough services that inconsistency and per-service maintenance actually hurts.

**Interview Answer:**
"A service mesh puts a small proxy next to every service instance that handles retries, timeouts, circuit breaking, and mutual TLS at the network level, instead of every team writing and maintaining that logic themselves in different languages. That gives you consistent behavior across the whole system, and lets you change something like retry policy centrally instead of redeploying every service. It does add an extra network hop and real operational complexity, so it tends to pay off once you have enough services that per-service inconsistency has become a real problem, not from day one."

---

## Q12. [Mid] Why is idempotency critical when one microservice retries a call to another, and how would you actually implement an idempotent endpoint?

**Answer:**

**Idempotency means running the same request twice has the same effect as running it once, and it matters because network failures make it impossible to know if a call actually succeeded before you decide to retry it.**

**The core problem**

- If a call to "charge payment" times out, you genuinely don't know if the payment service processed it and the response was lost, or if it never received the request at all.
- Retrying blindly risks charging the customer twice if the first attempt actually succeeded.

**How to implement it: idempotency keys**

- The caller generates a unique key (often a UUID) for each logical operation and sends it with the request.
- The receiving service stores which keys it has already processed, along with the result, before doing the actual work.
- On a retried request with the same key, the service returns the stored result instead of repeating the operation.

```
@PostMapping("/payments")
public ResponseEntity<PaymentResult> charge(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @RequestBody PaymentRequest request) {

    Optional<PaymentResult> existing = idempotencyStore.find(idempotencyKey);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get());
    }

    PaymentResult result = paymentProcessor.charge(request);
    idempotencyStore.save(idempotencyKey, result);
    return ResponseEntity.ok(result);
}
```

**Where the idempotency key gets checked and stored**

- Storing the key and the eventual result needs to happen atomically with the actual operation, otherwise a crash between "do the work" and "record the key" reopens the same race condition.

**Interview Answer:**
"A network failure doesn't tell you whether the request actually succeeded before the response got lost, so a retry could double-charge a customer if I'm not careful. I make the endpoint idempotent using an idempotency key the caller generates once per logical operation. The service checks if it's already processed that key and returns the stored result if so, otherwise it does the work and stores the result under that key. The key thing is that checking and storing the key has to happen atomically with the actual operation, or you just reintroduce the same race under a crash."

---

## Q13. [Senior] What is contract testing, like consumer-driven contracts with a tool such as Pact, and what problem does it solve that end-to-end integration tests don't?

**Answer:**

**Contract testing lets a service verify it still satisfies what its consumers actually expect from it, without needing every consumer and provider running together in one slow, flaky end-to-end environment.** For example, if the order service's mobile app consumer only ever reads the total and status fields from a response, a Pact contract capturing exactly that lets the order service safely add a new internal field without ever needing the mobile team's test suite to run.

**The problem with relying only on end-to-end tests**

- Spinning up every dependent service just to test one integration point is slow and flaky, since any one of ten services being down breaks the whole test run.
- End-to-end tests tend to be owned by nobody in particular, so they rot and get skipped under deadline pressure.

**How consumer-driven contract testing works**

- The consumer of an API writes a "contract" describing exactly what it expects from specific requests, the actual fields and structure it depends on.
- That contract is published (often through a broker like Pact Broker), and the provider service runs it against its own real implementation in its own test suite, without the consumer needing to be running at all.
- If the provider's response no longer matches what the consumer expects, the provider's build fails immediately, catching the break before it ever reaches production.

**Why this actually catches real bugs**

- It tests exactly the fields and behavior a consumer relies on, not an idealized full response, so it doesn't break on harmless additive changes but does catch changes the consumer actually depends on.

**Interview Answer:**
"Contract testing lets each consumer describe exactly what it needs from a provider's API as a contract, and the provider runs that contract against its own real code in its own test suite, without needing the consumer's service running at all. That's a lot faster and more reliable than full end-to-end tests across ten services, and it catches breaking changes exactly where they matter, in the fields a specific consumer actually depends on, right in the provider's own build, before it ships."

## Q14. [Senior] A dozen internal microservices depend on one shared API. How do you evolve that API without breaking all of them on every change?

**Answer:**

**You version the API explicitly and support the old and new versions side by side for a transition period, so consumers can migrate on their own schedule instead of being forced to update in lockstep with the provider.**

**Backward-compatible changes don't need a new version**

- Adding a new optional field, adding a new endpoint, or relaxing a validation rule usually doesn't break existing consumers, so it can ship without any version bump.

**Breaking changes need a real strategy**

- Explicit versioning, either in the URL path (`/v2/orders`) or in a header, lets you run the old and new contract simultaneously.
- The old version keeps working exactly as before while consumers migrate to the new one at their own pace, and you deprecate and eventually remove the old version once everyone's off it, with a clear communicated timeline.

**Supporting tools that reduce breakage**

- Contract testing (from the previous question) catches accidental breaking changes before they ship.
- Clear deprecation headers and monitoring which consumers still call the old version tells you when it's actually safe to remove it.

**The organizational piece**

- This is as much a communication problem as a technical one, consumers need advance notice and a real deadline, not a surprise removal.

**Interview Answer:**
"For backward-compatible changes, like adding an optional field, I don't need a new version at all. For actual breaking changes, I version the API explicitly, usually in the path like /v2/, and run both versions side by side so nobody is forced to update the moment I ship a change. I track which consumers are still calling the old version, give a clear deprecation timeline, and only remove the old version once usage has actually dropped to zero. Contract testing helps catch changes that would silently break a consumer before they ever reach production."

## Q15. [Senior] Monorepo versus polyrepo for a fifty-person engineering org running microservices: what are the real trade-offs, and which would you pick?

**Answer:**

**Monorepo makes cross-service changes and shared tooling easier at the cost of needing serious build tooling to keep it fast, while polyrepo gives clean ownership boundaries at the cost of painful cross-service changes.**

**Monorepo advantages**

- A single pull request can change a service and every consumer of its API in one atomic commit, which avoids the coordination pain of a breaking change spread across ten separate repos.

- Shared libraries, linting rules, and CI configuration live in one place, so upgrading a shared dependency is one change, not fifty.
- Code search and discovery is trivial, since everything is in one place.

**Monorepo challenges**

- Build and test times can explode without investment in tooling that only builds and tests what actually changed (like Bazel or Nx), otherwise every commit rebuilds everything.
- Access control is coarser by default, since it takes real tooling to restrict who can touch what inside one giant repo.

**Polyrepo advantages**

- Clean ownership: a team owns their repo outright, with their own CI pipeline, their own release cadence, and no risk of an unrelated team's broken build blocking their deploy.
- Each service can use different tooling, versions, or languages freely, without a shared repo-wide constraint.

**Polyrepo challenges**

- A change that spans five services means five pull requests, five reviews, and careful coordination about deployment order, which is genuinely painful.
- Shared code either gets duplicated across repos or pulled in as versioned packages, which adds its own update-and-drift overhead.

**A reasonable default at fifty people**

- Polyrepo with strong shared tooling (templated CI configs, shared library versioning, and contract testing to catch cross-service breaks) tends to be the more common real-world choice at this size, since coordination overhead across fifty engineers in one repo without heavy investment in build tooling gets painful fast. Teams that already have strong monorepo tooling experience can make monorepo work well too, this is a genuine trade-off, not a clear-cut answer.

**Interview Answer:**

"There's no universally correct answer here, it's a real trade-off. Monorepo makes cross-service changes atomic and keeps shared tooling in one place, but needs serious investment in incremental build tooling to stay fast at fifty engineers. Polyrepo gives each team clean ownership and their own release cadence, but a change spanning several services means coordinating multiple pull requests and deploy order by hand. At fifty people without existing monorepo tooling experience, I'd lean polyrepo with strong shared CI templates and contract testing to catch cross-service breaks, but I'd say this upfront as a trade-off, not a rule."

---

## Q16. [Senior] Walk me through what happens end-to-end when a "place order" request fans out across five microservices, and how you'd stop one slow or failing service from taking down the whole checkout flow.

**Answer:**

**The key idea is that the customer-facing part of checkout should only depend synchronously on the services that must succeed before you can tell them "your order is placed," and everything else should happen asynchronously afterward, protected by the same reliability patterns covered earlier in this book.**

**The synchronous, must-succeed path**

1. The client calls the API gateway, which routes the request to the order service.

2. The order service synchronously calls the inventory service to reserve stock, since the order genuinely cannot be placed if the item is unavailable.
3. The order service synchronously calls the payment service to charge the customer, since you cannot tell them the order succeeded before payment clears.
4. If both succeed, the order service commits its own local "order placed" record and returns success to the customer.

**The asynchronous, can-happen-after path**

1. The order service publishes an "order placed" event to a message queue.
2. The shipping service, notification service, and analytics service each independently consume that event and do their own work, sending a confirmation email, kicking off fulfillment, updating dashboards, without the customer waiting on any of them.

**Protecting the synchronous path from a slow or failing dependency**

- Timeouts on both the inventory and payment calls, so a hung dependency doesn't hang the whole request.
- Circuit breakers around each call, so a struggling dependency gets given a break instead of every request piling on and making it worse.
- If inventory reservation fails partway through and payment already succeeded, the saga pattern's compensating action refunds the payment, since the order can't proceed.
- If the queue publish itself fails after the order is committed, use the transactional outbox pattern, writing the event to a table in the same local transaction as the order, and a separate process reliably publishes it afterward, so the event isn't silently lost.

**Interview Answer:**

"I'd only call inventory and payment synchronously, because those are the two things that genuinely have to succeed before I can tell the customer their order is placed. Everything else, shipping, email, analytics, happens asynchronously off an 'order placed' event, so a slow notification service can never block or fail someone's checkout. On the synchronous path, I'd put timeouts and circuit breakers around both calls, and if payment succeeds but inventory reservation then fails, I'd run a compensating refund rather than leaving things half-done. And I'd use a transactional outbox so publishing that event is reliable even if the message broker has a hiccup right after the order commits."

---

**If you remember nothing else from this chapter:** *splitting a monolith into services you still deploy together every time gives you all the coordination cost of microservices with none of the independence.*

# Chapter 20: When Your Tech Lead Says This... (Rapid-Fire Judgment Calls)

None of these are "design a system" questions. Each one is a single opinionated statement a tech lead, a senior engineer, or a PR comment might actually make, some correct, some wrong, most somewhere in between depending on context. Use these to practice evaluating a claim on its merits instead of either agreeing with everything an authority figure says or reflexively contradicting it. The answer you want to give is rarely a flat yes or no, it's "here is when that is right, here is when it breaks, and here is what I would actually say back."

---

## Q1. [Mid] Your tech lead says: "Never delete data, just add isDeleted=true to every table." What is your reply?

**Answer:**

Soft delete is a good default for tables where recoverability or an audit trail has real business value, but a blanket rule for every table creates problems the tech lead probably hasn't hit yet.

*Where it breaks*

- Unique constraints on things like email or username still count soft-deleted rows, so a "deleted" user can block someone else, or themselves, from ever reusing that value unless the constraint is scoped to isDeleted = false.
- Every single query now has to remember the isDeleted = false filter, and the one place someone forgets, a new report, a cache warm job, an analyst's ad-hoc query, leaks deleted data back into the product.
- It doesn't satisfy legal right-to-erasure requirements (GDPR/CCPA), which mean actual removal or irreversible anonymization of personal data, not a flag on an intact row.
- High-churn tables (sessions, notifications) grow forever with no reclaim plan, bloating indexes and backups.

**Soft delete is the right default where recoverability matters, but it needs an archival job, a single query layer that enforces the filter automatically, and an explicit hard-delete path carved out for PII.**

**Interview Answer:**

"I'd keep soft delete for data where recovery or audit trail has real value, orders, financial records, but pair it with a scheduled purge job, put the isDeleted filter in one repository layer instead of every query, and carve out an explicit hard-delete or anonymization path for anything with personal data, since a flag alone doesn't satisfy a right-to-erasure request."

---

## Q2. [Mid] GET /users/{id}: if the user is inactive, return 403. What is your reply?

**Answer:**

403 Forbidden means "you are identified, but you are not allowed to do this," it is a statement about the caller's permission, not about the state of the resource. An inactive account is a property of the resource, so encoding it as a permission error conflates two different concerns.

*What breaks*

- If the inactive user is looking up their own profile, they get blocked from seeing why their own account is inactive, which is exactly the moment they need that information.
- An admin or support tool that legitimately needs to view inactive accounts now has to special-case around a status code that is supposed to mean "no permission," not "this record has this status."

*A better shape*

- Return 200 with the user object and an explicit field like "status": "inactive", and let the caller decide what to do with that information.
- If the actual intent is "hide this account's existence from other users," that is closer to a 404, not a 403, since 403 tells an attacker the ID is valid and merely restricted.

**Whether a user is active is resource state, not caller permission, so it belongs in the response body, not in the HTTP status code.**

**Interview Answer:**

"I'd push back on 403 here, since that status is about the caller's permission, not the resource's state. I'd return 200 with a status field on the user object, so the account owner can still see their own inactive status, and reserve 403 or 404 for actual permission or visibility decisions."

---

## Q3. [Junior] Your tech lead says: "Just add @CrossOrigin, and CORS errors will never come." What is your reply?

**Answer:**

That understates what CORS actually requires. @CrossOrigin (or an equivalent global CORS config) only allows cross-origin requests to the endpoints it is actually applied to, every other controller or a newly added endpoint someone forgets to annotate is still unconfigured and will still fail.

*The bigger trap*

- A wildcard origin ("*") combined with credentials (cookies, Authorization headers) is explicitly rejected by browsers, the CORS spec does not allow "allow everyone" plus "and send credentials" at the same time.
- Requests with custom headers or non-simple content types (like application/json) trigger a preflight OPTIONS request first, and if that is not handled correctly, the real request never goes out at all.

**CORS is not a single switch you flip once, it is a per-endpoint, per-environment, per-header configuration that has to be deliberately maintained.**

**Interview Answer:**

"@CrossOrigin only covers what it's applied to, so I'd still expect gaps on new endpoints, and if we ever add cookies or an Authorization header alongside a wildcard origin, the browser will reject it outright. I'd rather define an explicit allowed-origins list per environment and make sure preflight is handled correctly for non-simple requests."

---

## Q4. [Senior] Your tech lead says: "The frontend will send the user's role in every request, and the backend will use it for auth." What is your reply?

**Answer:**

If this means the client can set an arbitrary role field on a request and the backend trusts it at face value, that is a textbook broken access control vulnerability, a user just edits a header or a request body and becomes an admin.

*Why it matters*

- Authorization decisions must be derived from something the server itself vouches for, a signed JWT claim it issued, or a session looked up server-side, never a plain field the client is free to set.
- If what they actually mean is "the frontend forwards a signed JWT, and the backend reads the role claim out of that token," that is completely fine, the trust boundary is the signature, not the field name.

**Client-supplied data can inform the UI, but authorization must always be re-derived or verified server-side from something the client cannot forge.**

**Interview Answer:**

"I'd need to know exactly what 'sends the role' means. If it's an unsigned field the client can edit, that's a critical vulnerability, anyone can promote themselves. If it's a role claim inside a JWT the backend itself issued and verifies the signature on, that's standard and fine. The difference is entirely whether the backend can trust the source, not whether a role field exists."

---

## Q5. [Mid] Your tech lead says: "Each microservice should have its own API gateway." What is your reply?

**Answer:**

Usually this inverts the point of an API gateway. A gateway exists to centralize cross-cutting concerns, authentication, rate limiting, routing, TLS termination, so every service does not reimplement them. One gateway per microservice duplicates that logic everywhere and gives up the centralization that made a gateway worth having.

*When something like this is legitimate*

- A Backend-for-Frontend (BFF) pattern, where each frontend (web, mobile) gets its own tailored aggregation layer, is a different, deliberate pattern, not "a gateway per microservice."
- A service mesh sidecar handling per-service concerns like mTLS or retries is also a different layer than an API gateway and solves a different problem.

**A gateway is valuable precisely because it is shared; multiplying it per service reintroduces the duplicated cross-cutting logic a gateway is meant to eliminate.**

**Interview Answer:**

"I'd ask what problem they're trying to solve, because a gateway per microservice usually just duplicates auth, rate limiting, and routing logic everywhere instead of centralizing it. If the goal is a tailored API per frontend, that's a BFF pattern, which is a good idea, but it's a different thing from every backend service owning its own gateway."

---

## Q6. [Senior] Your tech lead says: "Queries are fast when cold and slow when warm." What does he mean, and how do you respond?

**Answer:**

This is a warning about misleading benchmarks, not a statement about how databases normally behave. A query run against a cold, mostly-idle system with an empty or lightly-loaded buffer pool and no contention can look deceptively fast. As the system warms up under real production conditions, working set exceeds cache size, other queries compete for buffer pool space, locks and connection pools see contention, that same query slows down.

*The implication*

- Benchmarking only against a freshly restarted, empty-cache system gives you numbers that will not hold up in production.
- You have to test under a realistic, loaded, cache-under-pressure state, with concurrent traffic, to know the number that actually matters.

**A cold-system benchmark measures the best case you will never see in production; the warm, contended state is the one that actually matters.**

**Interview Answer:**

"I take that as a warning that our benchmark numbers are probably too optimistic, since a cold cache and no contention is the best case, not the normal case. I'd want to re-run the same test under realistic concurrent load with a warmed, full cache before trusting any of those numbers for capacity planning."

---

## Q7. [Mid] Your tech lead says: "Every foreign key must be indexed." What is your reply?

**Answer:**

This is a good default, especially in Postgres, which does not automatically index foreign key columns the way some other databases do, but "must always" oversimplifies it.

*Why it is usually right*

- An unindexed FK forces a full table scan on the child table every time the parent row is deleted or updated, to check for or cascade to matching rows, which can turn a single delete into a production incident under load.
- It also speeds up the joins that foreign keys usually exist to support in the first place.

*Where it is not free*

- Every index adds write overhead and storage, and a foreign key that is rarely joined on, rarely deleted from on the parent side, or extremely low-cardinality may not be worth an index in every single case.

**It's the right default, not an absolute law: verify with EXPLAIN on the actual join and delete patterns rather than indexing every FK by rote.**

**Interview Answer:**

"I'd treat it as the correct default rather than a blanket rule, especially since Postgres doesn't auto-index foreign keys, so an unindexed one can turn a parent-row delete into a full table scan. I'd still check the actual query plans, because a rarely-touched, low-cardinality FK might not be worth the extra write and storage cost."

---

## Q8. [Mid] Your tech lead says: "Never use UUID as a primary key." What is your reply?

**Answer:**

There is a real, well-documented cost, but "never" is too strong.

*The real cost*

- A random UUIDv4 has no ordering, so inserts land at random points across a B-tree index, causing page splits and fragmentation, which hurts insert throughput and cache locality compared to a sequential integer.
- A UUID is also larger, 16 bytes versus 4 or 8, which adds up across every index that stores it.

*Why you might still want one*

- UUIDs can be generated independently on multiple nodes with no coordination, which matters for distributed systems, offline-first clients, or merging data from multiple sources.
- A sequential integer ID leaks the count and order of your records to anyone who can see it; a UUID does not.

*The actual best practice*

- Prefer a time-ordered identifier like UUIDv7 or a ULID when you need global uniqueness without coordination, since these keep most of the insert locality of a sequential key while remaining globally unique.

**The problem is randomness, not uniqueness, so a time-ordered UUID variant solves the real issue without giving up global, coordination-free ID generation.**

**Interview Answer:**

"I'd agree that a random UUIDv4 primary key hurts insert locality and bloats every index that references it, but I wouldn't ban UUIDs outright. If we need globally unique, coordination-free IDs, I'd reach for a time-ordered variant like UUIDv7 or a ULID, which keeps index locality close to a sequential integer while still being generated safely across multiple nodes."

---

## Q9. [Junior] Your tech lead says: "DELETE should always return the deleted object." What is your reply?

**Answer:**

That is a reasonable convenience, but "always" conflicts with the standard, idiomatic response for a successful delete, which is 204 No Content, precisely because there is no meaningful resource left to represent.

*The actual rule*

- If you choose to return the deleted resource for client convenience, for example so the UI can show an undo toast, you should return 200 OK with the body, not 204, since 204 explicitly forbids a response body.
- What matters most is that the choice is deliberate and consistent across the whole API, not a rule applied with no exceptions and no matching status code.

**The convention should be a consistent, deliberate API decision, and the status code must match whether or not a body is actually returned.**

**Interview Answer:**

"I'd be fine with returning the deleted object if the client genuinely needs it, but then the response should be 200, not 204, since 204 isn't allowed to carry a body. If we don't have a real use for the deleted object client-side, I'd default to 204 with no body, since that's the standard, expected shape for a successful delete."

## Q10. [Mid] Your tech lead says: "PUT /users/{id}, if the user doesn't exist, create it." What is your reply?

**Answer:**

This one is actually correct, and matches standard HTTP semantics. PUT means "replace the resource at this exact URI," and if nothing exists there yet, creating it is a valid, idempotent way to satisfy that.

*The condition that matters*

- This works cleanly when the client supplies a meaningful, predictable ID, a UUID it generated, or a natural key, since the client has to know the URI in advance to PUT to it.
- If IDs are server-generated (auto-increment), the client can't know a valid future ID up front, so create-via-PUT does not make sense there, and POST for creation plus PUT for update-only is the better split.
- A create-via-PUT should return 201 Created when it actually created something, and 200 OK when it replaced an existing resource.

**PUT-as-upsert is correct HTTP semantics when the client owns the ID; it stops making sense the moment the server is the one generating identifiers.**

**Interview Answer:**

"I actually agree with this one, PUT is defined as idempotent create-or-replace at a given URI, so upserting on PUT is standard, as long as the client is the one supplying the ID. If our IDs are server-generated auto-increment values, this wouldn't work, since the client can't know a valid ID before it exists, and I'd split create into POST and keep PUT for replace-only in that case."

---

## Q11. [Senior] Your tech lead says: "The frontend can directly query the database." What is your reply?

**Answer:**

For a production application with real users, this is almost always a serious anti-pattern, not a shortcut worth taking.

*What it breaks*

- It bypasses every layer of authorization and validation your application logic would normally enforce, the database itself has to become the entire security boundary.
- It exposes your schema and, usually, real credentials to an untrusted client, and any schema change now breaks every client directly coupled to it.
- There is no request-level auditing, rate limiting, or business-rule enforcement in the path at all.

*The one legitimate lookalike*

- Backend-as-a-service platforms with row-level security (Supabase-style Postgres RLS, Firebase security rules) look similar but are fundamentally different: authorization is still enforced, just inside the database itself via policies, not skipped entirely.

**Removing the backend layer does not remove the need for authorization, it just removes the place where authorization used to be enforced.**

**Interview Answer:**

"I'd push back hard on raw, unauthenticated direct DB access from the frontend, since that turns the database into the entire security boundary with no validation or auditing in front of it. If the goal is less

backend boilerplate, I'd look at a database with real row-level security policies, like Supabase's Postgres RLS, where authorization is still enforced, just declared closer to the data."

---

## Q12. [Mid] Your tech lead says: "The endpoint is removed, just return 404." What is your reply?

**Answer:**

404 conflates two different situations: "this specific thing was never here" (a typo'd URL, a resource that never existed) and "this used to exist and was intentionally retired." Collapsing both into 404 makes it harder for the engineers on the other end to tell a bug from a deliberate deprecation.

*A better shape*

- 410 Gone explicitly signals the resource existed and was intentionally, permanently removed, which is a different debugging signal than a plain not-found.
- Pairing that with a Deprecation or Sunset header, and pointing to migration docs, turns a confusing failure into an actionable one.

**404 says "nothing here, maybe you made a mistake"; 410 says "this was here on purpose, and it is gone on purpose," and that distinction is worth keeping.**

**Interview Answer:**

"I'd use 410 Gone instead of 404 for an intentionally retired endpoint, since 404 leaves the caller wondering whether they mistyped the URL or the endpoint never existed. 410 tells them clearly that it used to work and was deliberately removed, which is a much better debugging signal, especially paired with a deprecation notice pointing at the replacement."

---

## Q13. [Senior] Your tech lead says: "Both the public and internal APIs should accept the same JWT." What is your reply?

**Answer:**

As a blanket rule, this widens the blast radius of a leak more than it needs to. A token meant for public, user-facing calls and a token meant for trusted, internal service-to-service calls represent different trust boundaries, and collapsing them into one token means a leak of either compromises both.

*What actually should differ*

- An audience (aud) claim scoped to the specific API surface a token is valid for, so a public-facing token cannot be replayed against an internal-only endpoint even if it leaks.
- Internal service-to-service calls often warrant their own shorter-lived service identity tokens, or mTLS, rather than reusing a user's personal token at all.

**Using the same token format everywhere is fine; using the literal same token across trust boundaries means one leak compromises both.**

**Interview Answer:**

"I'd want separate tokens scoped by audience, even if they're both JWTs, rather than literally the same token accepted everywhere. If a public-facing token leaks, and it can also be replayed against internal endpoints, that leak just became a much bigger incident than it needed to be."

## Q14. [Mid] Your tech lead says: "The same JWT can be used across multiple devices." What is your reply?

**Answer:**

Technically true, a stateless JWT doesn't inherently care who holds it, but it's usually a design choice you don't actually want, for reasons of revocation and visibility rather than correctness.

*What you give up*

- You can't revoke access from one device (a lost phone) without also logging out every other device sharing that same token.
- You lose the ability to show a user "these are your active sessions" or apply device-specific policy, since there is only one token to reason about, not one per device.

**Sharing one token across devices is possible, but it trades away per-device revocation and visibility that most products actually want.**

**Interview Answer:**

"It would work mechanically, since a JWT doesn't know or care which device holds it, but I'd recommend issuing a separate access and refresh token pair per device or login instead, so revoking one compromised device doesn't force every other device to log out too."

---

## Q15. [Mid] Your tech lead says: "If a user tries to register with an existing email, return 400 Bad Request." What is your reply?

**Answer:**

400 signals that the request itself was malformed, missing fields, invalid JSON. A well-formed registration request that happens to conflict with existing state is a different situation, which is what 409 Conflict exists for.

*A separate concern worth raising*

- Returning any explicit "email already registered" message at all enables account enumeration, an attacker can probe arbitrary emails to learn who has an account. Some products deliberately respond the same way whether or not the email exists ("if this email is registered, we've sent a verification link") to avoid leaking that.

**Status code correctness (409, not 400) and information-leak avoidance (generic messaging) are two separate improvements worth making here, not one.**

**Interview Answer:**

"I'd use 409 Conflict rather than 400, since the request is well-formed, it just conflicts with existing state. Separately, I'd flag that telling the user exactly which email is taken enables account enumeration, so depending on how sensitive this product is, I might respond identically either way and let a verification email do the disambiguation instead."

---

## Q16. [Mid] Your tech lead says: "Let's partition the table by time." What is your reply?

**Answer:**

It's a strong default for append-heavy, time-series-shaped data, logs, events, orders, as long as the actual query patterns line up with it.

*Why it usually works*

- Old data can be dropped as a whole partition instead of a slow, row-by-row DELETE, which makes archival and retention cheap.
- The query planner can skip entire partitions outright when a query filters by the time column, keeping each partition small and its indexes fast.

*Where to check first*

- If most real queries filter by something other than time, a customer ID, a status, time-partitioning does not help pruning at all and just adds operational complexity.

**Time partitioning pays off exactly when most real queries already filter by time; verify that before committing to it.**

**Interview Answer:**

"I'd check what our actual queries filter on first. If most of them include a time range, partitioning by time is a great default, cheap archival, smaller indexes per partition, faster pruning. If most queries filter by something else entirely, time partitioning would not help pruning and I'd look at what the real access pattern actually is before choosing a partition key."

---

## Q17. [Mid] A teammate says: "GET works, but POST fails with a CORS error, why?" How do you explain it?

**Answer:**

This comes down to what counts as a "simple" cross-origin request under the CORS spec. A GET with no custom headers and a simple content type does not trigger a preflight, the browser just sends it and lets the response through if the server's Access-Control-Allow-Origin header matches.

*Why the POST is different*

- A POST sending application/json is not a "simple" request, so the browser first sends an automatic OPTIONS preflight asking permission before it will send the real request.
- If the server doesn't respond to that OPTIONS request with the right CORS headers, the browser blocks the POST before it ever reaches your handler, which looks like the API is broken even though the server-side POST code is never actually invoked.

**The GET skips a preflight because it qualifies as simple; the JSON POST does not qualify, so it needs an explicitly-handled OPTIONS response before the browser will send it at all.**

**Interview Answer:**

"The GET request qualifies as a CORS-simple request, so the browser sends it straight through. The POST is sending JSON, which isn't a simple content type, so the browser first fires an OPTIONS preflight to ask permission, and if that preflight response doesn't include the right CORS headers, the browser blocks the actual POST before our handler ever sees it."

---

## Q18. [Junior] Your tech lead says: "DELETE /users/{id} on success should return 204 instead of 200." What is your reply?

**Answer:**

This one is correct and matches standard practice. 204 No Content is the idiomatic response for a successful operation that has nothing meaningful left to return, which is exactly the case for a delete.

*One thing to watch*

- A 204 response must not include a body at all, if you want to return the deleted resource for convenience, that should be 200 instead, not 204 with a body attached.
- It's also worth deciding upfront what a second DELETE on an already-deleted resource should return, repeating 204 keeps the operation idempotent, while 404 is also defensible, but pick one and stay consistent.

**204 with an empty body is the correct default for a successful delete; the moment you want to return data, the status code has to change to 200 as well.**

**Interview Answer:**

"I agree, 204 is the right call for a delete with nothing left to return, and it keeps the response consistent with what 204 actually means. I'd just make sure we never attach a body to it, and separately decide whether calling delete twice on the same resource should keep returning 204 or move to 404, since that's worth being consistent about too."

---

## Q19. [Mid] Your tech lead says: "Always use cursor pagination, it's faster than offset." What is your reply?

**Answer:**

Cursor pagination genuinely wins at scale and depth, but "always" ignores real trade-offs offset pagination still solves better.

*Why cursor wins deep in a large table*

- OFFSET N forces the database to scan and discard N rows before returning the next page, so the cost grows with page depth, page 800 is far more expensive than page 1.
- Offset pagination can also skip or duplicate rows if data is inserted or deleted between page requests, since the "offset" shifts under a moving dataset.

*What offset still does better*

- Jumping directly to an arbitrary page number, and showing a total page count, is straightforward with offset and effectively impossible with a pure cursor, which only supports sequential next/previous.

**Cursor pagination is the right call for deep, high-write, infinite-scroll access patterns; offset is still the simpler, correct choice when users need to jump to an arbitrary page.**

**Interview Answer:**

"I'd use cursor pagination for anything with deep pages or a high write rate, like an infinite-scroll feed, since offset gets slower and less consistent the deeper you page. But if the product actually needs page-jump or a total page count, like a paginated admin table, offset is still the simpler, correct tool, so I wouldn't make cursor the rule for every endpoint."

---

## Q20. [Mid] Your tech lead says: "GET requests don't trigger a preflight, so CORS won't bother us there." What is your reply?

**Answer:**

That's true only for a genuinely simple GET, and it's incomplete even then, since skipping the preflight doesn't mean CORS stops enforcing anything at all.

*Where the statement breaks*

- A GET that carries a custom header, like a bearer token in a non-standard header, or a non-simple Accept value, does trigger a preflight, same as a non-simple POST would.
- Even a GET that skips the preflight is still subject to CORS at the response stage: the request itself can reach the server and execute, but the browser will block the frontend JavaScript from reading the response if Access-Control-Allow-Origin doesn't match. That's not a safe state for anything with side effects.

**Avoiding a preflight only skips an extra round trip; it does not mean CORS stops enforcing who is allowed to read the response.**

**Interview Answer:**

"That's only true for a genuinely simple GET with no custom headers. Add an Authorization header or a non-standard Accept value and it will trigger a preflight just like a POST would. And even when a GET skips the preflight, the browser still blocks the frontend from reading the response if our CORS headers don't match, so I wouldn't say CORS isn't a factor there at all."

---

## Q21. [Junior] Your tech lead says: "If the UI sends the wrong payload, just return 500." What is your reply?

**Answer:**

500 means the server hit an unexpected failure of its own; a malformed or invalid client payload is the client's fault, and HTTP has status codes specifically for that: 400 for a malformed request, 422 for a well-formed request that is semantically invalid.

*Why the distinction matters operationally*

- 5xx rates are usually what pages on-call engineers and drives alerting dashboards. Returning 500 for bad client input pages someone for a problem that isn't actually a server defect.
- It also breaks the basic 4xx-versus-5xx convention every HTTP client, proxy, and monitoring tool relies on to distinguish "your request was wrong" from "we broke."

**A 500 should mean the server failed unexpectedly; a bad request from the client belongs in the 4xx range so monitoring and alerting stay meaningful.**

**Interview Answer:**

"I'd push back on that, since 500 implies our server broke, which pages people and pollutes our error-rate dashboards for something that's actually the client's fault. I'd return 400 for a malformed payload or 422 if it's well-formed but semantically invalid, and reserve 500 for genuine, unexpected server-side failures."

---

## Q22. [Mid] Your tech lead says: "If the API receives an expired JWT, return 403." What is your reply?

**Answer:**

401 Unauthorized is the more correct choice here. 401 means "you are not authenticated," or your credentials are invalid or expired, which is exactly the situation. 403 Forbidden means "you are authenticated, but you are not allowed to do this," a different failure mode entirely.

*Why the distinction is practically useful*

- A 401 tells the client "re-authenticate or refresh your token and try again," which is actionable. A 403 suggests re-authenticating will not help, since the problem was framed as a permissions issue, not a credentials issue, which can send client retry logic down the wrong path.

**An expired token is an authentication failure, not a permissions failure, and the status code should tell the client which one it is so it knows whether refreshing the token will actually help.**

**Interview Answer:**

"I'd use 401, not 403, since the token being expired means the request isn't authenticated at all, it's not a matter of permissions. That distinction actually matters for the client, since 401 signals it should try refreshing the token, while 403 suggests refreshing wouldn't help."

---

## Q23. [Senior] Your tech lead says: "In Postgres, an UPDATE is internally a delete plus an insert." What is your reply?

**Answer:**

There's a real mechanism behind this, but the framing as literally "delete then insert" is not quite accurate, and it's worth correcting the mental model precisely.

*What actually happens*

- Postgres uses MVCC (multi-version concurrency control): an UPDATE doesn't delete the old row and create an unrelated new one. It marks the old row version as no longer current and writes a new row version linked to the same logical row, all inside one transaction, so the application sees one row being updated, not two independent operations.

*The part worth keeping from the statement*

- The practical consequence is real: old row versions (dead tuples) accumulate and need VACUUM to reclaim space, and frequent updates, especially to indexed columns, cause table and index bloat over time, which is probably what prompted the comment in the first place.

**It's row versioning under MVCC, not an unrelated delete-and-insert, but the bloat-and-vacuum consequence they're gesturing at is real and worth planning around.**

**Interview Answer:**

"I'd correct the framing a little: Postgres uses MVCC, so an update marks the old row version as dead and writes a new linked version, it's one logical row being updated transactionally, not a delete followed by an unrelated insert. But the underlying point stands, dead tuples do accumulate and need vacuuming, so frequent updates on hot tables are worth watching for bloat."

---

## Q24. [Junior] Your tech lead says: "Let's store everything as varchar, types don't matter much." What is your reply?

**Answer:**

Column types are not just documentation, they enforce integrity and correctness at the database layer, and losing them pushes that work into every piece of application code that touches the table.

*What breaks with all-varchar*

- A date or numeric column stored as text sorts and compares lexicographically, not numerically, so "9" sorts after "10" as a string, and range queries silently misbehave.

- The database can no longer validate that a value is actually a valid date or number, that check moves into application code, duplicated across every service that writes to the table.
- Numeric and date types are also smaller and faster to index and compare than variable-length text, and the query planner makes better selectivity estimates with real types.

**Types move validation and correct ordering into the database once; varchar-everything pushes that same work into every consumer, repeatedly, and loses it the first time someone forgets.**

**Interview Answer:**

"I'd push back on that. Storing dates and numbers as text breaks correct sorting and range comparisons, moves validation out of the database and into every piece of code that touches the table, and loses the storage and indexing efficiency real types give you. The type system is doing useful work for free, I wouldn't give it up."

---

## Q25. [Senior] Your tech lead says: "It's a hot key, not high traffic." What does he mean?

**Answer:**

He's distinguishing between overall system load and how that load is distributed. Aggregate traffic across the whole cluster can look completely normal on a dashboard while one specific key, one product ID, one celebrity's profile, absorbs a wildly disproportionate share of it, overwhelming the single shard, partition, or cache node responsible for that one key.

*Why the distinction changes the fix*

- Adding more overall capacity doesn't help a skewed distribution, the new capacity sits idle while the same one hot key keeps hammering the same node.
- The actual fixes are key-specific: caching that key more aggressively, replicating or splitting it, request coalescing, or spreading it across multiple physical partitions with a randomized suffix.

**Average load per node can look fine while one specific key is overwhelmed, so the fix has to target that key, not the cluster as a whole.**

**Interview Answer:**

"He means the overall traffic to the system is fine, average load looks healthy, but one specific key is getting a wildly disproportionate share of requests and overwhelming the single node responsible for it. Since that's a distribution problem, not a capacity problem, adding more servers everywhere won't help, we'd need to cache, replicate, or shard that specific key differently."

---

## Q26. [Mid] Your tech lead says: "Our system is multi-AZ." What does he mean?

**Answer:**

Multi-AZ means the system is deployed redundantly across multiple physically separate, independently-failing Availability Zones within the same region, separate power, cooling, and networking, so that one AZ having a bad day, a power outage, a network partition, does not take the whole system down.

*What it isn't*

- It's not the same as multi-region. Multi-AZ protects against a data-center-level failure within one region; multi-region spans geographically separate regions for disaster recovery and lower latency to distant users, a bigger and different guarantee with its own added complexity, like cross-region data replication and consistency trade-offs.

**Multi-AZ buys resilience to a single data center failing; it is not the same guarantee as multi-region, and conflating the two overstates how disaster-proof the system actually is.**

**Interview Answer:**

"He means the infrastructure is spread across multiple independent data centers within one region, so one of them failing doesn't take the whole system down. I'd be careful not to conflate that with multi-region, though, since multi-AZ doesn't protect against a region-wide outage or give us lower latency for users on the other side of the world."

---

## Q27. [Mid] Your tech lead says: "Just return 200 OK and put the error in the response body." What is your reply?

**Answer:**

As a default for a normal REST API, this is an anti-pattern. HTTP status codes exist so that infrastructure between the client and your handler, load balancers, monitoring, retries, caching, API gateways, can act correctly without parsing every response body.

*What it breaks*

- Error-rate dashboards and alerting that key off status codes go blind, since every response looks like a success.
- Every client now has to remember to parse the body and check for a hidden error field, and the one place that check is missed, a real failure gets silently treated as success.

*The narrow legitimate exception*

- GraphQL deliberately always returns 200 and encodes errors in a standardized errors array, because a single request can return multiple independent field-level results, some succeeding and some failing at once. That is a considered protocol design choice, not a shortcut for skipping status codes in a normal REST API.

**Status codes exist so infrastructure and monitoring can trust them without parsing bodies; hiding failures inside a 200 quietly disables all of that.**

**Interview Answer:**

"I'd push back on that for a normal REST endpoint, since it breaks every piece of tooling that keys off status codes, our monitoring, our clients' error handling, load balancer health checks. GraphQL does something similar deliberately, because one request can have multiple partial results, but that's a specific protocol decision, not a general reason to stop using status codes."

---

## Q28. [Junior] Your tech lead says: "A socket is basically the same as a port." What is your reply?

**Answer:**

They're related but not the same thing. A port is just a number, one field identifying an endpoint on a host. A socket is the full combination, protocol plus source IP and port plus destination IP and port, that identifies one specific, actual connection.

*Why the difference matters*

- A server listening on port 443 has one listening socket, but a separate connected socket for every single client connection, each distinguished by that client's unique IP and port. One port number can be involved in thousands of simultaneous sockets at once.

**A port is one field in the tuple that defines a socket; a socket is the specific connection, and a single port can be part of many sockets simultaneously.**

**Interview Answer:**

"Not quite, a port is just a number, one piece of the puzzle. A socket is the full tuple, protocol, source IP and port, destination IP and port, that identifies one actual connection. That's why a server can have one port open but thousands of simultaneous sockets, one per connected client."

---

## Q29. [Mid] Your tech lead says: "We should have a refresh token for the refresh token, a re-refresh token." What is your reply?

**Answer:**

That's reinventing a problem that already has a standard, well-understood solution: refresh token rotation.

*How rotation already solves this*

- Each time a refresh token is used, the server issues a brand-new refresh token and invalidates the old one, so the session can keep extending indefinitely without ever needing a second layer of refresh tokens.
- If an already-rotated-out refresh token is ever presented again, that is a strong signal of theft, reuse detection, and the entire token family can be revoked immediately.

**Rotation with reuse detection already gives you an indefinitely extendable, revocable session; stacking another refresh layer on top just adds more tokens to secure without solving anything new.**

**Interview Answer:**

"I'd point out that rotation already solves this, every time a refresh token is used, we issue a new one and invalidate the old, so the session extends indefinitely without a second layer. And rotation gives us something extra for free, if an old, already-used refresh token ever shows up again, that's a theft signal we can act on immediately."

---

## Q30. [Senior] Your tech lead says: "Let's shard orders by country." What is your reply?

**Answer:**

A reasonable default if the access patterns and data distribution actually support it, worth validating before committing to it.

*Why it can work well*

- Most reporting and operational queries for an orders system are often naturally scoped to one country already, and country-based sharding can double as a way to satisfy data-residency and regulatory requirements.

*Where it can go wrong*

- Country-level traffic is rarely balanced, a handful of large countries can dwarf the rest combined, creating a hot shard that defeats the point of sharding.

- Any query that needs to aggregate or join across countries, global reporting, a user who orders while traveling, becomes an expensive cross-shard fan-out.

**Sharding by country only pays off if real query patterns are already country-scoped and traffic is reasonably balanced across countries; validate both before committing.**

**Interview Answer:**

"I'd want to check two things first: whether our real queries are actually scoped by country already, and how skewed order volume is across countries, since a couple of large markets could easily overwhelm their own shard. If both check out, this can also help with data-residency requirements, but if we frequently need cross-country aggregation, that becomes an expensive fan-out on every one of those queries."

---

## Q31. [Junior] Your tech lead says: "The password is base64 encoded, so it's safe." What is your reply?

**Answer:**

This one is flatly wrong, and worth correcting immediately rather than diplomatically. Base64 is an encoding, not encryption and not hashing, it's fully and trivially reversible by anyone, with no key required, a single line in a browser console decodes it instantly.

*What it should be instead*

- A password must be hashed with a purpose-built, slow, salted algorithm like bcrypt or Argon2. Encoding and hashing solve completely different problems, and base64 provides none of the protection hashing does.

**Base64 is exactly as exposed as plaintext to anyone who intercepts or reads it; it provides zero actual protection.**

**Interview Answer:**

"I'd flag this immediately as a security issue, not a style preference. Base64 is reversible by anyone with no key at all, so an encoded password is exactly as exposed as plaintext. Passwords need to be hashed with something like bcrypt or Argon2, encoding was never meant to provide security in the first place."

---

## Q32. [Junior] Your tech lead says: "Return timestamps in the server's local time." What is your reply?

**Answer:**

This breaks the moment the system has more than one server, spans time zones, or crosses a daylight saving transition, and it's a bug that's genuinely hard to reconstruct after the fact.

*Why local time is the wrong wire format*

- Local time is ambiguous without also knowing which time zone and offset applied at that instant, and during a DST fall-back transition, some local times occur twice in the same day with no way to tell which occurrence a stored value meant.

*The standard fix*

- Always store and transmit timestamps in UTC, ideally ISO 8601 with an explicit offset or Z suffix, and let each client convert to whatever local time zone it needs to display.

**UTC on the wire and in storage is unambiguous everywhere; local time is only ever safe as a final display-layer conversion, never as the stored or transmitted value.**

**Interview Answer:**

"I'd push back on that, since local server time is ambiguous the moment we have more than one server or a client in a different time zone, and it actually becomes unrecoverable during a DST fall-back, when the same local time occurs twice. I'd store and send everything in UTC and let each client convert to its own local time only for display."

---

## Q33. [Mid] Your tech lead says: "GET /users?page=1&pageSize=0, treat pageSize=0 as return all records." What is your reply?

**Answer:**

That overloads an edge-case value with hidden, undocumented-by-convention behavior, and it is a real production risk, not just an odd API design choice.

*Why it is risky*

- pageSize=0 reads naturally as "give me zero items," so a client bug that accidentally computes a pageSize of 0 (an empty filter, a divide-by-zero somewhere upstream) would silently trigger a full, unbounded table dump instead of an empty page or a clear error.
- An unbounded response size is a real memory and latency risk on both the server and the client the moment the table is large.

**Overloading pageSize=0 to mean "everything" turns an easy client bug into an accidental full table dump instead of a harmless empty response.**

**Interview Answer:**

"I'd avoid that, since a pageSize of 0 reads as 'return nothing,' not 'return everything,' and a client bug that accidentally sends 0 would now trigger a full, unbounded dump instead of failing safely with an empty page. If we genuinely need a 'give me everything' mode, I'd make that an explicit, separate parameter, not a hidden meaning for an edge-case value."

---

## Q34. [Mid] Your tech lead says: "Never store JSON in Postgres, use MongoDB instead." What is your reply?

**Answer:**

Postgres has mature, well-indexed native JSON support (jsonb, with GIN indexes and containment and path query operators), so "never" gives up a genuinely good, well-supported feature for no clear reason.

*When Postgres jsonb is the better fit*

- If most of the data model is relational and only a handful of columns are genuinely flexible or schema-evolving, jsonb lets you keep transactional guarantees and joins across your structured tables and the flexible fields in one database, with no dual-write consistency problem and no second system to operate.

*When a document database might actually make more sense*

- If the overwhelming majority of the data model is genuinely document-shaped, deeply nested, evolving structure, with little need for cross-collection joins or multi-document transactions, a document database can be a better fit for that specific shape.

**The right choice depends on how much of the data model is genuinely document-shaped, not on rejecting a native, well-indexed feature Postgres already provides for free.**

**Interview Answer:**

"I'd push back on 'never,' since Postgres's jsonb type is well-indexed and lets us keep transactional guarantees and joins alongside our relational tables, all in one system. I'd only reach for a document database if the overwhelming majority of our data were genuinely document-shaped with little need for cross-table joins, not just because we have a few flexible fields."

---

## Q35. [Mid] GET /api/trades?page=800&size=50, what should it return?

**Answer:**

Assuming there are not at least 40,000 matching records (page 800 at 50 per page skips roughly the first 39,950), the correct, expected behavior is a 200 OK with an empty array, not an error. A page number beyond the available data is a valid, well-formed request that simply has no results, "no results" is not the same thing as "not found."

*The performance angle worth raising*

- Getting there with offset-based pagination means the database has to scan and discard roughly 39,950 rows before it can return the next 50, a cost that grows with page depth. That is exactly the deep-pagination performance cliff that motivates switching to cursor-based pagination for anything with pages this deep.

**An out-of-range page is a valid request with an empty result, not an error, but reaching page 800 via OFFSET is also the exact performance problem cursor pagination exists to avoid.**

**Interview Answer:**

"Assuming there aren't at least forty thousand matching trades, I'd expect a 200 with an empty array, since an out-of-range page is a valid request that just has no results, not something that should 404. I'd also flag that getting to page 800 with OFFSET means the database scanned and discarded around forty thousand rows to get there, which is exactly the kind of deep pagination cost that would push me toward a cursor-based approach instead."

---

**If you remember nothing else from this chapter:** *agreeing with everything an authority figure says, or disagreeing with everything they say, gives away less confidence than saying "it depends, and here's when."*

# Chapter 21: Real Interview Experience: Verbatim Questions Asked at Real Companies

Every question in this chapter was actually asked, word for word or close to it, at the company named next to it, based on real candidate-reported interview experiences. They're included as-is, across very different domains, a trading firm, a bank, a social network, a search engine, so you can see how the same core system design skills show up differently depending on what the company actually builds. Use these to calibrate: notice how a finance-flavored prompt leans harder on correctness and determinism, while a consumer-scale prompt leans harder on throughput and approximation.

---

## Q1. [Senior] Goldman Sachs asked: "Design a stock trading system."

**Answer:**

Start simple: a trading system lets users place buy and sell orders, matches those orders against each other, and reports back confirmed trades, all without ever losing an order or processing the same one twice.

*Core entities*

- Order: buy or sell, stock symbol, quantity, price, order type.
- Order Book: one per stock symbol, holding all pending buy orders and all pending sell orders separately.
- Trade: the record created when a buy and a sell order match.
- Account: each user's cash balance and stock holdings, updated when a trade executes.

*How matching actually works*

- Buy orders are sorted with the highest price first; sell orders are sorted with the lowest price first.
- A match happens the moment the highest buy price is greater than or equal to the lowest sell price, at which point a trade is created and both accounts are updated.

*Why correctness comes before speed*

- Every order and trade is written to a durable, ordered log first, so the system can recover its exact state after a crash.
- Each stock symbol is handled by one matching engine at a time (or its orders are processed through one ordered queue), so two orders for the same stock can never race each other and cause a double-fill.

**a trading system's one non-negotiable requirement is that no order is ever lost, duplicated, or double-filled, which is why correctness and ordering are designed for before raw speed.**

**Interview Answer:**

"I'd model it around an order book per stock symbol, with buy orders sorted by highest price and sell orders sorted by lowest price. A matching engine dedicated to each symbol continuously checks whether the best buy price meets or beats the best sell price, executes the trade, and updates both accounts atomically. I'd process each symbol's orders through a single ordered queue so there's no race condition between two orders for the same stock, and I'd persist every order and trade to a durable log first, since losing or double-processing a trade is the one failure mode a trading system can never have."

## Q2. [Junior] Goldman Sachs asked: "Design a grading system. (asked to a summer intern candidate)"

**Answer:**

Think of it as: students submit assignments, a grader (a person or an auto-grader) evaluates them, and the system stores and reports scores, applying rules like weighting and late penalties.

*Core entities*

- Student, Course, and Assignment (each assignment has a max score and a weight toward the final grade, like "worth 20% of the final grade").
- Submission: a student's answer to one assignment, with a timestamp.
- Grade: a separate record from the submission, holding the score and any feedback.

*Key decisions worth calling out*

- Keep Submission and Grade as two separate records, not one, so re-grading something never destroys the original submission.
- Make policies like the late penalty a configurable rule per course (for example, minus 10% per day late) rather than a hardcoded number, since every class handles lateness differently.
- Some assignments can be auto-graded instantly (multiple choice); others need a human reviewer and should sit in a review queue until graded.

**the actual skill being tested is turning an everyday, familiar concept into a clean set of entities and relationships, exactly like any other class-design question.**

**Interview Answer:**

"I'd model Students, Assignments, Submissions, and Grades as separate entities, with each assignment having a weight toward a final grade. I'd keep submissions and grades as separate records so a re-grade doesn't destroy the original submission, and I'd make policies like late penalties configurable per course rather than hardcoded, since every class handles that differently."

---

## Q3. [Senior] Morgan Stanley asked: "Design a system that processes multiple live data feeds and exposes an API to serve various queries over that data."

**Answer:**

This is a data ingestion pipeline problem: information is arriving continuously from several outside sources, each with its own format and speed, and it all needs to end up queryable through one clean API.

*Step by step*

- Ingestion: put one adapter in front of each feed source, whose only job is translating that source's specific format into one common internal event shape, so the rest of the system never has to think about individual vendor formats.
- Decoupling: publish every normalized event onto a message stream, like Kafka, so a slow or backed-up downstream consumer never causes the live feed itself to stall or drop data.
- Processing: stream processors read off that queue and update two different stores, a fast in-memory "latest value" store for real-time lookups, and a durable time-series store for historical or range queries.
- Serving: the API picks whichever store actually fits the question being asked, the in-memory store for "what's the value right now," the time-series store for "show me the last hour."

**the core trick is decoupling ingestion, which is unpredictable and bursty, from serving, which needs to stay fast and stable, by putting a queue in between them.**

**Interview Answer:**

"I'd put a normalizing adapter in front of each feed so every source becomes one common event format, then publish everything onto a stream like Kafka so ingestion never blocks on a slow consumer. From there, I'd maintain two stores, an in-memory latest-value store for real-time queries and a time-series store for historical ones, and let the API pick whichever store fits the query it's answering."

---

## Q4. [Senior] JPMorgan Chase asked: "Design a rate limiter with per-user configurable quotas, a resetting time window, and the ability to update quotas at runtime without restarting the service."

**Answer:**

A basic rate limiter enforces one fixed rule for everyone. This version adds three real requirements on top: different limits for different users, a window that resets over time, and the ability for an admin to change a limit live, with no redeploy.

*How to support that*

- Store each user's quota and window as data, not as a constant in the code, for example in Redis as user_id mapped to a limit and a window length.
- Enforce it with a token bucket per user: each user's bucket refills steadily up to their configured limit, and every request consumes one token, with an empty bucket meaning the request is rejected or delayed.
- Cache each user's config briefly in memory so the service isn't hitting Redis on every single request just to check the limit, refreshing that cache every few seconds.

*Why runtime updates just work*

- Because the limit is a value in a data store, not a number baked into the code, an admin updating that value takes effect on the very next request or the next cache refresh, no deployment involved.

**treating the quota as configurable data instead of a hardcoded constant is the single design choice that makes live updates possible at all.**

**Interview Answer:**

"I'd store each user's quota and window as data in Redis, not in code, so an admin changing a value takes effect immediately. I'd implement enforcement with a token bucket per user, refilling at a rate derived from their quota, and I'd cache each user's config briefly to avoid a Redis lookup on every single request. The key difference from a basic rate limiter is treating the limit itself as configurable data instead of a constant."

---

## Q5. [Mid] Citadel asked: "Design a thread-safe shared counter. What are the different ways to make it safe, and which would you pick?"

**Answer:**

A shared counter is one number that many threads read and increment at the same time, for example counting total requests processed. Without protection, two threads can read the same value, both add one, and an increment silently gets lost, a classic race condition.

*The options*

- Lock-based: wrap the increment in a mutex so only one thread updates it at a time. Simple and correct, but every thread waits its turn, which gets slow under heavy contention.
- Atomic operations: use a hardware-supported atomic increment, which updates the value in one indivisible step with no lock at all. Faster than locking under most levels of contention.
- Sharded counters: split the single counter into several smaller counters, one per thread or CPU core, each incremented independently with zero contention, and only add them together when someone actually needs to read the total.

**the right technique depends entirely on how often the counter is written versus read, there is no single universally correct answer.**

**Interview Answer:**

"For a simple case, I'd use an atomic increment instead of a lock, since it avoids threads blocking each other for something as small as adding one. If this counter is being hit extremely often by many threads at once, I'd shard it into one counter per thread or core, incremented independently, and only sum them up when someone actually reads the total, since that removes contention entirely on the hot write path."

---

## Q6. [Senior] Citadel asked: "Design a low-latency store for stock price time-series data. What makes this different from a normal database?"

**Answer:**

Stock price data has a very specific shape: extremely high write volume, one price tick after another for every symbol, written once and essentially never updated, and almost always queried by time range, like "show AAPL for the last hour."

*Why a normal database struggles here*

- A typical relational database is built around B-tree indexes optimized for random reads and updates, which is the opposite of "append constantly, read by time range."

*A design that fits the shape of the data*

- Use a time-series-oriented storage engine that writes data in time-ordered, append-only segments, which is extremely fast for this exact write pattern.
- Partition storage by symbol and time window, so a query for one symbol's last hour only has to touch one small segment instead of scanning everything ever written.
- Keep the most recent few minutes of data in memory for the lowest possible read latency, and asynchronously flush older segments to disk in a compressed format.
- Use delta encoding for compression, storing the difference from the previous value instead of the full number, since consecutive price ticks are usually close together.

**price data is append-only and time-ordered, so the storage engine should be built around fast sequential writes and time-range reads, not general-purpose random access.**

**Interview Answer:**

"I'd move away from a general relational database and use a time-series-oriented storage model: data partitioned by symbol and time window, written in append-only, time-ordered segments. I'd keep the last few minutes of data in memory for the lowest-latency reads, and flush older segments to disk in a compressed, delta-encoded format, since consecutive price ticks usually only change by a small amount."

## Q7. [Senior] Meta asked: "Design a large-scale recommendation system for a news feed. How do you think about data, features, models, and evaluation at a high level?"

**Answer:**

A feed recommendation system decides, out of thousands of possible posts, which handful to actually show one user, ranked by predicted relevance.

*Data*

- What the system knows: a user's past likes, comments, and shares, who they follow, and post metadata like author, content type, age, and engagement so far, all logged continuously as people use the product.

*Features*

- Raw data gets turned into signals a model can actually use, like "how often has this user engaged with this author before" or "how much engagement has this post gotten in the last hour," computed both from historical batch data and real-time signals.

*Model*

- At this scale, a two-stage approach is standard: a lightweight, fast model narrows thousands of candidate posts down to a few hundred, then a heavier, more accurate model ranks just that smaller set precisely.

*Evaluation*

- Before a model change ships to everyone, it runs as an A/B test on a small slice of users, comparing real metrics like time spent or return visits against the old model, since offline accuracy doesn't always predict real behavior.

**the two-stage pattern, a cheap filter followed by an expensive rank, is what makes recommendation at this scale computationally feasible at all.**

**Interview Answer:**

"I'd think of it as two stages: a lightweight model that quickly narrows thousands of candidate posts down to a few hundred, followed by a heavier, more accurate model that ranks just that smaller set. Features would come from both offline historical behavior and real-time signals like how a post is performing right now. And I'd never trust offline accuracy alone, any model change needs an A/B test measuring real engagement before it ships to everyone."

## Q8. [Senior] Google asked: "Given massive search log data and a fixed number of servers, design a system to produce a report of the top 1 million most frequent search queries."

**Answer:**

This is a classic "find the most frequent items in a dataset too big for one machine" problem, often called heavy hitters or top-K.

*Why the naive approach fails*

- Counting every query in one giant hash map on a single machine does not work, the data does not fit in one machine's memory and one machine cannot count fast enough.

*A MapReduce-style approach*

- Map: split the raw logs across many machines, and have each machine count the queries within its own chunk of data, producing many small (query, count) pairs.
- Shuffle: route all counts for the same query to the same reducer machine (by hashing the query string), so each reducer sees every occurrence of a given query across the entire dataset.
- Reduce: each reducer sums the counts it received for its queries, producing a final total count per query.

*Finding the top 1 million without sorting everything*

- Each reducer keeps only a min-heap capped at 1 million entries, the biggest counts it has seen so far, discarding anything smaller than the current smallest entry in that heap.
- The final step merges the per-reducer top lists into one global top 1 million.

**the two big ideas are splitting the counting work across many machines with MapReduce, and never holding more than "the top million so far" in memory with a bounded heap, instead of sorting the entire dataset.**

**Interview Answer:**

"I'd treat this as a MapReduce problem: split the logs across many machines, have each one count queries in its own chunk, then shuffle so all counts for the same query land on one reducer, which sums them. To find the top 1 million without sorting the entire dataset, I'd keep a min-heap capped at 1 million entries on each reducer, discarding anything smaller than the current minimum, then merge the per-reducer top lists at the end."

---

## Q9. [Mid] LinkedIn asked: "Design a system to integrate education certificates from external vendors, like Coursera or a university, into a user's LinkedIn profile."

**Answer:**

This is fundamentally a third-party integration problem: an outside vendor needs to tell LinkedIn "this user completed this certificate," and LinkedIn has to display it on the correct profile, safely, without letting anyone fake a credential.

*Step by step*

- Vendor integration: each vendor either calls a webhook to notify LinkedIn the moment a user completes a course, or LinkedIn periodically pulls updates from the vendor's API.
- Identity matching, the hardest part: instead of guessing based on a matching email address, the user explicitly connects their vendor account to LinkedIn through an OAuth-style consent flow, so there is never any ambiguity about which LinkedIn account a certificate belongs to.
- Verification: a certificate is only marked verified if it came from a vendor on a trusted, pre-approved allowlist, an arbitrary unverified claim is not treated as a real credential.
- Async processing: certificate events are handled through a queue, since a vendor might send a large burst of completions at once, like at the end of a course cohort, and that burst should never block or slow anything down.

**the hard problem here isn't storage, it's establishing trustworthy identity linkage between a vendor's user and the correct LinkedIn account, which is best solved with explicit consent, not guesswork.**

**Interview Answer:**

"I'd have the user explicitly connect their vendor account to LinkedIn through an OAuth-style consent flow, so there's no ambiguity about which LinkedIn account a certificate belongs to. Certificate events would come in through a queue, since a vendor might send a burst of completions at once, and I'd only mark a

certificate as verified if it came from a vendor on a trusted allowlist, rather than accepting any unverified claim at face value."

---

## Q10. [Mid] LinkedIn asked: "Design an online monitoring system. What are the core building blocks?"

**Answer:**

A monitoring system collects metrics from many services, stores them efficiently, and lets engineers query and alert on them, without becoming so heavy that it slows down the very services it watches.

*Core building blocks*

- Collection: a lightweight agent running alongside each service, emitting metrics like request count, latency, and error rate at regular intervals.
- Ingestion: metrics are sent through a queue rather than directly into storage, so a burst of metrics can never block or slow down the service producing them.
- Storage: metrics are timestamped numbers, so they belong in a time-series database, the same underlying shape of problem as the trading time-series question, append-heavy and queried by time range.
- Aggregation: raw per-instance metrics get rolled up, like an average latency across every instance of a service, so a dashboard isn't processing millions of raw data points.
- Alerting: a separate rules engine continuously checks metrics against thresholds, like error rate above 5% for 5 minutes, and fires a notification, kept independent from storage so changing a rule never risks the data pipeline.

**monitoring must never become a burden on the services it watches, which is why collection stays lightweight and every step is asynchronous, never a blocking call.**

**Interview Answer:**

"I'd break it into lightweight agents that emit metrics from each service, a queue-backed ingestion pipeline so bursts don't get lost or block the service being monitored, and a time-series database for storage, since metrics are timestamped numbers queried by time range. On top of that, I'd separate alerting, a rules engine watching for threshold breaches, from the dashboards that just visualize the data, so changing an alert rule never risks the storage pipeline."

---

## Q11. [Senior] Urban Company asked: "Design an Uber Carpool system (LLD). List the classes you'd create and how they interact."

**Answer:**

Regular ride-sharing matches one rider to one driver for one trip. Carpool is harder because several riders, headed to different destinations, can share the same car during overlapping trips, so the system has to decide who can ride together, not just who's closest.

*Core classes*

- Rider: requests a ride with a pickup location, destination, and time.
- Driver: has a current location and a car with a fixed seat capacity.
- RideRequest: one rider's request, pending until it gets matched to a trip.
- Trip: an active carpool trip, holding a driver, the list of matched RideRequests it currently carries, and a computed route of interleaved pickups and drop-offs.

- MatchingEngine: given a new RideRequest, decides whether it fits into an existing in-progress Trip or needs a brand new one.
- RouteCalculator: given a Trip's pickups and drop-offs, computes the most efficient order to visit them.

*How they interact*

- A new RideRequest arrives, and the MatchingEngine checks nearby in-progress Trips that still have a free seat.
- For each candidate Trip, the RouteCalculator estimates how much extra time inserting this rider would add for everyone already in the car.
- If that added detour stays under a small threshold, the rider joins the existing Trip; otherwise, a new Trip is created for them.

**the core new complexity versus regular ride-sharing is the MatchingEngine deciding whether adding a rider only slightly lengthens the trip for everyone already in it, a constraint one-rider-one-driver matching never has to consider.**

**Interview Answer:**

"The main difference from regular ride-sharing is that a MatchingEngine has to decide whether a new rider can join an already-in-progress Trip without adding too much detour for the riders already in the car, not just find the nearest free driver. I'd model a Trip as holding a driver and a list of matched ride requests, with a RouteCalculator computing the best order of pickups and drop-offs, and I'd only add a new rider to an existing trip if the estimated added detour stays under a small threshold, otherwise they get their own trip."

# Chapter 22: Top 10 Most Asked System Design Questions

These are the ten system design prompts that come up most often across interviews, so this chapter doubles as a quick calibration tool. If you've genuinely practiced system design, your answer here should come out structured and confident, not something you're improvising for the first time in the room.

---

## Q1. [Mid] Design a URL Shortener like bit.ly or TinyURL. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements, meaning what the system must actually let a user do:
- Given a long URL, generate a short URL (like bit.ly/ab12Cd) that redirects to it.
- Let a user optionally pick a custom short code (a "vanity" alias) if it is still available.
- Redirect anyone who visits the short URL to the original long URL almost instantly.
- Optionally let a link expire on a set date, and show the owner how many times it was clicked.
- Non-functional requirements, meaning how well the system needs to perform rather than what it does:
- Redirects must be very fast, well under 100 milliseconds, since a slow redirect feels broken.
- High availability: the redirect (read) path must almost never go down, even if link creation (write) has a hiccup.
- Short codes must never collide, meaning two different long URLs must never end up sharing one short code.
- The system is heavily read-skewed: people click links far more often than they create them.

**Scale/capacity estimate**

- Assume 100 million new short links are created per month.
- 100,000,000 / 30 days = about 3.3 million new links per day.
- 3,300,000 / 86,400 seconds in a day = about 40 writes per second on average.
- Assume a rough 100:1 read-to-write ratio (each link is clicked far more often than it is created).
- 40 writes/sec x 100 = about 4,000 redirect reads per second on average, with much higher spikes for viral links.
- Assume each stored record (short code, long URL, metadata) is about 500 bytes.
- Over 5 years: 100,000,000/month x 60 months = 6 billion links created.
- 6,000,000,000 x 500 bytes = 3,000,000,000,000 bytes, which is about 3 terabytes (TB) of storage, cheap and small compared to the traffic problem.

**Architecture**

A client asks the API to shorten a long URL; the service generates a unique short code, stores the mapping, and returns the short URL. When anyone later visits the short URL, the service checks a cache first, falls back to the database on a miss, and issues an HTTP redirect to the original URL.

```
Write path:
Client -> API Server -> ID Generator (unique code) -> Database (code -> long URL)
                                                   -> Cache (write-through)
```

```
Read path:
Client -> Load Balancer -> API Server -> Cache (hot codes) -> Database (cache miss)
                                      -> 301/302 Redirect back to Client
```

- API Server: accepts "shorten this URL" and "take me to this code" requests; stateless, so many copies can run behind a load balancer.
- ID Generator: produces a unique short code for every new long URL.
- Database: stores the mapping from short code to long URL, plus metadata like creation date and expiry.
- Cache (an in-memory, very fast lookup layer, such as Redis): holds the most frequently visited codes so most redirects never touch the slower database.
- Load Balancer: spreads incoming traffic across many API servers so no single machine gets overwhelmed.

**Key design decisions**

- **How do we generate unique short codes?**
- Option 1: hash the long URL (for example with MD5) and take the first 6 to 8 characters. Problem: different URLs can produce the same truncated hash, called a collision, so you need extra collision-checking logic.
- Option 2: use a counter, a number that only ever goes up, and convert that number into a short string using base62 encoding, which writes numbers using 62 symbols (a-z, A-Z, 0-9) instead of just 10 digits, so the same number needs fewer characters. **Using an auto-incrementing counter converted to base62 is the best choice because it guarantees a unique code by construction, with no collision checks needed.** A 7-character base62 code can represent over 3.5 trillion values, more than enough headroom.
- To stop every server fighting over one shared counter, hand out ranges in blocks: each API server asks a central coordinator for a block of 10,000 IDs at a time and uses them locally before asking for the next block.
- **Which redirect status code should we return, 301 or 302?**
- A 301 (permanent redirect) lets browsers cache the redirect and skip your server next time, which is faster but means you lose the ability to count that click and cannot change the destination later.
- A 302 (temporary redirect) always comes back to your server, slightly slower per click, but lets you log every click and change the destination anytime.
- 302 is the better default for most products, because click analytics is usually a requirement, and the extra latency is small.
- **How do we make reads fast at scale?**
- Put a cache in front of the database holding the most popular codes, since a small fraction of links usually receive the large majority of clicks.
- Use a Content Delivery Network (CDN), a network of servers spread around the world close to users, for extremely popular, long-lived links, so a redirect never has to travel back to one central data center.
- **How do we shard the database as it grows?**
- Split (shard) the database by short code, for example using the first character or a hash of the code to decide which machine owns it, so no single machine holds all the data or handles all the traffic.

**Hardest edge case**

- Two API servers generating a code for two different long URLs at the exact same instant must never produce the same code; the counter-plus-block approach solves this because each server owns a private range of IDs that never overlaps another server's range.
- A hand-picked vanity alias can collide with a code the counter already generated, so vanity aliases need an explicit uniqueness check against the database before being accepted.

**Interview Answer:**

I'd generate short codes using an auto-incrementing counter converted to base62, since that guarantees uniqueness without collision checks, and I'd hand out counter ranges in blocks to each server so they never step on each other. Writes go to a database keyed by short code; reads go through a cache first, since redirects are extremely read-heavy and a small number of links get most of the traffic. I'd use a 302 redirect instead of 301, since I want every click to hit my server for analytics. For scale, I'd shard the database by short code and cache hot codes near users.

---

## Q2. [Mid] Design Instagram: users upload photos, follow other users, and see a feed of photos from people they follow. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Upload a photo with a caption.
- Follow and unfollow other users.
- View a feed, a scrollable list of posts, from people you follow.
- Like and comment on posts.
- Non-functional requirements:
- Read-heavy: users view feeds far more often than they post, roughly 100 reads for every write.
- Low feed latency, so opening the app feels instant.
- High availability.
- Eventual consistency, meaning it is fine if a like count or new follower takes a few seconds to appear everywhere, is acceptable for likes and follower counts, but not for the photo itself.

**Scale/capacity estimate**

- Assume 500 million daily active users, each posting on average once every 10 days.
- 500,000,000 / 10 = 50 million posts per day.
- 50,000,000 / 86,400 seconds = about 580 writes per second on average.
- Assume each user checks their feed 5 times a day.
- 500,000,000 x 5 = 2.5 billion feed reads per day.
- 2,500,000,000 / 86,400 = about 29,000 reads per second, roughly 50 times the write rate.
- Assume an average uploaded photo is 2 MB after compression.
- 50,000,000 photos/day x 2 MB = 100 million MB, which is 100 terabytes (TB) of new photo data per day, far too much for a normal database and a reason photos live in dedicated object storage.

**Architecture**

Uploads go straight to cheap, large-scale object storage rather than a database, and only the resulting link plus metadata is recorded. The feed is precomputed for most users so reading it is a cheap cache lookup, not a live computation.

```
Client -> API Gateway -> Post Service -> Object Store (photo blobs) + CDN
                                      -> Metadata DB (post info, follow graph)
                                      -> Fanout Worker -> Feed Cache (per-user timeline)
Client -> Feed Service -> Feed Cache (fast read)
```

- API Gateway: the single front door for client requests, routes them to the right internal service.
- Post Service: handles uploads, writes the photo to object storage, and writes metadata (caption, owner, timestamp) to the database.
- Object Store (like Amazon S3): cheap, durable storage built for large files, not meant to be queried like a database.
- CDN: copies of the photo cached on servers physically close to users, so downloads are fast worldwide.
- Metadata DB: stores who posted what and when, and the follow graph, meaning who follows whom.
- Fanout Worker: a background process that, on a new post, pushes it into followers' precomputed feed lists.
- Feed Cache: an in-memory store holding each user's ready-to-serve list of recent posts.

**Key design decisions**

- **Should the feed be built with fanout-on-write or fanout-on-read?**
- Fanout-on-write means the moment someone posts, the post is immediately copied into every follower's precomputed feed list, making reads instant but making one post from a hugely popular account trigger millions of writes at once.
- Fanout-on-read means nothing is precomputed; the feed is assembled by fetching and merging posts from everyone you follow at the moment you open the app, which avoids wasted work but makes every read expensive.
- **The practical answer is a hybrid: fanout-on-write for ordinary users, but fanout-on-read for celebrity accounts with millions of followers, so one post never triggers millions of instant writes.** This is often called solving the "celebrity problem."
- **How do we store the follow graph efficiently?**
- Store it as a simple adjacency list, a table listing for each user who they follow, indexed in both directions so "who do I follow" and "who follows me" are both fast lookups.
- **How do we handle like and comment counts on a viral post without slowing it down?**
- Writing directly to a single "likes" counter on a viral post creates a hot row, meaning many requests fighting to update the exact same database row at once.
- Instead, route likes through a separate write-behind counter service that batches updates and flushes them periodically, instead of hammering the post's row on every single like.

**Hardest edge case**

- A celebrity with 50 million followers posting a photo: fanning that out to 50 million feed lists immediately would be far too slow and resource-hungry, which is exactly why the hybrid fanout approach above exists.

**Interview Answer:**

I'd split this into ingestion and delivery. Uploads go straight to object storage behind a CDN, with only metadata like caption and owner in the database, keeping writes cheap. For the feed, I'd use fanout-on-

write into a per-user cache for most users, so reading a feed is just a cache lookup. For celebrity accounts with huge follower counts, I'd switch to fanout-on-read and merge their posts into the feed at request time, since instantly fanning one post out to millions of followers is wasteful. I'd also move like counts to a write-behind counter service to avoid hot-row contention.

---

## Q3. [Mid] Design YouTube or Netflix: users upload and stream video at scale, with playback that adapts to network conditions. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Upload a video.
- Convert (transcode) it into multiple resolutions and qualities.
- Stream video to viewers, including pausing and resuming.
- Track views for later use in recommendations.
- Non-functional requirements:
- Massive read (viewing) traffic compared to uploads.
- Global low-latency delivery, meaning playback starts quickly anywhere in the world.
- Storage efficiency, since raw video files are large.
- Adaptive playback, meaning smooth viewing even on a weak or changing network connection.

**Scale/capacity estimate**

- Assume 500 hours of video are uploaded every minute.
- 500 x 60 = 30,000 hours uploaded per hour; x 24 = 720,000 hours of new video per day.
- Assume 1 billion hours of video are watched per day globally.
- 1,000,000,000 / 24 = about 41.6 million watch-hours per hour, which is about 41,600,000 / 3,600 = roughly 11,500 concurrent streams on average, with far higher peaks in the evening.
- Assume raw video averages 1 GB per hour, and transcoding into about 5 quality levels roughly doubles total stored size.
- 720,000 hours/day x 1 GB x 2 = about 1.4 petabytes (PB), meaning 1,400 terabytes, of new storage needed every single day.

**Architecture**

A video is uploaded to blob storage, then a pipeline transcodes it in parallel into multiple qualities and splits each into small segments. A manifest file lists what is available, and the player downloads segments and switches quality dynamically based on measured network speed.

```
Uploader -> Upload Service -> Raw Store (blob storage)
                           -> Transcoding Pipeline (queue + parallel workers)
                           -> Encoded Segments (multiple qualities) -> CDN
Client -> CDN (manifest + segments) -> Adaptive Bitrate Player
```

- Upload Service: accepts the raw file and stores it in blob storage.
- Transcoding Pipeline: a queue of jobs processed by many workers in parallel, converting raw video into several resolutions and bitrates, meaning how much data per second the video uses, split into small chunks.
- CDN: caches and serves video segments from locations close to the viewer.

- Adaptive Bitrate Player: the app that reads the manifest and requests the next segment at whatever quality current bandwidth supports.

**Key design decisions**

- **How do we deliver video that plays smoothly even on bad networks?**
- The naive approach, one file at one fixed quality, buffers and stalls whenever the network slows down.
- **The standard solution is adaptive bitrate streaming: split the video into many small segments at several quality levels, and let the player pick a different quality segment by segment based on measured bandwidth, instead of committing to one quality for the whole session.** This is commonly implemented with formats called HLS or DASH.
- **How do we control transcoding cost?**
- Transcoding every uploaded video into every quality immediately is expensive, and most videos are watched by very few people, a pattern called the "long tail."
- Pre-transcode popular or promoted content into all renditions right away; transcode long-tail content lazily, meaning only generate a rendition the first time it is actually requested, then cache the result.
- **How do we keep delivery fast worldwide?**
- Use a multi-tier CDN: edge servers close to users cache popular, recently watched content, while less popular catalog content sits in cheaper, more central storage and is pulled to the edge on demand.

**Hardest edge case**

- A viewer's network speed suddenly drops mid-video, for example walking off Wi-Fi onto mobile data; the player must detect this quickly and drop to a lower-quality segment without a visible stall, which is the entire point of the segment-by-segment adaptive design.

**Interview Answer:**

I'd focus on transcoding and delivery. On upload, a pipeline splits the video into chunks and transcodes each into multiple resolutions and bitrates in parallel, producing segmented output. On playback, the client reads a manifest and adaptively switches quality segment by segment based on measured bandwidth, so a viewer on a weak connection degrades gracefully instead of buffering. Everything is served through a multi-tier CDN, with popular content cached at the edge and long-tail content transcoded lazily, meaning only on first request, to control cost.

---

## Q4. [Mid] Design a Web Crawler that discovers and downloads pages across the internet at scale. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Starting from seed URLs, discover new links on each page.
- Download (fetch) each page.
- Extract new links to visit next.
- Store downloaded content for later indexing, meaning building a searchable catalog of it.
- Non-functional requirements:

- Scale to billions of pages.
- Be polite, meaning respect each site's rate limits and its robots.txt file, a text file websites use to tell crawlers which pages they may visit and how fast.
- Avoid infinite loops and re-downloading the same page repeatedly, called deduplication.
- Be resumable after a crash, without losing all prior progress.

**Scale/capacity estimate**

- Assume the goal is to crawl 10 billion pages, refreshed on average every 30 days.
- 10,000,000,000 / 30 = about 333 million fetches per day.
- 333,000,000 / 86,400 seconds = about 3,850 fetches per second on average.
- Assume an average page is 100 KB.
- 333,000,000 x 100 KB = about 33.3 terabytes (TB) of new content per day.
- Assume the frontier, the queue of URLs not yet visited, holds 1 billion pending URLs at any time, at about 100 bytes each.
- 1,000,000,000 x 100 bytes = 100 GB just to hold the queue, which is why it must be a distributed, sharded structure, not one machine's memory.

**Architecture**

URLs to visit sit in a distributed queue called the frontier, split by website (host), so each site's crawl speed can be controlled independently. Workers pull URLs, check the site's rules, fetch the page, extract new links, and check whether each link was already seen before re-queuing it.

```
Seed URLs -> Frontier (priority queue, sharded by host)
          -> Fetcher Workers -> robots.txt check -> HTTP GET
          -> Parser -> extract links + content
          -> Dedup (URL fingerprint store, e.g. Bloom filter) -> back to Frontier
          -> Content Store / Index
```

- Frontier: a distributed priority queue, meaning more important or more likely-to-have-changed pages get visited sooner, sharded by host.
- Fetcher Workers: pull a URL, check robots.txt and the rate limit, then download the page.
- Parser: reads the page's HTML and extracts both useful content and new links.
- Dedup store: a fast way to check "have I seen this URL before," typically a Bloom filter, a compact, probabilistic structure that can quickly say "definitely not seen" or "probably seen," using far less memory than storing every URL exactly.
- Content Store / Index: where downloaded content lands for a separate search-indexing system to process.

**Key design decisions**

- **How do we stay polite and avoid overwhelming any one website?**
- The tension is between crawling fast overall and never overwhelming a single host. **Sharding the frontier by host, and giving each host its own rate limiter and crawl-delay timer, is the single design choice that stops the crawler from either overwhelming small sites or crawling too slowly.** Because every URL for a host lands on the same shard, that shard alone controls the pace for that host, with no cross-cluster coordination needed.
- **How do we avoid re-crawling the same page millions of times?**
- Storing every seen URL exactly at a scale of tens of billions would need huge memory to check quickly.

- A Bloom filter trades a small, tunable false-positive rate for massive memory savings, and is checked before a slower, exact lookup in durable storage.
- **How do we decide how often to re-crawl a page?**
- A news homepage changes every few minutes; a static "About Us" page might not change for years.
- Track how often each page has historically changed, and use that history to set its priority in the frontier, so fast-changing pages get revisited more often.

**Hardest edge case**

- A poorly built site can generate effectively infinite URLs, for example a calendar page linking to "next month" forever; without a cap on crawl depth or URL patterns per host, the crawler can get stuck endlessly fetching one site's junk pages.

**Interview Answer:**

The two hard constraints here are politeness and dedup, so I'd design around those first. I'd shard the URL frontier by host so each host gets its own rate limiter, which keeps the crawler from hammering any one site regardless of overall throughput. For dedup, I'd use a Bloom filter to cheaply reject already-seen URLs before checking a slower, exact, persistent store. Fetcher workers pull from the frontier, respect robots.txt, parse content for new links, and feed them back in, with re-crawl frequency prioritized by how often a page has historically changed.

---

## Q5. [Senior] Design a Distributed Cache like Redis or Memcached, the cache system itself, not just how to use one. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Support get (read), set (write), and delete operations on key-value pairs.
- Support TTL (time to live), meaning a key automatically expires after a set amount of time.
- Support storing more total data than fits in one machine's memory.
- Non-functional requirements:
- Sub-millisecond latency, meaning responses well under one thousandth of a second.
- High throughput, meaning it can handle a very large number of operations per second.
- Horizontal scalability, meaning you add more machines to handle more load, rather than only making one machine bigger.
- Tunable consistency versus availability, meaning the system can favor either always giving an answer or always giving the latest correct answer, depending on the use case.

**Scale/capacity estimate**

- Assume 1 million requests per second at peak, and one node can handle about 50,000 requests per second.
- 1,000,000 / 50,000 = 20 nodes at bare minimum, so realistically 30 to 40 nodes with headroom and redundancy.
- Assume 500 million keys stored, each averaging 1 KB including overhead.
- 500,000,000 x 1 KB = about 500 GB of total data.
- If each machine offers 32 GB for cache data, 500 GB / 32 GB = about 16 machines needed for storage alone, consistent with a 30-to-40-node cluster overall.

**Architecture**

Clients hash each key to figure out which node owns it, then talk to that node directly. A coordination layer tracks which nodes are alive so the cluster adapts automatically when a node joins or fails.

```
Client -> Client-side hash ring / router
              |-> Cache Node A (in-memory store + eviction)
              |-> Cache Node B
              |-> Cache Node C (replica of A)
Coordinator/Gossip -> membership + failure detection
```

- Cache Node: a machine holding an in-memory, meaning RAM-based and extremely fast, hash table of key-value pairs, with an eviction policy such as LRU (Least Recently Used, meaning whatever hasn't been accessed in the longest time gets removed first when memory fills up).
- Hash ring / router: logic, often in the client library or a thin proxy, that decides which node is responsible for any given key.
- Coordinator/Gossip layer: nodes periodically tell each other what they know about cluster membership, keeping the ring updated automatically as nodes join or fail.

**Key design decisions**

- **How do we spread keys across nodes so resizing the cluster doesn't ruin everything?**
- Plain modulo hashing, deciding a key's node using key_hash % number_of_nodes, means adding or removing even one node changes the answer for almost every key, forcing a massive reshuffle.
- **Consistent hashing, where both nodes and keys sit on a conceptual ring and a key belongs to the next node clockwise from it, keeps key movement proportional to the size of the cluster change instead of the whole cluster.** Adding "virtual nodes," where each physical machine appears at many points on the ring, further smooths out uneven load.
- **How much replication and consistency should the cache use?**
- A cache generally favors speed and availability over perfect correctness, since it is usually a copy of data that lives durably somewhere else anyway.
- Use asynchronous replication, copying data to one or two backup nodes without waiting for that copy before responding, and accept a few seconds of possible staleness after a failure, rather than paying the latency cost of a quorum write, meaning waiting for a majority of replicas to confirm before responding.
- **What about a single super-popular key overwhelming one node, a "hot key"?**
- Even perfect hashing sends one key to exactly one node, so a viral key, like a trending post's ID, can overload that one node no matter how balanced the rest of the cluster is.
- Fix this with client-side request coalescing, combining many simultaneous requests for the same key into a single backend fetch, or by explicitly replicating known hot keys across multiple nodes.

**Hardest edge case**

- A node fails while holding the only up-to-date copy of keys that had not finished replicating yet; the system must decide between serving slightly stale data from a replica or returning an error, and for a cache the right default is almost always to serve the stale data.

**Interview Answer:**

I'd build this around consistent hashing with virtual nodes so the cluster can grow or shrink without remapping the entire keyspace, each node running an in-memory store with LRU eviction and TTL expiry. For availability, I'd replicate each key asynchronously to one or two other nodes and accept eventual consistency after a failover, since a cache should favor speed and availability over strict correctness. I'd also call out hot-key handling: a single very popular key can overwhelm one node regardless of hashing, so I'd add client-side request coalescing or targeted replication for detected hot keys.

## Q6. [Mid] Design a Notification System that fans out push, email, and SMS notifications to millions of users. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Let internal services trigger a notification to a user.
- Deliver it via push (a mobile app alert), email, SMS (text message), or in-app message.
- Respect user preferences and use templates, meaning pre-formatted message layouts, so messages look consistent.
- Non-functional requirements:
- Handle high throughput bursts, such as a breaking-news alert going out to tens of millions of devices at once.
- Guarantee at-least-once delivery, meaning a notification is never silently lost, even if it is occasionally delivered more than once.
- Avoid duplicate or spammy notifications to the same user.
- Fail over to a backup provider if a third-party provider is down.

**Scale/capacity estimate**

- Assume a breaking-news event triggers a notification to 50 million users within a 5-minute window.
- 50,000,000 / 300 seconds = about 167,000 notifications per second during the spike.
- Assume normal daily volume is 2 billion notifications per day across all channels.
- 2,000,000,000 / 86,400 = about 23,000 notifications per second on average, meaning the system must handle bursts around 7 times its average load.

**Architecture**

Instead of every internal service calling push, email, and SMS providers directly, they all call one central notification API, which places the request on a queue. Separate worker pools drain each channel's queue at a controlled pace and call the real third-party providers.

```
Producer Services -> Notification API -> Queue (per channel)
                                      -> Push Worker -> APNs/FCM
                                      -> Email Worker -> SES/SendGrid
                                      -> SMS Worker -> Twilio-like provider
                        Preference/Template Store, Idempotency Store
```

- Producer Services: internal systems that want to notify a user, such as "your order shipped."
- Notification API: the single entry point that writes requests to a durable queue rather than sending immediately.
- Queue (per channel): a separate line of pending messages for each channel, so a slowdown in one provider does not back up the others.
- Workers: pull messages off a queue, fill in the template, apply user preferences, and call the provider.
- Preference/Template Store: holds each user's opt-in and opt-out settings and message templates.
- Idempotency Store: tracks which notifications already sent, so a retried message is not delivered twice.

**Key design decisions**

- **How do we absorb huge traffic bursts without overwhelming providers?**
- **Putting a durable, per-channel queue between the producer services and the delivery workers is what absorbs a burst like a breaking-news alert, since the queue simply grows temporarily while workers drain it at a sustainable pace.** Without it, every request would hit the push or email provider directly and overwhelm it.
- **How do we guarantee a user isn't spammed with duplicate notifications?**
- Queues and providers typically only guarantee at-least-once delivery, meaning retries after a timeout can occasionally send the same message twice.
- Attach a unique idempotency key, a unique identifier for that specific send attempt, to every notification, and check the idempotency store before sending, so a retried send is recognized and skipped.
- **How do we handle a failing third-party provider?**
- If the primary provider starts failing or timing out, workers should automatically retry through a backup provider rather than dropping the message or endlessly retrying a dead provider.
- **How do we respect user preferences and avoid annoying people?**
- Check preferences, such as quiet hours, opted-out channels, and per-user rate limits, at the worker layer right before sending, not just when the notification was first triggered, since preferences can change in between.

**Hardest edge case**

- A user's device token, the unique ID used to send them a push notification, is stale because the app was uninstalled; the push provider reports a failure, and the system must detect this and stop retrying that dead token instead of endlessly resending into a void.

**Interview Answer:**

I'd put a durable, per-channel queue between the services that trigger notifications and the workers that actually deliver them, so a burst like a breaking-news alert to millions of devices gets absorbed by the queue instead of overwhelming push or email providers directly. Each worker pool applies templates and user preferences, then calls the provider, with retries and failover to a backup provider on failure. Since retries mean at-least-once delivery, I'd attach an idempotency key, a unique ID for that specific send, to every notification so a retried message never reaches the same user twice.

---

## Q7. [Senior] Design an E-commerce Checkout and Inventory System that handles orders without overselling stock. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Browse a catalog and add items to a cart.
- Check out: reserve inventory, charge payment, confirm the order.
- Never sell an item that is not actually in stock, meaning no overselling.
- Non-functional requirements:
- Handle flash-sale traffic spikes, meaning a huge, sudden burst of demand for a small number of popular items.
- Keep checkout latency low so the process feels responsive.

- Be resilient to payment provider failures, so a slow or failing payment processor does not leave orders in a broken state.

**Scale/capacity estimate**

- Assume a flash sale where 500,000 users try to buy 10,000 units of one item within 60 seconds.
- 500,000 / 60 = about 8,300 checkout attempts per second hitting that one item's inventory record, even though only 10,000 will succeed.
- Assume normal daily volume across the store is 5 million orders per day.
- 5,000,000 / 86,400 = about 58 orders per second on average, meaning the flash-sale spike is roughly 140 times normal peak load, concentrated on a single database row.

**Architecture**

Checkout is treated as a multi-step workflow, called a saga, across separate services, since inventory, payment, and orders typically live in different systems. Each step either succeeds and moves forward, or fails and triggers a compensating (undo) action for steps that already happened.

```
Client -> Checkout Service -> Inventory Service (reserve stock)
                           -> Payment Service (charge)
                           -> Order Service (persist order)
Saga Orchestrator coordinates: reserve -> charge -> confirm/rollback
```

- Checkout Service: the entry point that coordinates the checkout flow.
- Inventory Service: owns the stock count for every product and handles reserving and deducting it.
- Payment Service: charges the customer's payment method through a third-party processor.
- Order Service: persists the final confirmed order record.
- Saga Orchestrator: a coordinator, a saga being a sequence of steps with a defined undo action for each one, that drives the checkout sequence and calls the right compensating action if a step fails partway through.

**Key design decisions**

- **How do we guarantee we never oversell an item under heavy concurrency?**
- The classic bug: two customers both read "1 item left" and both get told "yes, you can buy it," because the check and the update were two separate steps with a gap in between.
- **Inventory deduction has to be one atomic, conditional database operation, such as "decrease stock by 1 only if current stock is greater than 0," rather than a separate read followed by a write from application code.** Otherwise two concurrent requests can both slip through the gap and succeed.
- **How do we avoid stock getting locked up by users who never finish paying?**
- Add a short-lived reservation: when checkout starts, hold the item's stock for a few minutes while payment completes.
- If payment does not complete in that window, automatically release the hold back into available stock, so abandoned carts do not starve other buyers.
- **How do we handle the extreme concentration of traffic on one item during a flash sale?**
- Sending every request straight to the same database row causes massive contention, meaning many requests fighting to update the exact same row at once.
- Funnel the hot decrement path through a queue, or assign a single partition, a dedicated, isolated slice of the system, per popular SKU (Stock Keeping Unit, a unique code identifying a specific product), so contention is contained rather than hitting the raw database directly.

**Hardest edge case**

- A payment charge succeeds, but the next step, confirming the order, fails due to a crash; the saga's compensating action must correctly refund the payment rather than leaving the customer charged with no order, which is why every step needs a defined undo action, not just a happy path.

**Interview Answer:**

The failure mode I'd design around first is overselling, so inventory decrement has to be an atomic conditional update in the database, never a read-check-then-write from application code. On top of that, I'd add a short-lived reservation when checkout starts, holding the stock while payment processes and auto-releasing it if the user abandons checkout. Checkout itself I'd model as a saga, a sequence of steps with defined undo actions: reserve inventory, charge payment, confirm order, with compensating steps like releasing stock or refunding if any step fails, since these operations span separate services.

---

## Q8. [Mid] Design a Search Autocomplete or Typeahead feature that suggests queries as a user types. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- After each keystroke, return the top few, say 10, suggested completions.
- Rank suggestions by popularity or relevance.
- Keep suggestions reasonably fresh as trends change.
- Non-functional requirements:
- Very low latency, meaning suggestions must appear within tens of milliseconds, or the feature feels laggy.
- Handle huge query volume, since a popular search box gets a request on nearly every keystroke from every active user.
- Freshness: a newly trending term should show up within minutes to hours, not days.

**Scale/capacity estimate**

- Assume 1 billion searches per day, and a user types an average of 15 characters before picking a suggestion.
- 1,000,000,000 x 15 = 15 billion keystroke-triggered lookups per day before any debouncing.
- 15,000,000,000 / 86,400 = about 173,000 lookups per second on average.
- Debouncing, meaning waiting a short pause before firing a request instead of firing on every keystroke, cuts this by roughly 3x, to around 58,000 requests per second, still far too high to rank live.

**Architecture**

Suggestions are precomputed offline for every possible prefix, the beginning portion of a term typed so far, and stored in a fast in-memory lookup structure. A background pipeline continuously reprocesses real search logs to keep those precomputed suggestions fresh.

```
Client (debounced keystrokes) -> Autocomplete Service -> In-memory Trie/Index (per shard)
Query Logs -> Aggregation Job (batch/stream) -> rebuild top-k per prefix -> push to Trie
```

- Client: waits briefly after each keystroke, called debouncing, before sending a request.
- Autocomplete Service: receives the current prefix and looks up the precomputed top results.

- In-memory Trie, a tree-like structure organized by shared prefixes that lets you quickly find all words starting with a given string, sharded by prefix range so no one machine holds everything.
- Aggregation Job: processes real search logs, counts how often each query happens, and recomputes the top-k, meaning top "some number" like top 10, completions for every prefix.

**Key design decisions**

- **How do we make each keystroke feel instant?**
- **Ranking every possible completion live, on every keystroke, at the request rate a busy search box generates, is far too slow, so the core trick is to precompute and cache the top-k suggestions for every prefix offline.** The online path then becomes a simple, fast lookup instead of a live ranking computation.
- **How do we keep suggestions fresh when trends change quickly, like breaking news?**
- A purely batch pipeline that reprocesses logs once a day would miss a trending term for up to 24 hours, too slow for something genuinely viral.
- Run a lightweight, fast streaming aggregator on top of the slower batch job, specifically to catch sudden spikes within minutes, while the batch job continues handling steady-state, long-tail ranking.
- **How do we scale the lookup structure itself?**
- Shard the trie or prefix map by prefix range, for example "a" through "m" on one set of machines and "n" through "z" on another, so each shard fits comfortably in memory.

**Hardest edge case**

- A brand-new term suddenly goes viral, like a celebrity's name trending within minutes, and the precomputed structure has no entry for it yet; without the fast streaming layer, users would see stale or empty suggestions at exactly the moment the term matters most.

**Interview Answer:**
Because latency budgets here are in the tens of milliseconds, I wouldn't rank anything live. I'd precompute the top-k suggestions for every prefix offline from query logs, store that as a trie or prefix map sharded by prefix range, and have the online path just do a fast lookup and return. Freshness is the trade-off that creates, so I'd add a lightweight streaming aggregator on top of the batch job specifically to catch trending queries within minutes, while the batch job handles steady-state ranking for everything else.

---

## Q9. [Senior] Design a Distributed File Storage system like Dropbox or Google Drive, supporting sync across multiple devices. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Upload and download files and folders.
- Sync changes automatically across a user's multiple devices.
- Share files with other users.
- Handle edits made while offline and reconcile them once back online.
- Non-functional requirements:
- Durability, meaning files essentially never get lost, even if individual disks or machines fail.
- Bandwidth efficiency, especially for large files with only small changes.
- A consistent view of the file and folder structure (metadata) across devices.

- Sensible conflict handling when the same file is edited in two places at once.

**Scale/capacity estimate**

- Assume 500 million users, each storing an average of 5 GB of files.
- 500,000,000 x 5 GB = 2.5 exabytes (EB), meaning 2,500 petabytes, of total stored data.
- Assume each user syncs 20 small file changes per day, each requiring only about 100 KB of actually-changed data thanks to chunking, explained below.
- 500,000,000 x 20 = 10 billion sync events per day.
- 10,000,000,000 / 86,400 = about 116,000 sync events per second that the metadata service must process.

**Architecture**

Files are broken into small chunks on the user's device before upload, and only chunks that are new or changed get uploaded. A separate metadata service tracks the file and folder structure, and a lightweight notification system tells other devices when something changed so they can pull just the missing pieces.

```
Client (watches local FS) -> Sync Client -> Chunker -> Block Store (S3-like)
                                          -> Metadata Service (file tree, versions)
Other Devices <- Notification Service <- Metadata Service (change events)
```

- Sync Client: software on the user's device that watches the local folder for changes.
- Chunker: splits each file into fixed-size chunks (blocks) and computes a content hash, a fingerprint derived from the chunk's actual bytes, for each one.
- Block Store: cheap, durable object storage holding the actual chunk data, referenced by its content hash.
- Metadata Service: tracks the folder and file tree, version history, and which chunks make up each version, separately from the raw chunk data.
- Notification Service: pushes a lightweight "something changed" event to a user's other devices, which then ask what changed and pull only the missing chunks.

**Key design decisions**

- **How do we avoid re-uploading a whole large file every time it changes slightly?**
- **Splitting files into content-addressed chunks, meaning each chunk is identified by a hash of its own content, and only uploading chunks that are actually new or changed, is what makes sync bandwidth-efficient.** An edit to one paragraph of a large document then only re-uploads the few chunks that actually changed, not the whole file, and identical chunks shared across files are deduplicated automatically.
- **What happens when two devices edit the same file at the same time while offline?**
- Silently picking one version as the winner and discarding the other risks real, permanent data loss.
- Instead, version every change, and when two versions have genuinely diverged, keep both: apply a default like last-writer-wins for the main file, but save the other version as a separate "conflicted copy" file for the user to manually reconcile.
- **How do we keep metadata consistent if a client crashes mid-sync?**
- The metadata service must treat each file update as its own atomic step, fully happening or fully not, so a crash mid-upload leaves the file at its last fully-completed version instead of a corrupted, half-updated state.

**Hardest edge case**

- A user edits the same document on their laptop and phone while both are offline, then reconnects both; the system must detect that the two versions have diverged in content, not just in timestamp, and preserve both rather than silently discarding one person's changes.

**Interview Answer:**

I'd split files into content-hashed chunks on the client, so only the blocks that actually changed get re-uploaded and identical blocks are deduplicated across files. A separate metadata service tracks the file tree and version history independent of block storage, and a lightweight notification service pushes change events to other devices, which then pull just the missing blocks. For conflicts, when two devices edit the same file offline and reconnect with diverging content, I wouldn't silently overwrite; I'd version both and surface a conflicted copy for the user to resolve.

---

## Q10. [Senior] Design a Ticket Booking System like BookMyShow or Ticketmaster that must handle many users trying to book the same seats at once. Walk me through your approach.

**Answer:**

**Requirements**

- Functional requirements:
- Browse events and showtimes, and view a seat map.
- Temporarily hold seats, then complete payment to book them.
- Receive a confirmation once booking succeeds.
- Non-functional requirements:
- Never double-book the same seat to two different people.
- Handle massive concurrent demand for popular events, sometimes called an "on-sale."
- Give users a fixed, reasonable amount of time to complete payment before releasing their held seats.

**Scale/capacity estimate**

- Assume a popular concert on-sale where 1 million users try to book seats in a 20,000-seat venue within the first 2 minutes.
- 1,000,000 / 120 = about 8,300 booking attempts per second at peak, competing over only 20,000 seats.
- Assume normal platform-wide traffic is 10 million bookings per day.
- 10,000,000 / 86,400 = about 116 bookings per second on average, meaning a hot on-sale is roughly 70 times normal peak load, all concentrated on one event's seat map.

**Architecture**

Each event has its own seat map stored in a fast key-value store. Selecting seats places a short-lived hold on them, which only becomes a permanent booking after payment succeeds, and a virtual waiting room throttles how many users are even allowed to attempt booking at once during extreme demand.

```
Client -> Booking Service -> Seat Lock Service (per-show seat map, short TTL hold)
                           -> Payment Service
                           -> Confirmation/Ticket Service
Queue/Waiting Room -> throttles entry during high-demand on-sales
```

- Seat Lock Service: holds each showtime's seat map in a fast in-memory store, like Redis, keyed by seat ID, so locking one seat is a cheap, localized operation.

- Booking Service: coordinates selecting seats, holding them, and moving to payment.
- Payment Service: charges the user once seats are confirmed.
- Confirmation/Ticket Service: issues the final ticket once payment succeeds.
- Queue/Waiting Room: a front-door throttle limiting how many users can enter the booking flow at once during extreme demand.

**Key design decisions**

- **How do we guarantee two people can never book the same seat?**
- The classic bug: two users both check that a seat is available in the small gap of time before either has actually claimed it, and both are told "yes, available."
- **Seat holds must be implemented as a single atomic compare-and-set operation, meaning "claim this seat only if its current state is still unclaimed," executed as one indivisible step such as a single Lua script or database transaction, never a separate read followed by a write.** Otherwise two concurrent requests can both slip through and succeed.
- **How long should a seat hold last, and what happens if payment isn't finished?**
- Give a short, fixed TTL, typically a few minutes, during which the seat is reserved for that user alone.
- If payment does not complete before the TTL expires, automatically release the seat back into the available pool, so a distracted user does not permanently block others from buying it.
- **How do we handle demand that vastly exceeds available seats?**
- Letting a million users hit the booking service and seat-lock layer simultaneously would overwhelm the system and produce a chaotic experience.
- Add a virtual waiting room in front of the booking service: users are queued and let into the booking flow at a controlled rate, protecting backend systems and keeping the process fair, first-come first-served, instead of a free-for-all race.

**Hardest edge case**

- A user completes payment right as their seat hold's TTL is about to expire, creating a race between "release the seat because the timer ran out" and "confirm the booking because payment just succeeded"; the system must check the seat's current hold owner as part of the atomic confirm step, so a just-in-time payment still wins cleanly.

**Interview Answer:**

The core risk is two people booking the same seat, so I'd make seat holds an atomic compare-and-set operation against the seat's state in a fast store like Redis, never a separate read-then-write, and give each hold a short TTL that auto-releases if payment doesn't complete in time. For genuinely hot on-sales where demand vastly exceeds supply, I'd add a virtual waiting room in front of the booking service to throttle how many users can attempt a booking at once, which protects the seat-lock layer and keeps the process fair instead of turning into an uncontrolled race.

---

**If you remember nothing else from this chapter:** *these ten come up constantly for a reason, each one forces you to touch requirements, trade-offs, and a hard edge case in a single sitting.*

# Chapter 23: Top 10 Most Asked LLD Questions

These ten low-level design prompts are asked often enough that you should have a clean, structured answer ready for each one. If you find yourself reconstructing your approach from scratch on one of these, that's a signal worth taking seriously rather than brushing off.

---

## Q1. [Senior] Design an Elevator System. What classes would you create, and how would you handle scheduling when multiple requests come in from different floors?

**Answer:**

- The core challenge is not drawing boxes and arrows, it is deciding, in real time, which elevator car should answer a new button press so that no one waits too long and cars do not zig-zag pointlessly between floors.

**Core classes**

- `Elevator`: represents one physical car. It tracks its id, current floor, direction (up, down, or idle), door state, and the list of floors it still needs to stop at.
- `ElevatorController` (also called a `Dispatcher`): the "brain" that owns every elevator in the building and decides which car should handle each new request.
- `Request`: a passenger's ask. There are two kinds: an `ExternalRequest` (someone presses "up" or "down" on a hallway button on some floor) and an `InternalRequest` (someone already inside the car presses a floor number).
- `Door`: a small helper class that models opening, closing, and reacting to an obstruction (like a hand or bag blocking the door).
- `ElevatorState`: represents what the elevator is doing right now, such as Idle, MovingUp, MovingDown, DoorOpen, or OutOfService. This uses something called the State pattern, which just means each state is its own small class that knows what actions are allowed from that state, instead of one giant class full of if/else checks on a status flag.

```
interface ElevatorState {
    void handleRequest(Elevator e, Request r);
}
class Elevator {
    int id, currentFloor;
    Direction direction;
    ElevatorState state;
    TreeSet<Integer> upStops, downStops;
}
```

**How it works**

1. A passenger on some floor presses a hallway button. This creates an `ExternalRequest` and sends it to the `ElevatorController`.
2. The controller looks at every elevator and scores how good a fit each one is. A car that is already heading in the right direction and would pass that floor anyway is a great fit. An idle car nearby is the next best option.
3. The controller assigns the request to the best-scoring elevator, which adds the floor to its sorted list of stops.

4. Once someone steps inside and presses a destination floor, that becomes an `InternalRequest`, which is just added to the same car's stop list, no separate assignment step needed.
5. Each elevator services its stops using a sweep algorithm (sometimes called SCAN or LOOK): it keeps moving in one direction, picking up every stop along the way, and only reverses direction once there are no more stops ahead of it in the current direction. Storing stops in two sorted sets, one for "stops while going up" and one for "stops while going down," makes this easy to implement.
6. When the car reaches a stop, it opens the door, waits, closes the door, and removes that floor from its stop list, then continues the sweep.

**The trickiest part**

- **The hardest part isn't the class diagram, it's making dispatch decisions safely when requests arrive at the same time from many floors while elevators are already mid-trip.**
- If two hallway requests arrive within milliseconds of each other, both could read the "current" state of the same elevator before either assignment is written back, and both end up assigned to the same car when a different car would have been better, or a request gets silently dropped.
- The fix is to make assignment a single, guarded operation: either put a lock around each elevator's stop list so only one assignment can modify it at a time, or route every assignment decision through one single-threaded dispatcher (sometimes implemented as what is called an actor, a worker that processes one message at a time from a queue) so there is never a race in the first place.
- You also need to re-check assignments if an elevator becomes unavailable between the moment it was assigned a request and the moment it actually arrives, for example if it goes into maintenance mode or fills to capacity.

**Interview Answer:**

I would split this into an Elevator class that tracks its own state and stops, and a Dispatcher that assigns hallway requests to whichever elevator has the least detour, using a sweep algorithm so each car finishes floors in one direction before reversing. Internal button presses just get added to that car's own stop list. I would model states like Idle, Moving, and DoorOpen using the State pattern, so each state only allows the actions that make sense from it. The part I would call out unprompted is protecting the shared stop lists from race conditions when many requests land at once, either with per-elevator locks or a single-threaded dispatcher.

---

## Q2. [Mid] Design a Vending Machine. Walk me through the classes you'd create and how it handles state (idle, selecting, dispensing, out of stock).

**Answer:**

- The core challenge is making sure the machine only allows the right action at the right moment, you cannot dispense a snack before payment, and you cannot accept a coin while it is mid-dispense, and doing that cleanly is a textbook example of the State pattern, which means representing "what the machine is currently allowed to do" as its own object instead of a pile of boolean flags.

**Core classes**

- `VendingMachine`: the main object. It holds a reference to the current state, the inventory, and how much money has been inserted so far.
- `VendingMachineState`: an interface with methods like `insertCoin()`, `selectProduct(code)`, `dispense()`, and `refund()`. The concrete states are `IdleState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`, and `SoldOutState`.
- `Inventory`: a lookup from a slot code (like "A3") to a `Slot`.

- `Slot`: holds one `Product` and how many units are left in that slot.
- `Product`: just a name and a price.
- `CoinBox` (or `ChangeDispenser`): tracks how many coins or notes of each denomination the machine currently holds, used to calculate change.

```
interface VendingMachineState {
    void insertCoin(VendingMachine m, Coin c);
    void selectProduct(VendingMachine m, String code);
    void dispense(VendingMachine m);
}
```

**How it works**

1. The machine starts in `IdleState`, waiting for a coin.
2. When a coin is inserted, the machine moves to `HasMoneyState` and adds the coin's value to the running balance.
3. The customer picks a product code. The machine looks up the `Slot` for that code.
4. If the slot is empty, the machine moves to `OutOfStockState`, refunds any inserted money, and returns to `IdleState`.
5. If the balance is less than the price, the machine stays in `HasMoneyState` and asks for more coins.
6. If the balance covers the price, the machine moves to `DispensingState`, decrements the slot's quantity, calculates any change owed, and physically releases the product.
7. After dispensing finishes, the machine resets balance to zero and returns to `IdleState`, ready for the next customer.
8. **Each state class only implements the transitions that are valid from it, so pressing "dispense" while in IdleState is simply not a wired-up action instead of a bug hiding inside a tangle of if-statements.**

**The trickiest part**

- Making exact change: the `ChangeDispenser` needs to work out the fewest coins or notes that add up to the change owed. This is basically the classic "coin change" algorithm from data structures class, except now it is constrained by real hardware, the machine can only give back coins it physically has. If it cannot make exact change, the safest design is to refuse the sale (or tell the customer only exact change is accepted) rather than shortchange them.
- Concurrency on the last unit: imagine this machine is actually a backend service controlling many physical dispensers, and two people tap "buy" on the last item in a slot at nearly the same instant. Both requests might read "quantity: 1" before either one writes back "quantity: 0", and both purchases could succeed even though only one item exists. The fix is to make the inventory decrement an atomic compare-and-decrement (check the quantity and reduce it in one indivisible step) guarded by a lock per slot, rather than a separate "read the quantity, then write the new quantity" sequence.

**Interview Answer:**

I would model the machine using the State pattern, with states like Idle, HasMoney, Dispensing, and OutOfStock, each implementing insertCoin, selectProduct, and dispense, so invalid actions simply are not available from that state. Inventory is a map from slot code to a Product plus a quantity. The two edge cases I always bring up are exact change, which is a coin-change style algorithm over the coin box that should refuse the sale if it cannot make change, and protecting the very last unit in a slot from being sold to two customers at once under concurrent purchase attempts.

## Q3. [Mid] Design a Library Management System. How do you model books, copies, members, and the checkout/return/reservation flow?

**Answer:**

- The key modeling insight is separating the abstract idea of a book, like "1984 by George Orwell," from a physical copy of it sitting on a shelf with its own barcode, because the library might own five physical copies of the same title and each one needs to be tracked separately.

**Core classes**

- `Book`: a catalog-level record with title, author, ISBN (a book's unique identifying number), and subject.
- `BookItem`: one physical copy of a `Book`. It adds a barcode, a shelf/rack location, and a status such as `AVAILABLE`, `LOANED`, `RESERVED`, or `LOST`.
- `Member`: a library user, with an id, contact details, and a capped list of current loans (most libraries limit how many books you can borrow at once).
- `Loan` (sometimes called `BookLending`): links one `BookItem` to one `Member`, along with the checkout date, due date, and logic to calculate a late fine.
- `Catalog`: supports searching for books by title, author, or subject.
- `Reservation`: represents a member waiting in line for a `Book` when every copy is currently checked out.
- `Library`: the top-level object that wires the catalog, members, and loans together.

```
class BookItem {
    String barcode;
    Book book;
    BookStatus status;
    LocalDate dueDate;
}
class Loan {
    BookItem item;
    Member member;
    LocalDate checkoutDate, dueDate;
    double calculateFine();
}
```

**How it works**

1. A member searches the `Catalog` for a title and finds a `Book` with, say, three physical copies.
2. To check out, the system looks for any `BookItem` for that `Book` with status `AVAILABLE`.
3. If found, its status flips to `LOANED`, and a new `Loan` record is created with today's date and a computed due date.
4. When the member returns the book, the system flips the `BookItem` status back and calculates any overdue fine based on how late the return is.
5. Before marking the item generally `AVAILABLE` again, the system checks the `Reservation` queue for that `Book`. If someone is waiting, the item's status becomes `RESERVED` for that specific member instead, and they get notified.
6. Only if nobody is waiting does the item become fully `AVAILABLE` for any walk-in member to borrow.

**The trickiest part**

- **The subtle part interviewers probe for is making the reservation queue interact correctly with concurrent returns and checkouts, because when the last copy of a popular book comes back, you**

**need to check "is someone waiting" before the item is shown as available to everyone, otherwise a walk-in customer could grab it ahead of someone who reserved it days earlier.**

- This race happens because "update the item's status" and "check the reservation queue" are really two separate steps, and if another checkout request sneaks in between them, it can see the item as available before the reservation logic has had a chance to claim it.
- The fix is to make the status update and the reservation check happen as one atomic unit, either using a database transaction or a lock scoped to that specific `BookItem`, so no other operation can interleave in the middle.

**Interview Answer:**

I separate Book, the catalog entry, from BookItem, one specific physical copy with its own status. Checkout and return operate on a BookItem and create or close a Loan record with due dates and fines. A Reservation queue per Book handles holds when every copy is checked out. The trickiest bit is making sure that when a copy is returned, checking the reservation queue and updating the item's status happen as a single atomic step, so a waiting reservation always wins over a walk-in checkout on that same copy.

---

## Q4. [Mid] Design an ATM Machine. What are the main classes and states, and how do you keep cash dispensing safe?

**Answer:**

- The core challenge is that an ATM withdrawal is really two separate real-world actions, taking money out of a bank account and physically releasing cash from a machine, and those two actions can fail independently of each other, so the design has to make sure they never get out of sync.

**Core classes**

- `ATM`: the main object, holding the current `ATMState` plus references to hardware parts like `CardReader`, `CashDispenser`, `Keypad`, and `Screen`.
- `ATMState`: represents where the customer is in the flow: `IdleState`, `HasCardState`, `AuthenticatedState`, `TransactionSelectionState`, `DispensingCashState`. Each state implements `insertCard()`, `enterPin()`, `selectTransaction()`, and `ejectCard()`, and simply does not allow actions that make no sense in that state.
- `Account`: the customer's balance, account number, and a reference to the bank's backend service.
- `BankService`: an interface representing the actual bank system, with methods like `authenticate()`, `debit()`, and `getBalance()`. The ATM itself never touches money directly, it just calls this service.
- `Transaction`: an interface with implementations `WithdrawalTransaction`, `DepositTransaction`, and `BalanceInquiryTransaction`, each knowing its own validation rules. This is an example of the Strategy pattern, which means you swap in different "how to do this specific task" objects behind one shared interface, instead of writing one big method full of type checks.
- `CashDispenser`: tracks how many notes of each denomination (like $20s, $50s) are loaded, and works out which notes to release for a given amount.

```
interface Transaction {
    boolean validate(Account acc);
    void execute(ATM atm, Account acc);
}
class CashDispenser {
    Map<Integer, Integer> denominationCount;
    List<Integer> calculateNotes(int amount);
}
```

**How it works**

1. The customer inserts a card, moving the ATM from `IdleState` to `HasCardState`.
2. They enter a PIN, which the `BankService` authenticates; on success the ATM moves to `AuthenticatedState`.
3. The customer picks a transaction type, and the ATM builds the matching `Transaction` object (withdrawal, deposit, or balance check) and moves to `TransactionSelectionState`.
4. For a withdrawal, the `Transaction.validate()` method checks things like sufficient balance and daily withdrawal limits.
5. If valid, the ATM asks the `BankService` to place a hold on the funds (reserve them without finalizing the debit yet), then moves to `DispensingCashState`.
6. The `CashDispenser` calculates the notes needed and physically releases them.
7. Only after the cash physically comes out does the ATM tell the `BankService` to confirm the debit. The card is ejected and the ATM returns to `IdleState`.

**The trickiest part**

- **The safe design treats a withdrawal like a two-phase commit: reserve the funds first, physically try to dispense the cash second, and only finalize the debit once dispensing actually succeeds.** A two-phase commit is a general pattern for making a two-step operation safe: you first "prepare" (reserve) both sides, and only "commit" for real once you know both sides can succeed.
- Without this, imagine the network drops right after the account is debited but before the cash tray opens, or the dispenser physically jams. The customer loses money with nothing to show for it, and that is the single worst failure mode an ATM can have.
- With the two-phase approach, if dispensing fails for any reason, the hold on the funds is simply released and the customer was never actually charged.
- Separately, `calculateNotes` needs a greedy or dynamic-programming style algorithm (a technique for breaking a number down into the fewest denominations), and it must fail cleanly, and release the hold, if the exact amount cannot be made with the notes currently loaded.

**Interview Answer:**

I model the ATM as a state machine with states like Idle, HasCard, Authenticated, and Dispensing, and I delegate the actual transaction logic to a Transaction interface with Withdrawal, Deposit, and BalanceInquiry implementations. The interesting design decision is treating a withdrawal as a two-phase operation: reserve the funds with the bank first, then attempt to physically dispense the cash, and only finalize the debit once the cash actually comes out, rolling back the hold otherwise. That avoids the worst case of debiting an account when no money was dispensed.

---

## Q5. [Mid] Design an LRU Cache as a class. How do you get O(1) get and put, and what would you change to make it thread-safe?

**Answer:**

- The core challenge is not a class hierarchy at all, it is picking and combining the right two data structures so that both looking up a value and marking it as "recently used" happen in constant time, LRU stands for Least Recently Used, meaning that when the cache is full, it throws away whichever item has gone the longest without being touched.

**Core classes**

- `LRUCache<K, V>`: the public-facing class, exposing `get(key)` and `put(key, value)`, with a fixed maximum size called `capacity`.

- `Node<K, V>`: an internal doubly linked list node holding a key, a value, and pointers to the previous and next nodes. A doubly linked list is just a chain of nodes where each node knows both its neighbors, which lets you remove or insert a node from the middle in constant time.
- Internally, a `HashMap<K, Node<K, V>>` gives O(1) lookup by key, while the doubly linked list, with `head` (most recently used end) and `tail` (least recently used end) sentinel nodes, gives O(1) reordering.

```
class LRUCache<K, V> {
    int capacity;
    Map<K, Node<K, V>> map;
    Node<K, V> head, tail; // sentinels

    V get(K key) { /* move node to front, return value */ }
    void put(K key, V value) {
        /* if exists, update + move to front;
           else insert at front; if over capacity, evict tail.prev */
    }
}
```

**How it works**

1. On `get(key)`, look up the node directly in the hashmap. If it exists, unlink it from its current spot in the list and re-insert it right after `head`, then return its value.
2. On `put(key, value)`, if the key already exists, update its value and move it to the front the same way.
3. If the key is new, create a node, insert it right after `head`, and add it to the hashmap.
4. If inserting a new node pushes the cache over `capacity`, remove the node just before `tail` (the actual least recently used item) from both the list and the hashmap.
5. A hashmap alone would give fast lookup but no sense of order. A linked list alone would give fast reordering but force a slow linear scan to find a key. Combining them, the hashmap stores a direct pointer straight to the node, so both "find it" and "move it to the front" are O(1).

**The trickiest part**

- **The design question interviewers actually care about is what happens when this cache needs to be thread-safe, because a naive `synchronized` block around every method serializes all access and kills performance under heavy read traffic.**
- The catch is subtle: in a normal cache you might think reads can be handled by a shared "read lock" while only writes need an exclusive lock. But in an LRU cache, even a `get()` mutates the linked list, because it has to move the accessed node to the front. That means there is no such thing as a pure read here, which rules out a simple read-write lock.
- A workable answer is to protect the whole map-plus-list structure with a single lock, such as `ReentrantLock`. For higher throughput at larger scale, you would shard the cache into several independently-locked segments by hashing the key, the same idea `ConcurrentHashMap` uses internally, trading a perfectly global sense of "least recently used" for much better concurrency.

**Interview Answer:**

I back it with a HashMap for O(1) key lookup and a doubly linked list for O(1) reordering, where the map's values are direct pointers to list nodes, so both get and put just unlink and relink a node near the head, and eviction removes from the tail. The interesting follow-up is thread safety: since even a read in an LRU cache mutates the list's order, a simple read-write lock will not work, so I would either wrap the whole structure in one lock or shard the cache into multiple independently-locked segments for better concurrency.

## Q6. [Senior] Design Splitwise. How do you model expenses, splits, and the settle-up logic between users?

**Answer:**

- The core challenge is separating two different concerns: how one expense gets divided among a group of people, and what the overall running balance is between every pair of people once you add up all their shared expenses.

**Core classes**

- `User`: id, name, email.
- `Group`: a set of users who share expenses together, like roommates or a trip group.
- `Expense`: an amount, a description, the `User` who paid, and a list of `ExpenseSplit` entries describing how it is divided.
- `ExpenseSplit`: an interface with implementations `EqualSplit`, `ExactSplit`, and `PercentSplit`, each knowing how to compute one participant's share of the total. This is the Strategy pattern again, swapping in different "how to divide this" logic behind one shared interface.
- `Balance` (or `BalanceSheet`): a running total between every pair of users, for example "A owes B $30," updated every time a new expense is added.
- `SettlementService`: works out and records payments that bring balances back to zero.

```
interface SplitStrategy {
    Map<User, Double> computeSplits(double amount, List<User> participants);
}
class Expense {
    User paidBy;
    double amount;
    SplitStrategy splitStrategy;
    List<User> participants;
}
```

**How it works**

1. Someone adds a new `Expense`, specifying who paid, the total amount, the participants, and which `SplitStrategy` to use (equal, exact amounts, or percentages).
2. The chosen strategy computes each participant's share. `EqualSplit` divides evenly, `ExactSplit` uses amounts the user typed in directly (which must add up to the total), and `PercentSplit` uses percentages (which must add up to 100).
3. For every participant except the payer, the system updates the pairwise `Balance` between that participant and the payer, adding to "participant owes payer."
4. Rather than storing every raw transaction and replaying history to answer "how much does X owe Y," the system keeps a running balance per pair, so that lookup is instant.
5. When a user wants to settle up, the `SettlementService` looks at everyone's net balance (total owed to them minus total they owe) and works out the smallest possible set of payments to zero everything out.

**The trickiest part**

- The signature hard part of Splitwise is debt simplification. Picture three friends where A owes B, B owes C, and C owes A the same amount. Naively that is three separate payments, but really everyone could just pay nothing, the debts cancel out. In general, the minimum number of payments needed is usually far fewer than the number of individual expenses.

- The standard approach treats each person's net balance as a single number (positive if people owe them, negative if they owe others), then repeatedly matches the person owed the most money against the person who owes the most money, settling the smaller of the two amounts between them, and repeating until everyone's balance is zero.
- **This greedy approach, always matching the biggest creditor against the biggest debtor, minimizes the number of settlement transactions and is the piece interviewers most want to see you reason through out loud.**

**Interview Answer:**

I model an Expense as an amount paid by one User and divided across participants using a pluggable SplitStrategy, equal, exact, or percentage, each producing a per-user share. Instead of storing raw transaction history, I maintain a running balance between every pair of users that updates whenever an expense is added. The genuinely hard part is settle-up: minimizing the number of payments needed by treating net creditors and net debtors as two groups and greedily matching the person owed the most against the person who owes the most until every balance hits zero.

---

## Q7. [Senior] Design a Movie/Ticket Booking System. How do you prevent two people from booking the same seat at the same time?

**Answer:**

- The class model itself is fairly simple, the real interview focus is on how you stop two different customers from both successfully booking the same physical seat at the same showtime.

**Core classes**

- `Movie`, `Theater`, `Screen`: standard catalog-style entities describing what is playing and where.
- `Show`: one specific `Movie` playing on one specific `Screen` at one specific time, along with a `SeatMap` for that showing.
- `Seat`: a seat number, a type (regular or premium), and a status that is specific to one show, such as `AVAILABLE`, `LOCKED`, or `BOOKED`.
- `SeatLock`: a temporary hold on a seat with an expiry timestamp, created the moment a user starts checking out.
- `Booking`: a finalized reservation linking a `User`, a `Show`, a set of seats, and a `Payment`.
- `PaymentService`: processes the payment and either confirms the booking or releases the held seats.

```
class SeatLock {
    String seatId, userId;
    Instant expiresAt;
}
interface SeatLockService {
    boolean tryLock(String showId, List<String> seatIds, String userId);
    void confirm(String showId, List<String> seatIds);
    void release(String showId, List<String> seatIds);
}
```

**How it works**

1. A user picks one or more seats on the seat map for a `Show`.
2. The system calls `tryLock`, which checks that every single requested seat is currently `AVAILABLE`, and if so, flips all of them to `LOCKED` with a short expiry, say five to ten minutes.
3. The user proceeds to payment while their chosen seats are held.

4. If payment succeeds within the time window, the seats move permanently to `BOOKED` and a `Booking` record is created.
5. If payment fails, or the hold simply times out, the seats are released back to `AVAILABLE`, either by a background job that periodically sweeps expired locks, or by a lazy check the next time someone tries to view or lock that seat.

**The trickiest part**

- **The core design decision is that locking a set of seats must be all-or-nothing, otherwise two users requesting overlapping seat sets could each successfully lock a different subset, leaving the system in a half-reserved, inconsistent state.**
- For example, if user A wants seats 5 and 6, and user B wants seats 6 and 7, and the system checks and locks seats one at a time instead of as one atomic group, it is possible for both A and B to end up thinking they hold seat 6.
- In a design running on a single server, this is solved with a synchronized check-then-set operation scoped to that show's seat map, so the whole "check every seat, then lock every seat" step happens as one uninterruptible unit.
- At larger scale, across multiple application servers, this atomic lock-with-expiry is usually pushed into a shared store like Redis, using either a small script (a Lua script, which Redis can run as a single atomic unit) or a `SETNX`-with-TTL style operation (TTL means "time to live," an automatic expiry) per seat key, so the lock, its expiry, and the fact that it covers multiple seats at once all happen in a single round trip instead of several separate steps that could race against each other.

**Interview Answer:**

Seats have a per-show status of available, locked, or booked. When a user starts checkout, I atomically lock every requested seat with a short expiry, all-or-nothing, so an overlapping request from another user never leaves seats half-reserved. If payment completes before the hold expires, seats become booked, otherwise a background sweep or a lazy check releases them. At scale I would implement that atomic multi-seat lock with expiry in a store like Redis, since it needs to happen in a single round trip rather than several separate check-then-set calls spread across app servers.

---

## Q8. [Mid] Design a Tic-Tac-Toe or Chess game. How would you structure the board, pieces, and move validation so it's extensible?

**Answer:**

- The interesting design decision is making move validation extensible without an explosion of if/else branches, and this matters far more for chess than tic-tac-toe, so the answer below is framed generally with chess called out specifically where it differs.

**Core classes**

- `Board`: an n by n grid of `Cell` objects, each optionally holding a `Piece`.
- `Piece`: an abstract class (used for chess) with subclasses `Pawn`, `Rook`, `Knight`, `Bishop`, `Queen`, and `King`, each implementing its own `getValidMoves(board, position)`. For tic-tac-toe there is no movement at all, so this collapses to a simple `Symbol` enum, either X or O.
- `Player`: an id, and for chess, a color or side.
- `Move`: the source position, destination position, which piece moved, and any piece that was captured.
- `Game`: owns the `Board`, the two `Player` objects, whose turn it is, and the move history. It exposes one entry point, `makeMove()`.

- `GameState`: represents the overall game status, such as `IN_PROGRESS`, `WIN`, `DRAW`, and, for chess specifically, `CHECKMATE` and `STALEMATE`.

```
abstract class Piece {
    Player owner;
    abstract List<Position> getValidMoves(Board board, Position current);
}
class Game {
    Board board;
    Player[] players;
    int currentPlayerIndex;
    boolean makeMove(Position from, Position to);
}
```

**How it works**

1. A player attempts a move by specifying a source and destination position.
2. `Game.makeMove()` looks up the `Piece` sitting at the source position.
3. It asks that piece for its list of valid moves given the current board, rather than containing any movement logic itself.
4. If the destination is in that list, the move is legal from a pure movement standpoint.
5. For chess specifically, the move is then simulated on a temporary copy of the board to check it would not leave the mover's own king in check; only if that check passes is the move actually committed.
6. The board is updated, the move is added to history, turn passes to the other player, and the game state is re-evaluated (checking for win, draw, checkmate, or stalemate as appropriate).

**The trickiest part**

- **Each Piece subclass owns its own movement rules, so Game.makeMove just delegates to piece.getValidMoves() instead of containing a giant switch statement in the game engine.** This is a clean example of the Strategy pattern, meaning each piece type carries its own "how do I move" behavior, and adding a brand new piece type or a chess variant means adding one new class, without touching `Game` at all.
- For tic-tac-toe, checking for a winner naively means rescanning the whole board after every move, which is fine at a tiny 3x3 size but wasteful in principle. The sharper answer is to only check the row, column, and diagonal (if applicable) that the very last move touched, which is a much smaller, incremental check.
- For chess, the genuinely hard part is that a move's legality is not purely local to the piece moving. A move can follow all of a piece's normal movement rules and still be illegal if it would leave your own king in check. That means `Game.makeMove` cannot just ask the piece "is this move valid," it must also simulate the move and re-run check detection before committing.

**Interview Answer:**

I would give each piece type its own class implementing getValidMoves, so the Game engine just asks the piece for its legal moves rather than branching on piece type itself, that is the extensibility win for chess variants. Game owns the board, the players, turn order, and move history, and exposes a single makeMove entry point. For tic-tac-toe I would only check the row, column, and diagonal touched by the last move instead of rescanning the whole board. For chess, the tricky part is validating that a move does not leave your own king in check, which means simulating the move before actually committing it.

---

## Q9. [Mid] Design a Logging Framework. How would you structure loggers, log levels, and output destinations so new destinations can be added easily?

**Answer:**

- This is a small system, but it is a great way to show you understand how to design for extensibility, specifically letting new log destinations get added without ever touching the core logging code.

**Core classes**

- `Logger`: the entry point application code calls, exposing `debug()`, `info()`, `warn()`, and `error()`, each of which just calls one shared `log(level, message)` method underneath.
- `LogLevel`: an ordered list of severities, `DEBUG < INFO < WARN < ERROR`, so a logger configured at `INFO` automatically ignores `DEBUG` calls without any extra code.
- `LogMessage`: a small data holder with the level, timestamp, message text, and information about which part of the code or thread produced it.
- `Appender`: an interface for "where do logs go," with implementations like `ConsoleAppender`, `FileAppender`, `NetworkAppender`, and `DatabaseAppender`, each implementing `append(logMessage)`. A single `Logger` can hold several appenders at once, so one log call can go to the console and a file simultaneously.
- `Formatter`: converts a `LogMessage` into an actual string, whether plain text or JSON (a structured text format), kept separate from `Appender` so the format and the destination can each change independently.
- `LoggerConfig`: the minimum log level and the list of appenders, usually wired together once at startup by a factory or a single shared object often called `LogManager`.

```
interface Appender {
    void append(LogMessage msg);
}
class Logger {
    LogLevel minLevel;
    List<Appender> appenders;
    void log(LogLevel level, String msg) {
        if (level.ordinal() < minLevel.ordinal()) return;
        LogMessage m = new LogMessage(level, msg, Instant.now());
        for (Appender a : appenders) a.append(m);
    }
}
```

**How it works**

1. Application code calls something like `logger.info("user logged in")`.
2. `Logger.log()` first compares the message's level against the logger's configured minimum level, and silently returns if the message is below that threshold.
3. If the message passes the threshold, a `LogMessage` object is created with the level, text, and current timestamp.
4. The logger loops through every configured `Appender` and calls `append()` on each one, so the same message can be written to the console, a file, and a remote logging service all from one call.
5. Adding a brand new destination, say shipping logs to a metrics pipeline, means writing one new class that implements `Appender`. Nothing inside `Logger` needs to change, which is the entire point of coding against an interface instead of a specific class.

**The trickiest part**

- **The design decision that actually matters in production is making append() asynchronous, so a slow appender, like one writing over the network, never blocks the application thread that triggered the log call in the first place.**

- If `append()` runs synchronously and a network appender hangs for even a second, every part of the application that tries to log something during that second gets stuck waiting too, which is a serious problem for a feature that is supposed to be invisible overhead.
- The standard fix is a producer-consumer setup: `Logger.log()` just pushes the `LogMessage` onto a thread-safe bounded queue (bounded means it has a maximum size so it cannot grow forever) and returns immediately, while one or more background worker threads pull messages off the queue and call the real appenders.
- You also need an explicit policy for what happens if the queue fills up faster than it can drain, during a sudden burst of log activity: options include dropping the oldest messages, dropping new ones, or blocking briefly. Stating this choice out loud in an interview shows you have thought about failure modes, not just the happy path.

**Interview Answer:**

Logger is the facade application code calls, delegating to a list of Appenders, console, file, network, each implementing a simple append method, so adding a new destination never touches Logger itself. LogLevel is an ordered list used to filter calls early and cheaply. The production-grade detail I would bring up unprompted is making logging asynchronous: log calls push onto a bounded queue and a background thread drains it to the real appenders, with an explicit overflow policy, so a slow network appender never blocks the application's main work.

---

## Q10. [Mid] Design a Car Rental System. How do you model vehicles, reservations, and pricing so new vehicle types or pricing rules are easy to add?

**Answer:**

- The main modeling decisions are grouping vehicles into categories, correctly answering "is anything available for these dates," and keeping the pricing logic pluggable so new rate rules do not require rewriting existing code.

**Core classes**

- `Vehicle`: an abstract class with a registration number and a status such as `AVAILABLE`, `RESERVED`, `RENTED`, or `MAINTENANCE`. Subclasses like `Car`, `SUV`, and `Van` carry attributes specific to that type.
- `RentalBranch` (or `Store`): one physical rental location, with its own `VehicleInventory`.
- `Reservation`: a `User`, a requested `Vehicle` category (resolved to one specific vehicle at pickup time), a start date, and an end date.
- `RentalAgreement`: created at the actual pickup moment, tracking real pickup and return times, mileage driven, and a link to the final `Bill`.
- `PricingStrategy`: an interface with implementations like `DailyRateStrategy`, `WeeklyDiscountStrategy`, and `LateFeeStrategy`, each computing cost differently depending on vehicle category and rental duration. This is once again the Strategy pattern, swapping out "how to price this rental" behind one shared interface.
- `Bill`: the base rental cost plus any late fees, insurance, or add-ons.

```
interface PricingStrategy {
    double calculatePrice(Vehicle v, LocalDate start, LocalDate end);
}
class Reservation {
    User user;
    VehicleCategory category;
    LocalDate startDate, endDate;
    ReservationStatus status;
```

```
}
```

**How it works**

1. A customer searches for a vehicle category (like "SUV") at a branch for a specific date range.
2. Instead of checking a single boolean "is this vehicle free" flag, the system checks, for each vehicle in that category, whether any of its existing reservations overlap with the requested date range. This means keeping, per vehicle, a sorted list of already-booked date ranges.
3. If an available vehicle is found, a `Reservation` is created, initially tied to the category rather than one exact vehicle, since the exact vehicle can be assigned later.
4. At pickup time, a specific `Vehicle` is assigned, and a `RentalAgreement` is created recording the actual pickup time and starting mileage.
5. At return time, the system records the return time and final mileage, then asks the currently configured `PricingStrategy` to calculate the cost based on vehicle category and duration, adding any late fees if the return was overdue.
6. A final `Bill` is generated combining the base rental cost with any extra fees or add-ons.

**The trickiest part**

- Pricing needs to vary along several dimensions at once, vehicle category, how long the rental lasts (with weekly discounts kicking in), the season, and late returns. Hardcoding a price formula directly onto the `Vehicle` class breaks almost immediately once a second pricing rule needs to be added, so pricing is instead injected as a swappable `PricingStrategy` at billing time.
- **Confirming a reservation must atomically verify that no overlapping reservation exists for that vehicle and then create the booking, otherwise two customers can walk away each believing they have secured the same car for the same week.**
- This overlap-check-then-create step needs the exact same locking discipline you would use for any other double-booking problem: if the check and the creation are two separate steps with no protection between them, two nearly-simultaneous reservation attempts can both pass the check before either one has actually recorded its booking, and both get confirmed for a vehicle that can only serve one of them.

**Interview Answer:**

Vehicle is an abstract class with Car, SUV, and Van subclasses, tracked per branch in a VehicleInventory. Reservation represents a request for a vehicle category over a date range, and availability is really an interval-overlap check against that vehicle's existing reservations, rather than a single status flag. Pricing is a pluggable PricingStrategy so daily rates, weekly discounts, and late fees can each evolve independently. The concurrency detail I would flag is that checking for overlap and creating the reservation must happen as one atomic step, otherwise two customers can both end up confirmed for the same car on overlapping dates.

---

**If you remember nothing else from this chapter:** *LLD questions reward you when you can turn a vague prompt into concrete classes and interfaces without reaching for more complexity than you've earned.*